TATA ELXSI TELIPORT SEASON 4 · CASE STUDY 3 (MCV)

VeriQ

Adaptive QoS Engine for Mixed Home Broadband Traffic

Finds the cause, protects what you chose, and proves it.

The complete project handbook · design v5 (final for implementation)
Problem · solution · architecture · implementation guide · testing & evaluation · KPIs & business case · demo, 3D/2D models & physical kit · submission kit

Version	Handbook v5.0 · 12 October 2026
Team	<Team name> · <College> · <Members>
Authoritative spec	docs/11_VeriQ_v5_Final_Design.md (Appendix C)
Evidence status	One preliminary Day-0 sandbox run; all other numbers are targets, starting parameters, assumptions or illustrations, and are labelled as such.

Confidential team working document. No Tata Elxsi endorsement is implied.

Contents

How to use this handbook 7

Part I · Context 8

1 Executive Summary 9
- 1.1 The problem in one page 9
- 1.2 VeriQ in one paragraph 9
- 1.3 The dominance invariant: “never worse than static SQM” 9
- 1.4 The three layers 10
- 1.5 What is new compared with QoSmate and cake-autorate 10
- 1.6 Key design decisions, C1–C14, in plain words 11
- 1.7 Evidence so far 11
- 1.8 Evaluation plan and primary outcome 12
- 1.9 Deliverables and timeline 12
- 1.10 Why judges should care 13
- 1.11 VeriQ at a glance 13

2 The Case and Its Requirements 14
- 2.1 TELIPORT Season 4 format 14
- 2.2 The Case 3 brief, summarised 14
- 2.3 Explicit requirements 14
- 2.4 Implicit requirements 15
- 2.5 Non-functional requirements 15
- 2.6 Assumptions and constraints 16
- 2.7 Requirement traceability matrix 16
- 2.8 Why Case 3 was chosen 18

3 Tata Elxsi Context 19
- 3.1 Company overview 19
- 3.2 The MCV vertical and home broadband 20
- 3.3 The NEURON link 21
- 3.4 How VeriQ maps to Tata Elxsi’s kind of work 21
- 3.5 Careful wording 22
- 3.6 What a Tata Elxsi panel is likely to probe 22
- 3.7 Pre-panel checklist 22

4 The Problem: Delay Under Load in a Shared Home Connection 24
- 4.1 How home broadband works 24
- 4.2 Queues, buffers and bufferbloat 25
- 4.3 Why throughput tests miss it 26
- 4.4 The home traffic mix and what each activity needs 27
- 4.5 Why generic QoS fails 27
- 4.6 Problem 1: capacity moves 28
- 4.7 Problem 2: the cause of a delay rise is not always the access link 28
- 4.8 Problem 3: households need intent, not knobs 29
- 4.9 The Day-0 evidence 29
- 4.10 Problem statement and success criteria 31

5 Landscape and Competitors 32
- 5.1 A taxonomy of approaches 32
- 5.2 The foundations: shapers and smart queues 32
- 5.3 SQM (sqm-scripts) 32
- 5.4 qosify 33
- 5.5 cake-autorate 33
- 5.6 QoSmate deep dive (read) 33
- 5.7 Commercial router “adaptive QoS” 36
- 5.8 ISP-side approaches 37
- 5.9 L4S and NQB 37
- 5.10 The rival student prototype: AQE 37
- 5.11 Comparison at a glance 38
- 5.12 Where VeriQ positions itself 38
- 5.13 The hypotheses VeriQ must prove 39

Part II · The solution 41

6 Solution Overview 42
- 6.1 The VeriQ idea 42
- 6.2 The dominance invariant 42
- 6.3 The nine principles 43
- 6.4 Three speeds 43

6.5 Three layers 43
6.6 The 250 ms control loop 44
6.7 What VeriQ is not 45
6.8 The v5 decision list (C1–C14) 45
6.9 A day in the life: one household evening 46

7 Architecture 48
7.1 Layers and planes 48
7.2 Component catalogue 49
7.3 Data plane objects 51
7.4 Deployment views 52
7.5 Process model and privileges 52
7.6 Interfaces 52
7.7 Data stores and persistence 53
7.8 Key flows 53
7.9 Quality attributes 54
7.10 Architecture decision records 55
7.11 Summary 56

8 Queue management 58
8.1 Concepts first 58
8.2 Why the shaper must sit just below the bottleneck 60
8.3 VeriQ service classes 60
8.4 The datapath 61
8.5 The three backends in depth 62
8.6 Link-layer presets 64
8.7 One-directional shaping 64
8.8 ACK and MSS handling 64
8.9 Per-device policers: honest semantics 65
8.10 AQM parameter scaling 65
8.11 DSCP policy and Wi-Fi WMM 65
8.12 Rate updates per backend 66
8.13 CPU cost considerations 66
8.14 Worked example: sizing the HFSC RT budget for 100/20 Mbit/s 66
8.15 How to explain this chapter to a judge 67
8.16 Queue-management checklist 67

9 Chapter 09 (pending) 68

10 Capacity Estimation and Rate Control 69
10.1 The quantities, and why they are never conflated 69
10.2 Reflectors and probing 70
10.3 Detectors 72
10.4 Direction attribution (C9) 73
10.5 The rate controller 73
10.6 The fast cut (C1): one step to 0.90·Ĝ 75
10.7 Upward exploration, BACKOFF and DRIFT 76
10.8 Bounded fallback episodes 77
10.9 The capacity estimate μ ± σ 77
10.10 Capacity memory (time-of-week prior) 78
10.11 Link events 78
10.12 Comparison with a delay-only autorate 78
10.13 Starting parameters 79

11 Chapter 11 (pending) 80

12 Safety 81
12.1 Safety philosophy 81
12.2 The Action Checker 82
12.3 Operating states 84
12.4 The reconciler 86
12.5 Watchdog and time-to-safe 87
12.6 Budgets and persistence 88
12.7 Failure domains 89
12.8 Safety case summary 89

13 Chapter 13 (pending) 91

14 Chapter 14 (pending) 92

15 User experience 93
15.1 UX principles 93
15.2 Personas 93
15.3 The setup wizard 94
15.4 The LuCI app 95
15.5 The household page 99

15.6 Command-line interface 101
15.7 Alerts 102
15.8 Internationalisation and accessibility 102
15.9 Error and empty states 102
15.10 UX study plan 103
15.11 UX acceptance checklist 103
16 **Chapter 16 (pending)** 104
17 **Chapter 17 (pending)** 105

Part III · Implementation 106
18 **Tech Stack and Repository** 107
18.1 Technology choices at a glance 107
18.2 Key decisions explained 107
18.3 Toolchain and version pinning 108
18.4 Repository layout, folder by folder 109
18.5 Development environment setup 110
18.6 Coding standards 111
18.7 Branching, commits and reviews 112
18.8 CI workflows 112
18.9 Third-party modules and licences 112
18.10 Sprint 0 bootstrap checklist 113
19 **Module Implementation Guide** 114
19.1 How the packages fit together 114
19.2 clock 114
19.3 config 115
19.4 model 116
20 **Chapter 20 (pending)** 117
21 **Chapter 21 (pending)** 118
22 **Build plan** 119
22.1 Planning assumptions 119
22.2 Priorities: P0, P1, P2 119
22.3 Sprint calendar 119
22.4 Task breakdown per sprint 120
22.5 Roles 124
22.6 RACI matrix 124
22.7 Rituals 125
22.8 Definition of done 125
22.9 The critical path 125
22.10 Buffers 126
22.11 Scope-cut ladder 126

Part IV · Testing and evaluation 127
23 **Test Strategy** 128
23.1 Principles 128
23.2 The test pyramid 128
23.3 Unit tests 129
23.4 Property and fuzz tests 132
23.5 Replay determinism 132
23.6 Integration tests in network namespaces 133
23.7 Package tests in the OpenWrt VM 133
23.8 Hardware-in-loop 134
23.9 Soak tests 134
23.10 Security tests 134
23.11 CI gates 135
23.12 Test data management policy 135
23.13 Traceability: test → requirement 135
24 **Evaluation Design** 137
24.1 What the evaluation must answer 137
24.2 Arms 138
24.3 Controller-only vs whole-system comparisons 139
24.4 Scenario catalogue 139
24.5 Tiers and the Stage 1–4 ladder 142
24.6 Fault blindness 142
24.7 Metric definitions 142
24.8 Statistics 144
24.9 The evidence ledger and the claims check 145
24.10 Threats to validity 146

24.11 Results reporting template 146
24.12 UX study protocol (SUS) 147
25 Lab Build Guide 148
25.1 What the lab must prove, and what it cannot 148
25.2 Hardware and software requirements 148
25.3 The lab topology 149
25.4 The ISP emulator 151
25.5 Traffic generators 152
25.6 Collecting WebRTC getStats 154
25.7 Ground-truth channels 155
25.8 The scenario YAML schema 155
25.9 The runner workflow 156
25.10 The evidence ledger and manifest 157
25.11 Installing the baselines 157
25.12 SUT-side faults 158
25.13 Reproducing Day-0 with the corrected fingerprint 158
25.14 The OpenWrt VM lab 159
25.15 The physical lab with the Pi 159
25.16 Troubleshooting 159
25.17 Safety notes 159
25.18 Lab readiness checklist 160
Part V · Business, KPIs and risk 161
26 Key performance indicators 162
26.1 Labels used in this chapter 162
26.2 The KPI tree 162
26.3 Technical KPIs 162
26.4 Product KPIs 164
26.5 Operator KPIs (pilot hypotheses) 165
26.6 Engineering KPIs 165
26.7 Competition KPIs 166
26.8 Dashboard mapping 166
26.9 From KPI to decision 167
26.10 Rules that keep the KPIs honest 167
27 Business case 168
27.1 The value chain 168
27.2 Stakeholder value 168
27.3 Tata Elxsi context 169
27.4 Indian deployment context 169
27.5 What a buyer could do instead 169
27.6 Packaging options 170
27.7 Cost model 170
27.8 Illustrative operator model 172
27.9 Go-to-market path 173
27.10 Pilot design 174
27.11 Scaling 174
27.12 IP and licensing 175
27.13 Presenting the business case in two minutes 175
27.14 Summary 175
28 Risks 176
28.1 Scoring 176
28.2 Technical risks 176
28.3 Schedule risks 177
28.4 Team risks 177
28.5 Evaluation-validity risks 178
28.6 Legal and licence risks 178
28.7 Demo-day risks 179
28.8 Heat map 180
28.9 Top-5 risk deep dive 180
28.10 Keeping the register alive 181
Part VI · Demonstration and submission 182
29 Live Demo and Video 183
29.1 Demo principles 183
29.2 Stage layout 183
29.3 Pre-flight checklist 183
29.4 The run sheet (about 7 minutes) 184
29.5 The judge's fault menu 185

29.6 Backup plans 186
29.7 What the presenter must explain personally 186
29.8 The 3-minute video 186
29.9 Recording tips 187
29.10 Rehearsal plan 188
30 3D Home-Network Twin 189
30.1 Purpose and rules 189
30.2 Scene design 189
30.3 Tech stack 191
30.4 The twin event stream 192
30.5 Step-by-step build guide 193
30.6 Assets 198
30.7 Performance tips 198
30.8 How to demo it 198
30.9 Live mode 199
30.10 Extending the twin 200
30.11 Acceptance checklist 200
31 Chapter 31 (pending) 201
32 Chapter 32 (pending) 202
33 Submission Kit 203
33.1 The 20-page submission blueprint 203
33.2 Pitch deck outline 205
33.3 The Round 1 one-slide recap 208
33.4 Panel Q&A bank (50 questions) 208
33.5 Judging-criteria self-assessment 212
33.6 Final pre-submission checklist 212
Part A–E · Appendices 214
Appendix A · Glossary 215
Appendix B · References 220
Appendix C · Authoritative specification (docs/11, VeriQ v5) 221
Appendix D · Implementation contracts (docs/09 v4.1 Appendix A, in force) 248
Appendix E · Change records 251

How to use this handbook

This handbook covers VeriQ v5 completely: the problem, the solution, the architecture, the implementation guide, testing and evaluation, the business case, and the demonstration assets. It is written for three audiences.

If you are…	Read first	Then
The build team (starting implementation)	Chapters 6–7 (overview, architecture), 18–21 (stack, modules, templates, flows), 22 (plan)	Chapters 8–14 for module detail as you build them; 23–25 for tests and the lab
Writing the ~20-page submission or slides	Chapter 1 (executive summary), Chapter 33 (submission kit: page-by-page blueprint)	Pull figures and tables from the chapters the blueprint names
Preparing for the panel	Chapter 33 (50-question bank), Chapter 14 (adverse conditions), Chapter 24 (evaluation design)	Chapter 5 (competitors) and Chapter 3 (Tata Elxsi context)

The labels used throughout

Judges reward honesty. Every number and claim in this handbook carries one of these labels, explicitly or through its section.

Label	Meaning	Example
Measured	Came from a run whose raw files are in the repository	Day-0 sandbox: FIFO added 266.1 ms upload p95 (one preliminary run)
Target	A goal the evaluation will test	Time-to-safe ≤ 12 s
Starting parameter	A default to be tuned in Stage 2 and recorded in the run manifest	θ_bb = 15 ms
Illustrative	An example made up to explain a mechanism	The 100 → 20 Mbit/s cut chart
Hypothesis	Expected behaviour, to be confirmed or refuted by an experiment	"A delay-only autorate cuts when all reflectors are lost"
(read)	A fact about another tool, taken from its README or source at a pinned version	QoSmate autorate averages RTT across reflectors (read)
Assumption	An input to a model that a pilot would replace	₹400 per support contact

Where the authoritative design lives

- **Authoritative specification:** `docs/11_VeriQ_v5_Final_Design.md`, reproduced in full as **Appendix C**. If any chapter disagrees with Appendix C, Appendix C wins.
- **Contracts still in force from v4.1:** `docs/09` Appendix A, reproduced as **Appendix D**. Where docs/11 changes one, it says so (change C1 replaces the L2 cut rule).
- **Change records:** `docs/12` (v5) and `docs/10` (v4.1), in **Appendix E**.
- Regenerate this PDF with `python3 tools/build_handbook.py`. Figures come from `tools/build_handbook_figures.py`.

Version

Item	Value
Handbook	v5.0, 12 Oct 2026
Design	VeriQ v5 (docs/11)
Measured evidence	Day-0 sandbox run only (preliminary). Every other number is a target, starting parameter, assumption or illustration
Comparator versions read	QoSmate v0.5.48 (c507843), luci-app-qosmate v1.0.14 (11c6e64); cake-autorate and sqm-scripts per their repositories
Status	Design complete; implementation starts with Sprint 0 (Chapter 22)

Part I · Context

The problem, the competition and the field

CHAPTER 1

Executive Summary

A judge who reads only this chapter should know what VeriQ is, what it claims and does not claim, what has been measured so far, and how the team will prove the rest.

1.1 The problem in one page

A home broadband connection is shared by very different kinds of traffic at the same time: a parent on a video call, a student in an online game, three televisions streaming, a laptop downloading a large ISO disk image, a phone uploading photos to a cloud backup, and a console installing an update. A call needs low, steady delay; a download wants throughput; a game needs low delay and jitter at a tiny bit rate.

The dominant source of bad experience in such a home is usually not a lack of bandwidth. It is **bufferbloat** [R1]: the modem or the ISP (Internet Service Provider) equipment has a large first-in, first-out (FIFO) buffer, a bulk transfer fills it, and every interactive packet then waits behind hundreds of milliseconds of queued data. A speed test reports a healthy number while the call freezes. The team's own Day-0 sandbox run measured exactly this pattern: an upload saturated by four TCP (Transmission Control Protocol) flows added 266.1 ms of delay at the 95th percentile behind a FIFO. That is one preliminary run, not a replicated result (see "Evidence so far" below).

The established fix is Smart Queue Management (SQM): shape the traffic in the home router to just below the bottleneck rate, so that the queue forms in the router, and manage that queue with a modern Active Queue Management (AQM) scheduler such as CAKE (Common Applications Kept Enhanced) [R5][R6][R10][R11]. This works well when the link rate is fixed and known. Three problems remain, and they are where existing tools differ:

1. **Capacity moves.** 4G/5G fixed wireless access (FWA), cable (DOCSIS, Data Over Cable Service Interface Specification) at peak hours, shared fibre (FTTH/PON, fibre to the home over a passive optical network) and Wi-Fi backhaul all vary. A static shaper rate is either too high (bloat returns) or too low (bandwidth is wasted).
2. **Delay can come from places the router cannot fix.** A slow far-away server, an ICMP (Internet Control Message Protocol) reflector that answers late, a weak Wi-Fi client, or an overloaded router CPU (central processing unit) all raise measured delay. A controller that cuts the shaper whenever delay rises will cut in all of these cases, wasting bandwidth without helping anyone.
3. **Households need intent, not knobs.** "Protect my exam call for the next hour" is the real request. A household also needs honest feedback on whether the protection helped.

Tata Elxsi's Case Study 3 asks for exactly this: an intent-aware QoS (Quality of Service) controller that classifies traffic without reading payload, estimates capacity, shapes and queues in both directions, verifies user experience, supports temporary intent with fairness, and keeps every automated action bounded, observable and reversible (Chapter 2).

1.2 VeriQ in one paragraph

VeriQ is an OpenWrt-native, adaptive home-QoS system with three layers. The first layer is QoSmate-class queue management, classification and user interface: every feature a QoSmate user relies on has an equivalent, and existing QoSmate configurations can be imported. The second layer is a diagnosis-informed, bounded, auditable rate controller that finds where the delay comes from before it cuts. The third layer is product-grade operation: signed packages, fail-safe behaviour, self-tests, explainability, a household page and footprint budgets. Packets stay in the Linux kernel (nftables, conntrack, CAKE); a single Go daemon, `veriqd`, observes, diagnoses, proposes, checks and only then applies changes.

The one-line pitch, used unchanged across the submission:

> *"Everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM."*

1.3 The dominance invariant: "never worse than static SQM"

No home router can promise perfect results under every adverse condition. If the ISP link fails, nothing inside the home can fix it. VeriQ therefore makes a narrower promise that can be tested:

> **Dominance invariant (design goal, tested in every scenario).** In every condition where VeriQ lacks evidence to act, it holds or reverts to a static CAKE configuration equivalent to well-tuned SQM. Its adaptive behaviour is only allowed where evidence supports it, through bounded steps. So by construction, VeriQ's worst case is "static SQM", and its best case is better.

Static CAKE is active from the first minute after installation. The adaptive controller first runs in SHADOW mode (it computes and logs decisions but applies none) for 24 hours by default. If the daemon crashes, the kernel configuration stays in place and a separate watchdog moves the shaper to a conservative SAFE rate (target: within 12 s). Where VeriQ is blind (no reflectors answer, a foreign tool

changes the queue, the daemon dies), it degrades to static behaviour rather than guessing. Chapter 14 tests the invariant against 32 adverse conditions, A1–A32.

> **Honesty check.** The invariant is a design goal and a test obligation, not a measured property. It becomes a claim only for the scenarios in which the evaluation (Chapter 24) shows it held.

1.4 The three layers

Layer	What it contains	Why it exists	Main chapters
1. Queue management, classification, UI	CAKE (default), HFSC (Hierarchical Fair Service Curve) and HTB (Hierarchical Token Bucket) backends; both directions via IFB (Intermediate Functional Block) and `act_ctinfo`; QoSmate-compatible rules, IP and domain sets, per-device policers, app catalogue, heuristics that may only demote; LuCI (OpenWrt web interface) app	Parity with the most capable community tool, plus a migration path	8, 9, 15, 20
2. Diagnosis-informed rate controller	Reflector pool with median aggregation, one-way delay where available, windowed-minimum baselines, evidence groups and levels L0–L3, a rate state machine, the Action Checker, budgets, receipts	Cut fast when the access link really dropped; hold when a cut cannot help; prove each action	10, 11, 12, 13, 14
3. Product-grade operation	Signed `.ipk` / `.apk` packages, procd services, watchdog, operating states, reconciler, self-tests, support bundle, household page, footprint profiles Lite/Standard/Pro	A design that a household or an operator could actually run	7, 12, 15, 16, 17

The control plane runs a 250 ms tick: Observe, Classify, Estimate, Diagnose, Propose, Check, Enforce, Receipt (Chapter 6). Per-packet work stays in the kernel; outcome assessment runs over 15–60 s windows; causal claims come only from repeated lab experiments.

1.5 What is new compared with QoSmate and cake-autorate

QoSmate [R13] and cake-autorate [R12] are mature, well-designed, community-proven tools. VeriQ learns from both and re-implements ideas clean-room; it copies no code. Facts about QoSmate below are read from its README and source at v0.5.48 (commit `c507843`); statements about behaviour are hypotheses until the head-to-head runs.

What VeriQ adopts (parity). QoSmate already offers CAKE, HFSC and HTB schedulers, download classification via conntrack information restored on ingress, DSCP (Differentiated Services Code Point) rules with sets and IPv6 suffix matching, rate limits, custom nftables rules, connection and statistics views, an auto-setup, a health check and an optional autorate daemon (read). VeriQ matches this surface feature by feature (Chapter 5) and imports QoSmate configurations.

What VeriQ changes or adds.

- **Diagnosis before action.** QoSmate's autorate averages RTT (round-trip time) across reflectors, uses 5/10 ms thresholds, cuts by ×0.85 and raises by ×1.02, keeps a baseline that only moves down after its first samples, sets latency to 9999 ms when no reflector replies, and does not check load in its decrease branch (read). The hypothesis that follows, to be tested, is that such a controller cuts in situations where a cut cannot help: remote-path delay, a lying reflector, an ICMP outage, an upward route change, an idle link. VeriQ instead requires two independent sources of evidence (for example far-reflector delay rising *and* throughput falling below the shaper rate while demand exists) before a fast cut, and holds when the evidence points to a remote, Wi-Fi or CPU cause.
- **Robust measurement.** Median of healthy reflectors with a quorum of at least two, never a mean; per-direction one-way deltas from ICMP timestamps [R16] where reflectors support them; a 10-minute windowed-minimum baseline that can rise after a route change.
- **A one-step evidence cut.** On a genuine drop at evidence level L2, VeriQ moves in one step to max(S_min, 0.90·Ĝ), where Ĝ is the median measured gateway throughput over the last second. Stepwise ×0.85 cuts would need several steps for a 100 → 20 Mbit/s drop.
- **Admissibility checking.** Every proposed action passes a deterministic Action Checker. The checker checks admissibility against the implemented rules; it is not a proof of safety.
- **Intent with fairness.** Authenticated, expiring leases ("Protect my call for 1 h"), limited to two active leases, always subject to a per-flow eligibility envelope that returns misbehaving flows to best effort.
- **Receipts.** Every action produces an IMPROVED, NOT_IMPROVED or INCONCLUSIVE receipt based on before/after measurements; receipts state association, not causation.
- **Product hygiene.** Packages through the package manager only (QoSmate's README installs by fetching scripts and offers an in-script update; read), operating states, SHADOW start, watchdog, tin self-test, signed model and catalogue.

cake-autorate, per its documentation, adjusts CAKE's rate from one-way delays to a set of reflectors and gates rate changes on load. VeriQ shares that philosophy and adds the near-hop probe, the independence-group diagnosis, the hold decision and the checker. Whether these additions produce better outcomes is the primary research question, not an assumption.

> **For judges.** In static conditions (fixed capacity, honest reflectors) both VeriQ and QoSmate's CAKE mode use CAKE, so **parity is expected and will be reported as parity**. VeriQ's expected advantages lie in the adverse and dynamic scenarios, in explainability, in safety and in user experience. These are design claims until tested.

1.6 Key design decisions, C1–C14, in plain words

Version 5 of the design made fourteen changes to the v4.1 specification. All other v4.1 contracts remain in force.

ID	Decision	In plain words
C1	One-step L2 cut to max(S_min, 0.90·Ĝ)	When the access link really drops, jump straight to just under what the link is now carrying, instead of shrinking by a fixed fraction several times. The ×0.75 cap applies only if no fresh Ĝ exists.
C2	Router daemon in Go	One static binary for every OpenWrt target, no Python on the router; Python stays for the lab, evaluation and ML (machine learning) training.
C3	OpenWrt product layer	Packages, configuration, services, web UI, access control, upgrade handling and CI (continuous integration), so the system is installable, not just demonstrable.
C4	Three queue backends	CAKE by default; HFSC for a hard realtime curve; HTB for the weakest CPUs. All map to the same four VeriQ service classes.
C5	Full classification engine	QoSmate-compatible rules, sets, policers and importer, plus the app catalogue, leases, envelope sanctions and the ML classifier.
C6	Wash DSCP by default in the product	Clear ISP-set and outgoing DSCP marks so they do not disturb home Wi-Fi priority or leak upstream; the class lives in conntrack marks instead. The lab profile leaves DSCP untouched.
C7	Heuristics only demote or reset	A heuristic can push a flow down to bulk, never up to priority. Preference needs an explicit basis.
C8	Windowed-minimum baseline	Each reflector's "normal delay" is the minimum of 1-s medians over 10 minutes of low-load samples, so it can rise after a route change.
C9	Direction attribution	Use one-way delay when available; otherwise blame only a direction that is at least 60% loaded, so an idle direction is not cut by mistake.
C10	Seven operating states, SHADOW first	INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE, DISABLED. Static CAKE helps from minute one; adaptation earns trust over 24 h.
C11	Watchdog as a procd instance	A small POSIX shell process checks a heartbeat and moves the shaper to SAFE if the daemon stops.
C12	QoSmate head-to-head and A1–A32	The evaluation compares VeriQ with QoSmate (whole system and controller only) and tests 32 adverse conditions.
C13	CPU_LIMITED diagnosis	A router that cannot keep up must not be mistaken for a drop in ISP capacity.
C14	Mark layout in bits 24–30	Class, provenance and sanction live in conntrack mark bits that avoid those used by mwan3 and pbr, so the kernel can enforce precedence and coexist with other packages.

One rule from the classification design deserves to be quoted exactly, because judges will test it: *"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."*

1.7 Evidence so far

The only measured data in this project is a single Day-0 sandbox run of `lab/quickstart_bufferbloat.sh` on 11 Oct 2026 [R51]: an emulated 20/5 Mbit/s line with an oversized FIFO at the ISP, four TCP flows for 15 s, ping during load.

Arm	Added delay p95, download	Added delay p95, upload
FIFO (no shaping)	63.6 ms	266.1 ms
Short-buffer shaper	55.4 ms	39.6 ms

> **Honesty check.** One preliminary run; no base RTT was applied (netem was unavailable in that kernel, so idle RTT was about 0.1 ms); IPv4 only; unreplicated. CAKE and fq_codel arms were skipped because the sandbox kernel lacked them. These numbers illustrate the problem; they say nothing yet about VeriQ, which has no code. The first Sprint 0 exit criterion is a full-kernel run with base RTT, CAKE and repetitions.

The data shows the expected shape (the upload FIFO is the worst offender; even a crude shaper helps sharply). It shows no CAKE result, no adaptive behaviour and no comparison with QoSmate or cake-autorate.

1.8 Evaluation plan and primary outcome

The evaluation (Chapter 24) is designed so that a tie or a loss is visible and reported.

- **Arms:** FIFO (to illustrate the problem only); static SQM with CAKE (the dominance-invariant floor); cake-autorate (co-primary comparator); QoSmate v0.5.48 with autorate, in HFSC default and CAKE mode, both whole-system and controller-only; VeriQ; and VeriQ ablations (no diagnosis, no classifier, no leases, no fallback, RTT only).
- **Scenarios:** S1–S13 (core) and A1–A32 (adverse), with random fault onset hidden from the controller.
- **Primary outcome:** inappropriate interventions in control-futile scenarios (S6 remote-path delay, S7 lying reflector, S10 Wi-Fi bottleneck, A32 probe-path disturbance), measured as count, cumulative magnitude $\int \max(0, S_{ref} - S(t))dt$ in Mbit·s, and duration below $0.95 \cdot S_{ref}$.
- **Co-primary (non-inferiority):** confirmation time after a genuine drop ≤ comparator + 1 s.
- **Secondary (non-inferiority):** bulk share of leftover bandwidth ≥ comparator − 5 percentage points (absolute).
- **Head-to-head secondary set** (Holm-corrected [R38]): added delay p95 per direction, WebRTC (browser real-time communication) freezes and concealment, DASH (adaptive video streaming) stalls, game RTT p99, page load time, Jain's fairness index [R36], recovery time, CPU and memory.
- **Statistics:** the run is the unit; a pilot of n = 5 sets the final n (expected 10–20); bootstrap confidence intervals [R37]; counterbalanced order; no p99.9 claims; missing data reported, never imputed.
- **Ladder and tiers:** Stage 1 (capability and smoke) to Stage 4 (hardware and resources); tiers H0 (smoke) to H4 (hardware soak).

Every published number must regenerate from raw data in the evidence ledger, enforced by a claims check in CI (principle P9).

1.9 Deliverables and timeline

Dates below are taken from the listings; confirm them on the official portal. Today is 12 Oct 2026.

Milestone	Date	Deliverable
Round 1	due 24 Oct 2026 (confirm)	One-slide entry (`submission/Round1_Case3_VeriQ.pptx`)
Round 2 window	14 Nov – 13 Dec 2026	Deck, multimedia and prototype; this handbook supports a ~20-page technical submission (Chapter 33)
Grand finale	date TBC	Live judge-controlled demo, video, 3D twin, physical kit

Priority	Scope	Target date
P0	Core and minimum product: CAKE backend and mark plumbing, tin self-test, prober and baselines, diagnosis, rate controller, Action Checker, reconciler, watchdog and states, rules, sets, leases, envelope, QoSmate importer, LuCI Overview/Autorate/Rules/Connections, packages for an x86-64 VM and a Raspberry Pi 4, H1 head-to-head runs, receipts, report, video	Round 2 (13 Dec 2026)
P1	Parity completion and demo assets: ML classifier with conformal abstention, HFSC/HTB, household page, Hindi strings, app catalogue, H2/H3 tiers, bench, UX study, retail router and Lite profile, 3D twin, physical kit	Finale (TBC)

Priority	Scope	Target date
P2	Product roadmap: exporter, public signed feed, apk line, prplOS/USP and RDK-B wrappers, hybrid backend, NQB/L4S experiments, home pilots	After the competition

Sprints 0–4 run from 12 Oct to 13 Dec (Chapter 22). The Sprint 0 exit criterion is concrete: Round 1 submitted, the repository skeleton and CI green, and a full-kernel lab run of FIFO versus CAKE with base RTT.

1.10 Why judges should care

- **Technical depth.** VeriQ engages with the real mechanisms: where the queue lives, why CAKE tins are priority and weights rather than reservations, why one bad reflector can move a mean, why throughput is censored when the shaper is the bottleneck.
- **Honesty.** Every number in the handbook is labelled as measured, target, starting parameter, assumption or illustrative. Expected weaknesses of other tools are hypotheses. Results where VeriQ ties or loses will be kept and shown.
- **Product readiness.** Installation, upgrade, rollback, uninstall, conflict detection, self-tests, support bundles, footprint budgets on three device classes, and a household page a non-technical user can operate.
- **Tata Elxsi fit.** The case belongs to the Media and Communications vertical, which works on broadband gateways (RDK-B and OpenWrt migration) and autonomous-network platforms (Chapter 3). VeriQ is an OpenWrt-native design with a documented porting path to prplOS and RDK-B. No endorsement or integration by Tata Elxsi is implied.

1.11 VeriQ at a glance

Aspect	VeriQ v5 (all values are targets or starting parameters unless stated)
One-line pitch	Everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM
Platform	OpenWrt 24.10 (fw4, kernel 6.6); lab on Ubuntu 24.04; reference router Raspberry Pi 4/5
Data plane	nftables table `inet veriq`; CAKE diffserv4 with `fwmark` on WAN egress and on `ifb-veriq` for download; HFSC and HTB alternatives
Service classes	RT (realtime), IV (interactive video), BE (best effort), BK (bulk)
Control plane	`veriqd` (Go, static binary), 250 ms tick, probes at 10 packets/s in total (2/s when idle)
Diagnosis	Evidence groups FAR, NEAR, THR, OWN, WIFI, CPU, CAP; levels L0–L3; labels WAN_LIKELY, REMOTE_LIKELY, WIFI_LIKELY, CPU_LIMITED, OWN_QUEUE, PROBE_FAULT, MIXED, UNKNOWN
Cut rule	L2: one step to max(S_min, 0.90·Ĝ); uncertain cases: bounded fallback (≤ 5% steps, ≤ 10% per rolling 10 min, 60 s cooldown that never refunds the budget)
Safety	Action Checker with refusal codes; reconciler with read-back; seven operating states; watchdog time-to-safe ≤ 12 s (target)
Classification	Header and timing only (NFLOG snaplen 40); 7-level precedence in the kernel; ML with Mondrian conformal abstention (α = 0.05) at P1
Intent	Leases of 15–120 min (max 4 h), ≤ 2 active, ≤ 1 per device, eligibility envelope always applies
Explainability	Receipts per action; template sentences, no LLM (large language model); household plain-language messages
Footprint	Lite ≤ 20 MB RAM; Standard ≤ 40 MB RAM and ≤ 5% of one A53 core; Pro ≤ 60 MB RAM (targets, measured in Stage 4)
Privacy	No payload, no cloud dependency, local retention (flow records ≤ 24 h in RAM, aggregates 7 days)
Measured so far	Day-0 sandbox only: FIFO added p95 266.1 ms up / 63.6 ms down; short-buffer shaper 39.6 / 55.4 ms (one run, no base RTT, IPv4)
Primary outcome	Fewer, smaller, shorter inappropriate rate cuts than delay-only autorate in S6, S7, S10, A32
Status	Design frozen for build; no controller code yet; 10 open decisions (O1–O10) have stated defaults

> **Why it matters.** The case asks for an AI-assisted controller that improves a measurable outcome over a deterministic baseline. VeriQ answers with a narrow, testable claim, a fair comparison against the strongest open tools, and a product that falls back to the accepted static solution whenever it is unsure.

CHAPTER 2

The Case and Its Requirements

This chapter states what the competition asks for, turns the Case 3 brief into numbered requirements, traces each to a VeriQ feature, a chapter and a test, and explains honestly why the team chose this case.

2.1 TELIPORT Season 4 format

TELIPORT is Tata Elxsi's national student innovation challenge. The facts below come from public listings summarised in docs/01 [R50]. Some listings conflict, so **confirm every date and format rule on the official portal.**

Item	What the listings say	What it means for the team
Eligibility	Pre-final and final-year B.Tech, M.Tech or integrated dual-degree students; teams of 2–3 from the same college (cross-branch allowed)	The build plan assumes three part-time members
Registration	Unstop shows 10 Oct 2026; GradPartners and a university notice show 23 Oct 2026	Confirm today
Round 1	Pick one problem statement; submit a single slide (PDF); due 24 Oct 2026 (confirm)	`submission/Round1_Case3_VeriQ.pptx`, slide 2
Round 2	Shortlisted teams: a 10-slide deck plus multimedia or prototype; window 14 Nov – 13 Dec 2026	About four weeks of build; a ~20-page technical document is planned too (Chapter 33)
Round 3	Grand finale presentation to senior leaders; date TBC	Demo assets (Chapters 29–32)
Rewards	₹2 lakh, ₹1.5 lakh and ₹1 lakh for the top three; pre-placement offers for national finalists; pre-placement interviews for Round 2 teams	Reaching Round 2 is already valuable
Intellectual property	Submitted ideas become Tata Elxsi's property	Keep the repository private; check the rules before publishing
Scale	Season 2 drew more than 6,000 teams (university report cited in docs/01)	Differentiation matters

2.2 The Case 3 brief, summarised

Case Study 3, "Adaptive QoS Engine for Mixed Home Broadband Traffic", is owned by the Media and Communications (MCV) business unit. Its content, summarised from the official PDF:

Background. A home connection carries calls, gaming, voice, adaptive video, downloads, updates and cloud backup at once. Static priority rules cannot tell which activity matters at a given moment, and high throughput is not enough if large queues add delay and jitter. An adaptive QoS (Quality of Service) controller should observe link conditions and broad traffic behaviour, schedule fairly and adapt policy, protecting interactive applications while bulk traffic keeps making progress. The goal is a credible reference implementation on openly documented interfaces, with a clear prototype–production boundary.

Challenge. Build an intent-aware QoS controller that classifies broad traffic categories without reading private payloads, estimates link capacity, applies queueing and shaping policies, verifies whether the policy improved user experience, and supports a temporary intent (such as prioritising a work call) while keeping fairness. Modules must be independently testable, with defined inputs, outputs, state transitions, failure handling and success conditions. Any AI must be compared with a deterministic baseline and improve a measurable outcome, not merely add a conversational interface.

Illustrative scenarios. (a) A large ISO download starts during a video conference; the controller limits the bulk queue and protects call latency. (b) WAN (wide area network) bandwidth drops from 100 to 20 Mbit/s; the controller recalculates shaping instead of building a large queue. (c) Three televisions stream while a gaming device needs low latency; service quality is balanced across the household. Teams may add generic, reproducible scenarios.

Constraints, expected outputs and evaluation criteria. The brief's ten constraints, nine expected outputs and five evaluation criteria appear below, almost word for word, as K1–K9 (plus F2), D1–D7 (plus F1–F4) and J1–J5.

2.3 Explicit requirements

Identifiers: F = functional, K = constraint, D = deliverable, J = judging criterion.

ID	Requirement (from the brief)
F1	Classify broad traffic categories without reading payload
F2	Expose classifier confidence and allow correction

ID	Requirement (from the brief)
F3	Estimate link capacity
F4	Apply queueing and shaping policy, changing it as demand and conditions change
F5	Verify whether a policy improved user experience
F6	Support temporary user intent while retaining fairness
F7	Decompose into independently testable modules with inputs, outputs, states, failure handling and success conditions
F8	Compare any AI with a deterministic baseline on a measurable outcome
K1	No decryption of application traffic
K2	Linux traffic control or another open, inspectable enforcement mechanism
K3	IPv4 and, where available, IPv6
K4	Policy rollback; no starvation of low-priority traffic
K5	Document third-party licences; no redistribution against a licence
K6	Publish generated data and impairment settings for reproduction
K7	Label emulation and explain its model and limits
K8	No credentials, keys or tokens in source control
K9	Remediation bounded, observable and reversible; failure returns to a known safe state
D1	Dashboard for latency, jitter, loss, throughput, queue depth, fairness
D2	Automated baseline-versus-optimised experiments
D3	API (application programming interface) or UI (user interface) for temporary intent
D4	Architecture diagram (device, edge, network, data, analytics, UI)
D5	Setup guide with exact versions, commands, configuration, verification
D6	Scripted demo: reset, inject, collect, report
D7	Known limitations and the prototype–production boundary
J1	Accuracy of classification and link estimation
J2	Reduction in interactive latency, jitter and packet loss
J3	Fairness and absence of starvation
J4	Adaptation speed and policy stability
J5	CPU overhead, reproducibility and clarity of results

2.4 Implicit requirements

A careful reading of the brief, and of what an MCV panel will expect, adds these.

- **I1 Queue placement.** Prioritisation is useless if the queue sits in the ISP (Internet Service Provider) modem; the router must become the bottleneck by shaping just below capacity in both directions.
- **I2 Distinguish causes.** “Recalculate shaping” after a drop implies not recalculating when the delay has another cause (remote server, Wi-Fi, router CPU, bad probe).
- **I3 Honest semantics.** “Protect a call” must not be presented as a bandwidth reservation when the scheduler only gives priority and weights.
- **I4 Abuse resistance.** A device must not gain priority simply by marking its traffic.
- **I5 Coexistence.** Work beside the firewall, multi-WAN and routing packages of a real router.
- **I6 Fair comparison.** “Deterministic baseline” means the strongest open tools, tuned per their documentation, not a straw man.
- **I7 Indian deployment reality.** Fibre (FTTH/PON), 4G/5G fixed wireless access (FWA) and cable links with variable capacity, mixed IPv4/IPv6, and low-cost routers with little RAM and CPU.

2.5 Non-functional requirements

Abbreviations used from here on: SQM (Smart Queue Management, a static shaper), CAKE (Common Applications Kept Enhanced), HFSC (Hierarchical Fair Service Curve), HTB (Hierarchical Token Bucket), IFB (Intermediate Functional Block), ML (machine learning), DSCP (Differentiated Services Code Point), SNI (Server Name Indication), ACL (access control list), LuCI (the OpenWrt web interface).

ID	Quality	Requirement (all values are targets)	Where measured
N1	Latency	Static added delay p95 at parity with SQM (≤ +2 ms); 250 ms tick; confirmation ≤ comparator + 1 s	H1 runs, S1, S2/A1
N2	Safety	Daemon failure never breaks connectivity; time-to-safe ≤ 12 s; actions checked, bounded, logged, reversible	S8/A15, checker unit tests, A1–A32
N3	Privacy	Headers and timing only (netfilter log sampling, 40-byte snap length); no cloud dependency; flow records ≤ 24 h in RAM, aggregates 7 days; export only by opt-in	Code review, data inventory (Chapter 17)
N4	Footprint	Lite ≤ 20 MB resident memory (RSS), ≤ 6 MB flash; Standard ≤ 40 MB RSS, ≤ 5% of one A53 core; Pro ≤ 60 MB RSS	Stage 4 / H4 on D3–D5
N5	Reproducibility	Every published number regenerates from raw data; manifests record versions and capabilities; deterministic replay of decision logs	Claims check in CI, replay tests
N6	Usability	Setup wizard ≤ 5 min; household tasks without technical terms; SUS (System Usability Scale) ≥ 75, formative	UX study (Chapter 15)
N7	Security	Local API is a root-only Unix socket; role-based rpcd ACLs; signed packages, model and catalogue	Security tests (Chapter 17)
N8	Portability	One Go binary for x86-64, ARM64, ARMv7, MIPS; OpenWrt 24.10 (23.05 best effort)	CI build matrix

2.6 Assumptions and constraints

These are stated so that judges can see the boundary of every claim.

- **Team and time.** Three part-time members (docs/11 plans for 3–4; the listings say 2–3); about nine weeks to Round 2. If Go ramp-up fails in Sprint 0, the controller is first built in Python in the lab with the same interfaces and golden tests (O7).
- **Hardware.** A full-kernel Ubuntu 24.04 laptop; a Raspberry Pi 4 or 5 with a USB 3 Gigabit Ethernet adapter; later one ARM64 retail router and one MIPS low-end device.
- **Emulation.** The ISP bottleneck is emulated with `tbf` and `netem`. Emulated links are not real DOCSIS, PON or 5G schedulers, and every such result is labelled.
- **Measurement.** The ISP queue is inferred, never observed. Public ICMP (Internet Control Message Protocol) reflectors may rate-limit probes. Without a cooperating capacity-test server, evidence level L3 is unavailable (O6).
- **Kernel features.** The `ctinfo cpmark` syntax and the fwmark-to-tin mapping must be verified by the self-test on each target; a compatibility mode exists if they differ (O2, O3).
- **Licences.** QoSmate (GPL-3.0) and cake-autorate (copyleft): ideas only, no code. VeriQ's licence: Apache-2.0 proposed, Tata Elxsi decides (O5).
- **No partnership.** No Tata Elxsi data, hardware, integration or endorsement is assumed.

2.7 Requirement traceability matrix

Each requirement maps to the VeriQ feature that satisfies it, the chapter that specifies it and the test that will show it (S1–S13 core scenarios, A1–A32 adverse conditions; unit tests in Chapter 23, self-test in Chapter 16).

Req.	VeriQ feature	Chapter	Test or scenario
F1	Header-and-timing classifier: rules, sets, app catalogue, demote-only heuristics, ML	9	H2 tier; S3, A10, A27; classifier honesty-kit report
F2	Conformal sets (act only on singletons); provenance and confidence shown per flow; user rules as corrections	9, 13, 15	Coverage, selection rate, selective accuracy
F3	Reflector prober, baselines, $\hat{G}$, μ/σ estimator, capacity memory	10	E1–E8 estimator tests (proposed); S2, A1–A3, A7

Req.	VeriQ feature	Chapter	Test or scenario
F4	CAKE/HFSC/HTB backends, both directions via IFB and `act_ctinfo`; rate controller with diagnosis-gated cuts	8, 10, 11, 20	Tin self-test; S1, S2, S4, A9
F5	Receipts (IMPROVED / NOT_IMPROVED / INCONCLUSIVE); harness metrics from WebRTC getStats and dash.js	13, 24	S1, S2, S5; receipt reproduction from the ledger
F6	Authenticated, expiring leases with admission; eligibility envelope; starvation metric	9, 15, 21	S5, A13
F7	19 Go packages with APIs, tests and acceptance criteria	7, 19	Unit, property and replay tests
F8	ML vs heuristics and port rules; controller vs SQM, cake-autorate, QoSmate; ablations	5, 24	H1, H2 tiers; primary outcome in S6, S7, S10, A32
K1	Privacy contract: NFLOG snaplen 40, no SNI or DNS reading from packets	9, 17	Code review; data inventory
K2	nftables and `tc`, inspectable with `nft -j` and `tc -s -j`	8, 20	Self-test; read-back in every tick
K3	IPv4 and IPv6 sets, rules, leases and probes; device rules resolve all IPv6 addresses	9, 10	S9, A20
K4	Reconciler, fallback undo, package rollback; envelope; bulk tin keeps progress	12, 16	A12, A13; starvation metric = 0 in every 10 s window
K5	Clean-room policy; licence register; permissive Go modules only	17, 18	Licence check before each release
K6	Scenario YAML files, evidence ledger with manifests and raw data	24, 25	Claims check in CI
K7	“Emulated” labels on every lab result; emulator model documented	25, 31	Report template review
K8	No secrets in the repository; fictitious examples	17	Secret scanning in CI
K9	Action Checker, budgets, operating states, watchdog to SAFE, decision log	12	Checker refusal tests; S8, A15, A30, A31
D1	LuCI Overview, Queues and Autorate pages; optional exporter and Grafana	15	Playwright LuCI tests
D2	Lab runner with arms, tiers H0–H4 and Stage 1–4 ladder	24, 25	H0 smoke, H1 core
D3	JSON-RPC `lease.request`, household page, `veriqctl lease add`	15, 21	S5; household ACL tests
D4	Layered architecture and datapath diagrams	7, 31	Submission review
D5	Lab build guide and OpenWrt install guide with pinned versions	16, 25	Fresh-machine dry run
D6	Scenario runner; judge-controlled demo	25, 29	Demo rehearsal checklist
D7	Limitations in each chapter; footprint budgets; open decisions O1–O10	14, 28, 33	Claims check
J1	Classifier report; E1–E8; diagnosis confusion matrix	9, 10, 11	H2; A-series ground truth
J2	CAKE shaping plus diagnosis-gated control	8, 10	Added delay p95, game RTT p99, WebRTC freezes
J3	Host isolation, bulk tin, envelope, starvation metric, Jain index	8, 24	S3, A10, A12
J4	One-step L2 cut, refractory periods, no-spiral rule	10	Confirmation time; oscillation in A2
J5	Footprint budgets, bench, ledger, explain sentences	13, 16, 24	H4 soaks; replay determinism

Req.	VeriQ feature	Chapter	Test or scenario
I1	Shaper just below the bottleneck in both directions	4, 8	Day-0 reproduction; S1, S4
I2	Evidence groups, labels and permissions	11	S6, S7, S10, A4–A8, A17, A32
I3	“Preference, not guarantee” wording; policers labelled as policers	8, 15	UI copy review
I4	Eligibility envelope and sanctions; DSCP trust list	9	S11, A11, A12
I5	Mark bits 24–30 only; conflict scan; reconciler that never fights	12, 16, 20	Self-test; A30
I6	Arms tuned per their own documentation, configs archived	24	Tuning log review
I7	Link presets; Lite profile; capacity memory; Hindi strings	8, 15, 16	A16, A22, A23; D5 Lite run
N1–N8	See the non-functional table	12–17, 23–24	As listed there

The three illustrative scenarios of the brief map directly: the ISO download during a call is S1; the 100 → 20 Mbit/s drop is S2 (and A1); the three televisions plus a game is S3.

2.8 Why Case 3 was chosen

docs/02 scored all five cases on eight weighted dimensions (standout chance and low competition 20% each; Tata Elxsi relevance and panel appeal 15% each; feasibility and tech stack 10% each; scaling and cost 5% each).

Case	Weighted score /10	Main reason
CS4 Medical device labeling (HLS)	8.40	Strongest strategic fit with the healthcare pivot; visual demo
CS3 Adaptive QoS (MCV)	**8.15**	Least crowded (estimated 5–10% of teams), fastest-growing vertical, objective metrics
CS5 VehicleGPT (TBU)	6.65	Most hype-crowded
CS2 Battery platform (IDV)	5.65	Crowded with EEE and mechanical teams
CS1 Predictive maintenance (ACTG)	4.75	Saturated public dataset

> **Honesty check.** The scorecard ranked CS4 slightly higher and recommended CS3 only if a tie-breaker held (someone has used `tc` or network namespaces; the team prefers measurement-heavy work; nobody wants to read regulations). The team chose CS3. Crowd estimates are reasoned guesses, not public data.

The rationale for CS3, as recorded in the repository:

1. **Small but strong crowd.** Few students have used `tc`; at least one other team already has a public working prototype (Chapter 5). The bar is depth.
2. **Fastest-growing vertical.** MCV grew about 22% year on year in Q1 FY27 (approximate, investor deck cited in docs/01); its gateway work makes QoS a real product feature (Chapter 3).
3. **Objective scoring.** Latency, jitter, fairness and CPU can be measured, so a rigorous team can win clearly.
4. **A known trap, faced openly.** CAKE already removes most bufferbloat and cake-autorate already handles variable links. VeriQ’s contribution must therefore be diagnosis, intent, safety and product quality, proved against those tools. This handbook is built around that test.

The main risk is skill: kernel networking is hard to learn in nine weeks. Chapter 22 answers it with a harness-first order, the O7 Python fallback and a scope-cut ladder.

CHAPTER 3

Tata Elxsi Context

Every TELIPORT case is owned by a Tata Elxsi business unit, and the panel that judges it will think in that unit's terms. This chapter summarises what the team knows about Tata Elxsi, explains why a home QoS (Quality of Service) engine is relevant to its Media and Communications (MCV) vertical, maps VeriQ's assets to the kinds of work that vertical sells, and lists the questions a Tata Elxsi panel is likely to ask.

> **Note.** All company facts below come from docs/01 (researched 11 Oct 2026) and the sources it cites [R46]–[R50]. They are public figures and press coverage, not insider knowledge. Tata Elxsi reports its Q2 FY27 results on 13 Oct 2026; re-check every number in this chapter after that date and before any slide is finalised.

> **Honesty check.** Nothing in this handbook claims that Tata Elxsi has endorsed, reviewed, integrated or expressed interest in VeriQ. Where this chapter says VeriQ "could" support a Tata Elxsi offering, that is the team's reading of public information, not a statement of the company's plans.

3.1 Company overview

Tata Elxsi is a design and technology services company. Its growth model, as read in docs/01, is to win engineering-services work and attach reusable accelerators and platforms to those deals, rather than to sell mass-market products of its own.

Item	Value (approximate where stated)	Source
FY26 revenue	₹3,757.4 crore; PBT (profit before tax) margin 23.4%	FY26 investor presentation, National Stock Exchange (NSE) filing, 21 Apr 2026 (cited in docs/01) [R48]
FY26 PAT (profit after tax)	About ₹628 crore, roughly 20% lower year on year (single-sourced)	Press coverage cited in docs/01
Q1 FY27 revenue	₹1,021.1 crore, the first quarter above ₹1,000 crore; +2.8% QoQ (quarter on quarter), +14.5% YoY (year on year)	Company press release, 14 Jul 2026 [R47]
Q1 FY27 PAT	₹170.6 crore (+18% YoY, −23% QoQ)	Press coverage cited in docs/01
FY27 outlook	Management guided to "higher single-digit" growth, with margin pressure from wage hikes in Q2	Q4 FY26 call coverage cited in docs/01
Leadership	Manoj Raghavan, Managing Director and CEO	docs/01
Next event	Q2 FY27 results and call, 13 Oct 2026	docs/01

3.1.1 The three businesses

docs/01 reads the approximate Q1 FY27 revenue mix from the NSE investor deck [R48]. The PDF text extraction was noisy, so the shares are approximate.

Vertical	Approximate share	Momentum (Q1 FY27)	What it is pushing (per docs/01)
Transportation (TBU)	~55%	+13.3% YoY reported (+6.7% in constant currency); flat QoQ	Software-defined vehicles, a multi-agent generative-AI (GenAI) tool for automotive software (DevStudio.ai), partnerships for intelligent cockpits
Media and Communications (MCV)	~33%	**Fastest growing: +22.2% YoY** (+11.5% in constant currency)	RDK-B broadband gateways, an OpenWrt-to-RDK-B migration accelerator, the NEURON autonomous-network platform, a GenAI AdTech modernisation deal
Healthcare and Life Sciences (HLS)	~9%	Declined through FY26; +1.7% QoQ in Q1 FY27	An "AI-first, design-led, regulatory-aware" pivot (CEO's words), regulatory and software-as-a-medical-device (SaMD) platforms

Industrial Design and Visualization (IDV) is the design arm and works across all verticals. Case 3 belongs to MCV.

3.1.2 Strategy signals that matter for Case 3

docs/01 draws five signals from the company's 2026 announcements. Three of them bear directly on how VeriQ should be pitched.

1. **"Domain plus AI" is the company story.** Each 2026 launch pairs deep domain knowledge with AI. The company is not looking for generic chat demos; it wants AI that is safe in demanding domains. VeriQ's ML (machine learning) is deliberately narrow: an

abstaining classifier with conformal confidence, sitting behind a deterministic Action Checker. That is the same message in a networking setting.

2. **A services company grows by attaching accelerators.** RegAI, NEURON and DevStudio.ai are examples docs/01 cites. A tool-agnostic component that plugs into a client's existing platform fits this model better than a "replace everything" product. VeriQ's design (one daemon, kernel-standard datapath, documented porting path) is shaped for that.
3. **Margins are under pressure, and AI-led efficiency is a stated lever.** Anything that shortens a client programme's validation or support effort is relevant. VeriQ's evaluation harness and its plain-language diagnosis are both pitched at effort reduction, as hypotheses to be shown.

3.2 The MCV vertical and home broadband

MCV is the vertical closest to the home network. docs/01 lists three broadband signals, all from public announcements [R49]:

- **RDK-B broadband gateways.** docs/01 lists a DOCSIS (Data Over Cable Service Interface Specification) 3.1 router for an operator among this work, and launch coverage of a first RDK-B release on a Qualcomm 5G fixed-wireless-access (FWA) platform "with QoS features".
- **An OpenWrt-to-RDK-B migration accelerator.** This signals that the team works with OpenWrt-based gateway code and helps clients move it onto the RDK-B stack.
- **NEURON**, an autonomous-network platform [R46], reported in docs/01 as deployed with Sky.

3.2.1 Why these technologies matter

Term	What it is	Why it matters to VeriQ
OpenWrt	An open Linux distribution for routers, with its own package manager, configuration system (UCI, Unified Configuration Interface), service manager (procd) and web UI (LuCI) [R40][R41]	VeriQ's primary target; the same platform as QoSmate, SQM (Smart Queue Management) and cake-autorate, all built around the CAKE (Common Applications Kept Enhanced) scheduler
RDK-B	Reference Design Kit for Broadband, an open-source software stack for operator broadband gateways [R44]	The platform MCV builds gateways on; VeriQ's porting path names an RDK-B component wrapper (needs a partner)
prplOS	A carrier-grade, OpenWrt-based gateway distribution from the prpl Foundation [R44]	The shortest path from VeriQ's OpenWrt packages to an operator-grade build
TR-181 and TR-369 (USP)	The Broadband Forum's device data model and its User Services Platform for remote management [R27]	How an operator would configure and monitor VeriQ across a fleet (`Device.X_VERIQ_*`, P2)
5G FWA	Fixed wireless access over a cellular network	Capacity varies with the radio and cell load, which is exactly where adaptive shaping and diagnosis matter

The common thread is that **QoS on the gateway is a product feature that operators pay to have engineered and validated**, and that the variable-capacity access technologies operators are deploying make static QoS harder. VeriQ addresses that problem with open interfaces only (Linux `tc`, nftables, conntrack), which keeps it portable across OpenWrt, prplOS and, with porting work, RDK-B.

3.2.2 Indian context

The Indian home-broadband mix illustrates why the problem is real, without needing statistics. Homes are served by fibre (FTTH over PON, passive optical networks), by cable, and increasingly by 4G/5G FWA, often with a low-cost operator-supplied router and Wi-Fi as the only in-home network. Several of VeriQ's design choices respond to this:

- **Variable capacity** on shared PON and FWA links motivates the diagnosis-gated controller and capacity memory ("evenings are usually ~60 Mbit/s", an illustrative UI sentence).
- **Low-cost hardware** motivates the Lite profile (target: 128 MB RAM, MIPS-class) and the HTB backend for weak CPUs.
- **Multilingual households** motivate English and Hindi strings at P1 and a household page that avoids technical terms.
- **Data protection.** India's Digital Personal Data Protection Act 2023 [R45] may treat household flow metadata as personal data. VeriQ's default is local-only processing with no transfer; any operator telemetry would need notice and consent and a legal review before any pilot. This is a design stance, not legal advice (Chapter 17).

3.2.3 From OpenWrt package to operator gateway

A gateway engineer on the panel will ask what changes between VeriQ on a hobbyist router and VeriQ inside an operator's product. The design answers in stages; each stage needs its own evidence before the next.

Stage	What changes	What stays the same	Evidence needed before moving on
1. Linux lab (D1)	Nothing platform-specific; systemd unit, plain-file config	Daemon, datapath templates, checker, tests	Stage 1–3 evaluation in network namespaces

Stage	What changes	What stays the same	Evidence needed before moving on
2. OpenWrt package (D2–D5)	procd services, UCI schema, LuCI app, rpcd ACLs (access control lists), signed feed	Same daemon binary, same nft and tc renders	Package tests in a VM, footprint on three device classes, H1 head-to-head
3. prplOS build	A TR-181/USP data model (`Device.X_VERIQ_*`) exposed through the USP agent	OpenWrt packaging largely reusable	Operator-style remote configuration test; partner review
4. RDK-B component	A component wrapper with TR-181 parameters; platform logging and service conventions	Control logic and kernel datapath, where the platform kernel offers CAKE, `act_ctinfo` and IFB (Intermediate Functional Block)	Kernel-capability self-test on the target SoC (system on chip); partner validation
5. Field pilot	SHADOW-mode deployment in 20–50 opt-in homes, then opt-in LIVE	All of the above	Disagreement report; stop and rollback criteria (crash loop, CPU over budget, more complaints than the control group)

Stages 3 to 5 are P2 and depend on a partner. The kernel features VeriQ relies on are standard Linux, but a vendor kernel may omit or patch them; that is why the self-test, not a table, decides whether the fwmark-to-tin mapping and `ctinfo` work on a given device (open decisions O2 and O3).

3.3 The NEURON link

NEURON is described by Tata Elxsi as an autonomous-network platform [R46]. Autonomous networks, as an industry idea, close a loop: observe the network, decide, act, and verify, with increasing automation and decreasing human effort.

VeriQ is a closed loop at the scale of one home. Its control loop observes, diagnoses with explicit evidence levels, proposes an action, checks it against rules, applies it, and writes a receipt that says whether the outcome improved. Three VeriQ properties map onto what any autonomous-operations platform needs from the devices at its edge:

- **Explainable local decisions.** Each action carries its evidence (L0–L3), its checker verdict and its receipt. A fleet platform could aggregate receipts instead of raw flow data.
- **Bounded autonomy.** Budgets, refractory periods, the SAFE state and the dominance invariant make local automation predictable, which is a precondition for letting it run unattended across many homes.
- **Shadow mode.** A new algorithm or model version runs in SHADOW for 24 hours and produces a disagreement report. That is the shape of a staged fleet rollout.

> **Honesty check.** This is a thematic fit, not a compatibility claim. The team has no access to NEURON's interfaces or data model, and no integration with NEURON is designed, built or implied. If a judge asks, say exactly that.

3.4 How VeriQ maps to Tata Elxsi's kind of work

The table maps VeriQ assets to the types of engagement docs/01 describes for MCV. "Possible use" is the team's hypothesis about value; none has been discussed with the company.

Engagement type (from public information)	VeriQ asset	Possible use	Caveat
Gateway software engineering for operators and vendors (RDK-B, DOCSIS, 5G FWA)	`veriqd` daemon, datapath templates (nft and tc), service classes, mark layout	A reference adaptive-QoS component for a gateway programme, or a starting point for one	Needs porting, platform QA and a partner; RDK-B wrapper is P2
OpenWrt-to-RDK-B migration	Platform-neutral daemon (Go, exec of standard `tc` / `nft`), UCI schema, QoSmate and SQM importers	An example of moving QoS configuration between ecosystems with an importer and a validated renderer	The importer targets QoSmate/SQM formats, not vendor firmware
Validation and testing of gateway firmware	Lab harness, scenarios S1–S13 and A1–A32, evidence ledger, claims check, tin self-test	A reusable, scripted QoS validation suite with ground truth and statistics	Emulated access links; real DOCSIS/PON/5G behaviour needs lab equipment
Autonomous and closed-loop operations (NEURON)	Receipts, evidence levels, checker verdicts, shadow reports	Edge-side evidence for fleet-level analytics	Compatibility not established
Operator customer experience and support	Household page ("the slowdown is outside your home"), support bundle with hashed identifiers	Fewer or shorter "slow internet" support calls	A hypothesis; the KPI is defined for a pilot (Chapter 26)
"Domain plus AI" accelerators	Abstaining ML classifier, conformal confidence, honesty kit, deterministic checker	A worked example of AI that is bounded, explained and evaluated against baselines	ML is P1; its accuracy is unmeasured

An illustrative operator model (Chapter 27) puts the gross modelled avoided cost at about ₹1.08 crore per year for one million homes, using the formula N × r × 12 × c × p with N = 1,000,000 homes, r = 1.5% of homes calling support per month, c = ₹400 per call and p = 15% of calls avoided. Every input is an assumption until a pilot measures it, and the net value subtracts integration, validation, maintenance and training costs.

3.5 Careful wording

Judges from the company will notice any overreach. Use the left column; never the right.

Say	Do not say
"Designed with a porting path to prplOS and RDK-B; needs a partner"	"Ready for RDK-B" or "integrates with Tata Elxsi's stack"
"Thematic fit with autonomous-network platforms such as NEURON"	"Plugs into NEURON"
"Could support validation engagements of the kind MCV performs"	"Tata Elxsi needs this" or "Tata Elxsi will use this"
"The checker checks admissibility against the implemented rules"	"The checker guarantees safety"
"Leases give bounded preferential treatment inside CAKE"	"Leases reserve bandwidth"
"Hypothesis, to be tested in the head-to-head"	"QoSmate fails when…"
"Illustrative operator model"	"Tata Elxsi will save ₹1.08 crore"

Two more rules from the competition itself: submitted ideas become Tata Elxsi's property, so keep the repository private and confirm the rules before publishing anything; and do not use the company's logo or brand in a way that suggests a partnership.

3.6 What a Tata Elxsi panel is likely to probe

The panel will include engineers who build gateways and leaders who sell programmes. Expect both kinds of question. Each item gives the likely question and the core of an honest answer.

- **"CAKE already fixes bufferbloat and QoSmate already exists. What is new?"** Diagnosis before cutting, bounded and checked actions, receipts, leases with an eligibility envelope, and product hygiene. In static conditions we expect parity with QoSmate's CAKE mode and will report it; our hypotheses concern adverse conditions (S6, S7, S10, A32).
- **"How would this run on an RDK-B gateway?"** The daemon only needs Linux `tc`, nftables, conntrack and a socket API. On RDK-B it would need a component wrapper and TR-181 parameters; that is P2 work requiring a partner. We have not tested on RDK-B.
- **"What is the CPU and memory cost on a real CPE (customer premises equipment) device?"** Targets per profile (Standard ≤ 40 MB resident memory, ≤ 5% of one Arm Cortex-A53 core), measured in Stage 4 on a Raspberry Pi, one ARM64 retail router and one MIPS device. CAKE's own kernel cost is measured separately by the bench and never attributed to the daemon.
- **"How does it behave on 5G FWA, where capacity moves every few seconds?"** Scenario A2 (10–50 Mbit/s oscillation) tests it; the no-spiral rule and the bounded fallback limit damage when evidence is mixed. Lab emulation is not a real radio scheduler, and we say so.
- **"Can an operator manage it across a fleet?"** Through a TR-181/USP data model and signed package channels, with SHADOW mode for staged rollouts. Designed, not built.
- **"What happens if your daemon crashes on a customer's router?"** The kernel configuration persists; the watchdog moves the shaper to SAFE (target ≤ 12 s); procd restarts the daemon, which adopts the existing state. Scenario S8/A15 measures time-to-safe.
- **"Where is the AI, and does it beat a baseline?"** The ML classifier is compared with QoSmate-style heuristics and port rules on a time- and application-split test set, reporting coverage, selection rate and selective accuracy. The controller is deterministic by design; its comparison is against SQM, cake-autorate and QoSmate.
- **"How do you handle privacy and India's DPDP Act?"** Headers and timing only, no payload, local-only by default, short retention, opt-in export, hashed identifiers in support bundles; legal review before any pilot.
- **"Why should an operator trust your numbers?"** Every published number regenerates from raw data in the evidence ledger; ties and losses are kept; the only measured data so far is the Day-0 sandbox run, labelled preliminary.
- **"What would you need from Tata Elxsi to take this further?"** Access to a reference gateway and real access links for validation, a decision on licence (Apache-2.0 proposed), and a partner for prplOS or RDK-B porting and a small SHADOW-mode pilot (20–50 opt-in homes, with stop and rollback criteria).

> **For judges.** VeriQ is a student reference implementation built on open interfaces, as the case asks. Its relevance to MCV is that it treats home QoS the way an operator programme must: as a bounded, explainable, testable product feature, validated against the strongest open tools.

3.7 Pre-panel checklist

- Read the Q2 FY27 results (13 Oct 2026) and update the company table.
- Re-verify each figure in this chapter against the cited source; drop any that cannot be re-verified.

- Rehearse the ten questions above with a timer (60 seconds per answer).
- Check every slide against the “Careful wording” table.
- Confirm the IP and publishing rules on the official portal.

CHAPTER 4

The Problem: Delay Under Load in a Shared Home Connection

A household buys a "100 Mbit/s plan", runs a speed test, sees close to 100 Mbit/s, and still has frozen video calls every evening. This chapter explains why. It starts from how home broadband is built, derives bufferbloat from first principles, shows why the usual quality-of-service (QoS) answers fall short, and ends with the precise problem VeriQ must solve and how success will be judged.

Everything in this chapter is either well-established networking knowledge, a fact from the project documents, or a number that is explicitly labelled. The only measured data in the whole project is the Day-0 sandbox run described in the section on Day-0 evidence below [R51].

4.1 How home broadband works

4.1.1 The path of a packet

A packet from a laptop to a video-call server crosses several very different links. Each has its own capacity and its own buffer.

```
laptop/phone/console/TV
       |  Wi-Fi (shared radio, half-duplex) or Ethernet
       v
HOME GATEWAY (router: NAT, firewall, DHCP, Wi-Fi AP)   <-- VeriQ runs here
       |  Ethernet
       v
ACCESS DEVICE (ONT / cable modem / DSL modem / FWA CPE)  <-- often a deep FIFO
       |  access link: fibre, coax, copper or radio
       v
ISP ACCESS NETWORK (OLT / CMTS / DSLAM / base station) -> aggregation -> BNG
       |
       v
INTERNET (transit, peering, CDN) -> remote server
```

Three facts about this path drive the whole design: - **The slowest link sets the pace.** In most homes the slowest link is the access link, and the upload direction is usually far slower than the download. - **Queues form in front of the slowest link.** That is where packets wait, so that is where delay is created. - **The home gateway is usually not the slowest link.** It receives packets over gigabit Ethernet or fast Wi-Fi and forwards them into a slower access link. Unless it deliberately slows itself down, its own queues stay empty and it cannot influence anything.

The access device (customer-premises equipment, CPE) is often supplied by the Internet service provider (ISP). In many Indian homes it is a combined fibre terminal and Wi-Fi router; in others the household adds its own router behind it. VeriQ assumes a router the household or operator can program, typically running OpenWrt.

4.1.2 Access technologies

Technology	Where the bottleneck usually sits	How capacity varies	Implication for QoS
FTTH over PON (fibre to the home over a passive optical network, e.g. GPON/XGS-PON)	The plan rate enforced by the ISP, or the shared optical tree when many homes are busy	Usually stable; can dip at peak hours when the tree or aggregation is congested	Static shaping works most of the time; evening congestion may sit in the ISP aggregation, beyond the home
DOCSIS cable (Data Over Cable Service Interface Specification)	The shared coaxial segment, especially upstream, which uses request-and-grant scheduling	Varies with neighbourhood load, often at peak hours	A static rate that is right at noon can be too high at 21:00
DSL (ADSL/VDSL2 over telephone copper)	The copper line; rate depends on line length and noise	Mostly stable after training; re-sync can change it	Encapsulation overhead (ATM cells on ADSL, PTM on VDSL2) must be modelled or the shaper overshoots
4G/5G fixed wireless access (FWA)	The radio link and the cell's load	Varies by second and by minute: signal, interference, other users	The strongest case for adaptive rate control
Wi-Fi inside the home	The radio between the access point and a weak or distant client	Varies with distance, walls, interference and the number of active stations	A slow client can be the bottleneck even when the access link is idle; shaping the WAN cannot fix it

Two further cases matter in practice: - **Mesh Wi-Fi and powerline backhaul** add another variable link between the gateway and the client. - **PPPoE sessions** (common on Indian FTTH and DSL) add 8 bytes of overhead per packet and can reconnect, which briefly takes the link down and may change the address.

> **Note.** The table states general mechanisms, not measurements of any particular ISP. VeriQ never assumes a technology; the setup wizard proposes a link-layer preset from the router's own interface configuration, and the controller learns from what it observes.

4.2 Queues, buffers and bufferbloat

4.2.1 Why buffers exist

Packets arrive in bursts. A laptop on gigabit Ethernet can emit dozens of packets in a few microseconds, while the upload link drains them at a few megabits per second. Without a buffer, the excess packets would be dropped at once. A buffer absorbs the burst and drains it over the following milliseconds. This is good queueing: it comes and goes quickly.

The classic sizing rule for one long-lived TCP (Transmission Control Protocol) flow is about one **bandwidth-delay product** (BDP): the link rate multiplied by the round-trip time (RTT). For example: - 5 Mbit/s × 50 ms = 250,000 bits = 31,250 bytes, about 21 full-size packets (illustrative values). - With many flows, an even smaller buffer is enough in theory.

4.2.2 How loss-based TCP fills any buffer

Most bulk transfers use loss-based congestion control (Reno or CUBIC). The sender keeps increasing its sending window until a packet is lost, takes the loss as a congestion signal, backs off, then grows again. If the buffer in front of the bottleneck is a plain first-in, first-out queue (FIFO) with a large limit, loss happens only when that buffer is full. So the sender keeps the buffer **near full all the time**. Model-based congestion control such as BBR [R9] tries to avoid this, but a home cannot choose the congestion control of every app and server.

The result is a **standing queue**: a queue that never drains, which adds delay without adding throughput [R2]. When the standing queue is large, the effect has a name.

> **Definition. Bufferbloat** is excessive delay caused by oversized buffers that fill under load and stay full [R1][R10]. The link stays busy, so throughput looks fine; but every packet, including a call's audio or a game's state update, waits behind the whole queue.

4.2.3 Worked example: a 300 ms buffer at 5 Mbit/s

Suppose the upload buffer in a modem holds 300 ms of data at its line rate of 5 Mbit/s. This is the exact configuration of the emulated ISP in the VeriQ Day-0 lab (`ISP_BUF_MS=300`, `UP_MBIT=5`).

Quantity	Calculation	Result
Bits in a full buffer	5,000,000 bit/s × 0.300 s	1,500,000 bits
Bytes in a full buffer	1,500,000 / 8	187,500 bytes (about 183 KiB)
Full-size packets	187,500 / 1,500	125 packets
Time to send one packet	12,000 bits / 5,000,000 bit/s	2.4 ms
Delay for a packet joining a full queue	125 × 2.4 ms	300 ms

The lab script sets exactly this limit: `limit = bytes_per_second × ISP_BUF_MS / 1000`, so 625,000 × 300 / 1000 = 187,500 bytes for the 5 Mbit/s upload.

Now hold the buffer size in **bytes** fixed and change the rate (illustrative):

Line rate	Delay for 187,500 queued bytes
1 Mbit/s	1,500 ms
5 Mbit/s	300 ms
20 Mbit/s	75 ms
100 Mbit/s	15 ms

This table explains two common observations: - **Upload hurts more than download.** The same kind of buffer in front of a slower link means far more delay. - **A buffer that is reasonable at one rate is bloated at another.** When an FWA link drops from 50 to 5 Mbit/s, the buffer that used to cost 30 ms now costs 300 ms.

Compare the 300 ms buffer with the BDP above: at 5 Mbit/s and a 50 ms RTT it is about six times larger than one BDP. The extra five-sixths buys no throughput; it only stores delay.

4.2.4 What the household feels

A queue of 250 ms ahead of a voice packet means that packet arrives 250 ms late. The effects stack up: - **Delay.** Conversation becomes awkward: people talk over each other. The ITU-T E-model treats delay as an impairment to call quality [R26]. - **Jitter.** The queue grows and shrinks as TCP probes and backs off, so delay varies from packet to packet. Real-time apps must enlarge their jitter buffers or discard late packets, which shows up as robotic audio and frozen video. - **Loss.** When the FIFO finally overflows, it drops whatever arrives, including the call's packets. - **Slow everything else.** DNS lookups, TLS handshakes and web page loads each need several round trips. Adding 250 ms to each round trip turns a snappy page into a sluggish one.

If both directions are bloated (for example a cloud backup uploading while a game update downloads), a round trip pays both queues.

4.2.5 The fix in principle: move the queue to where it can be managed

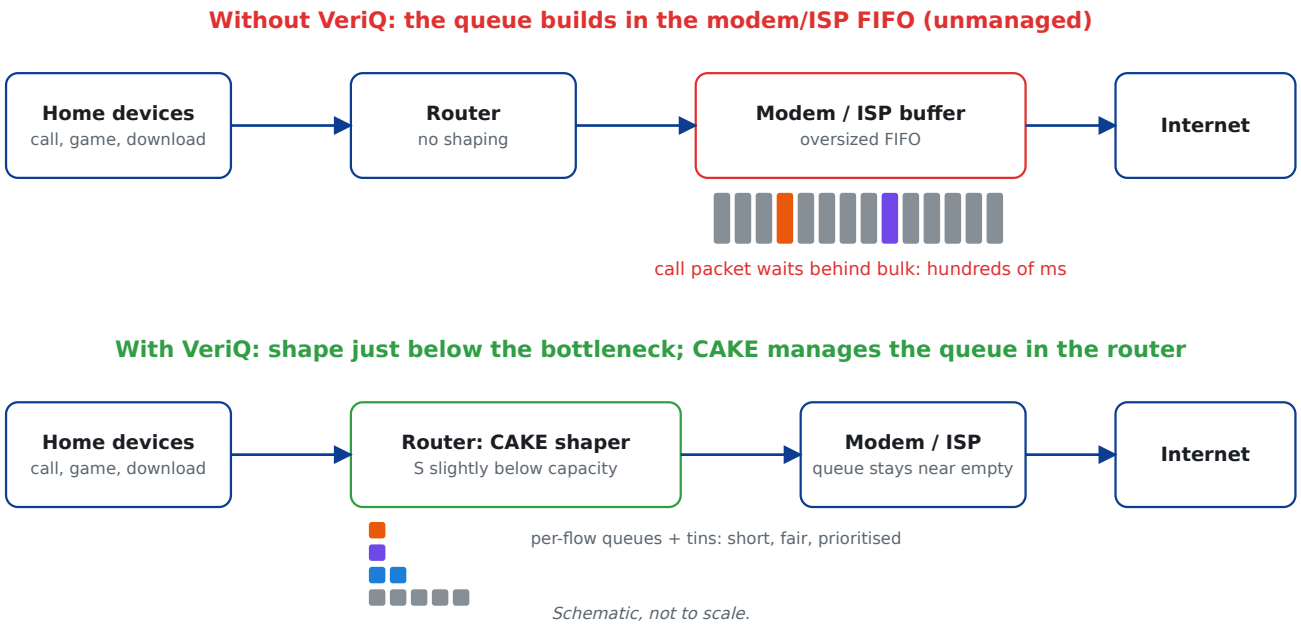


Figure 1. Queue in the modem's oversized FIFO versus the queue moved into the router's CAKE shaper.

The established fix has two parts [R5][R10][R11]: 1. **Shape just below the bottleneck.** The router limits its own sending rate to slightly less than the access link's capacity (for example 90–95% of it, a starting value). Now the router is the slowest point, so the queue forms in the router, and the modem's buffer stays nearly empty. 2. **Manage that queue intelligently.** The router runs an active queue management (AQM) algorithm with flow queueing. CoDel [R2][R3] drops or marks packets when the queue's sojourn time stays above a small target. fq_codel [R4] gives each flow its own queue. CAKE (Common Applications Kept Enhanced) [R5][R6] combines a shaper, flow isolation, per-host fairness and priority tins in one qdisc.

Neither part works alone: - **An AQM without a shaper does nothing** when the router is not the bottleneck: its queues are empty because the modem's queue is where packets wait. The lab README states this expectation for its `fq_codel` arm. - **A shaper without an AQM** (a short tail-drop buffer) reduces delay but drops bluntly and gives no fairness between flows.

The cost of the fix is a few percent of throughput, deliberately given up so that the router owns the queue. Chapter 8 covers the mechanisms in depth.

> **Why it matters.** Every adaptive feature in VeriQ rests on this foundation. If the shaper rate is above the true capacity, the queue moves back into the modem and no amount of classification or prioritisation in the router can help.

4.3 Why throughput tests miss it

A typical speed test opens several TCP connections, saturates the link for a few seconds and reports the average throughput. It may also report an "idle ping" measured **before** the load starts. Neither number shows what a call feels while someone else is downloading.

The Day-0 run (preliminary, one run, no base RTT [R51]) shows the gap directly. With a plain FIFO in the emulated ISP:

What a speed test would show	What a concurrent call would feel
Download goodput 19.1 Mbit/s on a 20 Mbit/s line	63.6 ms of added RTT at p95 while the download ran
Upload goodput 4.7 Mbit/s on a 5 Mbit/s line	266.1 ms of added RTT at p95 while the upload ran

By throughput alone, the FIFO looks excellent. By latency under load, the upload is badly bloated.

The metric that matters is **latency under load** (also called working latency or responsiveness): - measure delay to a reference point while the link is saturated, in each direction separately; - subtract the idle baseline; - report a high percentile (p95 or p99), not the mean, because calls and games suffer from the tail.

Public tools already do this: the Waveform bufferbloat test reports loaded latency per direction [R15], and the IETF responsiveness draft defines round trips per minute (RPM) under working conditions [R23]. The IETF Quality of Outcome draft goes further and maps network measurements to application outcomes [R24]. VeriQ's lab uses the same idea: `added_p95_ms` is the p95 RTT under load minus the idle median.

For judges. “Speed” and “responsiveness” are different properties. A connection can deliver its full plan rate and still be unusable for a call. VeriQ’s success is measured on responsiveness and on application outcomes (call freezes, video stalls, game delay), with throughput as a guard rather than the headline.

4.4 The home traffic mix and what each activity needs

A modern home runs very different traffic at the same time. The values below are **illustrative orders of magnitude** to show the contrast; they are not measurements and not requirements.

Activity	Traffic pattern	Most sensitive to	Throughput need (illustrative)	What hurts it most
Voice call (VoIP, app calling)	Small UDP packets at a steady rate, both directions	Delay, jitter, loss	Tens of kbit/s per direction	Any standing queue; tail drops in a full FIFO
Video conference (WebRTC-style)	Steady UDP media plus bursts at key frames; the sender adapts its bitrate	Delay, jitter, loss; then throughput	Hundreds of kbit/s to a few Mbit/s per direction	Upload bloat while a backup runs; sudden capacity drops
Online gaming	Small UDP state updates at a fixed tick rate (the lab’s synthetic game flow uses 64 packets/s)	Delay and jitter (p99 matters)	Tens to hundreds of kbit/s	Any queue; one late update is visible as lag
Cloud gaming	High-rate video down, small inputs up	Delay and jitter, and throughput	Several to tens of Mbit/s down	Bloat in either direction
Adaptive video streaming (DASH/HLS)	Large HTTP segments every few seconds; a buffer of tens of seconds	Throughput over seconds; tolerant of delay	Several Mbit/s per stream; more for 4K	Sustained throughput shortfall leading to rebuffering
Web browsing and apps	Many short flows, each several round trips	Delay (round trips), then throughput	Bursty	Every added round-trip delay multiplies page-load time
Large downloads (ISO images, games)	Few long TCP flows that fill any buffer	Total throughput (completion time)	As much as is available	Starvation, if others are prioritised without limits
OS and app updates	Long TCP or HTTP flows, often scheduled in the background	Throughput, eventually	As much as is available	Starvation; but they are the most common cause of bloat
Cloud backup and photo sync	Long upload flows	Upload throughput, eventually	As much upload as is available	They create the worst upload bloat
Peer-to-peer (e.g. BitTorrent of Linux ISOs)	Hundreds of flows, both directions	Throughput, eventually	Large	Per-flow fairness lets them win by flow count
Smart-home and IoT	Tiny periodic messages	Reliability	Very small	Rarely the issue

The table shows the core conflict: - **Interactive flows** (calls, games, page loads) need low delay but little bandwidth. - **Bulk flows** (downloads, updates, backups) need bandwidth but tolerate delay. - Both are legitimate. The case brief requires interactive applications to be protected **while bulk traffic continues to make reasonable progress**, and forbids starvation of low-priority traffic.

A second difficulty is that **importance changes with the moment**. The same laptop is a “work call” device at 10:00 and a “background update” device at 23:00. The same streaming service matters a lot on match night and very little when nobody is watching. No static table of ports or devices captures “what matters now”.

4.5 Why generic QoS fails

The obvious approaches each fail in a specific way. These are failure modes of **generic** approaches; mature open-source tools such as SQM and QoSmate already avoid several of them (see Chapter 5).

Generic approach	Why it falls short in a real home
Port-based rules (“prioritise port 3074”)	Most traffic now runs on TCP or QUIC over port 443, so ports no longer tell apps apart. Apps change ports, use CDNs, or tunnel. Port rules remain useful for known games and conferencing ranges, but they cannot be the whole answer.
Deep packet inspection (DPI)	TLS 1.3, QUIC and Encrypted Client Hello hide most of what DPI used to read. Signature databases need constant updates. Reading payload is a privacy problem, and the case explicitly forbids decrypting application traffic.
Trusting DSCP marks (Differentiated Services Code Point [R17])	Most apps do not mark. Some mark everything as high priority. ISPs may re-mark or bleach. Marks that leak into the home can change Wi-Fi access categories (WMM) [R18]. Trust without a behaviour check invites abuse.

Generic approach	Why it falls short in a real home
Priority queueing in the router without shaping	The queue is in the modem, not the router. The router's priority queues are empty, so the priority has nothing to act on.
Strict priority	A high-priority class that is allowed unlimited bandwidth starves everything below it. The case forbids starvation.
A static shaper rate	Right only while capacity does not move. Too high, and the queue returns to the modem; too low, and bandwidth is wasted every hour the link is healthy.
Shaping one direction only	Upload shaping is easy because it is the router's own egress. Download shaping needs a trick (an intermediate functional block, IFB, on the WAN ingress, or shaping on the LAN side). A setup that shapes only upload leaves download bloat untouched, and the reverse is also true.
Over-prioritisation	Marking many classes as "high" is the same as having no priority. QoSmate's own README warns that elevating too much traffic equates to no prioritisation at all (read).
Rules with no feedback	A rule is applied and nobody checks whether the call got better. When conditions change, the rule may now be useless or harmful, and nothing reports it.

> **Honesty check.** Encrypted traffic still exposes useful metadata: packet sizes, timing, direction and rates. Flow-fair queueing works well without knowing what any flow is. VeriQ's claim is not that classification is impossible, but that it must be payload-free, must admit uncertainty, and must never be the only basis for preference.

4.6 Problem 1: capacity moves

Static shaping assumes a fixed bottleneck rate. That assumption breaks on several common links: - **FWA radio links** change with signal strength, interference and the number of users in the cell. - **DOCSIS cable** and, to a lesser degree, **shared PON trees** slow down when neighbours are busy. - **ISP aggregation** can congest in the evening, beyond the access link. - **Mesh or powerline backhaul** can sit between the gateway and the access device. - **PPPoE reconnects and DSL re-syncs** change the link abruptly.

A static rate then faces a dilemma. Consider a line that delivers 100 Mbit/s most of the day but sometimes drops to 20 Mbit/s (illustrative, and the drop used in the case brief's own example):

Static choice	When capacity is 100 Mbit/s	When capacity drops to 20 Mbit/s
Shape at 90 Mbit/s	Low delay, 10% headroom given up	The router is no longer the bottleneck; the queue returns to the modem and bloat returns
Shape at 18 Mbit/s (90% of the worst case)	82% of the link's capacity is unused	Low delay

Adaptive rate controllers exist to escape this dilemma: they watch delay and throughput and move the shaper rate. Chapter 5 describes cake-autorate and QoSmate's autorate. They work, and they introduce the next problem.

4.7 Problem 2: the cause of a delay rise is not always the access link

Adaptive controllers usually decide from one signal: delay to a set of reflector hosts on the internet (often public DNS resolvers answering ICMP echo). When that delay rises above its baseline, they assume the access link has slowed and cut the shaper rate.

That inference is often right, but the same symptom has several causes:

Cause	What the reflector delay shows	Does cutting the home shaper help?	Better action
Access-link capacity dropped	Rises on all reflectors; throughput falls below the shaper rate while demand exists	**Yes**: the queue is now in the modem; a cut moves it back	Cut promptly, to just below the new capacity
Congestion beyond the ISP's first hop (remote path, a busy server, a transit link)	Rises on far reflectors; the first hop is flat; throughput is fine	**No**: the queue is not in the home's path segment	Hold the rate; tell the user the problem is outside
One reflector is slow or deprioritises ICMP	One reflector rises, the others do not	**No**	Exclude that reflector
All reflectors unreachable (ICMP blocked, outage)	No signal at all	**No** (there is no evidence either way)	Hold, and fall back to static behaviour
A Wi-Fi client is the bottleneck	Reflector delay flat; the client's station shows retries and low rates	**No**: the WAN is not the problem	Hold; tell the user which device and why
The router's CPU is saturated	Throughput below the shaper rate, with no matching delay rise from the access link	**No**: cutting the rate only hides the symptom	Hold; recommend a lighter queue backend

Cause	What the reflector delay shows	Does cutting the home shaper help?	Better action
The ISP deprioritises ICMP under load	Probe delay rises, but throughput and the router's own queues do not agree	**No**	Treat as a probe fault; hold or bounded steps only
The internet route changed to a longer path	Baseline RTT rises permanently	**No**: it is not queueing	Re-learn the baseline

A controller that cuts on every delay rise makes **inappropriate interventions**: it gives away bandwidth in situations where a cut cannot reduce delay. The household then gets both the original delay and a slower connection.

4.7.1 Worked illustration: a remote-path delay meets a delay-only controller

This is an **illustrative calculation**, not a measurement. Assume a delay-only controller that multiplies the rate by 0.85 about once per second while the delay delta stays above its threshold, with a floor at 25% of the base rate. These are QoSmate autorate's default factor, decrease refractory period and floor, read from its source (Chapter 5); whether QoSmate actually behaves like this in the scenario is a **hypothesis** to be tested in the head-to-head.

A remote server path gains 80 ms of delay for 60 s. Nothing in the home has changed and the access link is not full.

Time after onset	Rate (base 100 Mbit/s)
1 s	85
2 s	72
3 s	61
5 s	44
7 s	32
about 9 s	25 (floor)

The rate falls to the floor in about 9 s ($100 \times 0.85^9 \approx 23$, so the floor at 25 applies from the ninth step). The cumulative bandwidth given up, VeriQ's primary magnitude metric $\int \max(0, S_ref - S(t))dt$, is roughly: - about 390 Mbit·s during the first 9 s (15 + 28 + 39 + 48 + 56 + 62 + 68 + 73, one second each); - about 75 Mbit/s × 51 s ≈ 3,800 Mbit·s while held at the floor; - in total **about 4,200 Mbit·s** in one minute, plus the time to climb back afterwards.

None of that reduces the 80 ms, because the delay is not in the home's queue. VeriQ's primary research outcome counts exactly these interventions (count, cumulative magnitude in Mbit·s, and time below 0.95·S_ref) in control-futile scenarios.

> **Why it matters.** The cause problem is the gap VeriQ targets. The design goal is a controller that cuts as fast as existing tools when the access link really drops (the co-primary outcome) and holds when a cut cannot help (the primary outcome).

4.8 Problem 3: households need intent, not knobs

Even with perfect queue management and perfect capacity tracking, the router does not know what the family cares about right now. Households think in outcomes: - "Protect my daughter's exam call for the next hour." - "The console matters tonight; the TV updates can wait." - "Why is the internet slow, and is it our fault?"

Existing tools answer with knobs: DSCP classes, port ranges, HFSC curves, IP sets. Those are powerful for experts and opaque for everyone else. The case brief asks for something different: - **temporary intent**, such as prioritising a work call, **with fairness retained**; - classification that **exposes confidence and allows correction**; - verification of whether the policy **improved user experience**; - remediation that is **bounded, observable and reversible**, with rollback to a known safe state.

Intent also creates risks that a design must handle: - **Abuse.** If "priority" is free, everyone asks for it, or one device takes it all. - **Starvation.** A protected bulk transfer could starve everyone else. - **Stale intent.** A rule set for an exam that ended last week is still running. - **False comfort.** "Protected" is shown, but nothing measured whether the call improved.

VeriQ's answers (authenticated, expiring leases; an eligibility envelope on every preferred flow; receipts that compare before and after) are described in Chapters 6, 9 and 13. The precedence rule that ties them together is:

"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."

4.9 The Day-0 evidence

The team has one piece of measured data so far: a single sandbox run of the lab script on 11 October 2026 [R51]. It is reported here exactly, with its limits.

4.9.1 What was run

Item	Value
Script	`lab/quickstart_bufferbloat.sh` with `DURATION=15`
Topology	Four Linux network namespaces: client → router (arm under test) → emulated ISP → server
Emulated access link	`tc tbf` at 20 Mbit/s down and 5 Mbit/s up, each with a FIFO limit of 300 ms at line rate
Load	4 TCP flows (iperf3 3.16) for 15 s; ping every 0.1 s under load and every 0.2 s when idle
Metric	`added_p95_ms` = p95 RTT under load minus the idle median RTT; goodput from the iperf3 JSON
Arms that ran	`fifo` (no router shaping) and `shortfifo` (router `tbf` at 95% of line rate with a 15 ms tail-drop buffer)
Arms skipped	`fq_codel` and `cake`: the sandbox kernel lacked both qdiscs
Kernel and tools	Sandbox kernel 6.18.44; iproute2 6.1.0

4.9.2 Results (preliminary: one run, no base RTT, IPv4 only)

Arm	Direction	Added p95 delay	Goodput (same run)
fifo	Upload	**266.1 ms**	4.7 Mbit/s
fifo	Download	**63.6 ms**	19.1 Mbit/s
shortfifo	Upload	**39.6 ms**	4.4 Mbit/s
shortfifo	Download	**55.4 ms**	18.1 Mbit/s

4.9.3 How to read it

- **Upload FIFO: 266.1 ms.** This is close to the 300 ms ceiling from the worked example above, which is what loss-based TCP filling a 300 ms FIFO predicts. It is the clearest illustration of bufferbloat in the run.
- **Upload with a router shaper: 39.6 ms.** Moving the queue into the router reduced the upload figure by a factor of about 6.7 in this run, at a goodput cost of 0.3 Mbit/s (about 6%). The shaper here is a blunt tail-drop buffer with no AQM and no fairness.
- **Download: 63.6 ms vs 55.4 ms.** Shaping the download on the router's LAN side helped much less in this run. One plausible reason is that download shaping controls the upstream queue only indirectly: packets have already crossed the ISP's buffer before the router can delay them. This is an interpretation from one run, to be checked with replicated runs, CAKE's ingress mode and the IFB-based download path that VeriQ uses.
- **Goodput is almost unchanged across arms.** A speed test would rate all four rows as "good". Only the delay column shows the difference.

> **Honesty check.** These four delay numbers are the only measured results in the project. They come from **one** run, in an emulated topology, with **no base RTT** applied (the idle RTT was about 0.1 ms because netem was unavailable), **IPv4 only**, and **without the CAKE and fq_codel arms**. The original `environment.txt` in that run states "base RTT 20 ms (netem)"; that line is wrong and is corrected in the evidence README. Until the run is repeated at least 10 times per arm on a full kernel with netem, CAKE, fq_codel and IPv6, these figures are an illustration of the problem, not a reproduced result. Label them "preliminary, one run" wherever they appear.

4.9.4 Reproducing the Day-0 run

The recipe below follows `lab/README.md`. It needs an Ubuntu 24.04 machine (native or a VM that allows nested namespaces) with a full kernel. WSL2 kernels may lack CAKE.

```
# 1. Prerequisites
sudo apt install iproute2 iperf3 iputils-ping ethtool python3
sudo modprobe sch_cake sch_fq_codel sch_netem sch_tbf
# if modprobe fails: sudo apt install linux-modules-extra-$(uname -r)

# 2. Same settings as Day-0, but now with all four arms and a real base RTT
sudo DURATION=15 lab/quickstart_bufferbloat.sh            # fifo shortfifo fq_codel cake

# 3. Day-0 conditions for a like-for-like check (BASE_RTT_MS=0 mimics the missing
#    base RTT of the sandbox run; never publish these as realistic numbers)
sudo DURATION=15 DOWN_MBIT=20 UP_MBIT=5 ISP_BUF_MS=300 SHAPE_PCT=95 BASE_RTT_MS=0 \
     lab/quickstart_bufferbloat.sh fifo shortfifo

# 4. A faster line, closer to a modern FTTH plan
sudo DOWN_MBIT=100 UP_MBIT=20 lab/quickstart_bufferbloat.sh fifo cake
```

Each run writes `results/<timestamp>/` with the raw ping logs, the iperf3 JSON, `summary.csv` and `environment.txt`. Before quoting any number: 1. Open `environment.txt` and check that it says `base RTT: 20 ms (netem applied)` and `IPv6: enabled`. The current script records the capabilities it actually applied, so a missing capability is visible. 2. Check that every requested arm produced rows in `summary.csv`. A skipped arm leaves a `<arm>_apply.err` file explaining which qdisc was missing. 3. Repeat each configuration at least 10 times and report a median with a bootstrap confidence interval, not a single run (Chapter 24). 4. Copy the result directory into the evidence ledger with its manifest (Chapter 25).

The README states the expected pattern on a full kernel: large added delay for `fifo` (especially upload), little change for `fq_codel` alone, partial help from `shortfifo`, and a few milliseconds for `cake` at a small throughput cost. That pattern is an **expectation**, not a result, until the runs exist.

> **Implementation tip.** Offloads (GSO, GRO, TSO) are switched off on every lab interface so each shaper sees wire-sized packets. If you extend the script, keep this, or a shaper may release 64 KB "super-packets" and the delay figures become meaningless.

4.10 Problem statement and success criteria

4.10.1 The problem in one paragraph

A home connection is shared by interactive traffic (calls, games, browsing) and bulk traffic (downloads, updates, backups, streaming). When bulk traffic fills an oversized buffer in the modem or the ISP's equipment, every interactive packet waits behind it. The known fix (shape just below the bottleneck and manage the queue with CAKE) works while capacity is fixed and known. Three gaps remain: **capacity moves**, so a static rate is either bloated or wasteful; **delay rises have many causes**, and controllers that cut on every delay rise waste bandwidth where a cut cannot help; and **households need temporary, fair intent with honest feedback**, not technical knobs. VeriQ must close these gaps on an ordinary OpenWrt router, without reading payload, with every action bounded, checked, logged and reversible, and without ever leaving the household worse off than well-tuned static SQM.

4.10.2 Success criteria

Every value below is a **target** or a **pre-registered evaluation margin**, not a result. Chapters 23 and 24 define the metrics and statistics precisely.

Criterion	Metric	Target (to be measured)	Where tested
Dominance invariant	Behaviour wherever VeriQ lacks evidence	Holds or reverts to static CAKE equivalent to well-tuned SQM	Every scenario; A1–A32
Primary outcome	Inappropriate interventions in control-futile scenarios (S6, S7, S10, A32): count, cumulative magnitude ∫max(0, S_ref − S(t))dt in Mbit·s, duration below 0.95·S_ref	Lower than cake-autorate and QoSmate autorate	H1 head-to-head
Co-primary outcome	Confirmation time after a genuine capacity drop (S2/A1)	≤ comparator + 1 s (non-inferiority)	S2, A1
Secondary outcome	Bulk share of leftover bandwidth	≥ comparator − 5 percentage points (absolute)	S1, S4, S5
Static parity	Added delay p95 under load at fixed capacity	Parity with SQM (≤ +2 ms)	S1, S4
Call protection	WebRTC freezes and concealment with a concurrent download	0 freezes and added delay p95 ≤ 10 ms in S1 (docs/05 target)	S1
Fairness	Jain's index across hosts; starvation metric	Jain ≥ 0.9 in S3; starvation = 0	S3, A10, A12
Safety	Time from daemon failure to the SAFE static profile	≤ 12 s	S8, A15
Case coverage	Payload-free classification with confidence and correction; IPv4 and IPv6; rollback; no starvation; reproducible experiments	Each requirement traced to a feature and a test	Chapter 2 traceability matrix
Product footprint	veriqd memory and CPU on the Standard profile	≤ 40 MB RSS and ≤ 5% of one Cortex-A53 core	H4 hardware tier

> **For judges.** "VeriQ's promise is narrow and testable. When the access link really drops, it should respond at least as fast as leading open-source autorate controllers. When the delay comes from somewhere a rate cut cannot help, it should not cut. Wherever it cannot tell, it falls back to static SQM behaviour. Our only measured data today is one preliminary sandbox run that shows the bufferbloat problem itself; every other number in this handbook is a target that our lab will test."

CHAPTER 5

Landscape and Competitors

VeriQ does not enter an empty field. The core fix for bufferbloat (shape just below the bottleneck and run a smart queue) has existed in open source for years, and OpenWrt users already have several mature tools. This chapter maps that landscape fairly, describes the strongest open-source competitor, QoSmate, in detail from its source, and states exactly where VeriQ claims to differ and what it still has to prove.

Labels used in this chapter. - **(read)**: a fact from QoSmate's README or source at v0.5.48 (commit `c507843`, 28 Sep 2026) or luci-app-qosmate v1.0.14 (commit `11c6e64`) [R13]. The team read these files but has not yet run QoSmate. - **Per its documentation**: a statement about another tool from its documentation or the project's earlier notes, to be re-checked against the pinned version before publication. - **Hypothesis**: an expected behaviour that the head-to-head evaluation (Chapter 24) must test.

5.1 A taxonomy of approaches

Category	Examples	Strength	Cannot do alone	Relevance to VeriQ
Static shapers	HTB, HFSC, `tbf`; SQM with a fixed rate	Make the router the bottleneck	Track moving capacity	VeriQ's floor (the dominance invariant)
AQM and flow queueing	CoDel [R2][R3], fq_codel [R4], CAKE [R5][R6]	Short managed queues; flow and host isolation	Act on a queue in the modem	VeriQ's datapath core (CAKE by default)
Autorate controllers	cake-autorate [R12]; QoSmate autorate (read); CAKE `autorate-ingress` [R6]	Move the rate as capacity changes	Tell a capacity drop from delay caused elsewhere	The layer VeriQ redesigns
Rule and heuristic classifiers	qosify [R14]; QoSmate rules (read)	Mark known traffic; demote bulk	Express intent; show confidence; verify outcomes	Rule surface adopted; leases, abstention, receipts added
DPI-based commercial QoS	Retail router "adaptive QoS"	Recognisable app names	Work on encrypted traffic; explain decisions	Not adopted: headers and timing only
ISP-side approaches	Access-equipment AQM; subscriber shaping; managed CPE via TR-069/TR-369 [R27]; UDPST/TR-471 [R22]	Fix the operator's own segment	See home Wi-Fi or household intent	A deployment path via TR-181
New marking architectures	L4S [R20]; NQB [R21]	Low delay for cooperating flows	Help flows that do not use them	Complementary; later experiments
Measurement methods	Waveform [R15]; RPM [R23]; UDPST [R22]; QoO [R24]	Make latency under load visible	Change anything	Lab metrics; optional L3 evidence

Here AQM means active queue management, DPI deep packet inspection, CPE customer-premises equipment and L4S/NQB the low-latency architectures described later. The categories combine: SQM is a static shaper with an AQM; QoSmate adds a classifier and an optional autorate; VeriQ intends to add diagnosis, safety and intent layers on top.

5.2 The foundations: shapers and smart queues

Chapter 8 explains the mechanisms. CoDel drops or marks with ECN (Explicit Congestion Notification) when packets sit in the queue too long [R2][R3]; fq_codel runs CoDel per flow [R4]; CAKE adds a shaper, host isolation, NAT awareness, DiffServ tins (priority and weights, not reservations) and overhead compensation [R5][R6]; HFSC (Hierarchical Fair Service Curve) separates real-time from link-sharing service [R8]; HTB (Hierarchical Token Bucket) is a simpler hierarchy [R7]. All of them need a correct rate, and that is where the tools below differ.

5.3 SQM (sqm-scripts)

SQM ("Smart Queue Management") is the OpenWrt package family that brought CAKE and fq_codel to home routers [R11]. - It shapes upload on the WAN egress and download through an intermediate functional block (IFB) device on the WAN ingress. - It offers queue setup scripts such as `piece_of_cake.qos` (CAKE with a single tin) and `layer_cake.qos` (CAKE with DiffServ tins), plus fq_codel-based scripts. - It handles link-layer overhead (ATM, PTM, Ethernet framing) and offers a LuCI page (`luci-app-sqm`). - Rates are **static**: the user enters them, typically 80–95% of measured capacity (common community guidance).

Strengths. Simple, widely used, extensively documented, and effective on links with stable capacity. It is the reference arm in VeriQ's evaluation: "static SQM" is the floor the dominance invariant promises never to go below.

Limits. No adaptation to capacity changes, no traffic classification beyond what packets already carry in DSCP, no intent and no feedback about whether a setting helped.

5.4 qosify

qosify is an OpenWrt package for DSCP-based classification with CAKE [R14]. Per its documentation in the OpenWrt source tree: - an eBPF classifier sets DSCP values on packets from configured matches on ports, addresses and DNS-derived names; - behavioural heuristics move flows with a high packet rate to a bulk class and can favour flows with small average packet sizes; - CAKE with DiffServ tins then shapes and schedules by those DSCP values.

Strengths. Very low overhead (eBPF), compact configuration, and heuristics that capture the common case without user effort; VeriQ's earlier notes call qosify-style heuristics "the strongest simple rival" for classification.

Limits. Static rate; DSCP as the class carrier; no confidence or abstention; no intent; no verification of outcomes.

5.5 cake-autorate

cake-autorate is a community project that adjusts CAKE's bandwidth on links with variable capacity, such as LTE, 5G and satellite [R12]. VeriQ uses it as the co-primary comparator (pinned v3.x, tuned per its own documentation). The concepts below are **per its documentation**, as summarised in the project's design notes (docs/05); confirm each against the pinned release. - **Signals.** It reads achieved throughput from interface counters and measures delay to a set of reflectors. - **One-way delay (OWD).** With an ICMP-timestamp pinger, it estimates upload and download delay separately: the reflector's timestamps split the round trip into two one-way deltas whose constant clock offset cancels against a baseline [R16]. Echo-based pingers give RTT only. - **Per-reflector baselines.** Each reflector has its own baseline that rises very slowly and falls quickly. - **Bufferbloat detection.** A sample counts as "delayed" when its delta exceeds a threshold (about 30 ms by default in the version summarised); bufferbloat is declared when 3 of the last 6 samples are delayed. - **Load-gated rate changes.** Increases happen only under high load and low delay, in small steps (up to about ×1.04); decreases on bufferbloat are larger (down to about ×0.75); refractory periods separate changes (about 300 ms after a cut, about 1 s between drifts). - **Idle and reflector handling.** Probing sleeps when the link is idle, so an idle link produces no cuts; misbehaving reflectors are rotated out.

Strengths. A careful design (OWD, per-reflector baselines, multi-sample detection, load-gated changes, sleep at idle). VeriQ's v3.1 design took its fast-detector starting values from these published defaults rather than inventing constants.

Limits relative to VeriQ's question. It is delay-triggered: it does not try to tell an access-link drop from remote-path delay or a Wi-Fi problem. It controls the rate only; classification, intent and UI are out of scope.

> **Note.** cake-autorate is released under a copyleft licence (check its LICENSE file at the pinned version). VeriQ runs it only as an external comparator in the lab and copies no code.

5.6 QoSmate deep dive (read)

QoSmate is the most complete open-source home-QoS system for OpenWrt today. VeriQ v5 is explicitly designed to match its feature surface and to differ in the controller, safety and intent layers. Everything in this section is **(read)** from the README and source at v0.5.48 and luci-app-qosmate v1.0.14, unless marked otherwise.

5.6.1 Origins, requirements and structure

- QoSmate builds on the SimpleHFSCgamerscript by @dlakelan and integrates concepts from SQM, DSCPCLASSIFY and cake-qos-simple (README).
- It requires OpenWrt 23.05 or later with firewall4 and nftables; a legacy branch exists for 22.03 (README).
- The backend is a set of shell scripts: `/etc/init.d/qosmate` (procd service and CLI), `/etc/qosmate.sh` (renders nftables and tc), `/etc/qosmate-autorate.sh` (the autorate daemon) and a hotplug script, with UCI configuration in `/etc/config/qosmate`.
- The frontend, luci-app-qosmate, is a set of LuCI client-side JavaScript views with rpcd helpers (`luci.qosmate`, `luci.qosmate_stats`).

5.6.2 Schedulers

[illegible]	Structure (read)	Typical use (README)
`hfsc` (default)	HFSC root with a realtime class and four link-share classes; selectable leaf qdiscs	Gaming; older or less powerful routers
`cake`	CAKE on WAN egress and on the IFB, `diffserv4` by default, host isolation and NAT on	"Default settings usually work well"; no gaming-specific tuning needed
`hybrid`	HFSC root; realtime via the game qdisc, bulk via fq_codel, and CAKE for all other traffic	Combining a hard realtime class with CAKE's fairness
`htb`	HTB root with three classes and fq_codel leaves	Weak CPUs

5.6.3 The HFSC class tree (read)

`qosmate.sh` builds the following tree on each shaped interface (`RATE` in kbit/s; `DUR = max(25, 5 × 1500 × 8 / RATE)` ms):

Class	Role	Service curve (read)	DSCP mapped to it (priomap)
`1:11`	Realtime (“High Priority”)	`rt m1 gameburst d DUR m2 GAMERATE`	EF, CS5, CS6, CS7
`1:12`	Fast non-realtime	`ls m1 70% d DUR m2 30%` of RATE	CS4, AF41, AF42
`1:13`	Normal (default class)	`ls m1 20% d DUR m2 45%`	CS0
`1:14`	Low priority	`ls m1 7% d DUR m2 15%`	CS2, AF11
`1:15`	Bulk	`ls m1 3% d DUR m2 10%`	CS1

Further details (read): - `GAMEUP` / `GAMEDOWN` default to 15% of the direction’s rate plus 400 kbit/s, and `gameburst` is 10 × GAMERATE capped at 97% of RATE. - The realtime leaf (`gameqdisc`) can be `pfifo` (the default in the shipped config), `bfifo`, `fq_codel`, `red` or `netem`. The pfifo limit is `PFIFOMIN + MAXDEL × RATE / 8 / PACKETSIZE` packets, with defaults `PFIFOMIN = 5`, `MAXDEL = 24` ms and `PACKETSIZE = 450` bytes. - Non-game classes use `fq_codel` by default (`nongameqdisc`), or CAKE with `besteffort ack-filter`. - On the IFB (download) side, an extra class `1:2` carries the router’s own traffic.

Worked example (computed from the formulas read in the source, not measured). For a 20 Mbit/s upload (`UPRATE = 20000`): - GAMEUP = 20000 × 15 / 100 + 400 = 3,400 kbit/s; - gameburst = min(10 × 3,400, 0.97 × 20,000) = 19,400 kbit/s; - DUR = max(25, 5 × 12,000 / 20,000 = 3) = 25 ms; - pfifo limit = 5 + 24 × 20,000 / 8 / 450 ≈ 5 + 133 = 138 packets.

The HTB backend uses three classes (read): `1:11` priority (realtime and fast), `1:13` best effort (default) and `1:15` bulk, with priorities 1, 2 and 3, rate-scaled fq_codel leaves (interval `100 + 2 × 1500 × 8 / RATE` ms and a rate-dependent target), and bursts sized to about 10 ms of each class’s rate.

5.6.4 Download classification with ctinfo (read)

QoSmate’s key datapath idea, which VeriQ’s v4.1 design credits and re-implements differently: 1. Classification runs once, on **LAN → WAN** packets, in the nftables table `inet dscptag`, chain `dscptag` (hook `forward`, priority 0 by default; both configurable as `NFT_HOOK` and `NFT_PRIORITY`). 2. At the end of the chain, the chosen DSCP is stored in the connection-tracking mark: `ct mark set ip dscp or 128` (bit 7 flags “valid”). 3. On the WAN ingress, a tc filter runs `matchall action ctinfo dscp 63 128 mirred egress redirect dev ifb-<WAN>`: it restores the DSCP from the conntrack mark onto each downloaded packet, then redirects the packet to the IFB, where the download shaper runs.

So one rule that matches the upload direction also classifies the download direction of the same connection; the README states that mirrored inbound rules can never match.

> **Note.** VeriQ uses the same conntrack-plus-ctinfo principle but carries its class in private conntrack mark bits (24–30) and restores them to the packet mark (`ctinfo cpmark`) for CAKE’s `fwmark` tin selection, instead of carrying class in DSCP. That lets VeriQ wash DSCP freely and keep provenance bits for kernel-side precedence. QoSmate-style DSCP restore remains VeriQ’s documented compatibility fallback (open decision O3).

5.6.5 Classification surface (read)

- **Rules** (`config rule`): `name`, `proto`, `src_ip`, `src_port`, `dest_ip`, `dest_port`, `class` (a DSCP name from cs0–cs7, af11–af43 or ef), `counter`, `enabled`. Values accept lists, ranges, `!=` negation, `@set` references and IPv6 suffix matching (`::suffix/::mask`) for dynamic prefixes.
- **IP sets** (`config ipset`): static sets of addresses, ranges and subnets, or dynamic sets with a timeout, filled by `dnsmasq-full` from domain names (`nftset` into table `dscptag`). IPv4 and IPv6 must use separate sets.
- **Rate limits** (`config ratelimit`): per-target download and upload limits implemented as nftables meters that **drop** traffic over the rate, with a burst factor, negation and set references. The README states plainly that this is a policer, not a shaper or AQM.
- **Custom and inline rules**: a full custom nftables table, and inline statements included in the `dscptag` chain; both are validated (`validate_custom_rules`) before use.
- **Legacy options** (`VIDCONFPORTS`, `REALTIME4/6`, `LOWPRIOLAN4/6`) are kept but marked “use rules instead”.

5.6.6 Heuristics and link options (read)

Option	Mechanism (read)	Default (read)
`UDP_RATE_LIMIT_ENABLED`	UDP flows with DSCP above CS2 that exceed 450 packets/s (meter keyed by `ct id . ct direction`) are re-marked CS0	README table: 1; shipped config file: 0
`TCP_UPGRADE_ENABLED`	TCP flows under 150 packets/s (burst 150) that are not CS1 are marked AF42	1
`TCP_DOWNPRIO_INITIAL_ENABLED`	TCP connections that have carried fewer bytes than 500 ms at the download rate (`DOWNRATE × 500 / 8`) and have DSCP below CS4 (and not CS1) are set to CS0	1

Option	Mechanism (read)	Default (read)
`TCP_DOWNPRIO_SUSTAINED_ENABLED`	TCP connections that have carried more than 10 s worth of bytes at the download rate (`DOWNRATE × 10000 / 8`) and have DSCP below CS4 are set to CS1 (bulk)	1
`UDPBULKPORT`, `TCPBULKPORT`	Listed ports are treated as bulk	empty
`ACKRATE`	Pure ACKs over per-flow rate tiers are dropped with random probabilities (four meters)	empty in the shipped config; README suggests about 5% of UPRATE
`BWMAXRATIO`	Caps download relative to upload	20
`ACK_FILTER_EGRESS` (CAKE)	`auto` enables CAKE's ack-filter when download/upload ≥ 15	auto
`MSS`	SYN MSS clamped (536–1500) when either rate is below 3000 kbit/s	536
`WASHDSCPUP`, `WASHDSCPDOWN`	Reset DSCP to CS0 on egress (no leak to the ISP) and ingress (ISP marks do not reach LAN Wi-Fi WMM); CAKE uses its `wash` keyword, HFSC/HTB/hybrid use nftables	1 and 1

For a 90 Mbit/s download setting, the sustained threshold is 90,000 × 10,000 / 8 = 112,500,000 bytes, so a TCP connection is demoted to bulk after about 112.5 MB (computed from the formula, illustrative).

The README also documents **one-directional shaping**: a rate of 0 disables that direction (no IFB is created for `DOWNRATE = 0`), and the TCP down-prioritisation and ACK-filter `auto` features are switched off because their thresholds derive from the download rate.

5.6.7 User interface, setup and operations (read)

LuCI tabs (from the menu definition): Settings, HFSC, CAKE, Advanced, Rules, IP Sets, Rate Limits, Custom Rules, Connections, Statistics. - **Statistics** shows CAKE per-tin tables for egress and ingress (threshold, target, interval, peak, average and sparse delay, bytes, packets, drops, ECN marks) with charts, HFSC and hybrid per-class tables, and autorate charts with optional hourly history. - **Connections** shows the conntrack table with DSCP, packet and bit rates, and a filter.

Auto-setup (`/etc/init.d/qosmate auto_setup`, also from the UI): - runs a speed test with `speedtest-go`, or `python3-speedtest-cli`, installing one if missing and if flash space allows; - samples the WAN interface counters every second during the test and computes a **plateau rate**: the median of the 1-s samples that reach at least half of the peak; - if the tool reports more than 1.25 × what actually crossed the interface, warns and uses the measured value (`warn_if_overreported`); - optionally adds a gaming-device rule for a given IP; - the README cautions that router-based tests may underestimate and recommends LAN-side tests for precision.

Health check (`health_check`): config validity, service enabled state, presence of the `inet dscptag` table, presence of an hfsc/cake/htb qdisc on each shaped device (a disabled direction reports `tc:off`), required packages, and file integrity against an md5 registry; output as `status=…;errors=N`.

Distribution and updates: - the README installs the backend and frontend by fetching raw files from GitHub with `uclient-fetch`; - the init script offers `update` and `check_version` (release, snapshot, branch or commit channels), installs missing packages at runtime, preserves configuration across sysupgrade (`PRESERVE_CONFIG_FILES`), migrates configuration versions, and registers a hotplug handler for interface events; - extra CLI commands are `check_version`, `update`, `auto_setup` (and non-interactive variants), `validate_custom_rules` and `health_check`.

5.6.8 The autorate algorithm, step by step (read)

QoSmate's optional autorate daemon (`qosmate-autorate.sh`, managed by procd; off by default) is the part VeriQ's controller is designed to improve on. The description below paraphrases the source; it is not code.

```
every interval (default 500 ms):
  achieved_up   = rate from WAN tx byte counter since last loop
  achieved_down = rate from IFB tx byte counter since last loop
  every 2nd loop (about 1 s):
    for each reflector in "1.1.1.1 8.8.8.8 9.9.9.9" (in turn): ping -c 1 -W 1
    latency = MEAN of the RTTs that replied
    if no reflector replied: latency = 9999 ms (sentinel)
        -> if a previous good value exists, use that last good value instead
    baseline: first 10 samples = running average (always accepted)
              afterwards, only if BOTH directions < 50% load:
                  candidate = 0.9 * baseline + 0.1 * latency
                  accept only if candidate < baseline        # lower-only
  no rate decisions until 5 baseline samples exist
  delta = max(0, latency - baseline)                         # same value for both directions
  per direction, with load = achieved / current_rate:
    if delta > 10 ms and >= 1 s since this direction's last change:  rate *= 0.85
    elif load >= 75% and delta < 5 ms and >= 3 s since last change:  rate *= 1.02
    elif load < 50%:  drift toward base, every loop (x0.98 above base, x1.01 below)
    clamp to [min, max] = [25%, 105%] of the configured rate (base = 100%)
  direction guard: do not reduce a direction with load < 50% while the other is >= 75%
  apply changed rates with tc; write state to /tmp/qosmate-autorate-state
```

Additional facts (read): - The 5–10 ms band is a neutral zone: no change. - The decrease branch does **not** check load. - Rates restored from the state file are reused if the daemon restarts within 300 s. - History is kept in RAM, with optional hourly consolidation to flash. - The README recommends autorate for variable links (LTE, cable at peak) and says a stable FTTH link typically works well with a static setting.

5.6.9 What the read logic implies (hypotheses)

The logic above is simple and sensible for its main purpose: reacting to a real capacity drop on a variable link. The rows below are **hypotheses** derived from reading the code. They are not observed failures. Each one will be tested in the head-to-head (tiers H1 and H3).

Condition	Read fact that matters	Hypothesis to test	Scenario
Remote-path delay (+80 ms beyond the ISP's first hop)	Decrease branch has no load or cause check; the increase branch needs delta < 5 ms	Rate steps down ×0.85 about once per second to the 25% floor and stays there while the delay lasts. Recovery afterwards is about 70 s by drift at low load (×1.01 per 500 ms loop) or about 3.5 min under load (×1.02 per 3 s, about 70 steps); illustrative	S6 / A4
One lying reflector	Latency is the **mean** of three RTTs	With three reflectors at a 20 ms baseline, one rising to 51 ms moves the mean to 30.3 ms, a delta of 10.3 ms, which crosses the 10 ms threshold; a median would stay at 20 ms	S7 / A5
All reflectors unreachable	9999 ms sentinel replaced by the last good latency	The controller freezes on a stale value: if the outage began during congestion (delta > 10 ms), cuts may continue to the floor; otherwise it holds or may even raise under load without any fresh delay evidence	A6
Base RTT rises permanently (route change, +15 ms)	Baseline can only decrease after the first 10 samples	The delta stays elevated indefinitely: above 10 ms → repeated cuts toward the floor; between 5 and 10 ms → no upward probing	A7
Upload-only saturation	One latency value drives both directions; guard needs the other direction ≥ 75% loaded	When the idle direction is below 50% and the busy one ≥ 75%, the guard protects it; at intermediate loads both directions may be cut	S4 / A9
Idle link with delay noise	Decisions do not require load for decreases	A delay spike at idle can trigger a cut; drift returns the rate to base afterwards	A29
PPPoE reconnect	Interface checks end the loop if the device disappears; procd restarts the daemon	Behaviour around the outage depends on timing; to be observed rather than predicted	A14

> **Honesty check.** Some of these may never matter in practice. For example, public resolvers are usually stable reflectors, and many homes never see an upward route change. If the head-to-head shows QoSmate autorate making no more inappropriate interventions than VeriQ, the team reports parity and reconsiders the extra components. Earlier project notes described the 9999 ms value as reaching the decrease branch directly; the v0.5.48 source includes the last-good-value guard described above, and this handbook follows the source.

5.6.10 QoSmate's strengths, stated plainly

- **Mature and community-proven:** active development, an OpenWrt forum community, real users on real routers.
- **Choice of scheduler** for gamers, general users and weak CPUs.
- **A sound datapath:** conntrack plus ctinfo classifies both directions with one rule; washing protects Wi-Fi WMM and avoids leaking marks upstream.
- **A rich rule surface** with sets, IPv6 suffix matching, policers and validated custom nftables.
- **Useful heuristics out of the box**, a complete LuCI app with per-tin statistics, and practical setup (plateau-based auto-setup, health check, one-directional shaping, migration, hotplug).
- **Honest documentation:** it warns against over-prioritisation and calls rate limits policers.

VeriQ's feature-by-feature parity matrix (docs/11 §16) adopts most of these ideas, re-implemented clean-room. QoSmate is GPL-3.0; VeriQ takes ideas only, and its QoSmate importer is written from the README's documented configuration format.

5.7 Commercial router "adaptive QoS"

Many retail routers advertise an “adaptive QoS”, “smart QoS” or “gaming mode” feature. In general terms (the team has not audited any specific product), these combine a licensed DPI engine that matches traffic against a vendor-maintained signature database (destination, port, visible metadata such as the TLS server name), category or device rankings mapped to priority queues, and a bandwidth figure entered once or measured at setup. They are polished: app icons, usage charts and one-click modes.

Limits relevant to the case: - encryption (TLS 1.3, QUIC, Encrypted Client Hello) removes more of the metadata that signatures rely on; - decisions are opaque, and nothing reports whether a priority helped; - static rates bring bufferbloat back when capacity moves, and priority has nothing to act on while the queue sits in the modem; - reading application metadata raises privacy questions, and the case forbids decrypting traffic.

VeriQ takes one lesson from them: households respond to app and device names, not DSCP codes. Its app catalogue and household page aim for that, while staying header-only and explainable.

5.8 ISP-side approaches

Operators can attack the problem inside their own networks: - **AQM in access equipment.** Newer access standards build queue management into modems (DOCSIS 3.1, for example, specifies AQM for cable modems), which shortens the queue at the true bottleneck. - **Managed CPE.** Operators configure gateways through TR-069 or TR-369 (USP) using the TR-181 data model [R27], and can trigger IP-layer capacity tests (TR-471, built on UDPST [R22]). - **Low-latency deployment.** Operators are documenting L4S and NQB deployment experience [R25].

None of these sees inside the home: the household’s Wi-Fi, devices and priorities. The home gateway remains the natural place to manage the household’s uplink queue, diagnose Wi-Fi and accept intent. VeriQ’s porting path (prplOS with a TR-181/USP data model, an RDK-B component wrapper) would let an operator run the same logic on managed CPE; no Tata Elxsi integration or endorsement is implied.

5.9 L4S and NQB

Two IETF architectures aim to make low delay the norm rather than the exception: - **L4S** (Low Latency, Low Loss, Scalable throughput) [R20] uses the ECT(1) ECN codepoint to identify flows with scalable congestion control, and a DualQ Coupled AQM (DualPI2) to give them a shallow queue while keeping classic flows fair. - **NQB** (Non-Queue-Building per-hop behaviour) [R21] uses DSCP 45 to mark flows that send at low rates and do not build queues (for example, game state or DNS), so a network can give them a separate shallow queue. It is explicitly not a priority mechanism.

Both need cooperation from endpoints and networks. VeriQ’s stance: - it maps DSCP 45 to its interactive-video class (IV), not to realtime, because NQB means “does not build queues”, not “priority”; - the eligibility envelope still applies, so a misbehaving NQB-marked flow is sanctioned to best effort; - ECT(1) does not create L4S behaviour in CAKE; a DualPI2 arm and an NQB lane remain post-core experiments (P2).

5.10 The rival student prototype: AQE

A public repository (`sidhantiitian17/adaptive-qos-engine`, read on 12 Oct 2026 at commit `4040228`, not run) contains the exact Case 3 problem statement and a working prototype. It is almost certainly another team in the same case (docs/07).

What it does (read):

Module	AQE implementation
Classifier	XGBoost on per-packet IP length, TTL and inter-arrival time, trained on three iperf3 generators that stand in for apps; reports 99.1% accuracy (self-reported, on synthetic traffic)
Capacity estimator	SLoPS-style active probing (the pathload method [R28]) to a cooperating receiver, with a passive fallback
Policy and enforcement	CAKE bandwidth = 0.95 × estimate, diffserv4, plus iptables DSCP rewriting
Verifier	Three pings after each change; rollback if average RTT > 60 ms or loss exceeds a threshold
Intents	Natural-language parsing with a zero-shot model (confidence around 0.3 by its own note)
Evidence	SQLite evidence database, scenario runners, report generator, traceability matrix, known-limitations document, about 5,900 lines of Python and 97 tests

What it does well, and VeriQ adopts as general practice (ideas only; the repository has no licence, so no code may be copied): - active capacity probing on an idle link, credited to the original SLoPS paper; - a fixed estimator test matrix (VeriQ’s E1–E8); - an evidence ledger, a claims check in CI and a requirement traceability matrix; - deployment hygiene (install, uninstall, verify scripts) and a failure-injection script.

Where VeriQ's design differs (design claims, not results): - flow-level, payload-free, abstaining classification evaluated on real WebRTC and DASH traffic with time and application splits, never using TTL (which reflects the OS and path, not the app); - outcomes verified with application metrics and receipts that can say INCONCLUSIVE, instead of a fixed ping threshold that can roll back a good change after the WAN itself degrades; - diagnosis before acting; DSCP washed, with class carried in conntrack marks rather than rewritten upstream; - intent as authenticated, expiring leases; baselines that are real, tuned SQM, cake-autorate and QoSmate, with base RTT emulated and timing measured from the injected event.

> **For judges.** The pitch never names or attacks another team. The neutral version is: "Many CAKE-based controllers report ping. VeriQ reports what users feel, diagnoses the cause before acting, and compares itself honestly with leading open-source tools."

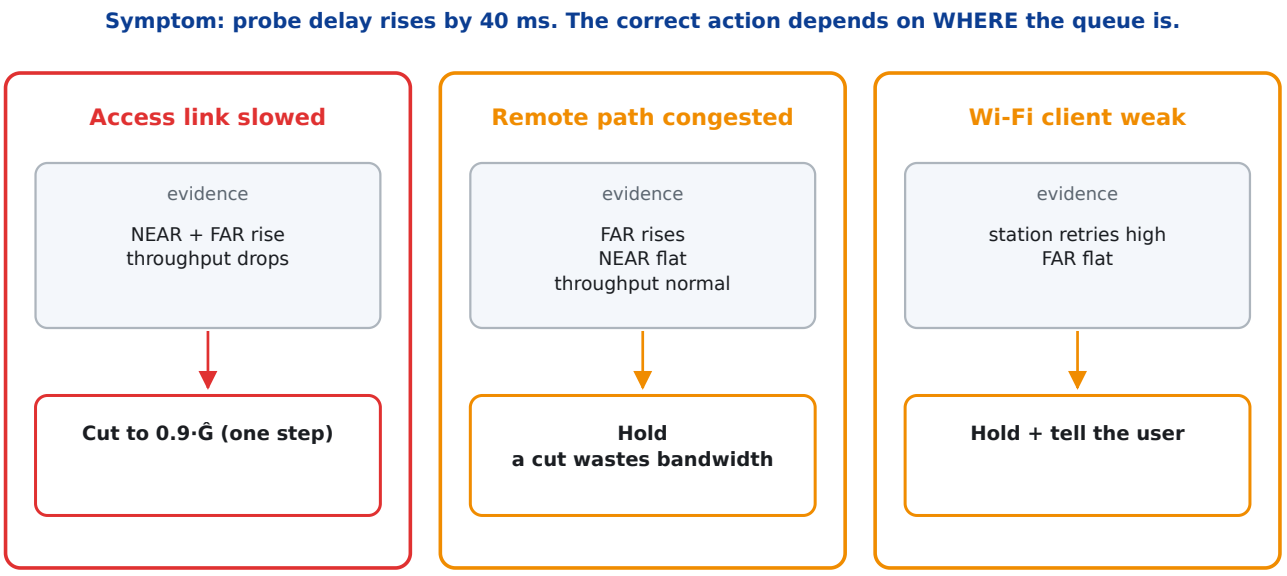
5.11 Comparison at a glance

Condensed from the v4.1 competitive table and the v5 parity matrix. "Designed" means specified, not yet built or measured.

Capability	SQM	qosify	cake-autorate	QoSmate v0.5.48 (read)	VeriQ v5 (designed)
Shaping both directions	CAKE or fq_codel, IFB for download	CAKE	Adjusts an existing CAKE setup	HFSC (default), CAKE, hybrid, HTB	CAKE (default), HFSC, HTB
Adaptive rate	No	No	Yes	Optional daemon	Yes, diagnosis-gated
Delay signal	—	—	Per-reflector baselines; OWD with timestamp pingers (per its docs)	Mean RTT of reflectors; lower-only baseline	Median of healthy reflectors, quorum ≥ 2; OWD where available; windowed-minimum baseline
Fault localisation before cutting	No	No	No (reflector rotation only)	Not found in the read files	Core design hypothesis (L0–L3 evidence)
Download class via conntrack	No	DSCP-based	No	DSCP restore via ctinfo	Private ct mark bits → CAKE fwmark
Rules, IP and domain sets, policers	No	Partial	No	Yes	Yes, plus device, app and schedule targets
Behavioural heuristics	No	Yes	No	Yes (promote and demote)	Demote or reset only, plus a per-flow eligibility envelope
Classifier confidence and abstention	No	No	No	No	Yes (conformal sets; P1)
Temporary user intent	No	No	No	Rules only	Authenticated, expiring leases
Checker, shadow mode, receipts	No	No	Logs	Not found in the read files	Core
UI and diagnostics	LuCI page	Config files	Logs	LuCI (10 tabs), health check	LuCI, household page, CLI, health check plus tin self-test
Maturity	Widely used	In the OpenWrt tree	Established community project	Active community project	Design complete; no code yet

Reading the table. Absence in the read files does not prove absence elsewhere. QoSmate and SQM are far more mature products today. VeriQ's possible advantages are design claims until the evaluation runs.

5.12 Where VeriQ positions itself



A delay-only autorate sees one number and cuts in all three cases (hypothesis to test).

Figure 2. Same delay rise, three causes (access link, remote path, Wi-Fi), each needing a different correct action.

VeriQ's one-line pitch is: *"Everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM."* That breaks into three layers: 1. **Parity layer.** QoSmate-class queue management, classification and UI, with a configuration importer. Here VeriQ aims to match, not beat, and expects parity in static conditions because both use CAKE. 2. **Controller layer.** Diagnosis before action: the same delay rise in the figure above leads to a fast one-step cut when the access link dropped, a hold when the delay is on the remote path, and a hold plus advice when a Wi-Fi client is the bottleneck. Every action is checked, bounded, logged and reversible. 3. **Product and intent layer.** Leases with an eligibility envelope, receipts that state association rather than causation, shadow mode before live control, a watchdog that falls back to static CAKE, signed packages and a household page.

VeriQ deliberately **differs** from QoSmate in four places, each with a reason: - **No download cap relative to upload** (`BWMAXRATIO`): a warning only, because a cap silently wastes download capacity; ACK problems are handled by CAKE's ack-filter. - **No nftables ACK thinning** (`ACKRATE`): fewer hand-tuned drop rules; no ACK drops on HFSC/HTB. - **No slow-TCP upgrade on CAKE** (`TCP_UPGRADE`): CAKE's sparse-flow boost already favours low-rate flows, and preference needs an explicit basis; on HFSC/HTB it becomes a "fast best-effort" sub-class, never realtime. - **No in-script self-update**: updates only through the package manager, for supply-chain safety.

5.13 The hypotheses VeriQ must prove

Hypothesis	Metric	Comparator	Scenarios	What would count against VeriQ
Fewer and smaller inappropriate interventions where a cut cannot help (primary)	Count, Mbit·s and time below 0.95·S_ref	cake-autorate; QoSmate autorate	S6, S7, S10, A32	No reduction, or a reduction that comes with worse delay
Responds to genuine drops as fast (co-primary)	Confirmation time after the drop	≤ comparator + 1 s	S2, A1	Slower by more than 1 s
Bulk traffic still progresses (secondary)	Bulk share of leftover bandwidth	≥ comparator − 5 pp	S1, S4, S5	Bulk starved or slowed beyond the margin
Parity in static conditions	Added delay p95; call freezes	SQM, QoSmate CAKE mode	S1, S4	Worse than static SQM by more than the parity margin (≤ +2 ms target)
Robust to reflector and baseline faults	Rate change attributable to the fault	QoSmate autorate	A5, A6, A7	Cuts caused by one reflector, an outage or a route change
Cuts only the loaded direction	Idle-direction rate change	QoSmate autorate	A9	Collateral cuts of the idle direction
Safe on failure	Time to SAFE; outage	—	S8, A15	Time-to-safe above 12 s, or any internet outage
Easier to use	Task success, time, SUS score	QoSmate LuCI	UX study (formative, 6–8 people)	No improvement in task success or SUS

Hypothesis	Metric	Comparator	Scenarios	What would count against VeriQ
Fits real routers	RSS, CPU, flash	QoSmate	H4 hardware tier	Standard profile above 40 MB RSS or 5% of one A53 core

Two rules govern how these results will be reported: - **Ties and losses are published**, not hidden. A parity result is reported as parity. - **Every published number regenerates from raw data** in the evidence ledger; until then, every VeriQ advantage in this chapter is a hypothesis.

> **For judges: "Why not just use QoSmate?"** "If your link is stable and you are happy tuning it, QoSmate is excellent, and VeriQ expects to match it there. VeriQ is for the cases QoSmate's autorate was not designed to reason about: delay that comes from somewhere a rate cut cannot fix, reflector faults, Wi-Fi and CPU bottlenecks. It also adds what a household needs: temporary protection with fairness, an explanation of every decision, and a tested design goal of never being worse than static SQM. Our head-to-head will show whether those design advantages are real; if they are not, we will say so."

Part II · The solution

What VeriQ is and how every part of it works

CHAPTER 6

Solution Overview

This chapter describes VeriQ as a whole: the idea, the one promise the design is built around, the nine principles, the three speeds and layers, the 250 ms control loop, what VeriQ deliberately does not do, the fourteen v5 changes (C1–C14) and one household evening. Every number here is a **starting parameter**, a **target** or **illustrative**; the only measured numbers in the project are the Day-0 sandbox values (Chapter 4).

6.1 The VeriQ idea

Home broadband is shared by video calls, gaming, streaming, downloads, backups and updates. When a bulk flow fills an oversized buffer in the modem or at the Internet Service Provider (ISP), every interactive flow waits behind it. The established fix is Smart Queue Management (SQM): shape just below the bottleneck rate so the queue forms in the router, where an Active Queue Management (AQM) scheduler such as CAKE (Common Applications Kept Enhanced) [R5] keeps it short and fair. VeriQ starts from that fix and addresses three problems it leaves open:

1. **Capacity moves.** 4G/5G Fixed Wireless Access (FWA), cable at peak hours, shared fibre (FTTH/PON) and Wi-Fi backhaul all vary. A static shaper rate is either too high, so the bloat returns, or too low, so bandwidth is wasted.
2. **Delay can come from places the router cannot fix:** a distant server, a misleading measurement target, a weak Wi-Fi client or an overloaded router CPU. A controller that cuts the rate whenever delay rises wastes bandwidth in all of these cases and helps none of them.
3. **Households think in intentions, not knobs.** “Protect my exam call for an hour” is the request; the household also needs an honest answer to “did it help?”.

VeriQ is an OpenWrt-native adaptive home QoS (Quality of Service) system. Its one-line pitch, from the v5 specification, is:

> *“Everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM.”*

“Everything QoSmate does” is the queue-management, classification and UI layer, built to parity with QoSmate [R13] and able to import its configuration. “Knows when not to cut” is the diagnosis-informed rate controller. “Proves what it did” is the receipt and decision-log machinery. “Never worse than plain SQM” is the dominance invariant.

For judges. The pitch makes one design claim (feature parity with QoSmate, checked in the parity matrix, Chapter 5) and one testable claim: fewer and smaller inappropriate rate cuts than delay-only controllers where cutting cannot help. That is the primary outcome (Chapter 24), and it is a hypothesis until the head-to-head runs.

6.2 The dominance invariant

No network system can promise perfect results under every condition: if the ISP link fails, nothing in the home can repair it. VeriQ makes a narrower, testable promise:

> **Dominance invariant (design goal, tested in every scenario).** In every condition where VeriQ lacks evidence to act, it holds or reverts to a static CAKE configuration equivalent to well-tuned SQM. Its adaptive behaviour is only allowed where evidence supports it, through bounded steps. So by construction, VeriQ’s worst case is “static SQM”, and its best case is better.

Four mechanisms make the invariant concrete:

- **Static CAKE from minute one.** On install, VeriQ applies a static CAKE shaper at the wizard’s base rate. The adaptive controller first runs in SHADOW mode, where it computes and logs but applies nothing, for 24 hours by default.
- **Evidence permissions.** A large cut is permitted only when two independent evidence sources agree that the access link has lost capacity. With weaker evidence, only small, budgeted and reversible steps are allowed.
- **Hold when blind.** If the measurement system fails (all probe targets unreachable, for example), VeriQ holds the current rate and then degrades to static behaviour instead of guessing.
- **SAFE state.** If the daemon dies, a separate watchdog installs static CAKE at 0.85 × min(last-known-good rate, base rate). The kernel configuration persists in any case, so a crash never removes shaping.

Honesty check. The invariant is a design goal, not a proof. First, the SAFE profile assumes capacity does not keep falling after it is installed. Second, a fallback step under uncertain evidence can leave VeriQ temporarily below a static SQM rate, by at most the fallback budget (starting value: 10% per rolling 10 minutes), and the step is undone if its receipt shows no improvement. Scenarios S13 and A1–A32 measure both.

6.3 The nine principles

Every principle in the specification is enforced by a named mechanism. A principle that nothing enforces is a slogan, so the table also says how a reviewer can check each one.

#	Principle	What it means in practice	Enforced by	How to check it
P1	The packet path is the kernel	No user-space process touches packets; veriqd only configures nftables and tc and reads statistics	Datapath design (Chapters 8, 20)	Kill veriqd during a call: traffic and shaping continue
P2	Never worse than static SQM	Hold or revert to static CAKE whenever evidence is missing	Operating states, SAFE profile, evidence permissions	Adverse-condition runs A6, A15, A30 compared with the SQM arm
P3	Find the cause before acting	Cut only where a cut can help; hold for remote, Wi-Fi and CPU causes	Diagnosis with evidence levels L0–L3 (Chapter 11)	Control-futile scenarios S6, S7, S10, A32
P4	Every action is checked, bounded, logged and reversible	Each proposed change passes the Action Checker; budgets limit cumulative change; receipts record the result	Action Checker, budgets, receipts (Chapters 12, 13)	One unit test per refusal code; decision log replay
P5	Preference needs an explicit basis	Classifier abstention alone never grants preference; heuristics may only demote	Kernel precedence with provenance bits; MARK_SET checker rule	An unknown flow without a lease stays in Best Effort
P6	Privacy by construction	Headers and timing only; no payload; no cloud dependency	Observer design, NFLOG snaplen 40 bytes	Inspect the sampling rule and the support bundle contents
P7	Honest semantics	A CAKE tin is priority and weight, not a bandwidth guarantee; a policer is not a shaper	UI wording and documentation	Review LuCI strings and receipts
P8	Product-safe	The daemon crashing never breaks the internet	Kernel config persists; procd respawn; separate watchdog	`kill -9` test with a time-to-safe target of ≤ 12 s
P9	Every published number regenerates from raw data	Results are produced by scripts from the evidence ledger	Evidence ledger and claims check (Chapter 24)	Regenerate any chart from the ledger

P5 is worth stating in the exact words the whole handbook uses, because judges will probe it:

> *"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."*

6.4 Three speeds

Fast decisions are simple and made in the kernel; slower decisions use more evidence; causal claims come only from controlled experiments.

Speed	Period	What happens	Where
Guardrails	Per packet	CAKE AQM, flow isolation, tin scheduling, eligibility-envelope meters, policers	Linux kernel
Control	250 ms tick; probes at 10 packets/s in total	Estimation, diagnosis, rate control, Action Checker, reconciliation	veriqd
Assessment	15–60 s windows	Receipts (IMPROVED / NOT_IMPROVED / INCONCLUSIVE), fallback undo, shadow disagreements	veriqd
Experiment	Runs of 2–5 min, repeated	Causal claims about performance	Lab harness, off the router

The v3.3 design named three speeds (fast guardrails, medium outcome check, slow experiments); v5 splits the fast speed into kernel and daemon rows. A receipt on the router is **operational evidence** (association); only the repeated lab experiment supports a causal claim.

6.5 Three layers

Layer	Contents	Relation to QoSmate
1. Queue management, classification and UI	CAKE (default), HFSC or HTB backends; QoSmate-compatible rules, IP and domain sets, policers; heuristics; app catalogue; ML classifier with abstention; leases; eligibility envelope; LuCI app, household page, CLI	Parity (adopt or improve), plus an importer for QoSmate configurations
2. Diagnosis-informed rate controller	Reflector probing with one-way delays, windowed-minimum baselines, detectors, diagnosis labels with evidence levels, the rate-controller state machine, the Action Checker, receipts	New design; advantages are hypotheses until the head-to-head
3. Product-grade operation	Signed packages, operating states (INIT to DISABLED), reconciler, watchdog, self-tests, explain API, footprint budgets	Improves on QoSmate's distribution and health check

Layer 1 delivers the first benefit, static CAKE, from installation. In static conditions VeriQ and QoSmate's CAKE mode should be at rough parity, because both use CAKE. Layer 2 is where VeriQ expects to differ, in dynamic and adverse conditions. Layer 3 makes the system fit to run unattended.

6.6 The 250 ms control loop

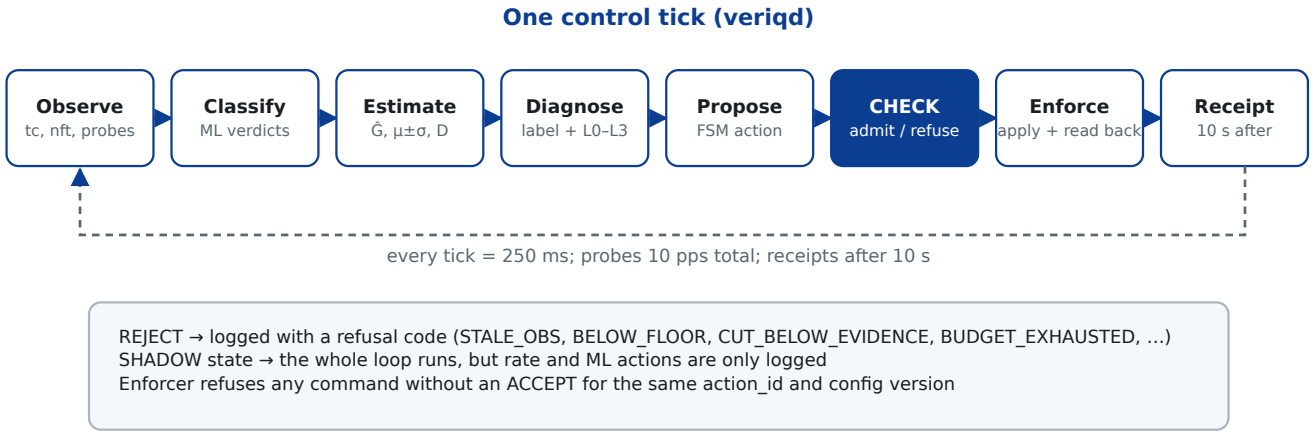


Figure 3. The 250 ms control tick: Observe, Classify, Estimate, Diagnose, Propose, Check, Enforce, Receipt.

Every 250 ms a single control goroutine inside veriqd runs the same sequence for each direction (upload and download). The steps below describe one tick.

1. **Observe.** The daemon takes an immutable snapshot; each signal carries its own monotonic timestamp. It holds shaper statistics (`tc -s -j`: bytes dequeued, per-tin delay and backlog), the latest reflector probe results, near-hop delay, conntrack and sampled-header events, Wi-Fi station statistics (polled every 5 s) and CPU softirq load. A source that fails is marked missing, never zero.
2. **Classify.** Most classification already happened in the kernel, where user rules, leases, app marks and heuristics are evaluated on each packet. In the daemon, flows that reached a checkpoint (8, 16 or 32 packets, or 10 s) are scored by the ML model. Only a singleton conformal prediction set can produce a verdict; anything else is abstention, which writes nothing.
3. **Estimate.** The estimator updates the gateway throughput $\hat{G}$ (median over the last second), the load ratio, the per-reflector baselines, the own-queue delay Q_own, the delay signal D (median of healthy reflectors, quorum ≥ 2) and the tracking estimate μ ± σ.
4. **Diagnose.** Evidence groups (FAR, NEAR, THR, OWN, WIFI, CPU, CAP) are combined into a label (OWN_QUEUE, WAN_LIKELY, REMOTE_LIKELY, WIFI_LIKELY, CPU_LIMITED, PROBE_FAULT, MIXED or UNKNOWN) and an evidence level from L0 (no usable signal) to L3 (two independent sources plus an active capacity test).
5. **Propose.** The rate-controller state machine turns the diagnosis into at most one action per direction: a one-step cut, a bounded fallback step, an upward probe, a slow drift toward the base rate, or nothing.
6. **Check.** The Action Checker, a pure function with no I/O, decides ACCEPT or REJECT with a refusal code. It verifies observation age, version match, operating state, the rate envelope, the step allowed by the evidence level and the budgets.
7. **Enforce.** In LIVE state, an accepted action is applied (`tc qdisc change … bandwidth X` for CAKE) and the result is read back from the kernel. In SHADOW state, the action is only logged.
8. **Receipt.** The decision, verdict and evidence are written to the decision log. An assessor opens a before/after window; about 10 s later it records whether the action IMPROVED, did NOT_IMPROVE or was INCONCLUSIVE.

Alongside the tick, the **reconciler** compares the desired kernel state with what is installed (self-healing or going DEGRADED), and the daemon writes a **heartbeat** every second for the watchdog.

Why it matters. The loop is slow enough to gather real evidence and fast enough to react within a second or two to a genuine capacity drop. The kernel guardrails handle everything faster than that, packet by packet, without waiting for the daemon.

6.7 What VeriQ is not

Stating the boundaries clearly prevents both over-claiming and misplaced criticism.

- **Not deep packet inspection.** VeriQ reads IP and transport headers, packet sizes and timing only. It never reads payload, TLS SNI, HTTP Host headers or DNS answers from forwarded packets. Domain sets come from the router's own DNS resolver configuration, and they stop working when clients use DoH, DoT or ECH directly; the UI says so.
- **Not a bandwidth guarantee.** CAKE tins provide priority and scheduling weights, not reservations. A protected call gets preferential treatment, bounded by its eligibility envelope; it is not promised a fixed rate. Per-device limits are policers (they drop), not shapers.
- **Not a cloud service.** Control, recovery, storage and explanation are all local. No flow data leaves the router unless the operator explicitly opts in.
- **Not a speed booster.** Shaping below capacity costs a few percent of throughput, traded for low delay under load.
- **Not a fix for problems outside the router's reach** (a congested remote server, a failing ISP link, poor Wi-Fi coverage). It can diagnose them, hold its rate and tell the user.
- **Not a self-updating script.** Code arrives only through signed packages and the package manager; there is no runtime download of code.
- **Not an on-device learner, and not an L4S/NQB implementation.** The ML model is trained off the router and shipped as a signed package. DSCP 45 (NQB) is only a hint mapped to Interactive Video; L4S and NQB lanes are post-competition experiments.
- **Not endorsed by or integrated with Tata Elxsi.** The portability path to prplOS and RDK-B is a design direction that would need a partner.

6.8 The v5 decision list (C1–C14)

The v5 specification records fourteen changes from v4.1. Everything else in v4.1, including the kernel datapath, the evidence levels, the precedence, leases, the envelope, the fallback budget and receipts, is preserved.

#	Decision in plain words	Why	How it is verified
C1	On strong evidence (L2) of a capacity drop, cut in **one step** to max(S_min, 0.90 × Ĝ). Fall back to ×0.75 of the current rate only when no fresh throughput measurement exists	The v4.1 limit of ×0.75 per step needed about six steps for a 100 → 20 Mbit/s drop, which would fail the co-primary timing margin	Confirmation time in S2/A1 compared with cake-autorate
C2	Write the on-router daemon in Go as one static binary; keep Python for the lab, evaluation and ML training	One cross-compiled binary for every OpenWrt target, no Python runtime on the router, bounded memory	Footprint measured in Stage 4
C3	Add a full OpenWrt product layer: packages, UCI, procd, LuCI, rpcd ACLs, hotplug, sysupgrade, signed feed, CI	Product readiness that can stand beside QoSmate	Package tests in the OpenWrt VM and on the Pi
C4	Offer three queue backends (CAKE default, HFSC, HTB) mapped onto the same four service classes; defer the hybrid backend	Parity with QoSmate's scheduler choice; weak CPUs cannot always run CAKE at line rate	Bench plus robustness tier H3
C5	Build a full classification engine: QoSmate-compatible rules, sets, policers, importer, app catalogue, alongside leases, envelope and ML	Parity plus a migration path for QoSmate users	Importer tests; classification tier H2
C6	In the product profile, wash DSCP in both directions by default; the lab profile leaves DSCP untouched	Protects Wi-Fi WMM from ISP-set DSCP and avoids leaking marks upstream; classification survives because the class lives in conntrack mark bits	Self-test
C7	Heuristics may only demote or reset a class; preference always needs an explicit basis	Makes "abstention never grants preference" apply to heuristics too	Unit tests
C8	Each reflector's baseline is the minimum over a sliding 10-minute window of 1-s medians taken at low load	A lower-only baseline cannot follow an upward route change	Scenario A7

#	Decision in plain words	Why	How it is verified
C9	Attribute a delay rise to a direction using one-way delays where possible, otherwise by load (cut only directions with load ≥ 60%)	Avoids cutting an idle direction by mistake	Scenario A9
C10	Extend the operating states to INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE, DISABLED; static CAKE from minute one, adaptive control after 24 h of SHADOW	Users get the SQM benefit at once; adaptive features earn trust	Shadow report
C11	Run the watchdog as a separate procd instance written in POSIX shell	OpenWrt uses procd, not systemd	Time-to-safe test
C12	Add a QoSmate head-to-head (whole system and controller only) and 32 adverse conditions A1–A32 to the evaluation	A measurable comparison with the strongest community tool	Evaluation tiers H1–H4
C13	Add the diagnosis label CPU_LIMITED	A CPU-bound router's throughput shortfall must not be read as a capacity drop	Scenario A17
C14	Fix the conntrack mark layout: CLASS in bits 24–26, provenance in 27–29, sanction in 30	Lets the kernel enforce precedence; avoids the bits used by mwan3 and pbr	Self-test and conflict scan

Each change has a "revert if" condition in docs/12; C1, for example, is reverted if false L2 cuts in the control-futile runs exceed the comparator's.

> **Implementation tip.** C1 touches both the controller and the Action Checker. The controller targets 0.90 × $\hat{G}$; the checker accepts targets down to max(S_min, 0.85 × $\hat{G}$) and refuses lower ones with `CUT_BELOW_EVIDENCE`. The gap is a tolerance, not a second cut.

6.9 A day in the life: one household evening

The walkthrough below follows one illustrative evening. **All numbers in this section are illustrative**, chosen to show each module acting; they are not measurements.

The household. A three-generation family in a tier-2 Indian city uses a 5G FWA connection with a plan of about 100/20 Mbit/s. VeriQ was installed on an OpenWrt router four days ago; the wizard set S_base to 85 Mbit/s down and 18 Mbit/s up, giving a download floor S_min = max(1, 0.10 × 85) = 8.5 Mbit/s and ceiling S_max = 1.05 × 85 ≈ 89 Mbit/s. After 24 hours of SHADOW with no anomalies, the controller moved to LIVE.

The people. Anita (a parent working from home) has an evening call with a client. Rohan (a student) wants to update a console game. His grandfather watches a streaming series on a tablet.

Time	Event	What VeriQ sees	Module acting	Action and result
18:55	Anita taps "Protect my call" for 60 min on the household page	Authenticated household session; laptop has no active lease; 0 leases active	`api`, `lease`, `checker`, `enforce`	LEASE_GRANT accepted; laptop's IPv4/IPv6 addresses added to the IV lease sets, timeout 3600 s
19:00	Call starts; series streaming on the tablet	Call flows hit the lease rule on the first packet; the stream is unmarked	Kernel precedence	Call → Video tin (provenance: lease); stream → Best Effort
19:10	Console game update and a phone OS update start	Sustained bulk TCP; heuristic H1 or an ML singleton BULK verdict	Kernel heuristics, `classify`	Update flows → Bulk tin; CAKE keeps the call's queue short; label OWN_QUEUE, no rate change
19:25	Anita's laptop starts a cloud sync upload during the call	The sync flow inherits the lease but exceeds the IV envelope (8 Mbit/s per flow)	Kernel envelope meter, `sanction`	Sync flow sanctioned to Best Effort for 60 s, refreshed while it keeps violating; the call keeps its Video tin
19:40	Cell load drops download capacity to about 36 Mbit/s	FAR delay +45 ms; own queue < 5 ms; $\hat{G}$ ≈ 36 Mbit/s with demand; upload one-way delay flat	`estimate`, `diagnose`	WAN_LIKELY at L2 (FAR + THR), download only
19:40	(same tick group)	Proposed target max(8.5, 0.90 × 36) = 32.4 Mbit/s	`control`, `checker`, `enforce`	RATE_CUT accepted (checker minimum 30.6); `tc qdisc change` on `ifb-veriq`; read back OK

Time	Event	What VeriQ sees	Module acting	Action and result
19:41	Receipt window closes	Median delay 10 s after is about 6 ms, against 34 ms before	`receipt`	IMPROVED; the explain sentence appears on the Overview page
19:45	Rohan tries to lease his console	Collapse rule: WAN_LIKELY at L2 with a drop > 50%	`lease`, `checker`	Refused (COLLAPSE); Anita's lease continues, with a warning
19:50	Anita renews her lease once for 30 more minutes	One renewal is allowed without re-approval; the total stays under the 4 h cap	`lease`, `enforce`	Set timeouts updated; the lease expires in the kernel at 20:25; its receipt compares call delay and loss before and during the lease
20:05	Cell capacity recovers	Load ≥ 75% for 2 s, delay delta < 5 ms, no diagnosis ≥ L1 in the last 10 s	`control`	PROBE_UP: ×1.04 per second (each raise checked); back near 85 Mbit/s in roughly half a minute
21:10	A congested path beyond the ISP's first hop slows Rohan's game server	FAR +60 ms; NEAR present and flat; no throughput censoring; load 0.4	`diagnose`, `control`	REMOTE_LIKELY at L2 → **hold**; household page: "The slowdown is beyond your provider's first hop; reducing your speed would not help"
22:30	House goes quiet	Load < 10% both ways for 30 s	`probe`, `control`, `estimate`	Idle probing at 2 packets/s; no cuts; DRIFT toward S_base after 120 s; evening capacity sample stored

Four moments in this evening deserve a closer look.

19:25, the lease is not a blank cheque. The eligibility envelope applies to every flow, leased or not. Her cloud sync exceeded the Video-class allowance, so the kernel demoted only that flow, in both directions, without waking the daemon, which learned of it from an NFLOG event. Her call was untouched.

19:40, one step instead of six. A delay-only controller would cut in fixed steps, for example ×0.85 per refractory period (QoSmate's defaults, read). VeriQ first asks where the delay comes from. Its own queue is nearly empty, so the queue has moved upstream; throughput fell while the downloads still wanted more (censoring). Two independent sources agree, so the evidence reaches L2 and C1 permits one cut to just below the measured throughput. One-way delays showed the upload unaffected, so only the download was cut (C9). The Overview page then shows a sentence such as:

> "19:40:12: download rate 85 → 32.4 Mbit/s. The far-reflector delay rose 45 ms, and throughput fell to 36 Mbit/s while downloads still wanted more (evidence L2: FAR + THR). Checker: ACCEPT. After 10 s: delay −28 ms → IMPROVED." *(illustrative)*

21:10, knowing when not to cut. The delay rise looks the same from the far reflectors, but the evidence pattern differs. The near hop is flat, throughput is not censored and the link is only 40% loaded. A cut would waste bandwidth and cannot fix a queue beyond the ISP's first hop, so VeriQ holds and explains. The primary outcome counts exactly these inappropriate interventions in control-futile conditions. What a delay-only controller does here is a **hypothesis** to be tested in scenario S6.

22:30, quiet hours teach little. At idle nothing is learned about capacity, so VeriQ neither cuts nor raises on delay alone. With three or more weeks of data in an hour-of-week bin, a reboot at the evening peak can start from a conservative historical rate instead of the full base rate.

Had veriqd crashed at 19:41, the kernel would have kept the 32.4 Mbit/s CAKE configuration, and within the 12 s target the watchdog would have installed the SAFE profile.

> **For judges.** "What does VeriQ do differently on a normal evening?" Static CAKE does most of the work. VeriQ adds a single evidence-based cut when capacity really drops, a hold with an explanation when the problem lies elsewhere, and per-flow limits that stop a protected device abusing its protection. Each action leaves a receipt.

CHAPTER 7

Architecture

This chapter is the map of VeriQ for the people who will build it. It describes the layers and planes, every internal package of the daemon, the kernel objects VeriQ owns, the deployment and process views, the interfaces and data stores, and the key runtime flows as numbered steps. It closes with the quality attributes the architecture is designed for and ten architecture decision records (ADRs). Detailed templates and the method-level API are in Chapters 20 and 21; package-level implementation guidance is in Chapter 19. All numbers are **starting parameters** or **targets** from the v5 specification unless labelled otherwise.

7.1 Layers and planes

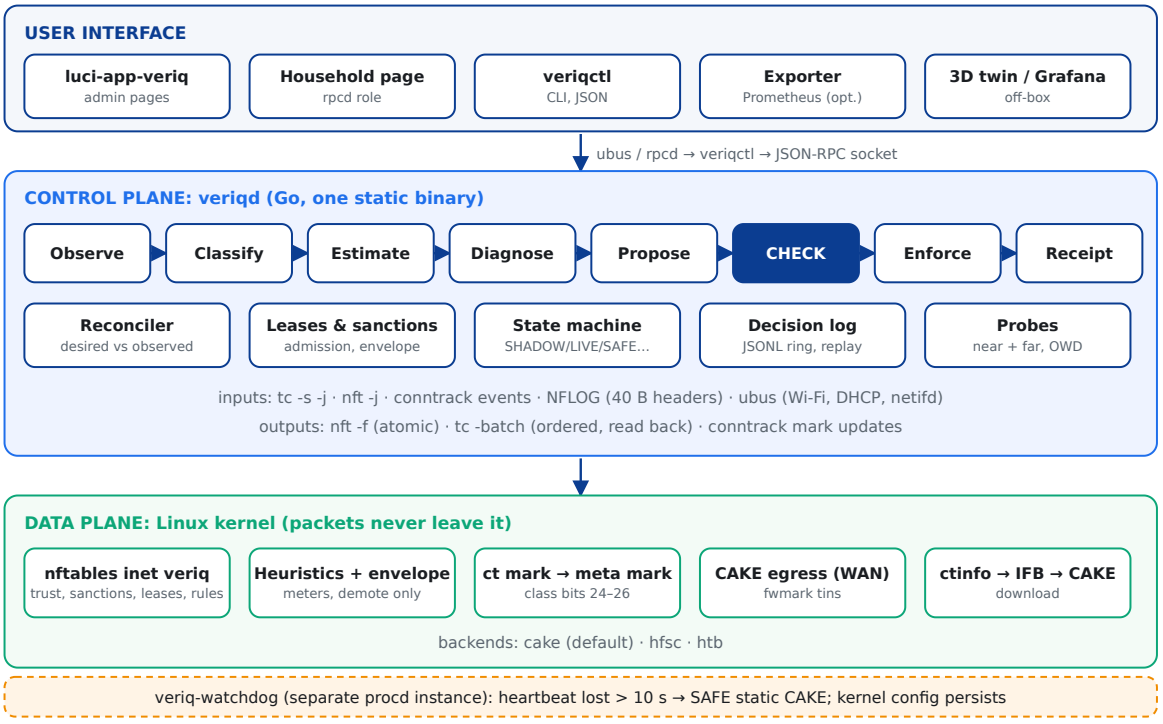


Figure 4. Layered architecture: user interface, control plane (veriqd), data plane (kernel) and the separate watchdog.

VeriQ is split into three planes plus an independent safety process. The split follows principle P1 (the packet path is the kernel) and principle P8 (the daemon crashing never breaks the internet).

Data plane (Linux kernel). All packets are handled here, at line rate, without user-space involvement. The data plane consists of: - the nftables table `inet veriq`, which classifies flows, enforces precedence, applies eligibility-envelope meters and sanctions, and copies the class into the packet mark; - the egress qdisc on the WAN device (CAKE by default, or an HFSC or HTB tree), which shapes upload; - an ingress hook on the WAN device that restores the class from conntrack with `act_ctinfo` and redirects packets to the Intermediate Functional Block device `ifb-veriq`, where a second qdisc shapes download.

Control plane (`veriqd`, Go). One daemon observes, estimates, diagnoses, proposes, checks and enforces. It never forwards or inspects payload. It writes to the data plane only through two channels: `nft -f` (an atomic transaction) and `tc -batch` (ordered steps, each read back). It reads from the kernel with `tc -s -j`, `nft -j`, conntrack netlink events, NFLOG (40-byte header samples) and raw ICMP sockets. It reads system state from ubus (Wi-Fi station statistics via iwinfo/hostapd, DHCP leases, netifd interface events).

User-interface plane. The LuCI app `luci-app-veriq` (administrator), the household page (a restricted rpcd role), the `veriqctl` command-line tool and the optional Prometheus exporter. The 3D twin and Grafana dashboards run off the router and only for demos and engineering. Every UI path reaches veriqd through one local socket.

Watchdog. `veriq-watchdog` is a POSIX shell loop under procd, sharing no code or memory with veriqd. Its only job is to notice a stale heartbeat and install the SAFE shaper.

The text diagram below is the same picture in compact form.

```
UI PLANE     luci-app-veriq | household page | veriqctl | exporter (opt.)
                 |  ubus / rpcd (luci.veriq -> veriqctl -j)
                 v
CONTROL      veriqd: observe -> classify -> estimate -> diagnose -> propose
PLANE                -> CHECK -> enforce -> receipt   (+ reconciler, leases,
                     state machine, decision log, metrics rings)
```

```
                |  nft -f (atomic) | tc -batch (ordered, read back)
                v
DATA PLANE    nft inet veriq: wash/trust -> rules/sets/leases -> heuristics
(kernel)        -> envelope meters -> ct mark -> meta mark
              WAN egress: CAKE diffserv4 fwmark (or HFSC/HTB)
              WAN ingress: act_ctinfo cpmark -> mirred -> ifb-veriq -> CAKE

WATCHDOG      veriq-watchdog (procd, sh): heartbeat stale > 10 s -> SAFE
```

Why it matters. Because the kernel holds the complete working configuration, the daemon is an *optimiser*, not a *dependency*. If it stops, the household keeps a correctly classified, shaped connection at the last applied rate, and the watchdog then moves to a conservative static rate.

7.1.1 Trust boundaries

Three boundaries matter for security and for failure analysis (Chapter 17 expands them): 1. **LAN devices → router.** Packets from LAN devices are untrusted input: their DSCP marks may be wrong or abusive. The eligibility envelope and sanctions in the kernel contain them. 2. **UI → veriqd.** All requests pass through rpcd ACLs (roles `admin` and `household`) and then a root-only Unix socket. veriqd has no network listener of its own. 3. **veriqd → kernel.** Every change crosses the Action Checker first, and the Enforcer refuses any command without an ACCEPT for the same action ID and version.

7.2 Component catalogue

The daemon is a single Go module (`cmd/veriqd`, `cmd/veriqctl` and the packages under `internal/`). The tables below list every internal package from the repository layout with its responsibility, inputs, outputs, period and failure behaviour. "Tick" means the 250 ms control tick.

7.2.1 Foundation and sensing packages

Package	Responsibility	Inputs → outputs	Period	Failure behaviour
`clock`	Monotonic time source behind an interface; a fake clock for tests and replay	OS monotonic clock → `time.Duration` timestamps	On demand	No wall-clock dependence in control timing, so an NTP step (A18) has no effect
`config`	Parse `/etc/config/veriq` and `/etc/veriq.d/`, apply the schema (Appendix B of docs/11), validate, migrate `config_version`	UCI files → typed `Config`, errors reported as `section.option`	At start and on reload	An invalid reload is rejected and the running configuration stays active
`model`	Shared types: `Dir`, `Class`, `Prov`, `Level`, `Label`, `Snapshot`, `Diagnosis`, `Action`, `Verdict`	— → types used by every package	—	Pure types, no behaviour
`observe`	Collect kernel and system observations: `tc -s -j` shaper stats, `nft -j` counters, conntrack events (netlink), NFLOG samples, Wi-Fi station stats (ubus), DHCP and neighbour tables	Kernel and ubus → timestamped samples on channels	tc/nft each tick; Wi-Fi every 5 s; conntrack, NFLOG and DHCP on events	A failed source yields samples with `OK=false`; downstream treats them as MISSING, never as zero
`probe`	ICMP echo and timestamp probing over raw sockets; reflector pool, qualification, health, outlier rejection, near-target resolution	Configured pool, WAN gateway → per-reflector deltas, health flags	10 probes/s in total (2/s idle); qualification at start and every 6 h	Unhealthy reflectors are excluded and replaced; with fewer than 2 healthy, the delay signal is MISSING (L0)
`estimate`	Windowed-minimum baselines, the delay signal D (median of healthy reflectors), Ĝ, load, Q_own, detectors, μ/σ, capacity memory	Samples → derived quantities and detector flags	Each tick; μ/σ every 1 s; capacity memory written at most hourly	Stale baselines are flagged; a baseline stale for over 30 min caps that reflector's evidence at L1

7.2.2 Decision packages

Package	Responsibility	Inputs → outputs	Period	Failure behaviour
`diagnose`	Combine evidence groups (FAR, NEAR, THR, OWN, WIFI, CPU, CAP) into a label and an evidence level L0–L3; apply the missing-near rules	Estimates and detectors → `Diagnosis{Label, Level, Sources, Dir}`	Each tick, per direction	Insufficient evidence gives UNKNOWN at L0–L1, which permits bounded fallback at most

Package	Responsibility	Inputs → outputs	Period	Failure behaviour
`control`	The rate-controller state machine (HOLD, PROBE_UP, BACKOFF, DRIFT; CUT and FALLBACK_STEP), refractory periods, fallback episodes and the rolling budget	Snapshot, diagnosis → at most one proposed `Action` per direction	Each tick, per direction	Proposes only; nothing reaches the kernel without a checker ACCEPT
`classify`	Compile user rules, sets and heuristics into nft text; load the app catalogue; run the ML tree ensemble and the conformal set builder	Config, NFLOG samples, conntrack counters → nft ruleset text; MARK_SET proposals for singleton verdicts	Compile on config change; ML at 8, 16, 32 packets and at 10 s	Missing model or insufficient calibration disables that checkpoint (abstain); the Lite profile runs without ML
`lease`	Lease table, admission, device address tracking, persistence and restore	API requests, DHCP/neighbour changes → LEASE_GRANT/REVOKE proposals, set-element updates	On request; checkpoint every 60 s	After a reboot, leases resume only if NTP is synced; remaining time is never extended
`sanction`	Record envelope violations (NFLOG group 8), detect standing queues in preferential tins, escalate penalties	NFLOG events, per-tin stats → SANCTION_ESCALATE proposals, sanction records	On events; standing-queue check each tick	The kernel enforces the first 60 s penalty on its own, so sanctions work while the daemon is down
`checker`	The Action Checker: a pure function returning ACCEPT or REJECT with a refusal code	Action, snapshot, budgets, lease table, state, limits → `Verdict`	Per proposed action	Shares the process with the controller, so they fail together; the watchdog covers that failure domain

7.2.3 Actuation, operation and interface packages

Package	Responsibility	Inputs → outputs	Period	Failure behaviour
`enforce`	Backends (`cake` , `hfsc` , `htb`) that render tc batches and rate changes; the nft renderer; execution and read-back	Accepted action + verdict → `tc -batch -` , `nft -f -` , observed state	On accepted actions	Refuses without a matching ACCEPT; on partial failure runs a compensating step or goes SAFE; at most 3 retries with capped backoff
`reconcile`	Compare desired (versioned) and observed kernel state; ownership checks; self-heal; restart adoption	Desired document, `tc -j` , `nft -j` → repair actions or a DEGRADED event	Each tick	Re-applies a vanished `inet veriq` table within one tick; never fights a foreign qdisc (goes DEGRADED)
`state`	Operating-state machine (INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE, DISABLED); atomic persistence; heartbeat and `safe.env`	Events from all packages → current state, state files	On events; heartbeat every 1 s	State files written by temp file + rename, so a power cut leaves the old or new file, never a torn one
`receipt`	Before/after assessment, outcome rules, lease receipts, shadow disagreements, explain templates	Decision log, metric rings → receipts, explain sentences	10 s windows for rate actions; 15–60 s for others	Too few samples or a concurrent change gives INCONCLUSIVE, never a guessed outcome
`api`	JSON-RPC 2.0 server on `/var/run/veriq.sock` ; method table; role hints for rpcd	Requests → typed messages to the control goroutine; responses	On request	Runs in its own goroutine; a slow client cannot stall the tick
`importer`	Convert QoSmate and SQM configurations into VeriQ UCI	`/etc/config/qosmate` , `/etc/config/sqm` → candidate config, mapping report	On demand (`--dry-run` supported)	Unmappable options are reported, and nothing is applied without validation
`telemetry`	Metric ring buffers, the decision log (JSONL, RAM ring ≤ 2 MB), hourly summaries, exporter text	All decisions and samples → log lines, metrics	Continuous; summaries hourly	Rings overwrite their oldest entries; the exporter is optional and LAN-only

Two binaries and one script complete the picture. `cmd/veriqd` wires the packages together, handles signals and owns the main loop. `cmd/veriqctl` is a thin JSON-RPC client used by people, by rpcd and by the hotplug script. The watchdog is a shell script installed by the `veriq` package as a second procd instance.

7.2.4 Concurrency model

- One **control goroutine** owns all decision state and runs the tick. Decision logic contains no shared mutable state and no mutex.

- **I/O goroutines** (prober, conntrack event reader, NFLOG reader, tc/nft sampler, Wi-Fi poller, API server) send typed messages over channels to the control goroutine.
- The decision log records every input message with its monotonic timestamp. `veriqctl replay log.jsonl` must reproduce the decisions bit for bit; this is a test requirement, not an aspiration.

This design makes the controller deterministic for a given input stream, which is what allows golden tests, replay of field incidents and the 3D twin's faithful replay.

> **Implementation tip.** Keep every call to `time.Now()` out of `checker`, `control` and `diagnose`. They receive time only through the snapshot. A single stray wall-clock read breaks replay determinism, and the replay test will catch it.

7.3 Data plane objects

VeriQ owns a fixed set of kernel objects and never modifies objects it does not own. Owned objects carry the comment `veriq:vN`, where N is the configuration version.

Object	Kind	Purpose	Notes
`table inet veriq`	nftables table	Holds all VeriQ chains, sets and meters	Removed in DISABLED; re-applied by the reconciler if a firewall restart flushes it
`chain cls_up`	Base chain, hook forward, priority mangle + 5	Classification pipeline on LAN → WAN packets: trust/wash, state bypass, sanctions, leases, user rules, app marks, heuristics, default	Precedence enforced with provenance (PROV) bits; also hosts the NFLOG sampling rule (exact position in Chapter 20)
`chain envelope`	Regular chain, jumped to for CLASS ∈ {RT, IV}	Per-flow eligibility meters; violation sets CLASS = BE, SANCT = 1, adds the flow to `@sanctioned`, logs to NFLOG group 8	Applies to every source of preference, leases included
`chain mark_out`	Base chain, hook postrouting, priority mangle + 10	Copies the CLASS bits from the ct mark to the packet mark with four constant-OR rules	Upload direction; CAKE reads `fwmark 0x07000000`
Lease sets `lease_rt4/6`, `lease_iv4/6`	nft sets with timeout	Device addresses with an active lease; element timeout = remaining lease time	Expiry happens in the kernel
`sanctioned`	nft set of `ct id` with timeout	Flows currently sanctioned; 60 s first penalty	Escalated by veriqd (×2 per offence, cap 15 min)
`state_bypass`, `trusted_dscp`	nft sets	DISABLED bypass flag; optional list of devices whose DSCP is trusted	`trust = all` (with envelope) is the default
Device and app sets	nft sets (`app_<id>_4/6`, device sets, user IP sets)	Targets for rules; domain sets filled by dnsmasq `nftset`	Domain sets need `dnsmasq-full`
Meters	nft meters keyed `ct id . ct direction`; per-device policer meters	Envelope rate/pps limits; user rate limits (labelled policers)	Envelope thresholds use S_base, so they do not churn
NFLOG groups 7 and 8	nft `log group`	7: header samples (`snaplen 40`, first 32 packets, ≤ 4000 packets/s in total); 8: sanction events	Read by veriqd over netlink
WAN egress root qdisc	CAKE `diffserv4 dual-srchost nat fwmark 0x07000000` (or HFSC/HTB tree + `fw` filters)	Upload shaping and tin selection	Overhead and MPU from the link preset
WAN ingress qdisc + filter	`ingress` qdisc; filter with `action ctinfo cpmark 0x7F000000 then action mirred egress redirect dev ifb-veriq`	Restores the class from conntrack for download packets	`cpmark` syntax verified by the Stage-1 self-test on each target
`ifb-veriq`	IFB device + root qdisc (CAKE `dual-dsthost nat ingress fwmark 0x07000000`, or HFSC/HTB)	Download shaping	Absent when download shaping is disabled (rate 0)

The conntrack mark layout is CLASS in bits 24–26 (`0x07000000`), PROV in bits 27–29 (`0x38000000`) and SANCT in bit 30 (`0x40000000`). Bits 0–23 and 31 are never touched, so VeriQ coexists with mwan3 and pbr; the health check scans the ruleset to confirm no other package uses bits 24–30. Chapter 20 gives the full templates and bit arithmetic.

> **Note.** The fwmark values 1–4 select CAKE tins Bulk, Best Effort, Video and Voice in tc's display order for diffserv4. The Stage-1 tin self-test is the source of truth for this mapping on each kernel, not the table in the specification (open decision O2).

7.4 Deployment views

View	Hardware	Platform	Purpose
D1 Lab	x86-64 laptop or desktop	Ubuntu 24.04, network namespaces, systemd unit, plain-file config backend	Development, CI integration tests, Stage 2–3 evaluation
D2 OpenWrt VM	x86-64 host with QEMU/KVM	OpenWrt 24.10 ×86-64 (netns or bridged)	Package and LuCI tests in CI
D3 Reference router	Raspberry Pi 4/5 + USB 3 gigabit Ethernet adapter	OpenWrt 24.10 (bcm27xx)	Demo kit, Stage 4 resource measurements
D4 Retail router	One ARM64 OpenWrt router with ≥ 512 MB RAM (for example, the Filogic 830 class)	OpenWrt 24.10	Product credibility, Stage 4
D5 Low-end check	One MT7621-class device (MIPS, 128–256 MB RAM)	OpenWrt, Lite profile	Proves the footprint floor

The same daemon binary logic runs in every view. Only the platform glue differs: procd, UCI and rpcd on OpenWrt; systemd and a plain-file config on Ubuntu. This is why the lab results in D1 are representative of the controller's decisions, while resource numbers must come from D3–D5.

7.5 Process model and privileges

Process	Started by	Privilege	Purpose
`veriqd`	procd via `/etc/init.d/veriq` (`START=95`, respawn `3600 5 5`, `nofile=4096`)	root; needs CAP_NET_ADMIN and CAP_NET_RAW; drops other capabilities via procd jail where available	Control plane
`veriq-watchdog`	procd, second instance of the same init script	root	Heartbeat check → SAFE
rpcd plugin `luci.veriq`	rpcd	Per ACL role (`admin`, `household`)	UI bridge; calls `veriqctl -j`
`veriqctl`	User shell, rpcd plugin, hotplug script	root (via rpcd for UI calls)	Commands and JSON output

procd's respawn parameters `3600 5 5` mean: a run shorter than 3600 s counts as a failure, procd waits 5 s before each restart, and it gives up after 5 consecutive failures. By then the watchdog has already installed SAFE, so the household keeps static CAKE.

7.6 Interfaces

Interface	Between	Transport and format	Access control
Local API	`veriqctl` ↔ `veriqd`	Unix socket `/var/run/veriq.sock`, line-delimited JSON-RPC 2.0	File mode 0600, root only
UI bridge	LuCI and household page ↔ rpcd ↔ `veriqctl -j`	ubus over HTTP (uhttpd)	rpcd ACLs; the household role may call only `veriq.status`, `veriq.lease_request`, `veriq.lease_list`, `veriq.lease_cancel_own`, `veriq.explain_simple`
System inputs	`veriqd` ← ubus	iwinfo/hostapd station stats, DHCP leases, netifd interface state	Local ubus
Interface events	hotplug `30-veriq` → `veriqd`	Event over the local socket on WAN ifup/ifdown	If veriqd is down, the script does nothing; the kernel config persists
Kernel write	`veriqd` → kernel	`nft -f -`, `tc -batch -` as processes	root, owned objects only
Kernel read	kernel → `veriqd`	`tc -s -j`, `nft -j`; conntrack and NFLOG via netlink; raw ICMP sockets	root
Watchdog	`veriqd` → `veriq-watchdog`	Files: `/var/run/veriq/heartbeat` (mtime), `/var/run/veriq/safe.env` (SAFE rates)	RAM filesystem, root
Metrics	exporter → LAN scraper	Prometheus text, default port 9567	Opt-in package; LAN zone only
Alerts	`veriqd` → user endpoint	JSON POST webhook	Opt-in; at most 1 per 5 min per alert type

The JSON-RPC method table (about 25 methods) and the rpcd ACL files are in Chapter 21.

7.7 Data stores and persistence

Data	Location	Write policy	Survives
Runtime state, heartbeat, `safe.env`, decision log ring (≤ 2 MB), metric rings	`/var/run/veriq`, `/tmp/veriq` (RAM)	Continuous	Daemon restart only (lost on reboot)
Budget state, sanction escalation, leases	`/etc/veriq/state/budget.json`, `sanctions.json`, `leases.json`	On change, ≤ 1 write per minute	Reboot and sysupgrade
Capacity memory and hourly summaries	`/etc/veriq/state/capacity.json` (≤ 10 KB)	≤ 1 write per hour	Reboot and sysupgrade
Desired-state version	`/etc/veriq/state/version.json`	On change, ≤ 1 write per minute	Reboot and sysupgrade
Configuration	`/etc/config/veriq`, `/etc/veriq.d/` (inline/custom nft, app overrides)	On user change	Reboot and sysupgrade (`/lib/upgrade/keep.d/veriq`)
Model and catalogue	Installed by the signed `veriq-model` and `veriq-catalog` packages	Package manager only	Package upgrades

Every state file has `schema` and `written_at` fields and is written atomically (temporary file, then rename). The write limits protect router flash from wear. Privacy limits also apply: flow records stay in RAM for at most 24 hours, and aggregates for 7 days.

> **Note.** Because `version.json` follows the flash-wear limit, the stored version can lag the kernel by up to a minute. A restart soon after a rate change may then fail the exact-match adoption test. That is safe by design: the reconciler takes the SAFE-then-INIT path instead of adopting. Measure how often this happens in the Stage 4 soak before deciding whether the version needs a separate, more frequent checkpoint.

7.8 Key flows

Each flow is a numbered list of steps. Chapter 21 gives the full sequence diagrams.

7.8.1 Startup and adoption

1. procd starts `veriqd` and `veriq-watchdog` (START=95, after the network and firewall).
2. `config` loads and validates `/etc/config/veriq`. If it is invalid, veriqd reports errors by `section.option` and leaves the kernel unchanged (handbook default; any existing VeriQ objects keep working).
3. `state` reads `version.json` and the other state files; budgets are restored, never refunded.
4. `reconcile` reads the kernel (`tc -j`, `nft -j`) and compares the owned objects with the persisted version.
5. **If they match**, VeriQ adopts the kernel state and resumes in its previous operating state.
6. **If they do not match** (or nothing is installed), it goes SAFE, then INIT. In INIT it installs static CAKE at the start rate, min(S_lkg, S_base) or the capacity-memory rate, and the `inet veriq` table, with each tc step read back.
7. The state moves to SHADOW for the first 24 hours after install or after an algorithm/model upgrade, otherwise to LIVE, subject to the conflict checks (no other shaper, no flow offloading).
8. `probe` qualifies reflectors in the background; the heartbeat starts; the first tick runs.

7.8.2 The control tick

1. The control goroutine drains its input channels and builds an immutable `Snapshot`.
2. `estimate` updates baselines, D, $\hat{G}$, load, Q_own and the detectors.
3. `diagnose` produces a `Diagnosis` per direction.
4. `control` proposes at most one action per direction.
5. `checker.Check` returns a `Verdict`.
6. In LIVE, an accepted action goes to `enforce`, which applies it and reads back; the desired version increments.
7. `telemetry` writes one decision-log line per direction, including "no action" and refused actions, in every state.
8. `reconcile` compares desired and observed state; `state` touches the heartbeat once per second and rewrites `safe.env` when rates change.

7.8.3 Lease request from the household page

1. A household member opens `/veriq/` on a phone and logs in as the rpcd user `household`.
2. The page calls `veriq.lease_request` with the device, class (IV by default, RT for "game console" mode), duration and reason.
3. rpcd checks the session and the ACL, then runs `veriqctl -j lease add …`, which sends a JSON-RPC request over the socket.

4. `lease` evaluates admission: at most 2 active leases, at most 1 per device, and no capacity collapse (S < 2 Mbit/s, or WAN_LIKELY at L2 with a drop of more than 50%).
5. `checker` evaluates LEASE_GRANT (`AUTH`, `ADMISSION`, `COLLAPSE`, plus the common checks).
6. On ACCEPT, `enforce` adds the device's IPv4 and all current IPv6 addresses to `@lease_iv4/6` (or `@lease_rt4/6`) with a timeout equal to the lease duration, and reads the sets back.
7. `lease` persists the record to `leases.json`; `receipt` starts collecting the device's before-lease metrics.
8. The page shows "Your call is protected until 21:30" (example), or the refusal reason.
9. At expiry, the kernel removes the set elements by itself. The lease receipt compares delay and loss before and during the lease.

7.8.4 Envelope violation

1. A flow in RT or IV exceeds its per-flow meter (starting values for RT: more than max(1 Mbit/s, 2% of S_base) or 450 packets/s).
2. In the kernel, the `envelope` chain sets CLASS = BE and SANCT = 1, adds the flow's `ct id` to `@sanctioned` with a 60 s timeout and logs an NFLOG group-8 event. Both directions of the flow are affected, because the sanction is keyed by flow.
3. `sanction` records the offence. On a repeat within 1 hour, it proposes SANCTION_ESCALATE (×2, cap 15 min); the checker requires a recorded violation (`NO_VIOLATION`, `CAP_EXCEEDED`).
4. The Connections page shows the flow as sanctioned with the reason; the device's other flows keep their class.
5. When the timeout passes without a further violation, the next packet finds SANCT set but the flow absent from `@sanctioned`; the kernel clears the sanction and the flow is classified again from the top. Silence counts as good behaviour.

7.8.5 Watchdog to SAFE

1. veriqd stops touching `/var/run/veriq/heartbeat` (crash, hang or `kill -9`).
2. The watchdog checks the heartbeat mtime every 2 s.
3. When the heartbeat is more than 10 s old and the state is not DISABLED, it runs `tc qdisc change` with the SAFE rates from `safe.env`: static CAKE at 0.85 × min(S_lkg, S_base). Rules and leases stay in place.
4. It logs to syslog and writes `state=SAFE`. The target time-to-safe is ≤ 12 s.
5. procd respawns veriqd. On start, the reconciler finds the SAFE configuration, does not adopt the old LIVE state, and moves SAFE → INIT.

7.8.6 Link down and up

1. netifd reports WAN down, or the WAN device loses carrier; hotplug sends the event to veriqd.
2. veriqd enters HOLD_LINKDOWN: rates frozen, probes paused, losses not counted as evidence.
3. On WAN up (or a PPPoE reconnect or IP change), veriqd re-resolves the near target (the WAN gateway, or the PPPoE peer).
4. Reflectors are re-qualified in the background; the rate resumes at the capacity-memory start rate.
5. After 10 s of healthy probes, the state returns to LIVE (or the prior state).
6. If the WAN interface is renamed or missing, the state goes DEGRADED with an alert, keeping the last kernel configuration.

7.8.7 Configuration change and reload

1. The user saves settings in LuCI (or edits UCI and commits).
2. procd's reload trigger for `veriq` calls the init script's `reload`.
3. veriqd re-parses and validates the configuration; errors are reported against `section.option`.
4. `classify` and `enforce` render a candidate nft ruleset and tc batch.
5. The candidate is checked with `nft -c -f`. A failing rule is reported by name with the nft error, and the previous ruleset stays active.
6. `checker` evaluates RULESET_APPLY: the candidate passed `nft -c`, version = desired + 1, and the state is not SAFE (unless this is the SAFE ruleset).
7. `enforce` applies the ruleset with `nft -f` (one atomic transaction) and any tc changes in order, reading each back.
8. If S_base changed, envelope meters are re-rendered. A backend change happens only on an explicit user request (BACKEND_SWITCH; otherwise `NOT_USER_INITIATED`).

7.9 Quality attributes

Attribute	Target (to be measured)	How the architecture achieves it	Verified by
Latency under load	Added delay p95 at parity with static SQM in static conditions; confirmation time after a genuine drop ≤ comparator + 1 s	Kernel CAKE handles every packet; the 250 ms tick; the C1 one-step cut; direction attribution	H1 head-to-head, S2/A1
Safety	Time-to-safe ≤ 12 s; no internet outage on a daemon crash	Kernel config persists; separate watchdog; Action Checker; budgets; reconciler; SHADOW start	S8/A15, fault-injection tests, checker unit tests

Attribute	Target (to be measured)	How the architecture achieves it	Verified by
Privacy	No payload read; no off-router flow export by default	NFLOG snaplen 40; local-only processing; RAM rings with retention limits; hashed support bundles	Code review, support-bundle inspection
Footprint	Standard profile ≤ 40 MB RSS, ≤ 10 MB flash, ≤ 5% of one A53 core; Lite ≤ 20 MB RSS	Static Go binary; no Python on the router; kernel writes only on change; bounded rings and sampling limits	Stage 4 bench and soak (D3–D5)
Portability	Builds for x86_64, aarch64 (A53, A76), arm A7, mipsel 24kc	`CGO_ENABLED=0` cross-compile; platform glue isolated in `openwrt/`	CI build matrix, VM and Pi package tests
Explainability	Every action has a decision-log line and a receipt	Single decision log; explain templates; precedence trace per flow	Replay tests, demo
Reproducibility	Every decision replays bit for bit	Monotonic timestamps on all inputs; single control goroutine	`veriqctl replay` test in CI

For judges. If asked "what happens when VeriQ itself fails?", walk through three layers of defence. First, the kernel keeps shaping at the last rate. Second, the watchdog installs SAFE static CAKE within the 12 s target. Third, procd restarts the daemon, and the reconciler only adopts a kernel state it can prove matches its own record.

7.10 Architecture decision records

Each ADR records the context, the decision, its consequences and the main alternative rejected.

7.10.1 ADR-01: Go for the on-router daemon

- **Context.** The daemon must run on five CPU architectures with bounded memory and without a large runtime. The team also needs readable code and good testing tools.
- **Decision.** Write `veriqd` and `veriqctl` in Go as static binaries (`CGO_ENABLED=0`, `-trimpath -ldflags "-s -w"`). Keep Python for the lab, evaluation and ML training (C2).
- **Consequences.** One build per architecture, no interpreter on the router (Python would cost roughly 10–15 MB of flash), fuzzing and race detection built in. The binary is larger than a C program, so the flash budget must be checked.
- **Rejected.** Python on the router (runtime size); C (slower development, memory-safety risk); shell, as used by QoSmate (read: `qosmate.sh` and `qosmate-autorate.sh` are POSIX sh scripts), which suits a configuration generator but not a stateful, unit-tested controller. Fallback if the team cannot ramp up in Sprint 0: the controller in Python in the lab for P0 (O7).

7.10.2 ADR-02: Run tc and nft as processes; use netlink only for conntrack, NFLOG and ICMP

- **Context.** Kernel configuration can be changed through netlink libraries or by running the iproute2 and nftables tools.
- **Decision.** Use `tc -batch -` and `nft -f -` for writes and their `-j` JSON output for read-back. Use netlink libraries only where event streams are needed: conntrack (`github.com/ti-mo/conntrack`, MIT), NFLOG (`github.com/florianl/go-nflog`, MIT), plus `golang.org/x/net/icmp` (BSD).
- **Consequences.** Each call costs a few milliseconds, which fits a budget of at most about two changes per second. The rendered text is exactly what an engineer would type, which makes debugging, support bundles and templates simple. The tools must be present (`tc-full`, `nftables`).
- **Rejected.** Full netlink for tc and nft: faster, but a large surface of hand-encoded messages that would be hard to verify across kernels.

7.10.3 ADR-03: Precedence enforced in the kernel

- **Context.** The seven-level precedence must hold even if the daemon is slow, restarting or down.
- **Decision.** Encode class, provenance and sanction in conntrack mark bits, and let nft rules overwrite a class only when their own provenance is higher than the stored one. Leases and sanctions are set elements with kernel timeouts.
- **Consequences.** Precedence, lease expiry and sanction expiry work without a daemon round trip. The ML classifier writes verdicts by conntrack update only when the stored provenance is lower. The nft ruleset becomes more involved and needs careful template testing.
- **Rejected.** Daemon-side precedence with per-flow mark writes: slower, and wrong whenever the daemon lags.

7.10.4 ADR-04: Class carried in conntrack mark bits and CAKE fwmark, not in DSCP

- **Context.** Download packets arrive from the WAN before any LAN-side classification, and DSCP may be washed or rewritten by the ISP.
- **Decision.** Copy CLASS bits to the packet mark on upload (`mark_out`). On download, restore the mark with `act_ctinfo cpmark` before redirecting to the IFB. CAKE selects tins with `fwmark 0x07000000`; HFSC and HTB use `fw` filters (C14).
- **Consequences.** Classification survives DSCP washing (which enables the C6 default), one rule covers both directions, and bits used by mwan3 and pbr are avoided. The `cpmark` syntax and the fwmark-to-tin mapping must be verified per kernel by the self-test.

If `cpmark` is unavailable, the compat mode restores DSCP with ctinfo instead (O3).

- **Rejected.** DSCP as the class carrier (QoSmate's ctinfo DSCP restore, read): it works, but couples classification to DSCP policy.

7.10.5 ADR-05: CAKE as the default backend

- **Context.** The default must give the strongest out-of-the-box behaviour with the least tuning.
- **Decision.** CAKE `diffserv4` with host isolation, NAT awareness and link-layer compensation is the default; HFSC and HTB are options for gamers and weak CPUs (C4).
- **Consequences.** Flow queueing, sparse-flow boost and per-host fairness come for free [R5], [R6]. CAKE's cost scales with rate and runs per qdisc on one CPU, so the setup bench recommends HFSC or HTB when CAKE cannot reach the target. Heuristic H3 (slow-TCP upgrade) is off on CAKE because the sparse-flow boost already covers it.
- **Rejected.** HFSC as the default (QoSmate's default, read): powerful but needs more tuning for a non-expert household.

7.10.6 ADR-06: Start in SHADOW with static CAKE active

- **Context.** A new adaptive controller has not yet earned the household's trust, but the household should benefit at once.
- **Decision.** Install static CAKE from minute one; run the controller in SHADOW for 24 hours by default, logging what it would do; move to LIVE if the shadow report shows no anomalies, otherwise ask the user (C10). Repeat SHADOW after an algorithm or model upgrade.
- **Consequences.** Immediate SQM-level benefit, a disagreement report for review, and a natural pilot mode for operators. Adaptive benefits are delayed by a day.
- **Rejected.** LIVE from install: faster benefit but riskier, and no pre-deployment evidence.

7.10.7 ADR-07: No runtime self-update

- **Context.** Code that updates itself from the internet is a supply-chain risk on a device that sits on every household packet.
- **Decision.** Updates arrive only through the package manager from a signed feed (usign for opkg, the apk signing key for apk). The model and catalogue are separate signed packages.
- **Consequences.** Predictable versions, rollback through the package manager and a clear audit trail. Updates are slower to reach users than a script fetch.
- **Rejected.** In-script update commands that fetch code at runtime (QoSmate's README installs by fetching raw scripts with `uclient-fetch` and offers an `update` command, read). That approach is convenient for a community tool but not what VeriQ wants for an operator-grade product.

7.10.8 ADR-08: Conformal abstention for the ML classifier

- **Context.** A payload-free classifier will be uncertain on many flows, and a wrong preference is worse than none.
- **Decision.** Use Mondrian (class-conditional) split conformal prediction at $\alpha = 0.05$ per checkpoint [R29], [R30]. Act only on singleton prediction sets; an empty or multi-class set means abstain, which writes nothing.
- **Consequences.** P5 holds by construction: uncertainty never grants priority. The selection rate may be low on unfamiliar networks, and that is acceptable because abstention defaults to Best Effort. Coverage, selection rate, selective accuracy and abstention must all be reported, because high coverage does not imply high selective accuracy.
- **Rejected.** Acting on the top class above a fixed probability threshold: simpler, but the threshold has no coverage interpretation.

7.10.9 ADR-09: LuCI client-side JavaScript views

- **Context.** The admin UI must feel native to OpenWrt users, including those migrating from QoSmate.
- **Decision.** Build `luci-app-veriq` with LuCI's client-side JS framework (`'require view'`, `'require form'`, `'require rpc'`), the same framework QoSmate's LuCI app uses (read). Charts use uPlot; strings use gettext `.po` files (English and Hindi at P1).
- **Consequences.** No server-side Lua, a small footprint, familiar look and feel, and a straightforward rpcd bridge. The household page is a separate vanilla-JS page under 100 KB.
- **Rejected.** A separate web server or single-page framework on the router: heavier, and a second authentication system.

7.10.10 ADR-10: Watchdog as a separate procd instance in POSIX shell

- **Context.** The watchdog must survive any failure of veriqd, including a bug in Go code shared by the daemon.
- **Decision.** Run `veriq-watchdog` as a second procd instance of `/etc/init.d/veriq`. It is a small POSIX shell loop that reads the heartbeat mtime every 2 s and applies SAFE rates from `safe.env` after 10 s of silence (C11).
- **Consequences.** A separate failure domain with almost no code to go wrong, and no dependency on the daemon's libraries. It can only install SAFE; it does not try to fix anything else.
- **Rejected.** A goroutine inside veriqd (shares the failure domain); systemd (not available on OpenWrt; still used for D1 on Ubuntu).

7.11 Summary

VeriQ's architecture puts every packet decision in the kernel and every judgement in one deterministic daemon, with a shell watchdog as the last line of defence. The daemon is organised as 19 single-purpose packages around one control goroutine. It writes to the kernel only through the Action Checker, and it owns a small, versioned set of kernel objects that it can read back, repair and remove. The flows

above define the exact order of operations for the cases judges and testers will probe. The ADRs record why each major choice was made, so that later changes are made deliberately rather than by drift.

CHAPTER 8

Queue management

This chapter covers the part of VeriQ that touches every packet: the shaper, scheduler and active queue management (AQM) in the Linux kernel. Concepts come first, then exactly how VeriQ uses them. All of it runs in the kernel, so it keeps working if the daemon `veriqd` stops (principles P1 and P8).

Acronyms used in this chapter: - **AQM**: active queue management, which drops or marks packets early to keep a queue short. - **CoDel**: Controlled Delay, an AQM algorithm [R2, R3]. - **fq_codel**: Flow Queue CoDel, a per-flow scheduler with CoDel in each queue [R4]. - **CAKE**: Common Applications Kept Enhanced, a shaper, scheduler and AQM in one qdisc [R5, R6]. - **HFSC**: Hierarchical Fair Service Curve [R8]; **HTB**: Hierarchical Token Bucket [R7]. - **qdisc**: queueing discipline, the Linux traffic-control object attached to an interface. - **IFB**: Intermediate Functional Block, a virtual device used to shape download traffic. - **DSCP**: Differentiated Services Code Point, the 6-bit priority field in the IP header [R17]. - **ECN**: Explicit Congestion Notification [R19]; **WMM**: Wi-Fi Multimedia, the 802.11 priority scheme [R18]. - **MTU**, **MSS**: maximum transmission unit and TCP maximum segment size; **MPU**: minimum packet unit. - **DRR**: deficit round robin; **pps**: packets per second.

Every number in this chapter is a **starting parameter** from docs/11, a value from a man page, or an **illustrative** calculation, unless it is explicitly labelled as the Day-0 measurement.

8.1 Concepts first

8.1.1 Shaping versus policing

A **shaper** enforces a rate by *delaying* packets. Packets that arrive faster than the configured rate wait in a queue and leave at the configured rate. The shaper therefore *owns* a queue, and what happens inside that queue (which packet goes next, which packet is dropped) is decided by the scheduler and the AQM attached to it.

A **policer** enforces a rate by *dropping* (or re-marking) packets that exceed it. It has no queue. A policer is cheap and simple, but it treats a TCP flow harshly: a burst above the rate loses several packets in a row, and the sender reacts with a sharp backoff. Policing also does nothing for latency, because it never decides the order in which packets leave.

Property	Shaper	Policer
Mechanism	Queue and delay excess packets	Drop or re-mark excess packets
Effect on TCP	Smooth; the AQM signals congestion early	Bursty losses; throughput below the limit is common
Effect on latency of others	Positive when combined with fair queueing and AQM	None
Memory and CPU	Higher	Lower
VeriQ use	The main WAN shaper (CAKE, HFSC or HTB)	Optional per-device limits (nft meters), labelled “policer”

> **Why it matters.** The case brief asks for queue management “in both directions”. Only a shaper that holds the queue in the router can fix bufferbloat. A policer can cap a device, but it cannot keep a call smooth. VeriQ’s UI never calls a policer a shaper (principle P7).

8.1.2 Active queue management: CoDel

A queue is good when it absorbs a short burst and bad when it stays full. CoDel separates the two by looking at **sojourn time**: how long each packet waited in the queue [R2, R3]. - CoDel tracks the minimum sojourn time over a sliding **interval** (default 100 ms). - If that minimum stays above a **target** (default 5 ms) for a whole interval, a standing queue exists. CoDel then drops (or ECN-marks) one packet. - While the standing queue persists, the next drop comes sooner: the spacing is `interval / sqrt(count)`. This control law increases pressure until the senders slow down. - When the minimum sojourn time falls below target, CoDel stops dropping.

Because CoDel reasons in time, not bytes, the same parameters work across a wide range of rates. They need adjustment only at very low rates, where one full-size packet already takes longer than the target to transmit (see “AQM parameter scaling” below).

8.1.3 Flow queueing: fq_codel

fq_codel hashes each packet’s 5-tuple into one of many queues (1024 by default) and runs CoDel in each queue [R4]. A deficit round robin scheduler serves the queues, a quantum of bytes at a time. Queues that were empty and have just received a packet go on a **new-flows** list that is served before the **old-flows** list.

The effect is that a sparse flow (a DNS query, a game update, a voice packet) waits behind at most one quantum from each busy flow, not behind a whole bulk queue. Bulk flows share the remaining capacity fairly, and each gets its own CoDel signal.

fq_codel has two limits that matter at home: - **It only helps where it is the bottleneck.** On a router whose WAN port runs at 1 Gbit/s in front of a 100 Mbit/s modem, fq_codel on the WAN port never builds a queue; the queue builds in the modem. fq_codel therefore needs a shaper in front of it (HTB or HFSC) or inside it (CAKE). - **Fairness is per flow, not per device.** A device with 200 BitTorrent flows gets 200 shares; a laptop with one video call gets one.

8.1.4 CAKE

CAKE combines a shaper, a flow-queueing scheduler, an AQM and several home-gateway features in one qdisc [R5, R6]. It is VeriQ's default backend, as it is SQM's and one of QoSmate's options [R11, R13].

8.1.4.1 DRR++ and the sparse-flow boost

CAKE schedules flows with **DRR++**, a refinement of fq_codel's new/old lists. A flow that has been idle and sends a packet is treated as *sparse* and served ahead of busy flows, as long as it stays within its quantum. CAKE uses set-associative hashing to reduce collisions between flows sharing a queue. Its AQM is **COBALT**, which combines CoDel with BLUE (a second AQM that reacts to queues that do not respond to CoDel, such as unresponsive UDP).

The sparse-flow boost is important for VeriQ's design: interactive, low-rate flows already receive low delay **without being classified**. That is why VeriQ turns the "slow-TCP upgrade" heuristic off on CAKE (Chapter 9, heuristic H3).

8.1.4.2 Host isolation and NAT awareness

With `dual-srchost` (upload) or `dual-dsthost` (download), CAKE first shares capacity fairly between **hosts**, then between the flows of each host. A device running many flows cannot crowd out a device running one.

On a home router, the WAN side sees only the router's public address after NAT. The `nat` keyword makes CAKE look up the connection-tracking entry to find the internal host address, so host isolation works on the WAN interface and on the IFB.

8.1.4.3 Diffserv tins: priority and weights, not reservations

With a diffserv preset, CAKE splits traffic into **tins**. Each tin has its own flow queues. VeriQ uses `diffserv4`, whose four tins are described in tc-cake(8) [R6]:

Tin (display order)	Default DSCPs per tc-cake(8)	Threshold	VeriQ class
Bulk	CS1 (and LE on newer kernels)	6.25% of the shaped rate	BK
Best Effort	everything not listed elsewhere	100%	BE
Video	AF4x, AF3x, CS3, AF2x, CS2, TOS4, TOS1	50%	IV
Voice	CS7, CS6, EF, VA, CS5, CS4	25%	RT

The threshold is the key to honest wording. A tin whose traffic stays below its threshold is served with priority. Above the threshold, the tin keeps only a reduced scheduling weight, so it cannot starve the others. Voice traffic beyond roughly a quarter of the shaped rate therefore loses its priority. **A tin is preference with a soft ceiling. It is not a reserved rate, and it is not a bandwidth guarantee.** The tin can still use spare capacity when nobody else needs it.

> **Note.** The DSCP lists above come from the man page and should be checked against the man page of the target iproute2 and kernel. VeriQ does not depend on them for classified traffic, because it selects tins with `fwmark` (below). They matter only for flows with no VeriQ class.

8.1.4.4 ACK filter, overhead, MPU, wash and ingress mode

- `ack-filter` removes redundant pure TCP ACKs from the upload queue when a newer ACK for the same flow is already queued. On a very asymmetric link (fast download, slow upload) this frees upload capacity without hurting the download.
- `overhead N` and `mpu M` tell CAKE how many bytes the link layer adds to each packet and the minimum size it bills, so the shaper counts what the bottleneck actually transmits. Keywords such as `ethernet`, `docsis` or `pppoe-ptm` set both values (see "Link-layer presets").
- `wash` clears the DSCP bits (not ECN) **after** tin selection [R6]. It stops marks leaking upstream or arriving from the ISP into the home.
- `ingress` is for shaping download traffic on an IFB. It makes CAKE count dropped packets as if they had been sent, because they already used the access link.
- `fwmark MASK` overrides tin selection. CAKE masks the packet mark, shifts the result right by the number of trailing zero bits of the mask, and uses the value to pick a tin. A zero result means "no override": CAKE falls back to the packet DSCP.

8.1.5 HFSC: service curves

HFSC schedules a class hierarchy with **service curves** [R8]. A curve is two linear pieces: slope `m1` for the first `d` milliseconds of a backlog period, then slope `m2`. A class can have up to three curves:

Curve	Meaning	Typical use
`rt` (real-time)	The class receives at least this service, regardless of the other classes; the two-piece shape decouples delay from long-term rate	The RT class for game and voice packets
`ls` (link-share)	How spare capacity is divided among siblings, in proportion to the curves	IV, BE and BK classes
`ul` (upper limit)	Cap on link-share service	The parent class, set to the shaped rate S

The trick that makes HFSC attractive to gamers is the `rt` curve with `m1 > m2`. A burst of small game packets after a quiet period is served at the high initial slope, so it leaves quickly, while the long-term rate stays at `m2`. A class with only an `rt` curve does not take part in link-sharing, so its long-term service is bounded by `m2`.

8.1.6 HTB: token buckets with borrowing

HTB gives each class an assured `rate` and a `ceil` up to which it may borrow unused capacity from its parent [R7]. When several classes want to borrow, lower `prio` values borrow first. `burst` and `cburst` size the token buckets; at high rates they must be large enough to cover the kernel's timer granularity, or the class undershoots its rate. `quantum` controls how borrowing is shared within one priority level.

HTB is simple and cheap per packet. It has no built-in AQM, so each leaf class needs its own qdisc (fq_codel in VeriQ).

8.1.7 Summary of the building blocks

Building block	Shapes the rate	Fairness	AQM	Priority mechanism	Relative CPU cost
fq_codel alone	No	Per flow	CoDel	None	Low
HTB + fq_codel leaves	Yes	Per flow, within each leaf	CoDel per leaf	`prio` and borrowing order	Lowest of the shapers
HFSC + fq_codel leaves	Yes	Per flow, within each leaf	CoDel per leaf	`rt` service curve	Low to medium
CAKE diffserv4	Yes	Per host, then per flow (DRR++)	COBALT	Tins with thresholds	Highest

The CPU column is qualitative. The bench on the target router decides the backend, not this table (see "CPU cost considerations").

8.2 Why the shaper must sit just below the bottleneck

A queue forms at the **bottleneck**, usually the access link (modem, ONT, DSLAM port or cell), whose buffers are often large, simple FIFOs. A router-side AQM can only manage a queue that is in the router. The fix is to shape slightly **below** the bottleneck rate: the router becomes the slowest point, the queue moves into CAKE, and the modem buffer stays nearly empty.

The Day-0 sandbox run illustrates the effect. It is one preliminary run in an emulated topology (20 Mbit/s down, 5 Mbit/s up, a 300 ms FIFO at the emulated ISP, no base RTT applied, IPv4 only, unreplicated) [R51]:

Arm (Day-0, measured, preliminary)	Added p95 delay, upload	Added p95 delay, download
FIFO at the bottleneck	266.1 ms	63.6 ms
Short-buffer shaper in the router	39.6 ms	55.4 ms

The upload case shows the mechanism: when the router shapes below the bottleneck, the 300 ms buffer at the emulated ISP no longer fills. CAKE was not available in that sandbox kernel, so the run says nothing yet about CAKE; the download numbers are too close to interpret from a single run. Chapter 25 describes how the lab replicates the run on a full kernel.

Illustrative arithmetic. A 300 ms buffer at 5 Mbit/s holds 5,000,000 × 0.3 / 8 = 187,500 bytes, about 125 full-size packets; a voice packet behind it waits 300 ms. Shaped at 4.5 Mbit/s, the modem buffer stays close to empty and the queue sits in CAKE, whose delay target is 5 ms.

How far below? Three effects push the margin up: - **Capacity varies.** A rate set at today's speed-test peak is too high in the evening; this is what VeriQ's controller addresses (Chapter 10). - **Overhead errors.** A shaper that under-counts link-layer bytes runs faster than its nominal rate (see "Link-layer presets"). - **Download shaping happens after the bottleneck.** Download packets have already crossed the access link; the router can only slow senders indirectly, by dropping or ECN-marking. Bursts still pile up upstream, so download needs more margin, and CAKE's `ingress` mode helps.

VeriQ's setup wizard sets `S_base` to **90% of the lowest of three capacity measurements** (docs/11 §11.1), a starting parameter. The lab script uses 95% for its fixed-rate arms. Both are starting points; the controller then moves `S` within `[S_min, S_max] = [max(1 Mbit/s, 0.10·S_base), 1.05·S_base]`.

8.3 VeriQ service classes

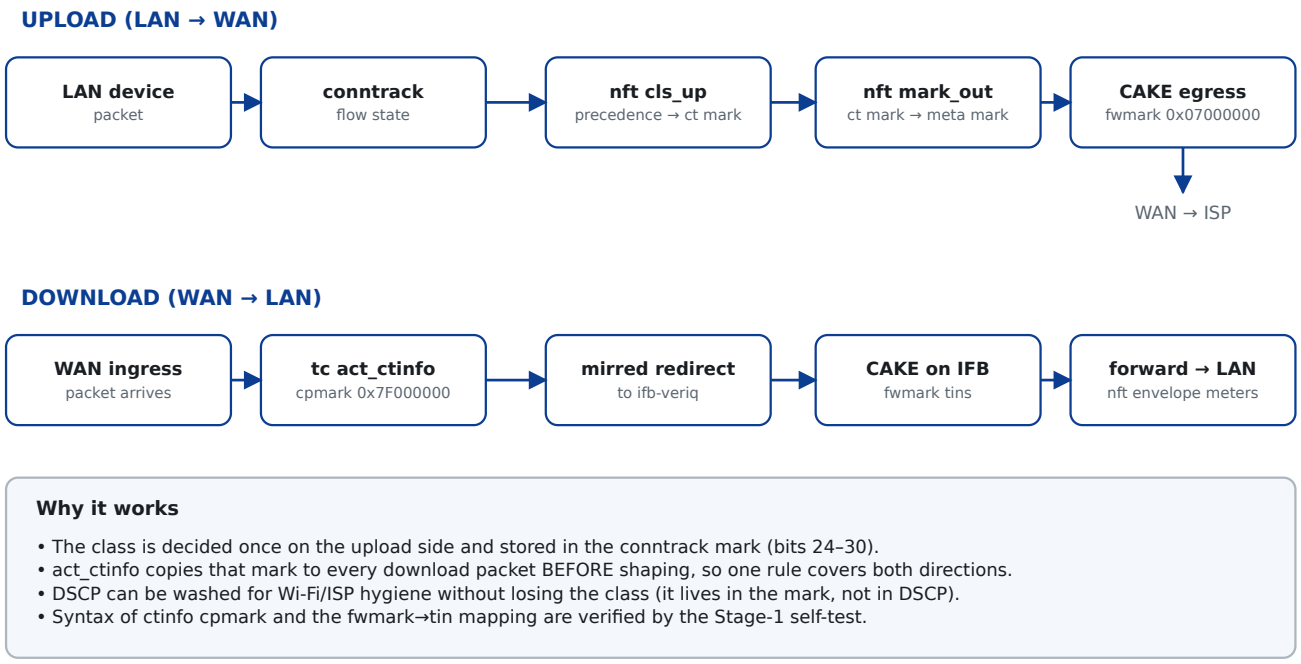
VeriQ defines **one service-class contract** and maps it onto three backends (docs/11 §4.1):

VeriQ class	Meaning	CAKE diffserv4 tin (fwmark value)	HFSC class	HTB class
RT	Realtime: voice, game state, call audio	Voice (4)	`1:11` rt curve	`1:11` prio 0
IV	Interactive video: call video, cloud-gaming stream	Video (3)	`1:12` ls high	`1:11` prio 0
BE	Best effort: web, apps, buffered streaming, unknown	Best Effort (2)	`1:13` ls normal (default)	`1:13` prio 1 (default)
BK	Bulk: downloads, updates, backups, P2P	Bulk (1)	`1:15` ls low	`1:15` prio 2

Design notes: - **Four classes, not eight.** Four is the smallest set that separates the needs in the home traffic mix (Chapter 4): tiny delay-critical packets, rate-hungry but delay-sensitive video, ordinary traffic, and traffic that should yield. More classes would not change what a household experiences, and they would make the per-tin self-test longer. - **Buffered streaming is BE, not IV.** Netflix-style players buffer tens of seconds; they need throughput, not low delay. Putting them above web browsing would make page loads worse for no gain. - **Unknown is BE.** A flow nobody can identify never gets preference (Chapter 9). - **fwmark values 1–4 select tins in tc's display order** for diffserv4 (Bulk, Best Effort, Video, Voice). This mapping must be verified by the Stage-1 tin self-test on every target kernel; the self-test is the source of truth, and the mapping table is configurable if a kernel differs (open decision O2).

> **For judges.** "RT" means *served first while it stays small*, not *reserved bandwidth*. If a misbehaving app pushes 10 Mbit/s of EF traffic, CAKE's Voice threshold and VeriQ's eligibility envelope (Chapter 9) both act. Preference is earned per flow and can be lost.

8.4 The datapath



Why it works

- The class is decided once on the upload side and stored in the conntrack mark (bits 24–30).
- act_ctinfo copies that mark to every download packet BEFORE shaping, so one rule covers both directions.
- DSCP can be washed for Wi-Fi/ISP hygiene without losing the class (it lives in the mark, not in DSCP).
- Syntax of ctinfo cpmark and the fwmark→tin mapping are verified by the Stage-1 self-test.

Figure 5. Packet path: LAN to nft classification, conntrack mark, packet mark and CAKE on WAN egress; WAN ingress through act_ctinfo and the IFB to CAKE.

8.4.1 Where the class lives

VeriQ stores a flow's class in **conntrack mark bits**, not in DSCP (decision C14; full layout in Chapter 20):

Bits	Mask	Field
24–26	`0x07000000`	CLASS: 0 none, 1 BK, 2 BE, 3 IV, 4 RT
27–29	`0x38000000`	PROV: provenance of the class (Chapter 9)
30	`0x40000000`	SANCT: sanctioned by the envelope
0–23, 31	—	Never touched (mwan3 and pbr use parts of these)

Because the class is attached to the connection, one decision on an upload packet covers the download packets of the same connection. Because it is not in DSCP, washing DSCP does not erase it.

8.4.2 Upload path, step by step

1. A LAN packet enters the router. Conntrack finds or creates the connection entry.
2. The nft chain `cls_up` (hook forward, priority mangle + 5) runs the precedence pipeline (Chapter 9) and may update the CLASS, PROV and SANCT bits of `ct mark`.
3. The chain `mark_out` (hook postrouting, priority mangle + 10) copies the class bits into the packet mark with four constant rules, one per class value.
4. CAKE on the WAN device, configured with `fwmark 0x07000000`, reads the packet mark, shifts it right by 24 and selects the tin.
5. CAKE queues, schedules, applies COBALT and, in the product profile, washes DSCP before transmission.

```
# Fragment of chain mark_out (illustrative; full template in Chapter 20).
# Copies CLASS from ct mark to meta mark without touching other bits.
ct mark & 0x07000000 == 0x04000000 meta mark set meta mark | 0x04000000
ct mark & 0x07000000 == 0x03000000 meta mark set meta mark | 0x03000000
ct mark & 0x07000000 == 0x02000000 meta mark set meta mark | 0x02000000
ct mark & 0x07000000 == 0x01000000 meta mark set meta mark | 0x01000000
```

Implementation tip. Constant-OR rules are easy to read back and count, and they do not depend on the nft version's support for register-to-register bitwise expressions. Packet-mark bits 24–26 must be zero before this chain; the health check confirms no other package sets them.

8.4.3 Download path: IFB and act_ctinfo

Linux can only shape traffic on *egress*. Download traffic arrives on the WAN device's *ingress*, before netfilter sees it. The standard solution, used by SQM and QoSmate as well [R11, R13], is to redirect ingress traffic to an IFB device and shape it on the IFB's egress.

The class must travel with it. That is the job of `act_ctinfo` [R7]:

1. A download packet arrives on the WAN device. A tc filter on the ingress hook matches all packets.
2. `action ctinfo cpmark 0x7F000000` looks up the packet's conntrack entry and copies bits 24–30 of `ct mark` into the packet mark.
3. `action mirred egress redirect dev ifb-veriq` sends the packet to the IFB.
4. CAKE on `ifb-veriq` (with `ingress`, `dual-dsthost`, `nat` and `fwmark 0x07000000`) selects the tin from the packet mark, queues and schedules.
5. The packet re-enters the stack, traverses netfilter (prerouting, forward) and reaches the LAN.

```
# Download plumbing sketch. Verify the cpmark syntax with tc-ctinfo(8) on the target
# iproute2: this is a Stage-1 self-test item (docs/11 section 4.2, open decision O3).
ip link add name ifb-veriq type ifb
ip link set ifb-veriq up
tc qdisc add dev "$WAN" handle ffff: ingress
tc filter add dev "$WAN" parent ffff: protocol all matchall \
    action ctinfo cpmark 0x7F000000 \
    action mirred egress redirect dev ifb-veriq
```

If `act_ctinfo cpmark` is missing or broken on a target, open decision O3 applies: VeriQ falls back to a **compat mode** that stores DSCP in conntrack and restores it on ingress with `ctinfo dscp 63 128`, the method QoSmate uses (read), and the health page shows "compat mode".

Implementation tip. The first packets of a new download connection may arrive before the upload side has classified the flow. They carry CLASS = 0 and are scheduled by CAKE's own DSCP rules. Marks never apply retroactively (docs/09 A.8). A device with an active lease is the exception: its first upload packet already matches the lease set.

Implementation tip (verify in Stage 1). CAKE's `wash` clears DSCP *after* tin selection. On the IFB, a CLASS = 0 packet is therefore placed by the DSCP it arrived with, which may have been set by the ISP. On upload, the same holds if the product-profile wash is done with CAKE's `wash` keyword rather than an nft rule before CAKE. docs/11 allows either method for egress washing. The recommended renderer choice is the nft rule at the end of the classification path for upload, so that CLASS = 0 means Best Effort in practice. The tin self-test must include a CLASS = 0 packet with a preferential DSCP in each direction, so the team sees what the kernel actually does. If the download case matters in practice, the team should record the chosen mitigation in docs/12.

8.5 The three backends in depth

All three backends implement the same four-class contract, read the same mark bits, and accept rate changes from the same controller (decision C4). The daemon talks to them through one Go interface (`Render`, `RateChange`, `Readback`; Chapter 19).

8.5.1 `cake` (default)

Egress (WAN), template:

```
tc qdisc replace dev $WAN root cake bandwidth ${S_up}kbit diffserv4 \
    dual-srchost nat fwmark 0x07000000 [ack-filter] [wash] \
    <preset keywords or overhead N mpu M> [rtt R]
```

Ingress (IFB), template:

```
tc qdisc replace dev ifb-veriq root cake bandwidth ${S_down}kbit diffserv4 \
    dual-dsthost nat ingress fwmark 0x07000000 [wash] \
    <preset keywords or overhead N mpu M> [rtt R]
```

Choices and reasons: - **`diffserv4`** is required, because the fwmark values 1–4 are defined against it. A QoSmate config with `diffserv3` or `diffserv8` is imported as `diffserv4` with a note (Chapter 9, importer). - **`dual-srchost` / `dual-dsthost` with `nat`** give per-device fairness on both directions (host isolation is on by default). - **`autorate-ingress` is never used.** CAKE's built-in rate estimator would fight VeriQ's controller over the same `bandwidth` value. - **`rtt`** stays at CAKE's default unless the user sets it. - Extra parameters from the user (`extra_egress`, `extra_ingress`) are validated: any option that conflicts with the contract (`bandwidth`, a diffserv preset, `fwmark`, `autorate-ingress`) is refused with an error against `cake.extra_*`.

When to use: whenever the bench shows that CAKE reaches at least 95% of the target rate with the shaping CPU at or below 85% (docs/11 §11.1). Only the bench on the actual device decides; the handbook quotes no per-device figures.

8.5.2 `hfsc`

Structure (both directions; on the IFB for download):

```
root 1: hfsc default 13      (stab overhead/linklayer from the preset)
└─ 1:1  ls m2 S   ul m2 S                parent: caps link-share at S
   ├─ 1:11 rt m1 B d D m2 R             RT  (leaf fq_codel, or pfifo with computed limit)
   ├─ 1:12 ls (high share)              IV  (leaf fq_codel)
   ├─ 1:13 ls (normal share)            BE  (leaf fq_codel) - default
   └─ 1:15 ls (low share)               BK  (leaf fq_codel)
Filters: tc fw filters, handle 0x0N000000/0x07000000 -> 1:1N (N = class)
```

The RT curve. `R` (m2) is the **RT budget**. Its VeriQ default is `max(400 kbit/s, 15% of S)` (docs/11 §4.3), configurable per direction. QoSmate's README gives its GAMEUP/GAMEDOWN default as 15% of the rate **plus** 400 kbit/s (read); the two defaults differ by at most 400 kbit/s, and the importer carries an explicit QoSmate value across unchanged. The burst slope `B` (m1) and duration `D` follow the shape QoSmate uses (read), adopted as starting values: `D = max(25 ms, 5 × 1500 B serialisation time at S)` and `B = min(10 × R, 0.97 × S)`.

The RT leaf. Two options: - `fq_codel` (default): per-flow fairness inside the RT class, with CoDel as a safety net. - `pfifo` with a computed packet limit, for users who want the classic "small FIFO for game packets" behaviour: `limit = pfifo_min + ceil(maxdel_ms × R / 8 / packet_size)`, with starting values `maxdel_ms = 24`, `pfifo_min = 5`, `packet_size = 450` bytes (Appendix B of docs/11). VeriQ sizes the limit against the RT class's own rate `R`, because an rt-only class drains at about `R` in the long run; QoSmate's pfifo formula uses the overall rate (read). Both are starting choices to be compared in H3.

Link-share shares for IV, BE and BK are starting values (proposed: IV 30%, BE 55%, BK 15% of S as `ls m2`), tuned in Stage 2. Shares only matter when the link is saturated; any class can use spare capacity.

When to use: gamers who want a hard realtime curve for a small set of packets, and mid-range CPUs where CAKE is borderline at the target rate. HFSC has no host isolation and no built-in ACK filter.

8.5.3 `htb`

Structure: HTB root with three leaves, because RT and IV share one class:

```
root 1: htb default 13
└─ 1:1  rate S ceil S
   ├─ 1:11 rate A_rt  ceil S prio 0      RT + IV  (fq_codel, tight target)
   ├─ 1:13 rate A_be  ceil S prio 1      BE       (fq_codel)  - default
   └─ 1:15 rate A_bk  ceil S prio 2      BK       (fq_codel, relaxed target)
```

Assured rates are starting values: `A_rt` uses the same RT-budget formula as HFSC, and the remainder is split between BE and BK (proposed 80/20). Every class has `ceil = S`, so idle capacity is borrowed in priority order. The `fast_be` option (Appendix B, default on) implements heuristic H3 as a "fast BE" sub-class on HTB and HFSC (Chapter 9). It never promotes to RT or IV.

When to use: the weakest CPUs (MIPS MT7621 class, the D5 low-end check) and gigabit plans on modest ARM devices where even HFSC is too expensive. The documented cost is **no host isolation** and only three classes.

8.5.4 Choosing a backend

Situation	Recommended backend	Reason
Bench: CAKE ≥ 95% of target and CPU ≤ 85%	`cake`	Best feature set: host isolation, sparse-flow boost, ACK filter
Competitive gamer, wants a strict realtime curve, CPU adequate	`hfsc`	`rt` curve serves game bursts at `m1`
CAKE fails the bench, HFSC passes	`hfsc`	Lower per-packet cost than CAKE
Both fail, or MIPS-class router	`htb`, or shape one direction only	Lowest per-packet cost
Gigabit download that does not bloat	Shape upload only (`down_base_kbit = 0`)	Download shaping costs the most CPU and may not be needed

Backend switches are **user-initiated only** (Action Checker rule `BACKEND_SWITCH`, refusal code `NOT_USER_INITIATED`). The controller never switches backends on its own.

8.6 Link-layer presets

Every link layer adds bytes around each IP packet. A shaper that ignores them counts fewer bytes than the bottleneck sends, so its effective rate is too high. The error is small for full-size packets and large for small ones.

Preset	CAKE keyword (values per tc-cake(8))	HFSC/HTB [illegible] equivalent
`ethernet` (default for fibre and Ethernet WAN)	`ethernet` (overhead 38, mpu 84)	`overhead 38 linklayer ethernet`
`ethernet-vlan`	`ethernet ether-vlan` (+4)	`overhead 42`
`docsis`	`docsis` (overhead 18, mpu 64)	`overhead 18`
`vdsl2-pppoe`	`pppoe-ptm` (overhead 30)	`overhead 30 linklayer ethernet`
`vdsl2-bridged`	`bridged-ptm` (overhead 22)	`overhead 22`
`adsl`	`conservative` (overhead 48, atm) unless the exact encapsulation is known	`overhead 48 linklayer atm`
`unknown` (non-DSL)	`overhead 44 mpu 84` (no ATM cell tax)	`overhead 44 linklayer ethernet`
`custom`	`overhead N mpu M [atm/ptm]`	user values

The wizard proposes a preset from the netifd protocol: PPPoE adds 8 bytes and a VLAN device adds 4. As QoSmate's guidance also says, **overestimating overhead is safer than underestimating it**: too much overhead wastes a little capacity; too little lets the queue escape to the modem. An ATM setting on a fibre link would waste about 10% of the rate, which is why the `unknown` preset avoids it.

Worked example (illustrative). A 20 Mbit/s upload carrying a TCP download's ACKs and a voice call: - A 1500-byte IP packet on Ethernet occupies 1500 + 38 = 1538 bytes on the wire. Ignoring overhead under-counts by 2.5%. - A 40-byte pure TCP ACK occupies 40 + 38 = 78 bytes, rounded up to the MPU of 84 bytes. Ignoring overhead under-counts by a factor of 2.1. - A 200-byte voice packet occupies 238 bytes; the under-count is 19%.

An upload dominated by ACKs and voice packets can therefore exceed the access rate by tens of percent if overhead is ignored, and the queue moves back to the modem exactly when the household is on a call.

> **Implementation tip.** On ATM-based ADSL, the cell tax (48 bytes of payload per 53-byte cell, plus padding of the last cell) depends on packet size, so only `linklayer atm` models it correctly. VDSL2 uses PTM, which has a small fixed overhead and no cell padding.

8.7 One-directional shaping

Setting `down_base_kbit = 0` or `up_base_kbit = 0` disables shaping in that direction, the same convention as SQM and QoSmate (read). For a disabled download direction, VeriQ creates no IFB, no ingress filter and no redirect, so the download path is untouched.

Consequences the user is told about: - The health page reports the direction as **"off"**, not "failed", and the controller adjusts only the shaped direction. - Heuristics H1 and H2 derive their byte thresholds from the download base rate; without one they are switched off, as in QoSmate (read). - `ack_filter = auto` cannot compute the download/upload ratio, so the user should set it explicitly.

8.8 ACK and MSS handling

ACK filter (CAKE). With `ack_filter = auto`, VeriQ enables CAKE's `ack-filter` on egress when the configured download/upload ratio is **≥ 15**, the same rule QoSmate uses (read). Example: a 100/20 Mbit/s plan (ratio 5) leaves it off; a 100/5 Mbit/s plan (ratio 20) turns it on. A separate **warning** appears when the ratio exceeds 20, because downloads may then be ACK-limited. VeriQ does not cap the download rate relative to the upload rate (QoSmate's `BWMAXRATIO` does; VeriQ deliberately differs, parity item 9 in docs/11 §16), because a cap would silently waste download capacity.

ACKs on HFSC/HTB. VeriQ drops no ACKs on these backends. Pure ACKs (TCP, under 100 bytes) of a classified flow ride in that flow's class, so a download in IV has its ACKs in IV. QoSmate's `ACKRATE` thinning by nft rules is not adopted (parity item 10): fewer hand-tuned drop rules means fewer surprising interactions.

MSS clamp. When either shaped rate is below **3000 kbit/s** (`mss_clamp = auto`), VeriQ clamps the TCP MSS on SYN packets to a configurable value between 536 and 1500 bytes, as QoSmate does (read). The reason is serialisation delay. At 1 Mbit/s, one 1500-byte packet takes 12 ms to transmit, and a voice packet that arrives just after it waits that long. With an MSS of 536 (packets of 576 bytes), the wait drops to about 4.6 ms. The cost is header overhead: 40 bytes of TCP/IP header per 576-byte packet is about 7%, compared with about 2.7% at 1500 bytes. On very slow uplinks (some FWA and rural DSL connections), that trade is worth making.

8.9 Per-device policers: honest semantics

The **Limit** button on the Devices page, and the `ratelimit` UCI section imported from QoSmate, create nft **meters** in the forward hook:

```
# Illustrative per-device download policer, 5 Mbit/s with burst_factor 1.0.
# 5000 kbit/s = 625 kbytes/second; burst = rate x burst_factor. Verify meter syntax.
ip daddr 192.168.1.60 meter dl_guest4 { ip daddr limit rate over 625 kbytes/second \
    burst 625 kbytes } counter drop
```

What this does and does not do: - It **drops** packets above the rate. It does not queue them, so it does not reduce anyone's latency. - For **upload**, the drop happens before the packet uses the access link, which saves upstream capacity. - For **download**, the packet has already crossed the access link. The drop works only because the sender's congestion control slows down. Download policing therefore wastes some capacity, and TCP throughput through a policer is often below the configured limit. - It is per address. A device with several IPv6 privacy addresses needs a device set (VeriQ maintains these automatically; Chapter 9).

The UI labels it **"policer (limit), not a shaper"** (principle P7), and the per-rule counter shows drops. Units follow QoSmate (read): the user enters kbit/s, and the renderer converts to kbytes/second by multiplying by 0.125.

8.10 AQM parameter scaling

CoDel's default target of 5 ms assumes that one full-size packet takes much less than 5 ms to send. On slow links that fails: at 2 Mbit/s a 1500-byte packet takes 6 ms, so a single packet in service already exceeds the target, and CoDel drops too aggressively. CAKE scales its own parameters internally; for the fq_codel leaves of HFSC and HTB, VeriQ computes them (docs/11 §4.5):

- `target = max(5 ms, 1.5 × MTU serialisation time at S)`;
- `interval = 100 ms + target`;
- `quantum = 300` bytes when `S < 100 Mbit/s` (so small packets get finer-grained turns), otherwise the default.

S (illustrative)	1500-byte serialisation	target	interval	quantum
1 Mbit/s	12.0 ms	18 ms	118 ms	300
2 Mbit/s	6.0 ms	9 ms	109 ms	300
5 Mbit/s	2.4 ms	5 ms (3.6 ms rounds up to the floor)	105 ms	300
20 Mbit/s	0.6 ms	5 ms	105 ms	300
200 Mbit/s	0.06 ms	5 ms	105 ms	default

The parameters are recomputed when `S` changes by more than a configured fraction, not on every controller tick, so a rate change does not rewrite every leaf. ECN marking stays on in CAKE and fq_codel; VeriQ itself sets no ECN bits.

8.11 DSCP policy and Wi-Fi WMM

VeriQ separates **classification** (where the class lives: ct mark) from **DSCP** (a header field other devices may read). That allows a clean policy (decision C6):

Profile	Ingress wash	Egress wash	Trust LAN DSCP
product (default)	on	on	yes, subject to the eligibility envelope
lab	off	off	yes
custom	user	user	per device set `@trusted_dscp`

Why wash ingress in the product profile. Many Wi-Fi access points map DSCP to a WMM access category (voice, video, best effort, background) [R18]. RFC 8325 recommends, for example, EF to the voice category, AF4x to the video category and CS1 to background. If the ISP's network delivers packets marked CS1 or EF, a home AP may put an ordinary download into the background or voice queue on the air. Washing on ingress removes marks that the household did not choose. Drivers differ in how they map DSCP to access categories, so a capture on the target AP is the only way to confirm the effect.

Why wash egress in the product profile. It avoids leaking marks upstream. Some networks treat marked packets worse, not better (QoSmate's README makes the same point, read).

Why the lab profile keeps DSCP. Controller-only comparisons against cake-autorate and QoSmate's autorate must use an identical datapath. Washing in one arm and not another would confound the comparison (Chapter 24).

What washing costs. With ingress wash on, LAN devices see CS0 on downloads, so a Wi-Fi AP cannot give a protected call the WMM voice or video category on the air. VeriQ's class still applies at the WAN shaper, where the access-link queue is. Re-marking download DSCP from the VeriQ class for WMM is **not** part of the v5 design; it is a candidate for later work, and it would need its own Wi-Fi test.

8.12 Rate updates per backend

The controller changes `S` through the Action Checker, and the enforcer applies an accepted change with an ordered, read-back sequence (docs/11 §4.3):

Backend	Rate change	Steps	Read-back
`cake`	`tc qdisc change dev X root cake bandwidth Ykbit`	One netlink message	`tc -s -j qdisc show dev X`: parse the bandwidth field
`hfsc`	Parent `1:1` ls/ul first, then leaves scaled proportionally (RT curve recomputed from the RT budget formula)	Several, in one ordered `tc -batch`	Each class read back; mismatch triggers a compensating step or SAFE
`htb`	Parent `1:1` rate/ceil first, then leaf rates and ceilings	Several, ordered	As for HFSC

Rules that apply to all backends: - A tc batch is **not atomic**. Steps are ordered so that each intermediate state is valid, each step is read back, and a partial failure leads to a compensating step or to SAFE, with at most 3 retries and capped backoff (docs/11 §8.3). - Every change carries the action ID and desired-state version. The enforcer refuses a command without an ACCEPT for the same ID and version. - The rate budget is about two changes per second at most, which suits process-based `tc` (each call costs a few milliseconds). - The SAFE profile is a static CAKE (or the configured backend) at `0.85 × min(S_lkg, S_base)`; the watchdog applies it with a single `tc qdisc change` from `/var/run/veriq/safe.env` (Chapter 12).

> **Implementation tip.** Read back the CAKE `bandwidth` field from `tc -s -j` rather than trusting the exit code. For HFSC, the read-back after a large cut also confirms that the RT class's `m2` (recomputed at about 15% of the new S) is well below the new parent rate.

8.13 CPU cost considerations

Shaping costs CPU in the kernel's softirq context, and the cost scales with **packets per second**, not bits per second.

Illustrative arithmetic. With full-size packets on Ethernet (1538 bytes on the wire): - 100 Mbit/s ≈ 100,000,000 / (1538 × 8) ≈ 8,100 pps; - 1 Gbit/s ≈ 81,000 pps.

A mix with many small packets has more packets per second at the same bit rate: with an assumed 600-byte average, about 2.4 times as many.

Factors that raise the cost: - **CAKE does more per packet** than HTB or HFSC with fq_codel (hashing, host fairness, NAT lookups, COBALT, overhead accounting). - **Download shaping passes through the IFB**, adding a redirect and a second qdisc pass. - **One qdisc runs on one CPU at a time**; multi-core routers do not spread one shaper across cores. CAKE's multi-queue option is deferred (parity item 37). - **Flow offloading bypasses the shaper**, so VeriQ refuses LIVE while it is on (A24).

How VeriQ handles it: 1. The wizard runs the **bench** (`veriqctl bench`): local veth/netns load, no WAN traffic, CAKE, HFSC and HTB at the target rate (docs/11 §12.5). 2. If CAKE reaches less than 95% of the target or the shaping CPU exceeds 85%, it recommends `hfsc`, `htb` or one-directional shaping. 3. At runtime, a CPU-bound shaper is diagnosed as **CPU_LIMITED** (decision C13) and the controller holds the rate, because a cut would not fix a CPU limit (A17). 4. The evaluation never attributes CAKE's kernel CPU cost to `veriqd`, or the reverse.

> **Honesty check.** This chapter quotes no CPU figures for specific routers. Any number the team publishes must come from the bench on the device, recorded in the evidence ledger.

8.14 Worked example: sizing the HFSC RT budget for 100/20 Mbit/s

Assume a 100/20 Mbit/s FTTH plan, with the wizard's `S_base` at 90/18 Mbit/s for download/upload. All values are **illustrative**, computed from the starting formulas above.

Step 1: RT budget `R = max(400 kbit/s, 15% of S)`. - Download: 15% × 90,000 = 13,500 kbit/s. `R_down = 13,500 kbit/s`. - Upload: 15% × 18,000 = 2,700 kbit/s. `R_up = 2,700 kbit/s`. - For comparison, QoSmate's formula (15% + 400 kbit/s, read) would give 13,900 and 3,100 kbit/s.

Step 2: check the budget against realistic demand. QoSmate's README notes that game packet rates often follow the server tick rate, for example about 20, 60 or 128 pps (read), and a voice call needs on the order of tens of kbit/s. As an assumption, take two consoles at 60 pps × 200 bytes (96 kbit/s each) and one voice call at 100 kbit/s: about 300 kbit/s per direction. The upload budget of 2,700 kbit/s is about nine times that demand, which leaves room for bursts and for several more devices.

Step 3: burst curve. `D = max(25 ms, 5 × 1500 × 8 / S)`. At 18,000 kbit/s, 60,000 bits / 18,000 kbit/s = 3.3 ms, so `D = 25 ms`. `B = min(10 × R, 0.97 × S) = min(27,000, 17,460) = 17,460 kbit/s` for upload. A burst of game packets after a quiet period is served at up to 17.5 Mbit/s for 25 ms, which is about 54,500 bytes, roughly 120 packets of 450 bytes.

Step 4: RT leaf limit (pfifo option). `limit = 5 + ceil(24 × 2,700 / 8 / 450) = 5 + ceil(18) = 23` packets for upload. At the RT rate, 23 packets of 450 bytes drain in about 31 ms, so the worst-case RT queue delay stays near the `maxdel_ms` intent.

Step 5: what happens after a rate cut. If the controller cuts download from 90 to 18 Mbit/s (Chapter 10, worked example), the RT budget is recomputed: `max(400, 15% × 18,000) = 2,700 kbit/s`. The RT class keeps the same *share*, not the same absolute rate; the household is told that protection scales with the available capacity.

Step 6: the envelope still applies. The RT class in HFSC accepts only flows that hold an RT class, and Chapter 9's envelope limits each RT flow to `max(1 Mbit/s, 2% of S_base)` and 450 pps. A flow that tries to use the RT budget for bulk transfer is sanctioned to BE.

8.15 How to explain this chapter to a judge

> **For judges.** "We move the queue from the modem into the router and manage it with flow queueing and AQM, as SQM, QoSmate and cake-autorate do [R10–R13]. VeriQ adds one class contract over three backends, the class carried in conntrack bits so it survives DSCP washing and covers both directions, a tin self-test on every kernel, and a bench that picks the backend. In static conditions we expect parity with any CAKE setup; tins are preference, not reserved bandwidth, and a policer is not a shaper."

8.16 Queue-management checklist

- ☐ CAKE, ctinfo and IFB modules present; fwmark → tin mapping verified by the tin self-test in **both** directions.
- ☐ `ctinfo cpmark` verified on the target iproute2, or compat mode active and shown.
- ☐ CLASS = 0 packets with a preferential DSCP tested in both directions (wash behaviour).
- ☐ Link preset confirmed; overhead not underestimated.
- ☐ Bench run and backend recommendation recorded; flow offloading off; no other QoS service on the WAN.
- ☐ Rate change read back for each backend; SAFE profile written to `safe.env`.

CHAPTER 9

Chapter 09 (pending)

This chapter has not been written yet.

CHAPTER 10

Capacity Estimation and Rate Control

A shaper only fixes bufferbloat when its rate sits just below the real bottleneck. Set it too high and the queue moves back into the modem or the ISP. Set it too low and the household pays for bandwidth it cannot use. On a fixed fibre line a well-tuned static rate is enough. On 4G/5G fixed wireless access (FWA), on shared PON or cable segments at peak hours, and on any link whose capacity moves, the rate has to follow the link.

This chapter specifies how VeriQ's daemon (`veriqd`) estimates the link and sets the shaper rate S in each direction. The next chapter covers diagnosis, which decides *whether* a cut is allowed at all.

> **Honesty check.** Every number in this chapter is a **starting parameter** from docs/11 §6 and docs/09 Appendix A, to be tuned in Stage 2 and recorded in the run manifest, or it is **illustrative** arithmetic. None is a measured result. No VeriQ controller code has run yet. Statements about QoSmate's autorate come from `qosmate-autorate.sh` at v0.5.48 @ c507843 and are marked **(read)**; what they imply in practice is a **hypothesis** for the head-to-head.

10.1 The quantities, and why they are never conflated

Autorate bugs often come from mixing up "what we configured", "what we pushed through" and "what the link could carry". VeriQ gives each its own symbol, source and unit (kbit/s, ms), and every sample carries a monotonic timestamp.

Symbol	Meaning	Source	Used for
S_dir	Configured shaper rate per direction (up, down)	Desired state, read back from `tc`	The control variable
S_base, S_min, S_max	Envelope: S_base is the nominal rate from the wizard; S_min = max(1 Mbit/s, 0.10·S_base); S_max = 1.05·S_base	UCI config (docs/09 A.2)	Hard bounds checked by the Action Checker
G_gw	Gateway throughput: bytes dequeued by our shaper per tick, overhead-compensated to the wire	`tc -s -j` every 250 ms tick	Raw throughput signal
Ĝ	Median of G_gw over the last 1 s (4 ticks)	Derived	Target of the fast cut (C1)
load	G_gw / S over the last 1 s	Derived	Gates exploration, fallback and attribution
μ, σ	EWMA tracking estimate of capacity and its dispersion, from flagged samples only	Estimator	Display, logging, capacity memory
D_dir	Delay signal: median over healthy far reflectors of their latest delta (age ≤ 1 s)	Prober	Detectors and diagnosis
Q_own	Own-queue delay: CAKE per-tin `av_delay` / `pk_delay`, and backlog / S	`tc -s -j`	Tells whether the queue is ours

Two further quantities appear only in the evaluation. **C** is the true access capacity, known only to the ISP emulator in the lab. **Receiver goodput** is application payload measured by iperf3 or the harness; it is lower than G_gw because of headers and retransmissions, and the controller never sees it (docs/09 A.3).

Two short reminders: **G_gw is not C** (one 8 Mbit/s stream gives G_gw ≈ 8 Mbit/s on a 100 Mbit/s link), and **Ĝ is not μ** (a cut uses the raw 1 s median Ĝ because it must reflect the link *now*; the slow EWMA μ would lag by many seconds).

> **Implementation tip.** Compute G_gw from the CAKE `bytes` counter delta. For overhead compensation, add `overhead × Δpackets` using the configured link-layer preset, so G_gw is comparable with the ISP's wire rate. Counter wrap and a qdisc re-creation (counters reset to zero) must both produce a "no sample" for that tick, never a huge negative or positive spike.

10.1.1 Censoring: what the router can and cannot learn

The central idea of the estimator is **censoring**. A router only sees the traffic that the household offers. It can learn about capacity only when demand is high enough to find the limit, and even then what it learns depends on *which* limit was hit.

Regime	What the router sees	What it learns about C	Sample flag
Application-limited	load well below 1; Ĝ follows demand; no delay rise	Nothing above Ĝ	none

Regime	What the router sees	What it learns about C	Sample flag
Shaper-limited	load ≥ 0.95; our own queue (Q_own) building; external delay D flat	Only a lower bound: C ≥ S (the link carried everything we let through)	lower bound
Capacity-limited	$\hat{G}$ well below S while demand exists; Q_own near zero; D rising	C ≈ $\hat{G}$ (the bottleneck is beyond our shaper and is draining at its own rate)	capacity

Worked examples (illustrative):

1. **Evening video only.** S = 95 Mbit/s, one 4K stream at 25 Mbit/s, load ≈ 0.26. The router learns nothing; the capacity could be 30 or 300 Mbit/s.
2. **Big download, healthy link.** Load = 0.99, CAKE's Best Effort tin shows `av_delay` around 4 ms, reflectors flat. The queue is ours, so the router learns only "C is at least about 95 Mbit/s".
3. **Capacity drop.** The FWA cell gets busy and the link falls to 20 Mbit/s. TCP slows, $\hat{G}$ falls to about 20 Mbit/s while S is still 95, our queue is empty, and the far reflectors show delay rising as the ISP-side buffer fills. Now $\hat{G}$ is a genuine capacity sample.

Only case 3 justifies a cut. Case 1 must never be used to "learn" that capacity is 25 Mbit/s, which is why the estimator flags samples.

10.2 Reflectors and probing

Delay is measured by sending small ICMP probes from the router to two tiers of targets: one **near** target just beyond the access link, and several **far** reflectors on the internet.

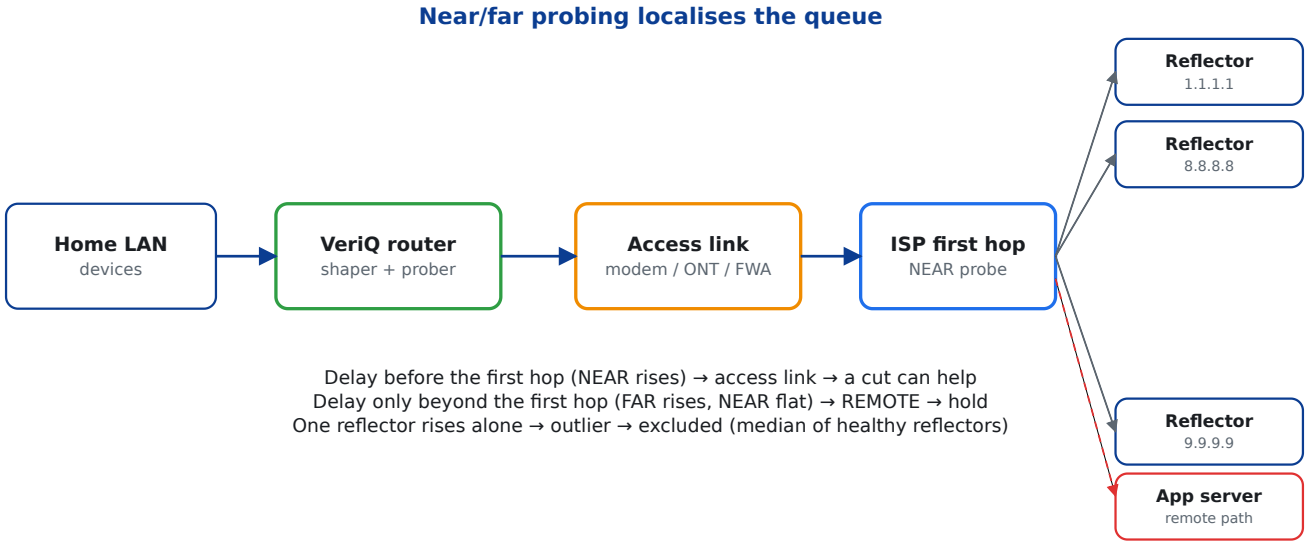


Figure 6. Near and far probe topology: the home router probes the ISP first hop (near) and several public reflectors (far); a far delay contains the access link, the near hop and everything beyond.

The two tiers are **overlapping observations, not additive segments**. A far round trip already contains the access link and the near hop. VeriQ never subtracts or adds segment delays (percentiles do not add anyway). It compares the *pattern of change* of each probe against its own baseline: near and far rising together points at the access link; far rising alone points beyond it.

10.2.1 The reflector pool

- **Far pool:** at least 8 candidates across at least 3 operators. The IPv4 defaults are 1.1.1.1, 1.0.0.1, 8.8.8.8, 8.8.4.4, 9.9.9.9, 149.112.112.112, 208.67.222.222 and 208.67.220.220, plus IPv6 equivalents when the WAN has IPv6. Users may add their own.
- **Active set:** 5 far reflectors at a time.
- **Near target:** the WAN default gateway (the PPPoE peer on PPPoE), re-resolved on every link-up because it can change after a reconnect.

All far reflectors sit behind the same access link, so for diagnosis they count as **one** evidence source (FAR). Five reflectors make that source *robust*; they do not make it five sources.

10.2.2 Qualification

At start and every 6 h, each candidate receives 10 ICMP echo and 10 ICMP timestamp probes. It is kept if it answers at least 90% of them and its idle RTT standard deviation is below 5 ms. Timestamp-capable reflectors are preferred because they give one-way deltas (next section). If fewer than two timestamp-capable reflectors qualify, VeriQ runs in RTT-only mode with load-based direction attribution (open decision O4).

10.2.3 Probe rate and idle mode

Mode	Total rate	Split	When
Normal	10 probes/s	1.6/s to each of 5 far reflectors (8/s), 2/s to the near target	Default
Idle	2 probes/s	Round-robin across the same set	Load < 10% in both directions for 30 s
Resume	10 probes/s	As normal	First tick with load ≥ 10%
Paused	0	—	HOLD_LINKDOWN

The probe order is randomised. The cost is tiny: with probes and replies under about 100 bytes each, 10 probes/s is roughly 16 kbit/s across both directions (illustrative). The cap is also etiquette: a reflector that rate-limits ICMP is rotated out, not probed harder.

10.2.4 ICMP timestamps and one-way deltas

An RTT rise cannot say which direction is congested. ICMP Timestamp messages (RFC 792 [R16]) can. The reply carries three fields, each in milliseconds since midnight UTC:

- `originate`: our send time (our clock);
- `receive`: when the reflector received the request (its clock);
- `transmit`: when the reflector sent the reply (its clock).

The router also notes `t_local_recv`, the arrival time of the reply on its own clock. Then:

```
up_raw   = receive       - originate    = OWD_up   + (clock_reflector - clock_router)
down_raw = t_local_recv  - transmit     = OWD_down + (clock_router - clock_reflector)
```

Each raw value contains the true one-way delay plus an **unknown clock offset** between the two machines. That offset can be tens or thousands of milliseconds, so `up_raw` alone is meaningless. But the offset is (nearly) **constant** over minutes. VeriQ therefore never uses the raw value; it uses the **change** against that reflector's own baseline:

```
delta_up   = up_raw   - base_up      # offset cancels: it is inside both terms
delta_down = down_raw - base_down
```

Worked example (illustrative). The reflector's clock runs 2,340 ms ahead of the router's. The true idle upload OWD is 6 ms, so `up_raw` reads 2,346 ms and the baseline settles at 2,346 ms. A cloud backup then fills the upload queue at the ISP and the true OWD rises to 46 ms. `up_raw` reads 2,386 ms, so `delta_up` = 40 ms. The 2,340 ms offset never appears in the answer.

Practical details: the resolution is 1 ms, enough for a 15 ms threshold; the fields wrap at midnight (86,400,000 ms), so differences are taken modulo that value; slow clock drift is absorbed by the windowed-minimum baseline; and reflectors that answer with zeros or a non-standard-time flag are marked "echo only" at qualification.

10.2.5 Baselines: the windowed minimum (C8)

A delta needs a baseline that represents "this path with no queue". VeriQ uses, per reflector and per direction:

> `base_i,dir` = the **minimum over a sliding 10-minute window of 1-second medians**, taken only from samples where load < 30% in **both** directions.

Each part has a reason. **1-second medians** remove single-probe spikes. The **minimum** is used because queuing delay is never negative, so the smallest recent value best estimates the propagation floor. The **sliding window** lets old values expire, so the baseline can rise after a route change. **Low-load samples only** stop a long download from inflating the floor.

Staleness rules:

- with no low-load sample for 10 min, the previous baseline is kept and flagged `stale`;
- if it stays stale for more than 30 min, that reflector's evidence is capped at L1.

Why a lower-only baseline fails: a route-change example. QoSmate's autorate (read) averages the first 10 latency samples into a baseline. After that it updates the baseline only when both directions are below 50% load, with a 9:1 EWMA, and only if the new value is *lower*. That is a sensible guard against inflating the baseline under load. But consider a permanent route change (adverse condition A7):

1. Idle RTT is 20 ms; the baseline is 20 ms.
2. The ISP reroutes and the idle RTT becomes 35 ms, permanently.
3. A lower-only baseline stays at 20 ms forever, so every sample shows a 15 ms "delay rise" even on an idle line.
4. With QoSmate's defaults (read: decrease threshold 10 ms, refractory 1 s, factor ×0.85, floor 25% of base), the decrease branch would fire about once per second. Starting from 95 Mbit/s, 0.85^n reaches the 25% floor after about 9 cuts, so within roughly 10 s the rate would sit at about 24 Mbit/s and stay there. This is a **hypothesis** derived from the read code, to be tested in A7.

Under VeriQ's rule, the 20 ms values leave the 10-minute window, and the minimum rises to 35 ms once enough low-load samples arrive. During re-learning, the far signal shows a rise. If the near hop is flat (the reroute is beyond the first hop), the diagnosis is REMOTE_LIKELY and VeriQ holds. If the evidence is weaker, the worst case is the bounded fallback: at most 10% of the anchor per rolling 10 minutes. There is no stuck-at-minimum state.

> **Implementation tip.** Keep the 600 one-second medians per reflector and direction in a monotonic deque (values increasing from front to back). Insertion and expiry are O(1) amortised and the minimum is the front element. Replay tests must cover a +15 ms step, a −15 ms step and a 40-minute high-load period (stale → L1 cap).

10.2.6 Reflector health and outlier detection

A reflector is marked **unhealthy** if any of these holds:

- loss above 20% over its last 20 probes;
- no reply for 3 s;
- **outlier:** its delta deviates from the median of the others by more than 3 × MAD for more than 10 s while the others are stable.

MAD is the median absolute deviation: take the median of the other reflectors' deltas, then the median of their absolute distances from it. It is a robust spread measure that one wild value cannot inflate. An unhealthy reflector is excluded at once, replaced from the pool, and cooled off for 10 min before it can be re-qualified.

Worked outlier check (illustrative). Four healthy reflectors show deltas of 1, 2, 2 and 4 ms. Their median is 2 ms; the absolute distances are 1, 0, 0 and 2, so MAD = 0.5 ms and the outlier limit is 2 + 3 × 0.5 = 3.5 ms away from the median. A floor on MAD (for example 1 ms) would stop a very quiet set from flagging normal jitter; this floor is an implementation suggestion, not part of docs/11, and must be tuned in S7. A fifth reflector that reads 45 ms for more than 10 s, while the other four stay put, is flagged and rotated out.

10.2.7 Aggregation: median and quorum

The per-direction delay signal is:

- `D_dir` = the **median** of the latest deltas of healthy far reflectors, each no older than 1 s;
- a **quorum of at least 2** healthy reflectors is required; otherwise the delay signal is MISSING, which means evidence level L0 for delay;
- **never a mean.**

When one-way deltas are unavailable, `D_rtt` is used in the same way, and direction is attributed by load (§ Direction attribution below).

Worked example: one lying reflector (adverse condition A5, scenario S7). Five reflectors, idle link, true queueing delay zero. One reflector starts deprioritising ICMP and its replies come back 120 ms late.

Reflector	Delta (ms)
R1	1
R2	2
R3	1
R4	3
R5 (lying)	120
Mean	**25.4**
Median	**2**

With the detection threshold θ_bb = 15 ms, the mean says "bufferbloat" and the median says "nothing is happening". The median ignores the liar from the first probe, before the outlier rule has even fired; the outlier rule then removes it after 10 s.

QoSmate's autorate (read) pings each configured reflector once per measurement (defaults 1.1.1.1, 8.8.8.8 and 9.9.9.9) and averages the RTTs of those that reply. With three reflectors at 1, 2 and 120 ms, the mean is 41 ms, well above its 10 ms decrease threshold. Whether this causes cuts in practice is a **hypothesis** that S7 tests; the averaging itself is read from the source.

> **Note.** A median of two values *is* their mean: with exactly two healthy reflectors, one bad reflector moves D by half its error, and the outlier rule cannot tell which one is lying. Hence an active set of 5, and a `healthy` count in every decision-log line so a reviewer can see when a decision rested on a thin quorum.

10.3 Detectors

Detectors turn raw signals into boolean facts, evaluated every 250 ms tick per direction. Diagnosis (next chapter) combines them.

Detector	Rule	Starting values
Bufferbloat beyond our shaper	`D_dir > θ_bb` in at least 3 of the last 6 ticks **and** `Q_own < 5 ms`	θ_bb = 15 ms
Own queue	`Q_own` (BE or BK tin) > 5 ms and load ≥ 0.95	CAKE's own target is 5 ms
Throughput censoring	$\hat{G} \le 0.80 \cdot S$ with demand present, `Q_own < 2 ms`, and a drop of at least 20% within 2 s	—
Near rise	Near delta > 10 ms in at least 3 of the last 6 ticks	—
CPU-bound	softirq on the shaping CPU ≥ 90% (from `/proc/stat`), $\hat{G} < 0.9 \cdot S$, and no D rise	—
Link down	netifd `ifdown`, or carrier lost on the WAN device	Immediate

Notes:

- **Bufferbloat.** "3 of the last 6" (1.5 s) filters single spikes yet reacts within about 0.75 s. `Q_own < 5 ms` matters: if our own CAKE queue is building, the delay is ours and CAKE already manages it.
- **Own queue** is the healthy state of a shaped link under load: no reason to act, but a source of lower-bound samples.
- **Throughput censoring.** "Demand present" means at least one bulk-class flow, or at least three active TCP flows with rates steady within ±10% for 2 s (cwnd-limited). The "20% drop within 2 s" part separates a sudden capacity loss from a slow change in usage.
- **CPU-bound** stops a router that cannot shape at line rate from reading its own shortfall as a capacity drop (C13, A17).

10.4 Direction attribution (C9)

Each direction has its own shaper and controller, and a cut in the wrong direction wastes capacity without helping.

With one-way deltas (enough timestamp-capable reflectors): use `D_up` and `D_down` directly. A rise in `delta_up` is an upload problem; a rise in `delta_down` is a download problem.

With RTT only:

1. If exactly one direction has load ≥ 60%, attribute the rise to that direction.
2. If both do, treat both as candidates, but cap the evidence per direction at **L1**, so only the bounded fallback can apply.
3. If neither does, it is not our access link: no cut, and the case goes to REMOTE_LIKELY or UNKNOWN.

Worked example (adverse condition A9, scenario S4; illustrative). A laptop starts a cloud backup at 18 Mbit/s on a 100/20 Mbit/s line while a family member streams video at 15 Mbit/s. Upload load is 0.95; download load is 0.16. RTT rises 60 ms.

- With timestamps, `delta_up` = +58 ms and `delta_down` = +2 ms: only the upload controller is involved.
- With RTT only, rule 1 applies: only upload has load ≥ 60%.
- Either way, the download rate stays where it was (pass criterion of A9: download S unchanged within ±5%).

QoSmate (read) feeds the same averaged latency into both directions, but it also has a guard: it does not reduce a direction below 50% load while the other is at 75% or more. In this example the guard would protect the download too. Differences can appear only where the guard does not apply (both directions moderately loaded, or neither loaded); whether they matter is a **hypothesis** for A9 and H1.

10.5 The rate controller

The controller runs once per tick, per direction, inside the single control goroutine. It has four persistent states and two transient actions.

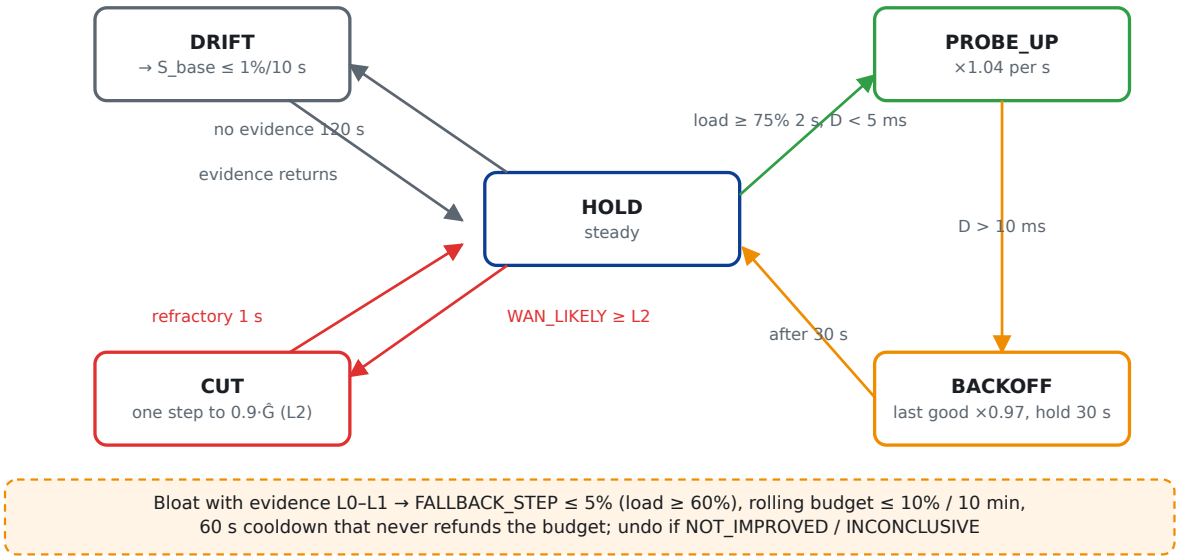


Figure 7. Rate controller states: HOLD, PROBE_UP, BACKOFF and DRIFT, with the transient CUT and FALLBACK_STEP actions.

State or action	Meaning	Leaves when
HOLD	Keep S; the normal resting state	Any rule below fires
PROBE_UP	Raising S ×1.04 per step to find spare capacity	Delay rises (→ BACKOFF), conditions fail (→ HOLD), or S_max is reached
BACKOFF	Returned to the last good S × 0.97 after a failed probe	30 s hold, then HOLD
DRIFT	No confirming evidence for 120 s: move toward S_base by at most 1% per 10 s	Any evidence arrives
CUT (transient)	One-step evidence cut (C1)	Immediately back to HOLD with a 1 s refractory period
FALLBACK_STEP (transient)	A bounded ≤ 5% step under uncertain evidence	Receipt after 10 s decides keep or undo

The global operating states (INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE, DISABLED) sit above this machine; they are specified in the Safety chapter.

10.5.1 The control loop, line by line

This is the pseudocode from docs/11 §6.5, unchanged:

```
every tick (250 ms), per direction d:
  obs  = snapshot()                       # each signal carries its timestamp
  diag = diagnose(obs, d)                 # §7 → (label, level, sources)
  act  = NONE
  if state == HOLD_LINKDOWN: continue
  if diag.label == WAN_LIKELY and diag.level >= L2:
      if fresh(obs.G_hat, 1 s):  target = max(S_min, 0.90 * obs.G_hat)          # C1
      else:                      target = max(S_min, 0.75 * S)
      if diag.level == L3:       target = max(S_min, 0.90 * C_measured)
      if target < S: act = RATE_CUT(target); episode.l2_cuts += 1
      if episode.l2_cuts > 3 and not improved(D): diag = MIXED   # no spiral
  elif bloat(d) and diag.level <= L1:            # uncertain but bloated
      act = FALLBACK_STEP(max(S_min, S - 0.05 * anchor)) if load >= 0.60
  elif explore_ok(d):                            # docs/09 A.4
      act = RATE_RAISE(min(S_max, 1.04 * S))     # 1 s refractory
  elif stale(120 s):
      act = DRIFT toward S_base, <= 1 % per 10 s
  if act: verdict = CHECKER(act, obs, versions, budgets, state)   # §8.1
          if verdict == ACCEPT and state == LIVE: enforce(act); readback()
          log(act, verdict, diag, obs)            # always, also in SHADOW
```

Reading it line by line:

1. **`snapshot()`**: one immutable `Snapshot` per tick; every signal carries its monotonic timestamp and an OK flag. Nothing later in the tick reads the kernel again, which makes deterministic replay possible.

2. `diagnose()`: returns a label, an evidence level L0–L3 and the sources used (for example `["FAR","THR"]`). This is the only input that can authorise a cut.
3. `act = NONE`: the default is to do nothing. "Hold" is not an action; it is the absence of one.
4. `HOLD_LINKDOWN`: when the WAN is down, the controller does not run; probe losses during an outage are not evidence.
5. **WAN_LIKELY at L2+**: the only path to a large cut; at least two independent sources point at the access link.
6. **Fresh Ĝ**: the C1 one-step cut to 90% of Ĝ, never below S_min (next section).
7. **No fresh Ĝ**: fall back to one ×0.75 step (the v4.1 cap).
8. **L3**: an active capacity test (UDPST to a cooperating server) agrees, so use its value with a 10% margin.
9. `target < S`: a "cut" that would raise the rate is dropped; raises happen only through exploration. Each cut increments the episode's L2 counter.
10. **No-spiral rule**: more than three L2 cuts in one episode without the delay improving downgrades the diagnosis to MIXED.
11. **Bloat at L0–L1**: a bounded fallback step of 5% of the episode anchor, only when this direction carries load ≥ 0.60.
12. `explore_ok`: the docs/09 A.4 preconditions; propose ×1.04, capped at S_max.
13. `stale(120 s)`: no confirming evidence for two minutes; drift back toward S_base.
14. **REMOTE, WIFI and CPU_LIMITED fall through every branch**, so `act` stays NONE and S is held; the UI explains why.
15. `CHECKER(...)`: every proposal goes to the pure-function Action Checker with the snapshot, versions, budgets and state.
16. `enforce` **only in LIVE**: in SHADOW the action is checked but not applied.
17. `log(...)` **always**: accepted, refused and shadow actions all produce a decision-log line.

10.6 The fast cut (C1): one step to 0.90·Ĝ

When diagnosis reaches **WAN_LIKELY at L2** with a fresh Ĝ, VeriQ cuts in **one step**:

```
S_new = max(S_min, 0.90 · Ĝ)       # Ĝ = median gateway throughput over the last 1 s
```

During a genuine capacity drop, with demand present and our queue empty, the bottleneck beyond our shaper drains at its own rate, and Ĝ *is* that rate. Setting S 10% below it moves the queue back into CAKE. The ×0.75 cap applies only when Ĝ is not fresh; at L3 the target is `max(S_min, 0.90 · C_measured)`.

The Action Checker enforces a lower limit on how far the cut may go: at L2 with fresh Ĝ the target must be at least `max(S_min, 0.85·Ĝ)`; without Ĝ at least `0.75·S`; at L3 at least `0.85·C_measured`. A deeper target is refused with `CUT_BELOW_EVIDENCE`. The checker therefore limits the cut from both sides: the diagnosis must permit it, and it may not go further than the evidence supports.

> **Note.** Ĝ is only meaningful when demand is present: an idle Ĝ is an application-limited sample and says nothing about capacity. This handbook's reading of the censoring rule and of adverse condition A29 ("zero cuts at load < 10%") is that `fresh(Ĝ)` must also require demand, so the cut path is closed on an idle link. Implement it inside `fresh()` and cover it with a unit test.

10.6.1 Worked example: 100 → 20 Mbit/s (illustrative)

This is scenario S2 and adverse condition A1. The plan is 100 Mbit/s down, S_base = 95 Mbit/s, so S_min = 9.5 Mbit/s and S_max = 99.75 Mbit/s. A call and a large download are running. At t_0 the ISP emulator drops capacity to 20 Mbit/s.

Time (illustrative)	What happens	S (Mbit/s)
t_0	Capacity falls to 20 Mbit/s; the ISP-side buffer starts to fill	95
t_0 + ~0.75 s	Far delta exceeds 15 ms in 3 of 6 ticks: bufferbloat detector true (FAR)	95
t_0 + ~1 s	Ĝ (1 s median) reads about 20.1 Mbit/s with demand present and Q_own < 2 ms: censoring detector true (THR)	95
next tick	Diagnosis WAN_LIKELY, L2, sources FAR + THR; proposed target max(9.5, 0.90 × 20.1) = 18.09	95
same tick	Checker: 18.09 ≥ max(9.5, 0.85 × 20.1 = 17.09); within [S_min, S_max]; snapshot fresh; versions match → ACCEPT	18.09
+ 10 s	Receipt: delay fell; outcome IMPROVED	18.09

The decision-log example in docs/11 §14.3 shows exactly this action (`RATE_CUT`, target 18090 kbit/s).

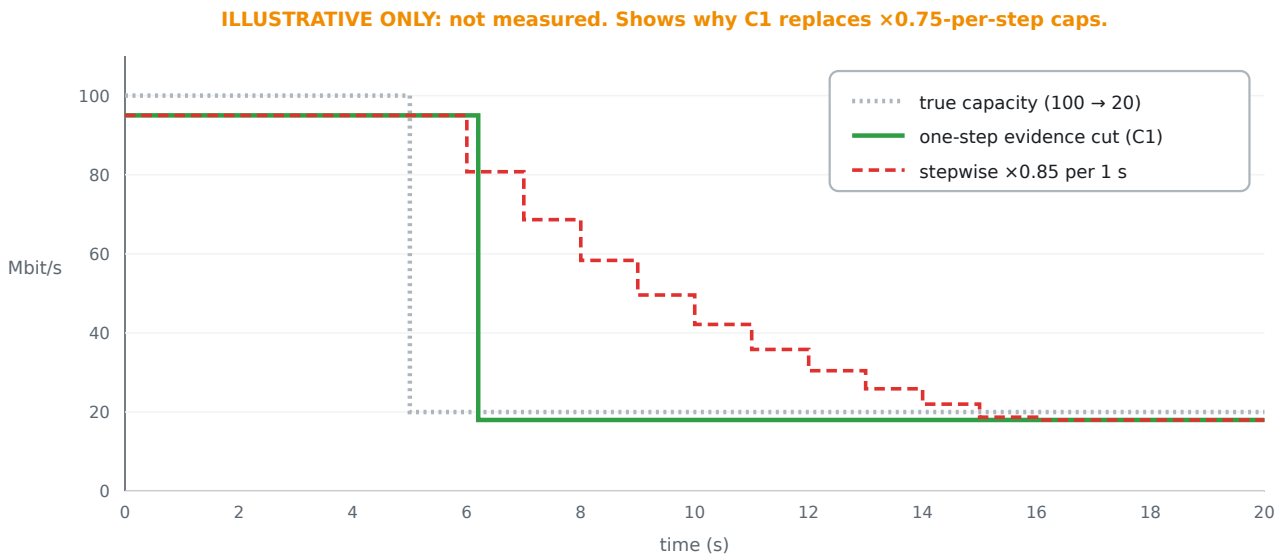


Figure 8. Illustrative S(t) for a 100 to 20 Mbit/s drop: a one-step evidence cut compared with repeated ×0.85 cuts (illustrative, not measured).

Compare the step counts (pure arithmetic, illustrative). To come within ±10% of 20 Mbit/s (S ≤ 22) from 95:

- **×0.85 per step:** $0.85^n \le 22/95$ needs n = 9 steps; at one step per second that is about 9 s after detection. With a floor of 25% of base (QoSmate's default, read), the rate cannot go below 23.75 Mbit/s, which is above the new capacity, so the queue would stay at the ISP (hypothesis for A1).
- **×0.75 per step (the v4.1 contract):** n = 6 steps, about 5–6 s. This is why C1 was introduced: it would likely fail the co-primary margin (confirmation ≤ cake-autorate + 1 s).
- **C1:** one step, about 1 s after onset.

10.6.2 Over-cut risk and its bounds

A one-step cut is only as good as $\hat{G}$. If TCP has just collapsed after a loss burst, $\hat{G}$ may read 15 Mbit/s on a 20 Mbit/s link, and the cut lands at 13.5 Mbit/s. That error is bounded: $\hat{G}$ is a 4-tick median, S_min and the $0.85 \cdot \hat{G}$ checker limit apply, and exploration recovers 13.5 → 18 Mbit/s in about 8 steps of ×1.04 after the 10 s quiet period, roughly 18 s in all (illustrative).

The revert condition for C1 in docs/12 is explicit: if false L2 cuts in the H1 control-futile runs exceed the comparator, C1 is reconsidered.

10.6.3 Refractory periods and the no-spiral rule

Timer	Value	Purpose
After a cut	1 s	Let the new rate take effect before judging again
Between raises	1 s	Each ×1.04 step must be seen before the next
After BACKOFF	30 s hold	Do not re-probe a ceiling just found
Fallback episode cooldown	60 s	Delays the next episode only; never refunds the budget

The **no-spiral rule** guards against a subtle failure. After a cut, our shaper is the limit, so $\hat{G} \le S_{new}$. If the delay rise had some other cause (for example a probe-path disturbance) and kept being read as WAN_LIKELY, each further cut would be 0.90 of an $\hat{G}$ that our own previous cut had lowered: 0.9, 0.81, 0.73 and so on, a ratchet toward S_min. The censoring detector already resists this, since after a cut the link is shaper-limited and $\hat{G}$ stops falling. The no-spiral rule is the backstop: more than three L2 cuts in one episode without the delay improving turns the diagnosis into MIXED, and from then on only the bounded fallback is allowed.

10.7 Upward exploration, BACKOFF and DRIFT

Upward exploration follows docs/09 A.4. **PROBE_UP** is entered only when all of these hold:

- load ≥ 75% of S for at least 2 s (enough demand to find out);
- the reflector-median delta is below 5 ms (no queue is building beyond us);
- no fallback episode is active;
- no diagnosis at L1 or above in the last 10 s.

In PROBE_UP, S rises ×1.04 per step with a 1 s refractory period, up to S_max. If the delta rises above 10 ms during PROBE_UP, the controller goes to **BACKOFF**: back to the last good S × 0.97, then a 30 s hold. Example (illustrative): probing from 90 to 93.6 Mbit/s, the delta reaches 12 ms, so S returns to 90 × 0.97 = 87.3 Mbit/s and stays there for 30 s.

Recovery example (A3; illustrative). Capacity returns from 20 to 100 Mbit/s while S = 18 Mbit/s. After the 10 s quiet period, ×1.04 per second reaches 95 Mbit/s in about 43 steps, roughly 53 s in total. QoSmate's defaults (read: ×1.02 per 3 s) need about 70 steps from its 23.75 Mbit/s floor, about 210 s. Faster reclaim is the A3 hypothesis; 53 s is still slow for an LTE cell, so the raise factor is a key tuning parameter.

DRIFT. With no confirming evidence for 120 s, S moves toward S_base by at most 1% per 10 s while σ widens; from 80 to 95 Mbit/s takes roughly 160 s (illustrative). It is a slow return to the nominal rate, not an exploration.

10.8 Bounded fallback episodes

When bufferbloat is present but diagnosis is uncertain (L0–L1, MIXED or UNKNOWN), VeriQ takes small, reversible steps. This rule is a hypothesis tested in S13.

- **Episode start:** the bufferbloat detector fires, the diagnosis is at most L1, and the direction has load ≥ 0.60. The current S is recorded as the **anchor S_o**.
- **Steps:** each step is at most 5% of S_o, followed by a refractory period.
- **Budget:** one global budget of at most **10% of the anchor per rolling 10 minutes**. Episodes cannot overlap, and all of them draw from the same budget. The budget state is persisted in `/etc/veriq/state/budget.json`, so a restart does not refund it either.
- **Cooldown:** a new episode may start only 60 s after the previous one ended. The cooldown never resets or refunds the budget.
- **Episode end:** bufferbloat clears for 30 s, a confident diagnosis appears, 5 min pass, or the regime changes.
- **Assessment:** a receipt 10 s after each step. IMPROVED keeps the step; NOT_IMPROVED or INCONCLUSIVE undoes it **after re-checking current conditions** (restore the pre-step rate if nothing changed, otherwise re-derive the rate from the current estimate).
- **Throughput guard:** while demand exists, aggregate throughput must not fall by more than the step size plus 2 percentage points, or the step is reverted.

10.8.1 Worked example: the cooldown does not refund the budget

Anchor S_o = 80 Mbit/s, so each step is 4 Mbit/s and the rolling budget is 8 Mbit/s (illustrative). This handbook's implementation reading, chosen as the conservative option: the budget counts every reduction *applied* inside the window, and an undo does not give it back.

Time (mm:ss)	Event	S (Mbit/s)	Budget used in window	Checker
00:00	Episode 1 starts (bloat, L1, load 0.85); step 1: 80 → 76	76	4 of 8	ACCEPT
00:10	Receipt IMPROVED: keep	76	4 of 8	—
00:12	Step 2: 76 → 72	72	8 of 8	ACCEPT
00:22	Receipt NOT_IMPROVED: conditions unchanged, so undo to 76	76	8 of 8	ACCEPT (revert)
00:52	Bloat clear for 30 s: episode 1 ends; cooldown starts	76	8 of 8	—
01:52	Cooldown over; episode 2 starts and proposes 76 → 72	76	8 of 8	REFUSED: `BUDGET_EXHAUSTED`
01:52	VeriQ raises an alert and holds	76	8 of 8	—
10:00	Step 1 leaves the 10-minute window	76	4 of 8	—
10:12	Step 2 leaves the window	76	0 of 8	—

Without this rule a controller could declare a fresh episode every 60 s and ratchet down 10% each time. The sequence is an Action Checker unit test (docs/09 A.1).

10.9 The capacity estimate μ ± σ

Every 1 s, a sample x = Ĝ is flagged:

- **capacity** (C ≈ Ĝ): taken during WAN_LIKELY at L2 or above;
- **lower bound** (C ≥ Ĝ): taken when shaper-limited (load ≥ 0.95, Q_own building, no D rise);
- **none**: application-limited, so nothing is learned.

Updates:

```
capacity sample:  e = x - mu ;  mu = mu + 0.2*e ;  sigma = sigma + 0.2*(|e| - sigma)
lower bound:      mu = max(mu, x) ;  sigma = 0.95*sigma
no sample:        sigma grows by 1% per 10 s
```

Implementation tip. Compute the residual `e` with μ *before* the update and use it in both lines. Write this choice into the unit test, because the two possible orders give different σ.

Worked example (illustrative). μ = 90, σ = 5 Mbit/s, then capacity samples of 20 Mbit/s arrive after a drop:

Sample	μ (Mbit/s)	σ (Mbit/s)
Start	90.0	5.0
1st: x = 20	76.0	18.0
2nd: x = 20	64.8	25.6
3rd: x = 20	55.8	29.4

μ moves slowly and σ widens while it is wrong, which is why **μ and σ never trigger a cut on their own**. μ ± σ is a tracking estimate for display and logging, not a calibrated confidence interval.

10.10 Capacity memory (time-of-week prior)

Many links follow a weekly rhythm, so VeriQ keeps 168 hourly bins (hour of week). Each bin holds the median of that day's **capacity** samples, kept for 4 weeks, in `/etc/veriq/state/capacity.json` (at most one write per hour, at most 10 KB).

It has exactly three uses:

1. **Start rate** after boot or link-up: `min(S_lkg, S_base)`, where S_lkg is the last known good rate, or `min(S_base, p20 of the bin)` once the bin holds at least 3 weeks of data. Example (illustrative): a reboot on Friday at 21:10, whose bin's 20th percentile is 55 Mbit/s, starts at min(95, 55) = 55 Mbit/s rather than at 95 and a burst of bloat (A16).
2. **A UI statement**, such as "Evenings are usually ~60 Mbit/s".
3. **Nothing else.** The prior never causes a cut by itself.

10.11 Link events

Event	Action
WAN down (netifd or carrier)	HOLD_LINKDOWN: freeze rates, pause probes, do not count losses as evidence
WAN up, PPPoE reconnect or IP change	Re-resolve the near target, re-qualify reflectors in the background, resume at the start rate, and return to LIVE (or the prior state) after 10 s of healthy probes
Interface renamed or missing	DEGRADED and an alert; keep the last kernel config
NTP clock step	No effect: control timing uses CLOCK_MONOTONIC; wall time only for logs and lease restore

The rule is that **an outage is not congestion**. QoSmate (read) returns 9999 ms from its latency function when no reflector answers; its main loop then substitutes the last good latency, if one exists, and logs a warning every fifth failed cycle. Rates hold if that value was within thresholds; if it was elevated, the decrease branch could keep firing (hypothesis for A6 and A14). In VeriQ the controller simply does not run during HOLD_LINKDOWN.

10.12 Comparison with a delay-only autorate

Aspect	QoSmate autorate (read, v0.5.48)	VeriQ (design, to be tested)
Delay signal	Mean RTT of the reflectors that reply; one shared value for both directions	Median of healthy reflectors' deltas, quorum ≥ 2; one-way deltas where available
Baseline	Average of the first 10 samples, then lower-only EWMA at low load	10-min windowed minimum of low-load 1-s medians; stale flag
Decrease	Delta > 10 ms → ×0.85, 1 s refractory, no load check in this branch	Only with diagnosis: one step to 0.90·Ĝ at L2; bounded 5% steps at L0–L1; hold for remote, Wi-Fi, CPU
Increase	Load ≥ 75% and delta < 5 ms → ×1.02, 3 s refractory	Load ≥ 75% for 2 s, delta < 5 ms, no recent diagnosis → ×1.04, 1 s refractory; BACKOFF
Envelope	min/base/max = 25/100/105%	S_min = max(1 Mbit/s, 10%), S_max = 105%

cake-autorate [R12], the co-primary comparator, uses one-way delays from ICMP timestamps where reflectors support them, rotates reflectors and gates rate increases on load (per its documentation); it is pinned and tuned in the Evaluation chapter. Both comparators are mature; in static conditions all arms sit on CAKE, and parity is the expected result.

10.13 Starting parameters

All values are starting parameters (docs/11 §6, Appendix B; docs/09 A.2–A.7), to be tuned in Stage 2 and recorded in the run manifest.

Parameter	Starting value	UCI option	Tested in
Control tick	250 ms	`controller.tick_ms`	E3, replay
S_min / S_max	max(1 Mbit/s, 0.10·S_base) / 1.05·S_base	`shaping.min_frac` , `max_frac`	S2, A1
Bufferbloat threshold θ_bb	15 ms, 3 of 6 ticks, Q_own < 5 ms	`controller.theta_bb_ms`	S2, S6
Near-rise threshold	10 ms, 3 of 6 ticks	—	S6, A19
Censoring	$\hat{G} \le 0.80 \cdot S$, drop ≥ 20% in 2 s, Q_own < 2 ms, demand present	—	S2, E5
CPU-bound	softirq ≥ 90%, $\hat{G} < 0.9 \cdot S$	—	A17
Fast-cut factor (C1)	$0.90 \cdot \hat{G}$; checker floor $0.85 \cdot \hat{G}$; ×0.75 without $\hat{G}$	`controller.cut_g_factor`	S2, A1
No-spiral	> 3 L2 cuts without improvement → MIXED	—	A2, A32
Raise	×1.04 per step, 1 s refractory	`controller.raise_factor`	A3
Explore preconditions	load ≥ 75% for 2 s; delta < 5 ms; no L≥1 for 10 s	`controller.explore_delta_ms`	A3
BACKOFF	delta > 10 ms → last good × 0.97; hold 30 s	`controller.backoff_delta_ms`	A2
DRIFT	stale 120 s; ≤ 1% per 10 s	`controller.stale_s`	A29
Fallback step / budget	5% of anchor / 10% per rolling 10 min	`fallback_step` , `fallback_budget`	S13
Fallback cooldown / episode cap	60 s (no refund) / 5 min	`fallback_cooldown_s`	S13
Fallback load gate	load ≥ 0.60	—	S13, A29
Probe rate / idle	10 probes/s / 2 probes/s (load < 10% for 30 s)	`probe.rate_pps` , `idle_rate_pps`	A6
Active far reflectors	5; pool ≥ 8 across ≥ 3 operators	`probe.active_far`	S7
Qualification	10 echo + 10 timestamp; ≥ 90% replies; idle std < 5 ms; every 6 h	—	S7, A6
Baseline	min of 1-s medians over 10 min; load < 30%; stale > 30 min → L1	—	A7
Health	loss > 20% of 20; silent 3 s; outlier > 3 × MAD for 10 s; cool-off 10 min	—	S7, A5
Quorum	≥ 2 healthy; DEGRADED after 60 s without	—	A6
Direction attribution	load ≥ 60% (RTT-only); both loaded → L1 cap	—	A9
Estimator	gain 0.2; σ × 0.95 on lower bound; +1% per 10 s idle	—	E1–E6
Capacity memory	168 bins, 4 weeks, p20 after ≥ 3 weeks	`controller.capacity_memory`	A16
Link-up settle	10 s of healthy probes	—	A14

For judges. “We do not cut because delay rose. We cut because two independent signals say the access link shrank, and then we cut once, to what the link is actually delivering. When the signals disagree we take at most 5% steps under a budget that cooldowns cannot refund, and when the delay comes from somewhere a cut cannot fix, we hold. Each of these is a design claim that the A-series scenarios test against QoSmate and cake-autorate.”

CHAPTER 11

Chapter 11 (pending)

This chapter has not been written yet.

CHAPTER 12

Safety

VeriQ changes the shaper rate of a household's internet connection several times a minute, perhaps while someone is in an exam call or a ranked match. A bad cut wastes bandwidth the family pays for. A bad raise brings bufferbloat back. A crashed controller must never take the internet down with it.

This chapter explains how VeriQ keeps its own actions within limits: the safety philosophy, the Action Checker (every rule and refusal code, with a worked example of each), the operating states, the reconciler, the watchdog and the SAFE profile, budgets and persistence, failure domains, and a safety-case summary.

> **Honesty check.** No VeriQ controller code has run yet. Everything below is a design with starting parameters (docs/11 §8, docs/09 Appendix A), and the tests named here are planned, not passed. The Action Checker *checks admissibility against the implemented rules*. It does not establish that an action is optimal or helpful, and it does not make the system "safe" in any absolute sense.

12.1 Safety philosophy

12.1.1 Four properties of every action

Principle P4 (docs/11 §2) says every action is **checked, bounded, logged and reversible**. Each word maps to a mechanism.

Property	What it means	Mechanism	Tested by
Checked	No command reaches the kernel without an ACCEPT for the same action ID and config version	Action Checker; the Enforcer refuses tokenless commands	One unit test per refusal code; replay
Bounded	Rates stay in `[S_min, S_max]`; step size depends on the evidence level; uncertain cuts draw on a finite budget	Rate envelope (docs/09 A.2), permission table (docs/11 §7.3), rolling fallback budget	Property (fuzz) tests
Logged	Every decision, including "do nothing" and every refusal, is a decision-log line; every applied or refused action has a receipt	Decision log (JSONL in a RAM ring), receipts	Replay determinism test
Reversible	Any state can return to a known static configuration without the daemon's help	SAFE profile, watchdog, persistent kernel config, condition-aware undo	Kill tests (S8, A15), S13

12.1.2 The dominance invariant

The safety design serves one promise (docs/11 §0.2):

> **Dominance invariant (design goal, tested in every scenario):** in every condition where VeriQ lacks evidence to act, it holds or reverts to a static CAKE configuration equivalent to well-tuned SQM. Its adaptive behaviour is only allowed where evidence supports it, through bounded steps. So by construction, VeriQ's worst case is "static SQM", and its best case is better.

SQM (Smart Queue Management, OpenWrt's `sqm-scripts` [R11]) here means a fixed-rate shaper using CAKE (Common Applications Kept Enhanced [R5]). "By construction" is a design argument. The evaluation tests it by comparing VeriQ with the static SQM arm in every scenario.

12.1.3 Defence in depth

Five layers fail independently; each catches what the one above misses.

```
Layer 5  procd              respawns veriqd (3600 5 5); restarts the watchdog
Layer 4  veriq-watchdog     separate process; heartbeat lost > 10 s -> SAFE rates
Layer 3  operating states   SHADOW / DEGRADED / HOLD_LINKDOWN / SAFE limit what may run
Layer 2  reconciler         desired vs observed; ordered apply; read-back; no fighting
Layer 1  Action Checker     pure function; every action ACCEPT or REJECT(code)
Layer 0  kernel guardrails  CAKE AQM, flow isolation, envelope meters: run per packet,
                            keep running if every user-space process is dead
```

Layer 0 matters most. The packet path is the kernel (principle P1), and user space only configures it. If `veriqd` stops, the last configuration keeps shaping.

12.1.4 What safety does not mean here

- **No bandwidth guarantee:** CAKE tins give priority and weights, not reservations.
- **No protection against the ISP:** if the access link fails, VeriQ detects it and holds.

- **No unconditional SAFE:** the SAFE profile assumes capacity stops falling (see below).
- **No proof of benefit:** ACCEPT means "admissible". Whether an action helped is the receipt's question, and causal claims need lab experiments.

12.2 The Action Checker

12.2.1 Position and contract

The checker sits between every proposer and the Enforcer (docs/09 A.1, docs/11 §8.1):

```
control FSM / lease API / classifier / rule editor
         |  proposed Action {id, type, dir, target, payload, basis, version}
         v
  +---------------+      REJECT(code, reason) --> decision log + receipt
  | ACTION CHECKER|
  +---------------+      ACCEPT(id, version)  --> Enforcer --> tc / nft --> read-back
```

- **Inputs:** the proposed action; the observation snapshot (each signal timestamped); the desired-state version and the last read-back kernel version; the budget state; the lease table; the sanction table; the operating state; the configured limits.
- **Output:** `ACCEPT(action_id, version)` or `REJECT(action_id, code, reason)`, both logged.
- **Purity:** no I/O and no clock reads; the tick time arrives in the snapshot. Equal inputs give equal verdicts, so the checker is table-testable and a decision log replays bit-for-bit.
- **Token rule:** the Enforcer refuses any command without an ACCEPT for the same `action_id` and version. The checker is the only door to the kernel.

Action IDs are ULIDs (Universally Unique Lexicographically Sortable Identifiers), which sort by creation time.

12.2.2 Evaluation order

When several rules fail, the order decides which code is reported. VeriQ fixes it:

1. `SAFE`, `REVERT_TO_RECHECKED` and `LEASE_REVOKE` are admissible in every state with no further checks (still logged).
2. `DUPLICATE`: an `action_id` already seen.
3. `STALE_OBS`: the signals the action depends on are older than 2 control cycles (500 ms).
4. `VERSION_MISMATCH`: desired version ≠ last read-back version.
5. The action-specific rules below.
6. `STATE_FORBIDS` **last**. In SHADOW, a controller action that passes steps 2–5 is logged with `shadow_verdict = ACCEPT`, then refused. The shadow report thus shows what LIVE *would* have done.

Placing the state check last is this handbook's reading of two requirements that must both hold: docs/09 A.1 tests "an action sent while in SHADOW" as a refusal, and docs/11 §6.5 logs the verdict "always, also in SHADOW".

12.2.3 Rule and refusal-code table

This merges docs/09 A.1 with the v5 additions of docs/11 §8.1. S is the current rate per direction; Ĝ ("G-hat") is the median gateway throughput over the last 1 s; `S_min = max(1 Mbit/s, 0.10·S_base)` and `S_max = 1.05·S_base`. All numbers are starting parameters.

Action	Rules checked (all must pass)	Refusal codes	Source
Any (except the three above)	Snapshot age ≤ 2 cycles; desired = read-back version; state permits (matrix below); `action_id` new	`STALE_OBS`, `VERSION_MISMATCH`, `STATE_FORBIDS`, `DUPLICATE`	A.1
RATE_CUT	Diagnosis WAN_LIKELY at L2 or L3 (otherwise only FALLBACK_STEP); new S ≥ S_min; L2 with fresh Ĝ: target ≥ max(S_min, 0.85·Ĝ); L2 without Ĝ: target ≥ 0.75·S; L3: target ≥ 0.85·C_measured	`STEP_EXCEEDS_LEVEL`, `BELOW_FLOOR`, `CUT_BELOW_EVIDENCE`	A.1 + §8.1 (C1)
RATE_RAISE	A.4 preconditions (load ≥ 75% for ≥ 2 s, delta < 5 ms, no fallback episode, no diagnosis ≥ L1 in 10 s); step ≤ ×1.04; 1 s since the last raise; new S ≤ S_max	`EXPLORE_PRECOND`, `ABOVE_MAX`	A.1, A.4
FALLBACK_STEP	Diagnosis L0–L1, MIXED or UNKNOWN with bloat; step ≤ 5% of the anchor; no overlapping episode; budget remainder ≥ step; 60 s cooldown elapsed; new S ≥ S_min	`EPISODE_ACTIVE`, `BUDGET_EXHAUSTED`, `COOLDOWN`, `STEP_EXCEEDS_LEVEL`, `BELOW_FLOOR`	A.1, A.7
DRIFT	Stale ≥ 120 s; toward S_base by ≤ 1% per 10 s; result in `[S_min, S_max]`	`EXPLORE_PRECOND`, `ABOVE_MAX`, `BELOW_FLOOR`	A.4 (codes: handbook mapping)

Action	Rules checked (all must pass)	Refusal codes	Source
LEASE_GRANT	Authenticated rpcd session; ≤ 2 active, ≤ 1 per device; ≤ 4 h; no capacity collapse (S < 2 Mbit/s, or WAN_LIKELY @ L2 with a > 50% drop)	`AUTH`, `ADMISSION`, `COLLAPSE`	A.1, §5.8
MARK_SET	Class ∈ 1..4; flow not sanctioned; basis is a lease, user rule, eligible app mark or singleton ML prediction, never abstention alone	`TIN_RANGE`, `SANCTIONED`, `NO_PREFERENCE_BASIS`	A.1
RULESET_APPLY	Candidate passed `nft -c`; version = desired + 1; not in SAFE unless it is the SAFE ruleset	`INVALID_RULESET`, `VERSION_MISMATCH`, `STATE_FORBIDS`	§8.1
BACKEND_SWITCH	Explicit user request only, never autonomous	`NOT_USER_INITIATED`	§8.1
SANCTION_ESCALATE	Recorded violation; ≤ ×2 per offence; result ≤ 15 min	`NO_VIOLATION`, `CAP_EXCEEDED`	§8.1
SAFE, REVERT_TO_RECHECKED, LEASE_REVOKE	Admissible in every state; logged	—	A.1, §8.1

Two notes: - **Why 0.85·Ĝ when the controller aims at 0.90·Ĝ?** The 5-point margin absorbs the change in Ĝ between proposal and check (both ≤ 2 cycles old) without letting a buggy controller cut far below the evidence. - **DRIFT codes:** A.1 gives drift no codes of its own, so this handbook reuses the closest existing ones. Record any dedicated code in docs/12.

12.2.4 State permission matrix

docs/11 §5.2 and §8.2 say what runs in each state. As checker rules they give the matrix below. “Logged” means: evaluated, would-be verdict recorded, `STATE_FORBIDS` returned. The DEGRADED, SAFE and HOLD_LINKDOWN cells for leases and rulesets resolve cases the specification leaves open; they are starting rules.

Action group	INIT	SHADOW	LIVE	DEGRADED	HOLD_LINKDOWN / SAFE
Rate actions (cut, raise, fallback, drift)	refused	logged	checked	refused	refused
ML MARK_SET (PROV = 3)	refused	logged	checked	refused	refused (earlier verdicts stay)
LEASE_GRANT	refused	checked	checked	refused	HOLD_LINKDOWN: checked; SAFE: refused
RULESET_APPLY	install only	checked	checked	only “take ownership”	SAFE ruleset only
BACKEND_SWITCH	refused	checked	checked	refused	refused
SANCTION_ESCALATE	refused	checked	checked	checked	checked

Sanction escalation stays available in DEGRADED and SAFE: the envelope protects *other* users, and a router’s trouble should not reward a misbehaving flow.

12.2.5 A worked example of every refusal

Setting: download, `S_base = 90 Mbit/s`, so `S_min = 9` and `S_max = 94.5 Mbit/s`; S = 90 Mbit/s; state LIVE unless stated. All numbers are illustrative.

Code	Situation	Why refused	Next
`DUPLICATE`	The household page times out and resends a lease request with the same `action_id`	Already decided	The first verdict stands
`STALE_OBS`	A `tc -s -j` read hangs; at tick 1003 a cut is proposed from a tick-1000 snapshot	750 ms > 2 cycles	Re-proposed next tick if the evidence holds
`VERSION_MISMATCH`	Someone runs `tc qdisc change … bandwidth 50mbit` by hand	Action builds on a state that no longer exists	Reconciler → DEGRADED
`STATE_FORBIDS`	In SHADOW, a valid L2 cut 95 → 18 Mbit/s	SHADOW applies no rate actions	Logged as a would-be cut
`STEP_EXCEEDS_LEVEL`	UNKNOWN @ L1, but a RATE_CUT to 70 Mbit/s is proposed (a bug)	At L1 only a ≤ 5% fallback step (4.5 Mbit/s)	S unchanged; bug visible in the log
`BELOW_FLOOR`	WAN_LIKELY @ L2, Ĝ = 8 Mbit/s, target 7.2	7.2 < S_min = 9	Controller should clamp to 9

Code	Situation	Why refused	Next
`CUT_BELOW_EVIDENCE`	WAN_LIKELY @ L2, fresh Ĝ = 20, target 15	Lowest admissible max(9, 17) = 17	Correct target 0.90 × 20 = 18
`EXPLORE_PRECOND`	Raise 90 → 93.6 at 60% load	Needs ≥ 75% for ≥ 2 s	No raise
`ABOVE_MAX`	At S = 92, raise to 95.7	95.7 > 94.5	Propose min(S_max, 1.04·S) = 94.5
`EPISODE_ACTIVE`	Episode open since 20:41:00; a new one proposed at 20:41:20	Episodes cannot overlap	The open episode continues
`BUDGET_EXHAUSTED`	Budget 9 Mbit/s per rolling 10 min; 4.5 debited at 20:30 and 20:33; third step at 20:36	Remainder 0	Next step possible after 20:40
`COOLDOWN`	Episode ended 20:50:00; new one at 20:50:40	40 s < 60 s	Retry after 20:51:00, still within budget
`AUTH`	Lease request with an expired rpcd session	Not authenticated	UI asks for login
`ADMISSION`	Two leases active; a third device asks	≤ 2 active	Reason and next free time shown
`COLLAPSE`	S = 1.8 Mbit/s; a new lease requested	S < 2 Mbit/s	Existing leases continue with a warning
`TIN_RANGE`	A corrupted model yields class 5	Classes are 1..4	Model integrity alert
`SANCTIONED`	Singleton {RT} for a flow sanctioned 20 s ago	Sanction (level 2) outranks classifier (level 6)	BE until the penalty expires
`NO_PREFERENCE_BASIS`	Conformal set {IV, WEB} for a QUIC flow; IV proposed	Abstention is no basis	Flow stays BE
`INVALID_RULESET`	User rule `dest_port '80-'`; `nft -c` fails	Does not compile	Previous ruleset stays; UI shows rule name and error
`VERSION_MISMATCH` (ruleset)	Two LuCI tabs save at once; the second builds on 811, desired is 812	Must be desired + 1	Second tab reloads
`NOT_USER_INITIATED`	CPU detector fires; controller proposes `cake` → `htb`	Maintenance only	UI shows a recommendation
`NO_VIOLATION`	Escalation for a flow with no recorded violation	No offence	Logged as a bug
`CAP_EXCEEDED`	Penalties 60 → 120 → 240 → 480 s; next proposed 960 s	> 900 s cap	Correct proposal: 900 s

For the precedence refusals, the governing rule is: *"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."*

12.2.6 Test list

The CI gate is 100% of refusal codes tested and ≥ 80% line coverage on `checker` (docs/11 §14.4).

- **One test per code**, 22 unique codes, with variants for `VERSION_MISMATCH` (rate, ruleset) and `STATE_FORBIDS` (SHADOW, DEGRADED, SAFE).
- **docs/09 A.1 scenarios:** stale snapshot; version mismatch after a foreign `tc` change; step too large for L1; S below the floor; budget exhausted across two episodes (cooldown must not refund); expired lease; abstained flow requesting preference without a lease; action in SHADOW; duplicate `action_id`.
- **Always admissible:** SAFE, REVERT_TO_RECHECKED and LEASE_REVOKE accepted in all seven states, even with a stale snapshot.
- **Order:** a duplicate stale action reports `DUPLICATE`; a stale SHADOW action reports `STALE_OBS`; a valid SHADOW action records `shadow_verdict = ACCEPT`.
- **Boundaries:** exactly 0.85·Ĝ accepted, 1 kbit/s lower refused; a raise to exactly S_max accepted.
- **Properties** (Go fuzzing, 10^6 cases, 0 failures): accepted rates never leave `[S_min, S_max]`; accepted fallback cuts in any 10-minute window never exceed 10% of the anchor; no ACCEPT follows an abstention-only basis.
- **Replay:** `veriqctl replay` of a recorded log reproduces every verdict.

12.2.7 What the checker does not do

It does not judge whether a cut was *wise*; that is diagnosis and the receipt. It checks only the rules it implements. And it shares a process with the controller, so a dead `veriqd` takes both down; that failure is covered by the kernel's persistent configuration and the watchdog.

12.3 Operating states

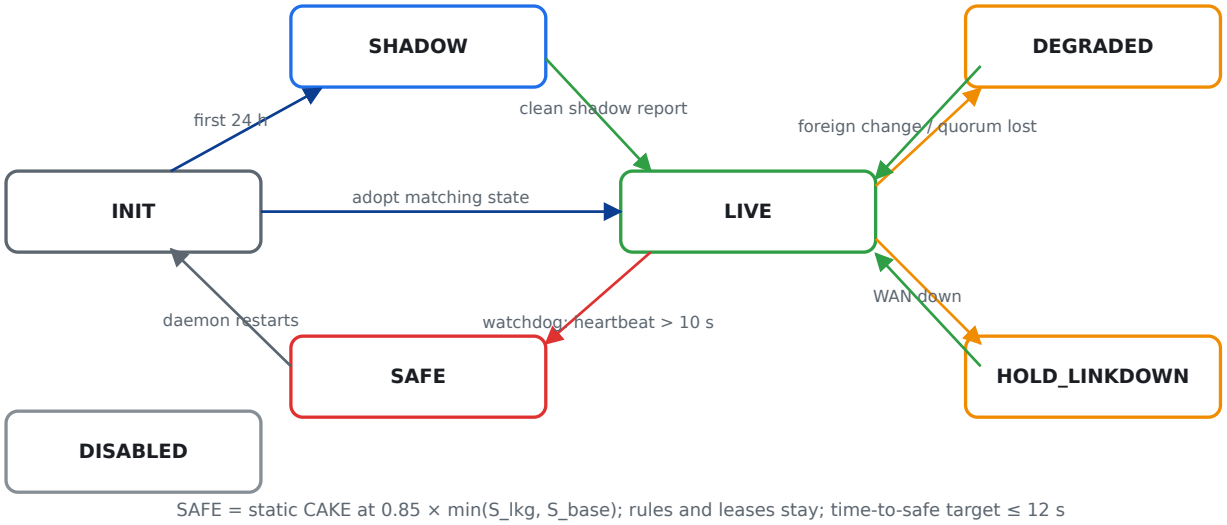


Figure 9. Operating states INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE and DISABLED, with their transitions.

12.3.1 The seven states

The states come from decision C10 (docs/11 §8.2). The product rule: **static CAKE is active from minute one**, and the adaptive controller earns trust in SHADOW before it may act.

State	Meaning	Kernel configuration	Controller
INIT	Starting; the reconciler adopts or installs	Static CAKE at the start rate	Off
SHADOW	First 24 h after install, and after a model or algorithm upgrade	Static CAKE + rules and leases	Computes and logs; applies no rate or ML actions
LIVE	Normal	Adaptive	Applies checked actions
DEGRADED	Foreign change, quorum lost > 60 s, or a component failing	Last good configuration, frozen	Holds; alerts
HOLD_LINKDOWN	WAN down	Frozen	Paused
SAFE	Watchdog or fatal error	Static CAKE at 0.85 × min(S_lkg, S_base); rules and leases kept	Off until recovery
DISABLED	User disabled	VeriQ objects removed	Off

S_lkg is the last-known-good rate: the latest rate that was read back correctly and whose receipt was not NOT_IMPROVED.

12.3.2 Transition table

From	Event (and guard)	To	Action on entry
boot or procd start	`veriqd` starts; config not disabled	INIT	Read `version.json` and the kernel state
INIT	Kernel matches the persisted version	persisted state	Adopt; restore timers, budgets, leases
INIT	Fresh install, or upgrade with a new algorithm or model	SHADOW	Install static CAKE + rules; start the 24 h timer
INIT	Kernel does not match the persisted version	SAFE → INIT → SHADOW	Install SAFE; reinstall from config (docs/09 §3.5); user may confirm LIVE at once
SHADOW	24 h elapsed and the report has no anomaly	LIVE	Notify the user
SHADOW	24 h elapsed with an anomaly	SHADOW	Ask the user to review and confirm
SHADOW	User confirms, or `veriqctl state set live`	LIVE	Log who confirmed
LIVE	`veriqctl state set shadow`	SHADOW	Freeze S
LIVE, SHADOW	Foreign change; quorum lost > 60 s; component failing; interface missing	DEGRADED	Freeze; alert banner

From	Event (and guard)	To	Action on entry
DEGRADED	Cause resolved or "take ownership", read-back matches	prior state	Log resolution
any but DISABLED	netifd `ifdown` or carrier lost	HOLD_LINKDOWN	Freeze rates; pause probes
HOLD_LINKDOWN	WAN up and 10 s of healthy probes	prior state	Re-resolve near target; capacity-memory start rate
any but DISABLED	Heartbeat > 10 s old (watchdog)	SAFE	Watchdog applies `safe.env`
any but DISABLED	Partial apply after 3 retries, or `state set safe`	SAFE	`veriqd` applies SAFE itself
SAFE	`veriqd` restarts	INIT	The reconciler decides
any / DISABLED	User disables / enables	DISABLED / INIT	Remove only owned objects / start

12.3.3 SHADOW, DEGRADED and HOLD_LINKDOWN

SHADOW runs the whole loop except the last step. Rules, leases and envelopes stay active because they are deterministic user configuration. After 24 h, `veriqd` builds a **shadow report**: would-be actions per type and label, the would-be rate trace beside the static rate, and anomalies. docs/11 §8.2 names three anomaly types: a crash, CPU over budget, and more than 5 disagreements per hour judged unsafe. With no ground truth on a product router, this handbook proposes these starting rules for "unsafe": a would-be cut at load < 10%; a would-be cut reaching S_min; more than 4 would-be reversals in a minute; more than 10 `STALE_OBS` or `VERSION_MISMATCH` refusals in an hour.

In the lab (S12), VeriQ's shadow decisions are compared with cake-autorate's real ones and judged against the injected fault. Only the lab can say who was right.

> **Note.** Shadow logs show *disagreements*, not avoided incidents. VeriQ never reports "SHADOW prevented N bad cuts".

DEGRADED means "I can no longer trust my picture of the world, so I stop changing it". The last good configuration keeps shaping. VeriQ never fights another tool for the qdisc: two controllers rewriting the same object would flap.

HOLD_LINKDOWN exists because an outage looks like a disaster to a delay-based controller: every probe is lost. VeriQ treats it as an event, not evidence. On link-up it resumes from `min(S_lkg, S_base)`, or from the 20th percentile of the capacity-memory bin once that bin has ≥ 3 weeks of data.

12.4 The reconciler

12.4.1 Desired versus observed

The reconciler makes "the configuration I think I applied" equal to "the configuration the kernel has".

- **Desired:** a versioned document (version N, a monotonic integer persisted across restarts) with the qdisc tree, rates, nft table, set elements and IFB.
- **Observed:** parsed every tick from `tc -j -s qdisc show` and `tc -j filter show` on the WAN device and `ifb-veriq`, and from `nft -j list table inet veriq`.
- **Ownership:** VeriQ owns table `inet veriq` (comment `veriq:vN`), qdiscs with handles in a reserved range, and the IFB `ifb-veriq`. Nothing else is modified; cleanup removes owned objects only.

12.4.2 Ordered apply and read-back

A `tc` batch is not atomic, so every change is ordered to keep each intermediate state a valid shaper, and each step is read back.

- **CAKE:** one command, `tc qdisc change … bandwidth X` (a single netlink message).
- **HFSC and HTB:** the root (upper-limit) class first, then the leaves, scaled proportionally, each read back. A failed step triggers a compensating step or SAFE. A unit test asserts that every prefix of the backend's `RateChange` sequence is a valid tree, for increases and decreases.
- **nft:** the table is applied with `nft -f` as one transaction, after `nft -c` passed.
- **Retries:** at most 3, capped exponential backoff; a failed apply starts no cooldown.

```
apply(change, version N+1):
  for step in backend.ordered_steps(change):
      run(step)                        # tc -batch -   or   nft -f -
      if readback() != expected(step):
          retry up to 3 (capped backoff)
          if still wrong: compensate(previous steps) or enter SAFE; return FAIL
  persist version N+1 (atomic write); return OK
```

12.4.3 Restart adoption

After a crash, upgrade or reboot, `veriqd` does not blindly reinstall, which would reset CAKE's flow state and glitch every active call. It reads the persisted version and the kernel state. If the hash of owned objects matches, it **adopts**: keeps the kernel state, restores budgets, sanctions and leases, and resumes in the persisted state. Otherwise it goes SAFE, then INIT, a fresh install and SHADOW.

12.4.4 Self-heal versus foreign change

Event	Example	Response
VeriQ's object vanished	A firewall restart flushes table `inet veriq`	**Self-heal:** re-apply within one tick; log (A31)
A foreign object replaced VeriQ's	SQM started by hand puts its qdisc on the WAN	**Do not fight:** DEGRADED; alert; offer "take ownership" (A30)

A vanished VeriQ object has no other owner, so restoring it conflicts with no one. A foreign object has an owner who presumably wants it there.

12.5 Watchdog and time-to-safe

12.5.1 Mechanism

The watchdog (C11) is a POSIX shell loop run by procd, OpenWrt's process manager, as the second instance of `/etc/init.d/veriq`. Being tiny and separate, it shares no failure mode with the Go daemon except the kernel.

- `veriqd` touches `/var/run/veriq/heartbeat` every 1 s.
- Every 2 s the watchdog checks the heartbeat's age.
- If it is > 10 s old and the state is not DISABLED, the watchdog runs `tc qdisc change` to the SAFE rates in `/var/run/veriq/safe.env`, logs to syslog, and writes `state=SAFE`.
- `veriqd` rewrites `safe.env` whenever rates change, so the watchdog computes nothing.
- procd respawns `veriqd` (`respawn 3600 5 5`); the reconciler then adopts.

An illustrative sketch, to be hardened during implementation:

```
#!/bin/sh
# veriq-watchdog: sketch only. Run by procd as instance "watchdog" of /etc/init.d/veriq.
RUN=/var/run/veriq
up_s() { cut -d. -f1 /proc/uptime; }        # monotonic seconds since boot

while :; do
    sleep 2
    st=$(cat "$RUN/state" 2>/dev/null)
    [ "$st" = "DISABLED" ] || [ "$st" = "SAFE" ] && continue
    hb=$(cat "$RUN/heartbeat" 2>/dev/null || echo 0)   # veriqd writes up_s here
    [ $(( $(up_s) - hb )) -le 10 ] && continue
    . "$RUN/safe.env"                      # WAN_DEV IFB_DEV SAFE_UP_KBIT SAFE_DOWN_KBIT
    [ "$SAFE_UP_KBIT" -gt 0 ] && \
        tc qdisc change dev "$WAN_DEV" root cake bandwidth "${SAFE_UP_KBIT}kbit"
    [ "$SAFE_DOWN_KBIT" -gt 0 ] && \
        tc qdisc change dev "$IFB_DEV" root cake bandwidth "${SAFE_DOWN_KBIT}kbit"
    echo SAFE > "$RUN/state"
    logger -t veriq-watchdog "heartbeat lost; SAFE rates applied"
done
```

> **Implementation tip.** docs/11 §8.4 checks the heartbeat file's *mtime*, which follows the wall clock. NTP may step that clock by an hour on a router without a real-time clock (A18), causing a false SAFE. The sketch writes monotonic uptime *into* the file and compares it with `/proc/uptime`, which no clock step disturbs. This is a recommended refinement; record it in docs/12 if adopted.

> **Implementation tip.** For `hfsc` and `htb`, a SAFE change is several ordered commands. Let `veriqd` pre-render them into a `tc -batch` file beside `safe.env`, so the watchdog stays backend-agnostic.

12.5.2 Time-to-safe arithmetic

The target is **time-to-safe ≤ 12 s** (docs/11 §8.4). In the worst case the daemon dies just after a heartbeat at t = 0. The age exceeds 10 s just after t = 10, and the 2 s check notices by t = 12. The `tc` commands take milliseconds. So the starting parameters meet the target by design; the *measured* value (S8, A15) is what goes into the submission. A threshold much closer to the 1 s heartbeat would trigger SAFE during brief scheduling stalls on a busy router.

During those 12 s the kernel keeps shaping at the last rate. Time-to-safe measures the return to a known static configuration, not an outage. There should be no outage.

12.5.3 Edge cases

Case	What happens	Mitigation
`veriqd` hung, not dead	procd does not respawn a live process; the watchdog applies SAFE	Proposal: watchdog then sends SIGTERM, SIGKILL after 5 s, so procd respawns
`veriqd` wakes after SAFE	Could apply a stale action	Each tick reads `state`; on SAFE it applies nothing and restarts via INIT
Watchdog dies	No effect while `veriqd` is healthy	procd respawns it; the health check reports it
`safe.env` missing or corrupt	No rates to apply	Written atomically at INIT before any rate change; watchdog logs and skips
Crash loop	procd stops respawning after 5 rapid failures	Kernel stays at SAFE; LuCI banner; support bundle captures the log

12.5.4 The SAFE profile and its assumption

SAFE is static CAKE at **0.85 × min(S_lkg, S_base)** per direction, rules and leases kept, cleanup limited to VeriQ's own objects.

Worked example (illustrative): S_base is 90 down and 45 up (Mbit/s). An evening dip had settled the download at S_lkg = 60; the upload stayed at 45. SAFE down = 0.85 × 60 = **51 Mbit/s**; SAFE up = 0.85 × 45 = **38.25 Mbit/s**. Taking the minimum keeps SAFE from jumping back to the plan rate during a known dip; the 15% margin absorbs moderate further fluctuation.

> **Honesty check.** SAFE **assumes capacity does not keep falling**. If it drops to 30 Mbit/s while SAFE holds 51, bufferbloat returns until `veriqd` is back. SAFE is a bounded static state equivalent to SQM, consistent with the dominance invariant, not protection against every future condition.

12.6 Budgets and persistence

12.6.1 What must survive a restart

State that limits VeriQ's own behaviour must persist, or a crash loop could escape the limit. docs/09 A.7: the rolling 10-minute cut budget is "never reset or refunded by cooldowns, new episodes or restarts".

State	Why it persists	File
Fallback budget debits (with timestamps)	A restart must not refund uncertain cuts	`budget.json`
Sanction escalation counters	A restart must not forgive a repeat offender	`sanctions.json`
Leases (device, class, expiry as wall clock + boot ID, remaining time every 60 s)	Protection survives a daemon restart	`leases.json`
Desired version	Restart adoption	`version.json`
Capacity memory (168 hourly bins, 4 weeks)	Start rate after boot	`capacity.json`

Each file has `schema` and `written_at` fields and is written atomically (temp file + rename), so a power cut leaves the old or the new file, never half of one.

Worked example (illustrative). Anchor 90 Mbit/s, so the budget is 9 Mbit/s per rolling 10 min. At 20:30:00 a 4.5 Mbit/s step is debited; at 20:30:30 the daemon crashes and restarts. On adoption `budget.json` still holds the debit, so only 4.5 Mbit/s remains until 20:40:00. After the 60 s cooldown, one more step is possible, and then the budget is exhausted. The cooldown delayed the second episode; it refunded nothing.

> **Implementation tip.** docs/11 §8.5 limits state writes to ≤ 1 per minute. Write budget *debits* before applying the step (write-ahead), or a crash inside the coalescing minute could refund one. Debits are rare (at most two 5% steps per 10 min), so the flash cost is negligible.

After a reboot, leases resume only if NTP is synced, and remaining time is never extended. During operation, lease expiry needs no daemon: each lease is a kernel set element whose timeout is the remaining time.

12.6.2 Flash-wear policy

Routers keep `/etc` on flash (SPI-NOR or NAND under JFFS2 or UBIFS), whose blocks endure a limited number of erase cycles. VeriQ keeps high-rate data in RAM.

Data	Location	Write policy
Runtime state, heartbeat, decision-log ring (≤ 2 MB), metrics rings	`/var/run/veriq`, `/tmp/veriq` (RAM)	Continuous
Budget, sanctions, leases	`/etc/veriq/state/*.json`	On change, ≤ 1/min (debits write-ahead)

Data	Location	Write policy
Capacity memory, hourly summaries	`/etc/veriq/state/`	≤ 1/hour
Config	`/etc/config/veriq`, `/etc/veriq.d/`	On user change

Everything under `/etc/veriq` survives sysupgrade (`/lib/upgrade/keep.d/veriq`). To check a device, compare the worst-case daily volume with its flash datasheet. With illustrative sizes:

```
state files : <= 1 write/min  x 1440/day x ~4 KB  = ~5.8 MB/day (upper bound)
capacity    : <= 1 write/hour x 24/day   x ~10 KB = ~0.24 MB/day
```

The bound assumes a state change every minute all day, which does not happen. Stage 4 measures real write counts on the reference router during the soak. Keeping budgets in RAM for the Lite profile is deliberately **not** offered: it would break the no-refund rule.

12.7 Failure domains

Failure	Effect on the household	Detection	Response	Residual risk
`veriqd` crashes or hangs	None at once; kernel keeps the last rate	Heartbeat age	SAFE ≤ 12 s; respawn; adoption	Capacity falls further while in SAFE
Checker bug admits a bad action	At most one bounded bad step	Receipt NOT_IMPROVED; property tests	Undo; envelope clamps	Bugs outside tested properties
Controller bug	Refusals pile up	Refusal counts	Checker refuses; S unchanged	Lost adaptation only
Watchdog dies	None while `veriqd` is healthy	procd; health check	Respawn	Both dying together
Foreign tool changes the qdisc	Different shaper	Read-back	DEGRADED; no fighting	User must resolve
Firewall flushes VeriQ's table	Classes lost briefly	Reconciler	Re-apply ≤ 1 tick	≤ 1 tick unclassified
WAN down	Outage (ISP)	netifd, carrier	HOLD_LINKDOWN	None caused by VeriQ
Reflectors unreachable	No delay signal	Quorum < 2	Hold (L0); DEGRADED after 60 s	Real change missed while blind
Router CPU saturated	Throughput below S	CPU detector	Hold; recommend a backend	Shaping quality falls
Power cut	Outage until boot	—	Atomic files; capacity-memory start	≤ 1 min of non-debit state
Corrupt model or catalogue	Wrong classes	Signature; `TIN_RANGE`	Reject; classifier abstains	Valid but poor model
Host or kernel failure	Everything stops	—	Outside VeriQ's domain	Config lives only while the host runs

12.8 Safety case summary

Each claim is linked to its planned evidence. All statuses are **planned**; this table is the work list for Sprint 2.

Claim	Argument	Planned evidence	Tests
SC1. No command reaches the kernel without an ACCEPT	Enforcer token for the same ID and version	Enforcer unit test; code review	Tokenless call refused; replay
SC2. Rates stay in `[S_min, S_max]`	`BELOW_FLOOR`, `ABOVE_MAX`; controller clamps	Property test, 10^6 sequences	Fuzz; S2; A1–A3
SC3. Uncertain cuts ≤ 10% of anchor per rolling 10 min, across restarts	Budget rule; persisted write-ahead debits	Property test; kill during an episode	S13; A6; A7
SC4. Abstention never grants preference	`NO_PREFERENCE_BASIS`; kernel PROV precedence	Unit test; per-tin counters	S11; A27
SC5. A daemon failure does not break the internet	Persistent kernel config; watchdog; procd	kill -9 under load	S8; A15
SC6. VeriQ does not fight other tools	Foreign change → DEGRADED	Integration test, foreign qdisc	A30
SC7. Invalid config never replaces working config	`nft -c`; atomic `nft -f`	Unit and integration tests	A25
SC8. SHADOW applies no adaptive action	`STATE_FORBIDS`; Enforcer token	Unit test; 24 h shadow soak	S12
SC9. Leases end on time without the daemon	Kernel set-element timeouts	Kill the daemon during a lease	S5

Claim	Argument	Planned evidence	Tests
SC10. Clock steps cause no spurious action	Monotonic control clock; uptime heartbeat	Fake-clock unit test; NTP step	A18

For judges. “Our checker does not guarantee safety, and we do not claim it does. It checks every action against 22 written refusal rules, each with its own unit test. Below it, the kernel keeps shaping if our daemon dies, and a separate watchdog restores a static, SQM-equivalent configuration within 12 seconds by design. We will kill the daemon live. What we cannot protect against, we state: a further capacity fall while in SAFE, and failure of the host itself.”

Why it matters. Operators fear automation they cannot audit or undo. SHADOW first, every action logged with a reason, and a documented path back to static behaviour are what make a pilot acceptable.

CHAPTER 13

Chapter 13 (pending)

This chapter has not been written yet.

CHAPTER 14

Chapter 14 (pending)

This chapter has not been written yet.

CHAPTER 15

User experience

A home QoS system succeeds only if people can set it up, trust it and correct it. The kernel datapath (Chapter 8) and the controller (Chapters 10–12) do the work. This chapter specifies the surfaces through which people meet that work: the setup wizard, the LuCI administration app, the household page, the command-line tool `veriqctl`, alerts, and the plan for testing all of these with real users.

Acronyms used in this chapter: LuCI is the OpenWrt web interface [R41]; UCI is OpenWrt's Unified Configuration Interface [R40]; rpcd is OpenWrt's RPC daemon that exposes ubus (the OpenWrt micro bus) to the web UI; ACL is an access-control list; CAKE is the Common Applications Kept Enhanced queue discipline [R5]; SUS is the System Usability Scale [R39]; WCAG is the Web Content Accessibility Guidelines; QR is a Quick Response code.

15.1 UX principles

Every screen in this chapter follows seven rules. They come straight from the design principles P1–P9 in docs/11 §2, translated into interface behaviour.

#	Principle	What it means in the interface	Example
U1	Benefit first, adaptation earned	Static CAKE is active from minute one; adaptive control starts in SHADOW and is promoted to LIVE only after 24 h or an explicit user choice	Overview shows "Observing: 17 h left" with a "Why?" link
U2	Every number carries its status	Measured, estimated and target values look different (plain, "≈", "target")	"Capacity ≈ 82 ± 6 Mbit/s (estimate)"
U3	Every action is explainable in one sentence	Each rate change, class decision and refusal has a template sentence (Chapter 13)	"Holding: the slowdown is beyond your provider's first hop"
U4	Honest semantics	"Prioritised", never "guaranteed"; a per-device limit is labelled "policer, not a shaper"	Devices → Limit dialog
U5	Progressive disclosure	Household page → LuCI pages → Advanced page → CLI	A parent never sees reflector tables
U6	Reversible by default	Cancel, undo and "previous ruleset kept" are visible on every change	A failing rule shows its nft error; the old rules stay active
U7	Local and private	No cloud account, no remote telemetry by default; the UI states what is stored	Settings → Privacy shows the retention values

Why it matters. Judges will try the UI themselves. The convincing moment is when a screen *explains itself*: a "Why" column in the connections table, or a plain refusal sentence when a lease is not admitted. QoSmate does not have these, and they are cheap because the daemon already produces the sentences.

15.2 Personas

Five personas cover the people who will touch VeriQ. They are design tools, not research findings: each one is a composite, and the UX study (later in this chapter) tests whether the assumptions hold.

Persona	Context (illustrative)	Main goal	Main surface
Priya, 38, parent working from home	Pune flat, FTTH plan with a nominal 100/50 Mbit/s, two children, smart TV, cloud backup	Calls that do not freeze when the children download games	Household page; Overview
Arjun, 20, student gamer	Family home on a 4G/5G fixed-wireless (FWA) connection whose speed varies through the evening	Low and stable game ping without hand-tuning	Devices; Autorate
Kamala, 71, non-technical elder	Lives with her son's family; daily video calls with grandchildren abroad	"Is the internet OK?" and one button that helps	Household page only
Ravi, 29, ISP support agent	Tier-1 helpdesk for a broadband provider; talks to customers by phone	Find out in two minutes whether the problem is the line, the Wi-Fi or the router	Support bundle; Diagnostics; "Why is it slow?"
Meera, 33, power user	Runs OpenWrt with QoSmate on a mid-range ARM router	Keep her rules, gain the controller and explanations, lose nothing	Import; Rules; Advanced; CLI

15.2.1 What each persona needs

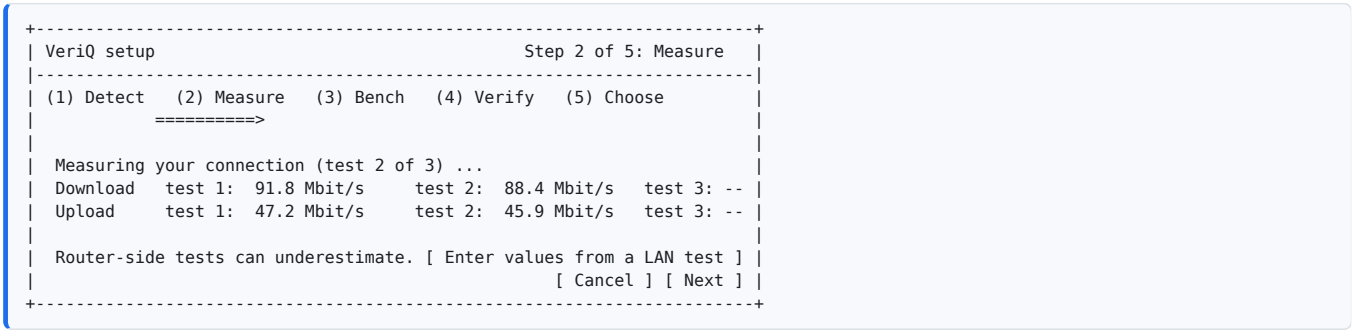
- **Priya** suffers unpredictability: a console update or cloud backup freezes her 7 pm meeting. She does not want to learn DSCP. VeriQ gives her a lease (Chapter 9) from her phone in three taps, a confirming sentence ("Your laptop is protected until 20:00") and a lease receipt afterwards. The eligibility envelope stops the lease being abused if her laptop starts a large download.
- **Arjun's** FWA capacity varies with cell load, so any static rate is wrong for part of the evening. VeriQ gives him a device rule (his console gets the RT class, Chapter 8), the adaptive controller, and the Autorate page, where he can see *why* the rate moved.
- **Kamala** needs one question answered on a phone with large text: "Is the internet OK?" Her son saved the household page to her home screen with a QR code. She sees a traffic light, one sentence in English or Hindi, and one large button; never a number she must interpret.
- **Ravi's** metric is time to a correct diagnosis. The customer can read out the "Why is it slow?" sentence, which separates *outside the home*, *Wi-Fi* and *router overloaded*, or download a support bundle with identifiers hashed. Any claim that this reduces support calls is a **hypothesis** for a pilot (Chapter 27).
- **Meera** will not switch if she must re-enter her QoSmate rules, IP sets and custom nft, or if she cannot inspect the controller. VeriQ gives her `veriqctl import qosmate --dry-run`, the same rule syntax (docs/11 §5.3), the same custom-nft files and a full CLI, plus provenance, evidence levels and receipts.

15.2.2 Key journeys and their targets

Journey	Persona	Surface	Target (to be measured)
First setup to a graded result	Priya's partner, Meera	Wizard	≤ 5 min
Protect a call	Priya, Kamala	Household page	≤ 3 taps after login, ≤ 20 s
Prioritise a console	Arjun	Devices → Protect, or a device rule	≤ 60 s
Diagnose a bad call	Ravi, Priya	"Why is it slow?" / Overview	correct cause named in ≤ 2 min
Undo a change	Meera	Rules (disable) or Receipts	≤ 30 s
Migrate from QoSmate	Meera	Import	no rule lost; differences listed

15.3 The setup wizard

The wizard runs on first open of Network → VeriQ, and again from Settings at any time. Its target is a configured, verified router in **≤ 5 minutes** (target, docs/11 §11.1). It has five steps; each shows progress and can be cancelled without changing the router.



15.3.1 Step 1: Detect

The wizard reads the WAN interface and its protocol from netifd (OpenWrt's network daemon). It proposes a link-layer preset from docs/11 §4.4: `ethernet` for fibre or Ethernet WAN, `docsis` for cable, `vdsl2-pppoe` for VDSL with PPPoE, and so on. A PPPoE session adds 8 bytes of overhead and a VLAN device adds 4; the wizard shows the resulting overhead with the note *"overestimating is safer than underestimating"*.

It then checks for conflicts (Chapter 16): SQM, qosify or QoSmate enabled; software or hardware flow offloading; other users of conntrack-mark bits 24–30. It also checks kernel capabilities (CAKE, `act_ctinfo`, IFB, nft meters and set timeouts). Each conflict appears as a card with a reason and a consent button, for example *"Flow offloading is on. Offloaded connections bypass the shaper, so VeriQ cannot manage them. [Turn off offloading] [Explain]"*. Nothing is disabled without consent.

15.3.2 Step 2: Measure

The wizard runs **three** capacity measurements: UDPST [R22] if a cooperating server is configured, otherwise `speedtest-go`. For each, it reads the WAN interface counters every second and takes the **interface-counter plateau**: the median of the 1 s samples that are at least half of the peak. This idea comes from QoSmate's auto-setup (`plateau_rate`, read). If the tool reports more than 1.25 × what actually crossed the interface, the wizard trusts the interface value (QoSmate's `warn_if_overreported` idea, read).

The base rate is **S_base = 0.90 × the lowest of the three plateaus**, per direction.

Worked example (illustrative numbers). Download plateaus are 91.8, 88.4 and 90.1 Mbit/s; the lowest is 88.4, so S_base,down = 0.90 × 88.4 ≈ 79.6 Mbit/s. Upload plateaus are 47.2, 45.9 and 46.6; S_base,up = 0.90 × 45.9 ≈ 41.3 Mbit/s. The screen shows both, with the button *"Use values from a LAN speed test instead"*, because a router CPU can limit its own test.

Edge cases: - **Spread too large.** If the highest plateau exceeds the lowest by more than 25% (a proposed threshold), the wizard says the line looks variable, recommends "Observe 24 h, then adapt", and mentions that capacity memory will learn evening patterns (Chapter 10). - **No internet or test failure.** The wizard asks for manual values and marks the verify step as "skipped". - **One direction not shaped.** Entering 0 disables shaping for that direction (the same convention as QoSmate and SQM); health then reports it as "off", not "failed".

15.3.3 Step 3: Backend bench

The wizard runs CAKE at S_base for 10 s against a local load generated on the router itself (veth pairs in a temporary namespace, no WAN traffic; Chapter 16). If CAKE reaches less than 95% of the target rate, or the shaping CPU exceeds 85%, the wizard recommends the `hfsc` or `htb` backend, or one-directional shaping.

Example message (illustrative): "At 79.6 Mbit/s download, CAKE reached 71 Mbit/s with the CPU at 97%. This router is too slow to run CAKE at this speed. Recommended: HTB backend (lowest CPU), or shape upload only." The user can accept or keep CAKE.

15.3.4 Step 4: Verify with a receipt

The wizard runs a bufferbloat test **before** (shaper bypassed for 20 s) and **after** (shaper active): saturating load in one direction at a time, with latency probes. It reports the added p95 delay (loaded p95 minus idle median) per direction and a letter grade. **The grade boundaries are VeriQ's own and are labelled as such in the UI.** They are not the Waveform grades [R15].

Grade	Added p95 delay	Plain-language label
A	≤ 5 ms	Excellent: calls and games unaffected
B	≤ 30 ms	Good: barely noticeable
C	≤ 60 ms	Fair: games may feel it
D	≤ 200 ms	Poor: calls may stutter
F	> 200 ms	Bad: calls likely to freeze

> **Honesty check.** Applying these boundaries to the only measured data we have, the Day-0 sandbox run [R51] (one preliminary run, no base RTT, IPv4 only): FIFO upload 266.1 ms → **F**, FIFO download 63.6 ms → **D**; short-buffer shaper upload 39.6 ms → **C**, download 55.4 ms → **C**. No VeriQ or CAKE grade has been measured yet.

The result screen is a small receipt: before, after, grade change, and the sentence *"This shows the difference in this test, at this time; it does not promise the same result under every condition."*

15.3.5 Step 5: Choose

- **Profile:** Lite, Standard (default) or Pro (footprints in Chapter 16). The wizard pre-selects Lite on devices with 128 MB of RAM.
- **Household page:** set a password; the wizard shows a QR code that opens `http://<router>/veriq/` (the household page, not the password itself).
- **Start mode:** "Observe 24 h, then adapt" (default, SHADOW) or "Adapt now" (LIVE). The first option shows the sentence *"Your connection is already protected by a fixed shaper. VeriQ will watch for 24 hours before it starts adjusting the speed by itself."*
- **Import:** if `/etc/config/qosmate` exists, offer *"Import your QoSmate rules, IP sets and limits"*, with a dry-run preview first. Import is never automatic.

15.4 The LuCI app

15.4.1 Information architecture

`luci-app-veriq` appears under **Network → VeriQ** and uses LuCI's client-side JavaScript views [R41], the same framework QoSmate uses. Charts use uPlot (MIT licence, about 50 KB) [R43].

QoSmate's menu (read, luci-app-qosmate v1.0.14 `menu.d`) has ten tabs: Settings, HFSC, CAKE, Advanced, IP Sets, Rules, Rate Limits, Custom Rules, Connections and Statistics. VeriQ reorganises the same capabilities around *tasks* rather than *mechanisms*:

QoSmate tab (read)	VeriQ page	Change
Settings (incl. auto-setup, health)	Overview + Wizard + Diagnostics	Status split from configuration
HFSC, CAKE, Advanced	Settings (basic) + Advanced	Backend parameters behind one backend selector
Rules, IP Sets	Rules	Adds device, app and schedule targets
Rate Limits	Devices → Limit; Rules → Policers	Labelled "policer"
Custom Rules	Advanced	Same two files: full table and inline

QoSmate tab (read)	VeriQ page	Change
Connections	Connections	Adds provenance, confidence, envelope status
Statistics	Queues + Autorate + Receipts	Adds evidence levels, reflector health, receipts
—	Devices, Apps	New

Every page has the same top band: the state badge, a banner slot for alerts, and a “Help” link to the relevant handbook section.

15.4.2 Overview

Purpose. Answer “Is it working, what is it doing, and why?” on one screen. Figure fig13 shows the wireframe.

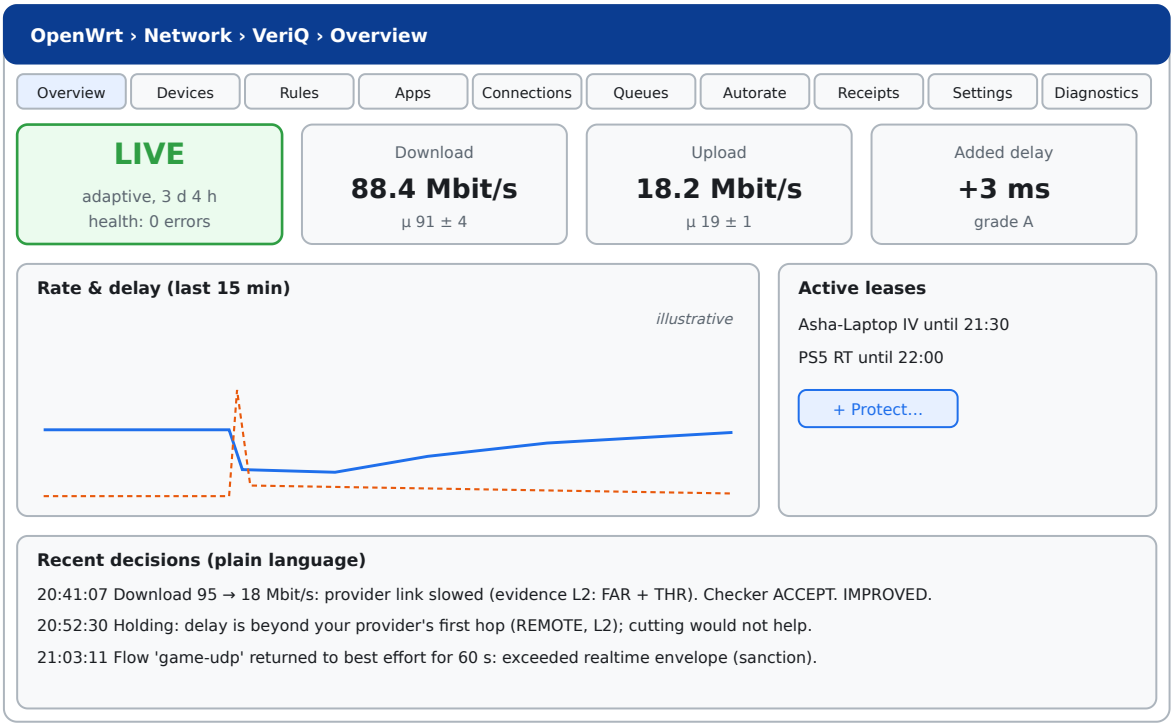


Figure 10. Wireframe of the LuCI Overview page: state badge, per-direction rate and estimate, live added delay and grade, active leases, the last five decisions as sentences, and a health summary.

```
+-------------------------------------------------------------------------+
| VeriQ  [ LIVE ]  backend: cake  profile: standard  model: m-2026.11.1   |
|-------------------------------------------------------------------------|
|  DOWNLOAD                          |  UPLOAD                            |
|  Shaper   82.0 Mbit/s              |  Shaper   41.0 Mbit/s              |
|  Capacity ~ 85 +/- 6 (estimate)    |  Capacity ~ 44 +/- 3 (estimate)    |
|  Added delay now  3 ms   Grade A   |  Added delay now  4 ms   Grade A   |
|  [sparkline: rate and delay, 15 min]   [sparkline: rate and delay, 15 min] |
|-------------------------------------------------------------------------|
|  Active protection: Priya-Laptop  IV  until 20:00  [Cancel]             |
|-------------------------------------------------------------------------|
|  Recent decisions                                                       |
|  20:41:07  Download 95 -> 18 Mbit/s: access link slowed (L2: FAR+THR).  |
|            After 10 s: delay -38 ms, IMPROVED.              [Details]  |
|  20:12:40  Holding: delay rise is beyond the provider's first hop. [Why] |
|-------------------------------------------------------------------------|
|  Health: OK   Self-test: passed 2 h ago   Reflectors: 5/5   [Diagnostics] |
+-------------------------------------------------------------------------+
```

Fields. - State badge: INIT, SHADOW (with time remaining), LIVE, DEGRADED, HOLD_LINKDOWN, SAFE or DISABLED (Chapter 12), each with a one-line meaning on hover. - Per direction: configured shaper rate S; the estimate μ ± σ (labelled “estimate”); live added delay; the grade from the last bufferbloat test. - Active leases with expiry and a Cancel button. - The last five decisions as sentences from the explain API (Chapter 13), including holds and checker refusals. - A health summary with a link to Diagnostics.

Actions. Run a bufferbloat test; switch SHADOW → LIVE (opens the shadow report first); open a decision’s details (explain view); cancel a lease.

Beyond QoSmate. QoSmate’s Settings page shows service status and health-check results (read). VeriQ adds the visible operating state, explainable decisions and the estimate with its uncertainty.

15.4.3 Devices

Purpose. Manage *people's devices*, not addresses.

```
+-------------------------------------------------------------------------+
| Devices                                        [ Search ] [ Add group ] |
|-------------------------------------------------------------------------|
| Name            Group   Addresses            Class mix      Now     Act. |
| Priya-Laptop    work    192.168.1.23 +2 v6   IV 60% BE 40%  6.1 Mb  [..] |
| PS5             games   192.168.1.40 +1 v6   RT 5% BK 95%   48 Mb   [..] |
| Kamala-Phone    family  192.168.1.51 +3 v6   BE 100%        0.4 Mb  [..] |
|-------------------------------------------------------------------------|
| [..] = [ Protect 1 h ] [ Limit (policer) ] [ Group ] [ Rename ] [ Rules ] |
+-------------------------------------------------------------------------+
```

Fields. Name (from DHCP or user-set), MAC address, every current IP address including IPv6 privacy addresses (resolved from DHCP leases and the neighbour table), group, class mix over the last hour, current rate.

Actions. - *Protect 1 h* creates an admin lease (IV by default, RT selectable). - *Limit* opens the policer dialog. The dialog heading reads *"Limit (policer, not a shaper): traffic above the limit is dropped, which can make some apps behave badly. Use only for guests or misbehaving devices."* - *Group* adds the device to a group that rules can target.

Beyond QoSmate. QoSmate rules and rate limits target IP addresses, ranges and sets (read). A VeriQ device rule follows the device when its IPv4 lease or IPv6 privacy address changes.

15.4.4 Rules

Purpose. Edit the user rules (precedence level 4, Chapter 9), IP sets, domain sets and subnet policers; import from QoSmate or SQM.

```
+-------------------------------------------------------------------------+
| Rules   [ Rules ] [ IP sets ] [ Domain sets ] [ Policers ] [ Import ]   |
|-------------------------------------------------------------------------|
| =  On  Name         Target           Match                Class   Hits  |
| =  [x] Console      device PS5       udp dport != 80,443  RT      1.2k  |
| =  [x] Work calls   group work, app  zoom                 IV      840   |
| =  [ ] Backups      device NAS       sched Mon-Fri 01-06  BK      0     |
| =  [x] Bad rule     192.168.1.300    tcp                  BE      --    |
|    !  Error in "Bad rule": invalid address. Previous rules still active. |
|-------------------------------------------------------------------------|
| [ Add rule ]  [ Validate ]  [ Save & apply ]          drag = to reorder  |
+-------------------------------------------------------------------------+
```

Fields. Every QoSmate rule option (name, proto, src/dest IP and port, `!=`, ranges, `@set`, IPv6 suffix masks, counter, enabled, DSCP class). VeriQ adds `device` / `group`, `app`, `vclass`, `schedule` and `direction` (docs/11 §5.3). The rule list shows hit counters.

Actions. Add, edit, enable or disable, reorder by drag, validate, save and apply. Validation compiles the whole candidate ruleset and runs `nft -c`. A failing rule is shown inline with its name and the nft error, and **the previous ruleset stays active**. The Domain sets tab carries a permanent note: *"Domain sets use this router's DNS. Devices that use encrypted DNS (DoH/DoT) bypass them."* The Import tab runs the importer in dry-run mode and shows a field-mapping table before anything is written.

Beyond QoSmate. Device, group, app and schedule targets; the importer; per-rule error placement. QoSmate also validates custom rules and shows a validation result (read).

15.4.5 Apps

Purpose. Let users prioritise or deprioritise known applications without knowing their ports.

```
+-------------------------------------------------------------------------+
| Apps  [ Search ]                        catalogue v2026.11 (signed) |
| App             Class  Source                  Verified    Choice   |
| Zoom Meetings   IV     vendor network page     2026-11-02  [Prio v] |
| OS updates (A)  BK     vendor documentation    2026-10-30  [Low  v] |
| Community app Y IV     community (UNVERIFIED)  --          [Def  v] |
+-------------------------------------------------------------------------+
```

Fields. App name, category, VeriQ class, source and verification date (docs/11 §5.4); unverified community entries are flagged. **Actions.** Prioritise, deprioritise or reset; each choice becomes a user rule with an `app` target, visible on the Rules page. If `dnsmasq-full` is missing, the domain part of an entry is greyed out with a link to Diagnostics.

Beyond QoSmate. QoSmate has no app catalogue; users write port or set rules themselves (read).

15.4.6 Connections

Purpose. Show live connections and, for each one, *why* it has its class.

```
+-------------------------------------------------------------------------+
| Connections  Filter: [ udp 192.168.1.40        ]  Poll: 2 s  [ Pause ]  |
|-------------------------------------------------------------------------|
| Proto Source             Destination        Class Why        Conf.  Env. pps |
```

```
| udp   192.168.1.40:3074   203.0.113.9:3074   RT    user rule  --      ok   62  |
| udp   192.168.1.23:51820  198.51.100.4:8801  IV    lease      --      ok   48  |
| tcp   192.168.1.40:50122  192.0.2.80:443     BK    heuristic  --      --   900 |
| udp   192.168.1.77:443    192.0.2.15:443     BE    abstained  {IV,BE} --  30  |
| udp   192.168.1.66:5004   198.51.100.7:5004  BE    SANCTIONED --      viol 2100|
+-------------------------------------------------------------------------------+
```

Fields. Protocol, endpoints, class, provenance ("Why": lease, user rule, app mark, ML, heuristic, default, system), confidence (the conformal set or "abstained"), envelope status (ok, violation, sanctioned with time left), pps and kbit/s. The filter uses QoSmate's syntax: space-separated terms, all of which must match (read).

Actions. Click a row to open the *explain flow* drawer: the precedence trace, the ML probability vector and conformal set, and the sanction history (Chapter 13). From the drawer: *Correct class* creates a user rule for that device and destination; *Protect device* creates a lease.

Beyond QoSmate. QoSmate's Connections tab shows DSCP, pps and bandwidth with AND filtering and zoom (read). The "Why", confidence and envelope columns are new; they make the precedence rule *"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."* visible on screen.

15.4.7 Queues

Purpose. Show what the qdiscs are doing, per class and direction.

```
+-------------------------------------------------------------------------+
| Queues   Direction: (o) Download ( ) Upload    Window: [ 5 min | 1 h ]    |
|-------------------------------------------------------------------------|
| Tin/class   Bytes    Packets  Drops  ECN marks  Av delay  Peak delay     |
| Voice (RT)  12 MB    61 k     0      0          0.4 ms    2.1 ms         |
| Video (IV)  310 MB   240 k    12     40         1.2 ms    6.8 ms         |
| BestEffort  2.1 GB   1.6 M    310    1.1 k      3.9 ms    14 ms          |
| Bulk (BK)   9.8 GB   7.0 M    4.2 k  9.0 k      11 ms     38 ms          |
| [ uPlot chart: per-tin rate and delay over the window ]                 |
+-------------------------------------------------------------------------+
```

Fields. Per tin (CAKE) or class (HFSC/HTB): bytes, packets, drops, ECN marks, average and peak delay, in both directions. Charts for the selected window come from RAM ring buffers; nothing is written to flash. **Beyond QoSmate.** QoSmate's Statistics page already shows per-tin and per-class tables and charts (read); this page is parity, with the same labels in both backends.

15.4.8 Autorate

Purpose. Make the controller inspectable: rate, evidence and budget.

```
+-------------------------------------------------------------------------+
| Autorate   Direction: (o) Download ( ) Upload            state: LIVE    |
|-------------------------------------------------------------------------|
| [chart: S, G-hat, mu +/- sigma, D over 15 min, with cut/hold markers]   |
| Evidence: L0 ....L1 ....L2 ##.......L1 .......  (timeline, 15 min)       |
| Fallback budget: [######----] 6% of 10% used in rolling 10 min          |
|-------------------------------------------------------------------------|
| Reflector        Health   Baseline   Delta    Timestamps   Note         |
| 1.1.1.1          ok       11.2 ms    +0.8 ms  yes                        |
| 9.9.9.9          ok       14.0 ms    +1.1 ms  yes                        |
| 8.8.8.8          EXCLUDED 13.1 ms    +41 ms   no          outlier 3xMAD  |
| near (gateway)   ok        1.9 ms    +0.2 ms  --                         |
+-------------------------------------------------------------------------+
```

Fields. Per direction: S, $\hat{G}$, $\mu \pm \sigma$, the delay signal D; the reflector table (health, baseline, delta, timestamp support, exclusion reason); the evidence-level timeline; episodes and the fallback-budget meter (Chapters 10–12). **Actions.** Pause adaptation (go to SHADOW), view the shadow report, open an episode's receipts. **Beyond QoSmate.** QoSmate shows autorate rates, latency, baseline and thresholds with charts (read). VeriQ adds evidence levels, per-reflector health and the budget meter, so a user can see *why* a cut did or did not happen.

15.4.9 Receipts

Purpose. List what VeriQ did and what followed.

```
+-------------------------------------------------------------------------+
| Receipts   Filter: [ All | IMPROVED | NOT_IMPROVED | INCONCLUSIVE ]     |
|            Type:   [ Rate | Lease | Shadow disagreements ]               |
|-------------------------------------------------------------------------|
| 20:41:07  RATE_CUT down 95->18  L2 FAR+THR  ACCEPT  IMPROVED  [open]    |
| 19:00:02  LEASE Priya-Laptop IV 60 min        ACCEPT  IMPROVED  [open]  |
| 18:22:15  FALLBACK_STEP up -5%  L1  ACCEPT  NOT_IMPROVED -> undone      |
| 17:03:40  (shadow) would cut 60->50; LIVE rule: hold   disagreement     |
+-------------------------------------------------------------------------+
```

Fields. Every receipt field (Chapter 13), with the footer *"Receipts show what happened before and after an action (association), not proof that the action caused it."* **Actions.** Filter, open, export as JSON. **Beyond QoSmate.** New.

15.4.10 Settings

Purpose. All everyday configuration on one page, in collapsible groups: *Basic* (WAN interface, rates, link preset, backend, profile, start mode); *DSCP policy* (product, lab or custom; wash in/out; trust); *Heuristics* (H1–H5 toggles; H3 shown as "off on CAKE" with the reason); *Envelope* (RT/IV limits, penalty); *Probing* (reflector list, rates, UDPST server); *Privacy* (retention, export). Errors appear next to the offending field and are reported as `section.option` (for example `shaping.down_base_kbit: must be ≥ 1000 or 0`). **Beyond QoSmate.** Mostly parity; the heuristics and envelope groups expose VeriQ-specific safety settings.

15.4.11 Advanced

Purpose. Expert controls kept out of the everyday path: inline nft statements (`/etc/veriq.d/inline.nft`) and the custom table (`/etc/veriq.d/custom.nft`), each with *Validate* and examples; CAKE extra parameters; HFSC and HTB parameters (RT budget, leaf type, MAXDEL, packet size); exporter on/off and port; webhook URL. A warning band reads *"Changes here can stop VeriQ from classifying correctly. Every change is validated before it is applied."* **Beyond QoSmate.** Parity: QoSmate offers the same two custom-rule areas with validation (read).

15.4.12 Diagnostics

Purpose. One-click checks that a support agent or power user can run.

```
+-------------------------------------------------------------------------+
| Diagnostics                                                             |
|-------------------------------------------------------------------------|
| [ Run health check ]  status=ok;errors=0          last run 20:10        |
| [ Run self-test ]     tins up: 4/4 ok  down: 4/4 ok  ctinfo: ok         |
| [ Run bench ]         cake 82 Mbit/s @ 41% softirq; htb @ 18%           |
| [ Bufferbloat test ]  down: A (3 ms)  up: A (4 ms)                      |
| [ Support bundle ]    preview contents -> download (identifiers hashed) |
+-------------------------------------------------------------------------+
```

Beyond QoSmate. QoSmate's `health_check` reports service, nft, tc, config, package and file-integrity status (read). VeriQ adds the tin self-test, mark-plumbing checks, probe checks, conflicts and the support bundle (Chapter 16).

15.5 The household page

The household page is a separate, mobile-first web page for people who should never see LuCI. Figure fig14 shows the three screens on a phone.

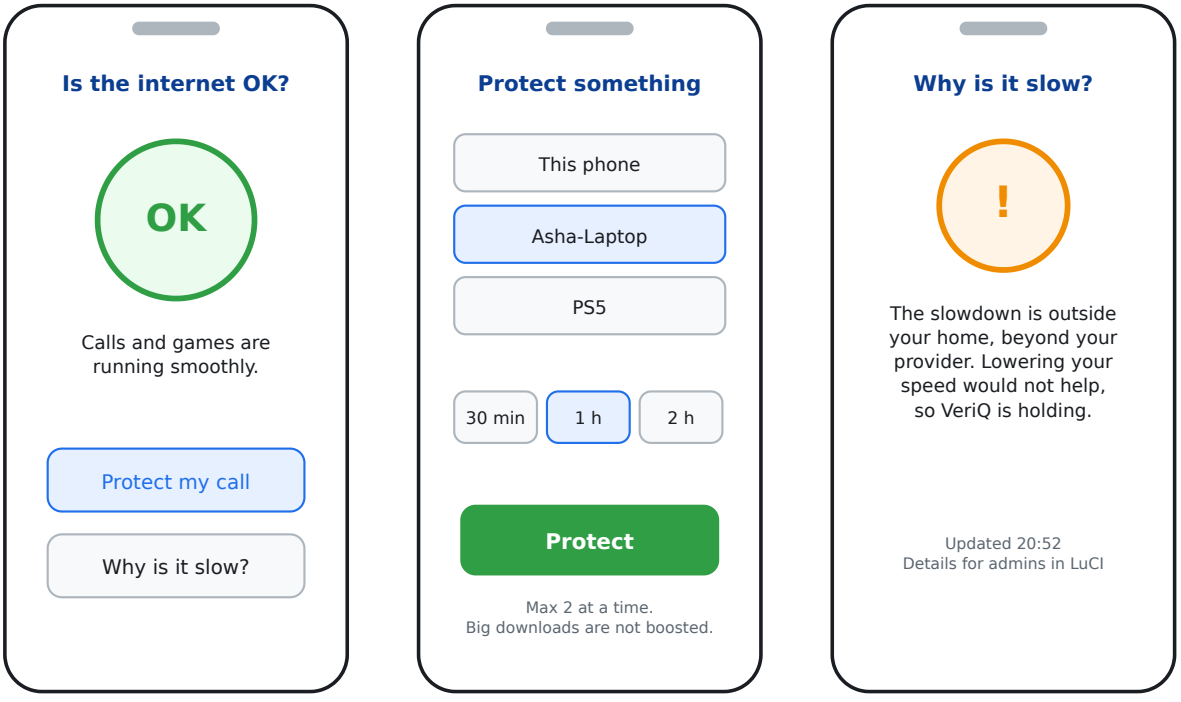


Figure 11. Phone wireframe of the household page: "Is the internet OK?", "Protect something" and "Why is it slow?".

15.5.1 Access and permissions

- **URL:** `http://<router>/veriq/`, static files served by uhttpd (package `veriq-household`).
- **Login:** the rpcd user `household`, with the password set in the wizard. The session is an ordinary rpcd session.

- **ACL:** the household role may call only `veriq.status`, `veriq.lease_request`, `veriq.lease_list`, `veriq.lease_cancel_own` and `veriq.explain_simple` (docs/11 §11.3). It cannot read device addresses, change rules or open LuCI.
- **Size:** vanilla JavaScript, no framework, under 100 KB in total (target).

15.5.2 Screen 1: "Is the internet OK?"

```
+----------------------------------+
|  Home internet                   |
|                                  |
|        ( GREEN )  OK             |
|                                  |
|  Calls and games should be       |
|  smooth right now.               |
|                                  |
|  Protected: Priya-Laptop         |
|  until 20:00                     |
|                                  |
|  [  Protect something   ]        |
|  [  Why is it slow?     ]        |
|                    English|हिंदी |
+----------------------------------+
```

The light is computed by veriqd and returned by `veriq.status`. Proposed rules: - **green:** state LIVE or SHADOW, and grade A or B on the live added delay over the last 5 minutes; - **amber:** grade C or D, or state DEGRADED, or a non-local diagnosis (remote, Wi-Fi, CPU); - **red:** SAFE, WAN down, or grade F.

Colour is never the only signal: each state also has an icon and a word (OK, Busy, Problem).

15.5.3 Screen 2: "Protect something"

```
+----------------------------------+
|  Protect something               |
|                                  |
|  What?                           |
|  (o) This phone                  |
|  ( ) Priya-Laptop                |
|  ( ) Living-room TV              |
|                                  |
|  For how long?                   |
|  [15 min][30 min][1 h][2 h]      |
|                                  |
|  Why? (optional) [exam call   ]  |
|                                  |
|  [      Protect      ]           |
+----------------------------------+
```

The choices map directly onto a lease (Chapter 9): the class defaults to IV ("Protect my call"); RT appears only as "Game mode" for devices in a games group. Durations are 15, 30, 60 or 120 minutes, with a maximum of 4 h and one renewal without re-approval. The reason is stored only in the local audit log.

When admission refuses, the reason is shown in plain words: - *"Two things are already protected. Ask an adult to cancel one, or wait until 20:00."* (more than two active leases); - *"This device is already protected until 19:45."* (one per device); - *"The internet connection is very weak right now, so protection would not help. VeriQ is keeping things fair for everyone."* (capacity collapse).

"This phone" needs the address of the browser that loads the page. uhttpd's ubus endpoint does not pass the client address to rpcd plugins by default, so the mechanism must be verified on the target. If it cannot be done cleanly, the option is hidden and the user picks the phone by name.

15.5.4 Screen 3: "Why is it slow?"

```
+----------------------------------+
|  Why is it slow?                 |
|                                  |
|  The slowdown is coming from the |
|  internet beyond your provider.  |
|  Lowering your speed would not   |
|  help, so VeriQ is holding.      |
|                                  |
|  What you can do:                |
|  - Try again in a few minutes.   |
|  - If it lasts, the website or   |
|    game server may be busy.      |
|                                  |
|  Checked at 20:12   [ Refresh ]  |
+----------------------------------+
```

The sentence comes from the latest diagnosis label (Chapter 11), mapped to plain language:

Diagnosis label	Household sentence (English draft)	Advice shown
OWN_QUEUE	“The connection is busy, but VeriQ is keeping it smooth.”	None needed
WAN_LIKELY	“Your internet connection slowed down. VeriQ lowered its limit to keep calls smooth.”	“If this happens every evening, tell your provider.”
REMOTE_LIKELY	“The slowdown is beyond your provider’s first hop. Lowering your speed would not help, so VeriQ is holding.”	“Try again later; the server may be busy.”
WIFI_LIKELY	“<device> has a weak Wi-Fi signal.”	“Move closer to the router, or use the 5 GHz network.”
CPU_LIMITED	“The router itself is overloaded.”	“Ask the admin to check VeriQ’s backend setting.”
PROBE_FAULT	“VeriQ cannot measure the connection right now, so it is not changing anything.”	None
MIXED / UNKNOWN	“VeriQ is not sure what is causing this, so it is only making small, careful changes.”	“If it lasts, run a speed test.”

15.6 Command-line interface

`veriqctl` talks JSON-RPC 2.0 over the Unix socket `/var/run/veriq.sock` (root only). Every command accepts `-j` for JSON output, which is exactly what the LuCI rpcd plugin consumes.

Command	Purpose
`veriqctl status [-j]`	State, rates, estimates, delay, leases, health summary
`veriqctl explain rate up|down`	The last rate decision with evidence, permission row, verdict and receipt
`veriqctl explain flow <ctid|tuple>`	Precedence trace, class, provenance, envelope, ML set, sanctions
`veriqctl lease add <device> <rt|iv> <minutes>` / `lease list` / `lease cancel <id>`	Lease management
`veriqctl receipts [--since 1h] [--outcome improved]`	Receipts
`veriqctl health` / `veriqctl selftest` / `veriqctl bench [--rate X]`	Checks
`veriqctl import qosmate [file] [--dry-run]` / `veriqctl import sqm`	Migration
`veriqctl shadow report [--since 24h]`	Shadow disagreements
`veriqctl state set live|shadow|safe`	Operating-state override
`veriqctl support-bundle [-o file.tar.gz]`	Support bundle
`veriqctl config validate` / `veriqctl version`	Configuration and version

Proposed exit codes: 0 success; 1 error; 2 usage error; 3 refused (by the checker, admission or validation), with the refusal code printed. Scripts can then tell “VeriQ said no” apart from “VeriQ is broken”.

All outputs below are **illustrative**; no part of VeriQ has run yet.

```
# veriqctl status
state      LIVE (since 2026-11-20 06:02, 14h39m)   backend cake   profile standard
download   S 82.0 Mbit/s   est 85 +/- 6   G-hat 61.3   load 0.75   D +2.1 ms   grade A
upload     S 41.0 Mbit/s   est 44 +/- 3   G-hat  3.2   load 0.08   D +0.9 ms   grade A
leases     1 active: Priya-Laptop IV until 20:00 (lease 7f3c)
reflectors 5/5 healthy (4 with timestamps)   near: ok
health     ok (selftest passed 2026-11-20 18:10)
```

```
# veriqctl explain rate down
20:41:07 download rate 95 -> 18 Mbit/s (RATE_CUT, id 01JC9X...)
evidence  WAN_LIKELY @ L2 from FAR (+42 ms) and THR (G-hat 20.1 at load 0.99)
permit    L2: one step to max(S_min, 0.90 * G-hat) = 18.1 Mbit/s
checker   ACCEPT (version 812)
receipt   10 s after: delay -38 ms, G-hat 18.0 -> IMPROVED
note      association, not causation
```

```
# veriqctl explain flow 3141592
flow      udp 192.168.1.77:443 -> 192.0.2.15:443 (QUIC)
trace     L1 state: pass | L2 sanction: none | L3 lease: none | L4 user: no match
          L5 app mark: dscp cs0 | L6 classifier: ML set {IV, BE} -> abstain | L7 default
class     BE (provenance: default)
envelope  n/a (not preferential)
rule      Classifier abstention alone never grants preference.
```

```
# veriqctl lease add PS5 rt 300
refused (3): LEASE_GRANT ADMISSION: duration 300 min exceeds the 240 min maximum
```

```
# veriqctl import qosmate --dry-run
read /etc/config/qosmate (QoSmate config)
settings  WAN=eth1 DOWNRATE=90000 UPRATE=45000 ROOT_QDISC=hfsc -> backend hfsc
rules     12 found: 11 mapped, 1 needs review (rule "dns-ef": class ef -> RT, see note)
ipsets    3 found: 3 mapped (2 static, 1 dynamic; dnsmasq-full present)
ratelimit 1 found: "Guests" -> policer (labelled policer, not a shaper)
custom    /etc/qosmate.d/custom_rules.nft -> /etc/veriq.d/custom.nft (validate needed)
nothing written (dry run). Re-run without --dry-run to apply.
```

The `selftest` output format is shown in Chapter 16.

15.7 Alerts

- **LuCI banners** appear on every VeriQ page for SAFE, DEGRADED, flow offloading detected, and a model abstaining on more than 80% of flows (docs/11 §11.5). Each banner names the cause and one action, for example *"VeriQ is in SAFE mode: the control daemon stopped responding at 20:03. Internet access continues with a fixed shaper. [View log] [Restart VeriQ]"*.
- **Webhook (optional):** a JSON POST to a user-set URL (for example a self-hosted ntfy or Home Assistant instance), limited to one message per 5 minutes per alert type. The payload contains no addresses or device names unless the user opts in.

```
{"source":"veriq","router":"home-gw","type":"SAFE","at":"2026-11-20T20:03:12+05:30",
 "message":"Watchdog engaged SAFE shaper; daemon heartbeat lost for 11 s",
 "state":"SAFE","link":"http://192.168.1.1/cgi-bin/luci/admin/network/veriq/overview"}
```

- **Household page:** no alert pushes; the traffic light reflects SAFE and link-down as red.

15.8 Internationalisation and accessibility

Languages. Strings are kept in gettext `.po` files. English and Hindi ship at P1; other languages can be added by the community, as QoSmate does with German and Simplified Chinese (read). Rules for translators and developers: - use whole-sentence templates with named placeholders, never concatenated fragments, because Hindi word order differs from English; use gettext plural forms; - keep Western digits and SI units (Mbit/s, ms) in both languages; - leave room for longer translated strings; check layouts with the real Hindi strings; - keep technical terms (CAKE, DSCP, nft) untranslated on admin pages; the household page avoids them.

Draft Hindi strings for the household page (to be reviewed by a native speaker before release):

Key	English	Hindi (draft)
status.ok	Internet is OK	इंटरनेट ठीक है
action.protect	Protect something	किसी को सुरक्षित करें
action.why	Why is it slow?	धीमा क्यों है?
lease.until	Protected until {time}	{time} तक सुरक्षित
advice.wifi	Move closer to the router	राउटर के पास जाएँ

Accessibility. - WCAG AA contrast (at least 4.5:1 for body text) on both pages, in light and dark themes. - Touch targets at least 44 × 44 CSS pixels on the household page. - Meaning never carried by colour alone (traffic light + icon + word; class colours + labels). - Every control labelled for screen readers; charts have a text summary ("Download delay stayed below 5 ms in the last 15 minutes"). - Keyboard operation in LuCI; no hover-only information. - The household page works without web fonts and in older mobile browsers; Devanagari text uses the phone's system font.

15.9 Error and empty states

Good empty states teach. Bad ones look like failures.

Situation	What the user sees	Action offered
veriqd not running	"The VeriQ service is not running. Your last shaper settings are still active in the kernel."	[Start] [View log]
First hours in SHADOW, no receipts	"No actions yet. VeriQ is observing for 24 h; decisions it *would* have made appear under Shadow disagreements."	[View shadow report]
No reflector quorum	"VeriQ cannot measure delay right now (no reflectors answering). It is holding the current speed."	Diagnostics

Situation	What the user sees	Action offered
`dnsmasq-full` missing	"Domain sets and app domains need dnsmasq-full."	[How to install]
Flow offloading on	"Offloaded traffic bypasses the shaper. VeriQ stays in SHADOW until offloading is off."	[Turn off] [Explain]
Rule fails `nft -c`	"Rule 'Bad rule' was not applied: <nft error>. Your previous rules are still active."	[Edit rule]
Lease not resumed after reboot	"Protection for PS5 was not restored because the router clock is not yet synchronised."	[Protect again]
Model abstaining > 80%	"The traffic classifier is unsure on this network. Rules, apps and protection still work."	[Learn more]

15.10 UX study plan

The UX study (P1) is **formative**: it finds problems to fix; it does not prove superiority.

- **Participants:** 6–8 (docs/11 §15.4), mixed: at least 2 who have used OpenWrt, at least 2 who have not, at least 1 aged over 60 for the household-page tasks.
- **Design:** within-subjects comparison of QoSmate's LuCI app and VeriQ's LuCI app on the same router image, with the order counterbalanced (half start with each).
- **Tasks:** (1) prioritise a game console; (2) find out why a video call is bad, with a scripted remote-delay fault in the lab; (3) undo a change made earlier. Extension for VeriQ only: (4) protect a call from the household page on a phone.
- **Measures:** task success (yes/partial/no), time on task, errors, and the SUS questionnaire after each system. Think-aloud notes are taken by a second team member.
- **SUS scoring [R39]:** ten items rated 1–5; for odd items subtract 1 from the rating, for even items subtract the rating from 5; sum the ten contributions and multiply by 2.5, giving 0–100. The target is **≥ 75 (formative)** (docs/11 §17).
- **Ethics and privacy:** written consent; no video of faces; screen recordings stored locally and deleted after analysis; participants can stop at any time.
- **Reporting:** per-task success and median time for both systems, every SUS score (not just the mean), and the top five problems found with their fixes. With n ≤ 8, results are reported as formative; no significance tests are claimed.

> **For judges.** "Our SUS target is 75, and the study is formative with 6–8 people. We will show you every score and every problem we found, including the tasks where QoSmate's UI did better."

15.11 UX acceptance checklist

A release passes UX acceptance when every item below is checked on the OpenWrt x86-64 VM and on the Raspberry Pi reference router (Chapter 16):

- [ ] The wizard completes in ≤ 5 min on a 100/50 Mbit/s lab link, including verify (target).
- [ ] The wizard can be cancelled at every step with no change left on the router.
- [ ] Grades shown match the A–F table, and the UI labels them as VeriQ's own boundaries.
- [ ] Every rate decision on Overview has a sentence and a Details link to the explain view.
- [ ] Every connection row has a provenance value; abstained flows show "abstained", never a guessed class.
- [ ] The policer dialog says "policer, not a shaper".
- [ ] No UI string uses "guarantee" for bandwidth or priority.
- [ ] A failing rule shows its name and nft error inline, and the previous ruleset stays active (tested).
- [ ] Domain-set screens show the encrypted-DNS (DoH/DoT) limitation.
- [ ] SAFE, DEGRADED, offload and model-abstention banners appear within one refresh of the condition.
- [ ] The household page loads in under 2 s on a mid-range phone over Wi-Fi (target) and is under 100 KB.
- [ ] The household role cannot call any method outside its ACL (tested with a direct ubus call).
- [ ] Lease refusals show a plain-language reason for every admission code.
- [ ] All household strings exist in English and Hindi; the Hindi has been reviewed by a native speaker.
- [ ] Contrast ≥ 4.5:1 and touch targets ≥ 44 px on the household page (checked with a contrast tool).
- [ ] Every chart has a text summary for screen readers.
- [ ] The support bundle shows its contents before download, and identifiers are hashed.
- [ ] `veriqctl` returns exit code 3 with the refusal code on every refusal path.
- [ ] The Playwright LuCI tests (Chapter 23) load every page without console errors.
- [ ] SUS and task results from the formative study are recorded in the evidence ledger.

CHAPTER 16

Chapter 16 (pending)

This chapter has not been written yet.

CHAPTER 17

Chapter 17 (pending)

This chapter has not been written yet.

Part III · Implementation

From design to code: stack, modules, templates, flows and plan

CHAPTER 18

Tech Stack and Repository

This chapter fixes the tools, languages and conventions the team uses to build VeriQ. It explains each technology choice and the alternatives that were rejected. It walks through the monorepo folder by folder, gives a reproducible setup for every development environment, and sets the coding, branching and continuous integration (CI) rules. It ends with the licence policy for third-party code. Chapter 19 then goes package by package through the daemon.

The guiding rule is the same as for the product: **one way to do each thing, pinned, reproducible and explainable to a judge in one sentence.**

18.1 Technology choices at a glance

Area	Choice	Alternatives considered	Why this choice
On-router daemon (`veriqd`, `veriqctl`)	Go, static binary, `CGO_ENABLED=0`	Python 3, Rust, C, ucode (OpenWrt's scripting language)	One cross-compiled binary for every OpenWrt target; no interpreter on flash; memory safety; goroutines fit the "one control loop plus many I/O readers" model (C2 in docs/11)
Kernel configuration	Run `tc -batch -` and `nft -f -` as processes; read back with `-j` (JSON)	Netlink libraries for qdiscs and nftables	The text is the same text a human or a test can read, diff and replay; `nft -f` is atomic; a few ms per call fits ≤ 2 changes/s (docs/11 §14.2)
Conntrack and NFLOG	Netlink (`ti-mo/conntrack`, `florianl/go-nflog`)	Polling `conntrack -L`; `/proc` parsing	Event-driven, no process spawn per flow, no cgo
Probing	Raw ICMP sockets (`golang.org/x/net/icmp`) plus a hand-built ICMP Timestamp message	External `ping` / `fping`; `irtt`	One-way deltas need ICMP Timestamp (RFC 792 [R16]); an in-process prober has exact send times and no fork cost
Local API	Line-delimited JSON-RPC 2.0 on a Unix socket (`/var/run/veriq.sock`, 0600)	HTTP/REST listener; gRPC; ubus object in the daemon	No network listener; trivial to call from `veriqctl` and the rpcd plugin; standard error model
ML training	Python: LightGBM [R31], scikit-learn, own split-conformal code	PyTorch; on-device training	Gradient-boosted trees are small, fast and well understood; training stays off the router (docs/11 §5.6)
ML inference	Pure-Go tree walk over an exported JSON model	LightGBM C API via cgo; ONNX runtime	No cgo, no shared library, bit-for-bit comparable with Python reference outputs
Admin UI	LuCI client-side JavaScript views [R41]	A separate single-page app with its own web server	Same framework as QoSmate and the rest of OpenWrt; served by uhttpd; ACLs through rpcd
Charts	uPlot [R43] (MIT, ~50 KB)	Chart.js; ECharts; server-side images	Small enough for router flash; fast with long time series
Household page	Vanilla JS, < 100 KB, no framework	React; Vue	Works on slow phones; fewer dependencies to audit (docs/11 §11.3)
Watchdog	POSIX `sh` under procd (C11)	A second Go binary; systemd	Shares nothing with `veriqd`, so a Go runtime fault cannot take both down
Lab harness and evaluation	Python 3.12 + shell, network namespaces	Mininet; containers only	Extends the existing `lab/quickstart_bufferbloat.sh`; Python has the statistics stack
3D twin	three.js / react-three-fiber [R43]	Unity; Blender renders	Runs in a browser, replays the JSON decision log
Documentation	Markdown → pandoc → HTML → PDF	LaTeX; Word; a docs site generator	Plain text in git, diffable, one build command; the same sources feed the handbook and the submission

> **For judges.** "Why Go and not Python?" Python on a router costs roughly 10–15 MB of flash for the runtime alone (docs/11 C2; an estimate, not a measurement) and has no single-binary story across five CPU architectures. Python stays where it is strongest: the lab, statistics and model training.

18.2 Key decisions explained

18.2.1 Go for the daemon

Four candidates were compared against the product needs: five target architectures (x86-64, two ARM64 profiles, ARMv7, MIPS little-endian), a RAM budget of ≤ 20 MB in the Lite profile (a target, docs/11 §12.2), no runtime downloads, and a team of students who must be productive within a sprint.

Language	Strengths	Why not chosen
Python 3	Team familiarity; fast prototyping; pyroute2	Interpreter and packages on flash; higher RSS; packaging across targets is fragile
Rust	Memory safety without a garbage collector; small binaries	Steeper learning curve for a 9-week build; cross-compiling for `mipsel_24kc` needs more toolchain work
C	Smallest footprint; native OpenWrt citizen	Memory-safety risk in a root daemon parsing untrusted input (JSON-RPC, netlink); slower development
ucode	Native to modern OpenWrt; tiny	Not suited to a 250 ms control loop with ring buffers, fuzzing and replay; limited test tooling
Go	Static binary; `GOARCH` / `GOMIPS` cross-compilation with no C toolchain; goroutines and channels; built-in fuzzing and race detector	Garbage collector and a larger binary than C; acceptable inside the budgets, and measured in Stage 4

The fallback is written into the spec: if the team cannot ramp up on Go in Sprint 0, the controller is first implemented in Python in the lab with the same interfaces and golden tests, and ported to Go in P1 (O7 in docs/11 §20).

18.2.2 Exec `tc` / `nft` rather than netlink for configuration

Netlink libraries for qdiscs exist, but CAKE, HFSC, `act_ctinfo` and `act_mirred` each have attribute sets that differ between kernel versions. Generating the same text an engineer would type has three advantages: - the rendered text is the unit of testing (golden files), of support (it goes into the support bundle) and of review; - `nft -f` applies a whole ruleset as one transaction, and `nft -c -f` validates it first without touching the kernel; - read-back uses the JSON output (`tc -s -j`, `nft -j`), which the same parser handles in the lab, the VM and on hardware.

The cost is a process spawn per change. At ≤ 2 rate changes per second that is a few milliseconds, inside the CPU budget. Conntrack and NFLOG are different: they are high-rate event streams, so they use netlink directly.

18.2.3 LightGBM, exported and walked in Go

The classifier is a LightGBM multiclass model per checkpoint (≤ 200 trees, depth ≤ 6; docs/11 §5.6). The Python pipeline in `ml/` trains it, calibrates the Mondrian conformal thresholds and exports both to a flattened JSON format. `veriqd` evaluates the trees with a pure tree walk of about 60 lines (chapter 19). The CI job compares Go probabilities against Python reference outputs on a fixed set of golden feature vectors, so the two implementations cannot drift apart silently.

18.2.4 uPlot inside LuCI

The Autorate and Queues pages draw long time series (rate, Ĝ, delay, per-tin delay). uPlot draws tens of thousands of points on a canvas with a small bundle. It is loaded as a static asset by the LuCI view; no CDN is used at runtime, because the router must work offline.

18.2.5 Pandoc for documents

The handbook, the submission and the docs are Markdown files. One `pandoc` command turns them into HTML and PDF. Figures are SVG files kept in the repository. This keeps every claim reviewable in a pull request, and lets the claims check (chapter 24) grep the text for numbers that lack a label.

18.3 Toolchain and version pinning

Exact versions are pinned in files, not in people's heads. The table gives the policy; the pinned value is recorded at Sprint 0 and changed only by a reviewed pull request.

Tool	Version policy	Pinned in	Notes
Go	Go 1.22 or later (docs/11 §14.1); one exact release for all builds	`go.mod` (`go` and `toolchain` lines), CI setup step	Release builds use the same toolchain as CI so `-trimpath` builds are reproducible
OpenWrt	24.10.x (fw4, kernel 6.6); best effort on 23.05	`openwrt/feed/sdk.lock` (SDK tarball name and checksum)	The apk-based release line is added when the SDK is available
Ubuntu (lab host)	24.04 LTS, x86-64, a full kernel with CAKE	`lab/README.md`; `environment.txt` fingerprint in every run	WSL2 kernels may lack CAKE (lab/README)
Python	3.12 (docs/05 §8)	`ml/requirements.txt`, `lab/requirements.txt` with hashes	Separate virtual environments for `ml/` and `lab/`

Tool	Version policy	Pinned in	Notes
LightGBM	One exact version for training and export	`ml/requirements.txt`	The model file records the trainer version
iproute2 / nftables	Whatever the target ships; recorded, not chosen	Self-test output and support bundle	Syntax that differs between versions is verified by the Stage-1 self-test
Node.js	One LTS release for Playwright and the twin build	`.nvmrc`, `package-lock.json`	Never needed on the router
pandoc	One release for the handbook build	`docs/Makefile`	The PDF engine is pinned with it

Implementation tip. Run `go version`, `tc -V`, `nft --version`, `uname -r` and `python3 --version` inside every lab run and write them into the run manifest. The Day-0 evidence already does this in `environment.txt`; keep the habit.

18.4 Repository layout, folder by folder

The monorepo follows docs/11 §14.1. Everything needed to build, test, package, evaluate and document VeriQ lives in one repository, so one commit identifies one consistent state of code, packaging, lab and documents.

```
veriq/
├── go.mod                 module github.com/<org>/veriq, Go 1.22+
├── cmd/veriqd/            daemon main: flags, wiring, signal handling
├── cmd/veriqctl/          CLI over /var/run/veriq.sock (JSON-RPC)
├── internal/              19 packages (chapter 19)
├── openwrt/               veriq, luci-app-veriq, veriq-household, feed/
├── ml/                    Python: datasets, training, conformal, export, honesty kit
├── lab/                   Python + shell: netns, ISP emulator, runner, evaluator, ledger
├── twin/                  3D twin (three.js) + replay loader
├── docs/                  specifications, handbook sources, figures
└── .github/workflows/     ci.yml, build.yml, lab.yml
```

Path	What lives there	Owner role (docs/11 §18)	Built and tested by
`go.mod`, `go.sum`	Module path, Go version, pinned dependencies with checksums	R2	Every Go job; `go mod verify`
`cmd/veriqd/`	`main.go` only: parse flags, load config, construct packages, start goroutines, handle SIGTERM (graceful) and SIGHUP (reload). No logic.	R2	Integration tests
`cmd/veriqctl/`	CLI commands of docs/11 §11.4; `-j` prints raw JSON for the rpcd plugin	R1	Unit tests for argument parsing; package tests
`internal/clock`, `model`, `config`	Foundations every other package imports	R2	`ci.yml` unit tests
`internal/observe`, `probe`, `estimate`, `diagnose`, `control`	The measurement and decision chain	R2	Unit, property and replay tests
`internal/classify`, `lease`, `sanction`	Classification, intent and envelope logic	R1 + R4 (ML)	Unit tests; golden vectors from `ml/`
`internal/checker`, `enforce`, `reconcile`, `state`	Safety core and kernel interaction	R2 + R1	One test per refusal code; netns integration
`internal/receipt`, `api`, `importer`, `telemetry`	Explanation, interfaces, migration, logs	R1 + R4	Unit tests; package tests
`openwrt/veriq/`	OpenWrt `Makefile` and `files/`: init script, default UCI config, uci-defaults, hotplug, rpcd plugin, ACLs, `keep.d`	R1	`build.yml` (SDK), VM tests
`openwrt/luci-app-veriq/`	`htdocs/luci-static/resources/view/veriq/*.js`, `menu.d`, `acl.d`, `po/{en,hi}`	R4	Playwright in the VM
`openwrt/veriq-household/`	`www/veriq/` static household page	R4	Playwright (mobile viewport)
`openwrt/feed/`	Feed index scripts, public signing keys, SDK lock file	R1	`build.yml` release job
`ml/`	Dataset builders, training, calibration, export, honesty-kit report	R4	`pytest`; golden-vector export consumed by Go tests
`lab/`	Topology, ISP emulator, traffic generators, scenario YAML, runner, evaluator, evidence ledger	R3	`lab.yml` (privileged runner)

Path	What lives there	Owner role (docs/11 §18)	Built and tested by
`twin/`	Three.js scene and replay loader for decision logs	R4	Manual demo checklist
`docs/`	Specifications, handbook Markdown, SVG figures, Makefile	All	Docs build job
`.github/workflows/`	CI definitions (below)	R1	—

Two rules keep the tree healthy: - **`internal/` is private by construction.** Go forbids imports of `internal/` packages from outside the module, so nothing outside VeriQ can depend on unstable APIs. - **Generated files are committed only when they are inputs to a test** (golden files, golden vectors). Build outputs (`.ipk`, `.apk`, binaries, PDFs) are CI artefacts, never committed.

18.5 Development environment setup

Five environments map onto the deployment views D1–D5 of docs/11 §3.3. Every developer needs D1; D2 runs in CI and on any laptop with KVM; D3–D5 are shared hardware.

18.5.1 Ubuntu laptop with a full kernel (D1)

A native Ubuntu 24.04 install (or a VM that allows nested network namespaces) is the main development machine. First confirm that the kernel has every module the datapath needs:

```
sudo apt install iproute2 nftables iperf3 iputils-ping ethtool conntrack \
     python3 python3-venv git make
for m in sch_cake sch_fq_codel sch_hfsc sch_htb act_ctinfo act_mirred ifb \
         nf_conntrack nfnetlink_log; do
  sudo modprobe "$m" && echo "ok   $m" || echo "MISSING $m"
done
# If anything is missing on stock Ubuntu:
#   sudo apt install linux-modules-extra-$(uname -r)
```

Then reproduce Day-0 before writing any code, so that the lab works on this machine:

```
sudo lab/quickstart_bufferbloat.sh fifo cake
```

18.5.2 Go toolchain

Install the Go release pinned in `go.mod` from go.dev [R43] into `/usr/local/go`, then:

```
go version                      # must match go.mod
go install honnef.co/go/tools/cmd/staticcheck@<pinned>
go install golang.org/x/vuln/cmd/govulncheck@<pinned>
go test ./...                   # unit tests, no root needed
sudo -E go test -tags netns ./internal/enforce/...   # kernel tests, root
```

Cross-compiling a router binary needs no C toolchain:

```
V=$(git describe --tags --always --dirty)
CGO_ENABLED=0 GOOS=linux GOARCH=arm64 \
  go build -trimpath -ldflags "-s -w -X main.version=$V" -o out/veriqd-arm64 ./cmd/veriqd
CGO_ENABLED=0 GOOS=linux GOARCH=mipsle GOMIPS=softfloat \
  go build -trimpath -ldflags "-s -w -X main.version=$V" -o out/veriqd-mipsle ./cmd/veriqd
```

`GOMIPS=softfloat` matters on MT7621-class devices, which have no hardware floating point.

18.5.3 OpenWrt SDK

Packages are built with the OpenWrt SDK for each target (docs/11 §12.1; OpenWrt SDK guide [R40]):

```
tar xf openwrt-sdk-24.10.*-x86-64_*.tar.zst && cd openwrt-sdk-24.10.*
./scripts/feeds update -a && ./scripts/feeds install luci-base
ln -s "$REPO/openwrt/veriq" package/veriq
ln -s "$REPO/openwrt/luci-app-veriq" package/luci-app-veriq
cp "$REPO/out/veriqd-amd64" package/veriq/files/veriqd   # prebuilt by CI
make defconfig && make package/veriq/compile V=s
```

The P0 package installs the CI-built static binary. For a later upstream submission the Makefile switches to `golang-package.mk`, which builds Go inside the OpenWrt build system (docs/11 §12.1).

18.5.4 OpenWrt VM (D2)

The x86-64 OpenWrt image runs in QEMU/KVM with two virtio NICs: one LAN, one WAN. In CI, the WAN side connects to the lab's ISP-emulator namespace through a tap device.

```
qemu-system-x86_64 -enable-kvm -m 512 -smp 2 -nographic \
  -drive file=openwrt-24.10-x86-64-generic-ext4-combined.img,format=raw,if=virtio \
  -netdev tap,id=lan,ifname=tap-lan,script=no,downscript=no \
  -device virtio-net-pci,netdev=lan \
  -netdev tap,id=wan,ifname=tap-wan,script=no,downscript=no \
  -device virtio-net-pci,netdev=wan
```

The image name is illustrative; use the exact file pinned in `openwrt/feed/sdk.lock`. Install the feed key, then `opkg install veriq luci-app-veriq`, and run `veriqctl health` and `veriqctl selftest`.

18.5.5 Raspberry Pi reference router (D3)

A Raspberry Pi 4 or 5 with a USB 3 Gigabit Ethernet adapter runs OpenWrt 24.10 (`bcm27xx` target). The on-board port is LAN; the USB adapter is WAN. Keep one SD card with a clean image for demos and one for development. Serial or HDMI console access is needed for watchdog tests, because a broken network config must not lock the team out.

18.5.6 Retail and low-end devices (D4, D5)

One ARM64 router with ≥ 512 MB RAM (Filogic 830 class) and one MT7621-class device run the Standard and Lite profiles respectively in Stage 4. They are flashed only after the VM and Pi tests pass. Keep the vendor firmware image and recovery instructions for each device next to it.

18.6 Coding standards

18.6.1 Go style

- `gofmt` formatting, `go vet` and `staticcheck` clean; CI fails otherwise.
- Package names are short nouns (`probe`, `checker`); no `util` or `common` packages.
- Units live in names: `RateKbit`, `DelayMs`, `Age time.Duration`. Rates are kbit/s as `float64` everywhere (docs/11 §14.2); conversion happens only at the tc renderer.
- All control timing uses the monotonic `clock.Clock` (chapter 19). Wall time appears only in logs, receipts and lease restore. A lint rule flags `time.Now()` outside `internal/clock` and `cmd/`.
- Exported identifiers have doc comments that state the unit and the owning goroutine.

18.6.2 Errors

- Return errors; never `panic` in control or enforcement code. `panic` is allowed only for programmer errors detected at start-up (for example a malformed built-in template).
- Wrap with context: `fmt.Errorf("enforce: tc batch step %d: %w", i, err)`. Define sentinel errors (`ErrStale`, `ErrForeign`) for conditions callers branch on, and test them with `errors.Is`.
- Every I/O goroutine recovers from panics, logs them, reports the component as failing to `state` (which can lead to DEGRADED) and restarts itself with capped backoff.
- A refusal by the Action Checker is **not** an error. It is a normal `Verdict` and is logged as a decision.

18.6.3 Logging

- Operational logs use the standard library `log/slog`, written to stderr; procd forwards them to the system log (`logread`). Default level `info`; `debug` only on request.
- Decisions are **not** operational logs. They go to the JSONL decision log (`telemetry`), one line per tick and direction, whose schema is fixed in docs/11 §14.3.
- Never log payload, full MAC addresses at `info` level, or anything under the privacy contract's "not allowed" column (docs/11 §5.1).

18.6.4 No cgo

`CGO_ENABLED=0` for every build. A dependency that needs cgo is rejected. This keeps the binary static, the cross-compilation trivial and the build reproducible.

18.6.5 Tests

- **Table-driven tests** for every pure function, with descriptive `name` fields that read as requirements ("cooldown does not refund budget").
- **Fake clock** for anything time-dependent; no `time.Sleep` in unit tests.
- **Golden files** for rendered nft/tc text and for importer output; update with `go test -update` and review the diff.
- **Golden vectors** from Python for the tree evaluator and the conformal set builder.
- **Fuzz tests** (`testing.F`) for parsers (UCI, tc JSON, nft JSON, JSON-RPC) and for controller properties (S_min ≤ S ≤ S_max, budget never exceeded).
- **Race detector** on in CI (`go test -race`).
- Coverage gate: ≥ 80% lines on `checker`, `control`, `diagnose`, `classify` (docs/11 §14.4); 100% of refusal codes have a test.

18.6.6 Concurrency rules

- One control goroutine owns all decision state. Other goroutines send typed messages over channels.
- No mutex in decision logic. Shared read-only data (compiled model, catalogue) is replaced by pointer swap on reload.
- Every goroutine takes a `context.Context` and exits on cancellation; `veriqd` shuts down within 2 s of SIGTERM (a target), leaving the kernel configuration in place.

18.7 Branching, commits and reviews

- **Trunk-based development.** `main` is always releasable. Work happens on short-lived branches (`feat/checker-refusals`, `fix/probe-wrap`), merged within a few days.
- **Pull requests** need one reviewer and green CI. Changes to `checker`, `control`, `enforce` or the nft/tc templates need a second reviewer from the owning role.
- **Commit messages** follow the Conventional Commits pattern (`feat(checker): add CUT_BELOW_EVIDENCE`), so the changelog can be generated.
- **Design changes** are not made in code review. A change to a contract in docs/11 goes into a short decision record and docs/12 first (docs/11 §20 "Decision: BUILD").
- **Releases** are signed tags (`v0.3.0`). The build job turns a tag into signed packages and a feed update. Version numbers follow semantic versioning; a change of algorithm or model version triggers SHADOW after upgrade (docs/11 §12.8).
- **Evidence runs** record the commit hash of `lab/` and of the system under test in the run manifest, so every published number regenerates (P9).

18.8 CI workflows

Workflow	Trigger	Jobs	Gate
`ci.yml`	Every push and pull request	`gofmt` check, `go vet`, `staticcheck`, `govulncheck`, `go test -race ./...`, coverage report, fuzz smoke (30 s per target), Python `pytest` for `ml/` and `lab/`, licence check of Go modules	All green; coverage gates; no new licence outside the allow-list
`build.yml`	Push to `main`, tags	Cross-compile `veriqd` / `veriqctl` for `x86_64`, `aarch64_cortex-a53`, `aarch64_cortex-a76`, `arm_cortex-a7_neon-vfpv4`, `mipsel_24kc`; SDK packaging (`.ipk`; `.apk` when available); binary size report; on tags: sign, SBOM (SPDX), publish feed	Every target builds; size within the profile's flash budget
`lab.yml`	Nightly, and on demand for PRs that touch `enforce`, `classify` or `lab/`	Netns integration (install, per-tin self-test, S1/S2/S6 smoke, kill → SAFE); OpenWrt VM boot, feed install, health/self-test, Playwright LuCI tests	Pass; time-to-safe ≤ 12 s (target)
Docs job (inside `ci.yml`)	Changes under `docs/`	pandoc build of the handbook; link and figure check; claims grep for unlabelled numbers	Builds without warnings

`lab.yml` needs root and kernel modules, so it runs on a privileged or self-hosted runner (docs/11 §14.4). Hosted runners are enough for `ci.yml` and `build.yml`.

18.9 Third-party modules and licences

VeriQ's own code is clean-room: QoSmate (GPL-3.0) and cake-autorate are used for ideas and as comparators only, never as code (docs/11 §13). Third-party Go modules linked into `veriqd` are restricted to **MIT, BSD or Apache-2.0** licences. The VeriQ licence itself is an open decision (O5: Apache-2.0 proposed; Tata Elxsi decides).

Module or component	Used in	Licence	Notes
`github.com/ti-mo/conntrack`	`observe` (events), `classify` / `sanction` (mark updates)	MIT	Named in docs/11 §14.2
`github.com/florianl/go-nflog`	`observe` (NFLOG group 7/8 reader)	MIT	Named in docs/11 §14.2
`github.com/mdlayher/netlink`	Transitive dependency of the two above	MIT	Verify on each `go.sum` update

Module or component	Used in	Licence	Notes
`golang.org/x/net` (`icmp`, `ipv4`, `ipv6`)	`probe`	BSD-3-Clause	Named in docs/11 §14.2
`golang.org/x/sys`	Transitive (sockets, capabilities)	BSD-3-Clause	—
Go standard library (`encoding/json`, `log/slog`, `net/netip`)	Everywhere	BSD-3-Clause	Preferred over any third-party equivalent
uPlot	`luci-app-veriq` charts	MIT	Shipped as a static file
LuCI framework	Admin UI host	Apache-2.0	Platform, not bundled
LightGBM, scikit-learn, NumPy, pandas	`ml/` (off-box only)	MIT / BSD-3-Clause	Never shipped to the router
three.js, react-three-fiber	`twin/`	MIT	Demo only
Playwright	VM and lab tests	Apache-2.0	Test only
Jitsi, dash.js, iperf3	Lab traffic generators	Apache-2.0 / BSD-3-Clause / BSD-3-Clause	Verify each LICENSE at the pinned version
QoSmate, cake-autorate, Flent, irtt	Lab comparators and tools, run unmodified as separate programs	GPL family (check each LICENSE)	Never linked, never copied; documented in `LICENSES.md`

Rules that CI enforces: - a licence scan of `go.sum` fails the build on any licence outside the allow-list; - every new dependency needs a one-line justification in the pull request (“why not the standard library?”); - `go.sum` and the Python lock files are committed, and module versions are pinned; - each release publishes an SPDX SBOM listing every shipped component and its licence.

> **Honesty check.** Licence identifiers in this table come from general knowledge of these projects. Each one must be confirmed against the LICENSE file at the pinned version before the first release, and the result recorded in `LICENSES.md`.

18.10 Sprint 0 bootstrap checklist

- [ ] Repository created private (submissions become Tata Elxsi IP; check the rules).
- [ ] `go.mod` with the module path and pinned Go version; empty `internal/` packages compile.
- [ ] `ci.yml` running `gofmt`, `vet`, `staticcheck`, `go test -race`.
- [ ] `clock`, `model` and `config` stubs merged with their first tests.
- [ ] `api` stub answering `version` over the Unix socket; `veriqctl version` works.
- [ ] Lab laptop passes the module check; Day-0 reproduced with a base RTT and the CAKE arm.
- [ ] SDK tarball pinned in `openwrt/feed/sdk.lock`; an empty `veriq` package installs in the VM.
- [ ] `LICENSES.md` and the licence allow-list in place before the first dependency is added.
- [ ] Branch protection on `main`; commit convention documented in `CONTRIBUTING.md`.

CHAPTER 19

Module Implementation Guide

This chapter is the build manual for `veriqd`. It takes each of the 19 packages under `internal/` in docs/11 §14.1 and states, for each one: - its purpose; - its public API as Go signatures; - its internal design and key algorithm; - how it handles errors; - the tests to write, by name; - the acceptance criteria; - an effort estimate.

Six pieces are given as full Go skeletons, because they carry the most risk and are easiest to get subtly wrong: - `checker.Check`, with every refusal code; - the control tick; - the reflector median and quorum; - the windowed-minimum baseline; - the conformal set builder; - the tree-ensemble evaluator.

The skeletons were written to compile against the types of docs/11 §14.2, but they are **starting code, not tested product code**. Import paths are shortened to `veriq/internal/...`; use the module path from `go.mod`. Every numeric constant is a **starting parameter** from docs/11 or docs/09 Appendix A, to be tuned in Stage 2 and recorded in the run manifest.

> **Note.** Effort figures are **assumptions** in person-days for one developer who already knows Go basics. They include unit tests but not lab evaluation. Treat them as planning inputs for chapter 22, not commitments.

19.1 How the packages fit together

19.1.1 Dependency rules

Packages form layers, and imports only point downwards. CI checks this with a small import-graph test.

```
cmd/veriqd, cmd/veriqctl
        │
   api ─┼─ importer ─ telemetry
        │
 control ── receipt ── lease ── sanction
   │  │                  │
   │  checker (imports model only; pure)
   │
 diagnose ── estimate ── probe ── observe
        │
 enforce ── reconcile ── state ── classify
        │
 config ── model ── clock          (imported by everyone)
```

- `model` and `clock` import nothing from VeriQ.
- `checker` imports only `model`. It must stay pure: no I/O, no clock, no logging. This is what makes it fully table-testable.
- `enforce` is the only package that runs `tc` or `nft` with side effects. `observe` and `reconcile` run them read-only.
- `api` never touches decision state. It sends requests to the control goroutine over a channel and waits for a reply.

19.1.2 Goroutines and channels in `cmd/veriqd`

`main.go` only wires the packages together. The runtime shape follows docs/11 §14.2: one control goroutine owns all decision state; I/O goroutines feed it typed messages.

```
probe.Run ──────── Result ──────┐
observe.TCSampler ─ QdiscStats ─┤
observe.Conntrack ─ FlowEvent ──┤
observe.NFLOG ──── PktHdr ──────┼──► control loop (single goroutine)
observe.WiFi ───── []StationStat┤      every 250 ms: build Snapshot,
hotplug/api ────── Call ────────┘      Tick, reconcile, receipts,
                                       heartbeat, telemetry.Append
telemetry.Exporter ◄── read-only copies (pointer swap per tick)
```

Each input channel is buffered and **lossy by design**: if the control goroutine falls behind, the reader drops the oldest message and counts the drop. A tick never waits for I/O. Missed inputs show up as stale samples (`Sample.OK = false` or an old `Sample.T`), which the checker and diagnosis already treat conservatively.

19.1.3 Build order

The order follows docs/11 §18. Sprint 0: `clock`, `model`, `config` and an `api` stub. Sprint 1: `enforce` (cake), `observe`, `probe`, the baselines in `estimate`, `telemetry`. Sprint 2: `diagnose`, `control`, `checker`, `reconcile`, `state`. Sprint 3: `classify` (rules, sets), `lease`, `sanction`, `importer`. Sprint 4: `receipt`. P1: the ML part of `classify` and the `hfsc`/`htb` backends.

19.2 clock

Purpose. Give every package a monotonic time source that tests and replay can control. Control timing must never use wall time, so an NTP step has no effect (A18 in docs/11 §10).

Public API.

```
package clock

type Clock interface {
    Now() time.Duration // monotonic, since process start
    After(d time.Duration) <-chan time.Duration
    NewTicker(d time.Duration) Ticker
}

type Ticker interface {
    C() <-chan time.Duration
    Stop()
}

func NewReal() *Real      // wraps time.Since(start): Go's monotonic reading
type Fake struct{ /* now, pending timers */ }
func (f *Fake) Advance(d time.Duration) // fires due timers in order
func Wall() time.Time    // the only wall-clock accessor; logs, receipts, leases
```

Internal design. `Real.Now` returns `time.Since(start)`. Go's `time.Time` carries a monotonic reading, so the difference is immune to wall-clock steps. The real ticker **drops** a tick rather than queueing a backlog. The fake clock keeps a sorted list of pending timers and fires them deterministically when `Advance` is called.

Errors. None; the package cannot fail.

Tests. `TestFakeAdvanceFiresInOrder`, `TestTickerDropsWhenSlow`, `TestRealMonotonicAcrossWallStep` (runs in a netns test with `date -s`).

Acceptance. No other package calls `time.Now()` for control decisions. A lint rule enforces this.

Effort (assumption). 1 day.

19.3 config

Purpose. Parse `/etc/config/veriq` (UCI format), apply defaults, validate every option against the schema in docs/11 Appendix B, and produce one immutable `*Config` with derived limits (S_min, S_max). It also migrates old `config_version` values.

Public API.

```
package config

type Config struct {
    Global     Global
    WAN        WAN
    Shaping    Shaping
    Cake       Cake
    HFSC       HFSC
    HTB        HTB
    Controller Controller
    Probe      Probe
    Classify   Classify
    Envelope   Envelope
    Leases     Leases
    Privacy    Privacy
    Alerts     Alerts
    Devices    []Device
    Rules      []Rule
    IPSets     []IPSet
    RateLimits []RateLimit
    Hash       string // sha256 of the canonical form; goes into receipts
}

type Issue struct {
    Section, Option string // reported as section.option, e.g. "shaping.backend"
    Msg             string
    Fatal           bool
}

func Parse(r io.Reader) (*File, error)                  // UCI syntax only
func Load(f *File) (*Config, []Issue)                    // defaults + validation
func LoadFile(path string) (*Config, []Issue, error)
func Migrate(f *File) (*File, []Issue, error)            // config_version → 5
func (c *Config) Envelope(d model.Dir) (min, base, max float64) // kbit/s
```

Internal design. - A hand-written UCI lexer handles `config <type> ['<name>']`, `option`, `list`, single and double quotes, and comments. No external library is needed. - `veriqd` reads the **committed** file, never `/tmp/.uci` staging, so a half-edited LuCI form can never reach the daemon. - Validation is table-driven: one row per option with type, range and allowed values. - Cross-field rules: - `min_frac · base`

`≥ 1000` kbit/s; - a backend's kernel module is present; - `profile = lite` forces `classify.ml = 0`; - **S_min ≤ 0.8 ×** the lowest capacity in the configured scenario set, when a scenario set is given (docs/09 A.2). - Derived values: `S_min = max(1000, min_frac·base)` and `S_max = max_frac·base`.

Errors. A fatal issue makes `veriqd` refuse the new config and keep running on the previous one. Reload therefore never makes things worse. `veriqctl config validate` prints every issue as `section.option: message`. Non-fatal issues (for example an unknown option) become warnings shown in LuCI.

Tests. `TestParseQuotesAndLists`, `TestParseRejectsUnterminatedQuote`, `TestDefaultsMatchAppendixB`, `TestSminFloor1000`, `TestLiteForcesMLOff`, `TestMigrateV4ToV5`, `TestIssueReportsSectionOption`, and `FuzzParse` (the parser must never panic).

Acceptance. - Appendix B parses to a `Config` whose fields equal a golden JSON dump. - 10^6 fuzz cases run without a panic. - An invalid reload leaves the running config unchanged (integration test).

Effort (assumption). 5 days.

19.4 model

Purpose. Hold the shared value types of docs/11 §14.2: `Dir`, `Class`, `Prov`, `Level`, `Label`, `OpState`, `Sample`, `Snapshot`, `Diagnosis`, `ActionType`, `Action` and `Verdict`. It has no logic beyond string conversion and JSON encoding.

Public API.

```
package model

type Dir uint8   // Up, Down
type Class uint8 // None=0, BK=1, BE=2, IV=3, RT=4 (same values as CLASS bits 24–26)
type Prov uint8  // None, Default, Heuristic, ML, AppMark, UserRule, Lease, System

const (
    ClassMask = 0x07000000 // CLASS, bits 24–26
    ProvMask  = 0x38000000 // PROV, bits 27–29
    SanctBit  = 0x40000000 // SANCT, bit 30
)

func (c Class) Mark() uint32 { return uint32(c) << 24 }
func (p Prov) Mark() uint32  { return uint32(p) << 27 }
func ClassOf(ctmark uint32) Class { return Class((ctmark & ClassMask) >> 24) }
func ProvOf(ctmark uint32) Prov   { return Prov((ctmark & ProvMask) >> 27) }

func (l Label) String() string // "WAN_LIKELY", ... as in the decision log
func (l Label) MarshalJSON() ([]byte, error)
func ParseLabel(s string) (Label, error)
```

Internal design. The numeric values of `Class` and `Prov` **are** the kernel mark bits (docs/11 §4.2), so conversion is a shift. JSON uses the upper-case names of the decision-log schema (docs/11 §14.3), so logs stay readable and stable when constants are reordered. `Snapshot` is passed by value and treated as immutable; slices inside it (`WiFi`) are copied on construction.

Errors. `ParseLabel` and friends return an error for an unknown name. Replay of an old log with an unknown label fails loudly rather than guessing.

Tests. `TestMarkRoundTrip` (all 8 × 8 class/prov pairs), `TestMarkNeverTouchesOtherBits` (bits 0–23 and 31 are preserved), `TestLabelJSONRoundTrip`, `TestSnapshotCopyIsDeep`.

Acceptance. The mark constants equal the values the nft renderer emits, checked by a shared golden test with `enforce`.

Effort (assumption). 1 day.

CHAPTER 20

Chapter 20 (pending)

This chapter has not been written yet.

CHAPTER 21

Chapter 21 (pending)

This chapter has not been written yet.

CHAPTER 22

Build plan

This chapter turns the v5 design into dated work. It covers the priority tiers (P0/P1/P2), the six sprints with a task-level breakdown, roles and a RACI (Responsible, Accountable, Consulted, Informed) matrix, team rituals, the definition of done, the critical path, the buffers, and an ordered scope-cut ladder for when the team falls behind.

Every effort figure in this chapter is a **planning assumption**, not a measurement. The sprint dates and exit criteria come from the authoritative design (docs/11 §18). The task breakdown, owners, dependencies and done criteria are this chapter's refinement of that plan.

22.1 Planning assumptions

The plan is only as good as its inputs. These are the inputs used here. Change them in the team's tracker if reality differs, and re-run the capacity check at the start of each sprint.

Item	Assumption used in this chapter	Why it matters
Team size	3 people, part-time. docs/01 records the TELIPORT rule as teams of 2–3 from the same college (confirm on the portal); docs/11 plans roles for 3–4	Four roles must fit into three (or two) people
Availability	About 18 focused hours per person per week alongside coursework (assumption)	Sets sprint capacity
Effort unit	1 person-day (pd) = 6 focused hours (assumption)	All task estimates use pd
Sprint capacity	3 people × 3 pd/week × 2 weeks ≈ 18 pd per two-week sprint (assumption)	Used for the load check per sprint
Lab machine	One Ubuntu 24.04 laptop with a full kernel (CAKE, netem, ifb, act_ctinfo available), able to run unattended overnight	H1 runs need machine time, not people time
Hardware	Raspberry Pi 4/5 and USB 3 GbE adapter ordered in Sprint 0	Package tests on D3 in Sprint 3
Code freeze for evaluation	The build evaluated in H1 is tagged before the full runs start	Results must refer to one pinned version

Honesty check. The load tables below show that Sprints 1–3 are **over capacity** under these assumptions. That is deliberate: the plan shows the real size of the work, and the scope-cut ladder at the end of the chapter says what goes first. A plan that fits on paper by hiding work is worse than one that admits overload.

22.2 Priorities: P0, P1, P2

Priority	Deadline	Contents (docs/11 §18)	Rule
P0	Round 2 submission, by 13 Dec 2026 (window 14 Nov–13 Dec; confirm on the portal)	The core and the minimum product: CAKE backend, mark plumbing, probing, diagnosis, rate controller, checker, reconciler, watchdog/SAFE, operating states, rules/sets/leases/envelope/heuristics, QoSmate importer, OpenWrt packages (VM + Pi), LuCI Overview/Autorate/Rules/Connections, receipts, H1 evaluation, report, video	Nothing in P1 starts while a P0 exit criterion is failing
P1	Finale (date to be confirmed on the portal)	ML classifier with conformal abstention; hfsc/htb backends; household page; Hindi; app catalogue; H2/H3 tiers; bench; UX study; retail router (D4); Lite profile on D5; 3D twin; physical kit	Each P1 item needs its own done criterion and must not regress a P0 result
P2	Post-competition roadmap	Exporter, public signed feed, apk line, prplOS/USP, RDK-B wrapper, hybrid backend, multi-queue CAKE, NQB/L4S experiments, home pilots	Listed in the submission as roadmap only; never presented as built

22.3 Sprint calendar

Sprint	Dates	Theme	Exit criterion (docs/11 §18)
0	Mon 12 Oct – Sat 24 Oct	Round 1, foundations, lab on a full kernel	Round 1 submitted; `go test ./...` green; lab FIFO vs CAKE run with a base RTT
1	Sun 25 Oct – Sat 7 Nov	Datapath, probing, V1 harness, baselines installed	Per-tin counters match intent in both directions; first call + bulk run reproducible
2	Sun 8 Nov – Sat 21 Nov	Diagnosis, controller, checker, safety; first LuCI pages	Stage 1 passes; H0 smoke; kill → SAFE ≤ 12 s (target)
3	Sun 22 Nov – Sat 5 Dec	Classification, importer, packages, H1 pilot	Packages install on the VM and the Pi; H1 n chosen
4	Sun 6 Dec – Sun 13 Dec	H1 full runs, receipts, report, video, submission	Every published number regenerates from the ledger
5+ (P1)	After Round 2 until the finale (TBC)	ML, extra backends, household page, demo assets, H2/H3, UX study	Finale demo

Key external dates: Round 1 is due **24 Oct 2026** (confirm on the portal; docs/01 notes conflicting listings). The Round 2 window is **14 Nov–13 Dec 2026**. The finale date is TBC. The internal deadlines are two days earlier than the external ones (see “Buffers”).

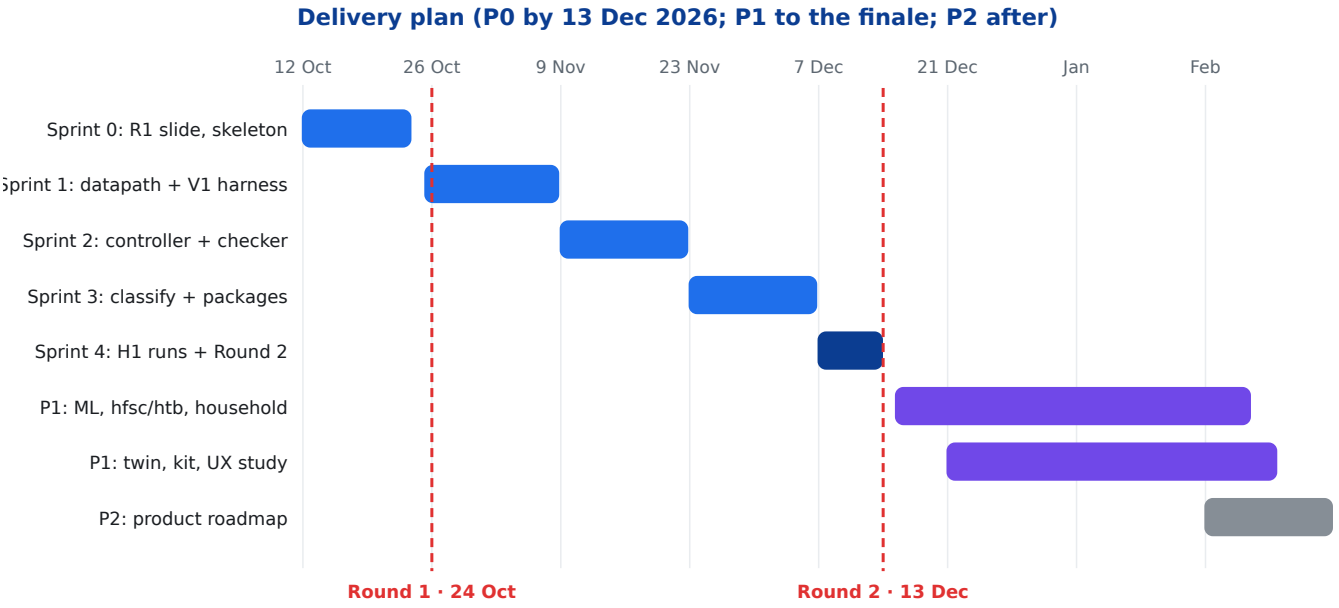


Figure 12. Sprint timeline from October 2026 to the finale, showing Sprints 0–5, the Round 1 and Round 2 deadlines, the evaluation freeze and the P1 track.

22.4 Task breakdown per sprint

Each table lists the task, the owner role (R1–R4, defined under “Roles”), the effort assumption in person-days, the dependencies and the done criterion. Task IDs are stable so the tracker, the decision log and the commit messages can refer to them.

22.4.1 Sprint 0: 12–24 Oct (Round 1 and foundations)

ID	Task	Owner · effort	Depends on	Done when
0.1	Confirm registration, Round 1 format and deadline on the official portal; record in the decision log	Lead · 0.2 pd	—	Decision-log entry with a screenshot of the portal page
0.2	Update the Round 1 slide (`submission/Round1_Case3_VeriQ.pptx`): one-line pitch, three layers, Day-0 numbers labelled “one preliminary run, no base RTT, IPv4 only”	R4 · 1.5 pd	0.1	PDF exported; claims check done on every number
0.3	Prepare the full-kernel Ubuntu 24.04 lab laptop; verify `modprobe sch_cake act_ctinfo ifb sch_netem`; record the environment fingerprint	R3 · 1 pd	—	Fingerprint file lists every capability as present
0.4	Re-run `lab/quickstart_bufferbloat.sh` with a base RTT applied and a CAKE arm; 3 runs per arm as a smoke check	R3 · 1 pd	0.3	Raw files in the ledger; summary regenerates with `lab/summarize.py`

ID	Task	Owner · effort	Depends on	Done when
0.5	Create the monorepo skeleton (docs/11 §14.1), `go.mod`, linters, `ci.yml` (lint, unit, race)	R1 · 1.5 pd	—	CI runs on every push and is green
0.6	Go packages `clock` (monotonic + fake), `model` (core types), `config` (UCI parser skeleton), `api` stubs (`status`, `version` over the Unix socket)	R2 · 3 pd	0.5	`go test ./...` green; `veriqctl version` answers
0.7	Go ramp-up check for every member (one table-tested function each); decide O7 (Python controller fallback) by 24 Oct	Lead · 0.5 pd	0.6	O7 decision recorded with the reason
0.8	Clean-room setup: `LICENSES.md`, rule that no GPL source is opened while writing the matching VeriQ module, pinned QoSmate commit noted	R1 · 0.5 pd	—	Policy committed; every member has read it
0.9	Draft the scenario YAML schema and the evidence-ledger layout (manifest, raw, derived)	R3 · 1 pd	0.3	Schema validates an example S2 file
0.10	Order the Pi, USB GbE adapter and switch (approximate prices in the business-case chapter)	R1 · 0.3 pd	—	Order placed; expected delivery logged

Load check: about 10.5 pd against a capacity of about 18 pd. Sprint 0 is light on purpose, because Round 1 and coursework compete for the same days.

22.4.2 Sprint 1: 25 Oct – 7 Nov (datapath, probing, V1 harness)

ID	Task	Owner · effort	Depends on	Done when
1.1	`enforce/cake`: `Render`, `RateChange`, `Readback` (parse `tc -s -j`) for egress and `ifb-veriq`	R1 · 3 pd	0.5, 0.6	Unit tests with recorded `tc -j` fixtures; install/remove leaves no residue
1.2	nft renderer: table `inet veriq` skeleton (`cls_up`, `mark_out`, lease and sanction sets), `nft -c` validation, atomic apply	R1 · 2.5 pd	0.6	Golden-file tests; an invalid rule keeps the previous ruleset
1.3	Download mark plumbing: ingress filter with `ctinfo cpmark + mirred` to `ifb-veriq`; verify syntax with tc-ctinfo(8) [R7]; decide O3 compat mode if broken	R1 · 1.5 pd	1.1	Marks visible on IFB packets in a capture
1.4	Tin self-test in netns: inject probe packets per class; check per-tin counters in **both** directions; confirm the fwmark → tin mapping (O2)	R1 · 2 pd	1.1–1.3	Self-test passes and is part of Stage 1
1.5	`observe/tcstats` and `nftstats`: per-tick sampling, overhead-compensated G_gw	R2 · 1.5 pd	1.1	G_gw within ±2% of iperf3-implied wire rate in a static run (target)
1.6	`probe`: ICMP echo + timestamp over raw sockets, reflector manager, qualification (10 + 10 probes, ≥ 90% replies, idle std < 5 ms)	R2 · 3 pd	0.6	Qualification output for the lab reflectors; midnight wrap test passes
1.7	`estimate`: per-reflector windowed-min baseline (C8), health and MAD outlier rule, median aggregation with quorum ≥ 2	R2 · 2 pd	1.6	Unit tests incl. one lying reflector and an upward route change
1.8	`telemetry`: decision-log JSONL ring (≤ 2 MB)	R2 · 1 pd	0.6	Log lines match the docs/11 §14.3 schema

ID	Task	Owner · effort	Depends on	Done when
1.9	V1 harness: netns topology, ISP emulator (tbf + netem schedules), Jitsi with JVB on the WAN side and p2p disabled, dash.js with local content, iperf3 bulk, WebRTC getStats collection, ledger manifest	R3 · 5 pd; R4 · 2 pd	0.3, 0.9	First call + bulk run reproduces: same scenario file gives the same metric set
1.10	Baseline installers: SQM static CAKE, cake-autorate (pinned), QoSmate v0.5.48 (+ autorate) in the OpenWrt VM where it needs OpenWrt services; configs archived	R3 · 2 pd	1.9, 1.11	Each arm passes Stage 1 (self-test, offloads off, media crosses the bottleneck)
1.11	OpenWrt 24.10 ×86-64 VM (D2) bring-up, attached to the lab topology	R4 · 1 pd	0.3	VM routes lab traffic; `tc -s qdisc` readable

Load check: about 26.5 pd against about 18 pd. This is the first overloaded sprint. The harness (1.9) and mark plumbing (1.3–1.4) are on the critical path and get first claim on time. If 1.10 slips, the QoSmate installer may finish in the first days of Sprint 2, but SQM and cake-autorate must be ready by 7 Nov, because H0 needs them.

22.4.3 Sprint 2: 8–21 Nov (controller and safety)

ID	Task	Owner · effort	Depends on	Done when
2.1	`checker.Check` as a pure function with one table test per refusal code (docs/09 A.1 + docs/11 §8.1)	R2 · 3 pd	0.6	100% of refusal codes covered; ≥ 80% line coverage (target)
2.2	`estimate` detectors (bufferbloat, own queue, censoring, near rise, CPU-bound, link down) and the μ/σ estimator	R2 · 2 pd	1.5, 1.7	Detector unit tests on recorded traces
2.3	`diagnose`: evidence groups → label + level, missing-near rules, direction attribution (C9)	R2 · 3 pd; R3 · 1 pd (test traces)	2.2	Label tests for WAN, REMOTE, WIFI, CPU, PROBE_FAULT, MIXED cases
2.4	`control` FSM: HOLD/PROBE_UP/BACKOFF/DRIFT, C1 one-step cut, fallback episodes with rolling budget and non-refunding cooldown; property tests (S_min ≤ S ≤ S_max)	R2 · 3 pd	2.1, 2.3	Fuzz property passes 10^6 cases with 0 failures (target)
2.5	`reconcile`: desired vs observed, ownership by comment `veriq:vN`, self-heal of a flushed table, foreign change → DEGRADED	R1 · 2.5 pd	1.1, 1.2	Firewall-restart test restores the table within one tick (target)
2.6	`state`: INIT/SHADOW/LIVE/DEGRADED/HOLD_LINKDOWN/SAFE/DISABLED, atomic persistence	R1 · 1.5 pd	0.6	Transition table tests pass
2.7	`veriq-watchdog` shell loop, `safe.env` writer, kill test	R1 · 1 pd	2.6	`kill -9` → SAFE in ≤ 12 s (target) over 10 trials
2.8	Replay determinism: `veriqctl replay log.jsonl` reproduces decisions	R2 · 1 pd; R3 · 0.5 pd	2.4	Replay of a recorded run is bit-identical
2.9	LuCI Overview and Autorate pages; rpcd plugin `luci.veriq` calling `veriqctl -j`	R4 · 4 pd	0.6, 1.11	Pages load in the VM; values match `veriqctl status`
2.10	Stage 1 for every arm; H0 smoke (S1, S2, S6 × all arms × 1 run)	R3 · 2 pd	1.10, 2.4	Stage-1 report archived; H0 runs in the ledger
2.11	Scenario YAMLs for the H1 set (S1, S2, S4, S6, S7, A6, A7, A9, A14, A32) with ground-truth channels; evaluator metrics (primary triplet, confirmation time, bulk share of leftover)	R3 · 3 pd	0.9, 1.9	Evaluator reproduces metrics from raw files with one command

Load check: about 27.5 pd against about 18 pd, and R2 alone carries about 12 pd. This sprint is the tightest point of the whole plan. The mitigations are: R3 writes the diagnosis test traces (2.3), R1 owns all safety work (2.5–2.7), and the LuCI pages (2.9) are the first P0 item on the cut ladder. The Round 2 window opens on 14 Nov in the middle of this sprint; nothing is submitted then.

22.4.4 Sprint 3: 22 Nov – 5 Dec (classification, packages, H1 pilot)

ID	Task	Owner · effort	Depends on	Done when
3.1	`classify` rule compiler: QoSmate-compatible rule options + device/app/schedule targets; kernel precedence with PROV comparison	R1 · 3 pd	1.2	Golden render tests; precedence property test (higher PROV never overwritten)
3.2	Heuristics H1, H2, H5 in nft (H3 off on CAKE)	R1 · 1 pd	3.1	Unit tests; bulk flow demoted in an S1 run
3.3	`lease`: table, admission (≤ 2 active, ≤ 1 per device, collapse refusal), set elements with timeout, persistence	R2 · 2 pd	2.1, 3.1	Lease expires in the kernel with the daemon stopped
3.4	`sanction`: envelope meters, NFLOG group 8 events, escalation (×2, cap 15 min)	R2 · 1.5 pd	3.1	S11 smoke: EF-marked bulk flow sanctioned within 1 s (target)
3.5	`importer` for QoSmate configs (rules, ipsets, ratelimits, settings), written from the README's documented format; `--dry-run`	R4 · 2 pd	3.1	Import of the QoSmate default config gives an equivalent rule set
3.6	LuCI Rules and Connections pages (provenance "why" column)	R4 · 3 pd	2.9, 3.1	Pages load; a rule edit round-trips through `nft -c`
3.7	OpenWrt packages: Makefile, procd init script with watchdog instance, uci-defaults, hotplug, keep.d; SDK builds for x86_64, aarch64_cortex-a53 and aarch64_cortex-a76	R1 · 3 pd	2.6, 2.7	`opkg install` succeeds on the VM and the Pi; health check passes
3.8	H1 pilot: n = 5 per arm per scenario; compute the n needed for the target CI half-width	R3 · 3 pd + machine time	2.10, 2.11	Pilot report with chosen n (expected 10–20)
3.9	Evaluation freeze: tag the build that H1 will evaluate (`eval-r2`), with the config and model versions recorded	Lead · 0.2 pd	3.1–3.4	Tag exists; the manifest records it
3.10	Submission skeleton and video storyboard	R4 · 1 pd	—	Outline approved by the team

Load check: about 20 pd against about 18 pd. The evaluation freeze (3.9) should happen by **Wed 2 Dec**, so that the full H1 runs can start before Sprint 4.

22.4.5 Sprint 4: 6–13 Dec (H1, receipts, submission)

ID	Task	Owner · effort	Depends on	Done when
4.1	H1 full runs, automated overnight, counterbalanced order	R3 · 2 pd + machine time	3.8, 3.9	All runs in the ledger, including failed or missing runs, reported as such
4.2	`receipt` assessor (IMPROVED/NOT_IMPROVED/INCONCLUSIVE) and explain templates	R2 · 2 pd	2.4	Receipts reproduce from the ledger
4.3	Statistics: bootstrap CIs (10 000 resamples), Holm correction, non-inferiority checks; plots generated from the ledger	R3 · 2 pd	4.1	One command regenerates every table and figure
4.4	Claims check: every number in the submission traced to a ledger entry or labelled target/assumption/illustrative	Lead + all · 1 pd	4.3	Claims sheet 100% traced
4.5	Demo video (recorded from real runs)	R4 · 2 pd	4.1	Honesty labels on every number shown
4.6	Submission document and deck	All · 3 pd	4.3	Internal review passed

ID	Task	Owner · effort	Depends on	Done when
4.7	Submit Round 2 on **Fri 11 Dec** (internal deadline)	Lead	4.4–4.6	Portal confirmation saved

Load check: about 12 pd of people time against about 10 pd available in these eight days, on top of the machine time. Writing (4.6) should start from the skeleton made in 3.10, not from a blank page.

Machine-time check (assumption). H1 has 10 scenarios. With 5 arms (SQM, cake-autorate, QoSmate whole-system, QoSmate controller-only, VeriQ), n = 10 and about 5 minutes per run including warm-up and reset, H1 needs 10 × 5 × 10 × 5 min ≈ 42 hours of lab time. At n = 20 it needs about 83 hours. Starting on 2 Dec and running about 20 hours a day gives roughly 180 hours by 11 Dec. That is enough, but only if the runner is fully unattended and restarts cleanly after a failure.

22.4.6 Sprint 5+ (P1): after Round 2 until the finale

The finale date is not yet known, so P1 tasks are planned in weeks relative to it (F − n = n weeks before the finale).

ID	Task	Owner · effort	Depends on	Done when
5.1	ML classifier: dataset builders, LightGBM training, Mondrian split conformal calibration, Go tree evaluator with golden vectors, honesty-kit report	R4 · 8 pd	3.1	Coverage, selection rate, selective accuracy and abstention reported per class, incl. shift tests
5.2	hfsc and htb backends with ordered rate changes and read-back	R1 · 4 pd	1.1	Tin/class self-test passes per backend
5.3	Household page (`/veriq/`), rpcd household role, English + Hindi strings	R4 · 4 pd	3.3	Three screens work on a phone; ACL limits verified
5.4	App catalogue (signed JSON, vendor-cited entries)	R4 · 2 pd	3.1	Each entry has a source and a `verified` date
5.5	H2 and H3 tiers; bench; H4 soaks on D3/D4/D5	R3 · 8 pd + machine time	4.1	Results in the ledger, reported with ties and losses
5.6	UX study (6–8 participants, SUS, formative)	R4 · 3 pd	5.3	Report with task success, time and SUS
5.7	Retail router (D4) and Lite profile on D5	R1 · 3 pd	3.7	Footprint measured against the budgets
5.8	3D twin (replay from the decision log)	R4 · 5 pd	1.8	Replays a real H1 run with measured/estimated/illustrative tags
5.9	Physical kit (ESP32 knob, buttons, LED strips)	R1 · 4 pd	3.7	Judge-controlled faults work offline
5.10	Finale rehearsal: 3 full dry runs, Q&A drill	All · 2 pd	all	Last two rehearsals complete without intervention

Suggested order: 5.2 and 5.9 by F − 4; 5.1, 5.3, 5.4 by F − 3; 5.5 and 5.6 by F − 2; 5.8 by F − 2; rehearsals in the final two weeks.

22.5 Roles

Role	Owns (docs/11 §18)	3-person team	2-person team
R1 Datapath & OpenWrt	nft/tc renderers, backends, packages, self-test, reconciler, states, watchdog	Person A	Person A
R2 Controller	probe, estimate, diagnose, control, checker, leases, sanctions, receipts	Person B	Person B
R3 Evaluation	lab, scenarios, baselines, statistics, ledger	Person C	Person A + B (alternate weeks)
R4 UX, ML & demo	LuCI, household page, importer, ML, twin, video	Shared: A takes LuCI, C takes video and twin, B takes ML (P1)	Shared; P1 ML moves to P2

Two hats rotate weekly across the team: - **Lead:** runs the weekly demo, owns the tracker and the external deadlines. - **Claims steward:** checks every new number in slides, documents and the handbook against the ledger or its label.

22.6 RACI matrix

R = does the work, A = accountable (one per row), C = consulted, I = informed.

Deliverable	R1	R2	R3	R4
Round 1 slide	C	C	C	A/R
Datapath (nft, tc, IFB, marks, self-test)	A/R	C	C	I
Probing, estimation, diagnosis, controller	C	A/R	C	I
Action Checker and its tests	C	A/R	C	I
Reconciler, states, watchdog, SAFE	A/R	C	C	I
Classification, leases, sanctions, importer	R	R	C	A (importer R)
OpenWrt packages and CI builds	A/R	I	C	C
LuCI pages and household page	C	C	I	A/R
Lab, scenarios, baselines, statistics	C	C	A/R	I
Claims check and evidence ledger	R	R	A/R	R
Demo video and submission	C	C	R	A/R
Physical kit and 3D twin (P1)	R	I	C	A/R

22.7 Rituals

Ritual	When	Length	Output
Async check-in	Daily, in the team chat	2 minutes each	Yesterday, today, blockers
Weekly demo	Every Saturday	45 min	Something runs live: a test, a run, a page. No slides
Sprint review and planning	Last Saturday of each sprint	90 min	Exit criterion checked; next sprint's load check
Decision log	Whenever a decision is made	—	Entry with context, options, decision, reason, revert condition (the docs/12 format)
Claims check	Weekly, and before every submission	30 min	Claims sheet updated
Risk review	At sprint planning	15 min	Risk register updated (see the Risks chapter)

The decision log is where open decisions O1–O10 are closed. **Rule:** no architecture change after the Sprint 2 exit criterion unless a measurement forces it, and then it is recorded as a change record entry.

22.8 Definition of done

A task is done only when every applicable item holds:

- The code is merged to the main branch through review by one other member.
- Unit tests exist and pass in CI with `-race`; checker, control, diagnose and classify keep ≥ 80% line coverage (target).
- No cgo; third-party modules are MIT/BSD/Apache only.
- Logging and decision-log fields are present for every action the code can take.
- The failure behaviour is implemented and tested (what happens when the input is missing or stale).
- The UCI options, if any, are added to the schema and validated.
- Docs are updated (module README or handbook note).
- Any number produced is in the ledger with its manifest, or labelled as a target or assumption.
- No secrets in the repository; example secrets are clearly fictitious.
- The weekly demo has shown it running.

22.9 The critical path

The longest dependency chain runs from the full-kernel lab to the Round 2 submission:

```
0.3 full-kernel lab
  |-> 1.9 V1 harness -> 1.10 baselines installed ----------------+
  |                                                              |
  +-> 1.1-1.4 CAKE backend + mark plumbing + self-test           |
       -> 1.5-1.7 observe, probe, baselines                      |
            -> 2.2-2.4 detectors, diagnosis, controller (+ 2.1)  |
                 -> 2.10 Stage 1 + H0 smoke  <-------------------+
                      -> 3.8 H1 pilot (n = 5)
                           -> 3.9 evaluation freeze (by Wed 2 Dec)
                                -> 4.1 H1 full runs (~42-83 h machine time)
                                     -> 4.3 statistics -> 4.4 claims check
                                          -> 4.7 submit (Fri 11 Dec)
```

Off the critical path, with some float: - LuCI pages (2.9, 3.6): about one week of float; first on the cut ladder. - Classification (3.1–3.4): must be in the evaluation freeze only for H2 scenarios, which are P1; for H1 it needs only the default classification and heuristics. - Packages (3.7): float until the submission, but the Pi install must work before the video.

> **Why it matters.** The controller work in Sprint 2 sits on the critical path and is the most overloaded block of the plan. Any slip there moves the evaluation freeze, and the H1 machine time cannot be compressed below the runs it needs. Protect R2's time first.

22.10 Buffers

- **Internal deadlines two days early:** Round 1 internal deadline Thu 22 Oct; Round 2 internal deadline Fri 11 Dec.
- **Machine-time buffer:** the evaluation freeze on 2 Dec leaves about twice the lab time needed for n = 10 and close to the time needed for n = 20.
- **Hardware buffer:** the Pi is ordered in Sprint 0, and every Sprint 3 package test can fall back to the VM (D2) if delivery slips.
- **Language buffer:** the O7 decision on 24 Oct allows the controller to be built in Python inside the lab for P0, with the same interfaces and golden tests, and ported to Go in P1.
- **No hidden buffer inside estimates.** Estimates are honest medians; the cut ladder is the buffer for overload.

22.11 Scope-cut ladder

Apply the steps in order. Each step is triggered by a dated condition, so the team does not argue about it under pressure.

Step	Trigger	Cut	Kept	Cost to the story
1	Sprint 2 exit missed by 21 Nov	LuCI Rules and Connections pages move to P1; CLI and Overview/Autorate stay	Full controller and safety	Product layer looks thinner in Round 2
2	Packages not installing on the Pi by 3 Dec	Pi install moves to P1; VM install only	Package tests in the VM	Hardware claim deferred to the finale
3	QoSmate installer not passing Stage 1 by 25 Nov	H1 runs QoSmate controller-only in the lab; whole-system QoSmate moves to P1	SQM and cake-autorate comparisons	Head-to-head is partial; say so in the submission
4	H1 pilot shows n > 20 needed, or machine time short	Reduce H1 to the primary and co-primary scenarios: S2, S6, S7, A32 (+ S1 as the static parity check)	Primary outcome, co-primary, parity check	Secondary outcomes reported as pilot data only
5	Controller not passing Stage 1 by 28 Nov	Apply O7: evaluate the Python lab controller with the same interfaces and golden tests	The research question	Product readiness of the controller deferred
6	Classification not ready for the freeze	Freeze with default classification + H1/H5 heuristics only	Controller evaluation	Intent features shown as design, not results

Never cut, at any step: - the Action Checker with its refusal tests, the watchdog and SAFE (the safety story); - the evidence ledger and the claims check (the honesty story); - the primary outcome scenarios and the co-primary comparison; - the honesty labels on every number.

> **For judges.** "We planned the work at task level with explicit effort assumptions, and the plan showed the controller sprint was overloaded. So we fixed the order in which scope is cut before we were under pressure. The safety and evidence parts were never on that list."

Part IV · Testing and evaluation

How we prove it, fairly

CHAPTER 23

Test Strategy

This chapter says how the team will show that VeriQ is built as specified. Chapter 24 asks whether the design works better than the alternatives. Tests check conformance ("the checker refuses a stale observation"); evaluation measures outcomes ("fewer inappropriate cuts than cake-autorate"). Judges notice when the two are mixed.

> **Honesty check.** No VeriQ code exists yet (12 Oct 2026), so none of the tests below has run. Every gate, count and time budget in this chapter is a **target** taken from docs/11 §14.4 or proposed here. The only measured data in the project is the Day-0 sandbox run [R51], and it is evaluation evidence, not test evidence.

Acronyms: **CI** = continuous integration (GitHub Actions); **SUT** = system under test; **netns** = Linux network namespace; **HIL** = hardware-in-the-loop; **VM** = virtual machine (run with **QEMU/KVM**); **ACL** = access control list; **RSS** = resident set size (memory a process holds); **SBOM** = software bill of materials.

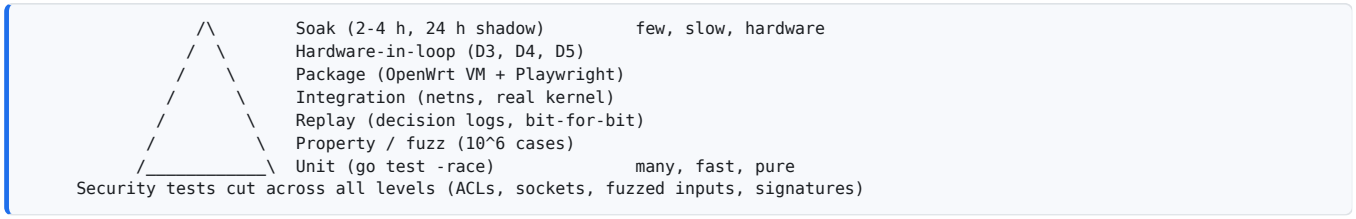
23.1 Principles

Six rules shape every test the team writes.

1. **Pure cores, thin shells.** The decision logic (`checker`, `control`, `diagnose`, `estimate`, `classify`) takes values and returns values, with no I/O and no clock reads. That makes it testable with plain table tests. I/O lives in thin adapters (`observe`, `probe`, `enforce`) that are tested at the integration level.
2. **Time is injected.** Every package reads time through the `clock.Clock` interface. Tests use a fake clock and step it tick by tick, so a 10-minute rolling budget is tested in milliseconds of wall time.
3. **One test per rule.** Every refusal code of the Action Checker has at least one test that triggers it and one test where the same input with a single field changed is accepted. A rule with no test does not count as implemented.
4. **Golden vectors across languages.** Where Go re-implements something trained or computed in Python (the tree ensemble and the conformal tables), the Python side writes reference outputs and the Go side must match them.
5. **Determinism is a feature.** The control goroutine is single-threaded and logs every input, so a recorded run can be replayed and must give the same decisions.
6. **The kernel is the real test of the datapath.** Rendered nft and tc text is checked against golden files, but only a real kernel (netns, VM, Pi) proves that packets reach the right CAKE tin. Each test has a stable name that the traceability matrix uses.

23.2 The test pyramid

docs/11 §14.4 defines six levels. This handbook adds two more (soak and security) that the specification implies through Stage 4 and §13. Higher levels are slower, fewer and closer to what a household sees.



Level	What it proves	Tooling	When it runs	Gate (target)
Unit	Each package does what its contract says	`go test -race`, table tests, fake clock	Every push	All pass; 100% of refusal codes; ≥ 80% line coverage on `checker`, `control`, `diagnose`, `classify`
Property / fuzz	Invariants hold for random input sequences	Go fuzzing (`testing.F`), seeded generators	Every push (short), nightly (long)	0 failures over 10^6 cases (nightly)
Replay	Decisions are a pure function of logged inputs	`veriqctl replay`	Every push on the stored corpus	Identical decision streams
Integration (netns)	The full datapath works in a real Linux kernel	`lab/` runner, privileged or self-hosted runner	Every pull request to `main`, nightly	All named checks pass
Package	The OpenWrt packages install, start, pass self-test, and the UI loads	QEMU OpenWrt 24.10 ×86-64 VM, Playwright	Nightly, and before each release	All pass
Hardware-in-loop	Footprint and behaviour on real routers	Lab bench with D3 (Pi), D4 (retail ARM64), D5 (MT7621 Lite)	Weekly from Sprint 3, before each release	Footprint budgets met (docs/11 §12.2)

Level	What it proves	Tooling	When it runs	Gate (target)
Soak	Nothing leaks, flaps or wears flash over hours	Bench + scripted load	Before Round 2 and before the finale	No crash; RSS flat; writes within policy
Security	Trust boundaries hold	Go fuzzing, Playwright, shell checks, `govulncheck`	Mixed: unit-level on every push, the rest nightly	All pass; no known-vulnerable module

23.3 Unit tests

23.3.1 Per-package test list

Each package in docs/11 §14.1 gets the named tests below (proposed names; keep them stable once written).

Package	Tests to write (Go test names)	Notes
`clock`	`TestFakeClockAdvance`, `TestMonotonicIgnoresWallStep`	The fake clock is used by every other package
`config`	`TestParseDefaults`, `TestValidateErrorsCiteSectionOption`, `TestMinFracFloor`, `TestSminBelowScenarioCapacity`, `TestConfigVersionMigrate`	Errors must name `section.option`; the floor check is S_min ≤ 0.8 × lowest scenario capacity (docs/09 A.2)
`model`	`TestClassFromDSCP`, `TestMarkBitsRoundTrip`, `TestProvOrder`	CLASS bits 24–26, PROV 27–29, SANCT 30 (C14)
`observe`	`TestParseCakeTinStats`, `TestParseHfscClassStats`, `TestParseNftJSON`, `TestGhatOverheadComp`, `TestQOwnFromBacklog`, `TestWifiStationParse`	Fixtures captured on each target
`probe`	`TestTimestampOneWayDelta`, `TestMidnightWrap`, `TestQualifyKeepsReplyRate90`, `TestUnhealthyLoss20`, `TestUnhealthyNoReply3s`, `TestIdleModeRate`, `TestNearReResolveOnLinkUp`	ICMP timestamp maths per RFC 792 [R16]
`estimate`	`TestBaselineWindowedMin10m`, `TestBaselineRisesAfterRouteChange`, `TestBaselineStaleFlag`, `TestBaselineStaleCapsL1`, `TestMedianNotMean`, `TestQuorumBelow2Missing`, `TestOutlier3MAD10s`, `TestDetectorBufferbloat3of6`, `TestDetectorCensoring`, `TestDetectorCPU`, `TestMuSigmaCapacitySample`, `TestMuSigmaLowerBound`, `TestCapacityMemoryP20`	Covers the A7 route change
`diagnose`	One test per row of the label table: `TestWANLikelyFarThr`, `TestWANLikelyNearMissingL2`, `TestRemoteLikely`, `TestRemoteNearMissingCapsL1`, `TestWifiLikely`, `TestCPULimited`, `TestProbeFault`, `TestMixedTwoLabels`, `TestMixedNoSpiral`, `TestUnknown`, `TestCorrelatedFarCountsOnce`, `TestL3NeedsCap`	Far reflectors count once
`control`	`TestFSMHoldToProbeUp`, `TestProbeUpBackoff`, `TestBackoffHold30s`, `TestDriftOnePercentPer10s`, `TestCutOneStepC1`, `TestCutWithoutGhat075`, `TestCutL3Rebaseline`, `TestRefractoryAfterCut`, `TestFallbackStep5pc`, `TestBudgetRolling10m`, `TestCooldownNeverRefunds`, `TestUndoNotImproved`, `TestLinkDownFreezes`, `TestDirectionAttributionLoad60`	100 → 20 Mbit/s in one cut

Package	Tests to write (Go test names)	Notes
`classify`	`TestRuleCompileQoSmateFields`, `TestRuleNegationAndSets`, `TestIPv6SuffixMask`, `TestDeviceAllIPv6`, `TestScheduleBoundaryDST`, `TestHeuristicsDemoteOnly`, `TestH3OffOnCake`, `TestTreeEvalGolden`, `TestConformalSetGolden`, `TestAbstainOnMulti`, `TestAbstainOnEmpty`, `TestCalibrationMin200`	Heuristics may only demote or reset (C7)
`lease`	`TestAdmissionMax2`, `TestAdmissionOnePerDevice`, `TestCollapseRefuses`, `TestExpiryOnMonotonic`, `TestRestoreNeedsNTP`, `TestRemainingNeverExtended`, `TestRenewOnce`	
`sanction`	`TestFirstViolation60s`, `TestEscalateDoubles`, `TestEscalateCap15m`, `TestResetAfter1hClean`, `TestOtherFlowsUnaffected`	
`checker`	One test per refusal code (next section), plus `TestAcceptHappyPath*` per action type	Pure function; no I/O
`enforce`	`TestCakeRenderGolden`, `TestHfscRenderGolden`, `TestHtbRenderGolden`, `TestRateChangeOrderHfsc`, `TestCompensateOnFailure`, `TestRefuseWithoutAccept`, `TestNftRenderGolden`	Golden files
`reconcile`	`TestAdoptMatchingVersion`, `TestMismatchGoesSafe`, `TestTableVanishedReapply`, `TestForeignQdiscDegraded`, `TestRetryMax3Backoff`	
`state`	`TestTransitionTable` (every allowed and forbidden edge), `TestShadowToLive24h`, `TestAnyToSafe`, `TestAtomicWriteRename`, `TestWriteRateLimit`	≤ 1 state write/min
`receipt`	`TestImproved07`, `TestNotImproved`, `TestInconclusiveConcurrent`, `TestInconclusiveFewSamples`, `TestLeaseReceipt`, `TestExplainTemplates`	Thresholds from docs/11 §9.1
`api`	`TestMethodTable`, `TestJSONRPCErrors`, `TestHouseholdHints`	
`importer`	`TestImportQoSmateRules`, `TestImportIPSets`, `TestImportRateLimits`, `TestImportLegacyPorts`, `TestImportSQM`, `TestDryRunNoWrite`	Clean-room fixtures from the README format
`telemetry`	`TestRingBound2MB`, `TestJSONLSchema`, `TestExporterLANOnly`	

23.3.2 The Action Checker: one test per refusal code

`checker.Check` is a pure function (docs/11 §14.2): each test builds its inputs and asserts the `Verdict`. The table merges docs/09 A.1 with docs/11 §8.1; thresholds are the specification's starting values, and tests change in the same commit as any tuning.

Refusal code	Action	How the test triggers it	Expected verdict
`STALE_OBS`	any	Snapshot timestamp 3 ticks (750 ms) older than the fake clock	Reject
`VERSION_MISMATCH`	any; also RULESET_APPLY	Action built on version 811 while read-back is 812 (simulates a foreign `tc` change); for RULESET_APPLY, candidate version ≠ desired + 1	Reject
`STATE_FORBIDS`	any	Valid cut proposed while the state is SHADOW (logged, not applied); also in SAFE and HOLD_LINKDOWN	Reject
`DUPLICATE`	any	Same `action_id` submitted twice	Second is rejected
`STEP_EXCEEDS_LEVEL`	RATE_CUT	A 30% cut proposed with diagnosis UNKNOWN at L1	Reject

Refusal code	Action	How the test triggers it	Expected verdict
`BELOW_FLOOR`	RATE_CUT	Target 8 Mbit/s with S_base 100 (S_min = 10)	Reject
`CUT_BELOW_EVIDENCE`	RATE_CUT	L2 with fresh $\hat{G}$ = 20 Mbit/s and target 15 Mbit/s (< 0.85·$\hat{G}$ = 17)	Reject
`EXPLORE_PRECOND`	RATE_RAISE	Raise while load is 50% (needs ≥ 75% for 2 s)	Reject
`ABOVE_MAX`	RATE_RAISE	Raise to 106 Mbit/s with S_max = 105	Reject
`EPISODE_ACTIVE`	FALLBACK_STEP	Second episode opened while the first is open	Reject
`BUDGET_EXHAUSTED`	FALLBACK_STEP	Two 5% steps in episode A, cooldown elapses, episode B proposes a third step inside the same rolling 10 min	Reject (the cooldown must not refund the budget)
`COOLDOWN`	FALLBACK_STEP	New episode 30 s after the previous one ended	Reject
`AUTH`	LEASE_GRANT	Missing or expired rpcd session token	Reject
`ADMISSION`	LEASE_GRANT	Third active lease, or a second lease for the same device	Reject
`COLLAPSE`	LEASE_GRANT	S = 1.5 Mbit/s, or WAN_LIKELY @ L2 with a 60% drop	Reject
`TIN_RANGE`	MARK_SET	Class value 5	Reject
`SANCTIONED`	MARK_SET	Flow present in the sanction table	Reject
`NO_PREFERENCE_BASIS`	MARK_SET	Preference requested for an abstained flow with no lease, rule, app mark or singleton; also a lease-based request after the lease expired on the monotonic clock	Reject
`INVALID_RULESET`	RULESET_APPLY	Candidate whose `nft -c` result is failure	Reject
`NOT_USER_INITIATED`	BACKEND_SWITCH	Switch proposed by the controller, not by an API call with a user session	Reject
`NO_VIOLATION`	SANCTION_ESCALATE	Escalation for a flow with no recorded violation	Reject
`CAP_EXCEEDED`	SANCTION_ESCALATE	Escalation beyond ×2 per offence or beyond 15 min	Reject
—	SAFE, REVERT_TO_RECHECKED, LEASE_REVOKE	Proposed in every state, with stale data	Accept (and logged)

Implementation tip. Mapping an expired lease to `NO_PREFERENCE_BASIS` follows from docs/09 A.1: MARK_SET needs a valid basis, and an expired lease is not one. If Chapter 12's final refusal table maps it differently, follow Chapter 12 and rename this test; the behaviour (reject) does not change.

Each case changes one field of a known-good input, so a failure points at exactly one rule:

```
func TestCheck_RefusalCodes(t *testing.T) {
    base := fixtures.GoodCut() // L2, fresh Ghat, LIVE, version 812, fresh snapshot
    cases := []struct {
        name   string
        mutate func(*fixtures.Input)
        code   string
    }{
        {"stale", func(in *fixtures.Input) { in.Snap.T -= 3 * tick }, StaleObs},
        {"version", func(in *fixtures.Input) { in.Act.Version = 811 }, VersionMismatch},
        {"floor", func(in *fixtures.Input) { in.Act.Target = 8000 }, BelowFloor},
        {"evidence", func(in *fixtures.Input) { in.Act.Target = 15000 }, CutBelowEvidence},
    }
    for _, c := range cases {
        t.Run(c.name, func(t *testing.T) {
            in := base.Clone()
            c.mutate(&in)
            v := Check(in.Act, in.Snap, in.Budgets, in.Leases, in.State, in.Limits)
            if v.Accept || v.Code != c.code {
                t.Fatalf("got %+v, want reject %s", v, c.code)
            }
        })
```

```
    }
}
```

A meta-test, `TestEveryRefusalCodeHasATest`, reads the list of code constants and fails if any code is not used in at least one test case. This is how the "100% of refusal codes" gate is enforced mechanically.

23.3.3 Golden vectors for the ML path

The classifier is trained in Python and evaluated in Go (docs/11 §5.6). - **Tree evaluator.** The export script writes 1,000 feature vectors per checkpoint (k = 8, 16, 32 and 10 s) with the LightGBM [R31] class probabilities. `TestTreeEvalGolden` requires a match within 1e-9 (target). - **Conformal set builder.** The calibration script writes the expected prediction sets for the same vectors. `TestConformalSetGolden` requires identical sets, including the singleton (act), multi-class (abstain) and empty (abstain) cases.

23.4 Property and fuzz tests

Property tests check rules over generated inputs. Go's native fuzzing (`testing.F`) mutates input sequences and keeps each failing input as a regression case in `testdata/fuzz/`.

Property	Statement	Package
Envelope	For every input sequence, S_min ≤ S ≤ S_max after every tick, in both directions	`control` + `checker`
Rolling budget	The sum of fallback reductions in any 10-minute window is ≤ 10% of the anchor, whatever the episode and cooldown pattern, including restarts that reload `budget.json`	`control`
Checker gate	The enforcer never applies an action without an ACCEPT for the same `action_id` and version	`enforce`
Precedence monotonicity	A class write with lower PROV never replaces a stored class with higher PROV	`classify` (Go model of the nft chain)
Abstention	A flow whose conformal set is not a singleton never receives RT or IV from the classifier	`classify`
Lease time	Remaining lease time never increases except by the single renewal	`lease`
No spiral	More than 3 L2 cuts in one episode without improvement always yields MIXED	`control`
Clock independence	Stepping the wall clock by ±1 h changes no decision (A18)	all, via the fake clock
Parser robustness	The UCI parser, the JSON-RPC decoder and the QoSmate importer never panic on arbitrary bytes	`config`, `api`, `importer`
Render validity	Every config the validator accepts renders an nft ruleset that `nft -c -f` accepts (run in the integration job, where nft exists)	`enforce`

Generators produce snapshot *sequences*, because the interesting failures are about history (two episodes, a cooldown, a restart, a third episode):

```
func FuzzControllerEnvelope(f *testing.F) {
    f.Add([]byte{0x10, 0x80, 0x03}) // seed: steady load, then a delay rise
    f.Fuzz(func(t *testing.T, seq []byte) {
        clk := clock.NewFake()
        c := control.New(testLimits(100_000), clk)
        for _, snap := range gen.Snapshots(seq, clk) { // decodes bytes into snapshots
            c.Tick(snap)
            for _, d := range model.Dirs {
                s := c.Rate(d)
                if s < c.Limits.Smin(d) || s > c.Limits.Smax(d) {
                    t.Fatalf("envelope violated: dir=%v S=%v", d, s)
                }
            }
            if c.Budget.UsedInWindow(10*time.Minute) > 0.10*c.Budget.Anchor() {
                t.Fatal("rolling budget exceeded")
            }
        }
    })
}
```

The 10^6-case gate runs nightly; each push runs about 30 s per target.

23.5 Replay determinism

The control goroutine owns all decision state, and the decision log records every input message with its monotonic timestamp (docs/11 §14.2). `veriqctl replay log.jsonl` feeds those inputs back into a fresh controller with a fake clock and must reproduce the decision stream bit for bit.

What makes this work: - **No hidden inputs.** Every input (snapshot, API call, link event, reload) enters through a logged channel message; no package under `internal/` reads time, environment or files except through an injected interface. - **No map-order dependence.** Go randomises map iteration order, so decision code sorts keys before iterating. - **Stable floating point.** Go may fuse multiply-add on some architectures (for example arm64), which can change the last bit of a result. Decision code uses explicit `float64(...)` conversions where the result must be exact, and replay identity is required **on the same architecture**. Across architectures, the test compares decisions (action type, target rounded to 1 kbit/s, verdict) instead of raw bytes. - **Versioned schema.** Each log line carries `config_version`, `model_version`, `backend` and `profile`. A replay against a different algorithm version is reported as "version differs", not as a failure.

The corpus in `testdata/replay/` holds the S1, S2 and S6 smoke logs plus logs from bug reports (hashed). A change that intentionally alters decisions updates the corpus in the same pull request, with the decision diff shown to the reviewer.

> **For judges.** Replay is what makes receipts and the 3D twin trustworthy: it proves the recorded decisions are what this exact code produces from those inputs.

23.6 Integration tests in network namespaces

The integration job builds the lab topology of Chapter 25 (clients → router SUT → ISP emulator → servers) in netns on a full Ubuntu kernel with `sch_cake`, `act_ctinfo` and `ifb` available. It runs `veriqd` exactly as on the router, with the plain-file config backend (docs/11 §12.9).

Test	What it checks	Pass criterion (target)
`it_install_teardown`	Install creates the nft table, qdiscs and IFB; DISABLED removes only VeriQ-owned objects	Foreign objects untouched
`it_tin_selftest_v4`, `it_tin_selftest_v6`	Marked probe packets of each class increment the expected CAKE tin counters in both directions	Every class in its tin; IPv6 identical (A20)
`it_mark_plumbing`	fwmark value → tin mapping (O2) and `ctinfo cpmark` on download	Mapping matches; otherwise compat mode is reported (O3)
`it_smoke_s1`, `it_smoke_s2`, `it_smoke_s6`	One short run of each H0 scenario	Run completes; ledger entry valid; no checker bypass
`it_kill_safe`	`kill -9 veriqd` under load	SAFE rates in place ≤ 12 s; traffic never stops (A15)
`it_firewall_restart`	`fw4 restart` flushes `inet veriq`	Table restored ≤ 1 tick (A31)
`it_foreign_qdisc`	Another tool replaces the WAN qdisc	DEGRADED; no flapping (A30)
`it_invalid_custom_nft`	Broken `custom.nft`	Candidate rejected; previous ruleset active (A25)
`it_link_down_up`	WAN veth down for 10 s, then up	No cut during the outage; resumes at the start rate (A14)
`it_lease_expiry_kernel`	Lease set element with a 60 s timeout; daemon stopped	Element expires in the kernel with no daemon (docs/11 §5.2)
`it_envelope_sanction`	EF-marked UDP flood from one LAN host	Sanctioned within 1 s; host's other flows unaffected (A11)
`it_reflector_lie`	One lab reflector adds 200 ms	Excluded; no rate change (A5)

Hosted runners may lack the privileges netns needs, so `lab.yml` runs on a self-hosted runner (the lab laptop), time-boxed to a target of ≤ 20 min per pull request.

23.7 Package tests in the OpenWrt VM

The package job boots OpenWrt 24.10 ×86-64 in QEMU/KVM (deployment view D2), with two virtual NICs: one for LAN and one for WAN towards an emulated ISP namespace on the host.

Test steps, in order: 1. Boot the image and wait for `ubus call system board`. 2. Add the CI-built feed key and URL; `opkg install veriq luci-app-veriq veriq-household` (and the `apk` equivalent on the apk-based line). 3. Check `uci-defaults`: `nf_conntrack_acct = 1`; QoSmate import is offered, not run. 4. Run `veriqctl health` and `veriqctl selftest`; require `errors=0`. 5. Conflict detection: enable SQM, restart VeriQ, expect LIVE to be blocked with an explanation; enable software flow offloading, expect the same (A24). 6. Upgrade to a package with a new algorithm version: config kept, `config_version` migrated, state goes to SHADOW. 7. Simulate sysupgrade keep: `/etc/veriq` is listed in `/lib/upgrade/keep.d/veriq`. 8. Uninstall: owned qdiscs, IFB and table removed; user config kept as `/etc/config/veriq-opkg`.

Playwright LuCI tests run a headless browser against the VM's uhttpd [R41]: - every VeriQ page (Overview, Devices, Rules, Apps, Connections, Queues, Autorate, Receipts, Settings, Advanced, Diagnostics) loads with no JavaScript console errors; - the setup wizard completes with a mocked measurement; - adding a rule with an invalid port shows the error inline and keeps the previous ruleset; - the household page logs in as `household`, creates a lease, sees it in the list and cancels it; - **negative ACL test:** the household session calling `rules.apply` or `state.set` gets an access-denied error; - with the language set to Hindi, page titles come from the Hindi `.po` file (no English fallback on translated keys).

23.8 Hardware-in-loop

HIL runs on the deployment views D3 (Raspberry Pi with a USB 3 GbE adapter), D4 (an ARM64 retail router with ≥ 512 MB RAM) and D5 (an MT7621-class device in the Lite profile).

Check	Method	Target
Footprint	RSS of `veriqd` and installed flash size, per profile	Lite ≤ 20 MB / ≤ 6 MB; Standard ≤ 40 MB / ≤ 10 MB; Pro ≤ 60 MB / ≤ 14 MB
veriqd CPU	`/proc/<pid>/stat` sampled under S1-like load	Lite ≤ 3% of one core; Standard ≤ 5% of one A53 core
CAKE CPU (separate)	`veriqctl bench` softirq at the configured rate	Recorded; never attributed to veriqd
Tin self-test on real NICs	As in the netns test, but through the physical WAN	Pass in both directions
Offload interaction	Vendor PPE or hardware offload on D4	Detected; LIVE blocked
Time-to-safe on hardware	`kill -9` under load	≤ 12 s

23.9 Soak tests

A soak finds what only appears over time: memory growth, descriptor leaks, unbounded rings, state flapping and flash wear.

- **Stage-4 soak:** 2–4 h on D3 and D4 with a mixed load script (call, DASH, bulk, periodic capacity steps from the emulator). Three soaks per device (tier H4).
- **Shadow soak:** 24 h in SHADOW with the same load, which also exercises the SHADOW → LIVE transition logic and the shadow report.

Measured	Pass criterion (target)
Crashes, respawns	0
RSS trend	No upward trend after the first 10 minutes (slope within noise)
Open file descriptors, goroutines	Bounded; return to the same level after each load cycle
Decision-log ring	≤ 2 MB; oldest lines evicted
Flash writes to `/etc/veriq/state`	≤ 1 per minute for state files; ≤ 1 per hour for capacity memory
Operating-state changes	Only those caused by injected events

23.10 Security tests

Security tests check the trust boundaries in docs/11 §13. Chapter 17 holds the threat model; this is the test list.

Test	Method	Pass criterion
Socket permissions	`stat /var/run/veriq.sock`	Mode 0600, owner root
No network listener	`netstat -lntu` (or `ss`) with the exporter off, then on	No VeriQ listener when off; exporter bound to LAN only when on
rpcd ACL roles	Playwright and `ubus call` with each role	Household role limited to its five methods
JSON-RPC fuzz	`FuzzAPIDecode` on the method dispatcher	No panic; malformed input returns an error
Malformed config and custom nft	Fuzzed UCI and nft snippets	Rejected before apply; previous state kept
NFLOG flood	Many new flows per second from one LAN host	Sampling capped at 4000 packets/s; veriqd CPU within budget
Signed artefacts	Install a model or catalogue package with a broken signature	Refused by the package manager
Support-bundle privacy	Generate a bundle and scan it for raw MACs and LAN IPs	None present; only stable hashes
Dependencies	`govulncheck ./...`; licence scan of `go.sum` modules	No known-vulnerable module; MIT, BSD or Apache only
SBOM	SPDX SBOM generated per release	Present and complete

Test	Method	Pass criterion
Lease abuse	Leased device runs a bulk download	Bulk flow sanctioned to BE; audit log records the requester (A13)

23.11 CI gates

The workflows follow docs/11 §14.1: `ci.yml`, `build.yml` and `lab.yml`, plus a nightly schedule and a claims check.

Workflow	Trigger	Jobs	Blocking gate
`ci.yml`	Every push and pull request	`gofmt`, `go vet`, staticcheck, unit tests with `-race`, coverage, short fuzz, replay corpus, refusal-code meta-test, `govulncheck`, licence scan	Yes: merge blocked on any failure or coverage below 80% on the four core packages
`build.yml`	Pull request to `main`, tags	Cross-compile `veriqd` for 5 targets (`CGO_ENABLED=0`, `-trimpath`), SDK packaging (`.ipk`, `.apk`), signing on tags	Yes: every target must build
`lab.yml`	Pull request to `main` (self-hosted), nightly	netns integration tests, H0 smoke scenarios	Yes for `main`
`nightly.yml`	Schedule	Long fuzz (10^6 cases), OpenWrt VM package tests, Playwright	Failure opens an issue; release blocked until fixed
`claims.yml`	Pull request touching `docs/`, `submission/` or `report/`	Recompute every published number from the evidence ledger	Yes: a number that does not regenerate within tolerance blocks the merge

A **release gate** adds: HIL footprint within budget on D3 and D4, one soak per device, and a signed SBOM.

> **Why it matters.** The claims check turns principle P9 ("every published number regenerates from raw data") into a CI rule. A slide cannot quote a number that the ledger cannot reproduce.

23.12 Test data management policy

Test data is versioned, reviewed and owned like code.

1. **Sources.** Only the team's own lab, seeded synthetic generators, and public datasets whose licence allows the use (for example CESNET DataZoo [R34], licence checked first). No real household traffic is captured; later pilot data follows Chapter 17.
2. **Headers only.** Packet fixtures are header-only (`snaplen 40`, as in the product), with addresses rewritten to documentation ranges (192.0.2.0/24, 2001:db8::/32).
3. **Location.** Fixtures ≤ 1 MB live in `testdata/`; pcaps, long logs and training sets live in the ledger's artefact store, referenced by content hash.
4. **Provenance.** Each fixture directory has a `SOURCE.md` with tool version, kernel or OpenWrt release, date and command, because `tc -s -j` field names can differ between versions.
5. **Measured vs synthetic.** File names say which is which (`*.measured.json`, `*.synthetic.json`). Synthetic data never appears in a report as a measurement.
6. **Golden updates.** Golden files change only in a commit whose message says why. The reviewer sees the diff of the golden output, not only the code.
7. **Retention.** Evaluation runs in the ledger are never deleted, including failed and unfavourable runs (docs/11 §15.4). Scratch CI artefacts expire after 30 days.
8. **No secrets.** Example credentials, keys and tokens in fixtures are visibly fake. The signing keys never enter the repo; CI uses repository secrets.

23.13 Traceability: test → requirement

The matrix maps test groups to the Case 3 requirements (docs/11 §19) and to VeriQ's own principles. Chapter 2 holds the full requirement matrix; Chapter 24 covers the evaluation scenarios.

Requirement	Tests (this chapter)	Level	Evaluation link (Chapter 24)
Classify without payload; confidence; correction	`TestRuleCompile*`, `TestConformalSetGolden`, `TestAbstainOn*`, header-only fixture rule, Connections-page Playwright test	Unit, package	H2, classifier report

Requirement	Tests (this chapter)	Level	Evaluation link (Chapter 24)
Estimate link capacity	`TestBaseline*`, `TestMedianNotMean`, `TestDetector*`, `TestMuSigma*`	Unit	E1–E8, S2, A1–A3
Queue management both directions, IPv4/IPv6	`it_tin_selftest_v4/v6`, `it_mark_plumbing`, render golden tests	Integration, HIL	S9, A20
Verify user experience	`TestImproved07`, `TestLeaseReceipt`, `it_smoke_s1`	Unit, integration	S1, S5 receipts
Temporary intent with fairness	`TestAdmission*`, `TestExpiryOnMonotonic`, `it_lease_expiry_kernel`, lease-abuse test	Unit, integration, security	S5, A13, starvation metric
Rollback; no starvation	`TestUndoNotImproved`, `TestCooldownNeverRefunds`, uninstall test, envelope tests	Unit, package	S13, A12
Bounded, observable remediation; safe state	All checker refusal tests, envelope and budget properties, `it_kill_safe`	Unit, property, integration	S8, A15
AI vs deterministic baseline	`TestTreeEvalGolden`, `TestHeuristicsDemoteOnly`	Unit	H2, ablations
Dashboard metrics	Playwright page loads, `TestExporterLANOnly`	Package	—
Reproducible experiments	Replay corpus, `claims.yml`, ledger schema tests	Replay, CI	All tiers
Licences, no secrets	Licence scan, SBOM, fake-credential rule	CI	—
P1 kernel-only packet path	`it_kill_safe` (traffic continues with the daemon dead)	Integration	S8
P5 preference needs a basis	`NO_PREFERENCE_BASIS` test, abstention property, `TestH3OffOnCake`	Unit, property	A27
P8 daemon crash never breaks the internet	`it_kill_safe`, time-to-safe on hardware	Integration, HIL	S8, A15

For judges. Ask us to name the test for any rule in the design. The answer should be a test name from this chapter and a CI run that shows it passing. Until the code exists, the honest answer is the planned test name and the sprint in which it lands (Chapter 22).

CHAPTER 24

Evaluation Design

This chapter is the evaluation protocol: what VeriQ must prove, against which tools, in which scenarios, with which metrics and statistics. It extends docs/09 §5 and docs/11 §15 and weakens nothing in them. Chapter 23 tests that the code matches the specification. This chapter asks whether the design helps households more than the tools they could install today.

> **Honesty check.** No evaluation run has happened yet. The only measured data is the Day-0 sandbox run [R51]: FIFO added p95 delay of 266.1 ms upload and 63.6 ms download, and a short-buffer shaper at 39.6 ms upload and 55.4 ms download. That was one preliminary run, with no base RTT applied, IPv4 only. Every other number in this chapter is a **target**, a **starting parameter** or an **illustrative** value, and is labelled as such.

Acronyms used here: - **RQ**: research question. - **SUT**: system under test. - **CI** (in statistics): confidence interval. - **OWD**: one-way delay. - **PLT**: page load time. - **DASH**: Dynamic Adaptive Streaming over HTTP. - **SUS**: System Usability Scale [R39]. - **pp**: percentage points (absolute). - **ISP**: internet service provider.

24.1 What the evaluation must answer

24.1.1 Research questions

The questions are fixed **before** the final runs (docs/11 §15.3) and do not change after the data is seen.

ID	Question	Outcome type	Scenarios
RQ1 (primary)	Does diagnosis-informed, bounded adaptation make **fewer and smaller inappropriate rate reductions** than delay-triggered autorate in disturbances the shaper cannot usefully control?	Superiority	S6, S7, S10, A32
RQ2 (co-primary)	Does VeriQ respond to a **genuine capacity drop** as fast as the comparator?	Non-inferiority, margin +1 s	S2 (= A1)
RQ3 (secondary)	Does VeriQ keep **bulk progress**?	Non-inferiority, margin −5 pp absolute	S1, S2, S4, S6
RQ4 (head-to-head set)	How does VeriQ compare with QoSmate, cake-autorate and static SQM on delay, call quality, streaming, gaming, page loads, fairness, recovery and resources?	Estimation, Holm-corrected	H1 scenarios
RQ5	Do classification, leases and the envelope deliver the intended treatment without starvation?	Estimation against targets	H2 scenarios
RQ6	Does VeriQ behave as designed under the adverse conditions A1–A32 and on real hardware?	Pass/fail against targets	H3, H4
RQ7	Is the VeriQ interface easier to use than QoSmate's for common household tasks?	Formative (small sample)	UX study

24.1.2 Hypotheses and the expected parity

Each RQ has a stated expectation, so the team cannot reinterpret results afterwards. - **RQ1.** VeriQ's inappropriate-intervention count and cumulative magnitude are lower than both cake-autorate's and QoSmate autorate's. The mechanism is the diagnosis gate: REMOTE_LIKELY, WIFI_LIKELY and PROBE_FAULT hold the rate. What QoSmate's autorate does in these scenarios is a **hypothesis** built from facts read in its source (mean RTT across reflectors, no load check in the decrease branch, a lower-only baseline after the first samples; docs/11 §10). - **RQ2.** VeriQ's confirmation time is within comparator + 1 s, and probably shorter, because the C1 cut reaches $\max(S_{min}, 0.90 \cdot \hat{G})$ in one step where a ×0.85 controller needs several (Chapter 10, illustrative). - **RQ3.** Bulk share of leftover bandwidth stays within 5 pp of the comparator. Holding during futile disturbances should help bulk, but a one-step cut slightly below true capacity costs a little. - **RQ4. Static conditions:** with fixed capacity and honest reflectors, VeriQ and QoSmate in CAKE mode should be at **parity**, because both use CAKE. Parity there is the expected result and is reported as parity, not spun. **Dynamic and adverse conditions:** VeriQ is expected to do better, which is where it must earn its added complexity. - **RQ5–RQ7.** Targets from docs/05 §4 and docs/11 §10, stated per scenario below.

> **For judges.** If a tuned QoSmate or cake-autorate ties VeriQ on RQ1, we report the tie and say which VeriQ components then do not earn their cost (docs/09 §4). The ablation arms exist to make that judgement possible.

24.2 Arms

24.2.1 The arm list

Arm	Version and mode	Purpose
FIFO	No QoS; the emulated ISP's oversized buffer	Illustrates the problem only; never a comparator
SQM	`sqm-scripts` [R11], `layer_cake.qos` (diffserv) or `piece_of_cake.qos`, static rate	The static reference: VeriQ's dominance-invariant floor
cake-autorate	Pinned v3.x release [R12] (exact tag in the manifest)	Co-primary comparator (docs/09)
QoSmate	v0.5.48 @ c507843 with autorate, luci-app-qosmate v1.0.14 [R13]; HFSC default mode and CAKE mode	Head-to-head (whole system), and controller-only on the same CAKE datapath
VeriQ	Product profile (whole system) or lab profile (controller-only)	System under test
VeriQ ablations	no-diagnosis, no-classifier, no-leases, no-fallback, RTT-only (no OWD)	Attribute any gain to a component

Two rules apply to every arm: - **Two enforcers never run at once.** Before each run the runner removes all qdiscs and nft tables on the router namespace and checks that `tc qdisc show` shows only the defaults. - **Every arm passes the Stage-1 self-test** before its runs count: per-tin counters match intent in both directions, offloads are off, and media crosses the ISP emulator.

24.2.2 Pinning procedure (all arms)

1. Record the exact version: release tag and commit hash for git projects, package version for OpenWrt packages.
2. Archive the complete configuration file used (`/etc/config/sqm`, the cake-autorate config file, `/etc/config/qosmate`, `/etc/config/veriq`) in the run's ledger entry.
3. Archive the resulting kernel state: `tc -s -j qdisc show`, `tc -j filter show` and `nft -j list ruleset`, captured after setup and before load starts. This is the **qdisc fingerprint** of the arm.
4. Do not change an arm's code. If a comparator has a bug that blocks the run, record it in the discrepancy log and report the run as failed for that arm. Never patch it silently.
5. Re-read the comparator's README and source at the pinned commit before publishing any comparison (docs/11 §10). Maintainers may change their code at any time.

24.2.3 Shared starting rate

Every shaped arm starts from the **same base rate**, measured the same way. The team uses VeriQ's wizard procedure for all arms: three capacity measurements of the emulated link, S_base = 90% of the lowest (docs/11 §11.1). That sits inside the 80–90% range QoSmate's README recommends for static settings (read), so it favours no one. Overhead and link-layer settings use the same preset in every arm, chosen per the SQM details guide [R11] for the emulated link (in the netns lab, plain Ethernet framing).

24.2.4 Tuning each comparator per its own documentation

The comparators must be tuned the way a competent user following their documentation would tune them. The team is expert in VeriQ and not in the others, which is a real risk to fairness (see Threats to validity).

SQM [R11]: - `script = layer_cake.qos` for whole-system runs (diffserv tins), `qdisc = cake`, `linklayer` and `overhead` as the shared preset; - `download` and `upload` set to the shared S_base; - ingress and egress both shaped (SQM shapes download via IFB); - no further tuning; SQM is the static floor by design.

cake-autorate [R12] (configure per its documentation; label any detail you are unsure of as "per its documentation"): - set its minimum, base and maximum shaper rates per direction to VeriQ's envelope: S_min = max(1 Mbit/s, 0.10·S_base), S_base, S_max = 1.05·S_base. Using the same envelope stops the floor, rather than the controller, from deciding S2; - point it at the same lab reflectors VeriQ uses, in the number its documentation recommends; - keep its default delay thresholds and step factors unless its documentation advises a change for the emulated link type; any change is recorded with the documentation passage that motivated it; - run it on the router namespace against the same CAKE instance it would control in a real deployment.

QoSmate [R13] (facts read from the README at v0.5.48): - static rates `UPRATE` and `DOWNRATE` = the shared S_base; - autorate enabled, with its defaults: `interval` 500 ms, `latency_increase_threshold` 5 ms, `latency_decrease_threshold` 10 ms, `adjust_down_factor` 85, `adjust_up_factor` 102, refractory 1 s after a decrease and 3 s after an increase; - **envelope-matched configuration** (primary comparison): the README allows overriding `min_dl_rate` and `min_ul_rate`, whose default is 25% of the configured rate. The team sets them to S_min (10% of base). With the default 25% floor, a 100 → 20 Mbit/s drop could never be followed, and QoSmate would lose S2 because of its floor, not its controller; - **default configuration** (sensitivity analysis): QoSmate exactly as shipped, reported beside the

envelope-matched result; - `reflectors` set to the same lab reflector IPs (space-separated list); - `WASHDSCPUP` and `WASHDSCPDOWN` on in whole-system runs, following its README guidance; `ROOT_QDISC` = HFSC (its default) in one set of whole-system runs and CAKE in another.

VeriQ: - whole system: product profile, defaults from Appendix B of docs/11, the reflector pool replaced by the lab reflectors; - controller-only: lab profile (DSCP untouched), classifier off, same CAKE line as the comparator.

> **Implementation tip.** Invite the comparators' maintainers or community to review the arm configurations before the final runs, and record their replies. A judge will trust "the QoSmate config was reviewed by its community" far more than "we tuned it".

24.3 Controller-only vs whole-system comparisons

The evaluation has two comparison types (docs/09 §5). They answer different questions and must not be mixed in one chart.

Aspect	Controller-only	Whole-system
Question	Is VeriQ's **rate controller** better?	Is VeriQ as a **product** better?
Datapath	One identical CAKE line (diffserv4, same overhead, nat, host isolation, same rates) for every arm	Each tool in its recommended configuration
What differs	Only the process that changes the CAKE bandwidth	Everything: scheduler, classification, heuristics, controller
Classification	Off or identical; DSCP untouched (VeriQ lab profile)	Each tool's own
Arms	cake-autorate, QoSmate autorate on CAKE mode, VeriQ (lab profile), VeriQ ablations	SQM, QoSmate (HFSC and CAKE modes), VeriQ (product profile)
Main RQs	RQ1, RQ2, RQ3	RQ4, RQ5
Validity check	The qdisc fingerprints of all arms are diffed before the runs; any difference other than the bandwidth value is fixed or recorded	Fingerprints archived; differences are the point

In controller-only runs QoSmate's autorate drives a CAKE qdisc set up by QoSmate in CAKE mode. Its CAKE options (for example `ack-filter`, `wash`, `nat`) are aligned with the shared line where its configuration allows, and the remaining differences are listed in the manifest.

24.4 Scenario catalogue

Every scenario has a machine-readable definition `lab/scenarios/<id>.yaml` (topology, traffic generators, fault schedule with a random onset in [60, 120] s, duration, ground-truth channels), a primary metric and a comparator (docs/11 §15.2). Chapter 25 gives the YAML schema and full examples.

Conventions for the tables: - capacities such as 100/20 Mbit/s are **illustrative scenario parameters**, chosen in the YAML; - "call" = a Jitsi WebRTC call with the video bridge on the WAN side and peer-to-peer disabled, so media crosses the emulated ISP; - "bulk" = iperf3 or curl transfers with persistent demand; - "Comparator" lists the arms the scenario's primary metric is compared against.

24.4.1 Core scenarios S1–S13

Definitions come from docs/05 §4 and are unchanged; metrics follow docs/11 §15.3.

ID	Traffic	Fault injected	Ground truth recorded	Primary metric (target)	Comparator
S1	Call + ISO download	None (constant capacity)	Emulator capacity and ISP queue length	Freezes (0) and added delay p95 (≤ 10 ms); bulk share of leftover (≥ 85%)	SQM, cake-autorate, QoSmate
S2	Call + bulk download	Capacity step 100 → 20 Mbit/s mid-call	Capacity schedule and step time	Confirmation time (co-primary, ≤ comparator + 1 s); diagnosed as WAN; settles < 2 s (target); no oscillation	cake-autorate, QoSmate autorate
S3	3 DASH players + game flow	None	Capacity; per-host demand	Game RTT p99 added ≤ 20 ms; 0 rebuffers at a sustainable bitrate; Jain ≥ 0.9	SQM, QoSmate (rules + heuristics)
S4	Call + backup upload	None (upload saturation)	Capacity; per-direction load	Upload added delay p95; download S unchanged ±5% (A9)	SQM, cake-autorate, QoSmate

ID	Traffic	Fault injected	Ground truth recorded	Primary metric (target)	Comparator
S5	Leased laptop call + others' bulk, lease 45 min	Lease request and expiry	Lease schedule; per-device demand	Leased device delay and freezes vs no-lease; others' progress > 0 in every 10 s; expiry on time	VeriQ no-leases ablation; QoSmate static rule
S6	Call + moderate bulk	+80 ms delay on the remote path beyond the ISP first hop	Fault location; access utilisation < 80%; ISP queue < 5 ms	**Primary triplet** (count, Mbit·s, duration)	cake-autorate, QoSmate autorate
S7	Call + moderate bulk	One reflector lies (spikes or deprioritised ICMP)	Which reflector lies; utilisation; ISP queue	**Primary triplet**; no rate change attributable to the reflector	cake-autorate, QoSmate autorate
S8	Call + bulk	Controller `kill -9`	Kill time	Time-to-safe ≤ 12 s; no traffic outage	QoSmate (procd restart; reported)
S9	IPv6-only client, call + bulk	None	Address family	Per-tin counters match intent; results equal to the IPv4 run	QoSmate (IPv6 rules)
S10	Call on a weak "Wi-Fi" client + bulk elsewhere	Client-link bottleneck (emulated in netns; real Wi-Fi in hardware runs)	Bottleneck location; WAN utilisation < 80%	**Primary triplet**; WIFI_LIKELY shown	cake-autorate, QoSmate autorate
S11	Bulk flow falsely marked DSCP 45, plus a call	Mis-marking from one host	Marking host and flow	Sanctioned within 1 s; not re-promoted by a lease or correction; lifts only after the penalty	QoSmate heuristics
S12	Mixed household load for 1 h	Capacity steps and remote-delay events	Event schedule	Disagreement report; each disagreement judged against ground truth	cake-autorate (enforcing)
S13	Call + bulk	Ambiguous evidence, plus an unrelated improvement inside the window	Injected ambiguity and the concurrent change	Budget and step limits respected; concurrent change → INCONCLUSIVE → condition-aware undo; throughput guard holds	VeriQ no-fallback ablation

Two scenarios need care in the lab: - **S10** cannot use real Wi-Fi in netns. There the "Wi-Fi" bottleneck is a rate limit and delay on the client's link plus injected station statistics, labelled **emulated**. The real-radio version runs on the physical kit in H4 and is reported separately. - **S12** runs cake-autorate as the only enforcer while VeriQ runs in an observe-only lab configuration that installs no qdisc and only logs what it would do. This keeps the rule that two enforcers never run at once. A disagreement is a difference in decisions; whether it mattered is judged only against the lab's ground truth (docs/05 §3A).

24.4.2 Adverse conditions A1–A32

Definitions, VeriQ's intended response and the pass criteria come from docs/11 §10. Several adverse conditions reuse a core scenario. Comparators marked "(hyp.)" are where VeriQ's advantage is a hypothesis drawn from read facts about the comparator.

ID	Traffic	Fault injected	Ground truth recorded	Primary metric (target)	Comparator
A1	As S2	Step 100 → 20 Mbit/s	Step time	Confirmation ≤ cake-autorate + 1 s; freezes ≤ comparator	cake-autorate, QoSmate (hyp.)
A2	Call + bulk	Capacity oscillates 10–50 Mbit/s every 5–20 s	Capacity schedule	Reversals/min ≤ comparator; delay p95 ≤ comparator	cake-autorate, QoSmate
A3	Bulk	Recovery 20 → 100 Mbit/s	Recovery time	Time to 90% of new capacity ≤ comparator	cake-autorate, QoSmate (hyp.)
A4	As S6	+80 ms remote path	Fault location	Primary triplet below comparator	cake-autorate, QoSmate (hyp.)
A5	As S7	Lying reflector	Which reflector	No attributable rate change	QoSmate (hyp.)
A6	Call + bulk	All reflectors unreachable	Block start and end	S within ±5% of pre-fault S; recovery with no manual action	QoSmate (hyp.)
A7	Call + light bulk	Base RTT +15 ms permanently (route change)	Change time	Cumulative magnitude below comparator; never stuck at minimum	QoSmate (hyp.)
A8	As S10	Client-side bottleneck	Location	Correct diagnosis shown; no WAN cut	cake-autorate, QoSmate

ID	Traffic	Fault injected	Ground truth recorded	Primary metric (target)	Comparator
A9	As S4	Upload saturation only	Per-direction load	Download S unchanged ±5%; upload delay p95 ≤ comparator	QoSmate (hyp.)
A10	500-flow BitTorrent of Linux ISOs + call	None	Per-host demand	Call freezes ≤ SQM; Jain ≥ 0.9	SQM, QoSmate
A11	High-rate UDP marked EF + call	Misbehaving app	Flow identity	Sanctioned within 1 s; device's other flows unaffected	QoSmate
A12	One host marks everything EF	Abusive host	Host identity	Starvation metric = 0 for other hosts	SQM, QoSmate
A13	As S5, leased device bulk downloads	Lease abuse	Flow identity	Others' bulk progress > 0 in every 10 s	VeriQ no-leases
A14	Call + bulk	PPPoE reconnect or WAN IP change	Outage window	No cut during outage; ≥ 90% of pre-event rate within 30 s under load	QoSmate (hyp.)
A15	As S8	`kill -9`	Kill time	Time-to-safe ≤ 12 s; no outage	QoSmate
A16	Peak-hour load	Router reboot	Reboot time	Bloat in first 60 s ≤ static SQM	SQM, QoSmate
A17	Call + bulk	Router CPU saturated by another process	CPU load schedule	No cut attributable to CPU; advice shown	QoSmate (hyp.)
A18	Call + bulk	NTP clock step ±1 h	Step time	No spurious events	— (property)
A19	Call + bulk	ISP aggregation congestion (shared queue at the emulator)	Emulator queue	Delay p95 ≤ comparator; inappropriate magnitude ≤ comparator	cake-autorate, QoSmate
A20	As S9	IPv6 and dual-stack	Family	Per-tin counters match intent for IPv6	QoSmate
A21	Download bulk on a ≥ 20:1 link	Asymmetry	Ratio	Download goodput ≥ SQM	SQM, QoSmate
A22	Call on ≤ 3 Mbit/s uplink	Very low uplink	Rate	Audio concealment ≤ SQM	SQM, QoSmate
A23	Gigabit bulk on a weak CPU (H4 hardware)	CPU limit	Bench result	No throughput collapse; recommendation logged	—
A24	Any	Flow offloading enabled	Offload state	Detected in 100% of runs	—
A25	Any	Invalid custom nft rule	Rule content	Previous ruleset still active	QoSmate
A26	Call + bulk	Cooperating UDPST server down	Server state	No L3 actions	—
A27	All traffic in one VPN tunnel	Encrypted tunnel	Tunnel	No preference without a basis	QoSmate
A28	Call + bulk	Radio-like random loss without delay	Loss schedule	No cut; loss shown	cake-autorate, QoSmate
A29	Idle link	Delay noise at idle	Load < 10%	Zero cuts at load < 10%	QoSmate (hyp.)
A30	Any	Foreign tool changes the qdisc	Change time	DEGRADED; no flapping	—
A31	Any	Firewall restart flushes rules	Restart time	Table restored ≤ 1 s	—
A32	Call + moderate bulk	ICMP deprioritised at the emulated ISP under load	Deprioritisation schedule; utilisation; ISP queue	**Primary triplet** below comparator	cake-autorate, QoSmate (hyp.)

24.4.3 Which scenarios feed the primary outcome

Only **control-futile** scenarios count for RQ1: S6, S7, S10 and A32. In each, the injected disturbance lies outside what the shaper can usefully control. The evaluator checks in **every run** that access-link utilisation stays below 80% and the ISP-emulator queue stays below 5 ms (docs/09 §5). A run that breaks this precondition is marked **invalid for RQ1** and reported in the exclusions table, never silently dropped.

Constant-capacity congestion (S1, S4) is excluded from RQ1, because there a cut can be correct. This prevents the primary outcome from rewarding a controller that simply never acts.

24.5 Tiers and the Stage 1–4 ladder

24.5.1 Tiers H0–H4

Tier	Scenarios	Arms	Runs per arm per scenario
H0 smoke	S1, S2, S6	All	1 (Stage 1)
H1 head-to-head core	S1, S2, S4, S6, S7, A6, A7, A9, A14, A32	SQM, cake-autorate, QoSmate (+autorate), VeriQ	Pilot n = 5, then n from the CI half-width rule (expected 10–20)
H2 classification and intent	S3, S5, S11, A10–A13, A20, A27	SQM, QoSmate (rules + heuristics), VeriQ	Same rule
H3 robustness	A2, A3, A15–A19, A21–A26, A28–A31	QoSmate, VeriQ (+ cake-autorate where relevant)	5–10
H4 hardware	Bench + soak on D3, D4, D5	VeriQ, QoSmate	3 soaks

S8 and S9 run inside H3 and H2 respectively (as A15 and A20). S10 joins H1 once the client-link emulation passes its Stage-1 check. S12 and S13 run once the corresponding features exist (Sprint 4 or P1). Scenarios not in a tier until then are reported as “not yet run”, not omitted.

24.5.2 Stage ladder

Stage	Purpose	Entry criterion	Exit criterion (target)	Tiers
Stage 1: capability and smoke	Prove the lab and every arm work	Lab on a full kernel; arms installed	Every arm passes self-test; media crosses the emulator; one smoke run each; no statistics	H0
Stage 2: core repeated comparisons	Answer RQ1–RQ3	Stage 1 passed; outcomes and margins frozen	H1 pilot (n = 5) → n chosen → full H1 runs; primary and co-primary analysed	H1
Stage 3: blind faults, mixed conditions, recovery	Robustness and ablations	Stage 2 analysed	H2 and H3 complete; ablations run; rejected stale action and controller kill demonstrated	H2, H3
Stage 4: hardware and resources	Product evidence	Packages on D3/D4/D5	Bench, footprint and 3 soaks per device	H4

Timeline (from docs/11 §18; confirm dates on the portal): H0 in Sprint 2 (8–21 Nov); the H1 pilot in Sprint 3 (22 Nov–5 Dec); full H1 runs in Sprint 4 (6–13 Dec, before the Round 2 deadline of 13 Dec 2026); H2, H3 and H4 in P1 before the finale.

24.6 Fault blindness

The controller must respond to telemetry, never to knowledge of the test. These rules hold in every run (docs/09 §5): - The fault onset is drawn at random in [60, 120] s from a seed recorded in the manifest. The controller never sees the seed, the scenario name, the fault schedule or future capacity. - The scenario runner and the evaluator run outside the router namespace. Nothing on the router can read their files. - **Measured**, **inferred** and **ground-truth** signals are stored in separate fields of the ledger. The emulator’s true queue length and capacity are ground truth; VeriQ’s estimates are inferred. - Harness-only metrics (WebRTC `getStats`, dash.js events, the game echo) are evaluation-only. The controller never reads them. - Arm order is counterbalanced, so a time-of-day or warm-up effect cannot favour one arm. - In the finale’s judge-chosen faults the same rule holds: the judge picks a fault, and VeriQ reacts from telemetry alone (docs/05 §4).

24.7 Metric definitions

Each metric is computed **per run** from raw ledger files by the evaluator. Formulas use these symbols: t_o = fault onset, t_c = fault clearance, S(t) = the arm’s shaper rate read back from tc, S_ref = S just before t_o, W = the measurement window.

Sample floors (docs/05 §3A, docs/09 §5): p95 needs ≥ 100 samples in the window, p99 needs ≥ 1,000, and p99.9 is not reported without ≥ 10,000. A metric below its floor is reported as “insufficient data”. Missing data is reported as missing and never imputed.

24.7.1 Delay and network metrics

```
added_delay_p95  = p95{ RTT_i : i in loaded window W } - median{ RTT_j : idle window }
                   per direction when the harness has one-way delay, else RTT
                   (same definition as the Day-0 run: load p95 minus idle median)
game_rtt_p99     = p99{ RTT of each echoed packet of the synthetic 64 pps UDP flow }
```

```
                  lost packets are counted as loss, not as RTT samples
goodput         = application bytes delivered / window duration    (receiver side)
```

The evaluator's delay probes run between harness clients and servers. Unlike the controller's reflector probes, they include the client's own link, so they measure what a household member feels. At 10 probes/s, a 60 s window gives 600 samples, enough for p95 but not p99. Game RTT at 64 packets/s gives 3,840 samples per minute, enough for p99.

24.7.2 Application metrics

```
freezeCount      = delta of inbound-rtp video freezeCount over W        [R35]
freeze_time      = delta of totalFreezesDuration over W
concealment      = delta concealedSamples / delta totalSamplesReceived  (audio, over W)
dash_stall_ratio = sum(stall durations) / (playback time + sum(stall durations))
dash_stalls      = number of stall events in W
plt              = loadEventEnd - startTime (Navigation Timing), median over 10 locally
                   cached sites per run (Playwright)
```

WebRTC statistics are defined by the W3C identifiers [R35]. dash.js stall events are taken from its event API in the pinned version [R43]; check the event names against that version.

24.7.3 Fairness and progress

```
jain_index       = (sum x_i)^2 / (n * sum x_i^2)                        [R36]
                   x_i = goodput of host i, over hosts with backlogged demand
leftover(t)      = S_lo(t) - sum(rate of protected flows at t)
bulk_share       = integral bulk_goodput(t) dt / integral leftover(t) dt
starvation       = number of 10 s windows with 0 bulk bytes while bulk demand exists
```

Jain's index is 1 when all backlogged hosts get equal goodput and 1/n when one host gets everything.

> **Note.** docs/09 defines leftover bandwidth as S − Σ rate(protected flows) but does not say which S. If S_lo is each arm's **own** shaper rate, an arm that cuts S is not penalised for the bulk bandwidth it gives up. This handbook therefore proposes S_lo(t) = the **arm-independent** reference rate (the scenario's shared S_base, or the emulated capacity schedule where capacity changes), with the arm's own S as a sensitivity analysis. The team must fix this choice in the analysis plan **before** the final runs.

24.7.4 Controller behaviour

```
onset            = first controller rate change after t0
confirmation     = earliest t1 >= t0 such that |S(t) - S_final| <= 0.10 * S_final
                   for all t in [t1, t1 + 5 s];  report t1 - t0          (docs/09 A.6)
                   S_final = median S over the last 10 s of the post-fault window
oscillation      = direction reversals per minute: consecutive rate changes of
                   opposite sign, ignoring changes smaller than 1% of S (dead band)
recovery         = time from t_c until S is within +/-10% of S_ref for 5 s
                   (A3 variant: time until S >= 0.90 * new capacity)
missed           = a genuine drop with no cut within 5 s of t0
time_to_safe     = t(SAFE rates read back from tc) - t(kill)
```

S_final and the 1% dead band are **analysis choices** proposed here; record them in the analysis plan before the final runs.

24.7.5 The inappropriate-intervention triplet (primary)

```
window           = [t0, t_c + 30 s]       (30 s grace for late reactions; analysis choice)
count            = number of rate reductions of any size in the window,
                   including drift-downs and fallback steps
magnitude        = integral over the window of max(0, S_ref - S(t)) dt     [Mbit*s]
duration         = total time in the window with S(t) < 0.95 * S_ref       [s]
precondition     = access utilisation < 80% and ISP-emulator queue < 5 ms throughout
```

Worked example (illustrative, not measured). S_ref = 90 Mbit/s. A delay-only controller cuts to 76.5 Mbit/s 2 s after onset, then to 65 Mbit/s 1 s later, and holds there for 40 s before climbing back over 20 s. Count = 2. Magnitude ≈ 1 × 13.5 + 40 × 25 + ½ × 20 × 25 = 13.5 + 1,000 + 250 ≈ 1,264 Mbit·s. Duration (S below 0.95 × 90 = 85.5 Mbit/s) ≈ 1 + 40 + 16.4 ≈ 57 s, because the linear climb from 65 to 90 Mbit/s passes 85.5 after 16.4 of its 20 s. A VeriQ run that holds scores 0, 0 Mbit·s and 0 s. Magnitude matters because a controller making many small cuts could have a low count and still waste a lot of bandwidth.

24.7.6 Resources

```
cpu_veriqd       = CPU time of veriqd / wall time, as % of one core (from /proc)
rss              = resident set size of veriqd, sampled every 10 s; report max and trend
cake_cpu         = softirq time on the shaping core, reported separately, never
                   attributed to the controller
```

24.8 Statistics

The statistical rules come from docs/09 §5 and docs/11 §15.4. They are fixed now, before any data exists.

24.8.1 The run is the unit

Packets and probe samples within a run are strongly correlated, so treating them as independent would give confidence intervals far too narrow. Each run produces **one value per metric** (for example, one p95 or one magnitude), and all inference uses those run-level values.

24.8.2 Sample size: pilot, then the half-width rule

The pilot runs n = 5 per arm per H1 scenario. From it, the team estimates the CI half-width w_pilot of each key difference and chooses the final n for a target half-width w_target. Because a CI's width shrinks roughly with 1/√n:

```
n_final ≈ n_pilot × (w_pilot / w_target)^2
```

Worked example (illustrative). The pilot gives a half-width of 0.9 s for the confirmation-time difference, and the team wants 0.4 s (well inside the 1 s margin). n_final ≈ 5 × (0.9/0.4)² ≈ 25. That is above the expected 10–20 runs, so the team either accepts a wider interval, cuts variance (for example by tightening the onset range), or adds lab time. Whatever is chosen is recorded before the final runs. With only five runs the pilot's own interval is rough, so round n up, not down.

24.8.3 Run order and resets

- Arms are interleaved in counterbalanced blocks: each block runs every arm once, in an order that rotates across blocks.
- Each run starts with a warm-up (the same for all arms) and ends with a full reset of qdiscs, nft tables, conntrack and the emulator.
- Random seeds (fault onset, traffic start jitter) are drawn per block, so all arms in a block see the same schedule. A paired analysis on blocks is then possible as a sensitivity check; the default analysis treats arms as independent samples.

24.8.4 Confidence intervals: bootstrap

Run-level differences are summarised with **bootstrap** 95% confidence intervals using 10,000 resamples [R37]. For two arms A and B with n_A and n_B runs: 1. resample n_A runs from A and n_B runs from B, with replacement; 2. compute the statistic (for example median_A − median_B); 3. repeat 10,000 times; the 2.5th and 97.5th percentiles give the CI.

Medians are the default summary for skewed metrics such as delay and time. For the primary triplet many runs may score exactly zero, and a median of mostly zeros carries little information. For RQ1 the team therefore reports the **mean** magnitude with its bootstrap CI, plus the **proportion of runs with at least one inappropriate reduction**.

24.8.5 The primary claim: superiority on two components

RQ1 claims "fewer **and** smaller". The team pre-registers the rule: VeriQ is declared better than a comparator only if the bootstrap CI of the difference (VeriQ − comparator) lies **entirely below zero for both** the mean count and the mean magnitude. Requiring both to pass is an intersection–union test, so it needs no multiplicity correction across the two components. Duration is reported but is not part of the decision.

RQ1 has two comparators: cake-autorate and QoSmate autorate. The claim is made separately for each, with Holm correction across the two.

24.8.6 Non-inferiority logic

A non-inferiority claim says "not worse than the comparator by more than a margin δ fixed in advance". It is decided from the CI of the difference, not from a p-value:

```
co-primary (confirmation time, lower is better), margin +1 s:
   difference d = VeriQ - comparator
   non-inferior if the upper CI bound of d <= +1 s

secondary (bulk share, higher is better), margin 5 pp:
   difference d = VeriQ - comparator   (in percentage points)
   non-inferior if the lower CI bound of d >= -5 pp
```

```
          better <-------------- 0 -------- +1 s ----------> worse
case A:   [-----d-----]                                     non-inferior (and lower)
case B:             [------d------]                         non-inferior
case C:                     [--------d--------]             inconclusive
case D:                                 [------d------]     inferior
```

Worked example (illustrative, not measured). In S2, twelve VeriQ runs have a median confirmation time of 1.7 s and twelve cake-autorate runs have 3.1 s. The bootstrap CI of the median difference is [−2.2, −0.5] s. Its upper bound (−0.5 s) is below +1 s, so VeriQ is non-inferior (case A). That it is also lower is reported as an observation, because superiority on RQ2 was not pre-registered. If instead the CI had been [−0.4, +1.3] s (case C), the team could **not** claim non-inferiority, even though the point estimate favours VeriQ. The honest wording would be: "inconclusive at this sample size".

24.8.7 Holm correction for the secondary set

The head-to-head secondary set (RQ4) has eight metrics: added delay p95 per direction, WebRTC freezes and concealment, DASH stalls, game RTT p99, PLT, Jain's index, recovery time, and CPU/RSS. Testing many metrics inflates the chance of a false "win". The Holm procedure [R38] controls this: sort the m p-values from smallest to largest and compare the i-th smallest with $\alpha/(m - i + 1)$, stopping at the first one that fails. p-values come from two-sided permutation tests on run-level values.

Worked example (illustrative p-values, m = 8, α = 0.05).

Rank i	Metric	p-value	Holm threshold α/(m − i + 1)	Decision
1	Upload added delay p95	0.003	0.05/8 = 0.00625	Significant
2	WebRTC freezes	0.009	0.05/7 = 0.00714	Not significant: stop
3	DASH stall ratio	0.012	0.05/6 = 0.0083	Not significant (stopped)
4–8	Others	≥ 0.04	—	Not significant

Only the first metric may be reported as a significant difference. Without the correction, three metrics would have looked "significant" at 0.05, which is exactly the over-claiming Holm prevents.

24.8.8 Other rules

- No p99.9 claims from small samples.
- Results where VeriQ ties or loses are kept and reported, in the same table and the same font.
- Every exclusion (precondition broken, arm crashed, harness fault) is listed with its reason.
- Bootstrap intervals and permutation tests are computed by the same evaluator script for every arm.

24.9 The evidence ledger and the claims check

24.9.1 Ledger layout

Every run writes raw files plus a manifest, and reports are generated **only** from the ledger (docs/05 M9, docs/11 P9).

```
ledger/
├── runs/
│   └── 2026-12-08T14-03-11Z_S6_qosmate_b03/
│       ├── manifest.json          # what, who, versions, seeds, checks
│       ├── config/                # archived arm config + qdisc fingerprint
│       ├── raw/                   # tc/nft, pings, iperf3 JSON, getStats, dash.js
│       ├── groundtruth/           # emulator capacity, ISP queue, fault schedule
│       ├── controller/            # decision log (VeriQ) or comparator logs
│       └── derived/metrics.json   # run-level metrics written by the evaluator
├── analysis/
│   ├── plan.md                    # frozen outcomes, margins, analysis choices
│   └── results/                   # tables and figures generated from runs/
├── claims.yaml                    # every number used in slides and the report
└── discrepancies.md               # every result that differed from expectation
```

The manifest records everything needed to reproduce the run:

```
{"run_id":"2026-12-08T14-03-11Z_S6_qosmate_b03","scenario":"S6","arm":"qosmate-autorate",
 "comparison":"controller-only","block":3,"seed":918273,"fault_onset_s":87.4,
 "versions":{"arm":"qosmate v0.5.48 c507843","kernel":"6.8.0","iproute2":"6.1.0",
             "harness_git":"<commit>","scenario_hash":"sha256:<hash>"},
 "stage1_selftest":"pass","offloads":"off","media_crosses_emulator":true,
 "precondition":{"util_max":0.71,"isp_queue_max_ms":2.3,"valid_for_primary":true},
 "status":"complete"}
```

The values in this example are placeholders, not results.

24.9.2 The claims check

`claims.yaml` lists every number that appears in a slide, the report or the README, with the query that regenerates it and a tolerance:

```
- id: rq1_magnitude_s6_vs_cake_autorate
  text: "Mean inappropriate magnitude in S6, VeriQ vs cake-autorate"
  query: "analysis/queries/rq1.py --scenario S6 --metric magnitude --vs cake-autorate"
  tolerance: 0.5   # Mbit*s, absolute
  used_in: ["submission/round2.pdf#page=12", "slides/deck.pptx#slide=9"]
- id: day0_fifo_up_p95
  text: "Day-0 FIFO added p95 upload (preliminary, one run)"
  query: "lab/summarize.py evidence/day0_sandbox_2026-10-11"
  expected: 266.1
  tolerance: 0.1
```

The `claims.yml` CI workflow (Chapter 23) runs every query and fails if a number does not regenerate within tolerance. The Day-0 values are in the ledger too, labelled preliminary, so even the Round 1 slide passes the same check.

24.10 Threats to validity

Threat	Type	Why it matters	Mitigation
Tuning asymmetry: the team knows VeriQ best	Comparator fairness	Comparators may be under-tuned	Tune per their docs; envelope-matched and default configs; invite community review; archive configs
Floor differences decide S2	Comparator fairness	QoSmate's default 25% floor cannot follow 100 → 20	Envelope-matched primary comparison; defaults as sensitivity
Emulation is not a real access network	External	tbf + netem queues differ from DOCSIS, PON and 5G schedulers	Label all results emulated; H4 hardware runs; no claim beyond the emulated conditions
Lab reflectors are not internet reflectors	External	Real ICMP rate limiting and route changes are richer	A5, A6, A32 inject reflector faults; a later field pilot in SHADOW
Wi-Fi emulated in netns	Construct	A rate-limited veth is not a radio	S10 labelled emulated; real-radio runs on the kit reported separately
Definition of "control-futile"	Construct	A scenario could hide real congestion	Precondition checked in every run; invalid runs listed
Metric choice favours VeriQ	Construct	Metrics designed by the SUT's authors	Outcomes and margins fixed in advance; the co-primary and secondary protect the comparators' strengths (speed, bulk)
Order and warm-up effects	Internal	Later runs may run warmer or slower	Counterbalanced blocks; identical warm-up; full reset
Host CPU contention	Internal	A busy laptop delays probes and media	Generator and browser CPU < 70% checked per run (docs/09 §5)
Offloads bypass the shaper	Internal	Results become meaningless	Stage-1 check per run; manifest field
Analysis flexibility	Conclusion	Choosing windows after seeing data	`analysis/plan.md` frozen before final runs; changes logged with reasons
Small n, zero-inflated outcomes	Conclusion	Wide or degenerate intervals	Pilot-based n; means plus proportions for RQ1; no p99.9
Many metrics	Conclusion	False wins	Holm correction; one pre-registered primary
Comparator code changes	Reproducibility	A later QoSmate release may behave differently	Pinned commits; re-read source before publishing; claims dated

24.11 Results reporting template

Every results section, in the submission and in the handbook updates, uses the same shape. The rows below are a **template**; the dashes are placeholders, not results.

Scenario / metric	VeriQ	Comparator	Difference [95% CI]	Verdict
S6 inappropriate magnitude (Mbit·s, mean)	—	cake-autorate: —	— [— , —]	Better / not shown / worse
S6 runs with ≥ 1 inappropriate reduction	—/n	—/n	—	—
S2 confirmation time (s, median)	—	cake-autorate: —	— [— , —]	Non-inferior / inconclusive / inferior
S1 bulk share of leftover (%, median)	—	cake-autorate: —	— pp [— , —]	Non-inferior / inconclusive / inferior
S1 added delay p95 up (ms, median)	—	QoSmate CAKE: —	— [— , —]	Parity expected

Each results section also states: 1. the number of valid runs per arm, and the exclusions with reasons; 2. the comparison type (controller-only or whole-system) and the arm versions; 3. the label "emulated" (netns) or the hardware used; 4. where VeriQ tied or lost, in the same prominence as where it won; 5. a "what would make us wrong" sentence (for example: "a tuned QoSmate with a median-of-reflectors patch would likely close the S7 gap; we have not tested that").

For judges. A good results slide has three numbers per claim: the estimate, the interval and n. If any of the three is missing, the number is not a result yet.

24.12 UX study protocol (SUS)

RQ7 is answered by a small formative study (docs/11 §15.4), scheduled for P1.

Design: - 6–8 participants, mixing the personas of Chapter 15: non-technical household members, a student gamer, and at least one experienced OpenWrt user; - within-subject: every participant uses both QoSmate's LuCI app and VeriQ's (LuCI plus the household page), on identical lab routers; - order counterbalanced: half start with QoSmate, half with VeriQ.

Tasks (the same for both systems, read aloud from a card): 1. "Give the game console priority." 2. "A video call is bad right now. Find out why." 3. "Undo the change you made in task 1."

Measures: - task success (yes, with help, no); - time on task (stopwatch, from the end of reading to the participant's "done"); - the System Usability Scale after each system [R39]: ten statements on a 1–5 scale. Score = 2.5 × [Σ(odd item − 1) + Σ(5 − even item)], giving 0–100. VeriQ's target is ≥ 75 (formative; docs/11 §17); - think-aloud notes and one open question: "What confused you most?"

Procedure and ethics: - written consent; no personal data beyond first name and persona; participants can stop at any time; - a pilot session with a teammate first, to fix the task wording; - the same facilitator script for both systems; the facilitator does not reveal which system the team built.

Analysis. Report per-task success, median times and SUS per system, with every individual score shown. With 6–8 participants there is no significance test: the study is **formative**, meant to find usability problems, and is reported as such, never as proof that VeriQ is easier.

Honesty check. The facilitator is from the VeriQ team, which can bias participants. Say so in the report, and if possible ask someone outside the team to run the sessions.

CHAPTER 25

Lab Build Guide

This chapter tells the team how to build the VeriQ evaluation lab from scratch, how to run a scenario in it, and how to keep every result traceable to raw data. It covers the Linux lab on a laptop (deployment view D1), the OpenWrt virtual machine (D2) and the physical lab around the Raspberry Pi (D3). The design rules come from docs/11 §14.4 and §15 and docs/09 §5. The starting point is the Day-0 script that already exists in the repository.

> **Honesty check.** Today the repository's `lab/` folder contains three files: `quickstart_bufferbloat.sh`, `summarize.py` and `README.md`. Everything else in this chapter (the extended topology, generators, scenario runner, evaluator and ledger tools) is the **build plan** for Sprints 0 and 1. File names such as `lab/run.py` are proposals. The only measured numbers are the Day-0 sandbox values [R51], quoted in the section on reproducing Day-0.

25.1 What the lab must prove, and what it cannot

The lab is where VeriQ's claims become evidence. It must inject a known fault at a random moment the controller cannot know, and record what really happened alongside what each arm did. Only then can "VeriQ held where a cut could not help" be checked against ground truth.

It therefore needs six capabilities: - an emulated access bottleneck with an oversized first-in-first-out (FIFO) buffer; - capacity schedules (S2, A1–A3) and delay injected beyond the Internet Service Provider (ISP) first hop only (S6, A4, A7); - faulty reflectors: lying, blocked or deprioritised (S7, A5, A6, A32); - real applications: a WebRTC call, Dynamic Adaptive Streaming over HTTP (DASH), game traffic, bulk and page loads; - ground-truth channels kept apart from the system under test (SUT); - an evidence ledger with manifests, so every number regenerates (principle P9).

The lab **cannot** prove how VeriQ behaves on a real ISP network, in a real home or on Wi-Fi radios it only emulates. Every number it produces is labelled "emulated", as the Day-0 notice in `lab/README.md` already requires. Stage 4 (hardware) and any later home pilot answer the remaining questions.

25.2 Hardware and software requirements

25.2.1 Hardware

Item	Minimum	Recommended	Why
Lab machine (D1)	4 cores, 16 GB RAM, 100 GB SSD, native Ubuntu 24.04	8 cores, 32 GB RAM	Browsers, Jitsi, iperf3 and the VM run together; generator CPU must stay below 70% (docs/09 §5)
Virtualisation	Hardware virtualisation (VT-x/AMD-V) for QEMU with the Kernel-based Virtual Machine (KVM)	Same	The OpenWrt VM (D2) for QoSmate and package tests
Reference router (D3)	Raspberry Pi 4 + USB 3 gigabit Ethernet adapter, OpenWrt 24.10	Raspberry Pi 5 with active cooler	Stage 4 resources and the demo kit (docs/06 §2)
ISP emulator for D3	An old laptop with two Ethernet ports (one may be USB)	Same, or a second Pi	Runs the same namespaces as D1, with physical ports
Switch	5-port gigabit, unmanaged	Same	Physical LAN

These sizes are **assumptions** from the workload, not measured requirements. Record the real machine in every run manifest.

> **Note.** Windows Subsystem for Linux 2 (WSL2) and many cloud sandboxes ship kernels without CAKE or netem. The Day-0 run itself was done in such a sandbox, which is why its CAKE and fq_codel arms were skipped [R51]. Use a native install, or a VM that allows nested namespaces.

25.2.2 Software

Component	Version policy	Used for
Ubuntu 24.04 LTS, generic kernel + `linux-modules-extra-$(uname -r)`	Record `uname -r`	Kernel qdiscs and actions
iproute2, nftables, conntrack-tools, ethtool, tcpdump, jq, ffmpeg, nginx, stress-ng	Distribution packages; record `tc -V`, `nft -v`	Topology, shaping, inspection, content
iperf3 (≥ 3.16, as in Day-0), curl, Python 3.12	Pinned in `lab/requirements.txt`	Bulk load, harness
Playwright for Python + its bundled Chromium	Pinned version	WebRTC and DASH clients, page loads

Component	Version policy	Used for
Docker Engine + docker-jitsi-meet	Pinned release tag	Jitsi Meet with the Jitsi Videobridge (JVB)
dash.js	Pinned release [R43]	DASH player
QEMU/KVM, OpenWrt 24.10 ×86-64 image	Pinned release	D2 VM
BitTorrent client and a local tracker (for example aria2 or Transmission, plus opentracker)	Pinned	A10 many-flow load, lab only

Install the basics as the existing README does, plus the extra tools:

```
sudo apt install iproute2 iperf3 iputils-ping ethtool python3 python3-venv \
  nftables conntrack tcpdump jq curl nginx ffmpeg qemu-system-x86 stress-ng
sudo apt install "linux-modules-extra-$(uname -r)"    # CAKE, netem, ctinfo on Ubuntu
python3 -m venv ~/vq-venv && ~/vq-venv/bin/pip install playwright pyyaml
~/vq-venv/bin/playwright install chromium
```

25.2.3 The full-kernel check

The Sprint 0 exit criterion is a FIFO-versus-CAKE run **with base round-trip time (RTT) applied** on a full kernel (docs/11 §18). Before any run, the lab's preflight must prove that every kernel feature VeriQ and the baselines need is present. This is a lab-side version of the Stage-1 capability check. A sketch of the proposed `lab/doctor.sh`:

```
#!/usr/bin/env bash
# lab/doctor.sh (proposed): fail fast if the kernel cannot run every arm.
set -u
fail=0
ok()  { printf 'PASS  %s\n' "$1"; }
bad() { printf 'FAIL  %s\n' "$1"; fail=1; }
for m in sch_cake sch_fq_codel sch_netem sch_tbf sch_htb sch_hfsc sch_prio \
         sch_ingress ifb act_ctinfo act_mirred cls_fw cls_u32 cls_matchall \
         nf_conntrack nft_ct; do
  modprobe "$m" 2>/dev/null && ok "module $m" || bad "module $m"
done
ip netns add vq-doctor
x() { ip netns exec vq-doctor "$@"; }
x ip link add d0 type dummy && x ip link set d0 up
x ip link add ifb-doc type ifb && x ip link set ifb-doc up
x tc qdisc add dev d0 root cake bandwidth 10mbit diffserv4 fwmark 0x07000000 \
  && ok "cake with fwmark" || bad "cake with fwmark"
x tc qdisc add dev d0 handle ffff: ingress && ok "ingress qdisc" || bad "ingress qdisc"
# verify syntax with tc-ctinfo(8) on this iproute2 [R7]
x tc filter add dev d0 parent ffff: matchall action ctinfo cpmark 0x7F000000 \
  action mirred egress redirect dev ifb-doc \
  && ok "ctinfo cpmark + mirred" || bad "ctinfo cpmark + mirred (see O3)"
x tc qdisc add dev ifb-doc root netem delay 1ms && ok "netem" || bad "netem"
x nft -c -f lab/doctor/features.nft && ok "nft sets/meters/log" || bad "nft features"
[ -d /proc/sys/net/ipv6 ] && ok "IPv6 enabled" || bad "IPv6 disabled"
ip netns del vq-doctor
exit $fail
```

`lab/doctor/features.nft` holds a throwaway table that uses a set with `flags timeout`, a dynamic set with a `limit rate over … kbytes/second` statement, a `ct packets < 33` match and `log group 7 snaplen 40`. These are the features VeriQ's ruleset relies on (docs/11 §5.2, §5.6). Check the exact syntax on the target nft version [R42].

If the `ctinfo cpmark` line fails, VeriQ's download plumbing cannot work as specified. The documented fallback is open decision O3: a QoSmate-style DSCP restore in "compat mode". Record it in the manifest; never fix it silently.

> **Implementation tip.** Run `doctor.sh` as the first step of every run, not only once. A kernel update on the lab machine can silently drop `linux-modules-extra`, and the runner must then refuse to start.

25.3 The lab topology

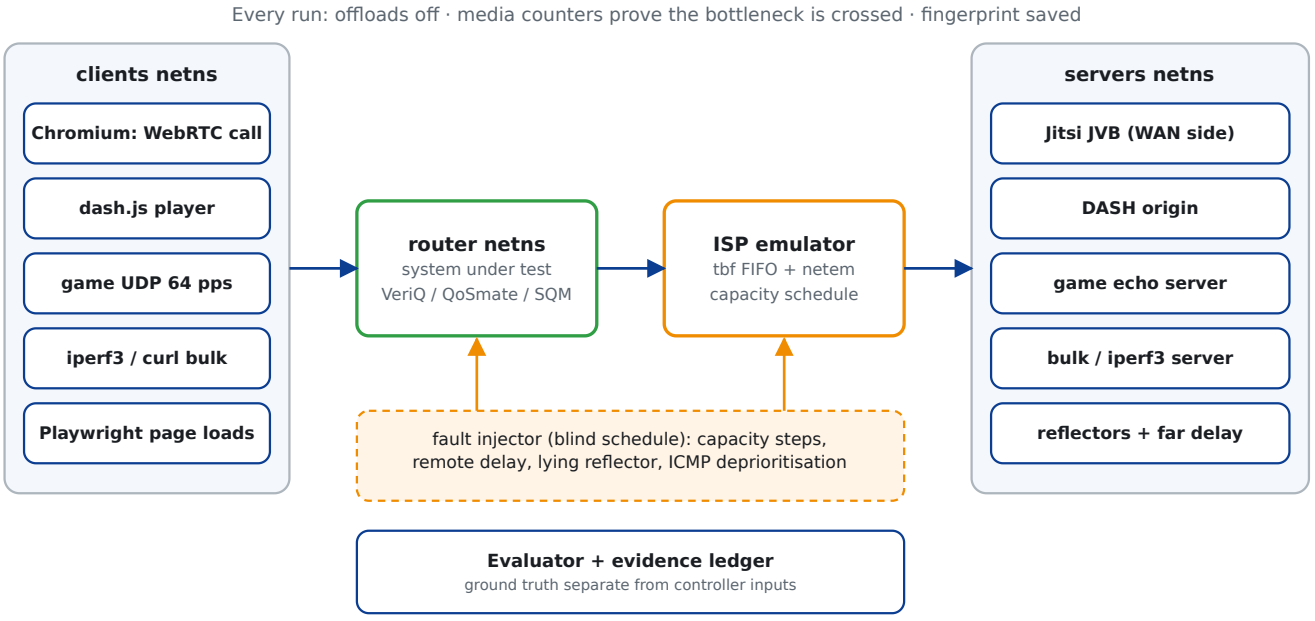


Figure 13. Lab topology: client namespaces, the router under test, the ISP emulator and the servers, with the evaluator and ledger outside the data path.

The Day-0 script uses four network namespaces (netns): `vq-lan` → `vq-rtr` → `vq-isp` → `vq-srv`. The evaluation lab keeps its naming, addressing style and helpers (`nsx`, `link`, `addr`), and adds: - **several clients** behind a LAN switch, so per-host fairness and leases can be tested; - a **core** namespace beyond the ISP first hop, where remote-path faults are injected without touching the near hop; - separate **reflector**, **server** and **peer** namespaces, so a single reflector can lie while the servers stay honest.

```
vq-c1 (call)  --+                                          +-- vq-refl (8 reflectors)
vq-c2 (DASH)  --+                                          |
vq-c3 (game)  --+-- vq-sw --- vq-rtr --- vq-isp --- vq-core -----+-- vq-srv (web,DASH,iperf3)
vq-c4 (bulk)  --+   bridge    SUT        bottleneck  base RTT,    |
                                         + near hop  remote delay +-- vq-peer (far call party,
                                                                  |            torrent seeds)
                                                                  +-- host: Jitsi/JVB (docker)
evaluator + ledger: root namespace, reads counters through "ip netns exec" (not in the path)
```

25.3.1 Namespaces and addresses

Namespace	Role	Interfaces and addresses (IPv4; IPv6 uses `fdc9:281b:1724::/64`)
`vq-c1` … `vq-c4`	Clients: call, DASH, game, bulk/torrent/page loads	`cN0` = 10.10.1.11–14/24, gateway 10.10.1.1
`vq-sw`	LAN switch (Linux bridge `br0`); Wi-Fi impairment point	`sw-cN` ports, `sw-rtr`
`vq-rtr`	Router, the SUT; one arm at a time	`rtr-lan` 10.10.1.1/24; `rtr-wan` 10.10.2.1/24
`vq-isp`	Access bottleneck (tbf FIFO) and the **near hop**	`isp-cpe` 10.10.2.2/24 (near target); `isp-core` 10.10.3.1/24
`vq-core`	"Internet": base RTT, remote delay, reflector faults	`core-isp` 10.10.3.2/24; `core-srv`, `core-refl`, `core-peer`, `core-host`
`vq-refl`	Reflectors 198.18.1.1–198.18.1.8 on a dummy device	`refl0` 10.10.5.2/24
`vq-srv`	nginx (web, DASH), iperf3, game echo, optional UDPST server	`srv0` 10.10.4.2/24
`vq-peer`	Far call participant; torrent seeders on aliases 10.10.60.0/24	`peer0` 10.10.6.2/24
root (host)	Jitsi stack via docker; evaluator; ledger	`host-core` 10.10.7.1/24

The reflector addresses come from 198.18.0.0/15, the range reserved for benchmarking, so a routing mistake can never send probes to a real public resolver. VeriQ's default reflector list contains public addresses (docs/11 §6.2). In the lab, the arm configuration overrides it with the lab reflectors, and the leak check described in the safety section proves that nothing escapes.

The router masquerades LAN IPv4 traffic on `rtr-wan` with a separate table, `inet labnat`, as a home router does. This makes CAKE's `nat` keyword meaningful. IPv6 is routed without translation. The NAT setting is recorded in the manifest.

25.3.2 Building it

The proposed `lab/topo.sh` reuses the Day-0 helpers. The core of it:

```
NS=(vq-c1 vq-c2 vq-c3 vq-c4 vq-sw vq-rtr vq-isp vq-core vq-refl vq-srv vq-peer)
for ns in "${NS[@]}"; do ip netns add "$ns"; nsx "$ns" ip link set lo up; done

nsx vq-sw ip link add br0 type bridge && nsx vq-sw ip link set br0 up
for i in 1 2 3 4; do
  link "vq-c$i" "c${i}0" vq-sw "sw-c$i"
  nsx vq-sw ip link set "sw-c$i" master br0 up
  addr "vq-c$i" "c${i}0" "10.10.1.1$i/24" "fd00:10:1::1$i/64"
  nsx "vq-c$i" ip route add default via 10.10.1.1
done
link vq-sw sw-rtr vq-rtr rtr-lan && nsx vq-sw ip link set sw-rtr master br0 up
addr vq-rtr rtr-lan 10.10.1.1/24 fd00:10:1::1/64
link vq-rtr rtr-wan vq-isp isp-cpe
addr vq-rtr rtr-wan 10.10.2.1/24 fd00:10:2::1/64
addr vq-isp isp-cpe 10.10.2.2/24 fd00:10:2::2/64
link vq-isp isp-core vq-core core-isp
addr vq-isp isp-core 10.10.3.1/24 fd00:10:3::1/64
addr vq-core core-isp 10.10.3.2/24 fd00:10:3::2/64
# ... core-srv, core-refl, core-peer, core-host in the same pattern ...
nsx vq-refl ip link add refl-lo type dummy && nsx vq-refl ip link set refl-lo up
for i in $(seq 1 8); do nsx vq-refl ip addr add "198.18.1.$i/32" dev refl-lo; done
nsx vq-rtr nft -f lab/topo/labnat.nft          # masquerade on rtr-wan (IPv4 only)
offloads_off                                   # every veth, as in the Day-0 script
```

Routing follows the Day-0 pattern: default routes point “up” towards the core, and `vq-core` holds the specific routes towards each server namespace. Forwarding is enabled in `vq-sw` (bridge only), `vq-rtr`, `vq-isp` and `vq-core`. Turn segmentation offloads (GSO, GRO, TSO) off on every veth, as `quickstart_bufferbloat.sh` already does. Otherwise the shapers see 64 KB super-packets instead of wire-sized ones, and both the delay figures and the per-tin counters are distorted.

> **Why it matters.** The near-probe target is `isp-cpe` (10.10.2.2), the router’s default gateway. VeriQ’s diagnosis separates “near” from “far” (docs/11 §7). Remote-path delay injected in `vq-core` raises the far deltas but leaves the near hop flat, which is exactly the signature of REMOTE_LIKELY. If the delay were injected in `vq-isp`, the scenario would test something else.

25.4 The ISP emulator

The ISP emulator is the `vq-isp` and `vq-core` pair. The two `tbf` bottlenecks and the near hop live in `vq-isp` (egress `isp-cpe` for download, `isp-core` for upload). The base RTT (lab default 20 ms, on one leg), remote delay, route changes and reflector faults live in `vq-core`. Because each element has its own interface, `tbf` and `netem` never share a qdisc tree.

25.4.1 Bottleneck and capacity schedules

The bottleneck follows `isp_bottleneck()` in the Day-0 script: `tbf` at the line rate, with a FIFO limit sized as `ISP_BUF_MS` of data at that rate. The Day-0 default of 300 ms is a lab parameter that represents an oversized modem buffer, not a measurement of any real modem.

A capacity change re-issues the `tbf` with all its parameters. The runner logs the command, its monotonic timestamp and the read-back value:

```
# capacity_step: download 100 -> 20 Mbit/s, buffer kept at 300 ms of the new rate
rate=20; bps=$(( rate * 1000000 / 8 )); lim=$(( bps * 300 / 1000 ))
nsx vq-isp tc qdisc change dev isp-cpe root tbf rate "${rate}mbit" \
  burst "$(burst_bytes "$bps")" limit "$lim"
nsx vq-isp tc -j qdisc show dev isp-cpe | jq '.[0].options.rate' >>"$RUN/truth/cap.jsonl"
```

There are two buffer modes, and the scenario declares which one it uses: - `buffer_mode: ms` (lab default): the byte limit is recomputed so that the buffer always holds the same time of data; - `buffer_mode: bytes`: the byte limit stays fixed, as in a modem with a fixed packet buffer, so the delay grows when the rate drops.

The choice changes how much bloat a drop creates. It is recorded in the manifest, and the two modes are never mixed within one comparison.

Capacity schedules generalise the step. A schedule is either a list of `(t, down_mbit, up_mbit)` points or a generator, such as “uniform 10–50 Mbit/s, dwell 5–20 s” for the LTE-like oscillation of A2. The generator draws from the run seed, so the same seed replays the same schedule for every arm.

25.4.2 Delay injection on remote paths only

```
# S6 / A4: +80 ms beyond the ISP first hop (near hop unaffected)
nsx vq-core tc qdisc change dev core-isp root netem delay 100ms limit 10000   # 20 -> 100
# A7: permanent route change, base RTT +15 ms
nsx vq-core tc qdisc change dev core-isp root netem delay 35ms limit 10000
# variant: only the application server is slow; reflectors and near hop stay flat
nsx vq-core tc qdisc replace dev core-srv root netem delay 80ms limit 10000
```

The `netem` limit of 10 000 packets (as in the Day-0 script) keeps netem from dropping packets at 100 Mbit/s with 100 ms of delay. Control-futile scenarios also have a precondition that the evaluator checks in every run: access utilisation below 80% and an ISP queue below 5 ms (docs/09 §5). If the precondition fails, the run is marked invalid for the primary outcome; it is not deleted.

25.4.3 Lying, blocked and deprioritised reflectors

```
# A5 / S7: reflector 198.18.1.3 lies (spiky delay); verify prio/u32 syntax with tc(8) [R7]
nsx vq-core tc qdisc replace dev core-refl root handle 1: prio bands 3 \
  priomap 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
nsx vq-core tc qdisc add dev core-refl parent 1:3 handle 30: netem delay 40ms 30ms
nsx vq-core tc filter add dev core-refl parent 1: protocol ip prio 1 u32 \
  match ip dst 198.18.1.3/32 flowid 1:3

# A6: all reflectors unreachable (ICMP blocked beyond the first hop)
nsx vq-core nft add table inet fault
nsx vq-core nft add chain inet fault fwd '{ type filter hook forward priority 0; }'
nsx vq-core nft add rule inet fault fwd ip daddr 198.18.1.0/24 meta l4proto icmp drop

# A32: ICMP deprioritised under load: ICMP-only delay and jitter while load runs
nsx vq-core tc qdisc replace dev core-isp root handle 1: prio bands 3 \
  priomap 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
nsx vq-core tc qdisc add dev core-isp parent 1:2 handle 20: netem delay 20ms limit 10000
nsx vq-core tc qdisc add dev core-isp parent 1:3 handle 30: netem delay 60ms 40ms
nsx vq-core tc filter add dev core-isp parent 1: protocol ip prio 1 u32 \
  match ip protocol 1 0xff flowid 1:3

# Near hop rate-limits ICMP (A32 variant; tests the missing-near rules)
nsx vq-isp nft add table inet near
nsx vq-isp nft add chain inet near in '{ type filter hook input priority 0; }'
nsx vq-isp nft add rule inet near in icmp type echo-request limit rate over 2/second drop
```

In the A32 sketch, band 2 keeps the 20 ms base RTT for ordinary traffic, and band 3 delays ICMP more. The `ip protocol 1` match catches ICMP only; add the IPv6 equivalent (next header 58) when the scenario runs dual-stack.

> **Note.** All namespaces share the host's clocks, and the Linux kernel answers ICMP timestamp requests, so lab reflectors give one-way deltas with **zero clock offset**, kinder than the internet. Some runs should block timestamp replies in `vq-refl` to force the RTT-only path (docs/11 §6.4, O4). Confirm in Stage 1 that the lab kernel answers timestamp requests.

25.4.4 Other network-side faults

Fault (scenario)	Injection	Ground truth recorded
Link down / reconnect (A14)	`ip link set isp-cpe down`, wait, `up`; in the VM, a PPPoE server in `vq-isp` (P1)	Down and up times
Wi-Fi bottleneck (S10, A8)	`netem rate 6mbit delay 4ms 3ms loss 1%` on `sw-c1` in `vq-sw` (illustrative values)	Port counters, impairment window
ISP aggregation congestion (A19)	A "neighbour" sender in `vq-isp` pushes cross-traffic through the `isp-cpe` bottleneck, so near and far deltas both rise	Cross-traffic rate
Capacity oscillation (A2), recovery (A3)	Capacity schedule generator	Every applied rate

The Wi-Fi case needs an honest caveat. VeriQ's WIFI evidence comes from station statistics read through ubus and hostapd (docs/11 §7.1), and a veth has none. The proposed `lab/wifi_shim.py` publishes station-like statistics **derived from the impaired port's own counters**, in every scenario (not only S10), so its presence leaks nothing. A D1 result validates the diagnosis logic, not Wi-Fi measurement.

25.5 Traffic generators

Every generator runs inside a client or server namespace (`ip netns exec`), so its traffic crosses the bottleneck. Each generator also writes a **flow truth record**: the 5-tuple it used and its intended class. This is the classification ground truth.

Generator	Where	What it measures	Scenario use
Jitsi call (2 participants, via JVB)	`vq-c1` ↔ JVB on the host ↔ `vq-peer`	WebRTC freezes, audio concealment, jitter, RTT	S1, S2, S4–S6, A-series
dash.js player	`vq-c2` ← nginx in `vq-srv`	Stalls, stall time, bitrate switches	S3, H1 secondary
Synthetic game, UDP 64 pps each way	`vq-c3` ↔ echo in `vq-srv`	RTT p99, loss, one-way delays	S3, H1 secondary
iperf3 / curl bulk	`vq-c4` ↔ `vq-srv`	Goodput, bulk share of leftover	Most scenarios
BitTorrent of Linux ISOs (lab only)	`vq-c4` ← seeders in `vq-peer`	Many-flow load, host fairness	A10

Generator	Where	What it measures	Scenario use
Playwright page loads (10 locally hosted sites)	`vq-c4` → nginx in `vq-srv`	Page load time (PLT)	H1 secondary

25.5.1 Jitsi with JVB on the WAN side, p2p disabled

A two-party Jitsi call sends media peer-to-peer by default. Then the media path depends on NAT traversal and bypasses the videobridge, so it can differ between runs, and two LAN participants would never cross the bottleneck at all. Two settings prevent this (docs/09 §6, item 1): 1. **Peer-to-peer (p2p) mode disabled**, so all media goes through the JVB. 2. **The JVB sits on the WAN side**, so every media packet of the LAN participant crosses `vq-isp` in both directions.

The simplest arrangement runs docker-jitsi-meet on the host, routed from `vq-core` through the veth `core-host`. The JVB advertises 10.10.7.1 (the variable name depends on the docker-jitsi-meet release; check the Jitsi handbook [R43]). The far participant runs headless Chromium in `vq-peer`. Disable p2p in the deployment's `config.js` (`p2p: { enabled: false }`) and, as a belt-and-braces measure, in the meeting URL:

```
https://10.10.7.1:8443/veriqlab#config.p2p.enabled=false&config.prejoinConfig.enabled=false
```

Both browsers use Chromium's fake capture devices, fed with test files, so no camera or real person is involved:

```
--use-fake-ui-for-media-stream --use-fake-device-for-media-stream
--use-file-for-fake-video-capture=/lab/media/pattern_720p.y4m
--use-file-for-fake-audio-capture=/lab/media/tone_speech.wav
```

Verify in every run that the selected candidate pair's remote address is the JVB address and that the UDP media port (10000 by default in docker-jitsi-meet) shows counters on `isp-cpe`. A call that silently fell back to TCP or TURN is a different experiment, and the run is marked invalid.

25.5.2 dash.js with local DASH content

Encode one test clip into a bitrate ladder with ffmpeg (for example 0.5, 1.5, 3 and 6 Mbit/s; illustrative values), package it as DASH and serve it from nginx in `vq-srv`. A small page loads the pinned dash.js build and logs player events with monotonic timestamps: buffer empty and loaded, playback stalled, and quality changes. A stall is a "buffer empty" event during playback, and the stall time runs until playback resumes. Check the event names against the pinned dash.js version [R43].

25.5.3 Synthetic game flow

A game flow is small, steady and latency-critical. The generator sends 64 packets per second each way. That rate is a lab parameter from docs/11 §15.3, not a measured profile of any game. Because all namespaces share `CLOCK_MONOTONIC`, the echo server can stamp the receive time, which gives **true one-way delays** as a ground-truth bonus.

```python
#!/usr/bin/env python3
"""lab/gen/game_udp.py (proposed): 64 pps UDP each way; echo server stamps receive time."""
import socket, struct, sys, time

FMT = "!Idd"  # seq, client send time, server receive time (shared CLOCK_MONOTONIC)


def server(port=27015):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(("0.0.0.0", port))
    while True:
        data, addr = s.recvfrom(2048)
        seq, t_send, _ = struct.unpack(FMT, data[:20])
        s.sendto(struct.pack(FMT, seq, t_send, time.monotonic()) + data[20:], addr)


def client(host, seconds, out, port=27015, pps=64, size=120):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.setblocking(False)
    pad, period = b"\0" * (size - 20), 1.0 / pps
    t0 = nxt = time.monotonic()
    seq, pending = 0, {}
    with open(out, "w") as f:
        f.write("seq,t_send,owd_up_ms,owd_down_ms,rtt_ms\n")
        while time.monotonic() - t0 < seconds:
            now = time.monotonic()
            if now >= nxt:
                s.sendto(struct.pack(FMT, seq, now, 0.0) + pad, (host, port))
                pending[seq], seq, nxt = now, seq + 1, nxt + period
            try:
                data, _ = s.recvfrom(2048)
            except BlockingIOError:
                time.sleep(0.0005)
                continue
            q, ts, tr = struct.unpack(FMT, data[:20])
            back = time.monotonic()
```

```
            pending.pop(q, None)
            f.write(f"{q},{ts:.6f},{(tr - ts) * 1e3:.3f},{(back - tr) * 1e3:.3f},"
                    f"{(back - ts) * 1e3:.3f}\n")
        for q, ts in pending.items():
            f.write(f"{q},{ts:.6f},,,\n")  # lost or still in flight: reported as missing


if __name__ == "__main__":
    if sys.argv[1] == "server":
        server()
    else:
        client(sys.argv[2], float(sys.argv[3]), sys.argv[4])
```

Game RTT p99 needs about 1,000 samples per window (docs/05 §3A). At 64 pps, that means windows of at least about 16 s; shorter windows report “insufficient data”.

25.5.4 Bulk: iperf3, curl and BitTorrent

- **iperf3** as in Day-0 (`-P 4`, `-R` for download, `-J` for JSON goodput), with one server port per direction; **curl** downloads of a large file model a browser download or an update.
- **BitTorrent** is needed for A10 (500 flows plus a call). Create a torrent of a freely redistributable Linux ISO with a local tracker URL on `vq-srv`. Run many seeders in `vq-peer` on address aliases from 10.10.60.0/24, and one leecher in `vq-c4` with a high peer limit. Disable DHT, peer exchange and local peer discovery in every client. The lab has no route to the internet, so no public swarm can be reached. For exact repeatability, also provide a synthetic “torrent-like” generator: N parallel HTTP range requests from many source addresses. Label each run with which one it used.

25.5.5 Playwright page loads

The secondary outcome uses page load time for 10 sites cached locally (docs/11 §15.3). Mirror 10 representative pages, with all page requisites, into nginx virtual hosts in `vq-srv`, and map their names in `/etc/netns/vq-c4/hosts` (`ip netns exec` bind-mounts it over `/etc/hosts`). Do **not** replay recordings inside the browser: that traffic would never cross the bottleneck. PLT is read from the Navigation Timing API: `loadEventEnd − startTime` of the navigation entry, with the browser cache cleared before each load.

25.6 Collecting WebRTC getStats

Call quality comes from the browser’s own statistics API, defined in the W3C WebRTC statistics specification [R35]. The harness wraps `RTCPeerConnection` before the Jitsi page loads, so it can poll every connection once per second. Inject the wrapper with Playwright’s `add_init_script`:

```
// lab/gen/getstats_hook.js (proposed): keep a handle on every peer connection.
(() => {
  const Orig = window.RTCPeerConnection;
  window.__vqPcs = [];
  window.RTCPeerConnection = function (...args) {
    const pc = new Orig(...args);
    window.__vqPcs.push(pc);
    return pc;
  };
  window.RTCPeerConnection.prototype = Orig.prototype;
  window.__vqSample = async () => {
    const keep = new Set(['inbound-rtp', 'remote-inbound-rtp', 'candidate-pair']);
    const out = [];
    for (const pc of window.__vqPcs) {
      if (pc.connectionState === 'closed') continue;
      (await pc.getStats()).forEach((s) => { if (keep.has(s.type)) out.push(s); });
    }
    return out;
  };
})();
```

The Python side calls `await page.evaluate("window.__vqSample()")` every second and appends each sample, with a monotonic timestamp, to `raw/webrtc_<participant>.jsonl`. The evaluator derives the metrics from counter differences, never from single snapshots:

Metric	Stats fields used [R35]
Video freezes	`freezeCount`, `totalFreezesDuration` (inbound-rtp, kind video)
Audio concealment ratio	Δ `concealedSamples` / Δ `totalSamplesReceived` (inbound-rtp, kind audio)
Jitter and loss	`jitter`, `packetsLost`, `jitterBufferDelay` / `jitterBufferEmittedCount`
Media-path RTT	`roundTripTime` (remote-inbound-rtp), `currentRoundTripTime` (nominated candidate pair)
Path validity	The remote candidate of the nominated pair is the JVB address

Some fields are Chromium-specific or recent; the manifest lists the pinned Chromium version and the fields actually present. An absent field is reported as missing, never as zero.

25.7 Ground-truth channels

Ground truth is what the evaluator knows and the arm under test must not. These channels are written to the run's `truth/` folder by root-namespace collectors:

Channel	Source	Rate
`capacity_applied`	Every `tbf` change, with command, monotonic time and read-back	On change
`isp_queue`	`tc -s -j qdisc show dev isp-cpe` / `isp-core`: backlog bytes ÷ rate = ISP queue delay	10 Hz
`isp_util`	Interface byte counters on `isp-cpe` and `isp-core` ÷ line rate	10 Hz
`faults`	Fault type, parameters, drawn onset, actual apply and clear times	On event
`flows`	Generator 5-tuples with intended class (call audio, call video, DASH, game, bulk, torrent, web)	Per flow
`game_owd`	One-way delays from the game generator	64 Hz
`headers.pcap`	`tcpdump -s 96` on `isp-cpe` (headers only)	Continuous
`cpu`	Per-core CPU of generators and browsers; softirq of the router namespace's cores	1 Hz

Fault blindness (docs/09 §5) is enforced in three ways: 1. The SUT runs as its own user, without read access to the ledger. 2. Its configuration never contains a scenario name. 3. Fault onsets are drawn from the run seed at random in [60, 120] s (docs/11 §15.2).

25.8 The scenario YAML schema

Each scenario has one machine-readable definition, `lab/scenarios/<id>.yaml` (docs/11 §15.2). The runner refuses a file that fails schema validation.

Key	Content
`id`, `version`, `title`, `kind`	Identity; `kind` ∈ genuine-drop, control-futile, congestion, classification, robustness, resources
`tiers`, `arms`	H0–H4 membership; arm identifiers from `lab/arms/`
`duration_s`, `warmup_s`	Run length and the warm-up that is excluded from metrics
`topology`	Profile (`netns-v1`, `vm-v1`, `pi-v1`), link rates, `isp_buffer_ms`, `buffer_mode`, `base_rtt_ms`, `ipv6`, `nat`
`traffic`	Generators: `id`, `type`, namespace, start and stop, parameters
`faults`	Injections: `id`, `type`, parameters, `onset_s` (fixed or `uniform: [a, b]`), duration
`ground_truth`	Channels to record
`preconditions`	Run-validity checks (media crosses the bottleneck, CPU, utilisation, ISP queue)
`outcomes`	Primary, co-primary and secondary metrics with comparator and margin
`targets`	Pass criteria, always labelled as targets

S2: a genuine capacity drop during a call (definition from docs/05 §4; co-primary outcome from docs/11 §15.3):

```yaml
id: S2
version: 1
title: "WAN download capacity 100 -> 20 Mbit/s during a call"
kind: genuine-drop
tiers: [H0, H1]
arms: [sqm_static, cake_autorate, qosmate_autorate_cake, veriq]
duration_s: 300
warmup_s: 30
topology:
  profile: netns-v1               # qosmate_* arms swap vq-rtr for the VM (same addresses)
  link: {down_mbit: 100, up_mbit: 20}          # lab parameters, not a real ISP
  isp_buffer_ms: 300
  buffer_mode: ms
  base_rtt_ms: 20
  ipv6: true
```

```
  nat: true
traffic:
  - {id: call, type: jitsi_call, ns: vq-c1, peer_ns: vq-peer, start_s: 0, stop_s: end}
  - {id: game, type: game_udp, ns: vq-c3, pps: 64, start_s: 0, stop_s: end}
  - {id: bulk, type: iperf3, ns: vq-c4, dir: down, streams: 4, start_s: 20, stop_s: end}
faults:
  - id: drop
    type: capacity_step
    dir: down
    to_mbit: 20
    onset_s: {uniform: [60, 120]}
    duration_s: until_end
ground_truth: [capacity_applied, isp_queue, isp_util, faults, flows, game_owd, cpu]
preconditions:
  media_crosses_bottleneck: true
  generator_cpu_max: 0.70
outcomes:
  co_primary: {metric: confirmation_time_s, event: drop, comparator: cake_autorate,
               margin_s: 1.0}
  secondary: [onset_time_s, missed_intervention, call_freeze_count, added_delay_p95_ms,
              game_rtt_p99_ms, bulk_leftover_share_pp]
targets:                                    # plans, not results
  - "confirmation_time_s(veriq) <= confirmation_time_s(cake_autorate) + 1.0"
  - "call_freeze_count(veriq) <= call_freeze_count(cake_autorate)"
```

A6: all reflectors unreachable (docs/11 §10; part of the H1 core set):

```
id: A6
version: 1
title: "All reflectors unreachable for 120 s while a call and a download run"
kind: robustness
tiers: [H1]
arms: [sqm_static, cake_autorate, qosmate_autorate_cake, veriq]
duration_s: 360
warmup_s: 30
topology:
  profile: netns-v1
  link: {down_mbit: 50, up_mbit: 10}
  isp_buffer_ms: 300
  buffer_mode: ms
  base_rtt_ms: 20
  ipv6: true
  nat: true
traffic:
  - {id: call, type: jitsi_call, ns: vq-c1, peer_ns: vq-peer, start_s: 0, stop_s: end}
  - {id: bulk, type: iperf3, ns: vq-c4, dir: down, streams: 2, start_s: 10, stop_s: end}
faults:
  - id: blackout
    type: reflector_block          # nft drop of ICMP to 198.18.1.0/24 in vq-core
    targets: all
    onset_s: {uniform: [60, 120]}
    duration_s: 120
ground_truth: [capacity_applied, isp_queue, isp_util, faults, flows, cpu]
preconditions:
  capacity_constant: true
  media_crosses_bottleneck: true
outcomes:
  scenario_metric: {metric: max_abs_rate_deviation_pct, s_ref: rate_before_onset}
  secondary: [rate_reductions_count, magnitude_mbit_s, recovery_time_s,
              call_freeze_count, added_delay_p95_ms]
targets:                                    # plans, not results
  - "max_abs_rate_deviation_pct(veriq) <= 5 during and after the blackout"
  - "recovery without manual action"
```

A6 is not one of the primary-outcome scenarios (S6, S7, S10, A32); its own pass metric is the rate deviation, and the intervention counts are reported alongside. For A6, the expected behaviour of a delay-only autorate is a **hypothesis** that the head-to-head will test. From the source (read), QoSmate's autorate sets latency to 9999 ms when no reflector replies, and its decrease branch does not check load (docs/11 §10).

25.9 The runner workflow

The proposed runner is `lab/run.py`. Its plan is generated from a seed, so the order and onsets can be reproduced:

```
sudo ~/vq-venv/bin/python lab/run.py plan --tier H1 --n 5 --seed 4242 -o plans/h1-pilot.json
sudo ~/vq-venv/bin/python lab/run.py exec plans/h1-pilot.json --ledger ledger/
~/vq-venv/bin/python lab/evaluate.py ledger/ --plan plans/h1-pilot.json
~/vq-venv/bin/python lab/claims_check.py --ledger ledger/ --claims docs/claims.yaml
```

For each run in the plan, the runner performs these steps: 1. **Preflight.** Run `doctor.sh` and check free disk space, clock source and CPU governor. 2. **Topology.** Tear down and rebuild all namespaces from `topo.sh`; no state carries over between runs. 3. **Arm up.** Install exactly one arm. The previous arm's qdiscs, IFB and nft tables must be gone (two enforcers never run at once). Run the per-tin self-test;

an arm that fails it does not count (docs/11 §15.1). 4. **Warm-up.** Start the generators; wait `warmup_s`. 5. **Collect.** Start the ground-truth collectors and the arm's own log capture. 6. **Inject.** Apply the faults at the drawn onsets; log the actual apply times. 7. **Stop.** At `duration_s`, stop the generators, then the collectors; copy the arm's logs and configuration. 8. **Validate.** Check the preconditions. A failed run is marked `invalid` with a reason and kept. 9. **Seal.** Write the manifest with SHA-256 hashes of every file; make the run folder read-only. 10. **Reset.** Arm down, topology down; a cool-down pause before the next run.

The run order is counterbalanced across arms, and a pilot of n = 5 per arm sets n (10–20 expected; docs/11 §15.2). Metrics are computed **only** from the ledger.

25.10 The evidence ledger and manifest

```
ledger/
  index.csv                          one line per run: run_id, scenario, arm, status
  plans/<plan_id>.json               seed, order, drawn onsets
  runs/<run_id>/
    manifest.json                    identity, versions, hashes, validity
    config/                          arm config exactly as applied (UCI, tc, nft, YAML)
    raw/                             ping, iperf3 JSON, webrtc_*.jsonl, dash_events.jsonl,
                                     game.csv, plt.jsonl, tc_stats.jsonl (router side)
    truth/                           capacity_applied, isp_queue, isp_util, faults,
                                     flows.csv, game_owd, headers.pcap, cpu
    sut/                             decision log (JSONL), receipts, arm logs
    metrics.json                     derived; deletable, regenerated by evaluate.py
```


```
{"run_id":"20261105T143012Z-S2-veriq-07","plan_id":"h1-pilot","seed":4242,
 "scenario":{"id":"S2","version":1,"sha256":"…"},
 "arm":{"id":"veriq","version":"0.3.0","git":"a1b2c3d","config_sha256":"…"},
 "faults":[{"id":"drop","onset_drawn_s":87.4,"applied_mono_ms":120734512}],
 "host":{"kernel":"6.8.0-…","iproute2":"6.1.0","nft":"1.0.9","iperf3":"3.16",
         "chromium":"…","dashjs":"…","jitsi":"…","cpu":"…","ram_gb":32},
 "capabilities":{"cake":true,"netem":true,"ctinfo_cpmark":true,"ipv6":true,
                 "offloads_off":true,"base_rtt_applied":true,"nat":true},
 "validity":{"status":"valid","media_crosses_bottleneck":true,"gen_cpu_max":0.41},
 "lab_git":"d4e5f6a","files_sha256":{"raw/game.csv":"…"},"operator":"R3"}
```


All values in this example are illustrative. The `capabilities` block is the lesson of Day-0: it records what was **actually applied**, not what was requested. Invalid and losing runs stay in the ledger and in the report (docs/11 §15.4). The claims check recomputes every published number from the raw files and fails the build if any differs beyond tolerance (docs/05 M9).

25.11 Installing the baselines

Each comparator is version-pinned and tuned **per its own documentation**, and its configuration is archived with every run (docs/11 §15.1). Tuning uses the same procedure for every arm: tune on the scenario's link profile under static conditions (S1), freeze the configuration by hash before the H1 runs, and never re-tune after seeing H1 results.

Arm	Where it runs	Installation and tuning
`fifo`	D1 `vq-rtr`	No qdisc changes; illustrates the problem only
`sqm_static`	D1 and D2	D2: the OpenWrt `sqm-scripts` package with `piece_of_cake` or `layer_cake`, tuned per the OpenWrt SQM guide [R11]. D1: sqm-scripts on Ubuntu if its documentation supports the version used (verify), otherwise the equivalent CAKE commands, labelled "SQM-equivalent" in the manifest
`cake_autorate`	D1 `vq-rtr` or D2	Pinned v3.x release [R12]. Per its documentation, it adjusts the bandwidth of CAKE qdiscs that already exist, so it runs on top of `sqm_static`. Its config names the upload interface (`rtr-wan`) and the download interface (the IFB), the min/base/max rates, and the reflector list, set to the lab reflectors. Thresholds are tuned per its documentation
`qosmate_*`	D2 only	QoSmate requires OpenWrt 23.05 or later with firewall4 and nftables (README, read). Install at the pinned commit `c507843` : take the README's `uclient-fetch` commands and replace `$LATEST_TAG` with the commit, rather than fetching the latest release. Set `ROOT_QDISC` to `hfsc` (default) or `cake`. Enable `config autorate` and set `reflectors` to the lab addresses

Arm	Where it runs	Installation and tuning
`veriq`	D1 (Linux build, plain-file config) and D2 (ipk from the local feed)	Lab profile: DSCP untouched (docs/11 §4.6), reflectors and near target overridden, start mode `live` for scenario runs; SHADOW runs are their own scenario (S12)

The **controller-only** comparison puts QoSmate (CAKE mode + autorate), cake-autorate and VeriQ on the same CAKE datapath at the same rates and overhead. The **whole-system** comparison runs each tool in its recommended configuration, which for QoSmate means the HFSC default (docs/09 §5).

QoSmate's README notes that its script installs missing packages and that other QoS services must be disabled first (read). In D2, provision the VM while it has temporary internet access, let QoSmate install its dependencies once, snapshot the VM, and run all scenarios from that snapshot with no internet access.

25.12 SUT-side faults

Fault	Scenario	Injection	Where
Controller crash	S8, A15	`kill -9` of the controller process	D1, D2, D3
Foreign qdisc	A30	`tc qdisc replace dev rtr-wan root fq_codel` from outside the arm	D1, D2
Firewall flush	A31	D1: `nft delete table inet veriq`; D2: `/etc/init.d/firewall restart`	D1, D2
Router CPU saturated	A17	`stress-ng` on the router's cores	**D2/D3 only**
NTP clock step ±1 h	A18	`date -s '+1 hour'`	**D2/D3 only**
Flow offloading on	A24	`uci set firewall.@defaults[0].flow_offloading=1`	D2/D3
Invalid custom nft	A25	A malformed rule in `/etc/veriq.d/custom.nft`	D1, D2

> **Implementation tip.** Never step the clock in D1: namespaces share the host clock, so `date -s` would shift the whole lab, evaluator included. CPU stress in D1 is unrealistic too, because softirq work cannot be confined to one namespace.

25.13 Reproducing Day-0 with the corrected fingerprint

The Day-0 evidence is one run, done on 11 Oct 2026 in a sandbox whose kernel lacked CAKE, fq_codel, netem and IPv6 [R51]. Its measured added p95 delay was:

Arm	Download	Upload
`fifo`	63.6 ms	266.1 ms
`shortfifo` (short-buffer shaper)	55.4 ms	39.6 ms

These are **preliminary, unreplicated** values: 20/5 Mbit/s, a 300 ms ISP buffer, `DURATION=15`, IPv4 only and **no base RTT** (idle RTT ≈ 0.1 ms). The original `environment.txt` wrongly stated "base RTT 20 ms (netem)". The evidence README corrects it, and the script now writes the capabilities it actually applied (`NETEM_OK`, `V6`).

To reproduce on a full kernel:

```
sudo lab/doctor.sh || exit 1
for i in $(seq -w 1 10); do
  sudo DURATION=15 OUT="results/day0-rerun/run$i" \
    lab/quickstart_bufferbloat.sh fifo shortfifo fq_codel cake
done
grep -h 'base RTT\|IPv6' results/day0-rerun/run*/environment.txt | sort | uniq -c
```

Then follow these steps: 1. **Check the fingerprint first.** Every `environment.txt` must say "netem applied" and "IPv6: enabled". If not, the run is not a reproduction. 2. **Expect different numbers.** With 20 ms of base RTT, TCP's window dynamics change, so the values will not match the sandbox ones. Do not "reconcile" them; report the rerun as new evidence beside the labelled sandbox values. 3. **Aggregate per arm** across the 10 runs: the median of `added_p95_ms` with a bootstrap confidence interval [R37], computed from the per-run `summary.csv` files. 4. **Store the reruns** as a new evidence folder with its own README. Never overwrite `evidence/day0_sandbox_2026-10-11/`.

This rerun is the Sprint 0 exit criterion: "lab FIFO vs CAKE run with base RTT" (docs/11 §18).

25.14 The OpenWrt VM lab

D2 runs OpenWrt 24.10 ×86-64 in QEMU/KVM, used as the router in place of `vq-rtr`. It is needed for QoSmate, for the package tests (docs/11 §14.4) and for the A17/A18 faults. QEMU runs inside a helper namespace `vq-vmh`, which holds two bridges. `br-vlan` joins `tap-lan` to the LAN switch, and `br-vwan` joins `tap-wan` to `vq-isp`. Taps created by QEMU in that namespace stay there.

```
nsx vq-vmh qemu-system-x86_64 -enable-kvm -cpu host -smp 2 -m 512 -nographic \
  -drive file=vm/openwrt-24.10-x86-64-generic-ext4-combined.img,format=raw,if=virtio \
  -netdev tap,id=lan,ifname=tap-lan,script=no,downscript=no \
  -device virtio-net-pci,netdev=lan \
  -netdev tap,id=wan,ifname=tap-wan,script=no,downscript=no \
  -device virtio-net-pci,netdev=wan
```

In the VM, set the LAN to 10.10.1.1/24 and the WAN to 10.10.2.1/24 with gateway 10.10.2.2, so the rest of the lab is unchanged. Disable offloads with `ethtool -K` (install the `ethtool` package) and keep software flow offloading off, except in A24. Packages come from a local feed served by nginx in `vq-srv`. Use a single vCPU for resource scenarios, so that CPU limits show up as they would on a small router.

25.15 The physical lab with the Pi

D3 replaces the router with a Raspberry Pi running OpenWrt 24.10 (bcm27xx). The on-board port is the WAN, and the USB 3 gigabit adapter is the LAN (docs/06 §2). The ISP-emulator laptop runs `vq-isp`, `vq-core`, `vq-refl`, `vq-srv` and `vq-peer`, with its two physical ports moved into namespaces (`ip link set enp2s0 netns vq-isp`). Real clients connect to the switch: the call laptop, a TV or streamer, a console and a download PC.

- **Clocks:** run a local NTP server in `vq-core`. The Pi and the clients synchronise to it, so logs merge on wall time; control timing still uses monotonic clocks.
- **Wi-Fi:** real WIFI evidence needs station statistics from hostapd on an OpenWrt access point. Record the radio, channel and client distance; Wi-Fi results are hardware-specific.
- **Thermals:** use the active cooler and record the SoC temperature; throttling can mimic CPU_LIMITED.
- **Stage 4:** bench and soak runs of 2–4 h (docs/09 §5) use this setup; RSS and CPU are compared with the footprint budgets of docs/11 §12.2, which are targets.

25.16 Troubleshooting

Symptom	Likely cause	Check	Fix
`Error: Specified qdisc kind is unknown`	Module missing (as in Day-0)	`modprobe sch_cake`; `doctor.sh`	Install `linux-modules-extra-$(uname -r)`
CAKE arm shows no improvement	Shaping on the wrong interface, or the rate is above the bottleneck	`tc -s qdisc show` on `rtr-wan` and the IFB: are packets and drops counting?	`tc` shapes egress only: download needs the IFB (or the LAN egress)
Delay figures erratic, goodput above line rate	Offloads on (GSO/GRO super-packets)	`ethtool -k <dev>` in each namespace	Turn offloads off on every veth
All traffic lands in the Best Effort tin	Marks not set, wrong fwmark mask, or `ctinfo` not restoring	Per-tin counters; `conntrack -L -o extended`; `nft monitor trace`	Fix the mark plumbing; run the tin self-test
Call media does not appear on `isp-cpe`	p2p still on, or the JVB is on the LAN side	getStats candidate pair; `tcpdump -ni isp-cpe udp port 10000`	Disable p2p; advertise the WAN-side JVB address
Browser CPU above 70%	720p fake video is too heavy	`cpu` channel	Use a 360p pattern; keep the same file for every arm

Useful checks:

```
nsx vq-isp tcpdump -ni isp-cpe -c 20 -v 'udp port 10000'      # media crosses; tos = DSCP
nsx vq-rtr tc -s -j qdisc show dev rtr-wan | jq '.[] | select(.kind=="cake") | .tins'
nsx vq-rtr nft list table inet veriq | grep -c 'counter packets'
nsx vq-rtr conntrack -L -o extended 2>/dev/null | grep -c 'mark='
```

The JSON field names of CAKE's per-tin statistics (sent packets, drops, ECN marks, average and peak delay, backlog) depend on the iproute2 version. Pin the parser to the version in the manifest and verify the names once on the target.

25.17 Safety notes

- **Privilege.** The lab needs root or `CAP_NET_ADMIN`, and CI needs a privileged runner or container (docs/11 §14.4) that can reconfigure the host's network. Use a dedicated machine or VM, and clean up namespaces with the script's `cleanup` trap.

- **Never run against networks you do not own.** Fault injection, probing at full rate, BitTorrent and bulk generators stay inside the namespaces. Leak check after every topology build: from `vq-rtr`, `ping -c 2 -W 1 1.1.1.1` and a lookup of a public name must **fail**. The host must forward nothing from `host-core` except to the Jitsi bridge network.
- **Content.** Torrent only freely redistributable Linux ISOs, with a local tracker and DHT, peer exchange and local discovery all disabled. Use test patterns and synthetic speech for calls, never recordings of real people.
- **Physical kit.** In D3, the ISP emulator is the only path out of the lab LAN. Probing etiquette applies if the kit is ever connected to a real line: at most 10 probes per second in total (docs/11 §13).
- **Labels.** Every chart and number from this lab says “emulated”, states its run count, and names its evidence folder.

> **For judges.** “Our numbers come from an emulated lab whose faults are injected at random times the controller cannot see. Every run’s raw data, configuration and fingerprint is in the ledger, and a script regenerates every number we publish. Day-0 is one preliminary sandbox run, and we say so.”

25.18 Lab readiness checklist

- `doctor.sh` passes on the lab machine; the health check and self-test pass in the VM and on the Pi.
- The Day-0 rerun has 10 runs with base RTT and IPv6 in the fingerprint.
- Media, game and bulk counters are verified on `isp-cpe` in S1.
- The leak check fails as expected.
- Every arm passes the per-tin self-test; configurations are hashed and frozen.
- S2 and A6 YAML files validate; their onsets are drawn from the seed.
- The ledger is sealed and read-only, and `evaluate.py` regenerates `metrics.json` from raw data alone.
- The claims check passes in CI.

Part V · Business, KPIs and risk

Why it matters beyond the competition

CHAPTER 26

Key performance indicators

A key performance indicator (KPI) is only useful if four things are fixed in advance: what exactly is counted, where the data comes from, what value counts as success, and who answers for it. This chapter defines VeriQ's KPIs in five families (technical, product, operator, engineering and competition) and maps each one to the dashboard or report where it appears.

> **Honesty check.** At the time of writing, **no VeriQ KPI has been measured.** The only measured numbers in the project are the Day-0 sandbox values (FIFO added p95: upload 266.1 ms, download 63.6 ms; short-buffer shaper: upload 39.6 ms, download 55.4 ms; one preliminary run, no base RTT applied, IPv4 only) [R51]. Every target below is a **target**, and every operator-side effect is a **hypothesis**.

26.1 Labels used in this chapter

Label	Meaning	Example
Target	A value the team commits to test against; fixed before the final runs	Time-to-safe ≤ 12 s
Non-inferiority target	VeriQ must not be worse than a comparator by more than a stated margin	Confirmation ≤ cake-autorate + 1 s
Comparative target	VeriQ must be better than a comparator; a tie or a loss is reported as such	Fewer inappropriate interventions than QoSmate autorate
Hypothesis	An expected effect that only a pilot or study can test	Fewer "slow internet" contacts
Formative	Measured on a small sample to guide design, not to prove a claim	SUS from 6–8 participants

Two measurement rules apply to every KPI (docs/09 §5, docs/11 §15.4): - the **run** is the experimental unit, not the packet; run-level values are compared with bootstrap confidence intervals [R37] and secondary outcomes are Holm-corrected [R38]; - missing data is reported as missing, never imputed; results where VeriQ ties or loses are kept.

26.2 The KPI tree

```
VeriQ succeeds
|
+-- Technical: does the controller and datapath do the right thing?
|   +-- Controller: primary triplet, confirmation time, bulk share, oscillation, recovery
|   +-- Experience: added delay, call freezes, concealment, DASH stalls, game RTT, PLT
|   +-- Fairness: Jain's index, starvation windows
|   +-- Classification: coverage, selection rate, selective accuracy, tin match
|   +-- Diagnosis: label accuracy (confusion matrix), estimator tests E1-E8
|   +-- Safety: time-to-safe, self-heal time, refusal-code coverage, flapping
|
+-- Product: would a household and a router vendor accept it?
|   +-- setup time, footprint (RSS, flash, CPU), SUS, install success, self-test pass
|
+-- Operator: would an ISP pilot it? (hypotheses until a pilot)
|   +-- contacts per 1000 homes, shadow disagreements, crash loops, SAFE time, opt-outs
|
+-- Engineering: is the team building it well?
|   +-- CI pass rate, coverage, property tests, replay determinism, sprint delivery
|
+-- Competition: does the submission land?
    +-- deadlines met, claims traced, labels present, demo rehearsal success, Q&A coverage
```

The technical family carries the research claim. The other four families decide whether that claim turns into something a household, an operator or a panel can use.

26.3 Technical KPIs

26.3.1 Controller outcomes

These are the outcomes fixed in docs/09 §5 and docs/11 §15.3. They are measured by the evaluator from the ledger, not by veriqd itself, so that the system under test never scores its own work.

KPI	Definition and formula	Data source	Target (label)	Owner
Primary: inappropriate interventions	In control-futile scenarios (S6, S7, S10, A32) only, per run: (1) count of rate reductions of any size; (2) cumulative magnitude ∫max(0, S_ref − S(t))dt in Mbit·s, where S_ref is the shaper rate just before the disturbance; (3) duration with S < 0.95·S_ref. A run counts only if the evaluator confirms access utilisation < 80% and ISP-emulator queue < 5 ms	Decision log (S over time), ISP-emulator ground truth	Comparative target: lower than cake-autorate and QoSmate autorate	R3
Co-primary: confirmation time	In genuine-drop scenarios (S2/A1): earliest t_1 after the event at t_o after which S stays within ±10% of its final value for 5 s; reported as $t_1 - t_o$. Onset (first rate change after t_o) is also reported	Decision log, fault schedule	Non-inferiority target: ≤ cake-autorate + 1 s	R3
Secondary: bulk share of leftover	Bulk goodput ÷ leftover bandwidth, where leftover = S − Σ rate(protected flows), over the run	Harness flow counters	Non-inferiority target: ≥ comparator − 5 percentage points (absolute)	R3
Missed interventions	Genuine drops with no cut within 5 s	Decision log, fault schedule	Target: 0 in S2/A1 runs	R2
Oscillation	Direction reversals of S per minute	Decision log	Target: ≤ comparator (A2)	R2
Recovery time	Time from fault clearance until S and delay return to their pre-fault regime	Decision log, harness	Target: ≤ comparator	R2
Reclaim time	Time to reach 90% of a raised capacity (A3)	Decision log	Target: ≤ comparator	R2

Worked example (illustrative, not measured). Suppose in an S6 run the comparator cuts a 90 Mbit/s rate to 60 Mbit/s for 40 s and then recovers. Its magnitude is (90 − 60) × 40 = 1200 Mbit·s, its count is at least 1 and its duration below 0.95 × 90 = 85.5 Mbit/s is 40 s. If VeriQ holds at 90 Mbit/s, all three are zero. If VeriQ instead takes one 5% fallback step for 10 s, its magnitude is 4.5 × 10 = 45 Mbit·s, its count is 1, and its duration below 85.5 Mbit/s is zero, because the step lands exactly on 0.95·S_ref. The KPI reports all three numbers, because a single count would hide the difference between one small step and one deep cut.

26.3.2 Experience and fairness

KPI	Definition and formula	Data source	Target (label)	Owner
Added delay p95	p95 of RTT under load − median idle RTT, per direction (the Day-0 metric), with a minimum sample count per run	Harness probes	Target: S1 ≤ 10 ms (docs/05); static parity with SQM ≤ +2 ms (docs/11 §17)	R3
WebRTC freezes	freezeCount and totalFreezesDuration from getStats [R35]	Browser getStats	Target: S1 0 freezes in the VeriQ arm; elsewhere ≤ comparator	R3
Audio concealment ratio	concealedSamples ÷ totalSamplesReceived [R35]	Browser getStats	Target: ≤ comparator (A22: ≤ SQM)	R3
DASH stall ratio	Total stall time ÷ session time; stall count	dash.js events	Target: S3 0 rebuffers at a sustainable bitrate	R3
Game RTT p99	p99 added RTT of the synthetic 64 pps UDP flow	Harness	Target: S3 ≤ 20 ms added	R3
Page load time	Median and p95 PLT over 10 locally cached sites	Playwright	Target: ≤ comparator (Holm-corrected secondary)	R3
Jain's fairness	$J = (\Sigma x_i)^2 \div (n \cdot \Sigma x_i^2)$ over per-host throughputs [R36]	Harness	Target: ≥ 0.9 across hosts (S3, A10)	R3
Starvation	Number of 10 s windows with 0 bulk bytes while bulk demand exists	Harness	Target: 0 (A12, A13)	R3

Worked example (illustrative). Three hosts receive 30, 30 and 20 Mbit/s. Then $\Sigma x = 80$, $\Sigma x^2 = 900 + 900 + 400 = 2200$, and $J = 6400 \div (3 \times 2200) \approx 0.97$. If one host takes 60 and the others 10 each, $J = 6400 \div (3 \times 3800) \approx 0.56$, which fails the target.

26.3.3 Classification

KPI	Definition and formula	Data source	Target (label)	Owner
Tin match	Share of self-test probe packets counted in the intended tin, per class and direction	Stage-1 self-test, `tc -s -j`	Target: 100%	R1
Conformal coverage	Share of prediction sets containing the true label, per class and checkpoint [R29]	ML test split (time and app held out)	Target: ≈ 1 − α = 0.95 on exchangeable data; reported under shift	R4
Selection rate	Share of flows with a singleton set (the flows VeriQ may act on)	ML test split	Reported, no fixed target (low selection is acceptable because abstention means BE)	R4
Selective accuracy	Accuracy among selected flows, per class	ML test split	Target: reported per class with CIs; compared with the QoSmate-style heuristic baseline	R4
Abstention rate (on device)	1 − selection rate, weekly	veriqd counters	Alert if > 80% (docs/11 §5.6)	R4
Sanction latency	Time from an envelope violation to the flow being in BE	nft counters, NFLOG events	Target: ≤ 1 s (S11, A11)	R2
Preference without basis	Flows in RT/IV whose PROV is not lease, user rule, app mark or ML singleton	Kernel set dump + decision log	Target: 0	R2

26.3.4 Diagnosis

KPI	Definition and formula	Data source	Target (label)	Owner
Label accuracy	Confusion matrix of diagnosed labels against the emulator's ground-truth cause, per scenario family	Decision log + ground-truth channel	Target: reported with CIs; REMOTE vs WAN confusion is the key cell	R2
Estimator tests E1–E8	Pass rate of the proposed estimator accuracy tests (step-down tracking, step-up tracking, oscillation, idle drift, censoring correctness, baseline re-learn, reflector-fault robustness, direction attribution)	Lab traces	Target: all pass before the evaluation freeze	R2
Upload-only cut correctness	Download S change during an upload-only saturation (A9)	Decision log	Target: within ±5%	R2

26.3.5 Safety

KPI	Definition and formula	Data source	Target (label)	Owner
Time-to-safe	Time from `kill -9` of veriqd to SAFE rates applied	Watchdog log, `tc` read-back	Target: ≤ 12 s	R1
Self-heal time	Time from a firewall restart flushing the table to `inet veriq` restored	Reconciler log	Target: ≤ 1 s (A31)	R1
Refusal-code coverage	Refusal codes with a passing unit test ÷ all refusal codes	CI	Target: 100%	R2
Flapping	Number of ownership changes per hour with a foreign tool present	Reconciler log	Target: 0 (A30)	R1
Probe-loss stability	S change during an all-reflectors-unreachable fault (A6)	Decision log	Target: within ±5% of the pre-fault S	R2

26.4 Product KPIs

KPI	Definition and formula	Data source	Target (label)	Owner
Setup time	Wall time from opening the wizard to "Done", median over study participants	UX study timing	Target: ≤ 5 min	R4
Footprint, Standard	veriqd RSS; installed flash; veriqd CPU as % of one A53 core, over a 2–4 h soak	`/proc`, `opkg` sizes, soak logs	Target: ≤ 40 MB RSS, ≤ 10 MB flash, ≤ 5% CPU	R1
Footprint, Lite	Same metrics on the D5 device	Same	Target: ≤ 20 MB RSS, ≤ 6 MB flash, ≤ 3% CPU	R1
Footprint, Pro	Same metrics	Same	Target: ≤ 60 MB RSS, ≤ 14 MB flash, ≤ 7% CPU	R1
SUS score	System Usability Scale score [R39] of VeriQ LuCI, with QoSmate LuCI as the comparison	UX study	Formative target: ≥ 75	R4
Task success	Share of participants completing: prioritise a console, diagnose a bad call, undo a change	UX study	Formative; reported per task	R4
Install success	Share of CI targets where the package installs and the health check passes	CI package tests	Target: 100% of CI targets	R1
Self-test pass	Share of supported devices where the tin self-test passes in both directions	Health check	Target: 100% of D2–D5	R1

> **Note.** CAKE's own CPU cost is part of the kernel and scales with the rate. The footprint KPI measures veriqd only. The bench reports CAKE's cost per device separately, and the evaluation never attributes one to the other (docs/11 §12.2).

26.5 Operator KPIs (pilot hypotheses)

These KPIs only make sense in an operator pilot (see the Business case chapter). They are listed now so that the pilot design and the data collection are ready before a pilot exists.

KPI	Definition and formula	Data source	Target (label)	Owner
"Slow internet" contacts	Contacts tagged slow/lag per 1000 homes per month, pilot vs control group	Operator ticketing system	Hypothesis: lower than control	Operator + R3
Shadow disagreement rate	Disagreements between VeriQ's would-be action and the active static configuration, per home-day, with the share judged unsafe	Shadow report	Gate: ≤ 5 unsafe per hour per home for SHADOW → LIVE (docs/11 §8.2)	R2
Crash loops	Homes where procd stopped respawning veriqd	Support bundle, fleet report	Stop criterion: any unexplained crash loop	R1
Time in SAFE or DEGRADED	Share of home-hours not in LIVE or SHADOW	State logs	Target set before the pilot; investigate every home above it	R1
CPU over budget	Homes where veriqd exceeds its profile budget	Soak and fleet metrics	Stop criterion: any persistent exceedance	R1
Opt-out rate	Pilot households leaving the pilot	Pilot records	Reported; reasons logged	Operator
Receipt outcome mix	Share of IMPROVED / NOT_IMPROVED / INCONCLUSIVE receipts	Receipts	Reported; a high INCONCLUSIVE share is information, not failure	R2

> **Honesty check.** A 20–50 home pilot cannot show a change in support contacts with any statistical confidence: at an illustrative 1.5% monthly contact rate, 50 homes produce under one contact a month. The pilot's job is safety and acceptance; the contact KPI needs a larger, controlled rollout.

26.6 Engineering KPIs

KPI	Definition and formula	Data source	Target (label)	Owner
CI pass rate	Green main-branch builds ÷ all main-branch builds, weekly	CI history	Target: ≥ 95%	Lead
Line coverage	Coverage of `checker`, `control`, `diagnose`, `classify`	`go test -cover`	Target: ≥ 80% each	R2
Property tests	Failures over 10^6 fuzz cases for the envelope, the budget and precedence monotonicity	Go fuzzing	Target: 0	R2
Replay determinism	Recorded runs whose replay reproduces every decision	`veriqctl replay`	Target: 100%	R2
Reproducible build	Two clean builds of the same tag produce identical binaries	CI	Target: identical (`-trimpath`, pinned modules)	R1
Sprint delivery	Exit criteria met on the sprint's last day ÷ exit criteria planned	Tracker	Target: 100% for P0; misses trigger the cut ladder	Lead
Open P0 defects	Defects blocking a P0 exit criterion	Tracker	Target: 0 at each sprint review	Lead
Licence compliance	Third-party modules with a non-permissive licence	`go-licenses`-style report	Target: 0	R1

26.7 Competition KPIs

KPI	Definition and formula	Data source	Target (label)	Owner
Deadlines met	Submissions made by the internal deadline (22 Oct, 11 Dec)	Portal confirmations	Target: 100%	Lead
Claims traced	Numbers in the submission traced to a ledger entry or labelled target/assumption/illustrative ÷ all numbers	Claims sheet	Target: 100%	Claims steward
Honesty labels	Figures and slides carrying a measured/estimated/illustrative tag ÷ all figures	Review checklist	Target: 100%	R4
Criteria coverage	Case 3 evaluation criteria with a dedicated slide or section and evidence: classification and estimation accuracy; latency, jitter and loss reduction; fairness and no starvation; adaptation speed and stability; CPU, reproducibility and clarity	Self-assessment	Target: 5 of 5	Lead
Demo rehearsal success	Full rehearsals completed without manual intervention ÷ rehearsals	Rehearsal log	Target: last 2 of 3 clean	R4
Reproduction from scratch	A member who did not build the lab reproduces one headline result from the setup guide	Dry-run log	Target: done before submission	R3
Q&A coverage	Questions in the bank drilled aloud by every presenter	Drill log	Target: all 50 at least once	Lead

26.8 Dashboard mapping

Each KPI is shown in one primary place. The rule is: household-facing screens never show research KPIs, and research reports never use numbers that the ledger cannot regenerate.

KPI group	Primary place	Secondary place	Refresh	Audience
Controller state, S, $\hat{G}$, μ±σ, D, evidence level	LuCI Autorate page	Grafana (exporter, P2)	Live (1 s)	Admin, demo

KPI group	Primary place	Secondary place	Refresh	Audience
Added delay and grade	LuCI Overview	Household page as a traffic light	Live	Admin, household
Per-tin bytes, drops, delay	LuCI Queues page	Grafana	Live	Admin
Primary, co-primary, secondary outcomes	Evidence report (generated from the ledger)	Submission results pages	Per evaluation	Judges, team
Experience and fairness	Evidence report	Demo video overlays (from real runs only)	Per evaluation	Judges
Classification metrics	Honesty-kit report	LuCI Connections (per flow: class, provenance, confidence or “abstained”)	Per model release; live per flow	Judges, admin
Safety KPIs	Stage-1 report, CI	LuCI Diagnostics (self-test, health)	Per build; on demand	Team, admin
Product KPIs	Bench and soak report; UX study report	LuCI Diagnostics (bench)	Per release	Team, judges
Operator KPIs	Pilot report	Fleet view via TR-369/USP [R27] (P2)	Weekly in a pilot	Operator
Engineering KPIs	CI dashboard, tracker	Weekly demo	Per push; weekly	Team
Competition KPIs	Claims sheet, rehearsal log	Sprint review	Weekly	Team

26.9 From KPI to decision

A KPI that never changes a decision is decoration. The table below links KPI results to the action the team takes. Several rows come straight from the revert conditions in the v5 change record (docs/12).

KPI result	Decision	Recorded in
False L2 cuts in H1 control-futile runs exceed the comparator	Revisit C1 (the one-step cut) per its revert condition; report the result as it is	Change record, evidence report
Confirmation time fails the +1 s margin in S2/A1	Investigate $\hat{G}$ freshness and refractory timing; no change to the outcome definition after the freeze	Decision log
Calibration below 200 samples per class at a checkpoint	That checkpoint is disabled (docs/11 §5.6); the classifier abstains there	Model release notes
Abstention > 80% on a network	UI states that the model is uncertain on this network; no on-device retraining	Alert log
Time-to-safe above 12 s in any trial	Blocks the Sprint 2 exit; fix before new features	Tracker
Footprint above the profile budget on a device	Recommend a lighter profile or backend for that device class; do not raise the budget silently	Bench report
SUS below the formative target	Redesign the failing task flow before the finale; report the score anyway	UX study report
Shadow report shows anomalies	Stay in SHADOW; the user must confirm LIVE (docs/11 §8.2)	Shadow report
Claims traced < 100% the day before submission	Remove or relabel the untraced number; never submit it	Claims sheet

26.10 Rules that keep the KPIs honest

- **Fix before running.** Outcomes, margins and comparators are frozen before the final runs, and the freeze is recorded in the decision log.
- **No small-sample tails.** No p99.9 claims from small samples; p99 only with a stated sample floor.
- **Same arms, same runs.** A comparative target is only evaluated on runs where both arms faced the same scenario in counterbalanced order.
- **Parity is a result.** In static conditions, VeriQ and QoSmate’s CAKE mode are expected to be at parity, because both use CAKE. A parity result is reported as parity, not as a win.
- **One owner per KPI.** The owner does not grade their own module’s KPI alone: the evaluator computes it, and the owner explains it.

> **For judges.** “Our KPI tree starts from the case’s five evaluation criteria and the pre-registered primary outcome. Each KPI has a formula, a data source, a labelled target and an owner. None of them is a result yet; the evidence report will fill them from the ledger, including the ones where we tie or lose.”

CHAPTER 27

Business case

The case brief asks the team to describe "the boundary between the prototype and a production-scale solution, including security, performance, hardware and operational considerations". This chapter covers the business side of that boundary: who gains what, how VeriQ could be packaged, what it costs to build and deploy, how an operator might value it, and how it would reach homes.

> **Honesty check.** Nothing in this chapter is measured. The operator model is **illustrative**: its inputs are assumptions carried over from the v4.1 specification (docs/09 §7), to be replaced with operator data. Component prices are **approximate, verify before purchase**. No Tata Elxsi endorsement, product plan or integration is implied anywhere in this chapter.

27.1 The value chain

VeriQ's value starts in the home and travels up through the operator to the engineering-services firm that builds gateway software. Each link has a different currency: the household feels fewer frozen calls, the operator sees fewer support contacts and better quality telemetry, and an engineering-services firm gains a reusable accelerator and a validation method.

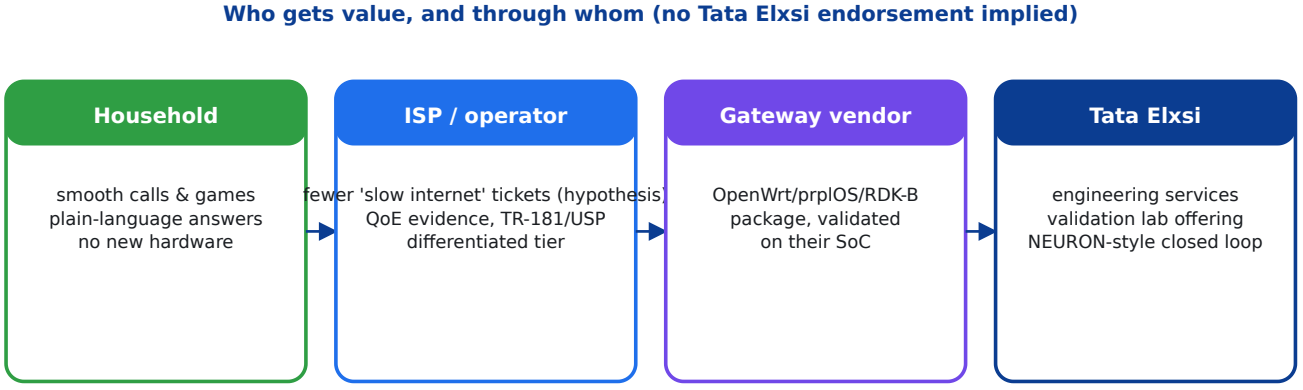


Figure 14. Stakeholder value chain: household, ISP/operator and Tata Elxsi, with what each gives and receives.

Read the chain from left to right:

1. **Household.** It runs mixed traffic on one line: calls, games, streaming, downloads, backups, updates. It wants calls that do not freeze and an honest answer to "why is it slow?".
2. **Gateway software.** VeriQ runs on the home router or the operator's gateway. It keeps the queue under its own control, cuts only when evidence supports it, and explains each action.
3. **Operator / ISP.** It sells the line and supports the customer. Bufferbloat and misdiagnosed slowness generate contacts that a speed test cannot explain, because the speed test shows full throughput.
4. **Gateway vendor or engineering-services firm.** It builds and validates gateway software for operators on platforms such as OpenWrt, prplOS and RDK-B [R44]. An open, tested QoS component with an evaluation harness shortens that work.

27.2 Stakeholder value

Stakeholder	Pain today	What VeriQ offers	Evidence still needed
Household (e.g. a parent working from home, a student gamer, an elder on video calls)	Calls freeze when someone downloads; no idea why; router QoS pages full of knobs	Static CAKE from minute one; adaptive control that holds when cutting cannot help; "Protect my call for 1 hour"; plain-language "why is it slow?"	H1 results; UX study (formative)
Operator customer support	"Slow internet" contacts where the speed test looks fine; little evidence from inside the home	Receipts and a support bundle that show what happened and where the delay came from (access link, remote path, Wi-Fi, router CPU)	Pilot: contacts per 1000 homes (hypothesis)
Operator product team	Hard to differentiate a broadband plan beyond Mbit/s	A "work-from-home ready" feature with measurable latency under load	Market validation (not in scope)

Stakeholder	Pain today	What VeriQ offers	Evidence still needed
Operator network operations	Fixed-wireless (FWA) and shared PON capacity varies; static shaping wastes capacity or lets bloat return	A diagnosis-gated controller that tracks genuine drops and records each decision	H1/H3 results; pilot shadow report
Gateway vendor / engineering services	Each operator project re-builds QoS, its tests and its documentation	A clean-room, permissively licensed core (licence decided by Tata Elxsi; Apache-2.0 proposed), a scenario library and an evidence ledger	Port to one carrier-grade stack (P2)
OpenWrt community	Strong tools exist (SQM, cake-autorate, QoSmate), each covering part of the problem	A QoSmate-compatible importer, packages for five architectures, a self-test	Community feedback after the feed is public (P2)

> **Why it matters.** The household gets the benefit, but the operator pays for gateways, support and firmware. A business case has to show value at the operator level, because that is where a deployment decision is made.

27.3 Tata Elxsi context

docs/01 records that Tata Elxsi's Media & Communications vertical (MCV) was its fastest-growing vertical in Q1 FY27, and that its public announcements in this area include RDK-B broadband gateway work (including RDK-B on a 5G FWA platform with QoS features, and an OpenWrt-to-RDK-B migration accelerator) and the NEURON autonomous-network platform [R46][R48][R49]. docs/01 also notes that Tata Elxsi grows by attaching accelerators to services deals.

That suggests a **thematic** fit for VeriQ as an engineering accelerator: an OpenWrt-native QoS component with a migration path to prplOS and RDK-B, and a closed-loop, evidence-graded controller that resembles the intent-based operations story in miniature. Compatibility with any Tata Elxsi product is **not established**, and no Tata Elxsi integration or endorsement is implied.

27.4 Indian deployment context

Three features of Indian home broadband shape the case. They are general observations, not market statistics.

- **Access mix.** Homes are served by FTTH/PON, cable, and increasingly 4G/5G fixed-wireless access (FWA). PON capacity is shared among neighbours, and FWA capacity changes with radio conditions and cell load. Both make a single static shaper rate a poor fit for at least part of the day, which is the "capacity moves" problem that VeriQ's controller addresses.
- **Who owns the router.** Many homes use the router or ONT supplied with the connection; others add a retail router. An operator-supplied gateway is where Option B applies. A retail OpenWrt router is where Option A applies.
- **Household diversity.** One line often serves several generations: a parent on work calls, a student in online classes or gaming, an elder on video calls with family. That is why the household page is mobile-first and plain-language, with Hindi alongside English at P1.

The low illustrative value per home (calculated below) also fits this context: in a price-sensitive market, the only form that can work is software on gateways already deployed, not extra hardware in the home.

27.5 What a buyer could do instead

A business case has to compare VeriQ with the buyer's alternatives, not with doing nothing. The comparison below is from an operator's point of view. Statements about other tools are general, or come from docs/09 §4 and docs/11 §16; VeriQ's advantages are design claims until H1 tests them.

Alternative	What the operator gets	What it leaves open	VeriQ's honest position
Do nothing (FIFO in the modem)	No cost	Bufferbloat under load; the Day-0 run measured 266.1 ms added upload p95 behind a FIFO (one preliminary run, no base RTT)	VeriQ's static CAKE layer alone addresses this; so does any well-tuned SQM
Static SQM (CAKE at a fixed rate) [R11]	Mature, simple, well understood	A fixed rate is too high when capacity drops and too low when it recovers	VeriQ holds this as its floor (the dominance invariant) and adapts only on evidence
QoSmate with autorate [R13]	A mature, community-proven OpenWrt QoS tool with several schedulers, rules, a LuCI app and an optional autorate	Per its source (read), its autorate averages reflector RTTs and has no load check in the decrease branch; whether that causes inappropriate cuts in practice is a **hypothesis** for H1	Parity expected in static conditions; possible gains in adverse conditions, explainability and safety, to be tested
cake-autorate [R12]	A dedicated, well-documented CAKE rate controller	Rate control only; no classification, intent or product UI	Co-primary comparator; VeriQ must be non-inferior on genuine drops

Alternative	What the operator gets	What it leaves open	VeriQ's honest position
Commercial router "adaptive QoS"	Bundled features, often signature-based classification and static rates	Closed; hard to evaluate; payload or signature inspection raises privacy questions	VeriQ is open and inspectable, and classifies from headers and timing only
Network-side approaches (L4S, NQB marking) [R20][R21]	Standards-based low latency where endpoints and networks support it	Needs application and network support; the home queue still matters	Complementary: VeriQ maps NQB to its IV class, and L4S/NQB experiments are P2

The table makes one point clear: VeriQ's business value rests on the same claim as its research value. If H1 shows parity with a tuned QoSmate or cake-autorate, the remaining case is explainability, safety and household UX, and the submission must say so.

27.6 Packaging options

Three packaging options fit the design. They are not exclusive: option C uses options A and B as its raw material.

27.6.1 Option A: open-source core

- **What:** `veriq`, `luci-app-veriq`, `veriq-household`, `veriq-catalog` and `veriq-model` as OpenWrt packages in a signed feed (docs/11 §12.1).
- **Licence:** decided with Tata Elxsi (O5; Apache-2.0 proposed). Third-party Go modules are MIT/BSD/Apache only.
- **Who pays:** nobody directly. The value is adoption, feedback and credibility.
- **Strengths:** reviewable code builds trust; the community tests on many devices; QoSmate users can import their configuration.
- **Weaknesses:** no revenue on its own; support load; the IP rules of the competition must be checked first (see "IP and licensing").

27.6.2 Option B: operator edition

- **What:** the same daemon, plus the operator-facing parts listed as P2 in docs/11:
  - a TR-181 data model (`Device.X_VERIQ_*`) and a TR-369/USP agent integration for fleet management [R27];
  - staged rollout of controller and model versions, with every home starting in SHADOW;
  - aggregated receipts and shadow reports per cohort;
  - support tooling: the support bundle and the explain API surfaced to agent dashboards;
  - an RDK-B component wrapper, which needs a partner.
- **Who pays:** an operator, as part of a gateway firmware programme.
- **Strengths:** this is where the operator value sits.
- **Weaknesses:** each gateway model needs a port, a bench and validation; Wi-Fi diagnosis depends on driver statistics; an operator's existing QoS features may overlap.

27.6.3 Option C: engineering-services accelerator

- **What:** a package of reusable assets that an engineering-services team brings to a gateway project:
  - the VeriQ core and its backends;
  - the netns lab, scenario library (S1–S13, A1–A32) and evidence ledger, as a validation method for any home-QoS feature;
  - the Stage 1 self-test and bench, as acceptance tests for a new gateway model;
  - the porting path to prplOS and RDK-B.
- **Who pays:** the operator or vendor, through a services engagement.
- **Strengths:** matches how docs/01 describes Tata Elxsi's growth model (accelerators attached to services deals); reuses the evaluation harness, which is valuable even when a customer keeps its own QoS code.
- **Weaknesses:** the decision belongs entirely to Tata Elxsi; the student team cannot validate demand.

Criterion	A: open-source core	B: operator edition	C: services accelerator
Primary customer	Enthusiasts, community	Operator	Operator or gateway vendor, via services
Extra work beyond P0/P1	Public feed, docs, support	USP/TR-181, fleet tooling, per-model ports	Packaging the harness and method for reuse
Main risk	Support load; IP rules	Integration cost per gateway model	Depends on a services pipeline the team does not control
What the team can show now	Packages, importer, self-test	Data-model mapping (design only)	The lab, the ledger and the H1 method

27.7 Cost model

27.7.1 Demo kit and lab bill of materials

The prices below start from docs/06 §2 (approximate, October 2026) and are widened into ranges. All are **approximate, verify before purchase**; prices vary by seller and month. The physical-kit chapter refines the kit extras.

Item	Role	Needed for	Approx. ₹ (verify before purchase)
Raspberry Pi 5 (4 GB) + active cooler + 27 W PSU + 64 GB microSD	VeriQ reference router (D3)	Minimum kit	7,500–10,000
USB 3.0 to 2.5 GbE adapter (RTL8156B class)	Second NIC on the Pi	Minimum kit	1,500–2,500
5-port gigabit switch	LAN side	Minimum kit	1,000–1,800
Team-owned laptop with two NICs	ISP emulator (tbf + netem) and local servers	Minimum kit	0 (owned)
Arcade buttons, jumper wires	Fault buttons	Minimum kit	200–500
Cat6 patch cables, power strip, USB hub	Wiring	Minimum kit	800–1,500
Second Pi 5 + 2.5 GbE adapter (instead of the laptop)	Dedicated ISP emulator	Full kit	9,000–12,500
7-inch HDMI display	Shows the 3D twin at the booth	Full kit	3,000–5,000
ARM64 OpenWrt router with ≥ 512 MB RAM	Retail-router check (D4)	Full kit	8,000–12,000
MT7621-class OpenWrt router (128–256 MB RAM)	Lite-profile floor (D5)	Full kit	2,500–5,000
ESP32 board, rotary encoder, buttons, LED strips, small OLED, enclosure material	Physical kit controls	Full kit	1,500–3,500
Spare microSD (cloned image) and spare PSU	Demo-day spares	Full kit	1,000–1,800

Bundle	Contents	Approx. total ₹ (verify)
Minimum demo kit	First six rows	11,000–16,300
Full kit	Minimum kit + all “Full kit” rows	36,000–56,100
Lab (D1, D2)	Team-owned Ubuntu laptop with a full kernel; OpenWrt VM in QEMU	0 (owned hardware, open-source software)

The minimum kit range agrees with docs/06’s estimate of roughly ₹11–18k. Software costs nothing: every tool in the stack is open source, and no cloud service is needed.

> **Implementation tip.** Buy the minimum kit in Sprint 0 (task 0.10 in the Build plan chapter). Buy the full-kit items only once the P0 exit criteria are met, so money follows evidence.

27.7.2 Development effort

For a student team, the main cost is time. The Build plan chapter estimates, as planning assumptions:

Scope	Person-days (pd, 6 h each)	Hours	Basis
P0 (Sprints 0–4)	about 97	about 580	Sum of the task tables
P1 (Sprint 5+)	about 43	about 260	Sum of the P1 task table
Total to the finale	**about 140**	**about 840**	Planning assumption

To turn effort into money for an organisation, use cost = effort × loaded day rate. The rate is the organisation’s own number; this handbook does not assume one.

Beyond the competition, production work is larger. The ranges below are the team’s **rough planning assumptions** for scoping only; a real estimate must come from the organisation that would do the work.

Work package (beyond P1)	Effort (assumption)	Why
Security review and hardening of daemon, packages and rpcd ACLs	2–4 person-months	Production gateways need an external review
Port and validation per gateway model (bench, Wi-Fi statistics, offload interaction)	1–3 person-months per model	Each SoC, driver and offload engine differs
TR-181 data model and USP agent integration (prplOS path)	2–4 person-months	Fleet management for operators

Work package (beyond P1)	Effort (assumption)	Why
RDK-B component wrapper	3–6 person-months, with a partner	Different component model and data-model plumbing
Fleet tooling: staged rollout, aggregated receipts and shadow reports	2–4 person-months	Needed before more than a pilot
Pilot support and analysis	1–2 person-months per pilot	Field issues, reports

27.7.3 Operator deployment costs

An operator's annualised cost K has four parts. The model keeps them as named variables, because they are unknown until a pilot.

```
K = (C_int + C_val) * M / L  +  C_train  +  C_maint

M        number of gateway models that will run VeriQ
C_int    one-time integration cost per model (port, firmware build, data model)
C_val    one-time validation cost per model (lab runs, bench, soak, certification steps)
L        amortisation period in years
C_train  yearly support-training cost (agents learn to read receipts and bundles)
C_maint  yearly maintenance (security updates, model and catalogue releases, re-verification)
```

Cost drivers worth naming: - **Gateway capability.** Not every gateway can run VeriQ: CAKE needs CPU headroom at the plan rate, and hardware flow offloading bypasses the shaper (docs/11 §12.4). Gigabit plans on weak CPUs need the hfsc/htb backend or one-directional shaping. - **Firmware cycles.** VeriQ ships inside the operator's firmware release train, so integration follows that cadence. - **Privacy and legal review.** Any operator telemetry needs notice and consent and a review under India's Digital Personal Data Protection Act 2023 [R45]; this handbook gives no legal advice.

27.8 Illustrative operator model

27.8.1 The formula

The gross modelled avoided cost of support contacts is:

```
Gross = N * r * 12 * c * p

N   subscribers on VeriQ-enabled gateways
r   monthly share of subscribers contacting support about lag or "slow internet"
c   cost per support contact (INR)
p   share of those contacts that VeriQ prevents
```

27.8.2 Inputs (all illustrative assumptions)

Input	Value	Status
N	1,000,000 subscribers	Assumption (docs/09 §7)
r	1.5% per month	Assumption; replace with operator ticket data
c	₹400 per contact	Assumption; replace with operator cost data
p	15%	Assumption; can only be measured in a controlled field trial

27.8.3 Step-by-step calculation

1. Contacts per month: N × r = 1,000,000 × 0.015 = 15,000.
2. Contacts per year: 15,000 × 12 = 180,000.
3. Yearly cost of those contacts: 180,000 × ₹400 = ₹7,20,00,000 = ₹7.2 crore.
4. Share prevented: ₹7.2 crore × 0.15 = ₹1,08,00,000 ≈ **₹1.08 crore per year (gross, illustrative)**.

Per home, that is ₹1.08 crore ÷ 1,000,000 ≈ **₹10.8 per home per year**. This small number is the most useful output of the model: any per-home cost (new hardware, a paid licence per device, a cloud service) would quickly exceed the value. The only viable form is software on gateways the operator already deploys, which is how VeriQ is designed.

27.8.4 Sensitivity

The two least certain inputs are p (no one has measured it) and c (it varies by operator and channel). The table varies both, with N = 1,000,000 and r = 1.5%/month.

p (share prevented)	c = ₹200	c = ₹400	c = ₹600
5%	₹0.18 crore	₹0.36 crore	₹0.54 crore
10%	₹0.36 crore	₹0.72 crore	₹1.08 crore

p (share prevented)	c = ₹200	c = ₹400	c = ₹600
15%	₹0.54 crore	**₹1.08 crore**	₹1.62 crore
20%	₹0.72 crore	₹1.44 crore	₹2.16 crore
25%	₹0.90 crore	₹1.80 crore	₹2.70 crore

The contact rate r matters just as much. With c = ₹400 and p = 15%:

r (monthly contact rate)	Contacts per year	Gross (illustrative)
0.5%	60,000	₹0.36 crore
1.0%	120,000	₹0.72 crore
1.5%	180,000	₹1.08 crore
2.0%	240,000	₹1.44 crore

Gross value scales linearly with N, so a 250,000-subscriber deployment at the base inputs gives ≈ ₹0.27 crore per year.

27.8.5 From gross to net

Net value subtracts what it costs to get there:

```
Net = Gross - integration - validation/maintenance - training
    = N * r * 12 * c * p  -  K
```

The break-even prevented share is p* = K ÷ (N × r × 12 × c). With the base N, r and c, the denominator is ₹7.2 crore. Two illustrative cases (K values are invented for the illustration, not estimates): - if K were ₹50 lakh per year, p* = 0.5 ÷ 7.2 ≈ 6.9%: VeriQ would need to prevent about 7% of lag contacts to pay for itself; - if K were ₹1 crore per year, p* ≈ 13.9%, close to the base assumption of 15%, so the case would be marginal.

> **Honesty check.** Four limits apply to this model: - p is unknown, and only a controlled trial can estimate it; - many "slow internet" contacts have causes VeriQ cannot fix (outages, Wi-Fi coverage, device problems), so p is bounded well below 100%; - the model counts only support contacts; other effects (fewer technician visits, lower churn, a premium tier) are possible but **unquantified** and are not added; - not every existing gateway can run VeriQ.
>
> The model shows **how** value would be quantified in an operator pilot, not **that** it exists.

> **For judges.** "We show the formula and let you change the inputs. At our illustrative inputs it is about ₹1.08 crore a year gross, or about ₹10.8 per home per year, which tells us VeriQ only makes sense as software on gateways the operator already owns. None of these inputs is measured."

27.8.6 Assumption register

Every input of the business case is listed here with how it would be replaced by real data. When a judge asks "where does this number come from?", this table is the answer.

Assumption	Current value	Status	Replaced by	Supplied by
Subscribers on VeriQ-enabled gateways (N)	1,000,000	Illustrative (docs/09)	Gateway models that pass the bench, times their installed base	Operator
Monthly lag contact rate (r)	1.5%	Illustrative (docs/09)	Ticket data tagged "slow/lag", 12 months	Operator
Cost per contact (c)	₹400	Illustrative (docs/09)	Blended cost across call centre, chat and visits	Operator
Share prevented (p)	15%	Illustrative (docs/09)	Controlled rollout with a matched control group	Joint trial
Annualised cost (K)	Not estimated	Unknown	Integration, validation, training and maintenance quotes	Operator and integrator
Component prices	Ranges above	Approximate, verify before purchase	Quotes on the purchase date	Team
Development effort	about 140 pd to the finale	Planning assumption	Tracked actuals per sprint	Team

27.9 Go-to-market path

The path goes from the lab to operators in stages. Each stage has an entry gate, so evidence leads and scale follows.

Stage	What happens	Entry gate	Evidence produced
0. Lab (now to the finale)	netns lab, OpenWrt VM, Pi; H0–H4 tiers	—	Evidence ledger; H1 results with CIs, including ties and losses
1. OpenWrt community (P2)	Public signed feed; installation by package manager only; importer for QoSmate and SQM users; feedback via the forum and issue tracker	Licence decided (O5); IP rules checked; security review of the packages	Install success across devices; bug reports; footprint on real devices
2. Partner port (P2)	prplOS with a TR-181/USP data model; RDK-B component wrapper with a partner	Stable feed; one carrier-grade platform partner	Data-model mapping; bench and soak on partner hardware
3. Operator pilot (P2)	20–50 opt-in homes, SHADOW first (design below)	Partner build passes Stage 1 and H4 soaks	Shadow report; safety KPIs; household feedback
4. Staged rollout	Cohorts grow step by step, every new cohort starts in SHADOW	Pilot met its stop/rollback criteria without triggering them	Contacts per 1000 homes vs control (a larger trial can measure this)

The path never skips SHADOW. Static CAKE is active from the first minute in every stage, so households get the SQM benefit while the adaptive layer earns trust (C10).

27.10 Pilot design

The v4.1 specification sketches a hypothesis-driven pilot (docs/09 §7). This section fills in the details.

27.10.1 Purpose

The pilot answers three questions: 1. Is VeriQ **safe** on real lines and real gateways (no crash loops, CPU within budget, no harm compared with static SQM)? 2. Do its **would-be decisions** look right on real traffic (shadow disagreements, judged where ground truth is available)? 3. Do households **accept** it (opt-outs, household-page use, feedback)?

It does **not** try to prove a reduction in support contacts. At the illustrative 1.5% monthly contact rate, 50 homes generate fewer than one lag contact per month, which no 20–50 home study can resolve.

27.10.2 Design

Element	Design
Homes	20–50 opt-in households with informed consent; a mix of FTTH/PON, cable and 4G/5G FWA where the operator offers them
Hardware	The operator's gateway running VeriQ, or a VeriQ router behind the operator's ONT or modem
Control group	Matched homes on the same access technologies without VeriQ, for contact and complaint comparison
Phase 1 (2 weeks)	Static CAKE active; controller in SHADOW; daily shadow reports
Gate	Shadow report without anomalies (crash, CPU over budget, > 5 unsafe disagreements per hour)
Phase 2 (4 weeks, assumption)	Opt-in LIVE for homes that passed the gate; receipts collected locally
Data	Local only by default; aggregate export only with explicit opt-in; support bundles hash LAN addresses and MACs
Analysis	Pre-registered: safety KPIs, disagreement rates, receipt outcome mix, opt-outs; contacts reported descriptively only

27.10.3 Stop and rollback criteria

Any of these stops LIVE operation for the affected homes, or for the whole pilot if it repeats: - a crash loop in any home; - veriqd CPU above its profile budget, persistently; - more complaints than the control group; - any sign that VeriQ made a home worse than static SQM in a comparable condition (the dominance invariant).

Rollback has three levels, all tested in the lab first: 1. `veriqctl state set safe` (static CAKE, rules and leases kept); 2. the package manager installs the previous version (state files stay compatible for one major version); 3. uninstall, which removes VeriQ's qdiscs, IFB and nft table and leaves the user configuration.

27.11 Scaling

- **Per home:** the control loop is local, on the gateway, with no cloud dependency. The datapath is in the kernel. Probing stays within 10 probes per second in total and drops to 2 per second when idle, which keeps reflector load small even across many homes.
- **Fleet:** configuration and status through TR-181/USP [R27]; staged cohorts; every controller or model update re-enters SHADOW for 24 h (docs/11 §12.8).

- **Updates:** only through signed packages; the model and the app catalogue are separate signed packages, so they can be updated without a new daemon.
- **Support:** the support bundle and the explain API give agents a shared, privacy-preserving view of what happened in the home.
- **Known limits:** CAKE runs on one CPU per qdisc in the kernels targeted, so multi-gigabit plans on weak CPUs need another backend or one-directional shaping; Wi-Fi diagnosis depends on driver statistics; reflector etiquette needs a reflector strategy agreed with the operator (an operator-hosted reflector or a cooperating UDPST server [R22] would also enable evidence level L3).

27.12 IP and licensing

- **Competition IP.** docs/01 records that submitted ideas become Tata Elxsi's property. Check the current rules on the portal before publishing anything, keep the repository private, share access with evaluators, and do not submit anything the team plans to patent.
- **VeriQ licence.** Open decision O5: Apache-2.0 proposed for the OpenWrt feed; Tata Elxsi decides.
- **Clean-room rule.** QoSmate (GPL-3.0) and cake-autorate (copyleft; check its LICENSE) are used for **ideas only**. No code is copied, and implementers do not open GPL source files while writing the matching VeriQ module. The QoSmate importer is written from the README's documented configuration format.
- **Third-party code.** Go modules are restricted to MIT/BSD/Apache licences. GPL lab tools are run as external programs and listed in `LICENSES.md`.
- **Content.** App catalogue entries cite vendor sources and carry a verification date; app names are used only to identify the apps. Demo video content is the team's own or openly licensed, with licences documented.
- **Data.** Public datasets used for training (for example, CESNET DataZoo [R34]) are used under their stated terms, and those terms are recorded in the model release notes.

27.13 Presenting the business case in two minutes

A suggested script for the business part of a pitch, with every number labelled as it should be spoken:

1. **The pain (20 s).** "One download can make a family's video call freeze. Our Day-0 sandbox run measured 266 ms of added upload delay behind a FIFO; that is one preliminary run, and we say so."
2. **Who pays (20 s).** "Households feel it; operators pay for it in support contacts that a speed test cannot explain, because the speed test shows full speed."
3. **The model (30 s).** Show the formula N × r × 12 × c × p. "At illustrative inputs it is about ₹1.08 crore a year gross for a million subscribers, about ₹10.8 per home. Change any input you like; the point is the shape, not the number."
4. **What it implies (20 s).** "At ₹10.8 per home, only software on existing gateways can work. That is what VeriQ is: OpenWrt packages today, a path to prplOS and RDK-B with a partner."
5. **The proof plan (30 s).** "We would not ask an operator to trust a model. A 20–50 home pilot starts in shadow mode with static CAKE active, has written stop criteria, and measures safety and acceptance. Support-contact effects need a larger trial, and we say that too."

27.14 Summary

Question	Answer
Who benefits first?	Households, from static CAKE on day one and diagnosis-gated adaptation after SHADOW
Who decides to deploy?	The operator, through its gateway firmware programme
What does the demo cost?	Approximately ₹11,000–16,300 for the minimum kit; ₹36,000–56,100 for the full kit (verify before purchase)
What is the illustrative operator value?	≈ ₹1.08 crore per year gross at the docs/09 inputs, ≈ ₹10.8 per home per year; net depends on integration, validation, maintenance and training
What would prove it?	A pilot for safety and acceptance first; a larger controlled rollout for support-contact effects
What fits Tata Elxsi?	Option C, a services accelerator built from the core and the evaluation harness; a thematic fit only, with no endorsement implied

CHAPTER 28

Risks

This chapter is VeriQ's risk register. It lists 36 risks in six categories (technical, schedule, team, evaluation validity, legal and licence, demo day), each with a likelihood, an impact, a mitigation, an early-warning trigger and an owner. It ends with a deep dive on the five highest-scoring risks and the routine that keeps the register alive.

Owners use the roles from the Build plan chapter: R1 Datapath & OpenWrt, R2 Controller, R3 Evaluation, R4 UX, ML & demo, plus the rotating Lead and Claims steward hats.

28.1 Scoring

Score	Likelihood (L)	Impact (I)
1 (low)	Unlikely before the finale	Local rework; no deliverable affected
2 (medium)	Plausible; has happened in similar projects	A P0 deliverable slips or loses part of its scope
3 (high)	Expected unless acted on	A headline claim, a submission or the live demo is at stake

Priority = L × I. A score of 6 or 9 is **red** (reviewed weekly, has a named contingency), 3 or 4 is **amber** (reviewed at each sprint planning), 1 or 2 is **green** (reviewed monthly). The scores are the team's judgement on 12 Oct 2026 (assumption) and must be re-scored as evidence arrives.

28.2 Technical risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
T1	`act_ctinfo cpmark` is unavailable or behaves differently on a target kernel or iproute2, so download packets do not carry the class	2 / 3 (6)	Verify syntax against tc-ctinfo(8) [R7] in Sprint 1; O3 compat mode (DSCP restore with `ctinfo dscp`, washed after the tin decision); self-test in both directions	Tin self-test fails for download on any target	R1
T2	The fwmark value → CAKE tin mapping differs on a kernel	1 / 3 (3)	The self-test is authoritative; the mapping table is configurable (O2)	Self-test counts land in the wrong tin	R1
T3	Fewer than 2 timestamp-capable reflectors qualify, so one-way delays are unavailable	2 / 2 (4)	RTT-only mode with load attribution (O4); evidence capped at L1 when both directions are loaded	Qualification report shows < 2 timestamp reflectors	R2
T4	The C1 one-step cut fires falsely in control-futile runs	2 / 3 (6)	Diagnosis gate at L2; checker rule `CUT_BELOW_EVIDENCE`; the "> 3 L2 cuts without improvement → MIXED" rule; docs/12 revert condition	H1 pilot shows false L2 cuts above the comparator	R2
T5	CAKE cannot reach the target rate on the Pi or a retail router CPU	2 / 2 (4)	Bench at setup; hfsc/htb backends (P1); one-directional shaping; report CAKE's cost separately from veriqd's	Bench reaches < 95% of the target rate or shaping CPU > 85%	R1
T6	Flow offloading or hardware NAT on a real router bypasses the shaper	2 / 2 (4)	Conflict detection blocks LIVE; one-click disable with consent; health check	Health check reports offloading enabled	R1
T7	Forking `tc` and `nft` costs too much on a low-end MIPS device	1 / 2 (2)	Budget of ≤ 2 changes per second; measure on D5; batch commands	Tick overruns on D5 in the soak	R1

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
T8	Reflector rate limiting or ICMP deprioritisation looks like congestion	2 / 2 (4)	Median with quorum, MAD outlier rule, rotation; PROBE_FAULT and MIXED labels; A5 and A32 scenarios	Outlier exclusions above a few per hour in soaks	R2
T9	The ML classifier lacks calibration data (≥ 200 samples per class per checkpoint)	2 / 1 (2)	Disable that checkpoint; classifier is P1; rules, leases and heuristics carry P0	Calibration report shows a class below the threshold	R4
T10	Wi-Fi station statistics are missing or inconsistent on a driver	2 / 1 (2)	WIFI evidence becomes MISSING; diagnosis falls back to other groups; documented limitation	ubus/iwinfo queries return no station data	R2
T11	The reconciler fights another tool and causes flapping	1 / 2 (2)	Foreign change → DEGRADED, never fight; A30 test	More than one ownership change per hour in a test	R1

28.3 Schedule risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
S1	The Round 1 deadline or format differs from the listings (docs/01 records conflicting dates)	2 / 3 (6)	Confirm on the official portal on 12 Oct; internal deadline 22 Oct	Portal shows a different date or format	Lead
S2	Sprint 2 controller work overruns (R2 carries about 12 pd against about 6 pd of personal capacity)	3 / 3 (9)	Pairing; R3 writes diagnosis traces; R1 owns safety; LuCI first on the cut ladder; O7 Python fallback	Checker or FSM tests not green by 18 Nov	Lead + R2
S3	H1 machine time is insufficient (≈ 42 h at n = 10, ≈ 83 h at n = 20, assumption)	2 / 3 (6)	Evaluation freeze by 2 Dec; unattended runner with clean restart; overnight runs; cut ladder step 4	Pilot shows n > 20 or the runner needs manual restarts	R3
S4	Examinations and coursework peak in November–December	3 / 2 (6)	Capacity check at each sprint planning; exam weeks marked in the tracker; front-load Sprint 1	Any member's availability falls below half for a week	Lead
S5	Pi or adapter delivery is delayed	1 / 2 (2)	Order in Sprint 0; VM (D2) fallback for every package test	Order not shipped by 5 Nov	R1
S6	The finale date is announced at short notice	2 / 2 (4)	P1 plan in weeks relative to the finale; finale-critical items (kit, rehearsals) first	Date announced < 4 weeks ahead	Lead
S7	Integration surprises when packaging for OpenWrt (fw4 interplay, procd behaviour)	2 / 2 (4)	Start the VM in Sprint 1; package tests in CI from Sprint 3	First `opkg install` on the VM fails after 3 Dec	R1

28.4 Team risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
P1	A member becomes unavailable for weeks (bus factor of one per role)	2 / 3 (6)	Code review by a second member; module READMEs; the two-person role mapping in the Build plan	Unplanned absence longer than one week	Lead

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
P2	Go ramp-up is slower than planned	2 / 2 (4)	Ramp-up check in Sprint 0; O7 Python controller in the lab for P0, same interfaces and golden tests	O7 check on 24 Oct fails	Lead
P3	Scope creep: new features before P0 exits	2 / 2 (4)	Decision-log rule; no architecture change after the Sprint 2 exit unless a measurement forces it	A new feature appears in a sprint without a decision-log entry	Lead
P4	Knowledge silo between controller and evaluation (the evaluator must not depend on the controller's internals)	1 / 2 (2)	Evaluator computes KPIs from the ledger only; R3 reviews controller logs, R2 reviews evaluator metrics	Evaluator imports veriqd code	R3

28.5 Evaluation-validity risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
E1	Comparators are mis-tuned, so comparisons are unfair (QoSmate, cake-autorate, SQM)	2 / 3 (6)	Tune each per its own documentation; pin versions; archive configs; controller-only and whole-system comparisons; invite review of configs	A comparator fails Stage 1 or its config deviates from its docs without a recorded reason	R3
E2	Results show parity or a loss for VeriQ	2 / 3 (6)	Parity in static conditions is expected and pre-stated; retention rule (ties and losses reported); the story moves to explainability, safety and UX	H1 pilot shows no difference in the primary outcome	Lead + R3
E3	Emulation does not represent real DOCSIS, PON or FWA schedulers	3 / 2 (6)	Label all emulation; physical kit and soak on real hardware; known-limitations section	A reviewer asks for real-line evidence that does not exist	R3
E4	The controller is not blind to the fault schedule	1 / 3 (3)	Random onset in [60, 120] s; ground truth in separate fields; the controller never sees scenario names	Any code path in veriqd reads a scenario file	R3
E5	Harness artefacts: generator CPU saturation, media not crossing the bottleneck, offloads left on	2 / 2 (4)	Per-run checks (generator CPU < 70%, bottleneck counters, offloads off, fingerprint regenerated)	A run fails a precondition check	R3
E6	Underpowered statistics	2 / 2 (4)	Pilot n = 5 sets n; bootstrap CIs; no p99.9 claims; Holm correction	CI half-width above the target after the pilot	R3
E7	Day-0 numbers are quoted without their labels (no base RTT, one run, IPv4 only)	2 / 2 (4)	Claims check; exact label wording in every document; replace with replicated results when available	Claims check finds an unlabelled Day-0 number	Claims steward

28.6 Legal and licence risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
L1	GPL contamination: QoSmate (GPL-3.0) or cake-autorate code enters VeriQ	1 / 3 (3)	Clean-room rule (ideas only, no GPL file open while writing the matching module); importer from the README format; review of every module's provenance	A diff resembles GPL source, or a member reports having copied	R1
L2	Competition IP: submitted ideas become Tata Elxsi's property (docs/01)	2 / 2 (4)	Read the rules on the portal; keep the repository private; no patent plans; licence decided with Tata Elxsi (O5)	Rules text changes or is unclear	Lead
L3	Third-party module licence is not permissive	1 / 2 (2)	MIT/BSD/Apache only; licence report in CI	CI licence report flags a module	R1
L4	Privacy: pilot or demo data includes personal data (DPDP Act 2023 [R45])	1 / 3 (3)	Local-only processing; hashed identifiers in support bundles; no real household data in the competition; counsel review before any pilot	Any plan to collect data outside the lab	Lead
L5	Demo media or catalogue content used outside its licence or trademark terms	1 / 2 (2)	Own or openly licensed media; licences documented; app names used only to identify apps	Media without a recorded licence in the repository	R4
L6	Secrets committed to the repository (forbidden by the case brief)	1 / 2 (2)	Secret scanning in CI; example secrets clearly fictitious; household password never in config examples	Scanner alert	R1

28.7 Demo-day risks

ID	Risk	L / I (score)	Mitigation	Trigger	Owner
D1	Venue network or Wi-Fi interferes with the demo	2 / 3 (6)	Run fully offline: local Jitsi, local DASH content, wired links; never depend on venue Wi-Fi	Venue gives no power or table space for wired setup	R4
D2	Hardware fails on the day (SD card, PSU, adapter)	2 / 3 (6)	Spare cloned microSD and PSU; second Pi; backup video recorded from real runs	Any hardware fault in the last rehearsal	R1
D3	The live demo shows no visible difference (for example, a static condition where parity is expected)	2 / 2 (4)	Script uses conditions where differences are expected (genuine drop, remote delay, kill → SAFE); explain parity honestly	Rehearsal shows no difference in a chosen scene	R4
D4	A judge asks for a fault not on the menu	2 / 1 (2)	Explain that the response comes from telemetry, not a scripted label; unknown cases fall to hold or bounded fallback	—	Lead
D5	Time overrun in the presentation	2 / 2 (4)	Timed rehearsals; a 3-minute cut of the demo; one owner per section	Rehearsal over time by > 10%	Lead
D6	Power or adapter problems at the venue	1 / 2 (2)	Power strip with surge protection, universal adapters, pre-checked cable list	—	R1

28.8 Heat map

```
               Impact 1       Impact 2                Impact 3
Likelihood 3   -              S4, E3                  S2
Likelihood 2   T9, T10, D4    T3, T5, T6, T8, S6, S7, T1, T4, S1, S3, P1,
                              P2, P3, E5, E6, E7,     E1, E2, D1, D2
                              L2, D3, D5
Likelihood 1   -              T7, T11, S5, P4, L3,    T2, E4, L1, L4
                              L5, L6, D6
```

Red risks (score ≥ 6): S2, T1, T4, S1, S3, P1, E1, E2, D1, D2, S4, E3.

28.9 Top-5 risk deep dive

The five below were chosen from the red risks by asking which ones, if they happened, would most damage the central claim or the submission.

28.9.1 1. S2: the controller sprint overruns (score 9)

- **Why it is top:** the diagnosis, controller and checker sit on the critical path. Any slip moves the evaluation freeze, and H1 machine time cannot be compressed.
- **Early warnings:** checker refusal-code tests not all green by Wed 18 Nov; the FSM property test not running by Fri 20 Nov.
- **Mitigation now:** the build plan moves all safety work to R1 and diagnosis traces to R3; LuCI pages are the first item on the cut ladder; R2's time is protected from demo work until 5 Dec.
- **Contingency:** apply cut-ladder step 5 (O7). The controller is built in Python inside the lab with the same interfaces and golden tests, evaluated in H1, and ported to Go in P1. The research question survives; product readiness of the controller is deferred, and the submission says so.
- **Residual risk:** medium. The Python path costs time too, but less, because the lab already runs Python.

28.9.2 2. E1 and E2: unfair comparisons, or parity (score 6 each)

- **Why it is top:** the whole contribution is comparative. A mis-tuned QoSmate or cake-autorate would make any win worthless, and a judge who knows these tools will check.
- **Early warnings:** a comparator config that differs from its documented defaults without a recorded reason; an H1 pilot with overlapping CIs on the primary outcome.
- **Mitigation now:** pin versions (QoSmate v0.5.48 @ c507843; cake-autorate pinned v3.x); tune each tool by its own documentation; archive every config; run both controller-only and whole-system comparisons; state in advance that static conditions should show parity.
- **Contingency:** if the primary outcome shows parity, report it, keep the ablations, and present VeriQ's value as the safety, explainability and household layers, with the controller result stated plainly. Do not move the goalposts after the freeze.
- **Residual risk:** low for fairness, medium for parity. Parity is a legitimate result.

28.9.3 3. T1: download mark plumbing does not work on a target (score 6)

- **Why it is top:** without class marks on download packets, half of the datapath's classification fails, and every per-tin result in the download direction is in question.
- **Early warnings:** `tc` rejects the ctinfo `cpmark` syntax; the self-test shows download packets in the Best Effort tin regardless of class.
- **Mitigation now:** verify the syntax in Sprint 1 on the lab kernel and the OpenWrt VM; the tin self-test checks both directions on every target; packet captures on `ifb-veriq`.
- **Contingency:** O3 compat mode: restore class via DSCP with `ctinfo dscp`, as QoSmate does (read), with DSCP washed after the tin decision; the health page shows "compat mode".
- **Residual risk:** low, because a known fallback exists and is testable.

28.9.4 4. S3: not enough lab time for H1 (score 6)

- **Why it is top:** the H1 runs are the evidence. Too few runs give wide intervals; rushed runs risk artefacts (E5).
- **Early warnings:** the runner needs manual restarts; a single run takes much longer than about 5 minutes; the pilot suggests n > 20.
- **Mitigation now:** evaluation freeze by Wed 2 Dec; automated reset between runs; counterbalanced overnight batches; per-run precondition checks so bad runs are flagged, not silently kept.
- **Contingency:** cut-ladder step 4 (reduce H1 to S2, S6, S7, A32 and S1). Report the remaining scenarios as pilot data with their n.
- **Residual risk:** medium; it depends on runner reliability, which is tested in Sprint 2 with H0.

28.9.5 5. D2 and D1: hardware failure or venue interference on demo day (score 6 each)

- **Why it is top:** the live demo is where judges form their impression, and the kit is the most fragile part of the project.
- **Early warnings:** any fault in rehearsal; the venue cannot provide wired power and table space.

- **Mitigation now:** fully offline kit, wired links only, spare cloned microSD and PSU, a second Pi, and a backup video recorded from real runs with honesty labels.
- **Contingency:** switch to the backup video and the 3D twin replay of a real run; keep the kill → SAFE demonstration, which needs only the Pi and a laptop.
- **Residual risk:** low.

28.10 Keeping the register alive

- **Weekly (red risks):** the Lead reviews each red risk's trigger at the Saturday demo.
- **Each sprint planning (all risks):** re-score L and I; add new risks; close risks whose window has passed (for example S1 after 24 Oct).
- **On a trigger:** the owner opens a decision-log entry stating the trigger, the action taken (mitigation or contingency) and whether the cut ladder moves.
- **Before each submission:** the Claims steward checks E7 and L1–L6 explicitly.

> **For judges.** "Our biggest risk was our own schedule, not the technology: the controller sprint was overloaded on paper, so we decided in advance what to cut and kept the safety and evidence parts off that list. Our biggest evaluation risk was an unfair comparison, so we pinned versions, tuned each tool by its own documentation and committed to reporting parity as parity."

Part VI · Demonstration and submission

Showing it: demo, video, 3D and 2D models, kit, submission

CHAPTER 29

Live Demo and Video

The judges will spend a few minutes with VeriQ. In that time they must see three things: a real problem, a controller that responds to evidence rather than to a script, and a system that fails safe. This chapter turns the judge-controlled demonstration of docs/09 §7 and the playbook in docs/06 into a minute-by-minute run sheet. It adds backup plans for every step, a 3-minute video script with honesty labels, and recording tips. The hardware is described in Chapter 32, the 3D twin in Chapter 30, and the charts in Chapter 31.

> **Note.** docs/06 predates v5. Where it uses older terms (a `CANARY` state, a "conf 0.93" diagnosis, p-values in receipts, "guaranteed rates" for intents), this chapter follows docs/11: operating states INIT/SHADOW/LIVE/DEGRADED/HOLD_LINKDOWN/SAFE/DISABLED, diagnosis labels with evidence levels L0–L3, receipts with IMPROVED/NOT_IMPROVED/INCONCLUSIVE outcomes, and leases that give CAKE tin preference, not a reserved rate.

29.1 Demo principles

Five rules govern every minute of the demo. They come from the credibility rules in docs/06 §10 and the demonstration plan in docs/09 §7.

1. **The judge drives.** The judge turns the capacity knob, presses the fault buttons and chooses the order. VeriQ reacts to its own telemetry, never to a hidden scenario label. The ESP32 panel talks only to the ISP emulator (and, for the kill button, to a restricted SSH command); it never tells `veriqd` what fault was injected.
2. **Fully offline.** The call server, the DASH content, the file server and the reflectors all live on the ISP emulator. Venue Wi-Fi and internet access play no part.
3. **Every number on screen is labelled.** Live counters from the kit are *measured (emulated link)*. Capacity estimates and diagnoses are *estimated*. House graphics and particles are *illustrative*. Design limits such as "time-to-safe ≤ 12 s" are *targets*.
4. **One idea per step.** Each step demonstrates one claim from the design and names the scenario that tests it (S1–S13, A1–A32).
5. **A backup exists for every step.** If any step fails, the presenter switches to the backup within 15 seconds and says so plainly.

> **For judges.** The physical kit is an emulated ISP bottleneck (tbf with an oversized FIFO, plus netem delay) in front of a real OpenWrt router running VeriQ. The call, the downloads, the shaping and the controller are real. The access link is emulated, and the kit says so on its panel.

29.2 Stage layout

The table holds four screens and the kit panel. Arrange them so that a judge standing in front of the panel can see all four without turning.

Position	Device	Shows	Label on screen
Left	Call laptop	Jitsi Meet call with the remote test-pattern participant; overlay with WebRTC (Web Real-Time Communication) `freezeCount`, `totalFreezesDuration` and round-trip time from `getStats` [R35]	Measured (emulated link)
Centre-left	Hub laptop	LuCI Overview and Autorate pages: state badge, S per direction, μ±σ, reflector table, last five decisions as sentences	Measured / estimated
Centre-right	7-inch display or monitor	3D twin in live mode (Chapter 30)	Live; tags per object
Right	Hub laptop second window	Scoreboard: true emulator capacity vs VeriQ S, time since last fault, checker verdicts	Truth (emulator) vs estimated
Front	Kit panel	Capacity knob, WAN DROP, REMOTE DELAY, LYING REFLECTOR, KILL (under a flip cover), RESET; LED bars per class; OLED with state and S	Measured

The scoreboard is the honesty anchor. It shows the emulator's true capacity next to VeriQ's configured rate S, so the judge can see how closely, and how quickly, VeriQ tracks a change it was never told about.

29.3 Pre-flight checklist

Run this list from the printed card, not from memory. The person who runs it signs the card.

T – 60 min - Power on the emulator, then the router, then the hub laptop. Confirm the router is in LIVE (the kit profile uses start mode "Adapt now"; say so if asked, because the product default is 24 h of SHADOW). - Run `veriqctl health` and `veriqctl selftest` on the router. Both must pass, including the tin self-test in both directions. - Run the emulator's `emu-agent selfcheck` (Chapter 32): every reflector answers echo and timestamp, and the servers respond. - Start the Jitsi call between the call laptop and the remote participant. Confirm video and the stats overlay. - Press RESET on the panel. The emulator returns to 100/20 Mbit/s, no faults.

T – 15 min - Start the bulk client (four TCP flows to the file server) and check that the call stays smooth. The baseline is ready when the LuCI Overview shows grade A or B for both directions. - Open the recorded comparator clip (S6 for a delay-only autorate arm) and the backup video in paused players. - Start screen recording on the hub laptop. The recording is the evidence that the demo was live.

T – 2 min - Close notifications on all laptops. Set screen sleep to "never". - Check that the KILL flip cover is closed and the ARM key switch is on.

29.4 The run sheet (about 7 minutes)

The timings below assume a 7-minute slot. For a 5-minute slot, skip Step 6 and shorten the frame. For a 10-minute slot, let the judge choose two extra faults from the fault menu.

Time	Step	Judge action	What the screens show	What the presenter says
0:00–0:40	Frame	—	Kit panel idle, call running, bulk running	"This is a real OpenWrt router running VeriQ, in front of an emulated ISP link with an oversized buffer. Our Day-0 sandbox run measured 266 ms of added upload delay behind a FIFO; that was one preliminary run. Everything you will press, VeriQ has to work out from its own measurements."
0:40–1:40	1. Call plus bulk	Optional: judge flips VeriQ off and on in LuCI	Off: call RTT climbs, freezes appear. On: static CAKE returns, RTT falls	"This is bufferbloat. With VeriQ off, the queue sits in the emulated modem. With it on, the queue moves into the router, where CAKE manages it. This part is plain SQM; VeriQ's floor is never worse than this." (S1)
1:40–2:50	2. Genuine drop	Judge presses WAN DROP (100 → 20 Mbit/s) or turns the knob	Scoreboard: true capacity falls. Within a few seconds S drops in one step. Explain sentence: "FAR + THR, evidence L2, Checker: ACCEPT". Receipt after 10 s	"VeriQ saw two independent signals: far-reflector delay rising and throughput stuck below the shaper while downloads wanted more. That is evidence level L2, so it cut once, to 90% of measured throughput. The receipt 10 seconds later says whether delay improved." (S2, A1)
2:50–3:50	3. Misleading remote delay	Judge presses REMOTE DELAY (+80 ms beyond the ISP first hop)	Far delay rises, near hop flat, no throughput shortfall. Diagnosis REMOTE_LIKELY @ L2. S unchanged. Beside it, the recorded comparator run of the same event	"Delay rose, but the near hop is flat and our link is not full. Cutting cannot help, so VeriQ holds and tells the household why. On the right is a recorded lab run of a delay-only autorate on the same event: watch what its rate does." (S6, A4)
3:50–4:40	4. A rejected action	Judge runs a prepared foreign `tc qdisc change` on the router from the hub laptop	Reconciler detects a foreign change: state DEGRADED, banner in LuCI. The decision log shows any action proposed against the old version refused (`VERSION_MISMATCH` , or `STATE_FORBIDS` once DEGRADED)	"Something other than VeriQ changed the queue. VeriQ does not fight it. It freezes, raises an alert, and its checker refuses any action built on a version it no longer owns. The owner decides: take ownership, or leave it." (A30)

Time	Step	Judge action	What the screens show	What the presenter says
4:40–5:50	5. Kill the controller	Judge lifts the flip cover and holds KILL for 1 s	OLED: heartbeat age counting up; at ≤ 12 s (target) state SAFE; the call keeps running; procd restarts `veriqd`, which adopts the kernel state	"We just killed the controller with `kill -9`. The kernel configuration stays in place, so the internet never stopped. The watchdog moved the shaper to the SAFE rate, and the measured time-to-safe is on the scoreboard." (S8, A15)
5:50–6:30	6. Intent (optional)	Judge taps "Protect my call, 30 min" on the household page	Lease receipt starts; Connections page shows provenance "lease" for the call flows	"A household member asked for protection. It is authenticated, it expires, at most two leases can be active, and a leased bulk download would still be returned to best effort by the envelope." (S5)
6:30–7:00	Close	—	The evidence ledger folder and the claims-check badge	"Everything you saw is in the decision log and will regenerate from raw data. What we measured today is labelled; everything else is a target until the head-to-head runs report."

Implementation tip. Prepare the foreign change as a one-line script on the hub laptop, e.g. `ssh root@192.168.1.1 tc qdisc change dev eth0 root cake bandwidth 50mbit`, and print it on a card. A judge should not have to type an interface name.

29.4.1 Notes on each step

Step 1 is optional because toggling VeriQ off and on costs about 30 seconds. If the slot is short, show the recorded FIFO-vs-VeriQ clip from the video instead and say it is a recording. When VeriQ is re-enabled it goes through INIT and installs static CAKE at the start rate at once; this is the C10 product rule at work.

Step 2 must not be over-explained. Say the two evidence sources, the evidence level, the one-step cut and the receipt. If the judge asks "why 90%?", answer: "$\hat{G}$ is the median measured gateway throughput over the last second; we leave a 10% margin so the queue stays in our shaper. The checker refuses anything below 85% of $\hat{G}$."

Step 3 is the most important step, because it is the primary outcome of the evaluation: inappropriate interventions in control-futile scenarios. The comparator clip must be a real recorded run from the evidence ledger, with the arm name, version, run ID and date on screen. If the head-to-head data shows the comparator holding as well, show that and say so; it is a parity result, not a failure.

Honesty check. One recorded comparator run is an illustration, not a statistical result. Say: "This is one run; the full comparison, with confidence intervals across all runs, is in our report." The statement that a delay-only autorate cuts on remote delay is a hypothesis derived from its source code (read), until the H1 runs measure it.

Step 4 shows the checker and reconciler without any special demo code. The refusal code shown must be the one the checker actually produced; if the controller had no action pending, show the DEGRADED transition and the reconciler's log event instead, and say that no action was refused because none was proposed. A second, scripted option is a replay: `veriqctl replay` of a recorded log in which the tc sample is delayed by more than two ticks, so the checker returns `STALE_OBS`. Label it "replayed with an injected delay". If time allows, the presenter can open the explain view and show the full decision-log line.

Step 5 prints the measured time-to-safe on the scoreboard. If the measured value exceeds 12 s, show it anyway and say it missed the target.

Step 6 works best when the judge holds the phone. If no phone is available, open the household page in a phone-sized browser window on the hub laptop.

29.5 The judge's fault menu

If a judge wants to choose, hand them the printed menu below. Each entry names the expected response; the expectation is a design target, and the screen shows what actually happened.

Panel input	What the emulator does	Expected VeriQ response (target)	Tests
Capacity knob	Sets the emulator line rate (tbf) between 5 and 100 Mbit/s	Cut on a drop at L2; PROBE_UP ×1.04 per second on recovery	A1, A2, A3
WAN DROP	Steps the line rate 100 → 20 Mbit/s	One-step cut to max(S_min, 0.90·$\hat{G}$)	S2, A1

Panel input	What the emulator does	Expected VeriQ response (target)	Tests
REMOTE DELAY	Adds 80 ms netem delay beyond the ISP first hop	REMOTE_LIKELY; hold S	S6, A4
LYING REFLECTOR	Delays and jitters ICMP replies from one reflector	Outlier excluded; no rate change attributable to it	S7, A5
KILL	`kill -9` of `veriqd` on the router (restricted SSH command)	SAFE ≤ 12 s; no outage; adoption on restart	S8, A15
RESET	Removes all faults, restores 100/20 Mbit/s	PROBE_UP back toward S_base; receipts close	A3
Foreign tc change (card)	Changes the root qdisc behind VeriQ's back	DEGRADED; proposed actions refused (`VERSION_MISMATCH` or `STATE_FORBIDS`)	A30

29.6 Backup plans

Demos fail. The team rehearses each backup at least twice, so that switching takes less than 15 seconds.

Failure	Detect	Backup	What to say
Jitsi call drops or will not start	No video, overlay frozen	Switch the overlay to the synthetic game flow (64 packets/s UDP) and its live RTT chart	"The call client failed; this synthetic flow measures the same queue."
Twin display freezes	No movement for 5 s	Reload the page; if that fails, open the replay of the rehearsal run	"This is the replay of this morning's run, from the same log format."
A step's effect does not appear	Scoreboard shows no change after 10 s	Press RESET, wait 20 s, retry once; then explain what should have happened and show the recorded clip	"It did not behave as expected; here is the decision log for what it saw."
ESP32 panel dead	No OLED, no LEDs	Trigger the same faults from the hub laptop's `kitctl` commands (Chapter 32)	"The panel failed; the fault injection is the same command."
Router or emulator dead	No call, no LuCI	Run the same scenario in the netns lab on the hub laptop (deployment view D1)	"This is the same daemon in our namespace lab."
Everything dead	—	Play the 3-minute video, then the twin replay	"Here is the recorded run; all files are in the evidence ledger."

The worst outcome is pretending. If a step fails, say so, and show the decision log. Judges trust a team that can explain a failure from its own logs.

29.7 What the presenter must explain personally

docs/09 §7 names five topics that a presenter must explain without notes. Each team member rehearses all five, because the judge chooses who answers.

1. **Queue placement:** why shaping just below the bottleneck moves the queue into the router.
2. **Classification uncertainty:** conformal sets, abstention, and why abstention never grants preference.
3. **False vs missed interventions:** a cut that cannot help costs bandwidth; a missing cut costs delay. The primary outcome measures the first, and the co-primary outcome protects against the second.
4. **The experiment design:** arms, scenarios, the run as the unit of analysis, bootstrap confidence intervals and Holm correction.
5. **Failure recovery:** watchdog, SAFE, procd respawn, reconciler adoption.

29.8 The 3-minute video

The video serves three uses: the Round 2 submission, the backup for a failed live demo, and a shareable summary. It must stand alone without narration from the team. Its structure follows docs/06 §3, updated for v5 and for the honesty rules.

29.8.1 Rules for the video

- **Only real recordings.** Every chart and counter comes from a run in the evidence ledger. A split screen of FIFO and VeriQ is two runs of the same scripted scenario with the same seed, recorded one after the other, and the caption says so.
- **Placeholders stay visible until filled.** The script below uses `[MEASURED: …]` for numbers that come from a run. A placeholder never ships; if a number is not measured yet, cut the line.
- **Corner tags.** Each shot carries a small tag in a fixed corner: MEASURED (emulated link), ESTIMATED, ILLUSTRATIVE or TARGET.
- **One parity shot.** Show one result where VeriQ did not beat a baseline, such as static conditions, where VeriQ and SQM should be at parity because both use CAKE.

29.8.2 Shot list and narration

Time	Shot (visual)	Narration (spoken)	On-screen text	Tag
0:00–0:12	Black screen; white text appears line by line	“One download can add hundreds of milliseconds to your video call. Watch.”	“Day-0 sandbox: +266 ms upload p95 behind a FIFO (one preliminary run)”	MEASURED (emulated)
0:12–0:35	Split screen, two runs of S1: left FIFO, right VeriQ. Freeze counter and RTT chart under each	“Same call, same download, same emulated link. On the left, a plain FIFO router. On the right, VeriQ.”	“Two runs, same scenario and seed, recorded sequentially”	MEASURED
0:35–0:55	Download starts on both; counters diverge	“On the left, the queue fills in the modem and the call freezes. On the right, the queue lives in the router, where CAKE keeps it short.”	“Freezes: [MEASURED: n] vs [MEASURED: n]”	MEASURED
0:55–1:25	Hand presses WAN DROP; scoreboard shows true capacity 100 → 20; S line drops in one step; explain sentence appears	“Now the link drops to a fifth. VeriQ sees two independent signals, far-reflector delay and a throughput shortfall, and cuts once, to just below what the link still carries.”	Explain sentence from the decision log; “Confirmation: [MEASURED: s]”	MEASURED + ESTIMATED
1:25–1:55	Hand presses REMOTE DELAY; VeriQ S flat; beside it the comparator arm’s S from a recorded run	“Next, the delay comes from far away, beyond the provider. A cut cannot help here. VeriQ holds and says why. A delay-only controller has no way to tell the difference.”	“REMOTE_LIKELY @ L2: hold” and the comparator’s arm, version and run ID	MEASURED
1:55–2:15	3D twin: queue towers for the same run; evidence halo changes colour; receipt card	“Every action is checked, logged and followed by a receipt that says whether things improved.”	Receipt card text; “twin replays the decision log”	MEASURED / ESTIMATED / ILLUSTRATIVE per object
2:15–2:35	Hand lifts the flip cover, presses KILL; OLED counts; SAFE; call continues	“If the controller dies, the internet does not. A watchdog moves the shaper to a safe rate.”	“Time-to-safe: [MEASURED: s] (target ≤ 12 s)”	MEASURED
2:35–2:52	Results chart with confidence intervals from the H1 runs; parity row highlighted	“Across repeated runs, here is where VeriQ helped, and here, in static conditions, where it simply matches SQM, as we predicted.”	“n = [MEASURED] runs per arm; 95% bootstrap CIs”	MEASURED
2:52–3:00	Logo and one-line pitch	“VeriQ: everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM.”	Pitch; “TELIPORT S4, Case 3”	—

> **Honesty check.** The pitch line states the dominance invariant as a design goal. If the evaluation found a scenario where VeriQ was worse than static SQM, the closing narration must say so, for example: “...and in one scenario we are still working on, it did not.”

29.8.3 If results are not ready

Round 2 is due by 13 December 2026, and the H1 runs land in Sprint 4 (6–13 Dec). If the results chart is not ready, replace the 2:35 shot with the evaluation plan: arms, scenarios, primary outcome and margins, tagged TARGET. Never show a chart of planned numbers styled like a results chart.

29.9 Recording tips

Capture. - Use OBS Studio with one scene per shot type: split screen, kit close-up, twin, chart. Record at 1920×1080, 30 fps. - Film the panel with a phone on a fixed mount, at an angle that shows the hand and the OLED in one frame. - Show a millisecond wall clock (NTP-synchronised on all hosts) in a corner of every screen capture. It lets the editor align the panel video, the screen capture and the decision log, and it lets a judge check that the reaction followed the button press. - Record the decision log, tc statistics and WebRTC stats for every take. Each take gets a run ID in the evidence ledger, so every number on screen traces to a file.

Sound. - Use a clip-on microphone in a quiet room, recorded separately from the screen. Record the narration after the edit is locked, so the timing fits. - Speak slowly. At about 150 words per minute, the narration above runs close to 3 minutes.

Editing. - Use Kdenlive or DaVinci Resolve. Cut on actions such as button presses, not on narration pauses. - Never speed up a segment that shows a reaction time. If a segment must be shortened, cut it and show a visible “cut” marker; a judge must not mistake an edit for a faster response. - Burn in captions for every narrated line. Judges often watch without sound. - Export H.264 MP4. Keep a lossless master.

Publishing. - Teliport rules say submissions become Tata Elxsi’s property (docs/06 §11). Upload the video as private or unlisted, and share it only through the submission portal until the rules have been checked. - Store the final file, its edit project, and the run IDs it uses in the evidence ledger under `video/`.

29.9.1 Video acceptance checklist

Check	Pass when
Length	≤ 3:00 including titles
Every number	Traces to a run ID in the ledger; no `[MEASURED]` placeholder remains
Tags	Every shot carries MEASURED, ESTIMATED, ILLUSTRATIVE or TARGET
Split screen	Caption states “two runs, same scenario and seed, recorded sequentially”
Comparator	Arm name, version and run ID visible
Parity	At least one shot shows a tie or a loss
Day-0	Stated as one preliminary emulated run
Words	No “guarantee”, no “proven”, no superiority claim without the CI chart
Captions	Present and correct
Wall clock	Visible in each screen capture; panel and screen in sync

29.10 Rehearsal plan

- **Two weeks before the finale:** a full dress rehearsal with a team member playing a hostile judge who presses buttons in a random order and asks questions mid-step.
- **One week before:** rehearse every backup in the table above, with a stopwatch. A backup that takes more than 15 seconds gets simplified.
- **Day before:** pack the kit using the checklist in Chapter 32, run the pre-flight checklist once at the hotel or hostel, and charge every device.
- **After every rehearsal:** add any surprise to `docs/discrepancies.md` and fix the cause, not the script.

CHAPTER 30

3D Home-Network Twin

Bufferbloat is invisible. A judge cannot see a queue, a reflector or an evidence level; they can only see a call that freezes or does not. The 3D twin makes those internal quantities visible: a house with its devices, a pipe to the provider whose width is the shaper rate, glass towers that fill with queueing delay, beacons in the sky for the reflectors, and a halo around the router that shows how much evidence VeriQ has. This chapter specifies the twin completely, from its honesty rules to code skeletons, assets, performance and live mode.

The twin is a P1 deliverable (Sprint 5+, before the finale, docs/11 §18). It lives in the `twin/` folder of the repository (docs/11 §14.1). It runs off the router, in a browser on the demo laptop, and never touches the packet path.

30.1 Purpose and rules

30.1.1 What the twin is for

- **Explaining** the control loop to a judge in about 30 seconds: where the queue is, what VeriQ measured, what it concluded, what it did, and whether it helped.
- **Debugging** for the team: scrubbing a recorded run shows at a glance when a reflector went bad or when an action was refused.
- **Evidence presentation:** a replay of a ledger run can sit next to the result chart for the same run.

30.1.2 What the twin is not

- Not a simulator: it draws recorded or live telemetry and computes no network behaviour.
- Not a packet animation: particles are illustrative and drawn from aggregates.
- Not part of the product: the router does not serve it and does not depend on it.

30.1.3 The rules

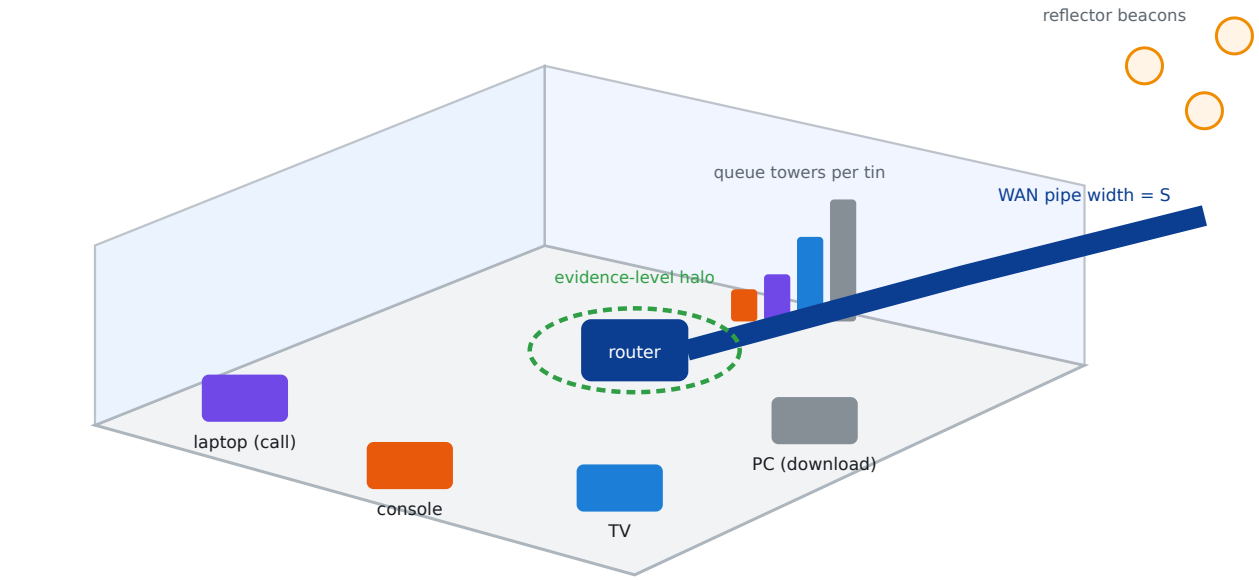
These rules extend the 3D replay rules in docs/09 §7 and the visual encodings in docs/05 (module M8). They are checked by the twin's own tests.

1. **It replays real event streams.** Every frame comes from a run in the evidence ledger (replay mode) or from the router's live API (live mode). No frame is hand-written, and no event is invented to make a scene look better.
2. **Every visible object carries a tag:** *measured*, *estimated*, *illustrative* or *truth*. The tag decides the visual encoding, as shown in the table below, and a legend explains it on screen at all times.
3. **The inferred upstream queue is drawn separately from ground truth.** VeriQ can only infer the queue in the ISP equipment from its probe deltas. In lab replays, the emulator's true queue is drawn beside it as a wireframe, which makes estimator accuracy visible.
4. **Aggregates only.** Flows are summarised per device and class. No per-packet animation and no per-flow export beyond what the decision log and stats already hold.
5. **Clicking an event opens its record:** the observations, diagnosis, proposed action, checker verdict, read-back and receipt, plus the application metrics for that time.
6. **Privacy.** The twin shows device names from the run's manifest. Demo runs use lab device names, never a real household's.

Tag	Meaning	Visual encoding	Examples
measured	Read from counters or probes on the router	Solid surfaces, full opacity	S (read back from tc), per-tin delay, Ĝ, reflector deltas
estimated	Computed by VeriQ from measurements	Outlined or translucent, with a band where a spread exists	μ±σ, diagnosis and level, inferred upstream queue
illustrative	Decoration that helps reading	Muted colours, a small "illustrative" note in the legend	House, rooms, furniture, particles
truth	Ground truth from the lab emulator, available only in lab replays	Dashed wireframe, labelled "TRUTH (lab only)"	True capacity, true ISP queue, the injected fault

> **Honesty check.** In live demos on the physical kit, truth comes from the ISP emulator, which is part of the lab. On a real home connection there is no truth channel, and the twin must hide all truth objects. The loader enforces this: a stream whose header says `"source":"router"` cannot contain truth frames.

30.2 Scene design



Timeline scrubber: replays the decision log; each object tagged measured / estimated / illustrative
Click an event → observations, diagnosis, checker verdict, receipt

Figure 15. 3D twin scene layout sketch.

The scene reads left to right, the same way the traffic flows: the home, the router, the access link, the provider's first hop, then the internet. The camera's default view is a three-quarter view from above, so every element is visible at once.

30.2.1 Layout

```
[study]   [living room]            first hop          sky: far reflector beacons
laptop    TV  console              (near beacon)        o   o   o   o   o
   \        |    /                      |
    \       |   /                       |              remote servers
[hall] ROUTER ==== WAN pipe (down/up) ==== ISP block ===== cloud ====== [server]
       queue towers per tin           inferred ISP queue      |
       evidence halo                  (+ truth wireframe)   remote-delay zone
[bedroom] phone/tablet
```

30.2.2 Objects and mappings

Object	Represents	Data field	Visual mapping	Tag
House, rooms, furniture	The home	Manifest (device → room)	Static low-poly model	illustrative
Device model	A LAN device	`devices[]` in the header	One model per device type; a name label	illustrative (model), measured (label)
Lease ring	An active lease on the device	`event.kind = lease`	Gold ring around the device until expiry; a countdown label	measured
Sanction badge	An envelope sanction on one of the device's flows	`event.kind = sanction`	Small red badge with the remaining penalty time	measured
Flow particles	Throughput per device and class	`flows` frames	Particle density ∝ kbit/s; colour = class	illustrative
Router and state badge	`veriqd` operating state	`tick.state`	Badge text and colour (LIVE, SHADOW, DEGRADED, SAFE …)	measured
Queue towers	Own queue per CAKE tin, per direction	`tins[].av_delay_ms`, `pk_delay_ms`	Glass tower; liquid height = average delay, a thin line = peak delay; red flash on new drops; small sparks on ECN marks	measured
WAN pipe	Configured shaper rate per direction	`tick.S`	Two lanes (down, up); lane width ∝ S	measured
Capacity sleeve	Capacity estimate	`tick.mu`, `tick.sigma`	Translucent sleeve around each lane from μ − σ to μ + σ	estimated
Truth ring	True emulator capacity	`truth.cap_down`, `truth.cap_up`	Dashed ring on each lane	truth

Object	Represents	Data field	Visual mapping	Tag
Inferred ISP queue	Delay beyond VeriQ's shaper	Derived: max(0, D − Q_own of the probe tin)	Hatched tower at the ISP block	estimated
Truth ISP queue	Emulator queue	`truth.isp_queue_ms`	Wireframe tower beside the hatched one	truth
Near beacon	WAN default gateway probe	`reflectors[role = near]`	Beacon on the ISP block; pulse height = delta	measured
Far beacons	Active far reflectors	`reflectors[role = far]`	Beacons on an arc in the sky; green healthy, amber outlier, grey unhealthy with a cool-off timer; double ring if ICMP timestamps are supported	measured (delta), estimated (health)
Evidence halo	Diagnosis and evidence level	`tick.diag`	Ring around the router; thickness and brightness grow with L0–L3; text "WAN_LIKELY @ L2: FAR + THR"	estimated
Action arcs	An action and its verdict	`event.kind = action`	Short arc from the router to the pipe; green for ACCEPT, red with the code for REJECT	measured
Receipt card	Outcome of an action	`event.kind = receipt`	Card that floats up 10 s after the action: IMPROVED, NOT_IMPROVED or INCONCLUSIVE	measured
Remote-delay zone	Injected remote delay (lab)	`truth.fault`	Dashed amber zone on the cloud link	truth

30.2.3 Scale choices

- **Pipe width.** Width is linear in S, clamped to a minimum of 4% of the maximum so a 1 Mbit/s lane stays visible. Linear scaling matters: a judge must see that 20 Mbit/s is a fifth of 100 Mbit/s.
- **Tower height.** Delay spans three orders of magnitude (1 ms to 1 s), so heights use a log scale: `h = H · log10(1 + d) / log10(1 + 1000)` for delay d in ms. Rings on the tower mark 5, 30, 60 and 200 ms, the boundaries of the wizard's bufferbloat grades (VeriQ's own boundaries, labelled as such).
- **Colours per class.** RT, IV, BE and BK use the class palette of Chapter 31, so the twin, LuCI, the LED bars on the kit and the charts all agree.

30.2.4 Camera presets

Key	Preset	Use
1	Overview	Default; whole scene
2	Router close-up	Queue towers and halo
3	Access link	Pipe, sleeve, truth ring, ISP queue towers
4	Sky	Reflector beacons
5	Device	The device selected in the side panel

30.3 Tech stack

Layer	Choice	Why	Alternatives considered
Rendering	three.js [R43]	WebGL standard; large community; MIT licence	Babylon.js (heavier); Unity WebGL (large download, a separate toolchain)
Scene graph	React with react-three-fiber and drei	Declarative scene; components match the object table; drei gives text, outlines, camera controls	Plain three.js (more glue code)
State	zustand	Small store; components read only the slice they need, which avoids re-rendering the scene every frame	React context (re-render storms)
Timeline chart	uPlot [R43]	Same chart library as LuCI; fast for long series	Chart.js
Build	Vite	Fast dev server; static output that can be opened offline	webpack

Layer	Choice	Why	Alternatives considered
Replay converter	Python (in `twin/tools/`)	Same language as the lab and evaluator	—
Live bridge	Python with the `websockets` package	Small; runs on the hub laptop	Node.js

All front-end dependencies are MIT-licensed; check each licence when adding a package, and pin exact versions with `npm install --save-exact`, so `package-lock.json` fixes the build. The twin is built to static files (`twin/dist/`) and opened from the hub laptop with no internet access.

> **Why it matters.** A twin that needs internet access, a cloud service or the router's CPU would contradict the design: control is local, and the router does nothing but route. The twin is a separate, off-box viewer.

30.4 The twin event stream

30.4.1 Sources

A replay is built from files that the lab already records for every run (Chapter 25):

- the **decision log** (JSONL, docs/11 §14.3), one line per tick per direction, with snapshot, diagnosis, action, verdict and read-back;
- **receipts**, with outcome and windows;
- **per-tin statistics** from `tc -s -j qdisc`, sampled every 250 ms by the lab runner;
- the **reflector table** (health, baseline, delta, timestamp support);
- **lease, sanction and state events**;
- **application metrics** from the harness: WebRTC `getStats` [R35], DASH player events, the synthetic game flow;
- **emulator truth** (lab only): true capacity, true queue, the fault schedule.

30.4.2 Format `twin.v1`

A replay file is JSONL. The first line is a header; every later line is a frame with a type and a time `t` in milliseconds since the run start (monotonic). Tags are declared once in the header, per field path, so the renderer can look them up and frames stay small.

```
{"type":"header","schema":"twin.v1","source":"ledger","run_id":"S2-veriq-r07",
 "scenario":"S2","arm":"veriq","t0_wall":"2026-11-20T20:40:00.000+05:30",
 "devices":[{"id":"d1","name":"Call laptop","kind":"laptop","room":"study"},
            {"id":"d2","name":"Download PC","kind":"desktop","room":"living"}],
 "tags":{"tick.S":"measured","tick.Ghat":"measured","tick.D":"measured",
         "tick.mu":"estimated","tick.sigma":"estimated","tick.diag":"estimated",
         "tins":"measured","reflectors.delta_ms":"measured",
         "reflectors.health":"estimated","truth":"truth","flows":"illustrative"}}
```

Typical frames (values are illustrative, chosen to match the example in docs/11 §14.3):

```
{"type":"tick","t":61250,"dir":"down","state":"LIVE","S":95000,"Ghat":20100,
 "load":0.99,"D":42.1,"qown":0.8,"near":39.0,"mu":21000,"sigma":1500,
 "diag":{"label":"WAN_LIKELY","level":2,"sources":["FAR","THR"]}}
{"type":"tins","t":61250,"dir":"down","tins":[
 {"tin":"voice","av_delay_ms":0.9,"pk_delay_ms":2.1,"drops":0,"ecn":0,"kbit":800},
 {"tin":"bulk","av_delay_ms":4.6,"pk_delay_ms":11.0,"drops":3,"ecn":12,"kbit":17000}]}
{"type":"reflectors","t":61000,"items":[
 {"id":"near","role":"near","health":"ok","delta_ms":39.0,"ts":false},
 {"id":"r3","role":"far","health":"ok","delta_ms":41.0,"ts":true}]}
{"type":"event","t":61250,"kind":"action","id":"01JC-demo","action":"RATE_CUT",
 "dir":"down","from":95000,"to":18090,"verdict":"ACCEPT","code":null}
{"type":"event","t":71250,"kind":"receipt","action_id":"01JC-demo","outcome":"IMPROVED"}
{"type":"truth","t":60000,"cap_down":20000,"cap_up":20000,"isp_queue_ms":180.0,
 "fault":"capacity_step"}
{"type":"app","t":61000,"device":"d1","webrtc":{"freezeCount":0,"rtt_ms":38}}
{"type":"flows","t":61000,"device":"d2","class":"BK","kbit":17000,"prov":"heuristic"}
```

Reflector IDs in replays are short aliases (`r1` … `r8`) mapped to addresses in the run manifest, so a screenshot shows no addresses.

30.4.3 Field mapping

Source	Source field	Twin field	Drives
Decision log	`snapshot.S`	`tick.S`	Pipe lane width
Decision log	`snapshot.Ghat`, `snapshot.load`	`tick.Ghat`, `tick.load`	Particle flow speed on the pipe; side panel
Decision log	`snapshot.D`, `snapshot.near.d`	`tick.D`, `tick.near`	Inferred ISP queue; near beacon

Source	Source field	Twin field	Drives
Decision log	`snapshot.QOwn`	`tick.qown`	Inferred ISP queue (subtracted)
Decision log	`diag`	`tick.diag`	Evidence halo and its text
Decision log	`state`	`tick.state`	Router badge
Decision log	`action`, `verdict`, `applied`	`event` (kind `action`)	Action arc; event panel
Receipts	`outcome`, windows	`event` (kind `receipt`)	Receipt card
Estimator log	μ, σ	`tick.mu`, `tick.sigma`	Capacity sleeve
tc -s -j	per-tin `av_delay`, `peak_delay`, `drops`, `ecn_mark`, bytes	`tins[]`	Queue towers
Prober	reflector table	`reflectors`	Beacons
Lease/sanction log	grant, revoke, expiry; sanction, escalation, recovery	`event`	Lease ring; sanction badge
Harness	WebRTC, DASH, game flow	`app`	Device label; side panel charts
Emulator	capacity schedule, tbf backlog, fault	`truth`	Truth ring, truth tower, fault zones

Implementation tip. Check the exact JSON key names of the CAKE tin statistics in `tc -s -j` output on the target iproute2 version before writing the converter. The converter maps them to the stable twin names above, so a key name change touches one function.

30.4.4 The converter

`twin/tools/build_replay.py` reads one run folder from the evidence ledger and writes `replay.twin.jsonl`. It refuses to write a frame without a source file, and it copies the run manifest's hash into the header, so every replay traces to its run.

```
"""Build a twin.v1 replay from one evidence-ledger run folder."""
import json, pathlib, sys

def tick_frames(path):
    for line in open(path, encoding="utf-8"):
        rec = json.loads(line)
        snap = rec["snapshot"]
        yield {"type": "tick", "t": rec["t_mono_ms"], "dir": rec["dir"],
               "state": rec["state"], "S": snap["S"], "Ghat": snap["Ghat"],
               "load": snap["load"], "D": snap["D"], "qown": snap["QOwn"],
               "near": snap["near"]["d"] if snap["near"]["ok"] else None,
               "diag": rec["diag"]}
        if rec.get("action"):
            yield {"type": "event", "t": rec["t_mono_ms"], "kind": "action",
                   "id": rec["action"]["id"], "action": rec["action"]["type"],
                   "dir": rec["dir"], "to": rec["action"].get("target"),
                   "verdict": "ACCEPT" if rec["verdict"]["accept"] else "REJECT",
                   "code": rec["verdict"].get("code")}

def main(run_dir, out):
    run = pathlib.Path(run_dir)
    manifest = json.loads((run / "manifest.json").read_text())
    t0 = manifest["t0_mono_ms"]
    frames = list(tick_frames(run / "decision_log.jsonl"))
    # tins_frames(), reflector_frames(), receipt_frames(), app_frames() and
    # truth_frames() follow the same pattern; truth only if manifest["lab"] is true.
    for f in frames:
        f["t"] -= t0
    frames.sort(key=lambda f: f["t"])
    header = {"type": "header", "schema": "twin.v1", "source": "ledger",
              "run_id": manifest["run_id"], "manifest_sha256": manifest["sha256"]}
    with open(out, "w", encoding="utf-8") as fh:
        for obj in [header, *frames]:
            fh.write(json.dumps(obj, separators=(",", ":")) + "\n")

if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])
```

Time alignment. veriqd times are monotonic; harness and emulator times are wall-clock. The run manifest records the offset between them at the start and end of the run; the converter applies it and warns if the drift exceeds 50 ms (a starting value).

30.5 Step-by-step build guide

30.5.1 Step 1: scaffold

```
cd veriq/twin
npm create vite@<x.y.z> . -- --template react  # pin the create-vite version you tested
npm install --save-exact three @react-three/fiber @react-three/drei zustand uplot
npm run dev                                    # http://localhost:5173
```

Folder layout:

```
twin/
├── src/
│   ├── data/      loader.js, index.js, live.js
│   ├── store.js
│   ├── scene/     Scene.jsx, House.jsx, Router.jsx, WanPipe.jsx, QueueTower.jsx,
│   │              Beacon.jsx, EvidenceHalo.jsx, Device.jsx, Particles.jsx
│   ├── ui/        Timeline.jsx, EventPanel.jsx, Legend.jsx, SidePanel.jsx
│   └── palette.js
├── public/models/ house.glb, devices.glb, router.glb
├── public/replays/ *.twin.jsonl
└── tools/          build_replay.py, bridge.py, validate.py
```

30.5.2 Step 2: load and index a replay

The loader parses JSONL once, splits frames by type and keeps each list sorted by time. Lookups during playback use binary search, so scrubbing a 10-minute run stays instant.

```
// src/data/loader.js
export async function loadReplay(url) {
  const text = await (await fetch(url)).text();
  const lines = text.split('\n').filter((l) => l.trim().length > 0);
  const header = JSON.parse(lines[0]);
  if (header.schema !== 'twin.v1') throw new Error(`unsupported schema ${header.schema}`);
  const byType = { tick: [], tins: [], reflectors: [], event: [], truth: [], app: [],
    flows: [] };
  for (let i = 1; i < lines.length; i++) {
    const f = JSON.parse(lines[i]);
    if (f.type === 'truth' && header.source === 'router') {
      throw new Error('truth frames are not allowed in a router stream');
    }
    (byType[f.type] ??= []).push(f);
  }
  for (const list of Object.values(byType)) list.sort((a, b) => a.t - b.t);
  const tEnd = Math.max(...Object.values(byType).map((l) => (l.length ? l.at(-1).t : 0)));
  return { header, byType, tEnd };
}

// Last frame at or before t (binary search). Optional filter, e.g. by direction.
export function frameAt(list, t, pred = () => true) {
  let lo = 0, hi = list.length - 1, best = -1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (list[mid].t <= t) { best = mid; lo = mid + 1; } else { hi = mid - 1; }
  }
  for (let i = best; i >= 0 && i > best - 8; i--) if (pred(list[i])) return list[i];
  return null;
}

export function tagOf(header, path) {
  const tags = header.tags || {};
  return tags[path] ?? tags[path.split('.')[0]] ?? 'untagged';
}
```

30.5.3 Step 3: the playback store

```
// src/store.js
import { create } from 'zustand';

export const useTwin = create((set, get) => ({
  replay: null,         // { header, byType, tEnd }
  t: 0,                 // playback time, ms since run start
  playing: false,
  speed: 1,             // 0.25, 0.5, 1, 2, 4
  selectedEvent: null,
  mode: 'replay',       // 'replay' | 'live'
  setReplay: (replay) => set({ replay, t: 0, playing: false }),
  seek: (t) => set({ t: Math.max(0, Math.min(t, get().replay?.tEnd ?? 0)) }),
  toggle: () => set({ playing: !get().playing }),
  advance: (dtMs) => {
    const { playing, t, speed, replay } = get();
    if (!playing || !replay) return;
    const next = t + dtMs * speed;
    set({ t: Math.min(next, replay.tEnd), playing: next < replay.tEnd });
  },
```

```
  select: (ev) => set({ selectedEvent: ev }),
}));
```

30.5.4 Step 4: the scene

The scene components read the store inside `useFrame` and mutate three.js objects through refs. They do not use React state for per-frame values, so React does not re-render the scene 60 times a second.

```jsx
// src/scene/Scene.jsx
import { Canvas, useFrame } from '@react-three/fiber';
import { OrbitControls } from '@react-three/drei';
import { useTwin } from '../store';
import { House } from './House';
import { Router } from './Router';
import { WanPipe } from './WanPipe';
import { QueueTowers } from './QueueTower';
import { Beacons } from './Beacon';
import { EvidenceHalo } from './EvidenceHalo';

function Clock() {
  const advance = useTwin((s) => s.advance);
  useFrame((_, delta) => advance(delta * 1000));
  return null;
}

export function Scene() {
  return (
    <Canvas camera={{ position: [6, 9, 14], fov: 45 }} dpr={[1, 1.5]}>
      <ambientLight intensity={0.6} />
      <directionalLight position={[5, 10, 5]} intensity={0.8} />
      <Clock />
      <House />
      <Router position={[0, 0, 0]} />
      <QueueTowers position={[0, 0, 1.5]} />
      <EvidenceHalo position={[0, 0.05, 0]} />
      <WanPipe from={[0.8, 0.6, 0]} to={[8, 0.6, 0]} />
      <Beacons origin={[12, 4, 0]} />
      <OrbitControls makeDefault />
    </Canvas>
  );
}
```

```jsx
// src/scene/WanPipe.jsx: two lanes; width = S (measured), sleeve = mu ± sigma (estimated)
import { useRef } from 'react';
import { useFrame } from '@react-three/fiber';
import { useTwin } from '../store';
import { frameAt } from '../data/loader';

const MAX_KBIT = 110000;     // scene scale; set from the header's S_max
const MAX_W = 0.6;           // metres in scene units
const widthOf = (kbit) => Math.max(0.04 * MAX_W, (kbit / MAX_KBIT) * MAX_W);

function Lane({ dir, z, length }) {
  const core = useRef();
  const sleeve = useRef();
  const truth = useRef();
  useFrame(() => {
    const { replay, t } = useTwin.getState();
    if (!replay) return;
    const tick = frameAt(replay.byType.tick, t, (f) => f.dir === dir);
    if (tick) {
      const w = widthOf(tick.S);
      core.current.scale.set(1, w, w);
      const hasMu = tick.mu != null && tick.sigma != null;
      sleeve.current.visible = hasMu;
      if (hasMu) {
        const wHi = widthOf(tick.mu + tick.sigma);
        sleeve.current.scale.set(1, wHi, wHi);
      }
    }
    const tr = frameAt(replay.byType.truth, t);
    truth.current.visible = Boolean(tr);
    if (tr) {
      const w = widthOf(dir === 'down' ? tr.cap_down : tr.cap_up);
      truth.current.scale.set(1, w, w);
    }
  });
  return (
    <group position={[length / 2, 0, z]}>
      <mesh ref={core} rotation={[0, 0, Math.PI / 2]}>
        <cylinderGeometry args={[0.5, 0.5, length, 24]} />
        <meshStandardMaterial color={dir === 'down' ? '#3a7bd5' : '#7b8794'} />
      </mesh>
      <mesh ref={sleeve} rotation={[0, 0, Math.PI / 2]}>
```

```
        <cylinderGeometry args={[0.5, 0.5, length, 24, 1, true]} />
        <meshStandardMaterial transparent opacity={0.18} color="#ffffff" />
      </mesh>
      <mesh ref={truth} rotation={[0, 0, Math.PI / 2]}>
        <cylinderGeometry args={[0.5, 0.5, length, 24, 1, true]} />
        <meshBasicMaterial wireframe color="#ffb000" />
      </mesh>
    </group>
  );
}

export function WanPipe({ from, to }) {
  const length = to[0] - from[0];
  return (
    <group position={from}>
      <Lane dir="down" z={-0.45} length={length} />
      <Lane dir="up" z={0.45} length={length} />
    </group>
  );
}
```

```
// src/scene/QueueTower.jsx: one tower per CAKE tin and direction (measured)
import { useRef } from 'react';
import { useFrame } from '@react-three/fiber';
import { useTwin } from '../store';
import { frameAt } from '../data/loader';
import { CLASS_COLOURS } from '../palette';

const H = 3.0;
const heightOf = (ms) => (H * Math.log10(1 + Math.max(0, ms))) / Math.log10(1001);
const TINS = [['voice', 'RT'], ['video', 'IV'], ['besteffort', 'BE'], ['bulk', 'BK']];

function Tower({ dir, tin, cls, x }) {
  const liquid = useRef();
  const peak = useRef();
  const lastDrops = useRef(0);
  useFrame(() => {
    const { replay, t } = useTwin.getState();
    const f = replay && frameAt(replay.byType.tins, t, (fr) => fr.dir === dir);
    const s = f?.tins.find((x2) => x2.tin === tin);
    if (!s) return;
    const h = Math.max(0.001, heightOf(s.av_delay_ms));
    liquid.current.scale.y = h;
    liquid.current.position.y = h / 2;
    peak.current.position.y = heightOf(s.pk_delay_ms);
    const flash = s.drops > lastDrops.current;
    liquid.current.material.emissiveIntensity = flash ? 1.0 : 0.0;
    lastDrops.current = s.drops;
  });
  return (
    <group position={[x, 0, dir === 'down' ? -0.6 : 0.6]}>
      <mesh position={[0, H / 2, 0]}>
        <cylinderGeometry args={[0.18, 0.18, H, 16, 1, true]} />
        <meshPhysicalMaterial transparent opacity={0.15} roughness={0.1} />
      </mesh>
      <mesh ref={liquid}>
        <cylinderGeometry args={[0.16, 0.16, 1, 16]} />
        <meshStandardMaterial color={CLASS_COLOURS[cls]} emissive="#ff0000" />
      </mesh>
      <mesh ref={peak}>
        <torusGeometry args={[0.19, 0.01, 6, 24]} />
        <meshBasicMaterial color="#222222" />
      </mesh>
    </group>
  );
}

export function QueueTowers(props) {
  return (
    <group {...props}>
      {['down', 'up'].flatMap((dir) =>
        TINS.map(([tin, cls], i) => (
          <Tower key={dir + tin} dir={dir} tin={tin} cls={cls} x={i * 0.45 - 0.7} />
        )))}
    </group>
  );
}
```

```
// src/scene/EvidenceHalo.jsx: ring thickness and brightness by evidence level (estimated)
import { useRef } from 'react';
import { useFrame } from '@react-three/fiber';
import { Text } from '@react-three/drei';
import { useTwin } from '../store';
import { frameAt } from '../data/loader';
```

```
const LEVEL_STYLE = [
  { tube: 0.02, opacity: 0.25 },   // L0
  { tube: 0.04, opacity: 0.45 },   // L1
  { tube: 0.07, opacity: 0.70 },   // L2
  { tube: 0.10, opacity: 0.95 },   // L3
];

export function EvidenceHalo(props) {
  const ring = useRef();
  const label = useRef();
  useFrame(() => {
    const { replay, t } = useTwin.getState();
    const tick = replay && frameAt(replay.byType.tick, t, (f) => f.dir === 'down');
    if (!tick?.diag) return;
    const st = LEVEL_STYLE[tick.diag.level] ?? LEVEL_STYLE[0];
    ring.current.scale.setScalar(1 + st.tube * 4);
    ring.current.material.opacity = st.opacity;
    const src = (tick.diag.sources || []).join(' + ');
    label.current.text = `${tick.diag.label} @ L${tick.diag.level}${src ? ': ' + src : ''}`;
  });
  return (
    <group {...props}>
      <mesh ref={ring} rotation={[-Math.PI / 2, 0, 0]}>
        <torusGeometry args={[1.1, 0.05, 8, 64]} />
        <meshBasicMaterial transparent color="#f2a33a" />
      </mesh>
      <Text ref={label} position={[0, 2.2, 0]} fontSize={0.22} color="#ffffff">
        {' '}
      </Text>
    </group>
  );
}
```

The halo above follows the download direction; a toggle in the side panel switches it to upload. Beacons, devices and particles follow the same pattern: look up the frame at time t, then mutate refs. Particles use one `InstancedMesh` per class, with the instance count set from the device's kbit/s (illustrative), so thousands of particles cost a handful of draw calls.

30.5.5 Step 5: timeline and event panel

The timeline is a strip at the bottom of the screen:

- a uPlot chart of S and the true capacity (when available) for both directions;
- event markers above it (action, refusal, receipt, lease, sanction, state change, fault);
- a play head that the user drags to seek.

Clicking a marker calls `select(event)`, and the event panel shows the full decision-log line for that tick, the receipt (if any) and the application metrics in a ±10 s window.

```
// src/ui/EventPanel.jsx
import { useTwin } from '../store';
import { frameAt } from '../data/loader';

export function EventPanel() {
  const ev = useTwin((s) => s.selectedEvent);
  const replay = useTwin((s) => s.replay);
  if (!ev || !replay) return null;
  const tick = frameAt(replay.byType.tick, ev.t, (f) => f.dir === ev.dir);
  const receipt = replay.byType.event.find(
    (e) => e.kind === 'receipt' && e.action_id === ev.id);
  return (
    <aside className="event-panel">
      <h3>{ev.action} {ev.dir}: {ev.verdict}{ev.code ? ` (${ev.code})` : ''}</h3>
      <p>Evidence: {tick?.diag.label} @ L{tick?.diag.level}
        [{tick?.diag.sources.join(', ')}]</p>
      <p>S {tick?.S} kbit/s, Ĝ {tick?.Ghat} kbit/s, D {tick?.D} ms, Q_own {tick?.qown} ms</p>
      <p>Receipt: {receipt ? receipt.outcome : 'pending or none'}</p>
      <p class="tag">Tags: measured (S, Ĝ, D), estimated (diagnosis)</p>
    </aside>
  );
}
```

30.5.6 Step 6: legend and tag audit

The legend is always visible in the top-right corner and shows the four tag encodings. A development-mode **tag audit** walks the scene graph and logs any mesh whose `userData.tag` is missing; the CI test (below) fails if the audit finds one. Every component sets `userData.tag` on its root group.

30.5.7 Step 7: tests

Test	What it checks
`validate.py` on every replay	Header present, schema `twin.v1`, every frame has `type` and `t`, times sorted, no truth frames in router streams
Golden replay	A short recorded S2 run loads, and at t = 70 s the lane width and halo text match stored values
Tag audit	No untagged mesh in the scene
Performance smoke	A 10-minute replay plays at 4× for 60 s with no frame longer than 50 ms on the demo laptop (target)
Live-mode fallback	When the WebSocket closes, the UI shows "LIVE: disconnected" within 2 s and stops moving

30.6 Assets

30.6.1 Modelling in Blender

- **Style:** low-poly, flat-shaded, matte. The twin should look like a clear diagram, not a game. Muted colours for the house, so the measured objects (towers, pipe, beacons) stand out.
- **Budget (targets):** house ≤ 15k triangles; each device model ≤ 1.5k; total scene ≤ 50k triangles; textures ≤ 1024 × 1024, preferably none (vertex colours).
- **Names:** name objects for the code: `room_study`, `room_living`, `anchor_router`, `anchor_device_01`, `anchor_isp`, `anchor_sky`. Code positions dynamic objects at these anchors, so the model can change without code changes.
- **Units:** 1 Blender unit = 1 scene unit; origin at the router anchor; +Y up in three.js.

30.6.2 Export

- File → Export → glTF 2.0, format **glTF Binary (.glb)**.
- Include: selected objects or the visible collection; apply modifiers; +Y up.
- Leave Draco compression off at first. Turn it on only if the file exceeds about 2 MB, and then ship the Draco decoder files locally in `public/`, because the demo runs offline.
- Load with drei's `useGLTF('/models/house.glb')`.

30.6.3 Third-party assets

CC0 low-poly packs (for example Kenney, as suggested in docs/06) are acceptable for furniture and devices. Record each asset's source and licence in `twin/ASSETS.md`. Do not use assets whose licence forbids redistribution, and do not use real brand logos on devices.

30.7 Performance tips

- **Mutate, do not re-render.** Read the store with `useTwin.getState()` inside `useFrame` and write to refs. Subscribe React components only to values that change rarely (selected event, mode).
- **Instancing** for particles and repeated device models.
- **Pre-index** frames by type and direction at load time; never filter the full array per frame.
- **Parse in a Web Worker** if a replay is larger than about 20 MB, so the UI does not stall at load.
- **Clamp the pixel ratio** (`dpr={[1, 1.5]}`) on high-density laptop screens.
- **Pause rendering when paused:** use `frameloop="demand"` while playback is paused and call `invalidate()` on interaction.
- **Avoid shadows and post-processing** on integrated graphics; the twin needs readability, not realism.
- **Target:** a steady 60 fps on the demo laptop, at least 30 fps on the 7-inch booth display (targets, to be measured).

30.8 How to demo it

1. **Open on the overview** (key 1) with a replay of the rehearsal run of S2 loaded and paused just before the drop. Point at the legend first: "solid is measured, outlined is estimated, dashed is lab truth, the house is decoration."
2. **Play at 1×.** The truth ring shrinks at the drop. Then the far beacons rise, the halo thickens to L2 with "WAN_LIKELY: FAR + THR", the green action arc fires, and the lane shrinks to the truth ring. Ten seconds later, the receipt card rises.
3. **Click the action marker** to open the event panel and read the evidence aloud.
4. **Switch to the S6 replay** (remote delay): the far beacons rise, the near beacon stays flat, the halo reads REMOTE_LIKELY, and the lane does not move. "Delay rose, but there was nothing to cut."
5. **Switch to live mode** if the kit is running, and let the judge press a button.

Bookmarks jump to the key moments of each prepared replay; each is a real ledger run, named on screen by its run ID.

> **For judges.** If a judge asks “is this animation scripted?”, open the event panel and the run ID, and offer to load any other run from the ledger. The twin draws only what the logs contain.

30.9 Live mode

30.9.1 Architecture

```
Router (VeriQ)                          Hub laptop (off-box)
veriqd ── rpcd (luci.veriq) ── HTTP /ubus, LAN, read-only ──► bridge.py ── WS ──► twin
       └─ veriq-exporter (P2, opt.) ── HTTP :9567, LAN ──────►    │
                                                                  └─► live.twin.jsonl
```

The router serves no WebSocket and gains no new listener. A small bridge on the hub laptop polls the router over the LAN and pushes `twin.v1` frames to the browser:

- **Default (P1):** rpcd over HTTP. The bridge logs in as a read-only rpcd user whose ACL allows only status, queue statistics, autorate state and recent events (Chapter 21 defines the methods). This is the path LuCI itself uses.
- **When the exporter exists (P2):** scrape the Prometheus text endpoint on the LAN (default port 9567) for counters, and use rpcd only for events.

The bridge writes every frame it sends to `live.twin.jsonl`. A live demo can therefore be replayed afterwards, frame for frame, and filed in the ledger.

Live streams have `"source":"router"` in their header, so truth frames are refused. On the physical kit, a separate `truth` channel from the ISP emulator may be enabled; the header then says `"source":"lab-live"` and the legend shows the truth tag.

30.9.2 Bridge skeleton

```
"""bridge.py: poll the router over rpcd and push twin.v1 frames over WebSocket."""
import asyncio, json, time
import requests                     # Apache-2.0
import websockets                   # BSD-3-Clause

ROUTER = "http://192.168.1.1/ubus"
NULL_SESSION = "0" * 32
OBJECT = "luci.veriq"               # rpcd plugin; method names per Chapter 21
CLIENTS = set()

def ubus(session, obj, method, args=None):
    body = {"jsonrpc": "2.0", "id": 1, "method": "call",
            "params": [session, obj, method, args or {}]}
    reply = requests.post(ROUTER, json=body, timeout=1.0).json()
    code, data = reply["result"][0], (reply["result"][1:] or [None])[0]
    if code != 0:
        raise RuntimeError(f"ubus {obj}.{method} failed with code {code}")
    return data

def login(user, password):
    return ubus(NULL_SESSION, "session", "login",
                {"username": user, "password": password})["ubus_rpc_session"]

async def poll(session, log):
    t0 = time.monotonic()
    while True:
        status = ubus(session, OBJECT, "status")
        t = int((time.monotonic() - t0) * 1000)
        frames = to_frames(status, t)          # map fields to tick/tins/reflectors
        for f in frames:
            line = json.dumps(f, separators=(",", ":"))
            log.write(line + "\n")
            websockets.broadcast(CLIENTS, line)
        await asyncio.sleep(0.25)

async def handler(ws):
    CLIENTS.add(ws)
    try:
        await ws.wait_closed()
    finally:
        CLIENTS.discard(ws)

async def main():
    session = login("kit", open("/etc/veriq-kit/password").read().strip())
    with open("live.twin.jsonl", "a", encoding="utf-8") as log:
        log.write(json.dumps({"type": "header", "schema": "twin.v1",
                              "source": "router", "mode": "live"}) + "\n")
        async with websockets.serve(handler, "127.0.0.1", 8765):
            await poll(session, log)

def to_frames(status, t):
```

```
    raise NotImplementedError("map the status fields of Chapter 21 to twin.v1 frames")

if __name__ == "__main__":
    asyncio.run(main())
```

For brevity the skeleton makes a blocking HTTP call inside the event loop; the real bridge should use a thread executor or an async client. It binds to `127.0.0.1`, so only the hub laptop's browser can connect.

30.9.3 Live client

```
// src/data/live.js: append live frames to the same structures the replay uses
import { useTwin } from '../store';

export function connectLive(url = 'ws://127.0.0.1:8765') {
  const replay = { header: { schema: 'twin.v1', source: 'router', tags: {} },
    byType: { tick: [], tins: [], reflectors: [], event: [], truth: [], app: [],
      flows: [] }, tEnd: 0 };
  useTwin.setState({ replay, mode: 'live', playing: false });
  const ws = new WebSocket(url);
  ws.onmessage = (msg) => {
    const f = JSON.parse(msg.data);
    if (f.type === 'header') { replay.header = f; return; }
    if (f.type === 'truth' && replay.header.source === 'router') return;
    const list = (replay.byType[f.type] ??= []);
    list.push(f);
    if (list.length > 20000) list.splice(0, 5000);   // keep memory bounded
    replay.tEnd = Math.max(replay.tEnd, f.t);
    useTwin.setState({ t: replay.tEnd });            // follow the newest frame
  };
  ws.onclose = () => useTwin.setState({ mode: 'live-disconnected' });
  return ws;
}
```

Because live frames go into the same structures as a replay, every scene component works unchanged in both modes. The timeline in live mode shows the last 5 minutes and lets the presenter scrub back.

30.9.4 Security and privacy in live mode

- The rpcd user `kit` is read-only and exists only on the demo kit image.
- The bridge and the twin run on the hub laptop and talk only over the LAN and localhost.
- Nothing is sent off the laptop. The live log is filed in the ledger like any other run.

30.10 Extending the twin

- **Comparator overlay:** load two replays of the same scenario (VeriQ and a comparator arm) and draw the comparator's lane as a ghost pipe, with both arms and run IDs labelled.
- **Wi-Fi layer:** station signal and retry rate as a halo around each wireless device, from the WIFI evidence group.
- **Operator view (P2 idea):** one small twin per pilot home, from aggregated receipts; it needs consent and the DPDP Act review of Chapter 17 first.

30.11 Acceptance checklist

Item	Done when
Replay of any ledger run	`build_replay.py` + `validate.py` pass; the twin loads it
Tags	Legend visible; tag audit reports zero untagged meshes
Truth separation	Truth objects appear only in lab streams; inferred and true ISP queues drawn side by side
Event panel	Every action marker opens its decision-log line, verdict and receipt
Offline	Runs from `twin/dist/` with the network cable unplugged
Performance	Targets above met on the demo laptop and the booth display
Live mode	Bridge connects as the read-only user; disconnection shown within 2 s; live log saved
Licences	`twin/ASSETS.md` lists every third-party asset and its licence

CHAPTER 31

Chapter 31 (pending)

This chapter has not been written yet.

CHAPTER 32

Chapter 32 (pending)

This chapter has not been written yet.

CHAPTER 33

Submission Kit

This chapter turns the handbook into what the judges will see: a ~20-page technical submission, a pitch deck, the Round 1 slide, and the answers to the panel's questions. It ends with an honest self-assessment and a pre-submission checklist.

Everything here follows one rule from the rest of the handbook: **the submission may only say what the evidence ledger supports.** Today (12 Oct 2026) the only measured numbers are the Day-0 sandbox values [R51]. Every other number must be labelled as a target, starting parameter, assumption or illustration.

> **Note.** Dates used in this chapter: Round 1 is due 24 Oct 2026 (per public listings [R50]; confirm on the portal). The Round 2 window is 14 Nov–13 Dec 2026. The finale date is TBC. The listings describe Round 1 as a single slide (PDF) and Round 2 as a 10-slide presentation plus multimedia or prototype (docs/01). The ~20-page document below is the technical report behind that deck; attach or link it only if the portal allows, and confirm formats and page limits first.

33.1 The 20-page submission blueprint

33.1.1 Principles for the document

Write for two readers: a senior leader who reads pages 1–2 and the figures, and a network engineer who checks every number.

- **Page 2 must stand alone:** problem, idea, dominance invariant, what is measured and what is only planned.
- **One idea and one figure per page**, with a bold one-line takeaway.
- **Every number carries a tag**: *measured* (with n, the environment fingerprint and the ledger path), *target*, *starting parameter*, *assumption* or *illustrative*.
- **Show one place where VeriQ did not win**, once results exist. A parity result in static conditions is expected and is reported as parity.

33.1.2 Page-by-page plan

The plan totals about 7,000 words plus 16–18 figures. Budgets are upper limits. "Ch." is the handbook chapter to pull from.

Page	Title	Content	Pull from	Words
1	Cover	Title "VeriQ: adaptive home QoS that knows when not to cut"; team, college, case number; the one-line pitch; a small legend of the honesty tags (measured / target / illustrative)	Ch. 1	100
2	Executive summary	Problem in four sentences; VeriQ in one paragraph; the dominance invariant ("never worse than static SQM"); what is new vs QoSmate and cake-autorate (stated as design claims); evidence so far (Day-0 only); primary outcome; deliverables P0/P1; a 6-row "at a glance" table	Ch. 1	450
3	The problem: queues, not bandwidth	Bufferbloat from first principles; the worked buffer example; the Day-0 table (FIFO 266.1 / 63.6 ms vs short-buffer shaper 39.6 / 55.4 ms added p95, one run, no base RTT, IPv4 only); variable capacity; the cause problem; the intent problem	Ch. 4; fig01_bufferbloat	350
4	Case 3 requirements and traceability	The case challenge and constraints in a compact table: requirement → VeriQ feature → test/scenario; non-functional requirements (privacy, safety, footprint, reproducibility)	Ch. 2	350

Page	Title	Content	Pull from	Words
5	Landscape and positioning	SQM, qosify, cake-autorate, QoSmate (read), commercial DPI-based QoS in general terms; the condensed comparison table; the three hypotheses VeriQ must prove; “same symptom, three causes”	Ch. 5; fig21_threecases	400
6	Solution overview	The nine principles P1–P9 and what enforces each; three speeds; the 250 ms loop narrative; what VeriQ is not (no payload inspection, no bandwidth guarantee, no cloud)	Ch. 6; fig06_control_loop	350
7	Architecture	Layers and planes; the veriqd packages in one table; deployment views D1–D5; process model on OpenWrt	Ch. 7; fig02_architecture	300
8	Datapath and queue management	Four service classes and the three backends (cake default, hfsc, htb); mark layout; upload via `mark_out`, download via `act_ctinfo` → IFB; DSCP policy (product washes, lab does not)	Ch. 8, 20; fig03_datapath, fig05_markbits	350
9	Classification and intent	Privacy contract; the 7-level precedence with the exact abstention sentence; user rules; heuristics that only demote; ML with Mondrian conformal abstention and the honesty kit; leases and the eligibility envelope	Ch. 9; fig04_precedence	400
10	Capacity estimation and rate control	S, G_gw, $\hat{G}$, μ/σ, D, Q_own never conflated; reflectors, median and quorum; windowed-min baseline; FSM; the C1 one-step cut with the illustrative 100 → 20 Mbit/s example	Ch. 10; fig07_rate_fsm, fig20_cut_example	400
11	Diagnosis	Evidence groups and independence; labels; levels L0–L3; the permission table; missing-near rules; one worked case (remote delay → hold)	Ch. 11; fig09_diagnosis, fig10_reflectors	350
12	Safety	Action Checker (refusal codes table, short); operating states; reconciler; watchdog and SAFE; budgets that cooldowns never refund; the safety case in five lines	Ch. 12; fig08_opstates	400
13	Receipts and explainability	Receipt fields; IMPROVED / NOT_IMPROVED / INCONCLUSIVE; association, not causation; two explain sentences; household messages	Ch. 13; fig19_receipt	350
14	Adverse conditions	The cut-fast / hold / degrade-to-static pattern; a 10-row extract of A1–A32 (A1, A4, A5, A6, A7, A9, A14, A15, A17, A32) with the delay-only behaviour labelled hypothesis	Ch. 14; fig18_adverse_map	300

Page	Title	Content	Pull from	Words
15	Product: OpenWrt and UX	Packages, signed feed, procd/UCI/hotplug, footprint budgets (targets); setup wizard; LuCI Overview; household page; QoSmate importer	Ch. 15, 16; fig12_openwrt_stack, fig13_luci_overview, fig14_household	400
16	Security, privacy and licences	Threat table (short); data inventory and retention; DPDP Act 2023 note (not legal advice); signing, SBOM; clean-room policy (GPL ideas only)	Ch. 17	300
17	Evaluation design	Research questions; arms incl. QoSmate head-to-head; tiers H0–H4; Stage 1–4; metric definitions; statistics (run as the unit, bootstrap, Holm, non-inferiority)	Ch. 24, 25; fig11_lab_topology	400
18	Results	Only ledger-generated, measured numbers with n, CIs and the fingerprint; at least one parity or loss shown; until H1 runs exist, this page shows the Day-0 table and the run plan, clearly labelled	Ch. 24, 26; ledger	350
19	Business case and Tata Elxsi fit	Value chain; packaging options; the illustrative operator model (≈ ₹1.08 crore/yr *gross modelled avoided cost*) with the net formula; the porting path (prplOS/USP, RDK-B); no endorsement implied	Ch. 3, 27; fig22_value_chain	350
20	Plan, risks and known limitations	Sprints to 13 Dec; top-5 risks; known limitations (emulation, CAKE single-threaded, Wi-Fi diagnosis needs driver stats, download shaping needs responsive senders); references	Ch. 22, 28; fig15_gantt	400

Implementation tip. Put every data-derived number behind a placeholder such as `{{h1.s6.count.veriq.median}}` that a script fills from the ledger. The claims check then compares each rendered number with its recomputed value, which is how principle P9 reaches the PDF.

33.1.3 Figures and tables policy

- Results charts (page 18) are generated by the plotting scripts in Ch. 31 from the ledger, never drawn by hand.
- Every chart carries a footer tag: “measured, n = ..., fingerprint ...”, “illustrative, not measured” or “target”.

33.2 Pitch deck outline

The master deck has 14 slides. The **10-slide cut** for the Round 2 format is marked in the last column; the remaining slides become backup slides for Q&A or the finale. Each note is about 40 seconds of speech.

#	Slide	On the slide	Speaker notes	10-slide cut
1	Title	“VeriQ: adapts, measures, explains.” Team, college, Case 3 (MCV). One-line pitch.	“Home broadband is shared by calls, games, streaming and downloads. VeriQ is an OpenWrt-native QoS system that does everything a mature tool like QoSmate does, plus a controller that knows when not to cut, proves what it did, and is designed never to leave the household worse off than plain SQM.”	Yes

#	Slide	On the slide	Speaker notes	10-slide cut
2	Queues, not bandwidth	fig01; the Day-0 table, tagged “one preliminary run, no base RTT, IPv4 only”	“In our first sandbox run, a saturating upload behind an oversized FIFO added 266 ms at the 95th percentile. That is one run, with no base RTT, so we treat it as an illustration. The fix is known: shape just below the bottleneck with CAKE. Three problems remain: capacity moves, delay often comes from places the router cannot fix, and households need intent, not knobs.”	Yes
3	Same symptom, three causes	fig21: access-link drop, remote server, Wi-Fi; three correct actions	“A delay rise can mean the access link dropped, a far server is slow, or a phone has weak Wi-Fi. Only the first one is fixed by cutting the shaper. Delay-only autorate controllers cut in all three cases, we expect; that expectation is a hypothesis we test head-to-head.”	Yes
4	The VeriQ idea	Dominance invariant; three layers; three speeds	“VeriQ’s promise is testable: where it lacks evidence it holds, or reverts to static CAKE equivalent to well-tuned SQM. Adaptive moves happen only where evidence supports them, in bounded steps. So its worst case is static SQM.”	Merge into 5
5	Architecture	fig02; veriqd in Go; kernel datapath; watchdog	“No user-space process touches packets. The kernel does nftables classification and CAKE shaping in both directions. veriqd runs a 250 ms loop: observe, classify, estimate, diagnose, propose, check, enforce, receipt. A separate watchdog moves the shaper to a SAFE static profile if the daemon stops.”	Yes
6	Diagnosis before action	fig09 and the permission table	“Evidence comes in independent groups: far reflectors, the near hop, throughput censoring, our own queue, Wi-Fi stats, CPU. Two independent groups give level L2. Only WAN_LIKELY at L2 allows a fast cut. Remote, Wi-Fi or CPU causes mean hold and explain. Uncertainty allows only small, budgeted steps.”	Yes
7	Fast where it should be	fig20 (illustrative); C1	“When the evidence is strong, VeriQ cuts in one step to 90% of the measured gateway throughput, instead of many ×0.85 steps. In the illustrative 100 to 20 Mbit/s case that is one step instead of about ten. The co-primary outcome checks that we settle no more than 1 s later than cake-autorate.”	Backup

#	Slide	On the slide	Speaker notes	10-slide cut
8	Intent with fairness	Precedence (fig04); leases; envelope	“A parent can protect a call for an hour from a phone page. That lease selects a preferential CAKE tin; it is priority, not a bandwidth guarantee. Every preferential flow stays inside a per-flow envelope, so a leased bulk download falls back to best effort. Classifier abstention never grants preference.”	Yes
9	Safe and explainable	fig08 states; checker refusal example; a receipt	“Every action passes the Action Checker, which checks admissibility against the implemented rules. Every decision is logged and explained in a sentence. Receipts report IMPROVED, NOT_IMPROVED or INCONCLUSIVE, and say association, not causation. New installs observe for 24 hours in SHADOW while static CAKE already helps.”	Yes
10	A product, not a script	fig12; packages; LuCI; household page; importer	“VeriQ ships as signed OpenWrt packages with a LuCI app, a household page, health checks with a tin self-test, a support bundle, and an importer for existing QoSmate configs. Footprint budgets are targets measured in Stage 4.”	Backup
11	How we will prove it	Arms, tiers, primary outcome, statistics	“We compare against tuned SQM, cake-autorate and QoSmate with its autorate, version-pinned, in the same lab. The primary outcome is inappropriate interventions in scenarios where cutting cannot help: count, magnitude and duration. The run is the unit; we use bootstrap CIs and Holm correction, and we publish ties and losses.”	Yes
12	Results	Ledger-generated charts only	“These are the measured results: n runs per arm, confidence intervals, environment fingerprint. Here is where we tied, and here is where we lost and why.” (Until H1 data exists, this slide shows the Day-0 table and the run plan, labelled.)	Yes
13	Value for operators and Tata Elxsi	fig22; porting path; illustrative model	“For an operator, fewer ‘slow internet’ complaints is the hypothesis a pilot would test. The illustrative model gives about ₹1.08 crore per year gross modelled avoided cost for a million homes, before integration and validation costs. The design is OpenWrt-first, with a path to prplOS and RDK-B through a partner. No endorsement is implied.”	Merge into 12/14

#	Slide	On the slide	Speaker notes	10-slide cut
14	Limits, roadmap, demo	Known limitations; P1/P2; demo QR	“What we do not claim: no bandwidth guarantees, no fix for a broken ISP link, emulated bottlenecks labelled as emulation. Next: the ML classifier, HFSC/HTB backends, the household page in Hindi, and a home pilot in SHADOW mode. Now, please pick a fault on the kit.”	Yes

For judges. Put the honesty legend (measured / target / illustrative) in the footer of every slide master. It costs nothing and answers half the credibility questions before they are asked.

33.2.1 Delivery notes

- Round 2 timing is not yet published: prepare an 8-minute talk, a 5-minute live demo (Ch. 29) and a 3-minute fallback video.
- One speaker per section, matching roles R1–R4; never read the slide.
- End on a fault the judge chooses, not on a summary slide.

33.3 The Round 1 one-slide recap

The Round 1 file exists at `submission/Round1_Case3_VeriQ.pptx`: the official template’s details slide plus one content slide (problem, idea, bounded-by-design properties, pipeline, four differentiators, proof plan, targets, stack, industry path). It predates v5 in places. Before the 24 Oct deadline (confirm on the portal), make these edits so Round 1 does not contradict Round 2:

Slide text now	Change to	Why
“added 266 ms (p95) in our Day-0 lab (emulated, 5 Mbit up)”	Keep, and add “one preliminary run, no base RTT”	Honesty rule: the Day-0 run is unreplicated
“a remote slowdown never triggers a wasteful cut”	“is designed not to cut for a remote slowdown; we measure inappropriate interventions”	Absolute claim; corrected in v4 and still untested
“Python + pyroute2 control plane”	“Go daemon (veriqd) on OpenWrt; Python for lab and ML”	C2; Python remains the fallback (O7)
“shaper = μ − kσ, change-point fast path”	“diagnosis-gated: one-step cut to 0.9 × measured throughput at L2; hold for remote/Wi-Fi/CPU causes”	C1 and the permission table; μ/σ never trigger cuts on their own
“improved / degraded / inconclusive”	“IMPROVED / NOT_IMPROVED / INCONCLUSIVE”	v5 receipt outcomes
Baselines list without QoSmate	Add “QoSmate (+autorate) head-to-head”	C12
“13 scenarios”	“13 core scenarios + 32 adverse conditions”	C12
“settle < 2 s after WAN drop” (target)	“confirmation ≤ cake-autorate + 1 s (target)”	Co-primary outcome
`<Team Name>`, `<College>`, `<Member …>` placeholders	Real details	Template fields

If space is tight, drop the stack line before any honesty label.

33.4 Panel Q&A bank (50 questions)

Each answer takes 20–40 seconds. Learn the substance, not the wording: answer, then evidence, then the limit. If the honest answer is “not measured yet”, say so and name the scenario that will measure it. Apart from the Day-0 values, every number here is a target, starting parameter or illustration.

33.4.1 Fundamentals and the datapath

Q1. CAKE already fixes bufferbloat. What did you add? In static conditions, CAKE plus a well-set rate is excellent, and VeriQ uses CAKE by default, so we expect parity there. We add what CAKE does not do: tracking a moving capacity, finding the cause of a delay rise before cutting, intent leases with an eligibility envelope, checked and logged actions, and receipts. Whether the controller beats existing autorate tools is our primary hypothesis, tested head-to-head.

Q2. Why shape at all? Doesn’t that waste bandwidth? Yes, slightly. Shaping a little below the bottleneck moves the queue from the modem’s oversized FIFO into the router, where CAKE can manage it. The cost is a few percent of throughput. The wizard sets S_base to 90% of the lowest of three measurements (a starting parameter), and upward exploration reclaims capacity only under load with low delay.

Q3. How do you shape download traffic, which arrives from the ISP? An ingress filter on the WAN runs `act_ctinfo` to copy the class bits from the conntrack mark into the packet mark, then redirects the packets to an IFB device, where CAKE shapes them as egress. So one classification on the upload side covers both directions. The exact `cpmark` syntax is a Stage-1 self-test item on each target, with a documented DSCP-restore fallback (O3).

Q4. Why carry the class in the conntrack mark instead of DSCP? The class survives DSCP washing, which the product profile does by default to protect Wi-Fi WMM from ISP-set marks. VeriQ's class marks do not leave the router. And provenance bits next to the class let the kernel enforce "higher precedence wins" without a round trip to the daemon. We use bits 24–30 only, which avoids the defaults of mwan3 and pbr; the health check verifies no other package uses them.

Q5. Do leases guarantee bandwidth to the protected device? No. A lease selects a preferential CAKE tin. CAKE tins give priority and scheduling weight, not reservations. We say "protected", never "guaranteed", and we measure the effect: each lease produces a receipt comparing the device's tin delay and drops before and during the lease.

Q6. How do you prevent starvation of bulk traffic? Two mechanisms and one measurement. CAKE's tin scheduling keeps serving lower tins, and every preferential flow is held to a per-flow eligibility envelope (starting values: RT ≤ max(1 Mbit/s, 2% of S_base) and ≤ 450 pps; IV ≤ max(8 Mbit/s, 25% of S_base)). We then measure starvation as any 10-second window with zero bulk bytes while bulk demand exists. The target is zero.

Q7. Why three queue backends? CAKE is the default and the most integrated. Some CPUs cannot run CAKE at line rate, so HTB is the lowest-CPU option and HFSC offers a hard realtime curve that some gamers prefer. All three map onto the same four VeriQ classes, so rules and leases do not change. The bench recommends a backend; switching is never autonomous.

Q8. What about Wi-Fi? Most home delay is in the Wi-Fi. VeriQ diagnoses Wi-Fi but does not control it. Station statistics from hostapd/iwinfo (retries, expected throughput) let it label a case WIFI_LIKELY, hold the WAN rate, and tell the user which device is affected and why. Airtime control needs driver support and is outside the current scope; we list it as a limitation.

33.4.2 Capacity estimation, control and diagnosis

Q9. How do you know the capacity dropped and not that a far server got slow? We look for two independent kinds of evidence. A genuine access-link drop shows far-reflector delay rising and throughput falling below the shaper rate while demand is present and our own queue is empty, or a rising near hop. A slow remote path shows far delay rising while the near hop stays flat and throughput is unaffected. The first allows a cut; the second means hold.

Q10. How fast do you react to a 100 → 20 Mbit/s drop? Our design target is a confirmation time no more than 1 s slower than tuned cake-autorate in the same runs. At evidence level L2, VeriQ cuts in one step to max(S_min, 0.90·Ĝ), where Ĝ is the median gateway throughput over the last second. Illustratively, that is one step to about 18 Mbit/s instead of about ten ×0.85 steps. It is unmeasured until the S2/A1 runs.

Q11. Why not cut faster on any delay rise, just to be safe? A cut that cannot help takes bandwidth and fixes nothing. That is our primary outcome: inappropriate interventions (count, cumulative magnitude in Mbit·s, and time below 0.95·S_ref) in scenarios where the shaper cannot help, such as remote delay or a lying reflector. Where the evidence is uncertain but bloat is present, VeriQ takes small, budgeted steps only.

Q12. What are your reflectors, and is it polite to ping them? By default, public anycast DNS resolvers from at least three operators, plus the WAN gateway as the near target; users can add their own. Probing totals 10 packets per second, dropping to 2 per second when idle. Unresponsive or rate-limiting reflectors are marked unhealthy and rotated out.

Q13. What if one reflector lies, for example by deprioritising ICMP? We take the median of healthy reflectors, never the mean, with a quorum of at least two. A reflector whose delta deviates from the others by more than 3 × MAD for more than 10 s is excluded, replaced and cooled off for 10 minutes. With five active reflectors, one liar cannot move the median. QoSmate's autorate averages RTT across reflectors (read); whether that matters in practice is our hypothesis for scenario S7.

Q14. How do you get one-way delay without synchronised clocks? ICMP timestamp replies (RFC 792) carry the reflector's receive and transmit times. Upload and download deltas each contain an unknown but constant clock offset, so we use only their changes against a per-reflector baseline; the offset cancels. When fewer than two reflectors support timestamps, we fall back to RTT and attribute direction by load.

Q15. What if the base RTT changes after a route change? The baseline is the minimum over a sliding 10-minute window of 1-second medians taken at low load. It can rise as well as fall. A lower-only baseline cannot rise, so after an upward route change it would see a permanent "delay" and keep cutting; that is condition A7 in our matrix.

Q16. Can the controller spiral down to the floor? It is designed not to. Cuts need L2 evidence; after more than three L2 cuts in an episode without improvement, the diagnosis becomes MIXED and only bounded fallback applies. Fallback steps are at most 5%, with at most 10% per rolling 10 minutes (starting parameters), and the 60 s cooldown never refunds that budget. The floor is S_min = max(1 Mbit/s, 0.10·S_base). The property tests check the envelope and the budget under random inputs.

Q17. What happens when the WAN goes down, or PPPoE reconnects? VeriQ enters HOLD_LINKDOWN: rates freeze, probes pause, and losses are not counted as congestion evidence. On link-up it re-resolves the near target, re-qualifies reflectors and resumes at the start rate from capacity memory. A delay-only controller may read the outage as extreme latency and cut (QoSmate's autorate sets 9999 ms when no reflector replies; read).

Q18. Your estimator: is μ ± σ a confidence interval? No. μ and σ are an EWMA tracking estimate and its dispersion, learned only from flagged samples: capacity samples during a confirmed drop, lower bounds when shaper-limited, nothing when traffic is application-limited. They are displayed and logged, but they never trigger a cut on their own.

Q19. Why a deterministic controller and not reinforcement learning? The controller must be auditable, bounded and replayable bit-for-bit from the decision log. A learned policy would be hard to bound, hard to explain to an operator and hard to test against every refusal code. We use ML where its uncertainty can be managed by abstention: flow classification. Diagnosis rules and thresholds are starting parameters, tuned on lab data.

33.4.3 Classification and privacy

Q20. How do you classify encrypted traffic without reading payloads? From headers and timing only: protocol, a port bucket, packet sizes and inter-arrival statistics of the first 32 packets per flow (NFLOG with a 40-byte snap length), byte ratios, DSCP and ECN. No payload, no SNI, no DNS answers read from packets. A LightGBM model decides at 8, 16 or 32 packets.

Q21. What is your classifier's accuracy? We do not have a number yet, and we will not quote one without the method. We will report coverage, selection rate, selective accuracy and abstention per class, plus shift tests. Splits are by time and by application, following the SIGCOMM 2025 lessons on leaky splits and shortcut features [R32].

Q22. What does "confidence" mean in VeriQ? We use class-conditional split conformal prediction at $\alpha = 0.05$ [R29]. For each flow the model outputs a set of plausible classes. VeriQ acts only when the set has exactly one class; empty or multi-class sets mean abstain, and the flow stays best effort. The Connections page shows the set or "abstained" for every flow.

Q23. What if the classifier is wrong? Its reach is limited by design. ML sits at precedence level 6, below user rules, leases and app marks, and it can never grant preference by abstaining. *"Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks."* A wrong RT/IV verdict is still held by the envelope, and the user can correct it with a rule.

Q24. Where is the AI, and does it beat a deterministic baseline? The ML classifier is compared with QoSmate-style heuristics and with rules only, and the "no-classifier" ablation measures its contribution to call and game outcomes. If the ablation shows no gain, we report that and keep the deterministic path; the case asks for a measurable improvement, not a conversational interface.

Q25. A device marks all its traffic EF. What happens? Each flow is checked against the envelope in the kernel. A flow that exceeds it is sanctioned to best effort for 60 s, refreshed while it keeps violating and escalated on repeat offences up to 15 minutes. Conforming small flows from the same device keep their class, and the device is not punished as a whole.

Q26. Does VeriQ send any data to the cloud? No. Control, recovery and storage are local. Flow records stay in RAM for up to 24 h and aggregates for 7 days. Nothing is exported unless the owner opts in, and the support bundle hashes addresses and MACs. Under India's DPDP Act 2023, household flow metadata may count as personal data, so any operator telemetry would need notice and consent; this is a design note, not legal advice.

Q27. Can a household member abuse leases? Leases need the household password, via an rpcd role limited to five methods. At most two leases are active, one per device, for at most 4 hours. Every request is logged, and the envelope still applies.

33.4.4 Safety and failure

Q28. What if VeriQ fails? The internet keeps working. The shaping rules live in the kernel and persist if the daemon dies. A separate watchdog checks a heartbeat every 2 s and, if it is older than 10 s, sets the static SAFE rates; the time-to-safe target is 12 s or less. procd restarts the daemon, which adopts the kernel state if it matches the last committed version. We will demonstrate this live with a kill.

Q29. And what if the evaluation fails, and VeriQ does not beat the comparators? Then we say so. Results where VeriQ ties or loses are kept and reported. Each design change has a revert condition; for example, the one-step cut (C1) is reverted if false L2 cuts in control-futile runs exceed the comparator. A parity result still leaves a useful product, but we would not claim a better controller.

Q30. Does the Action Checker guarantee safety? No. It checks admissibility against the rules it implements: observation freshness, version match, operating state, rate envelope, step size for the evidence level, budgets, lease validity and the preference basis. Each refusal code has a unit test. It does not prove an action is optimal or helpful; receipts and experiments address that.

Q31. What does SAFE assume? SAFE is static CAKE at $0.85 \times \min(\text{last-known-good } S, S_{base})$. It assumes capacity does not keep falling. If the line collapses further, SAFE can be too high and bloat can return; we state this assumption rather than hide it.

Q32. What if someone runs SQM or QoSmate at the same time? VeriQ detects conflicts (other QoS services, foreign qdiscs on the WAN, flow offloading, other users of our mark bits) and refuses to go LIVE until they are resolved. If a foreign tool changes the qdisc while VeriQ runs, it moves to DEGRADED and does not fight. Two enforcers never run at once, in the product or in the lab.

Q33. Why start in SHADOW for 24 hours? Static CAKE helps from minute one, so the household benefits immediately. Meanwhile the adaptive controller computes and logs what it would do without applying it. If the shadow report shows no crashes, no CPU over budget and few unsafe disagreements, it moves to LIVE; otherwise the user decides. Trust is earned, not assumed.

33.4.5 Evaluation and evidence

Q34. What have you measured so far? One preliminary sandbox run: behind a FIFO, added p95 delay was 266.1 ms upload and 63.6 ms download; with a short-buffer shaper, 39.6 ms and 55.4 ms. No base RTT was applied, it was IPv4 only, and CAKE was not available in that kernel. It illustrates the problem; it is not a result about VeriQ.

Q35. What is emulated? The ISP bottleneck: `tbf` with an oversized FIFO and `netem` for delay, inside Linux network namespaces, with capacity schedules and injected faults. Every number from the lab carries the label "emulated" and a fingerprint of the tools and kernel features actually applied. Real DOCSIS, PON and 5G schedulers differ; that is a stated limitation.

Q36. How do you measure what users feel, not just ping? Real encrypted WebRTC calls through a Jitsi server on the WAN side with peer-to-peer disabled, so media crosses the bottleneck; we read freezes and audio concealment from getStats [R35]. We also measure dash.js stalls, a synthetic 64 pps game flow's p99 RTT and page load times. These are evaluation-only signals; the controller never sees them.

Q37. How many runs, and which statistics? The run is the unit, not the packet. A pilot of n = 5 per arm sets n for the target CI half-width, expected to be 10–20 runs. We use bootstrap confidence intervals with 10,000 resamples, counterbalanced run order, and Holm correction for secondary outcomes [R37, R38]. We make no p99.9 claims from small samples, and missing data is reported, not imputed.

Q38. How do you stop the controller from "knowing" the test? Fault onsets are random and hidden: the controller never sees scenario names, future capacity or fault times. Ground truth sits in fields only the evaluator reads.

Q39. Isn't comparing against QoSmate unfair if you tune your own system and not theirs? Every arm is version-pinned, tuned according to its own documentation, and its configuration is archived. We run two comparisons: controller-only (same CAKE datapath, different rate controller) and whole-system (each tool in its recommended configuration). Every arm must pass the same Stage-1 self-test before its runs count.

Q40. Can a 10-second receipt prove an action helped? No, and we do not claim it does. A receipt reports association: IMPROVED, NOT_IMPROVED or INCONCLUSIVE, and any concurrent change (another action, a load change, a link event, too few samples) makes it INCONCLUSIVE. Causal claims come only from randomised, repeated lab experiments.

33.4.6 Product and deployment

Q41. Will this run on a real home router? It is designed to. The daemon is a static Go binary built in CI for five OpenWrt targets (x86-64, two ARM64 cores, ARMv7, MIPS). The footprint targets for the Standard profile are ≤ 40 MB RSS and ≤ 5% of one Cortex-A53 core; a Lite profile targets 128 MB MIPS devices. These are targets measured in Stage 4 on a Pi, a retail ARM64 router and an MT7621-class device; a Pi demo certifies only that device.

Q42. Why Go and not Python or C? One cross-compiled static binary for all targets, no Python runtime on the router (roughly 10–15 MB of flash saved, an estimate), bounded memory and no cgo. Python stays in the lab and ML training. If the team cannot ramp up in time, the controller runs in Python in the lab for P0 with the same interfaces and golden tests (O7).

Q43. How does it scale to gigabit fibre or millions of homes? Per home, CAKE's CPU cost grows with the rate; on weak CPUs the bench recommends HTB or one-directional shaping, and multi-queue CAKE is deferred. Across homes, the loop is local, so scaling is fleet management: a TR-181/TR-369 (USP) data model is on the porting path [R27].

Q44. How do users install and update it safely? Through the package manager from a signed feed: no `curl | sh`, no runtime self-download of code. Models and the app catalogue are signed packages too. Upgrades keep the config and return to SHADOW if the algorithm or model changed; rollback is the previous package.

Q45. How do you handle IPv6? Sets, rules and probes cover both families. Device rules resolve to the device's IPv4 address and all current IPv6 addresses, privacy addresses included. Scenario S9 checks that per-tin counters match intent for IPv6 traffic. Our Day-0 sandbox had no IPv6, so this is untested today.

33.4.7 Business, Tata Elxsi fit and competition

Q46. Why should an operator care? Our hypothesis is fewer "slow internet" complaints and support calls, because the gateway can tell whether a slowdown is in the home, the access link or beyond it. An illustrative model (1,000,000 homes, 1.5% contact rate per month, ₹400 per contact, 15% avoidable) gives about ₹1.08 crore per year gross modelled avoided cost. Every input is an assumption until a pilot measures it, and integration and validation costs come off the top.

Q47. How does this fit Tata Elxsi? Media & Communications is Tata Elxsi's fastest-growing vertical (+22.2% YoY, per our company research in Ch. 3), with work on RDK-B broadband gateways, an OpenWrt-to-RDK-B migration accelerator and the NEURON autonomous-network platform [R46–R49]. VeriQ is OpenWrt-first with a porting path to prplOS and RDK-B. That is a thematic fit; no integration or endorsement is implied, and porting would need a partner.

Q48. Why not just use QoSmate? For many users QoSmate is an excellent choice: it is mature, well designed and community-proven, and VeriQ re-implements most of its queue and classification features (clean-room) and imports its config. VeriQ differs deliberately in four places and adds a diagnosis-gated controller, the Action Checker, SHADOW/SAFE, receipts, leases and a household page. In static conditions we expect parity; our advantages are design claims until the head-to-head runs.

Q49. cake-autorate already handles variable links. Why another controller? cake-autorate is the reference for delay-based autorate on variable links, and it is our co-primary comparator. VeriQ's question is narrower: can diagnosis reduce cuts that do not help, such as for remote delay, a lying reflector or Wi-Fi, without reacting more slowly to genuine drops? If it cannot, the extra machinery is not justified, and we will say so.

Q50. Other teams built CAKE-based controllers too. How are you different? We do not compare ourselves with named teams. Our differences are methodological: we report what users feel (freezes and stalls), diagnose the cause before acting, abstain on uncertain flows, compare against tuned, version-pinned open-source tools with confidence intervals, and regenerate every published number from raw data.

33.5 Judging-criteria self-assessment

The case PDF lists five evaluation criteria. Readiness uses three steps: **Designed** (specified), **Built** (implemented and unit-tested), **Evidenced** (measured, in the ledger). Today every row is at most Designed; the Round 2 column is a plan. Re-score before each submission.

Case criterion	What VeriQ offers	Status (12 Oct 2026)	Evidence planned by Round 2	Main gap to close
Accuracy of classification and link estimation	Conformal abstention with honesty-kit reporting; diagnosis-gated estimator with median/quorum and windowed-min baselines	Designed	E1–E8 estimator tests; diagnosis confusion matrix vs ground truth. The classifier report moves to P1	ML ships after Round 2; say so on the slide
Reduction in interactive latency, jitter and loss	CAKE in both directions with mark plumbing; leases; envelope	Designed (Day-0 shows the problem only)	H1 core runs: added delay p95, WebRTC freezes, DASH stalls, vs SQM, cake-autorate, QoSmate	Static parity is expected; gains are expected in dynamic and adverse scenarios
Fairness and absence of starvation	Host isolation, envelope, sanctions, measured bulk share	Designed	Jain's index (A10), starvation metric, bulk share of leftover (secondary outcome)	Envelope values are starting parameters
Adaptation speed and policy stability	C1 one-step cut; refractory periods; no-spiral rule; bounded fallback	Designed	S2/A1 confirmation time vs cake-autorate + 1 s; oscillation (A2); primary outcome in S6/S7/S10/A32	The co-primary margin is the risk to watch (C1 revert rule)
CPU overhead, reproducibility, clarity of results	Go daemon, footprint budgets; evidence ledger and claims check; receipts and explain API	Designed; lab script and Day-0 raw files exist	Pi CPU/RSS measurements; every number regenerated by the claims check	Retail-router (D4) and Lite (D5) runs are P1

The case also lists expected outputs; map each to an artefact the judges can tick off:

Expected output (case PDF)	VeriQ artefact	Priority
Traffic-class and link-capacity estimator	`classify` (rules, heuristics; ML in P1) and `estimate` / `diagnose` packages	P0 (ML P1)
Dynamic QoS policy engine and enforcement	`control`, `checker`, `enforce` (cake backend), `reconcile`	P0
Dashboard (latency, jitter, loss, throughput, queue depth, fairness)	LuCI Overview, Autorate and Queues pages; Grafana for evaluation	P0 (partial)
Automated baseline-vs-optimised experiments	`lab/` scenario runner, arms, ledger	P0
API or UI for temporary intent	`lease.request` JSON-RPC, `veriqctl lease add`; household page	P0 (household page P1)
Architecture diagram (device, edge, network, data, analytics, UI)	fig02 plus the deployment views	P0
Setup guide with exact versions and verification	Lab build guide (Ch. 25) and the package install steps	P0
Scripted demo: reset, inject, collect, report	Scenario YAML + runner + report generator; judge-controlled demo	P0
Known-limitations section	Page 20 of the report; the limitations list in Ch. 1	P0

> **For judges.** "On 12 October everything was at the design stage, apart from our lab script and one preliminary run. Here is what has moved to Built and Evidenced since, and what is still only Designed." Showing the gaps convinces more than hiding them.

33.6 Final pre-submission checklist

Run this for every submission (slide, deck, report, video): one person reads, another checks.

Claims and numbers - Every number is tagged measured, target, starting parameter, assumption or illustrative. - Every measured number shows n, the environment fingerprint and its ledger path, and the claims check regenerated it within tolerance. - The Day-0 numbers appear exactly as 266.1 / 63.6 ms (FIFO) and 39.6 / 55.4 ms (short-buffer shaper), with “one preliminary run, no base RTT, IPv4 only”. - Search the text for “guarantee”, “always”, “never fails”, “proven”, “best”, “%”. Each hit is either removed or clearly correct (for example “not a bandwidth guarantee”). - The abstention sentence is quoted exactly as in Ch. 9. - The Action Checker “checks admissibility against the implemented rules”; it never “guarantees safety”. Policers are called policers, not shapers. - QoSmate facts are marked “(read)” with v0.5.48; expectations about it are marked as hypotheses. Static parity is stated. - At least one tie or loss is shown once results exist. - Drafts (QoO, RPM) are called drafts; UDPST is said to need a cooperating server.

Labels on visuals - Emulated results say “emulated”. - Illustrative figures (for example fig20) say “illustrative, not measured”. - The ISP queue is “inferred”; ground truth is shown only as evaluator data. - The 3D twin and physical kit show measured, estimated or illustrative tags.

Licences, IP and secrets - No code, text or figures copied from QoSmate (GPL-3.0), cake-autorate or any competitor repository; ideas are cited. - The third-party licence list is complete (Go modules MIT/BSD/Apache only; lab tools such as Flent and irtt documented as external tools). - The VeriQ licence is stated as “Apache-2.0 proposed; Tata Elxsi decides” (O5). - **IP:** per the rules summarised in docs/01, submitted ideas become Tata Elxsi’s property. Re-read the official rules, keep the repository private unless the rules allow publishing, and share access with evaluators only. - No credentials, keys or tokens in the repository or the slides; example secrets are clearly fictitious. - No implied Tata Elxsi endorsement or integration anywhere, including the business slide.

Format and logistics - Round 1 slide edits from the table above are done, and the team placeholders are filled. - File formats, page and slide limits, and deadlines are confirmed on the portal (Round 1: 24 Oct 2026, to confirm). - Company figures are re-checked after Tata Elxsi’s Q2 FY27 results (scheduled for 13 Oct 2026 per docs/01). - The PDF opens on a second machine; fonts are embedded; figures are readable in greyscale. - The demo video has a backup copy offline, and the live demo has its fallback plan (Ch. 29).

Part A–E · Appendices

Glossary, references, the authoritative specification, contracts and change records

Appendix A · Glossary

Terms, acronyms, symbols and identifiers used in this handbook, following the v5 design (docs/11). Numbers are starting parameters or targets unless stated otherwise; formal sources are cited as [R#].

Symbols and VeriQ identifiers

Several identifier families share letters; name the family on first use (for example “adverse condition A6”, “heuristic H3”, “tier H1”).

Term	Definition
A1–A32	The 32 adverse conditions of the v5 matrix, each with a detection, a bounded response, a test and a target pass criterion.
C1–C14	The v5 changes from v4.1 (docs/12); for example C1 is the one-step L2 cut.
D	The delay signal per direction: the median over healthy reflectors of their latest delta against baseline (D_up, D_down, or D_rtt).
D1–D5	Deployment views: D1 Ubuntu netns lab, D2 OpenWrt VM, D3 Raspberry Pi reference router, D4 retail ARM64 router, D5 low-end MIPS device (Lite profile).
E1–E8	Estimator accuracy tests (proposed), such as step-down tracking and reflector-fault robustness.
G_gw	Gateway throughput: bytes dequeued by the shaper per second, overhead-compensated.
Ĝ (G-hat)	The median of G_gw over the last 1 s (four ticks); the input to the C1 cut target.
H0–H4	Evaluation tiers: H0 smoke, H1 head-to-head core, H2 classification and intent, H3 robustness, H4 hardware.
H1–H5 (heuristics)	Kernel heuristics that may only demote or reset, for example H1 sustained bulk TCP; H3 is off on CAKE.
L0–L3	Diagnosis evidence levels: L0 no usable signal, L1 one source, L2 two independent sources, L3 L2 plus an agreeing active capacity test.
load	G_gw / S over the last second, per direction.
μ, σ	An EWMA tracking estimate of capacity and its dispersion; not a confidence interval, and never a cut trigger on its own.
O1–O10	Open decisions, each with a default until data decides.
P0, P1, P2	Delivery priorities: P0 by Round 2 (13 Dec 2026), P1 for the finale, P2 the post-competition roadmap.
P1–P9 (principles)	The nine design principles, from “the packet path is the kernel” (P1) to “every published number regenerates” (P9).
Q_own	Own-queue delay from CAKE per-tin delay or backlog / S.
R1–R3	The v4.1 core refinements: Action Checker and reconciler, mark plumbing, evidence-graded diagnosis.
R1–R4 (roles)	Team roles: R1 datapath and OpenWrt, R2 controller, R3 evaluation, R4 UX, ML and demo.
S	The configured shaper rate per direction (S_up, S_down), read back from tc.
S_base, S_min, S_max	The rate envelope: S_base is the configured nominal rate; S_min = max(1 Mbit/s, 0.10·S_base); S_max = 1.05·S_base.
S_ref	The shaper rate just before a disturbance; the reference for measuring inappropriate interventions.
S1–S13	The core scenarios, for example S2 a 100 → 20 Mbit/s drop and S6 remote-path delay.
Stage 1–4	The evaluation ladder: capability and smoke; core repeated comparisons; blind faults and recovery; hardware and resources.
θ_bb	The bufferbloat detector threshold on D (starting value 15 ms).

Terms A–Z

Term	Definition
ACK filter	A CAKE option that drops redundant pure TCP ACKs in the queue, useful on very asymmetric links; VeriQ enables it automatically when the download/upload ratio is ≥ 15.
Action Checker	VeriQ's pure-function gate before enforcement; it checks admissibility against the implemented rules and returns ACCEPT or a refusal code.
act_ctinfo	A tc action that copies conntrack information (a DSCP value or the conntrack mark) onto packets; VeriQ uses its `cpmark` mode on WAN ingress [R7].
Added delay	Latency under load minus idle latency; VeriQ reports its p50/p95/p99 per direction.
Adoption	On restart, the reconciler resumes using the kernel state if it matches the last committed version, instead of reinstalling it.
AQM	Active Queue Management: dropping or ECN-marking packets early to keep queues short, as CoDel does [R2].
ATM cell tax	The overhead of carrying packets in 53-byte ATM cells on ADSL; ignoring it makes a shaper overestimate capacity.
Backend	The tc scheduler VeriQ renders: `cake` (default), `hfsc` or `htb`, all mapped to the same four classes.
BACKOFF	Rate-controller state after delay rises while probing up: last good rate × 0.97, then hold 30 s.
BBR	Bottleneck Bandwidth and Round-trip propagation time: a TCP congestion control that models the path instead of reacting only to loss [R9].
Bench	Measures each backend's throughput and CPU on the router to recommend one.
Bootstrap CI	A confidence interval from resampling run-level results (10,000 resamples in VeriQ's plan) [R37].
Bufferbloat	Excess delay caused by oversized buffers that fill under load, so interactive packets wait behind bulk data [R1].
CAKE	Common Applications Kept Enhanced: a Linux qdisc combining a shaper, flow queueing, COBALT AQM, host isolation and diffserv tins [R5, R6].
Capacity memory	A 168-bin hour-of-week record of capacity samples, used only to choose a start rate after boot or link-up.
Censoring	When the shaper limits throughput, only "capacity ≥ S" is learned, not the true capacity; application-limited periods teach nothing.
Claims check	A CI script that recomputes every published number from the evidence ledger and fails if one does not match.
Clean-room	Building VeriQ from documented ideas without opening or copying GPL source while writing the corresponding module.
CoDel	Controlled Delay: an AQM that drops when a packet's sojourn time stays above a target for an interval [R2, R3].
Conformal prediction	A distribution-free method that turns model scores into prediction sets with a target coverage level [R29, R30].
Confirmation time	The time after an injected event until S stays within ±10% of its final value for 5 s; the co-primary outcome.
Conntrack	The Linux connection-tracking subsystem that records each flow's state, counters and a 32-bit mark.
Coverage	The fraction of prediction sets that contain the true label; the target is 1 − α on exchangeable data.
cpmark	The act_ctinfo mode that copies (masked) conntrack mark bits into the packet mark.
CPU_LIMITED	Diagnosis label for a router whose shaping CPU is saturated; VeriQ holds instead of cutting.
ct mark	The conntrack mark; VeriQ stores CLASS in bits 24–26, PROV in 27–29 and SANCT in bit 30.
DASH	Dynamic Adaptive Streaming over HTTP; evaluated with the dash.js player for stalls [R43].
Decision log	A JSONL record of every tick's inputs, diagnosis, action and verdict, kept in a RAM ring; enables bit-for-bit replay.

Term	Definition
DEGRADED	Operating state after a foreign change, prolonged quorum loss or a failing component: keep the last good config and alert.
Diffserv4	CAKE's four-tin mode: Bulk, Best Effort, Video, Voice.
DISABLED	Operating state chosen by the user; VeriQ removes its objects.
Dominance invariant	VeriQ's design goal: where it lacks evidence it holds or reverts to static CAKE equivalent to well-tuned SQM, so its worst case is static SQM.
DoH / DoT / ECH	DNS over HTTPS, DNS over TLS and Encrypted Client Hello; they bypass the router's resolver, so domain sets stop working.
DPDP Act	India's Digital Personal Data Protection Act, 2023 [R45].
DRIFT	Rate-controller state with no confirming evidence for 120 s: move toward S_base by at most 1% per 10 s.
DRR++	The deficit round-robin scheduler with sparse-flow priority used inside fq_codel and CAKE.
DSCP	Differentiated Services Code Point: six bits in the IP header that request a per-hop behaviour [R17].
dual-srchost / dual-dsthost	CAKE host-isolation modes giving fairness first between hosts, then between their flows.
ECN	Explicit Congestion Notification: routers mark packets CE instead of dropping them [R19].
Envelope (eligibility)	The per-flow rate and packet-rate limits a flow must respect to keep RT or IV treatment; violations are sanctioned.
Evidence ledger	The store of raw run files and manifests from which every result is regenerated.
Explain API	`veriqctl explain` methods that render a decision or a flow's classification as template sentences, without an LLM.
Fallback episode	Bounded cuts under uncertain evidence: ≤ 5% steps, ≤ 10% per rolling 10 min; the cooldown never refunds the budget.
fq_codel	Flow-queue CoDel: per-flow queues scheduled by DRR with CoDel on each [R4].
FTTH / PON	Fibre to the home over a passive optical network shared by several homes.
fw4	OpenWrt's nftables-based firewall.
fwmark	The packet mark; CAKE's `fwmark` option selects a tin from masked mark bits.
FWA	Fixed Wireless Access: home broadband over a 4G/5G cellular link, with variable capacity.
Go	The language of veriqd, compiled as a static binary without cgo [R43].
HFSC	Hierarchical Fair Service Curve: a scheduler with separate real-time and link-sharing curves [R8].
HOLD	Rate-controller state that keeps S unchanged.
HOLD_LINKDOWN	Operating state while the WAN is down: rates frozen, probes paused, losses not used as evidence.
Honesty kit	Classifier reporting rules: time and app splits, no shortcut features, per-class metrics, shift tests [R32].
HTB	Hierarchy Token Bucket: a class-based shaper with rates and ceilings; VeriQ's lowest-CPU backend.
ICMP timestamp	An ICMP message carrying originate, receive and transmit times, used for one-way delay deltas [R16].
IFB	Intermediate Functional Block: a virtual device that lets ingress traffic be shaped as egress.
Inappropriate intervention	A rate reduction where cutting cannot help; the primary outcome (count, Mbit·s, duration).
INCONCLUSIVE	Receipt outcome when a concurrent change or too few samples prevents attribution.
INIT	Starting operating state; static CAKE at the start rate while the reconciler adopts or installs.
IV / RT / BE / BK	VeriQ service classes: interactive video, realtime, best effort, bulk.
Jain's index	A fairness index from 1/n (one user gets all) to 1 (equal shares) [R36].

Term	Definition
L4S	Low Latency, Low Loss, Scalable throughput: an architecture using ECT(1) and a dual-queue AQM [R20].
Lease	An authenticated, expiring grant of RT or IV treatment for a device; enforced as a kernel set element with a timeout.
LightGBM	A gradient-boosted decision tree library; VeriQ trains with it off-box and evaluates the trees in Go [R31].
LIVE	Normal operating state: checked actions are applied.
LuCI	OpenWrt's web interface, with client-side JavaScript views [R41].
MAD	Median absolute deviation; used to detect an outlier reflector (> 3 × MAD for > 10 s).
MCV	Media & Communications: the Tata Elxsi vertical that owns Case 3.
Mondrian conformal	Class-conditional conformal prediction, calibrated separately per class.
MPU	Minimum packet unit: the smallest size a shaper charges for a packet.
MSS clamp	Rewriting the TCP maximum segment size in SYN packets; VeriQ clamps when a direction is below 3000 kbit/s.
NAT awareness	CAKE's `nat` option, which looks up conntrack to find internal hosts for fairness.
netem / tbf	tc qdiscs used in the lab: netem adds delay or loss; tbf emulates the ISP line rate and buffer.
netns	Linux network namespace; the lab builds clients, router, ISP and servers as namespaces.
NEURON	Tata Elxsi's autonomous-network platform [R46]; a thematic fit only.
NFLOG	A netfilter target that sends packet headers to user space; VeriQ uses a 40-byte snap length.
nftables	The Linux packet-classification framework; VeriQ owns table `inet veriq` [R42].
NOT_IMPROVED	Receipt outcome when delay did not fall after an action.
NQB	Non-Queue-Building per-hop behaviour (DSCP 45) for flows that do not build queues [R21]; VeriQ maps it to IV.
OpenWrt	A Linux distribution for routers; VeriQ's primary product platform [R40].
OWD	One-way delay; VeriQ uses only changes in one-way deltas, because clock offsets cancel.
Overhead	Per-packet link-layer bytes the shaper adds to its accounting (for example 38 for Ethernet).
Policer	A rate limiter that drops excess traffic without queueing it; not a shaper.
PPPoE	Point-to-Point Protocol over Ethernet, common on DSL and some fibre; adds 8 bytes.
prplOS	The prpl Foundation's carrier-grade OpenWrt-based distribution [R44].
PROBE_FAULT	Diagnosis label for lost quorum or an outlier reflector: hold and rotate.
PROBE_UP	Rate-controller state that raises S by ×1.04 per second under load with low delay.
procd	OpenWrt's init and process supervisor; it respawns veriqd [R40].
Provenance (PROV)	Three mark bits recording why a flow has its class (default, heuristic, ML, app mark, user rule, lease, system).
qdisc	A Linux queueing discipline attached to an interface.
QoO	Quality of Outcome: an IETF draft framework for scoring network quality against application needs [R24].
QoSmate	A mature OpenWrt QoS tool with HFSC/CAKE/HTB and optional autorate; VeriQ's head-to-head comparator [R13].
Quorum	At least two healthy reflectors; otherwise the delay signal is missing.
Receipt	An action's before/after record and outcome; association, not causation.
Reconciler	Compares desired with observed kernel state every tick; self-heals, adopts, or goes DEGRADED.
Reflector	A remote host that answers probes; near (first hop) or far (internet).
Refractory period	Minimum time between rate changes (1 s after a cut, 1 s between raises).

Term	Definition
REMOTE_LIKELY	Diagnosis label for delay beyond the ISP's first hop; VeriQ holds.
RDK-B	Reference Design Kit for Broadband, an operator gateway software stack [R44].
RPM	Round-trips per minute: the responsiveness metric of an IETF draft [R23].
rpcd	OpenWrt's RPC daemon, which exposes methods over ubus with ACLs.
SAFE	Operating state on watchdog or fatal error: static CAKE at 0.85 × min(last-known-good S, S_base).
Sanction	Demotion of an envelope-violating flow to BE for 60 s, escalating on repeat offences.
SBOM	Software bill of materials, published per release in SPDX format.
SHADOW	Operating state that logs but does not apply rate or ML actions; 24 h by default.
Singleton	A conformal prediction set with exactly one class; the only case where VeriQ acts on ML.
SLoPS	Self-Loading Periodic Streams: an available-bandwidth probing method [R28].
Sparse-flow boost	CAKE and fq_codel give new or low-rate flows priority, which already helps interactive traffic.
SQM	Smart Queue Management: OpenWrt's sqm-scripts, applying a static shaper with fq_codel or CAKE [R11].
Starvation	Any 10 s window with zero bulk bytes while bulk demand exists.
SUS	System Usability Scale, a 10-item usability questionnaire [R39].
Tin	A CAKE priority band with its own threshold and weight; not a bandwidth reservation.
Tin self-test	Injecting marked probes of each class and checking per-tin counters in both directions.
TR-181	The Broadband Forum Device:2 data model for gateways [R27].
TR-369 / USP	The Broadband Forum User Services Platform for managing devices at scale [R27].
UCI	Unified Configuration Interface: OpenWrt's configuration system under `/etc/config` [R40].
UDPST	UDP Speed Test protocol (RFC 9946, TR-471) for IP capacity; needs a cooperating server [R22].
UNKNOWN	Diagnosis label when no rule matches; also used for flows the classifier abstains on.
veriqd / veriqctl	The VeriQ daemon and its command-line client over a local Unix socket.
Wash	Clearing DSCP bits; VeriQ's product profile washes both directions after classification.
Watchdog	`veriq-watchdog`, a procd shell instance that applies SAFE rates if the heartbeat is older than 10 s.
WAN_LIKELY	Diagnosis label for an access-link capacity drop; at L2 it permits the one-step cut.
WebRTC getStats	The browser API that reports freezes, concealed audio samples and other call statistics [R35].
WIFI_LIKELY	Diagnosis label when a client's Wi-Fi link is the bottleneck; VeriQ holds and advises.
WMM	Wi-Fi Multimedia: four 802.11 access categories that DSCP can map onto [R18].

Appendix B · References

Queue management and congestion - R1. J. Gettys, K. Nichols, “Bufferbloat: Dark Buffers in the Internet”, ACM Queue 9(11), 2011. - R2. K. Nichols, V. Jacobson, “Controlling Queue Delay”, ACM Queue 10(5), 2012. - R3. RFC 8289, “Controlled Delay Active Queue Management” (CoDel), 2018. - R4. RFC 8290, “The Flow Queue CoDel Packet Scheduler and Active Queue Management Algorithm”, 2018. - R5. T. Høiland-Jørgensen, D. Täht, J. Morton, “Piece of CAKE: A Comprehensive Queue Management Solution for Home Gateways”, IEEE LANMAN 2018 (arXiv:1804.07617). - R6. tc-cake(8) manual page (iproute2): https://manpages.debian.org/unstable/iproute2/tc-cake.8.en.html - R7. tc-hfsc(8), tc-htb(8), tc-fq_codel(8), tc-ctinfo(8), tc-mirred(8) manual pages (iproute2). - R8. I. Stoica, H. Zhang, T. S. E. Ng, “A Hierarchical Fair Service Curve Algorithm for Link-Sharing, Real-Time and Priority Services”, ACM SIGCOMM 1997. - R9. N. Cardwell et al., “BBR: Congestion-Based Congestion Control”, ACM Queue 14(5), 2016. - R10. Bufferbloat project, “What can I do about Bufferbloat?”: https://www.bufferbloat.net/projects/bloat/wiki/What_can_I_do_about_Bufferbloat/

Tools and comparators - R11. sqm-scripts: https://github.com/tohojo/sqm-scripts and OpenWrt SQM guide: https://openwrt.org/docs/guide-user/network/traffic-shaping/sqm-details - R12. cake-autorate: https://github.com/lynxthecat/cake-autorate - R13. QoSmate (v0.5.48 @ c507843): https://github.com/hudra0/qosmate and luci-app-qosmate (v1.0.14): https://github.com/hudra0/luci-app-qosmate - R14. qosify (OpenWrt package for DSCP-based classification with CAKE), OpenWrt source tree. - R15. Waveform bufferbloat test: https://www.waveform.com/tools/bufferbloat

Standards and drafts - R16. RFC 792, Internet Control Message Protocol (incl. Timestamp/Timestamp Reply), 1981. - R17. RFC 2474, Definition of the Differentiated Services Field (DS Field), 1998; RFC 4594, Configuration Guidelines for DiffServ Service Classes, 2006. - R18. RFC 8325, Mapping Diffserv to IEEE 802.11, 2018. - R19. RFC 3168, The Addition of Explicit Congestion Notification (ECN) to IP, 2001. - R20. RFC 9330, 9331, 9332: Low Latency, Low Loss, and Scalable Throughput (L4S) architecture, ECN protocol, DualQ Coupled AQM, 2023. - R21. RFC 9956, A Non-Queue-Building Per-Hop Behavior (NQB PHB), May 2026: https://www.rfc-editor.org/rfc/rfc9956.html - R22. RFC 9946, UDP Speed Test Protocol for IP capacity (UDPST), Apr 2026: https://www.rfc-editor.org/rfc/rfc9946.html; Broadband Forum TR-471. - R23. draft-ietf-ippm-responsiveness-09 (RPM): https://www.ietf.org/archive/id/draft-ietf-ippm-responsiveness-09.html - R24. draft-ietf-ippm-qoo-11, Quality of Outcome: https://datatracker.ietf.org/doc/draft-ietf-ippm-qoo - R25. draft-livingood-low-latency-deployment-16: https://www.ietf.org/ietf-ftp/internet-drafts/draft-livingood-low-latency-deployment-16.html - R26. ITU-T G.107, The E-model: https://www.itu.int/ITU-T/recommendations/rec.aspx?rec=12505 - R27. Broadband Forum TR-181 (Device:2 data model) and TR-369 (User Services Platform, USP): https://usp.bbf.org/

Measurement, ML and statistics - R28. M. Jain, C. Dovrolis, “Pathload: a measurement tool for end-to-end available bandwidth”, PAM 2002 (SLoPS). - R29. A. N. Angelopoulos, S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification”, arXiv:2107.07511. - R30. V. Vovk, A. Gammerman, G. Shafer, Algorithmic Learning in a Random World, Springer, 2005. - R31. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”, NeurIPS 2017. - R32. Zhao et al., “The Sweet Danger of Sugar”, ACM SIGCOMM 2025: https://arxiv.org/abs/2507.16438 - R33. Sharma et al., “Estimating WebRTC Video QoE Metrics Without Using Application Headers”, ACM IMC 2023: https://arxiv.org/abs/2306.01194 - R34. CESNET DataZoo datasets: https://cesnet.github.io/cesnet-datazoo/datasets_overview/ - R35. W3C, Identifiers for WebRTC’s Statistics API: https://www.w3.org/TR/webrtc-stats/ ; MDN RTCInboundRtpStreamStats: https://developer.mozilla.org/en-US/docs/Web/API/RTCInboundRtpStreamStats - R36. R. Jain, D. Chiu, W. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems”, DEC-TR-301, 1984. - R37. B. Efron, R. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993. - R38. S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure”, Scandinavian Journal of Statistics 6(2), 1979. - R39. J. Brooke, “SUS: A ‘quick and dirty’ usability scale”, 1996.

Platforms and tooling - R40. OpenWrt documentation: procd init scripts https://openwrt.org/docs/guide-developer/procd-init-scripts ; UCI https://openwrt.org/docs/guide-user/base-system/uci ; SDK https://openwrt.org/docs/guide-developer/toolchain/using_the_sdk ; dnsmasq IP sets https://openwrt.org/docs/guide-user/base-system/dhcp#ip_sets - R41. LuCI (OpenWrt web interface), client-side JavaScript API: https://github.com/openwrt/luci - R42. nftables wiki: https://wiki.nftables.org - R43. The Go programming language: https://go.dev ; three.js: https://threejs.org ; uPlot: https://github.com/leeoniya/uPlot ; Mermaid: https://mermaid.js.org ; dash.js: https://github.com/Dash-Industry-Forum/dash.js ; Jitsi handbook: https://jitsi.github.io/handbook/ - R44. prpl Foundation (prplOS): https://prplfoundation.org/ ; RDK (RDK-B): https://rdkcentral.com/ - R45. Government of India, The Digital Personal Data Protection Act, 2023 (MeitY).

Tata Elxsi and TELIPORT (from docs/01; use only as docs/01 uses them) - R46. Tata Elxsi NEURON autonomous network platform: https://www.tataelxsi.com/platforms/neuron-autonomous-network-platform - R47. Tata Elxsi Q1 FY27 press release: https://www.tataelxsi.com/news-and-events/press-releases/tata-elxsi-delivers-steady-growth-at-28-qoq-and-145-yoy-in-q1-fy27 - R48. Tata Elxsi investor presentations (NSE filings): https://nsearchives.nseindia.com/corporate/TELNSE_14072026183436_SEIntimationPresentation.pdf - R49. Tata Elxsi RDK-B launch coverage: https://www.angelone.in/news/share-market/tata-elxsi-launches-rdkb - R50. TELIPORT Season 4 listings: https://gradpartners.in/e/Tata-Elxsi-TELIPORT-Season-4-347 ; https://unstop.com/hackathons/tata-elxsi-teliport-season-4-gl-bajaj-institute-of-management-greater-noida-1767967/amp

Project evidence - R51. VeriQ Day-0 sandbox evidence: repository `evidence/day0_sandbox_2026-10-11/` (preliminary, one run).

Appendix C · Authoritative specification (docs/11, VeriQ v5)

Prepared 12 Oct 2026. ***This document supersedes docs/09 (v4.1) as the single authoritative specification.*** *Every v4.1 controller contract (Appendix A of docs/09: checker, rate floor, exploration, ownership, marks, settling, coverage) remains in force unless this document changes it. Each change is listed in §0.3 and in docs/12. Every number is a* ***starting parameter*** *to be tuned in Stage 2 and recorded in the run manifest; none is a validated limit. Statements about other tools are labelled* ***read*** *(from their source code or docs at a pinned commit) or* ***hypothesis*** *(expected behaviour, to be tested). This document is meant to be implemented as written: open questions that genuinely remain are listed in §20, each with the default to use until data decides.*

Inputs. - docs/01–10 and the official Case 3 brief. - QoSmate backend v0.5.48 @ `c507843` (28 Sep 2026) and luci-app-qosmate v1.0.14 @ `11c6e64` (26 Sep 2026). Both were read in full: README, default config, `qosmate.sh`, `init.d/qosmate`, `qosmate-autorate.sh` and the LuCI views. Neither was run. - The Day-0 sandbox evidence (`evidence/day0_sandbox_2026-10-11/`).

0. Decision summary

0.1 What VeriQ v5 is

VeriQ is an OpenWrt-native, adaptive home-QoS system with three layers: 1. **QoSmate-class queue management, classification and UI.** Every feature a QoSmate user relies on has an equivalent, and existing QoSmate configs can be imported. 2. **A diagnosis-informed, bounded, auditable rate controller.** This is the v4.1 core: it finds where the delay comes from before it cuts. 3. **Product-grade operation.** This covers signed packages, fail-safe behaviour, self-tests, explainability, household UX and footprint budgets.

One-line pitch: *"Everything QoSmate does, plus a controller that knows when not to cut, proves what it did, and never leaves the household worse off than plain SQM."*

0.2 The design invariant: "never worse than static SQM"

"Perfect results under any adverse condition" is not a promise any network system can keep: if the ISP link fails, nothing in the home can fix it. VeriQ makes a stronger and *testable* promise instead:

> **Dominance invariant (design goal, tested in every scenario):** in every condition where VeriQ lacks evidence to act, it holds or reverts to a static CAKE configuration equivalent to well-tuned SQM. Its adaptive behaviour is only allowed where evidence supports it, through bounded steps. So by construction, VeriQ's worst case is "static SQM", and its best case is better.

The adverse-conditions matrix (§10) lists 32 conditions. For each it gives the detection, the bounded response, the scenario that tests it, and what a delay-only autorate is expected to do (**hypothesis**).

0.3 Changes from v4.1 (complete list)

#	Change	Type	Why
C1	**Fast-cut contract at L2:** target `S_new = max(S_min, 0.90·Ĝ)` in **one step**, where Ĝ is the median gateway throughput over the last 1 s of the episode. The ×0.75 cap applies only when Ĝ is unavailable.	**Correction**	The v4.1 cap of ×0.75 per step needs ~6 steps for a 100 → 20 Mbit/s drop, so it would fail the co-primary non-inferiority margin (confirmation ≤ cake-autorate + 1 s). Discovered in the v5 audit.
C2	**On-router daemon in Go** (`veriqd`, static binary). Python remains for the harness, evaluation and ML training.	Decision	OpenWrt product readiness: one cross-compiled binary for all targets, no Python runtime on the router (≈ 10–15 MB flash saved), bounded memory.
C3	**OpenWrt product layer** (packages, UCI, procd, LuCI app, rpcd ACLs, hotplug, sysupgrade, signed feed, CI matrix).	Scope (user-directed)	The user requires product readiness that beats QoSmate.
C4	**Queue backends:** `cake` (default), `hfsc` (realtime class, low-CPU option), `htb` (lowest CPU). All map onto the same four VeriQ service classes. HTB moves from post-core into the product layer; `hybrid` is deferred (§20).	Scope	Parity with QoSmate's scheduler choice; weak CPUs cannot always run CAKE at line rate.

#	Change	Type	Why
C5	**Classification engine:** QoSmate-compatible rules, static and dynamic IP sets, domain sets, IPv6 suffix match, inline/custom nft, per-device policers, MSS clamp and ACK handling, plus app catalogue, leases, envelope sanctions and the ML classifier. Per-device rate limits move from "deliberately not core" into the product layer.	Scope	Parity plus the VeriQ additions.
C6	**Product DSCP default:** wash ingress ON, wash egress ON (as QoSmate). The **lab profile** keeps DSCP untouched for controller-only comparisons. Classification is unaffected because VeriQ carries class in conntrack mark bits, not in DSCP.	Change of default	Protects Wi-Fi WMM from ISP-set DSCP; avoids leaking marks upstream.
C7	**Heuristics may only demote or reset; preference always needs an explicit basis** (lease, user rule, eligible app mark, or ML singleton).	Clarification	Makes the "abstention never grants preference" rule apply to heuristics too.
C8	**Baseline estimator:** a per-reflector windowed minimum (10 min) of 1 s medians, taken from low-load samples. It replaces the v4.1 wording "slow-up, fast-down" with an exact rule.	Precision	Handles upward route changes, a known failure mode of lower-only baselines.
C9	**Direction attribution:** use ICMP-timestamp one-way deltas where reflectors support them; otherwise attribute a rise by load (cut only directions with load ≥ 60%).	Precision	Avoids collateral cuts of an idle direction.
C10	**Operating states extended:** `INIT, SHADOW, LIVE, DEGRADED, HOLD_LINKDOWN, SAFE, DISABLED`, with a product rule: static CAKE is active from minute one, and the adaptive controller runs in SHADOW for 24 h (default) before LIVE.	Product	Users get SQM benefit at once; adaptive features earn trust.
C11	**Watchdog on OpenWrt** is a separate procd instance (`veriq-watchdog`, POSIX shell), not systemd.	Platform	OpenWrt uses procd.
C12	**Evaluation adds QoSmate head-to-head** (whole-system and controller-only), plus 32 adverse conditions as scenarios A1–A32.	Scope	The user's goal is to beat QoSmate measurably.
C13	**Diagnosis label CPU_LIMITED** (§7.2)	Precision	CPU-bound throughput must not read as a capacity drop (A17)
C14	**Mark layout** CLASS 24–26, PROV 27–29, SANCT 30 (§4.2)	Precision	Kernel-side precedence; coexists with mwan3/pbr mark bits

Unchanged and locked: the kernel datapath, with CAKE as the default core; the three-speed design; R1 checker and reconciler; R2 mark plumbing; R3 evidence levels L0–L3 with permission table and missing-near rules; the 7-level precedence; the single UNKNOWN rule; leases and admission; the eligibility envelope; the rolling fallback budget; receipts; shadow mode; the primary outcome (inappropriate interventions in control-futile scenarios) and its margins; Appendix A of docs/09 except where C1 and C8 sharpen it.

1. Problem statement (short; full version in the handbook)

Home broadband is shared by video calls, gaming, streaming, downloads, backups and updates. When any bulk flow fills an oversized buffer at the modem or ISP, every interactive flow waits behind it. This is **bufferbloat**: hundreds of milliseconds of extra delay at full throughput. The Day-0 sandbox run measured 266 ms of added upload delay at p95 behind a FIFO; that is one preliminary run, not a replicated result.

The established fix is to shape just below the bottleneck rate and use a smart AQM/scheduler (CAKE) so that the queue lives in the router, where it is managed. Three problems remain, and they are exactly where existing tools differ:

1. **Capacity moves.** LTE/5G FWA, DOCSIS at peak hours, shared PON and Wi-Fi backhaul all vary. A static rate is too high (bloat returns) or too low (bandwidth wasted).

2. **Delay can come from somewhere the router cannot fix:** a far server, a lying reflector, a weak Wi-Fi client, an overloaded router CPU. Delay-triggered autorate controllers cut the rate anyway. That wastes bandwidth and does not help.
3. **Households need intent, not knobs:** “protect my exam call for an hour”. They also need honest feedback on whether it helped.

Case 3 requirements: - classify traffic without reading payload, expose confidence and allow correction; - estimate capacity; - manage queues in both directions, for IPv4 and IPv6; - verify the user experience; - support temporary intent with fairness; - support rollback and avoid starvation; - keep remediation bounded and observable; - compare an AI approach with a deterministic baseline; - provide a dashboard and reproducible experiments.

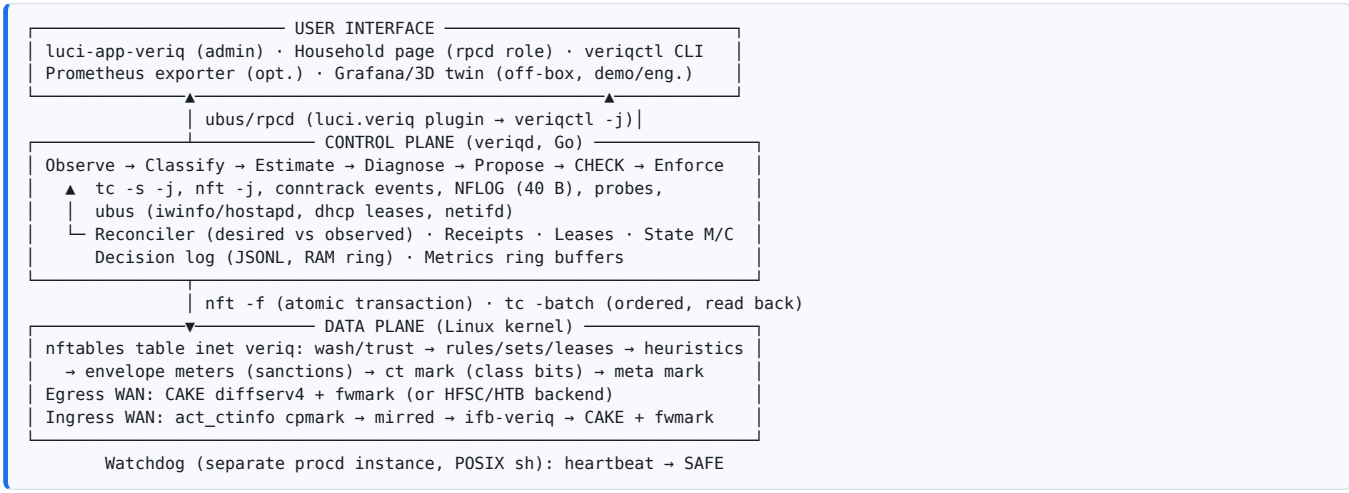
The full traceability matrix is in §19.

2. Principles (each one is enforced somewhere in the design)

#	Principle	Enforced by
P1	The packet path is the kernel: no user-space process touches packets.	Datapath (§4)
P2	Never worse than static SQM (dominance invariant).	Operating states, SAFE, evidence permissions
P3	Find the cause before acting; cut only where a cut can help.	Diagnosis L0–L3 (§7)
P4	Every action is checked, bounded, logged and reversible.	Action Checker, budgets, receipts (§8)
P5	Preference needs an explicit basis; uncertainty never grants priority.	Precedence + MARK_SET check (§5.8)
P6	Privacy by construction: headers and timing only; no payload; no cloud dependency.	Observer design, NFLOG snaplen 40 (§5.6)
P7	Honest semantics: CAKE tin preference is not a bandwidth guarantee; policers are not shapers.	UI wording, docs
P8	Product-safe: the daemon crashing never breaks the internet.	Kernel config persists; watchdog; procd (§12)
P9	Every published number regenerates from raw data.	Evidence ledger, claims check (§15)

3. Architecture

3.1 Layers and planes



3.2 Three speeds

Speed	Period	What	Where
Guardrails	per packet (kernel)	CAKE AQM, flow isolation, tin scheduling, envelope meters, policers	Kernel
Control	250 ms tick; probes at 10 pps total	Estimation, diagnosis, rate control, checker, reconcile	veriqd

Speed	Period	What	Where
Assessment	15–60 s windows	Receipts (IMPROVED/NOT_IMPROVED/INCONCLUSIVE), fallback undo, shadow disagreements	veriqd
Experiment	runs of 2–5 min, repeated	Causal claims	Harness (off-box)

3.3 Deployment views

View	Hardware	Purpose
D1 Lab	Ubuntu 24.04 ×86-64, network namespaces	Development, CI integration tests, Stage 2–3 evaluation
D2 OpenWrt VM	OpenWrt 24.10 ×86-64 in QEMU/KVM (netns or bridged)	Package and LuCI tests, CI
D3 Reference router	Raspberry Pi 4/5 + USB 3 GbE adapter, OpenWrt 24.10 (bcm27xx)	Demo kit, Stage 4 resources
D4 Retail router	One ARM64 OpenWrt router with ≥ 512 MB RAM (e.g. Filogic 830 class)	Product credibility, Stage 4
D5 Low-end check	One MT7621-class (MIPS, 128–256 MB RAM) device, *Lite* profile	Footprint floor; proves the Lite profile works

3.4 Process model on OpenWrt

Process	Started by	Privilege	Purpose
`veriqd`	procd (`/etc/init.d/veriq`), respawn 3600 5 5	root (needs CAP_NET_ADMIN, CAP_NET_RAW); drops other capabilities via procd jail where available	Control plane
`veriq-watchdog`	procd, second instance	root	Heartbeat check → SAFE
`rpcd` plugin `luci.veriq`	rpcd	per ACL	UI bridge (calls `veriqctl -j`)
`veriqctl`	user/CLI/rpcd	root via rpcd	Commands, JSON

The local API is a Unix socket `/var/run/veriq.sock` (mode 0600, root), carrying line-delimited JSON-RPC 2.0. veriqd provides no network listener of its own; the optional exporter listens on the LAN only (default port 9567).

4. Data plane: queue management

4.1 Service classes (one contract, three backends)

VeriQ class	Meaning	CAKE diffserv4 tin (fwmark value)	HFSC class	HTB class
RT	Realtime: voice, game state, call audio	Voice (4)	`1:11` rt curve (realtime)	`1:11` prio 0
IV	Interactive video: call video, cloud-gaming stream	Video (3)	`1:12` ls high	`1:11` prio 0
BE	Best effort: web, apps, buffered streaming, unknown	Best Effort (2)	`1:13` ls normal (default)	`1:13` prio 1 (default)
BK	Bulk: downloads, updates, backups, P2P	Bulk (1)	`1:15` ls low	`1:15` prio 2

fwmark values 1–4 select CAKE tins in tc's display order for diffserv4 (Bulk, Best Effort, Video, Voice). This mapping must be **verified by the Stage-1 self-test** (§12.6) on every target kernel, because the self-test is the source of truth, not this table.

4.2 Mark layout (conntrack mark, 32 bit)

Bits	Mask	Meaning
24–26	`0x07000000` (**CLASS**)	0 = none (CAKE falls back to the packet DSCP), 1 = BK, 2 = BE, 3 = IV, 4 = RT
27–29	`0x38000000` (**PROV**)	Provenance of the class: 0 none, 1 default, 2 heuristic, 3 ML, 4 app mark, 5 user rule, 6 lease, 7 system (probes, SAFE)
30	`0x40000000` (**SANCT**)	Sanctioned: an eligibility-envelope violation forces BE

Bits	Mask	Meaning
0–23, 31	—	**Never touched.** Coexists with mwan3 (default `0x3F00`) and pbr (default `0x00FF0000`); the health check verifies that no other package uses bits 24–30

Provenance bits let the **kernel** enforce precedence without a round trip to the daemon: a rule may overwrite the class only when its own provenance is higher than the stored one (§5.2).

Plumbing: - **Upload:** chain `mark_out` (hook postrouting, priority mangle + 10) copies the class bits to the packet mark with four constant-OR rules, e.g. `ct mark & 0x07000000 == 0x04000000 meta mark set meta mark | 0x04000000`. CAKE on WAN egress uses `fwmark 0x07000000`. - **Download:** a tc ingress filter on WAN applies `action ctinfo cpmark 0x7F000000`, then `action mirred egress redirect dev ifb-veriq`. CAKE on `ifb-veriq` uses `fwmark 0x07000000`. The `cpmark` syntax must be verified against `tc-ctinfo(8)` on the target iproute2; this is a Stage-1 self-test item. - **HFSC/HTB backends:** tc `fw` filters match `handle 0x0N000000/0x07000000` → class.

4.3 Backends

Backend	When	Egress	Ingress	Notes
`cake` (default)	CPU headroom at the configured rate (§12.5 bench)	`cake bandwidth S_up diffserv4 dual-srchost nat fwmark 0x07000000 [ack-filter] [wash] overhead … mpu …`	Same on `ifb-veriq` with `dual-dsthost nat ingress`	CAKE's sparse-flow boost already favours short interactive flows, so the "slow-TCP upgrade" heuristic is **off** for CAKE (§5.5)
`hfsc`	Gamers wanting a hard realtime curve; mid-range CPU	HFSC root, 5 classes; RT class `rt m1/d/m2` sized from the RT budget; leaves: RT = `fq_codel` (or `pfifo` with computed limit), others `fq_codel`	Same on IFB	RT budget defaults to `max(400 kbit, 15% of S)` (as QoSmate's GAMEUP formula), configurable
`htb`	Weakest CPUs	HTB root, 3 classes (RT+IV merged high, BE, BK), `fq_codel` leaves, rate-scaled parameters	Same on IFB	Lowest CPU cost; no host isolation (documented)

Rate updates (from the controller) always apply in a valid order: - **CAKE:** `tc qdisc change … bandwidth X`, a single netlink message. - **HFSC/HTB:** first the root/upper-limit class, then the leaves, scaled proportionally, each step read back. If a step fails, a compensating step runs, or the backend goes to SAFE.

4.4 Link-layer and overhead

Preset	CAKE keyword (values per tc-cake(8))	HFSC/HTB `stab` equivalent
`ethernet` (default for fibre/Ethernet WAN)	`ethernet` (overhead 38, mpu 84)	`overhead 38 linklayer ethernet`
`ethernet-vlan`	`ethernet ether-vlan` (+4)	`overhead 42`
`docsis`	`docsis` (overhead 18, mpu 64)	`overhead 18`
`vdsl2-pppoe`	`pppoe-ptm` (overhead 30)	`overhead 30 linklayer ethernet`
`vdsl2-bridged`	`bridged-ptm` (overhead 22)	`overhead 22`
`adsl`	`conservative` (overhead 48, atm) unless the exact encapsulation is known	`overhead 48 linklayer atm`
`unknown` (non-DSL)	`overhead 44 mpu 84` (no ATM cell tax: ATM framing on a fibre link would waste ~10% of the rate)	`overhead 44 linklayer ethernet`
`custom`	`overhead N mpu M [atm/ptm]`	user values

The wizard proposes a preset from the netifd protocol (PPPoE adds 8; a VLAN device adds 4). As in QoSmate's guidance, **overestimating is safer than underestimating.** For HFSC/HTB, the same overhead goes into `stab overhead … linklayer …`.

4.5 Other queue features

Feature	Behaviour	Default
One-directional shaping	`S_dir = 0` disables shaping for that direction, with no IFB for download (the same convention as QoSmate/SQM). Health reports the direction as "off", not "failed".	both on
ACK filter	CAKE `ack-filter` on egress when the download/upload ratio is ≥ 15 (`auto`)	auto

Feature	Behaviour	Default
ACK handling on HFSC/HTB	No ACK dropping. Pure ACKs (< 100 B TCP) of a classified flow ride that flow's class.	—
MSS clamp	When upload or download S < 3000 kbit/s, clamp SYN MSS to 536–1500 (configurable)	auto
Host isolation	CAKE `dual-srchost` / `dual-dsthost` with `nat`	on
BW ratio warning	Warn if download/upload > 20 (ACK-limited downloads)	warn only
Per-device policer	nft meter `limit rate over X kbytes/second burst B` → drop, per IP/set. The UI labels it a **policer, not a shaper**.	none
AQM parameters (fq_codel leaves)	`target = max(5 ms, 1.5 × MTU serialisation time at S)`, `interval = 100 ms + target`, quantum 300 when S < 100 Mbit/s	auto
ECN	CAKE/fq_codel ECN marking on; VeriQ sets no ECN bits	on

4.6 DSCP policy

Profile	Ingress wash	Egress wash	Trust LAN DSCP
product (default)	on (protects Wi-Fi WMM from ISP marks)	on (no leak upstream)	yes, subject to the eligibility envelope
lab	off	off	yes
custom	user	user	per device set `@trusted_dscp`

VeriQ classification survives washing because the class lives in **ct mark bits**, not DSCP. Washing happens *after* classification: on egress via CAKE `wash` or an nft rule at the end of the chain, and on ingress via CAKE `wash` on the IFB.

5. Classification engine

5.1 Inputs VeriQ may use (privacy contract)

Allowed	Not allowed
IP/IPv6 header fields, L4 ports and protocol, TCP flags, DSCP/ECN	Payload bytes of any forwarded packet
Packet sizes and timestamps (first 32 packets per flow, headers only: NFLOG with `snaplen 40`)	TLS SNI, HTTP Host, DNS answers read from packets
Conntrack counters (bytes, packets, duration)	Any per-flow export off the router (unless the operator opts in)
Local configuration: device names, user rules, domain sets populated by the router's own DNS resolver (dnsmasq `nftset`)	Cloud lookups of destinations

Domain sets use the router's own resolver, the same mechanism as QoSmate's dynamic sets. They are configuration the user chose, not inspection. They stop working when clients use DoH, DoT or ECH directly; the UI states this.

5.2 Pipeline and precedence (kernel-resident)

Evaluation runs in nft table `inet veriq`, chain `cls_up` (hook forward, priority mangle + 5), on **LAN → WAN packets**, the same direction model as QoSmate. - Download packets get their class from the ct mark via `act_ctinfo` **before** shaping, so one rule covers both directions. - The order below **is** the v4.1 precedence (levels 1–7). PROV comparisons enforce "higher level wins".

Step	Level	Rule (pseudo-nft)	Notes
0	—	Trust: `ip saddr != @trusted_dscp ip dscp != cs0 ip dscp set cs0` (only if `trust = list`)	Default `trust = all` (with envelope)
1	1 (state)	If `@state_bypass` contains `1`, return (set by veriqd in DISABLED)	SAFE does **not** bypass classification
2	2 (sanction)	`ct mark & SANCT != 0 ct id != @sanctioned` → clear SANCT, CLASS, PROV (recovered, fall through). `ct mark & SANCT != 0` → keep class BE, return	The kernel enforces the penalty timeout; silence counts as good behaviour

Step	Level	Rule (pseudo-nft)	Notes
3	3 (lease)	`ip saddr @lease_rt4` → CLASS = RT, PROV = 6, goto `envelope`; `ip saddr @lease_iv4` → IV, PROV = 6, goto `envelope` (likewise for IPv6)	Lease = a set element with **timeout = remaining lease time**, so expiry happens in the kernel with no daemon involved
4	4 (user)	`ct mark & PROV == PROV_USER` → goto `envelope_or_return` (sticky). Then the compiled user rules in order; first match sets CLASS + PROV = 5	User rules = QoSmate-compatible rules + device/app/schedule targets
5	5 (app mark)	Only if `PROV < 4`: `ip dscp {ef, cs5, 44}` → RT; `{af41, af42, af43, cs4, 45}` → IV; `{cs1, 1}` → BK; PROV = 4	NQB (45) maps to IV, not RT: NQB means "does not build queues", not "priority"
6	6 (classifier)	ML singleton verdicts are written by veriqd via conntrack update (PROV = 3) only when the stored PROV < 3. Heuristics run in the kernel only when `PROV ≤ 2`, and **only demote or reset** (§5.5)	Abstention writes nothing
7	7 (default)	Nothing: CLASS = 0, so CAKE uses the packet DSCP (BE after washing)	—
E	envelope	For CLASS ∈ {RT, IV}: per-flow meters (§5.7); violation → CLASS = BE, SANCT = 1, `add @sanctioned {ct id timeout 60s}` (keyed by flow, so both directions are sanctioned), NFLOG group 8 → veriqd	Applies to every source of preference, leases included

Operating states and the pipeline: - In **SHADOW**, veriqd does not write ML verdicts. It logs what it *would* write. User rules, leases and envelopes stay active, because they are deterministic user configuration. - In **SAFE**, ML verdicts written earlier stay; no new ones are written. - In **DISABLED**, the table is removed.

5.3 User rules (QoSmate-compatible, extended)

UCI section `rule`. Every QoSmate rule option is accepted with the same meaning: - `name`, `proto`, `src_ip`, `src_port`, `dest_ip`, `dest_port`, `counter`, `enabled`; - `class` given as a DSCP name (cs0–cs7, af11–af43, ef); - `!=` negation, `@set` references, ranges, and IPv6 suffix masks `::suffix/::mask`.

A DSCP `class` maps to a VeriQ class as follows: - cs5, cs6, cs7, ef → RT; - cs4, af4x → IV; - cs1 → BK; - everything else → BE.

The original DSCP is also written to the packet when `set_dscp '1'` is given, which keeps QoSmate-style DSCP marking possible.

VeriQ additions:

Option	Meaning
`device` / `group`	Target a named device or group (resolved to that device's IPv4 **and all current IPv6 addresses**, privacy addresses included, via DHCP leases and the neighbour table)
`app`	Target a catalogue app (§5.4): its port and domain sets
`vclass`	`rt | iv | be | bk`, as an alternative to `class`
`schedule`	`Mon-Fri 09:00-17:00` etc. veriqd enables or disables the rule at boundaries (local time; DST-safe re-render)
`direction`	`up` (default, covers both via conntrack) or `down` (matches the reply tuple, for router-initiated or special cases)

Validation: - every rule is compiled into a candidate ruleset; - the candidate is checked with `nft -c -f`; - a failing rule is reported with its name and the nft error, and **the previous ruleset stays active**; - apply is atomic (`nft -f` is a single transaction).

Inline and custom nft: - `/etc/veriq.d/inline.nft` holds statements only, included in `cls_up` at step 4 before the user rules (the same semantics as QoSmate's inline rules); - `/etc/veriq.d/custom.nft` holds a separate table `inet veriq_custom`; - both are validated before apply.

5.4 App catalogue

A catalogue is a signed JSON file shipped in package `veriq-catalog`, with user overrides in `/etc/veriq.d/apps.d/*.json`. Each entry has the form:

```
{"id":"zoom","name":"Zoom Meetings","category":"conferencing","vclass":"iv",
 "match":{"udp_dport":["8801-8810"],"domains":["zoom.us"]},
```

"source":"vendor network-requirements page (URL)","verified":"2026-10-xx","confidence":"vendor-documented"}

- Entries **must** cite a vendor or primary source and carry a `verified` date. Unverified community entries are flagged in the UI.
- Domain matches create dnsmasq `nftset` entries for sets `app_<id>_4` and `app_<id>_6`. This needs `dnsmasq-full`; the health check reports it if missing.
- The starter catalogue (P1) covers the conferencing apps the team can verify, the major consoles' documented ports, and OS update domains for demotion. Each entry is re-verified before release.

5.5 Heuristics (kernel, demote or reset only)

ID	Heuristic	Action	Default (cake)	Default (hfsc/htb)	Origin
H1	**Sustained bulk TCP**: `ct bytes > 10 s × S_down_base` (as QoSmate's `first10s`) or (duration > 30 s and average rate > 30% of S)	CLASS = BK, PROV = 2	on	on	adapted from QoSmate
H2	**New-flow reset**: flows below the 500 ms byte threshold that carry a preferential DSCP from an untrusted source	CLASS = 0 (DSCP decides)	on	on	adapted from QoSmate `mark_500ms`
H3	**Slow-TCP upgrade** (< 150 pps → priority)	—	**off**: CAKE's sparse-flow optimisation already boosts low-rate flows, and upgrading gives priority without a basis (violates P5)	on, but as **"fast BE" sub-class**, never RT/IV	QoSmate, modified
H4	**High-rate UDP in a preferential class** (> 450 pps by default)	Handled by the envelope (§5.7)	on	on	QoSmate `udp_meter`, generalised
H5	**Bulk ports** (`bulk_tcp_ports`, `bulk_udp_ports`)	CLASS = BK	user	user	QoSmate `TCPBULKPORT` / `UDPBULKPORT`

5.6 ML classifier (user space, abstaining)

Sampling: - an nft rule `ct packets < 33 log group 7 snaplen 40 queue-threshold 32`, rate-limited to 4000 packets/s in total, sends headers only; - veriqd also reads conntrack accounting (`nf_conntrack_acct = 1`, set by the package); - the **Lite profile** disables sampling and uses heuristics and rules only.

Checkpoints: a decision may be made at k = 8, 16 and 32 packets, and at 10 s (counters only).

Features (no addresses, no payload): - protocol and a port bucket (well-known class, ephemeral, registered); - packet-size statistics per direction (mean, std, min, max, p25/p50/p75); - inter-arrival statistics, burst count, up/down byte ratio, pps, duration; - DSCP seen, ECN codepoint, and the SYN/ACK flag pattern of the first packets.

Model and output: - LightGBM multiclass per checkpoint (≤ 200 trees, depth ≤ 6), trained off-box in Python, exported to JSON and evaluated in Go (pure tree walk, no cgo); - classes are {RT, IV, STREAM, WEB, BULK}, mapped STREAM → BE, WEB → BE, BULK → BK; - uncertainty: **class-conditional (Mondrian) split conformal**, α = 0.05 per checkpoint. VeriQ acts only on **singleton** sets; empty or multi-class sets mean abstain (BE, PROV unchanged). - Calibration needs ≥ 200 samples per class per checkpoint, otherwise that checkpoint is disabled.

Honesty kit (applies to every number reported): - splits by time and by application, so test apps are unseen in training; - no address or port shortcut features beyond the port bucket; - coverage, selection rate, selective accuracy and abstention reported per class (docs/09 A.9), plus a confusion matrix, and the shift tests (new app version, different network).

On-device drift signal: weekly abstention rate and class mix. If abstention exceeds 80%, the UI says the model is uncertain on this network. VeriQ never auto-retrains on the device.

Model updates: only through the signed package `veriq-model`; the model version is shown in the UI and written into every receipt.

5.7 Eligibility envelope and sanctions

Class	Per-flow envelope (starting values)	Kernel mechanism
RT	≤ max(1 Mbit/s, 2% of S_base) **and** ≤ 450 pps (QoSmate's UDP threshold, adopted as the starting value)	nft meter keyed `ct id . ct direction`, `limit rate over … kbytes/second` / `… /second`
IV	≤ max(8 Mbit/s, 25% of S_base)	same

Class	Per-flow envelope (starting values)	Kernel mechanism
Standing queue in a preferential tin ≥ 3 s	Detected by veriqd from CAKE per-tin `pk_delay` /backlog; it sanctions the top contributing flow	veriqd → conntrack update + set add

Penalty: - the first violation sets a 60 s timeout, kernel-refreshed on each repeat; - on repeated offences within 1 h, veriqd escalates the timeout (×2 per offence, cap 15 min) and resets the count after 1 h clean (as in v4.1); - the device and its other flows are unaffected.

Envelopes use **S_base**, not the moving S, so the kernel rules do not churn. They are re-rendered when S_base changes, for example after the wizard runs.

5.8 Leases (authenticated intent)

Property	Value
Who	Admin (LuCI) or Household role (§11.3); each request is authenticated by the rpcd session
What	A device (or a single app on a device: device + app sets) gets RT or IV for a duration. The default is IV ("Protect my call"); RT is for "game console" mode
Duration	15 / 30 / 60 / 120 min; max 4 h; renewable once without re-approval
Admission	≤ 2 active leases, ≤ 1 per device; refused under capacity collapse (S < 2 Mbit/s or WAN_LIKELY @ L2 with a > 50% drop); the reason is shown
Enforcement	Set elements `@lease_rt4/6` , `@lease_iv4/6` with timeout = remaining time; device address changes are tracked by veriqd, which updates the elements
Persistence	Lease records (device, class, expiry as wall-clock + boot-id, remaining time checkpointed every 60 s) in `/etc/veriq/state/leases.json` . After a reboot, leases resume only if NTP is synced; the remaining time is never extended
Envelope	Always applies; a leased bulk transfer is returned to BE by the envelope
Receipt	Every lease produces a receipt comparing the leased device's delay and loss before and during the lease (§9)

6. Capacity estimation and rate control

6.1 Quantities (never conflated; docs/09 A.3)

Symbol	Meaning	Source
S_dir	Configured shaper rate per direction	Desired state (read back from tc)
S_base, S_min, S_max	Envelope: S_min = max(1 Mbit/s, 0.10·S_base); S_max = 1.05·S_base	Config (docs/09 A.2)
G_gw	Gateway throughput: dequeued bytes of the shaper, overhead-compensated	`tc -s -j` every tick
Ĝ	Median G_gw over the last 1 s (4 ticks)	Derived
load	G_gw / S over the last 1 s	Derived
μ, σ	EWMA tracking estimate of capacity and its dispersion, from flagged samples only	Estimator (§6.6)
D_dir	Delay signal: median over healthy reflectors of their latest delta (age ≤ 1 s)	Prober (§6.3)
Q_own	Own-queue delay: CAKE per-tin `av_delay` / `pk_delay` and backlog / S	`tc -s -j`

6.2 Reflector pool and probing

Pool. The pool has ≥ 8 far candidates across ≥ 3 operators: - IPv4: 1.1.1.1, 1.0.0.1, 8.8.8.8, 8.8.4.4, 9.9.9.9, 149.112.112.112, 208.67.222.222, 208.67.220.220; - IPv6 equivalents where the WAN has IPv6; - user-addable.

The **near** target is the WAN default gateway (the PPPoE peer on PPPoE), re-resolved on every link-up.

Qualification at start and every 6 h: send 10 echo and 10 ICMP-timestamp probes. A reflector is kept if it replies to ≥ 90% and its idle RTT std is < 5 ms. Timestamp-capable reflectors are preferred, because they give one-way deltas. The active set is 5 far reflectors.

Rate: - 10 probes/s in total (v4.1 envelope), round-robin across the active set: each far reflector gets 1.6/s and the near target 2/s; - **idle mode** (load < 10% both ways for 30 s): 2 probes/s in total; - full rate resumes on the first tick with load ≥ 10%.

One-way deltas: - ICMP timestamp replies (RFC 792) give `originate`, `receive` and `transmit` in ms since midnight UTC; - `up = receive − originate` and `down = t_local_recv − transmit`, each containing an unknown but constant clock offset, so **only their changes against a per-reflector baseline are used**; - midnight wrap is handled; the resolution is 1 ms.

Baselines (C8): - `base_i,dir` = the **minimum over a sliding 10-min window of 1-s medians**, taken only from samples where load < 30% in both directions; - with no low-load sample for 10 min, the previous baseline is kept and flagged `stale`; - if it stays stale for > 30 min, that reflector's evidence caps at L1; - this rule rises after a route change and falls after an improvement; a lower-only EWMA cannot rise.

Health. A reflector is unhealthy if any of these holds: - loss > 20% over its last 20 probes; - no reply for 3 s; - **outlier**: its delta deviates from the median of the others by > 3 × MAD for > 10 s while the others are stable.

An unhealthy reflector is excluded at once, replaced from the pool, and cooled off for 10 min.

Aggregation: - `D_dir` = median of the latest deltas of healthy reflectors (age ≤ 1 s); - **quorum ≥ 2**, otherwise the delay signal is MISSING (evidence L0 for delay); - **never a mean**; - when one-way deltas are unavailable, `D_rtt` is used and §6.4 attributes direction.

6.3 Detectors (per tick = 250 ms)

Detector	Rule	Starting values
Bufferbloat (beyond our shaper)	`D_dir > θ_bb` in ≥ 3 of the last 6 ticks **and** `Q_own < 5 ms`	θ_bb = 15 ms
Own queue	`Q_own(BE or BK tin) > 5 ms` and load ≥ 0.95	CAKE's own target is 5 ms
Throughput censoring	Ĝ ≤ 0.80·S with demand present (≥ 1 bulk-class or ≥ 3 active TCP flows with cwnd-limited behaviour: rates steady ±10% for 2 s) and `Q_own < 2 ms`, and a drop ≥ 20% within 2 s	—
Near rise	near delta > 10 ms for ≥ 3 of the last 6 ticks	—
CPU-bound	softirq on the shaping CPU ≥ 90% (from `/proc/stat`) and Ĝ < 0.9·S and no D rise	—
Link down	netifd `ifdown`, or carrier lost on the WAN device	immediate

6.4 Direction attribution (C9)

- **With one-way deltas:** use `D_up` and `D_down` directly.
- **With RTT only:**
  1. If exactly one direction has load ≥ 60%, attribute the rise to it.
  2. If both do, treat both as candidates, but evidence per direction is capped at **L1**, so only bounded fallback applies.
  3. If neither does, it is not our access link: no cut, and the case goes to REMOTE/UNKNOWN.

6.5 Rate controller (per direction)

States: `HOLD`, `PROBE_UP`, `BACKOFF`, `DRIFT`, plus the transient actions `CUT` and `FALLBACK_STEP`, and the global states of §8.

```
every tick (250 ms), per direction d:
  obs  = snapshot()                      # each signal carries its timestamp
  diag = diagnose(obs, d)                # §7 → (label, level, sources)
  act  = NONE
  if state == HOLD_LINKDOWN: continue
  if diag.label == WAN_LIKELY and diag.level >= L2:
      if fresh(obs.G_hat, 1 s):  target = max(S_min, 0.90 * obs.G_hat)          # C1
      else:                      target = max(S_min, 0.75 * S)
      if diag.level == L3:       target = max(S_min, 0.90 * C_measured)
      if target < S: act = RATE_CUT(target); episode.l2_cuts += 1
      if episode.l2_cuts > 3 and not improved(D): diag = MIXED   # no spiral
  elif bloat(d) and diag.level <= L1:            # uncertain but bloated
      act = FALLBACK_STEP(max(S_min, S - 0.05 * anchor)) if load >= 0.60
  elif explore_ok(d):                            # docs/09 A.4
      act = RATE_RAISE(min(S_max, 1.04 * S))     # 1 s refractory
  elif stale(120 s):
      act = DRIFT toward S_base, <= 1 % per 10 s
  if act: verdict = CHECKER(act, obs, versions, budgets, state)   # §8.1
          if verdict == ACCEPT and state == LIVE: enforce(act); readback()
          log(act, verdict, diag, obs)            # always, also in SHADOW
```

Refractory periods: 1 s after a cut; 1 s between raises; 30 s hold after BACKOFF. **Undo:** a FALLBACK_STEP whose receipt (10 s after) is NOT_IMPROVED or INCONCLUSIVE is undone once conditions are re-checked (docs/05 v3.3 rule).

6.6 Estimator μ, σ

Every 1 s, a sample is flagged as one of: - **capacity**: C ≈ Ĝ. Taken during WAN_LIKELY @ ≥ L2 (throughput censoring with delay rising). - **lower bound**: C ≳ Ĝ. Taken when shaper-limited (load ≥ 0.95, Q_own building, no D rise). - **none**: application-limited, so nothing is learned (docs/09 §3.3).

Updates: - On a capacity sample: `μ ← μ + 0.2(x − μ)`, `σ ← σ + 0.2(|x − μ| − σ)`. - On a lower bound: `μ ← max(μ, x)`, and σ shrinks by 5%. - σ widens by 1% per 10 s without samples.

μ and σ are displayed and logged. **They do not trigger cuts on their own**: cuts need diagnosis evidence.

6.7 Capacity memory (time-of-week prior)

- 168 hourly bins (hour of week) holding the median of the day's **capacity** samples, kept for 4 weeks.
- Stored in `/etc/veriq/state/capacity.json`, written at most hourly, ≤ 10 KB.

Uses: 1. **Start rate** after boot or link-up: `min(S_lkg, S_base)`, or, when the bin has ≥ 3 weeks of data, `min(S_base, p20 of the bin)`. 2. The UI shows a statement such as "Evenings are usually ~60 Mbit/s". 3. The prior **never** causes a cut by itself.

6.8 Link events

Event	Action
WAN down (netifd/carrier)	State HOLD_LINKDOWN: freeze rates, pause probes, don't count losses as evidence
WAN up / PPPoE reconnect / IP change	Re-resolve the near target, re-qualify reflectors in the background, resume at the §6.7 start rate, LIVE (or the prior state) after 10 s of healthy probes
Interface renamed or missing	DEGRADED, alert; keep the last kernel config
Clock step (NTP)	No effect: all control timing uses CLOCK_MONOTONIC; wall time is used only for logs and lease restore

7. Diagnosis (R3, extended)

7.1 Evidence sources (independence groups)

Source	Group	Notes
Far-reflector median delta	**FAR**	All far reflectors behind the same access link count as **one** source
Near-hop delta	**NEAR**	MISSING if unreachable or rate-limited (docs/09 A.7)
Throughput censoring	**THR**	§6.3
Own-queue (CAKE tin stats)	**OWN**	—
Wi-Fi station stats (retries, signal, expected throughput, airtime via ubus/hostapd/iwinfo)	**WIFI**	Polled every 5 s
CPU softirq	**CPU**	—
Active capacity test (UDPST to a cooperating server, ≤ 5 s, at most once per 30 min, never during a lease)	**CAP**	Optional, gives L3

7.2 Labels and rules

Label	Rule (all conditions)	Max level
OWN_QUEUE	OWN high, load ≥ 0.95, FAR not rising beyond Q_own	— (normal: CAKE handles it)
WAN_LIKELY	FAR rising **and** (THR **or** NEAR rising with OWN empty)	L2; L3 with CAP agreeing
WAN_LIKELY (near missing)	FAR rising **and** THR	L2 (docs/09 A.7)
REMOTE_LIKELY	FAR rising, NEAR present and flat, THR absent, load not explaining it	L2; **L1 if NEAR missing**
WIFI_LIKELY	The affected client's station shows retries > 30% or expected throughput < its flow demand, while WAN load < 0.6 and FAR flat	L2 (WIFI + FAR flat)
CPU_LIMITED (v5)	CPU detector true	L2 (CPU + no FAR rise)
PROBE_FAULT	Quorum lost, or one reflector an outlier	—
MIXED	Two or more labels qualify, or more than 3 L2 cuts without improvement	≤ L1
UNKNOWN	None of the above	L0–L1

7.3 Permissions (enforced by the Action Checker)

Diagnosis @ level	Permitted
WAN_LIKELY @ L2	One-step cut to `max(S_min, 0.90·Ĝ)` (C1); without a fresh Ĝ, ≤ ×0.75
WAN_LIKELY @ L3	Re-baseline to `max(S_min, 0.90·C_measured)`
REMOTE / WIFI / CPU_LIMITED @ ≥ L2	**Hold** S; notify (UI explains the cause and the advice)
L0–L1, MIXED, UNKNOWN with bloat	Bounded fallback only (≤ 5% steps, ≤ 10% per rolling 10 min, cooldown 60 s that never refunds)
PROBE_FAULT	Hold; rotate the reflector

8. Safety: checker, states, reconciler, watchdog

8.1 Action Checker (docs/09 A.1, updated by C1)

All A.1 rules and refusal codes remain. Changes and additions:

Action	Added or changed rule	Refusal code
RATE_CUT	At L2 with fresh Ĝ: target ≥ max(S_min, 0.85·Ĝ). At L2 without Ĝ: target ≥ 0.75·S. At L3: target ≥ 0.85·C_measured. Below that → refused	`CUT_BELOW_EVIDENCE` (new), `BELOW_FLOOR`, `STEP_EXCEEDS_LEVEL`
RULESET_APPLY (new)	Candidate passed `nft -c`; version = desired + 1; not in SAFE unless it is the SAFE ruleset	`INVALID_RULESET`, `VERSION_MISMATCH`
BACKEND_SWITCH (new)	Only on an explicit user request (maintenance), never autonomous	`NOT_USER_INITIATED`
SANCTION_ESCALATE (new)	Escalation ≤ ×2 per offence, cap 15 min; only for flows with a recorded violation	`NO_VIOLATION`, `CAP_EXCEEDED`
LEASE_REVOKE	Always admissible (user or expiry)	—

Claim limit: the checker checks admissibility against the implemented rules only.

8.2 Operating states (C10)

State	Meaning	Kernel config	Controller
INIT	Starting; reconciler adopts or installs	Static CAKE at the start rate	Off
SHADOW	Default for the first 24 h after install, and after a model or algorithm upgrade	Static CAKE + rules/leases	Computes and logs, never applies rate/ML actions
LIVE	Normal	Adaptive	Applies checked actions
DEGRADED	Foreign change detected, quorum lost for > 60 s, or a component failing	Last good config, frozen	Holds; alerts
HOLD_LINKDOWN	WAN down	Frozen	Paused
SAFE	Watchdog or fatal error	Static CAKE at 0.85 × min(S_lkg, S_base); rules/leases kept	Off until recovery
DISABLED	User disabled	VeriQ objects removed	Off

Transitions: - SHADOW → LIVE after 24 h, if the shadow report has no anomalies (crash, CPU over budget, > 5 disagreements/h judged unsafe); otherwise the user confirms in the UI; - any state → SAFE on watchdog; - SAFE → INIT on daemon restart (the reconciler decides); - DEGRADED → LIVE once the foreign change is resolved, or when the user chooses “take ownership”.

8.3 Reconciler (docs/09 §3.5, OpenWrt specifics)

- **Every tick:** compare the desired document (version N) with the observed state (`tc -j -s qdisc/filter show dev WAN/ifb-veriq`, `nft -j list table inet veriq`, owned via the comment `veriq:vN`).
- **Self-healing:** if table `inet veriq` vanishes (for example after a firewall restart), it is re-applied from the desired state within one tick and an event is logged. If a foreign qdisc replaces ours, VeriQ goes DEGRADED and does not fight.
- **Restart adoption:** if the kernel state matches the persisted version, adopt it and resume; otherwise go SAFE, then INIT.
- **Ordered apply:** tc steps are ordered so every intermediate state is valid; each is read back; on a partial failure, compensate or go SAFE; at most 3 retries with capped backoff.

8.4 Watchdog (C11)

- `veriq-watchdog` is a POSIX sh loop under procd (the second instance of `/etc/init.d/veriq`).

- Every 2 s it checks the mtime of `/var/run/veriq/heartbeat`, which veriqd touches every 1 s.
- If the heartbeat is > 10 s old and the state is not DISABLED, it runs `tc qdisc change` to the SAFE rates from `/var/run/veriq/safe.env`, which veriqd writes whenever the rates change. It logs to syslog and writes `state=SAFE`.
- **Time-to-safe target ≤ 12 s.**
- procd respawns veriqd (respawn threshold 3600 s, timeout 5 s, retry 5); the reconciler then adopts the state.

8.5 Persistence and flash wear

Data	Location	Write policy
Runtime state, heartbeat, decision log ring (≤ 2 MB), metrics rings	`/var/run/veriq`, `/tmp/veriq` (RAM)	Continuous
Budget state, sanctions escalation, leases	`/etc/veriq/state/*.json`	On change, ≤ 1 write/min
Capacity memory, hourly summaries	`/etc/veriq/state/`	≤ 1 write/hour
Config	`/etc/config/veriq`, `/etc/veriq.d/`	On user change

Atomic writes (temp file + rename). Everything under `/etc/veriq` is kept across sysupgrade (`/lib/upgrade/keep.d/veriq`).

---

9. Receipts and explainability

9.1 Receipts

The schema is unchanged from docs/09 §3.6, with three fields added: `model_version`, `backend`, and `profile`.

Outcome rules for a rate action: - **IMPROVED:** median D in the 10 s after < 0.7 × the 10 s before, and $\hat{G}$ not reduced by more than the cut. - **NOT_IMPROVED:** D not reduced. - **INCONCLUSIVE:** a concurrent change (another action, a load change > 30%, a link event, or fewer than 20 samples).

Lease receipts compare the leased device's flows' tin delay (CAKE per-tin `av_delay` for their tin) and drops before and during the lease.

Receipts state **association, not causation**.

9.2 Explain API (template sentences, no LLM)

`veriqctl explain rate down` returns the last decision with its evidence, the permission row, the checker verdict and the receipt. The UI renders it as a sentence, for example:

> "20:41:07: download rate 95 → 18 Mbit/s. The far-reflector delay rose 42 ms, and throughput fell to 20 Mbit/s while downloads still wanted more (evidence L2: FAR + THR). Checker: ACCEPT. After 10 s: delay −38 ms → IMPROVED."

`veriqctl explain flow <id>` returns the precedence trace (which levels matched), the class and provenance, the envelope status, the ML probability vector and conformal set (or "abstained"), and the sanction history.

The household view shows only plain-language summaries, such as "Your call is protected until 21:30" or "The slowdown is coming from the internet beyond your provider's first hop; reducing your speed would not help, so VeriQ is holding."

---

10. Adverse-conditions matrix (A1–A32)

Reading the last column. - "Delay-only autorate" means a controller that cuts on reflector delay without diagnosis. - Statements marked **(read)** come from QoSmate `qosmate-autorate.sh` v0.5.48 @ `c507843`: - RTT is averaged across reflectors; - if no reflector replies, latency is set to 9999 ms; - the baseline only moves down after the first 10 samples; - the decrease branch does not check load, and both directions use the same latency value. - What follows from those facts is a **hypothesis** until the head-to-head runs (§15). Maintainers may change their code at any time; re-read the source at the pinned commit before publishing a comparison.

ID	Condition	VeriQ detection	VeriQ bounded response	Delay-only autorate (hypothesis)	Test	Pass criterion (target)
A1	Capacity step down 100 → 20 Mbit/s	FAR + THR → WAN_LIKELY L2	One-step cut to 0.9·$\hat{G}$ (C1)	Cuts in steps of ×0.85 per 1 s refractory (read: QoSmate defaults)	S2	Confirmation ≤ cake-autorate + 1 s; call freezes ≤ comparator
A2	Capacity oscillates 10–50 Mbit/s every 5–20 s (LTE)	Repeated L2 episodes	Cut on each L2; PROBE_UP ×1.04 with BACKOFF; no spiral (> 3 cuts without improvement → MIXED)	Saw-tooth tracking	A2	Oscillation (reversals/min) ≤ comparator; delay p95 ≤ comparator

ID	Condition	VeriQ detection	VeriQ bounded response	Delay-only autorate (hypothesis)	Test	Pass criterion (target)
A3	Capacity recovers 20 → 100	Load ≥ 75%, D < 5 ms	PROBE_UP ×1.04/s	Raise +2% per 3 s (read: defaults) → slower reclaim	A3	Time to 90% of the new capacity ≤ comparator
A4	Remote-path delay (+80 ms beyond the ISP first hop)	FAR up, NEAR flat, no THR → REMOTE L2	**Hold**	Cut toward the minimum (decrease branch has no load check; read)	S6	Inappropriate interventions: count/magnitude/duration below comparator (primary)
A5	One reflector lies (spikes or deprioritised ICMP)	Outlier > 3 × MAD for > 10 s	Exclude and rotate; median unaffected	The mean is shifted by one reflector (read: averaging)	S7	No rate change attributable to the reflector
A6	All reflectors unreachable (ICMP blocked/outage)	Quorum lost → L0, PROBE_FAULT	Hold at the current S; DEGRADED after 60 s; fall back to static	Latency = 9999 ms → decrease branch every refractory period down to the minimum (read)	A6	S stays within ±5% of the pre-fault S; recovery needs no manual action
A7	Base RTT rises permanently (+15 ms route change)	Windowed-min baseline re-learns within 10 min of low-load samples	Fallback-bounded at worst (≤ 10%/10 min) while it re-learns	Lower-only baseline → permanent delta > threshold → repeated cuts (read: baseline update rule)	A7	Cumulative magnitude below comparator; no stuck-at-minimum
A8	Wi-Fi client is the bottleneck	WIFI + FAR flat → WIFI_LIKELY	Hold; tell the user which device and why	No reaction expected (WAN probes unaffected)	S10	Correct diagnosis shown; no WAN cut
A9	Upload saturation only (cloud backup)	OWD up-delta, or load attribution	Cut **upload only**	Both directions cut on the same RTT (read)	S4	Download S unchanged (±5%); upload delay p95 ≤ comparator
A10	500-flow BitTorrent + call	OWN + host isolation; H1/H5 → BK	CAKE per-host fairness; bulk tin	Same datapath	A10	Call freezes ≤ SQM; Jain index across hosts ≥ 0.9
A11	High-rate UDP marked EF (misbehaving app)	Envelope meter	Sanction the flow → BE, device unaffected	QoSmate demotes UDP > 450 pps (read)	S11	Sanctioned within 1 s; other flows of the device unaffected
A12	A device marks all its traffic EF	Envelope on every flow	Each violating flow sanctioned; conforming small flows keep RT	—	A12	No starvation of other hosts (starvation metric = 0)
A13	Lease abuse (leased device runs a bulk download)	Envelope applies to leases	The bulk flow goes to BE; the lease continues for conforming flows	—	S5 variant	Others' bulk progress > 0 in every 10 s window
A14	PPPoE reconnect / WAN IP change	netifd hotplug	HOLD_LINKDOWN → resume at the start rate	Losses during the outage read as 9999 ms → cuts (read); slow regrowth	A14	No cut during the outage; ≥ 90% of the pre-event rate within 30 s under load
A15	Daemon crash (kill -9)	procd + watchdog heartbeat	Kernel config persists; SAFE ≤ 12 s; adopt on restart	(QoSmate: procd restarts the daemon; read)	S8	Time-to-safe ≤ 12 s; no internet outage
A16	Reboot during peak hour	Capacity memory	Start at min(S_lkg, S_base, p20 of the bin)	Starts at base	A16	Bloat in the first 60 s ≤ static SQM
A17	Router CPU saturated (another service)	CPU detector → CPU_LIMITED	Hold (no cut); recommend the hfsc/htb backend or a lower rate	Throughput shortfall + delay may read as congestion → cuts	A17	No cut attributable to CPU; advice shown
A18	NTP clock step ±1 h	Monotonic clock for control	No effect	—	A18	No spurious events
A19	ISP aggregation congestion (evening)	FAR + NEAR rise with throughput ≈ S	WAN_LIKELY @ L1–L2 → bounded or fast cut by evidence	Cuts	A19	Delay p95 ≤ comparator; inappropriate magnitude ≤ comparator

ID	Condition	VeriQ detection	VeriQ bounded response	Delay-only autorate (hypothesis)	Test	Pass criterion (target)
A20	IPv6-only / dual-stack clients	v4 and v6 sets, probes on both families	Identical classification	QoSmate supports v6 rules (read)	S9	Per-tin counters match intent for IPv6
A21	Highly asymmetric link (≥ 20:1)	Ratio check	`ack-filter` (CAKE, auto); warning	QoSmate: ACK filter auto ≥ 15 (read)	A21	Download goodput ≥ SQM
A22	Very low uplink (≤ 3 Mbit/s)	Config	MSS clamp; fq_codel target scaling (HFSC/HTB)	QoSmate: MSS clamp < 3000 kbit (read)	A22	Call audio concealment ≤ SQM
A23	Gigabit fibre on a weak CPU	Bench at setup	Backend recommendation; one-directional shaping where the download does not bloat	—	A23	No throughput collapse; recommendation logged
A24	Flow offloading enabled	Health check	Refuse LIVE; explain; one-click disable with consent	QoSmate docs: disable other QoS first	A24	Detected 100%
A25	Invalid custom nft rule	`nft -c`	Reject the candidate; keep the previous ruleset	QoSmate validates custom rules (read)	A25	Previous ruleset still active
A26	Cooperating UDPST server down	CAP unavailable	Max level L2	—	A26	No L3 actions
A27	All traffic inside one VPN tunnel	Classifier abstains	BE; the user may lease the device	Same	A27	No preference without a basis
A28	Radio loss without delay (lossy LTE)	Loss not used as congestion	No cut	—	A28	No cut; loss shown
A29	Idle link (no demand)	Censoring rules	No cut on delay at idle; DRIFT to base	Decrease branch can still trigger on delay at idle (read)	A29	Zero cuts at load < 10%
A30	Foreign tool changes the qdisc	Reconciler	DEGRADED; never fights	—	A30	No flapping
A31	Firewall restart flushes rules	Reconciler self-heal	Re-apply ≤ 1 tick	—	A31	Table restored ≤ 1 s
A32	Probe-path disturbance (ICMP deprioritised at the ISP under load)	NEAR/FAR inconsistent with THR/OWN	PROBE_FAULT or MIXED → hold/bounded	Cuts	A32	Primary-outcome scenario; inappropriate interventions below comparator

Summary of the response pattern: VeriQ cuts fast where evidence supports a cut (A1–A3, A19), holds where a cut cannot help (A4–A8, A17, A28, A29, A32), and degrades to static behaviour where it is blind (A6, A15, A30). That pattern is the dominance invariant made concrete.

11. User experience

11.1 Setup wizard (LuCI, ≤ 5 minutes target)

1. **Detect.** Detect the WAN device and protocol (netifd), propose a link preset (§4.4), check for conflicts (SQM, qosify, QoSmate, flow offloading, hardware NAT) and check kernel capabilities (§12.6).
2. **Measure.**
   - Run three capacity measurements: UDPST if a server is configured, otherwise `speedtest-go`.
   - Take each as the **interface-counter plateau**: the median of 1-s samples ≥ ½ of the peak, adopted from QoSmate's `plateau_rate`. If the tool reports > 1.25 × what crossed the interface, use the measured value (QoSmate's `warn_if_overreported` idea).
   - S_base is 90% of the lowest of the three.
   - Router-side tests may underestimate. The user can type in values from a LAN test instead.
3. **Backend bench.** Run CAKE at S_base for 10 s with a local load. If it reaches < 95% of the target, or the shaping CPU is > 85%, recommend `hfsc` or `htb`, or one-directional shaping.

4. **Verify with a receipt.** Run a bufferbloat test before and after (load plus probes). Show an A–F grade per direction using the added p95 delay: A ≤ 5 ms, B ≤ 30, C ≤ 60, D ≤ 200, F > 200 (grade boundaries are VeriQ's own, labelled as such).
5. **Choose.**
   - Profile: Lite, Standard or Pro.
   - Household page password and QR code.
   - Start mode: "Observe 24 h, then adapt" (default) or "Adapt now".
   - Import from QoSmate, if detected.

11.2 LuCI app (`luci-app-veriq`, Network → VeriQ)

Page	Contents	Beyond QoSmate
Overview	State badge (LIVE/SHADOW/SAFE…), current S per direction with μ±σ, live added delay, bufferbloat grade, active leases, last 5 decisions as sentences, health summary	Explainable decisions; state machine visible
Devices	Devices from DHCP and the neighbour table (name, MAC, all IPs incl. IPv6 privacy addresses), class mix, current rate, buttons: *Protect 1 h* (lease), *Limit* (policer), *Group*	Device identity survives IP changes
Rules	Rule editor (QoSmate fields + device/app/schedule), counters, enable/disable, drag order, validation errors inline; IP sets (static/dynamic), domain sets; **Import QoSmate/SQM**	Device/app/schedule targets; importer
Apps	Catalogue browser: enable "prioritise" or "deprioritise" per app; source and verification date shown	—
Connections	Live conntrack table: proto, endpoints, class, **provenance (why)**, confidence/abstained, envelope status, pps, kbit/s; filter syntax as QoSmate (space-separated AND terms)	"Why" column
Queues	Per-tin/class bytes, packets, drops, ECN marks, av/peak delay for both directions; time-series charts	—
Autorate	Rate, Ĝ, μ±σ, D per direction; reflector table (health, baseline, delta, timestamp support); evidence level timeline; episodes and the budget meter	Evidence levels; reflector health
Receipts	List of receipts (filter by outcome), lease receipts, shadow disagreements	—
Settings	Basic (WAN, rates, preset, backend, profile), DSCP policy, heuristics toggles, envelope values, probing, privacy	—
Advanced	Inline and custom nft, CAKE extra parameters, HFSC/HTB parameters, exporter, webhook alerts	—
Diagnostics	Health check, self-test (§12.6), bench, bufferbloat test, support bundle download	Automated self-test

Technology: - LuCI client-side JS views (`'use strict'; 'require view'; 'require form'; 'require rpc'`), the same framework as QoSmate; - charts drawn with **uPlot** (MIT, ~50 KB); - strings in gettext `.po` with **English and Hindi** at P1 (further languages via the community).

11.3 Household page (non-technical, mobile-first)

- **URL and login:** served at `/veriq/` (static files, uhttpd), logging in as rpcd user `household`. The ACL allows only `veriq.status`, `veriq.lease_request`, `veriq.lease_list`, `veriq.lease_cancel_own` and `veriq.explain_simple`.
- **Screens:**
  1. "Is the internet OK?" (traffic light + one sentence);
  2. "Protect something": pick a device or "this phone", a duration, and a reason;
  3. "Why is it slow?" (last diagnosis in plain words with advice: move closer to Wi-Fi, the problem is outside the home, the router is overloaded).
- **Accessibility:** large touch targets, WCAG AA contrast, works without JavaScript frameworks (vanilla JS, < 100 KB).

11.4 CLI

```
veriqctl status [-j]                  veriqctl explain rate up|down
veriqctl explain flow <ctid|tuple>    veriqctl lease add <device> <rt|iv> <minutes>
```

```
veriqctl lease list|cancel <id>      veriqctl receipts [--since 1h] [--outcome improved]
veriqctl health                      veriqctl selftest
veriqctl bench [--rate X]            veriqctl import qosmate [/etc/config/qosmate] [--dry-run]
veriqctl import sqm                  veriqctl shadow report [--since 24h]
veriqctl state set live|shadow|safe  veriqctl support-bundle [-o file.tar.gz]
veriqctl config validate             veriqctl version
```

11.5 Alerts

- LuCI banner for: SAFE, DEGRADED, offload detected, or a model abstaining > 80% of the time.
- Optional webhook (JSON POST to a user-set URL, e.g. ntfy/Home Assistant), rate-limited to 1 per 5 min per type.

---

12. OpenWrt productisation

12.1 Packages

Package	Contents	Depends
`veriq`	`veriqd`, `veriqctl`, `/etc/init.d/veriq` (+ watchdog instance), `/etc/config/veriq`, `/etc/uci-defaults/90-veriq`, hotplug `/etc/hotplug.d/iface/30-veriq`, `/usr/libexec/rpcd/luci.veriq`, rpcd ACLs, `/lib/upgrade/keep.d/veriq`, the bench helper	`kmod-sched-cake`, `kmod-sched-ctinfo`, `kmod-ifb`, `kmod-sched-core`, `tc-full`, `nftables` (fw4), `kmod-nft-core`, `conntrack` accounting sysctl
`luci-app-veriq`	LuCI views, menu, ACL, `.po`	`luci-base`, `veriq`
`veriq-household`	`/www/veriq/` static household page	`veriq`, `uhttpd`
`veriq-model`	Classifier model + conformal tables (signed)	`veriq`
`veriq-catalog`	App catalogue (signed)	`veriq`
`veriq-exporter`	Prometheus text endpoint (LAN only)	`veriq`
(optional) `kmod-sched-hfsc`, `kmod-sched-htb`, `kmod-sched-fq_codel`	Backends	—

Builds: - CI cross-compiles `veriqd` with the official Go toolchain (`CGO_ENABLED=0`; `GOARCH` arm64/arm/mipsle/amd64; `GOMIPS=softfloat`; `-trimpath -ldflags "-s -w"`); - the binary is packaged with the OpenWrt **SDK** for 24.10 (`.ipk`) and for the apk-based release line (`.apk`); - for upstream submission later, switch to `golang-package.mk`.

Targets in CI: - `x86_64`; - `aarch64_cortex-a53` (Pi 3/4, many Filogic/RK devices); - `aarch64_cortex-a76` (Pi 5); - `arm_cortex-a7_neon-vfpv4`; - `mipsel_24kc` (MT7621).

Distribution: - a signed feed (usign for opkg, the apk signing key for apk) hosted on GitHub Pages; - installation: add the feed key and URL, then `opkg install veriq luci-app-veriq` (or `apk add …`); - **no `curl | sh` and no runtime self-download of code.** Updates come only through the package manager. (QoSmate's README installs by fetching raw scripts with `uclient-fetch` and offers an in-script `update` command; read.)

12.2 Footprint budgets (targets, measured in Stage 4)

Profile	Features	RAM (RSS)	Flash (installed)	CPU (veriqd)	Minimum device
Lite	Rules, sets, heuristics, envelope, autorate + diagnosis (no ML, no NFLOG), hfsc/htb/cake	≤ 20 MB	≤ 6 MB	≤ 3% of one core	128 MB RAM, MIPS 880 MHz
Standard (default)	+ ML classifier, receipts, household page	≤ 40 MB	≤ 10 MB	≤ 5% of one A53 core	256 MB RAM, ARMv8
Pro	+ exporter, UDPST client, long history	≤ 60 MB	≤ 14 MB	≤ 7%	512 MB RAM

CAKE itself runs in the kernel and its cost scales with the rate. The bench (§12.5) measures it per device; the evaluation never attributes CAKE's CPU cost to veriqd or the reverse.

12.3 procd, UCI and hotplug

- **Init script:**
  - `START=95`, `USE_PROCD=1`;
  - instances `veriqd` (respawn `3600 5 5`, `procd_set_param limits nofile=4096`) and `watchdog`;
  - `procd_add_reload_trigger veriq`, plus an interface trigger on the WAN `network.interface`;
  - `service_triggers` reload on config change;
  - `reload` = re-render, validate (`nft -c`), checker, apply.

- **UCI:** a full schema in Appendix B; validated by veriqd at load time (`veriqctl config validate`), with errors reported against `section.option`.
- **uci-defaults:** on first install, if `/etc/config/qosmate` exists, offer an import (never automatic). Set `net.netfilter.nf_conntrack_acct=1`. Migrate config versions (`option config_version`).
- **Hotplug (iface):** on WAN ifup/ifdown, send the event to veriqd over the socket. If veriqd is down, the script does nothing; the kernel config persists.

12.4 Conflict detection

Conflict	Detection	Action
SQM / qosify / QoSmate enabled	`/etc/init.d/<svc> enabled`; qdiscs on WAN not owned by VeriQ	Block LIVE; offer import + disable (with consent)
Software flow offloading	`uci get firewall.@defaults[0].flow_offloading`	Block LIVE; explain the bypass; one-click disable with consent
Hardware flow offloading	`flow_offloading_hw`, vendor PPE	Same, with a throughput warning on weak CPUs
Other users of ct mark bits 24–30	Scan the nft ruleset for `ct mark set` touching those bits	Refuse to start the classifier; explain
Packet steering / multi-queue NIC (performance)	`/etc/config/network` globals	Informational

12.5 Bench

`veriqctl bench`: - creates temporary veth/netns load on the router itself, with no WAN traffic; - reports the throughput reached and softirq CPU for CAKE, HFSC and HTB at the target rate; - stores the results and uses them for the backend recommendation.

12.6 Health check and self-test (beyond QoSmate's `health_check`)

Check	Method
Service/config	procd running, config valid, state not SAFE/DEGRADED
Kernel capabilities	`tc qdisc add … cake` on a dummy device; ctinfo action; IFB; nft features (meters, sets with timeout, `log group`)
Ownership	Owned objects match the desired version
Tin self-test	Inject marked probe packets of each class through the shaped path (router-originated via a dedicated test netns or from a LAN helper); verify the per-tin counters increment as expected in **both directions**
Mark plumbing	Verify `fwmark` value → tin mapping and `ctinfo cpmark`
Probes	Quorum, timestamp support, near target present
Conflicts	§12.4
Package integrity	`opkg` / `apk` verify of installed files (checksums from the package manager, not a custom md5 list)

The output is human-readable plus JSON (`status=…;errors=N`, compatible in spirit with QoSmate's `health_check`).

12.7 Support bundle

`veriqctl support-bundle` produces a tar.gz containing: - config with LAN addresses and MACs replaced by stable hashes; - `tc -s -j` and `nft -j` dumps; - the last 2 h of the decision log; - health and self-test output, version, kernel and board info.

It contains no packet data. It is shown to the user before download.

12.8 Upgrade, rollback, uninstall

- **Package upgrade:** config is kept, `config_version` is migrated, and the controller enters SHADOW for 24 h if the algorithm or model version changed.
- **Rollback:** the package manager installs the previous version; the state files stay compatible for one major version.
- **Uninstall:** a `prerm` script stops the service, removes the owned qdiscs, IFB and nft table, and leaves the user config in `/etc/config/veriq-opkg`.

12.9 Compatibility

- **Supported:** OpenWrt 24.10.x (fw4, kernel 6.6) and the following release line.
- **Best effort:** 23.05.
- **Not supported:** iptables-only firmwares.
- **Other Linux:** the same daemon runs on Ubuntu (lab D1) with a systemd unit and a plain-file config backend.

Portability path: - **prplOS** (OpenWrt-based, carrier-grade): adds a TR-181/USP data model `Device.X_VERIQ_*` through the USP agent; - **RDK-B**: a component wrapper over the same daemon with TR-181 parameters. This needs a partner; no Tata Elxsi integration or endorsement is implied.

13. Security and privacy

Threat	Mitigation
LAN device spoofs DSCP for priority	Eligibility envelope; optional trust list; sanctions are per flow
Lease abuse by a household member	Leases need the household password (rpcd); admission ≤ 2; envelope; audit log of who requested what
Local API abuse	Unix socket 0600 root; rpcd ACL roles (admin, household); no network listener except the opt-in exporter (LAN zone only)
Supply chain	Signed packages and signed model/catalogue; reproducible builds (Go `-trimpath`, pinned module versions, `go.sum`); SBOM (SPDX) per release
Malformed config or custom nft	Validation before apply; previous ruleset kept
NFLOG flood (many new flows)	nft `limit rate 4000/second` on sampling; per-flow cap 32 packets; the queue drops oldest first
Probe abuse / reflector etiquette	≤ 10 pps total, idle mode 2 pps, randomised order, honours ICMP rate limiting (unhealthy → rotate)
Privacy (headers reveal behaviour)	Local only; ring buffers (7 days of aggregates, flow records ≤ 24 h in RAM); export only with explicit opt-in; support bundle hashes identifiers
Legal (India)	The Digital Personal Data Protection Act 2023 may treat household flow metadata as personal data. Default is local-only processing with no transfer; any operator telemetry requires notice and consent; to be reviewed by Tata Elxsi counsel before any pilot
Licences	VeriQ is clean-room: QoSmate (GPL-3.0) and cake-autorate (copyleft; check its LICENSE) **ideas only, no code copied**. Implementers do not open GPL source files while writing the corresponding VeriQ modules; the QoSmate importer is written from the README's documented config format. Licence of VeriQ itself: decided with Tata Elxsi (Apache-2.0 proposed for the OpenWrt feed). Third-party Go modules restricted to permissive licences (MIT/BSD/Apache)

14. Implementation specification

14.1 Repository layout (monorepo)

```
veriq/
├── go.mod                      # module github.com/<org>/veriq, Go 1.22+
├── cmd/veriqd/                 # daemon main: flags, wiring, signal handling
├── cmd/veriqctl/               # CLI (talks JSON-RPC over /var/run/veriq.sock)
├── internal/
│   ├── clock/                  # Clock interface (monotonic); fake clock for tests
│   ├── config/                 # UCI parser + validator + schema (Appendix B)
│   ├── model/                  # shared types: Direction, Rate, Class, Prov, Diagnosis, Action…
│   ├── observe/                # tcstats (tc -s -j), nftstats, conntrack (netlink), nflog, wifi (ubus), dhcp/neigh
│   ├── probe/                  # ICMP echo/timestamp (raw socket), reflector manager, qualification
│   ├── estimate/               # baselines, detectors, μ/σ, capacity memory
│   ├── diagnose/               # evidence groups → label + level (§7)
│   ├── control/                # rate controller FSM (§6.5), fallback episodes, budgets
│   ├── classify/               # rule compiler, app catalogue, heuristics config, ML (tree eval + conformal)
│   ├── lease/                  # lease table, admission, persistence
│   ├── sanction/               # envelope events, escalation
│   ├── checker/                # Action Checker (pure function) + refusal codes
│   ├── enforce/                # backends: cake, hfsc, htb; nft renderer; tc batch; read-back
│   ├── reconcile/              # desired vs observed, ownership, self-heal
│   ├── state/                  # operating-state machine, persistence (atomic writes)
│   ├── receipt/                # receipt assessor, explain templates
│   ├── api/                    # JSON-RPC server, method table, ACL hints
│   ├── importer/               # qosmate + sqm config importers
│   └── telemetry/              # ring buffers, decision log (JSONL), exporter
├── openwrt/
│   ├── veriq/                  # Makefile + files/ (init, config, uci-defaults, hotplug, rpcd, acl, keep.d)
│   ├── luci-app-veriq/         # htdocs/luci-static/resources/view/veriq/*.js, menu.d, acl.d, po/{en,hi}
│   ├── veriq-household/        # www/veriq/
│   └── feed/                   # feed index, keys (public), CI scripts
├── ml/                         # Python: dataset builders, training, conformal calibration, export, honesty kit
├── lab/                        # Python + shell: netns topology, ISP emulator, scenario runner, evaluator, ledger
├── twin/                       # 3D twin (three.js) + replay loader
```

```
├── docs/
└── .github/workflows/          # ci.yml (lint, unit, race), build.yml (cross + SDK), lab.yml (netns integration)
```

14.2 Core types and interfaces (Go)

```go
type Dir uint8            // Up, Down
type Class uint8          // None=0, BK=1, BE=2, IV=3, RT=4
type Prov uint8           // None, Default, Heuristic, ML, AppMark, UserRule, Lease, System
type Level uint8          // L0..L3
type Label uint8          // OwnQueue, WANLikely, RemoteLikely, WiFiLikely, CPULimited, ProbeFault, Mixed, Unknown

type Sample struct { V float64; T time.Duration /* monotonic */; OK bool }

type Snapshot struct {        // one per tick, immutable
    T          time.Duration
    S, Ghat    [2]Sample      // kbit/s
    Load       [2]Sample
    D          [2]Sample      // ms, median of healthy reflectors (or RTT)
    OWDAvail   bool
    QOwn       [2]Sample      // ms
    Near       Sample         // ms delta, OK=false → MISSING
    Healthy    int            // healthy far reflectors
    CPUSoftirq Sample
    WiFi       []StationStat
    Version    uint64         // last read-back kernel version
}

type Diagnosis struct { Label Label; Level Level; Sources []string; Dir Dir }

type ActionType uint8 // RateCut, RateRaise, FallbackStep, Drift, LeaseGrant, LeaseRevoke, MarkSet,
                      // RulesetApply, BackendSwitch, SanctionEscalate, Safe, RevertToRechecked
type Action struct {
    ID      string        // ULID
    Type    ActionType
    Dir     Dir
    Target  float64       // kbit/s for rate actions
    Payload any           // lease, mark, ruleset hash…
    Basis   Diagnosis
    Version uint64        // desired version this action builds on
}

type Verdict struct { Accept bool; Code string; Reason string }

// Pure: no I/O, no clock reads. Fully table-tested.
func Check(a Action, s Snapshot, b Budgets, l LeaseTable, st OpState, cfg Limits) Verdict

type Backend interface {        // cake, hfsc, htb
    Render(desired Desired) (tcBatch []string, err error)   // full install
    RateChange(dir Dir, kbit float64) ([]string, error)     // ordered steps
    Readback(dev string) (Observed, error)                  // parse tc -s -j
}

type Enforcer interface {
    Apply(a Action, tok Verdict) error   // refuses without Accept for a.ID + a.Version
}
```

Concurrency model: - A single **control goroutine** owns all decision state and runs the 250 ms tick. - I/O goroutines (prober, conntrack events, NFLOG reader, tc/nft sampler, API server, Wi-Fi poller) send typed messages over channels. - No shared mutable state, and no mutex in decision logic. - Deterministic replay: the decision log records every input message with its monotonic timestamp, so `veriqctl replay log.jsonl` reproduces the decisions bit-for-bit (this is a test requirement).

Exec vs netlink: - tc and nft are run as processes (`tc -batch -`, `nft -f -`, `-j` for read-back). Each call costs a few ms, which fits a budget of ≤ 2 changes/s. - Conntrack updates and events use netlink (`github.com/ti-mo/conntrack`, MIT). - NFLOG uses `github.com/florianl/go-nflog` (MIT). - ICMP uses `golang.org/x/net/icmp` (BSD) plus a hand-built timestamp message. - No cgo.

14.3 Data schemas

Decision log line (JSONL, RAM ring, exported on demand):

```jsonl
{"t_mono_ms":123456789,"t_wall":"2026-11-20T20:41:07.250+05:30","tick":493827,
 "dir":"down","state":"LIVE","snapshot":{"S":95000,"Ghat":20100,"load":0.99,"D":42.1,"QOwn":0.8,
 "near":{"ok":true,"d":39.0},"healthy":5,"owd":true},
 "diag":{"label":"WAN_LIKELY","level":2,"sources":["FAR","THR"]},
 "action":{"id":"01JC…","type":"RATE_CUT","target":18090},
 "verdict":{"accept":true},"applied":{"version":812,"readback_ok":true},
 "config_version":812,"model_version":"m-2026.11.1","backend":"cake","profile":"standard"}
```

Receipt: as docs/09 §3.6 plus `model_version`, `backend` and `profile` (§9.1).

State files: `leases.json`, `budget.json`, `sanctions.json`, `capacity.json` and `version.json`, each with `schema` and `written_at` fields.

14.4 Testing pyramid (all automated in CI)

Level	What	Tooling	Gate
Unit	Checker (one test per refusal code), detectors, FSM transitions, baselines, rule compiler, importer, tree evaluator vs Python reference outputs (golden vectors), conformal set builder	`go test -race`	100% of refusal codes; ≥ 80% line coverage on `checker`, `control`, `diagnose`, `classify`
Property	FSM never violates the envelope ($S_{min} \le S \le S_{max}$) or the budget under random input sequences; precedence monotonicity (a higher PROV never overwritten by a lower one)	Go fuzzing (`testing.F`)	0 failures over 10^6 cases
Replay	Recorded decision logs replay bit-for-bit	`veriqctl replay`	identical
Integration (netns)	Full datapath in Ubuntu netns: install, per-tin self-test, scenarios S1/S2/S6 smoke, kill → SAFE	`lab/` runner in GitHub Actions (privileged runner or self-hosted)	pass
Package	OpenWrt x86-64 VM boots, installs the feed, runs health/self-test, LuCI pages load (Playwright)	QEMU + Playwright	pass
Hardware-in-loop	Pi + retail router: bench, soak 2–4 h, footprint	lab bench	budgets met

15. Evaluation (extends docs/09 §5; nothing there is weakened)

15.1 Arms (version-pinned, tuned per their own docs, configs archived)

Arm	Purpose
FIFO (no QoS)	Illustrates the problem only
SQM static CAKE (`sqm-scripts`, piece_of_cake/layer_cake)	The static reference; VeriQ's dominance-invariant floor
cake-autorate (pinned v3.x)	Co-primary comparator (docs/09)
QoSmate v0.5.48 (+ autorate), HFSC default and CAKE mode	Head-to-head (whole system), plus controller-only (QoSmate autorate on the same CAKE datapath)
VeriQ (product profile)	System under test
VeriQ ablations: no-diagnosis, no-classifier, no-leases, no-fallback, RTT-only (no OWD)	Attribute gains to components

Two enforcers never run at once. Every arm passes the Stage-1 self-test before its runs count.

15.2 Scenario catalogue

- S1–S13 (docs/05 §4, unchanged definitions) plus A1–A32 (§10).
- Each scenario has:
  - a machine-readable definition `lab/scenarios/<id>.yaml` (topology, traffic generators, fault schedule with random onset $\in [60, 120]$ s, duration, ground-truth channels);
  - a primary metric;
  - its comparator.

Tiers:

Tier	Scenarios	Arms	Runs
H0 smoke	S1, S2, S6	all	1 each (Stage 1)
H1 head-to-head core	S1, S2, S4, S6, S7, A6, A7, A9, A14, A32	SQM, cake-autorate, QoSmate(+autorate), VeriQ	pilot n = 5 → n for CI half-width (expected 10–20)
H2 classification & intent	S3, S5, S11, A10–A13, A20, A27	SQM, QoSmate (rules + heuristics), VeriQ	same rule
H3 robustness	A2, A3, A15–A19, A21–A26, A28–A31	QoSmate, VeriQ (+ cake-autorate where relevant)	5–10
H4 hardware	Bench + soak on D3/D4/D5	VeriQ, QoSmate	3 soaks

15.3 Outcomes (fixed before the final runs)

- **Primary** (docs/09): inappropriate interventions in control-futile scenarios (S6, S7, S10, A32): count, cumulative magnitude (Mbit·s) and duration below 0.95·S_ref. The evaluator verifies access utilisation < 80% and ISP queue < 5 ms.
- **Co-primary** (non-inferiority): settling confirmation time in S2/A1 ≤ comparator + 1 s.
- **Secondary** (non-inferiority): bulk share of leftover bandwidth ≥ comparator − 5 pp (absolute).
- **Head-to-head secondary set** (Holm-corrected):
  - added delay p95 per direction;
  - WebRTC freezes and concealment;
  - DASH stalls;
  - game RTT p99 (synthetic 64 pps UDP flow);
  - page load time (Playwright, 10 sites cached locally);
  - Jain's fairness across hosts;
  - recovery time;
  - CPU and RSS.
- **Expectation stated in advance:** in **static** conditions (fixed capacity, honest reflectors), VeriQ and QoSmate's CAKE mode should be roughly at **parity**, because both use CAKE. VeriQ's expected wins are in the adverse and dynamic scenarios (A-series), in explainability, in safety and in UX. A parity result there is reported as parity.

15.4 Statistics

As in docs/09: - the run is the unit; - bootstrap CIs (10 000 resamples); - counterbalanced order; - Holm correction for secondary outcomes; - no p99.9 claims from small samples; - missing data is reported, never imputed; - results where VeriQ ties or loses are kept and reported.

UX study (P1, small): 6–8 participants, a task-based comparison (QoSmate LuCI vs VeriQ LuCI). Tasks: prioritise a console, diagnose why a call is bad, undo a change. Measures: task success, time, and the System Usability Scale (SUS). The sample is small and is reported as formative, not confirmatory.

---

16. QoSmate parity matrix (feature by feature)

Legend: - **Adopt** = same capability, re-implemented clean-room; - **Improve** = same purpose, stronger design; - **Modify** = deliberately different, with the reason given; - **Defer** = not in P0/P1; - **New** = VeriQ-only.

QoSmate facts are from the README and source at v0.5.48 / luci 1.0.14 **(read)**.

#	QoSmate feature (read)	VeriQ	Verdict	Note
1	Root qdisc: HFSC (default), CAKE, Hybrid, HTB	cake (default), hfsc, htb	Adopt (3 of 4)	Hybrid deferred (O1)
2	HFSC realtime class with `gameqdisc` (pfifo/bfifo/fq_codel/red/netem)	RT class leaf: fq_codel (default) or pfifo with computed limit	Adopt (partial)	red/bfifo deferred; in-router netem replaced by the lab harness
3	GAMEUP/GAMEDOWN = 15% + 400 kbit	RT budget, same default formula	Adopt	—
4	MAXDEL, PFIFOMIN, PACKETSIZE	Computed pfifo limit (advanced)	Adopt	—
5	CAKE diffserv3/4/8, host isolation, NAT, ack-filter auto (ratio ≥ 15), RTT, extra params	diffserv4 (needed for the fwmark mapping), dual-src/dsthost, nat, ack-filter auto, rtt, extra params	Adopt	CAKE `autorate-ingress` not used (it would conflict with the controller)
6	Link presets, overhead, MPU, link compensation, VLAN keyword (shared by all qdiscs)	Presets via CAKE keywords + `stab` for HFSC/HTB	Adopt	—
7	One-directional shaping (rate 0)	Same convention	Adopt	—
8	WASHDSCPUP/DOWN (default on)	Product default both on; lab profile off	Adopt	Class survives washing (ct mark)
9	BWMAXRATIO (caps download relative to upload)	Warning only	Modify	Not capping avoids silently wasting download capacity; ACK issues handled by ack-filter
10	ACKRATE (nft ACK thinning by rate)	Not adopted; CAKE ack-filter; no ACK drops on HFSC/HTB	Modify	Fewer hand-tuned drop rules

#	QoSmate feature (read)	VeriQ	Verdict	Note
11	UDP_RATE_LIMIT (> 450 pps → cs0)	Eligibility envelope (rate + pps) for all preferential classes	Improve	Same threshold as the starting value
12	TCP_UPGRADE (< 150 pps → AF42)	Off on CAKE (sparse-flow boost); “fast BE” on HFSC/HTB	Modify	Preference needs a basis (P5)
13	TCP_DOWNPRIO initial (500 ms) / sustained (10 s)	H2 / H1	Adopt	Rate-relative thresholds
14	UDPBULKPORT / TCPBULKPORT	H5	Adopt	—
15	Legacy VIDCONFPORTS, REALTIME4/6, LOWPRIOLAN4/6	Converted to rules by the importer	Adopt	—
16	MSS clamp when a rate < 3000 kbit	Same	Adopt	—
17	NFT_HOOK / NFT_PRIORITY	Fixed default (forward, mangle + 5), advanced override	Adopt	—
18	DSCP rules: proto, src/dst IP, ports, lists, `!=`, ranges, `@set`, IPv6 suffix `::x/::mask`, counters	Same syntax accepted	Adopt	—
19	—	Rules by **device/group** (all IPs incl. IPv6 privacy), **app**, **schedule**	New	Device identity survives address changes
20	IP sets static/dynamic (dnsmasq-full nftset), timeout	Same + automatically managed device sets	Improve	—
21	Rate limits: nft meters, burst factor, targets with negation and sets	Same, labelled “policer”	Adopt	—
22	Custom rules (full table) + inline rules, validated	Same	Adopt	—
23	Connections tab: conntrack, DSCP, pps, kbit/s, AND filter, zoom	Same + **class provenance, confidence, envelope status**	Improve	Explainability
24	Statistics: per tin/class, autorate charts, hourly history (optional flash)	Same + evidence levels, reflector health, receipts	Improve	—
25	Autorate: mean RTT of reflectors, 5/10 ms thresholds, ×0.85/×1.02, refractory 1/3 s, min/base/max 25/100/105%, drift to base, state file	Diagnosis-gated controller: median of healthy reflectors, OWD, windowed-min baseline, one-step evidence cut, floor 10%, checker, budgets, link-down hold	Improve (**hypothesis until H1**)	§10 A4–A7, A9, A14, A29
26	Auto-setup: router speed test, interface plateau, over-report warning, gaming-IP rule	Same + bench + before/after receipt + import	Improve	—
27	`health_check` (service, nft, tc, config, packages, md5 integrity)	Same + tin self-test, mark plumbing, probes, conflicts	Improve	—
28	CLI via `/etc/init.d/qosmate` (status, update, auto_setup, expand_config, validate_custom_rules, check_version)	`veriqctl` (§11.4)	Adopt	—
29	Self-update from GitHub (release/snapshot/branch/commit)	Package manager only; feed channels stable/beta	Modify	Supply-chain safety
30	Preserve config on sysupgrade	keep.d	Adopt	—
31	Config migration	`config_version` + uci-defaults	Adopt	—
32	Hotplug on interface events	Same + HOLD_LINKDOWN	Improve	—
33	Installs missing packages at runtime	Declarative package dependencies	Modify	—
34	Buildroot packages	SDK CI builds for 5 arches, signed feed, ipk + apk	Improve	—

#	QoSmate feature (read)	VeriQ	Verdict	Note
35	i18n: de, zh_Hans	en + hi (P1), community `.po`	Adopt	—
36	Issue template, manual troubleshooting with tcpdump	Support bundle + automated self-test	Improve	—
37	Multi-queue CAKE option (USE_MQ)	Deferred	Defer	Kernel-dependent
38	—	Leases, household page, receipts, shadow mode, watchdog/SAFE, checker, ML with abstention, capacity memory, explain API, exporter, evaluation harness	New	—

Bottom line (honest): VeriQ matches QoSmate's queue management and classification surface. It deliberately differs in four places (9, 10, 12, 29) and gives the reason for each. It adds the controller, safety and intent layers. Whether the controller *performs* better is the H1 question. Whether the UI is *easier* is the UX-study question.

17. KPIs and business (summary; full model in the handbook)

KPI family	KPI	Target (to be measured)
Technical	Primary: inappropriate interventions (count / Mbit·s / s) in control-futile scenarios	Lower than cake-autorate and QoSmate autorate
Technical	Confirmation time after a genuine drop	≤ comparator + 1 s
Technical	Added delay p95 under load (static)	Parity with SQM (≤ +2 ms)
Technical	Time-to-safe	≤ 12 s
Product	Setup time (wizard)	≤ 5 min
Product	Footprint (Standard)	≤ 40 MB RSS, ≤ 5% of one A53 core
Product	SUS score (UX study)	≥ 75 (formative)
Operator (pilot)	"Slow internet" tickets per 1000 homes	Hypothesis: reduction vs control group
Business	Gross modelled avoided cost	Formula in the handbook; illustrative only

18. Delivery plan

Priorities: - **P0 (Round 2, by 13 Dec 2026):** the core and the minimum product. - **P1 (finale; date to be confirmed on the portal):** parity completion and demo assets. - **P2:** the product roadmap after the competition.

Sprint	Dates	Deliverables (P0 unless marked)	Exit criterion
0	12–24 Oct	Round 1 slide; repo skeleton (§14.1); CI (lint, unit); Go: `clock`, `model`, `config`, `api` stubs; lab on a full-kernel Ubuntu laptop (CAKE present)	Round 1 submitted; `go test ./...` green; lab FIFO vs CAKE run with base RTT
1	25 Oct–7 Nov	`enforce/cake` + nft renderer + mark plumbing; tin self-test; `observe/tcstats`; `probe` (echo + timestamp, qualification); baselines; decision log; **V1 harness** (Jitsi, JVB on WAN side, p2p off; dash.js; bulk); baselines installed (SQM, cake-autorate, QoSmate)	Per-tin counters match intent both directions; first call + bulk run reproducible
2	8–21 Nov	`diagnose`, `control` FSM, `checker` (all refusal tests), `reconcile`, watchdog, SAFE, states; LuCI Overview + Autorate	Stage 1 passes; H0 smoke; kill → SAFE ≤ 12 s
3	22 Nov–5 Dec	`classify` (rules, sets, device sets, leases, envelope, heuristics), QoSmate importer (rules, ipsets, ratelimits, settings); LuCI Rules + Connections; OpenWrt packages (x86-64 VM + Pi 4); H1 pilot (n = 5)	Packages install on VM + Pi; H1 n chosen

Sprint	Dates	Deliverables (P0 unless marked)	Exit criterion
4	6–13 Dec	H1 full runs; receipts; report with claims check; demo video; Round 2 submission	Every published number regenerates
5+ (P1)	after Round 2	ML classifier + conformal; hfsc/htb; household page; Hindi; app catalogue; H2/H3; bench; UX study; retail router D4; Lite on D5; 3D twin; physical kit	Finale demo
P2	post-competition	Exporter, public signed feed, apk line, prplOS/USP, RDK-B wrapper, hybrid, MQ-CAKE, NQB/L4S experiments, home pilots	—

Roles (team of 3–4, part-time):

Role	Owns
R1 Datapath & OpenWrt	nft/tc renderers, backends, packages, self-test
R2 Controller	probe, estimate, diagnose, control, checker, reconcile
R3 Evaluation	lab, scenarios, baselines, statistics, ledger
R4 UX, ML & demo	LuCI, household page, ML, twin, video

With 3 people, R4 is shared.

19. Requirement traceability (Case 3)

Requirement	Where
Classify broad categories without payload; confidence; correction	§5.1, §5.2, §5.6 (conformal), §5.3 (user rules = correction), §11.2 Connections
Estimate link capacity	§6
Queue management, shaping, both directions, IPv4/IPv6	§4, A20
Verify user experience	§9, §15 (WebRTC/DASH/game/PLT)
Temporary intent with fairness	§5.8 leases, §5.7 envelope, starvation metric
Rollback, no starvation	§8, §12.8, A12/A13
Bounded, observable, reversible remediation; safe state	§6.5, §7.3, §8
AI vs deterministic baseline with measurable outcome	§15.1 (QoSmate heuristics, SQM, ablations)
Dashboard metrics	§11.2, exporter, Grafana
Automated experiments, scripted demo, reproducibility	§14.4, §15, evidence ledger
Licences, no secrets, emulation labelled	§13, lab notices
Known limitations, prototype vs production boundary	§10, §12.2, §20

20. Remaining decisions and the defaults to use now

#	Question	Default until data decides
O1	Hybrid backend (HFSC + CAKE)?	Not offered; revisit only if H3 shows hfsc or cake alone failing a gaming scenario
O2	fwmark value → CAKE tin mapping on each kernel	Use 1–4 = Bulk, BE, Video, Voice; the self-test is authoritative; the mapping table is configurable if a kernel differs
O3	`act_ctinfo cpmark` unavailable or broken on a target	Fallback mode: QoSmate-style DSCP restore (`ctinfo dscp 63 128`), with DSCP as the class carrier, washed after the CAKE tin decision; health shows "compat mode"
O4	Fewer than 2 timestamp-capable reflectors qualify	RTT-only mode with load attribution (§6.4)
O5	VeriQ licence	Apache-2.0 proposed; Tata Elxsi decides
O6	UDPST server	None by default (L3 unavailable); the lab provides one

#	Question	Default until data decides
O7	Team cannot ramp up on Go in Sprint 0	Implement the controller in Python in the lab (same interfaces, same golden tests) for P0 and port it to Go in P1; the spec is language-neutral
O8	SHADOW duration	24 h; tune after the pilot
O9	Envelope thresholds	Starting values in §5.7; tune in S11/A11
O10	Wi-Fi diagnosis thresholds	30% retries; tune on hardware

Decision: BUILD. No further architecture revision before Sprint 2's exit criterion. Changes after that come from measurements and are recorded in docs/12.

UCI configuration reference (spec Appendix B) (`/etc/config/veriq`)

```
config global 'global'
    option enabled '1'
    option config_version '5'
    option profile 'standard'          # lite | standard | pro
    option start_mode 'shadow24h'      # shadow24h | live
    option state_override ''           # '' | shadow | safe

config wan 'wan'
    option interface 'wan'             # netifd interface; device resolved automatically
    option ifb 'ifb-veriq'
    option preset 'ethernet'           # §4.4
    option overhead ''                 # custom only
    option mpu ''
    option linklayer ''                # atm | ptm | ''

config shaping 'shaping'
    option backend 'cake'              # cake | hfsc | htb
    option down_base_kbit '90000'      # 0 = do not shape download
    option up_base_kbit '45000'        # 0 = do not shape upload
    option min_frac '0.10'             # S_min = max(1000, min_frac * base)
    option max_frac '1.05'
    option dscp_policy 'product'       # product | lab | custom
    option wash_ingress '1'
    option wash_egress '1'
    option trust 'all'                 # all | list (uses @trusted_dscp set)
    option ack_filter 'auto'           # auto | 1 | 0
    option mss_clamp 'auto'
    option host_isolation '1'

config cake 'cake'
    option rtt ''                      # ms; '' = CAKE default
    option extra_egress ''
    option extra_ingress ''

config hfsc 'hfsc'
    option rt_budget_up_kbit ''        # '' = max(400, 15% of S)
    option rt_budget_down_kbit ''
    option rt_leaf 'fq_codel'          # fq_codel | pfifo
    option maxdel_ms '24'
    option pfifo_min '5'
    option packet_size '450'

config htb 'htb'
    option fast_be '1'

config controller 'controller'
    option tick_ms '250'
    option theta_bb_ms '15'
    option explore_delta_ms '5'
    option backoff_delta_ms '10'
    option raise_factor '1.04'
    option cut_g_factor '0.90'
    option fallback_step '0.05'
    option fallback_budget '0.10'      # per rolling 10 min
    option fallback_cooldown_s '60'
    option stale_s '120'
    option capacity_memory '1'

config probe 'probe'
    option rate_pps '10'
    option idle_rate_pps '2'
    option active_far '5'
    list reflector4 '1.1.1.1'
    list reflector4 '1.0.0.1'
    list reflector4 '8.8.8.8'
    list reflector4 '8.8.4.4'
```

```
    list reflector4 '9.9.9.9'
    list reflector4 '149.112.112.112'
    list reflector4 '208.67.222.222'
    list reflector4 '208.67.220.220'
    option near 'auto'                # auto = WAN gateway / PPPoE peer
    option udpst_server ''            # optional cooperating server (L3)

config classify 'classify'
    option ml '1'                     # forced 0 in the lite profile
    option sample_packets '32'
    option sample_limit_pps '4000'
    option conformal_alpha '0.05'
    option heuristic_bulk '1'         # H1
    option heuristic_newflow '1'      # H2
    option heuristic_slowtcp 'auto'   # H3: off on cake, fast-BE on hfsc/htb
    option bulk_tcp_ports ''
    option bulk_udp_ports ''

config envelope 'envelope'
    option rt_kbit_min '1000'
    option rt_frac '0.02'
    option rt_pps '450'
    option iv_kbit_min '8000'
    option iv_frac '0.25'
    option standing_queue_s '3'
    option penalty_s '60'
    option penalty_cap_s '900'

config leases 'leases'
    option max_active '2'
    option max_minutes '240'
    option household_user 'household'

config privacy 'privacy'
    option flow_retention_h '24'
    option aggregate_retention_d '7'
    option export '0'

config alerts 'alerts'
    option webhook_url ''

config device
    option name 'Rahul-Laptop'
    option mac 'aa:bb:cc:dd:ee:ff'
    option group 'work'

config rule
    option name 'Console'
    option device 'PS5'
    option proto 'udp'
    list dest_port '!=80'
    list dest_port '!=443'
    option vclass 'rt'
    option counter '1'

config ipset
    option name 'streaming'
    option mode 'dynamic'
    option family 'ipv4'
    option timeout '2h'

config ratelimit
    option name 'Guests'
    list target '192.168.2.0/24'
    option download_limit '5000'
    option upload_limit '2000'
    option burst_factor '1.0'
```

Appendix D · Implementation contracts (docs/09 v4.1 Appendix A, in force)

*Every number below is a **starting parameter** to be tuned in Stage 2 and recorded in the run manifest. None is a validated safety limit.*

A.1 Action Checker (R1): between the Policy Engine and the Enforcer

Position. `Guardrails/Policy (M4) → proposed action → ACTION CHECKER → Enforcer (M5)`. The Enforcer refuses any command without an ACCEPT token for the same `action_id` and config version.

Inputs: - the proposed action (type, target, `action_id`); - the observation snapshot (each signal with its timestamp); - the desired-state version and the last read-back kernel version; - the budget state (fallback, retry); - the lease table; - the sanction table; - the operating state.

Output: `ACCEPT(action_id, version)` or `REJECT(action_id, code, reason)`. Both go to the decision log and the receipt.

Action	Rules checked (all must pass)	Refusal codes
any	snapshot age ≤ 2 cycles; desired version = read-back version; state ∈ {LIVE} (SHADOW → logged, not applied); `action_id` not seen before	`STALE_OBS`, `VERSION_MISMATCH`, `STATE_FORBIDS`, `DUPLICATE`
RATE_CUT	step allowed by the diagnosis level (A.7); new S ≥ floor S_min (A.2)	`STEP_EXCEEDS_LEVEL`, `BELOW_FLOOR`
RATE_RAISE	exploration preconditions (A.4); step ≤ ×1.04; new S ≤ S_max	`EXPLORE_PRECOND`, `ABOVE_MAX`
FALLBACK_STEP	no overlapping episode; rolling-budget remainder ≥ step; episode cooldown elapsed	`EPISODE_ACTIVE`, `BUDGET_EXHAUSTED`, `COOLDOWN`
LEASE_GRANT	token valid; ≤ 2 active, ≤ 1 per device; not in capacity collapse	`AUTH`, `ADMISSION`, `COLLAPSE`
MARK_SET	tin ∈ 1..4; flow not sanctioned; the preference source is a lease, a user correction, an eligible app mark or a single-class prediction (**never abstention alone**)	`TIN_RANGE`, `SANCTIONED`, `NO_PREFERENCE_BASIS`
SAFE / REVERT_TO_RECHECKED	always admissible; still logged	—

Tests (Stage 1 unit and fault tests; one test per refusal code): - stale snapshot; - version mismatch after a foreign `tc` change; - step too large for L1; - S below the floor; - exhausted budget across two episodes (the cooldown must not refund the budget); - expired lease; - an abstained flow requesting preference without a lease; - an action sent while in SHADOW; - a duplicate `action_id`.

Claim limit. The checker checks admissibility against the implemented rules only.

A.2 Rate envelope and floor

- Per direction: `S_min ≤ S ≤ S_max`, with `S_base` = the configured nominal rate.
- **S_min = max(1 Mbit/s, 0.10 · S_base)**. For a 100 Mbit/s plan that is 10 Mbit/s. That sits below the 20 Mbit/s target of the 100 → 20 scenario, so S2 can settle around 0.9 × 20 Mbit/s.
  - Rule: `S_min ≤ 0.8 ×` the lowest capacity in the scenario set, checked at configuration time.
  - A floor of 25% of base, as in some tools' defaults, would be 25 Mbit/s and could **not** follow a drop to 20 Mbit/s.
- `S_max = 1.05 · S_base`.

A.3 Capacity estimate vs gateway throughput vs receiver goodput

Quantity	Where measured	Use
G_gw: gateway throughput	CAKE dequeued bytes (`tc -s`), including headers; overhead-compensated to the wire	Estimator input
Receiver goodput	Application payload at the endpoints (iperf3, harness)	**Evaluation only**; lower than G_gw (headers, retransmissions)
μ, σ	EWMA of G_gw samples taken while shaper-limited or queue-building (censored samples flagged)	Tracking estimate, not a calibrated CI

The estimator never treats an application-limited G_gw as capacity (§3.3).

A.4 Upward-exploration lifecycle

States: `HOLD → PROBE_UP → (BACKOFF) → HOLD`, plus `DRIFT` when idle.

- **Enter PROBE_UP** only when all hold:
  - load ≥ 75% of S for ≥ 2 s;
  - reflector-median delta < 5 ms;
  - no active fallback episode;
  - no diagnosis at ≥ L1 in the last 10 s.
- **In PROBE_UP:** raise ×1.04 per step with a 1 s refractory period, up to S_max.
- **BACKOFF:** if the delta rises above 10 ms during PROBE_UP, return to the last good S × 0.97, then HOLD for 30 s.
- **DRIFT (idle/stale):** if there is no confirming evidence for 120 s, move toward S_base by ≤ 1% per 10 s. σ widens while drifting.

A.5 Configuration ownership and versions

- **What VeriQ owns:** its nft table `veriq`, qdiscs with handles in a reserved range, and the IFB `ifb-veriq`. Each owned object records the config version in a comment or a state file.
- **Readback:** compares the hash of owned objects with the desired version.
- **Foreign objects:** never modified. Their presence on the shaped interfaces sends VeriQ to DEGRADED.
- **Disable/cleanup:** removes owned objects only.
- **Version numbers:** a monotonic integer, persisted across restarts.

A.6 Settling onset vs confirmation

For an injected event at t_0: - **onset** = the first controller rate change after t_0; - **confirmation** = the earliest time t_1 after which S stays within ±10% of its final value for 5 s.

Both are reported. The co-primary outcome uses confirmation time.

A.7 Diagnosis evidence levels, near-probe absence, cooldown

- **L0–L3** as defined in §3.3. Correlated sources count once (all reflectors behind the same access link = one source).
- **Allowed cut per level:**

Level	Allowed action
L2 WAN_LIKELY	×0.75
L3	Re-baseline to the measured capacity minus margin
L0–L1 / MIXED / UNKNOWN	Bounded fallback only (≤ 5% per step)

- **Near probe missing** (first hop unreachable, ICMP rate-limited, or a route change):
  - the near signal is MISSING;
  - REMOTE_LIKELY cannot exceed **L1**, because near-vs-far discrimination is impossible;
  - WAN_LIKELY may still reach **L2** only from two other independent sources: the far-reflector median **plus** throughput censoring (G_gw below S with demand present, own queue empty), or an L3 capacity test;
  - otherwise bounded fallback applies.
- **Episode cooldown (60 s):** it delays the next episode only. The **rolling 10-minute cumulative-cut budget** (≤ 10% of the anchor) is never reset or refunded by cooldowns, new episodes or restarts (the budget state is persisted).

A.8 First-packet and default mark handling

- A new connection has ct mark 0, so there is no `fwmark` tin. CAKE then applies its normal diffserv4 classification on the packet DSCP (usually Best Effort).
- **Leases:** a device with an active lease gets its mark from the **first packet**, because an nft rule matches the device address and the lease is known in advance.
- **Classified flows:** the mark is set after k packets (5/10/20/30). Marks never apply retroactively.
- **Sanctioned flows:** the mark is cleared to Best Effort.
- **External DSCP:** untouched unless `wash` is configured.

A.9 Selection rate vs conformal coverage

Metric	Meaning
Coverage	Fraction of prediction sets containing the true label. Target $1 - \alpha = 0.95$ on exchangeable held-out data; drift breaks it.
Selection rate	Fraction of flows with a singleton set, i.e. the flows VeriQ may act on
Selective accuracy	Accuracy among selected flows
Abstention rate	1 − selection rate (also counts empty sets)

All four are reported, per class and under time/app shift. A low selection rate is acceptable because abstention defaults to best effort. High coverage does **not** imply high selective accuracy.

> **Note.** Change C1 (Appendix C, §0.3) replaces the L2 cut rule in A.7 above: at L2 the cut is one step to max(S_min, 0.90·$\hat{G}$); ×0.75 applies only when no fresh $\hat{G}$ is available.

Appendix E · Change records

12 · VeriQ v5 change record (vs. v4.1)

The authoritative spec is now `docs/11_VeriQ_v5_Final_Design.md`. docs/09 (v4.1) remains valid wherever docs/11 does not change it; its Appendix A contracts stay in force, with the A.7 cut rule replaced by C1. The handbook (`docs/VeriQ_Handbook_v5.pdf`) is generated from docs/11 and the earlier docs.

#	Change	Type	Reason	Verification	Revert if
C1	L2 fast cut = one step to max(S_min, 0.90·Ĝ); ×0.75 only without fresh Ĝ	Correction	×0.75/step needs ~6 steps for 100 → 20 and would fail the co-primary margin	S2/A1 confirmation time vs cake-autorate	False L2 cuts in H1 control-futile runs exceed comparator
C2	On-router daemon in Go; Python for harness/ML	Decision	Single static binary for all OpenWrt targets; no Python runtime on routers	Footprint in Stage 4	Team cannot ramp up (O7 fallback: Python in the lab for P0)
C3	OpenWrt product layer (packages, LuCI, rpcd, procd, feed, CI)	Scope (user-directed)	Product readiness that beats QoSmate	Package tests (VM, Pi)	—
C4	Backends cake / hfsc / htb; hybrid deferred	Scope	Parity; weak CPUs	Bench + H3	—
C5	Classification engine with QoSmate-compatible rules, sets, policers, importer, app catalogue	Scope	Parity + migration path	Importer tests; H2	—
C6	Product DSCP default: wash both directions; lab profile untouched	Default change	Protect Wi-Fi WMM; no upstream leak	Self-test	ISP-specific issues reported
C7	Heuristics may only demote or reset	Clarification	Consistency with “preference needs a basis”	Unit tests	—
C8	Baseline = 10-min windowed minimum of low-load 1-s medians	Precision	Handles upward route changes	A7	—
C9	Direction attribution (OWD or load-based)	Precision	No collateral cuts of an idle direction	A9	—
C10	Operating states extended; SHADOW 24 h by default with static CAKE active	Product	Immediate SQM benefit; adaptive trust	Shadow report	—
C11	Watchdog = procd instance (POSIX sh)	Platform	OpenWrt	Time-to-safe test	—
C12	Evaluation adds the QoSmate head-to-head and A1–A32	Scope	Measurable comparison	H1–H4	—
C13	Diagnosis label CPU_LIMITED added	Precision	CPU-bound throughput must not read as a capacity drop	A17	—
C14	Mark layout: CLASS 24–26, PROV 27–29, SANCT 30 (avoids mwan3 and pbr bits)	Precision	Kernel-side precedence; coexistence	Self-test + conflict scan	—

Preserved without change: - CAKE default core and kernel datapath; - three speeds; - R1/R2/R3; - L0–L3 permissions (except C1) and the missing-near rules; - 7-level precedence and the single UNKNOWN rule; - leases and admission; - envelope and sanctions; - the rolling budget and a cooldown that never refunds it; - receipts and shadow mode; - the primary outcome and its margins; - the statistics rules; - the claims discipline.

10 · VeriQ v4 change record (vs. Project Description v3.3)

The original PDF is preserved as `docs/VeriQ_Project_Description.pdf`. The revised version is `docs/VeriQ_Project_Description_v4.pdf`.

A. Material changes

Existing behaviour/claim (v3.3 location)	Decision	Problem	Change	Expected benefit	Cost/risk	Verification	Revert/defer if
“Tested against…” (Exec summary p.3); “Properties claimed and tested” (§10.1 p.17)	Correct	Planned work read as executed	“Will be tested against…”; “Properties designed and to be tested”	Credibility	None	Text audit	—
“A slow far-away server never triggers a wasteful cut” (p.3)	Correct	Absolute claim	“is designed not to trigger…; measured as unnecessary-intervention rate”	Testable claim	None	Primary RQ	—
Encrypted traffic / port rules / static QoS / “many tools shape only download” (pp.3–4, §2.2, §4.2)	Correct	Overbroad; ignores mature tools (QoSmate, SQM, cake-autorate)	Qualified wording; encrypted traffic still exposes useful metadata; fair queuing stays effective	Accuracy	None	Text audit	—
Sanction rule (§6.4 p.11)	Correct	Could punish legitimate bulk traffic; recovery forced traffic generation	Sanctions only for preferential-eligibility violations; allowance = per-flow eligibility envelope; silence counts as no violation	Fairness, no perverse incentive	Envelope tuning	S11 + bulk-progress metric	Envelope causes false sanctions > 1% of flows
Precedence (§6.4, §10)	Correct	Unknown flows on authorized devices undefined	Final 7-level precedence; leases cover UNKNOWN flows on the leased device	Consistent user-intent semantics	Leased device’s bulk may get preference (bounded by thresholds + envelope)	S5 with mixed traffic	Leased bulk harms others’ progress
Day-0 numbers (§8.1 p.14)	Correct	Fingerprint wrongly claimed a 20 ms netem base RTT	Labelled preliminary; raw files + corrected fingerprint in `evidence/` ; script fixed to record applied capabilities	Honest evidence	None	`evidence/…/README.md`	—
Business model (§9.2 p.16)	Correct	Read as savings/ROI	“Gross modelled avoided cost”; net formula with integration/maintenance; no “every gateway”	Credibility	None	Text audit	—
CAKE single-core statement (§10.2 p.17)	Correct	Version scope	Kernel-scope wording; multi-queue CAKE conditional, not VeriQ evidence	Accuracy	None	Text audit	—
“Exact budget already specified” (§11.2 p.18)	Correct	Overstated	“Proposed, unvalidated contract”	Accuracy	None	Text audit	—
Jitsi demo (§11.1 p.18)	Correct	Local call could bypass the bottleneck (P2P/hairpin)	JVB on WAN side, P2P disabled, counters prove media crosses the ISP emulator	Valid demo	Setup effort	Stage 1 media-path check	—
R1 Action Checker + reconciler	**Add (core)**	No admissibility gate; partial applies and foreign changes unspecified	Deterministic checks; desired/observed reconciliation; restart adoption; failure domains	Fewer unsafe or stale actions; demoable rejection	~1 week	Stale/duplicate/foreign-change tests	Adds > 1 cycle of latency to fast cuts

Existing behaviour/claim (v3.3 location)	Decision	Problem	Change	Expected benefit	Cost/risk	Verification	Revert/defer if
R2 Mark plumbing (ct mark → CAKE fwmark; ctinfo on download)	**Add (core)**	"Internal marks select tins" had no mechanism, especially for download	nft + act_ctinfo + IFB + CAKE `fwmark`; DSCP untouched; per-tin verification	Real, verifiable preference in both directions	iproute2/kernel feature dependency	Per-tin counters + pcap	Target kernel lacks ctinfo or fwmark → fall back to DSCP-based tins (documented)
R3 Evidence-graded diagnosis + robust aggregation + capacity observability	**Add (core; repairs M3/M4)**	Correlated signals counted as independent; mean aggregation fragile; S/G/A/C conflated; censoring ignored	L0–L3 evidence levels, permission table, median-of-healthy reflectors with quorum, single global fallback budget, `[min, base, max]` envelope with drift-to-base	Fewer unnecessary cuts (hypothesis)	Possible slower reaction	Primary RQ, non-inferiority margin	Settle-time margin violated
Evaluation plan (§8)	Correct/extend	No QoSmate baseline; packets treated as samples; FIFO-heavy	QoSmate (+autorate) pinned; controller vs whole-system comparisons; run = unit; pilot-based n; blind faults; evaluation Stages 1–4	Comparisons a judge can trust	More setup	Ledger + claims check	—

A2. v4.1 synchronisation pass (no new features)

Issue	Change	Where
Action checker advertised but missing from architecture and workflow	Checker placed between M4 Policy and M5 Enforce in the diagram, the module contracts, the runtime loop (new "Check" step) and milestone M-B; inputs, refusal codes, per-action rules and tests specified. "Guarantees admissibility" → "checks admissibility against the implemented rules"	docs/09 §R1 + Appendix A.1; PDF §5, §7.1, §7.2, §10
UNKNOWN-flow contradiction	One rule everywhere: *Classifier abstention alone never grants preference. A valid authenticated lease may grant preference to UNKNOWN traffic, subject to eligibility and operating-state checks.*	docs/09 precedence; PDF §3, §5.1, §10
Evidence-level naming clash; cooldown vs budget; missing near probe	L0–L3 = diagnosis evidence, E1–E8 = estimator tests, Stage 1–4 = evaluation ladder. The 60 s episode cooldown never resets or refunds the rolling 10-minute cut budget. Missing near probe: REMOTE capped at L1; WAN reaches L2 only via far median + throughput censoring	docs/09 Appendix A.7; PDF §6.1, §6.6
Primary outcome could reward never acting	Primary = inappropriate interventions in **control-futile** scenarios (remote delay, lying reflector, Wi-Fi bottleneck, probe-path disturbance; evaluator confirms no local congestion). Report count, cumulative magnitude ∫max(0, S_ref−S)dt and duration below 0.95·S_ref. Constant-capacity congestion excluded	docs/09 §evaluation; PDF §8.5
Bulk margin mismatch (−10% vs 5 pp)	One margin: bulk share of leftover bandwidth ≥ tuned cake-autorate − 5 percentage points (absolute). The −10% relative margin is withdrawn. Confirmation time ≤ comparator + 1 s in genuine drops. (The v3.4 text is not in this repo; reconciled to the 5 pp margin as stated)	docs/09; PDF §8.5
Implementation contracts missing	One authoritative appendix: rate floor/ceiling (S_min = max(1 Mbit/s, 0.10·S_base), compatible with 100→20 Mbit/s), capacity vs gateway throughput vs receiver goodput, upward exploration lifecycle, app-owned config versions, first-packet/default marks, settling onset vs confirmation, selection rate vs conformal coverage	docs/09 Appendix A.2–A.9; PDF references → docs/09

B. Preservation matrix (case requirements → location in v4)

Case requirement	v4 location
Classify broad categories without reading payload; expose confidence; allow correction	§3.4 precedence, classifier (docs/05 M2), corrections level 4
Estimate link capacity	§3.3 (S/G/A/C, observability)
Queueing and shaping; both directions; IPv4/IPv6	§3.1–3.2 (CAKE egress + IFB ingress)
Verify user experience	§3.6 receipts, §5 metrics (WebRTC/DASH)
Temporary intent with fairness	§3.4 leases, admission, envelope; starvation metric
Rollback; no starvation	§3.5 reconciler/SAFE; §5 starvation definition
Bounded, observable, reversible remediation; safe state	§3.3 fallback budget, §3.5 checker, watchdog, SAFE assumption
AI vs deterministic baseline with measurable outcome	§5 (QoSmate-style heuristics, port rules; ablations)
Dashboard metrics	docs/05 M8; §7 replay
Automated experiments; scripted demo; reproducibility	§5 ladder, ledger, claims check, `evidence/`
Licences, no secrets, emulation labelled	§4 (QoSmate GPL-3.0: ideas only), docs/05 M9
Known limitations; prototype vs production boundary	§7 envelope, §8 audit

C. Unresolved / evidence gaps

- No controller code exists yet.
- No full-kernel run; CAKE/fq_codel/netem/IPv6 are untested.
- The act_ctinfo cpmark syntax still needs verifying on the target iproute2.
- QoSmate and cake-autorate have not been run as baselines yet.
- The hypothesis about QoSmate's mean-of-reflectors behaviour is untested.
- The envelope and fallback parameters are unvalidated.